파이썬 자료구조

Data Structures in Python

김수환 · 김덕환 지음



인하대학교

파이썬 자료구조

Data Structures in Python

김수환 · 김덕환 지음

인하대학교

김수환
- 부산외국어대학교 소프트웨어전공 교수
- 서울대학교 계산통계학과 졸업
- 한국과학기술원 전산학과 석사, 박사

김덕환
- 인하대학교 전기전자공학부 교수
- 서울대학교 계산통계학과 졸업
- 한국과학기술원 전산학과 석사, 박사

파이썬 자료구조

© 김수환 김덕환
발행일 2025년 10월 31일
지은이 김수환 김덕환
발행처 인디펍
발행인 민승원
출판등록 2019년 01월 28일 제2019-8호
전자우편 cs@indiepub.kr
대표전화 070-8848-8004
팩스 0303-3444-7982
정가 비매품
ISBN 979-11-6756744-4 (93000)

이 책은 저작권법에 따라 보호받는 저작물이므로 무단 전재와 복제를 금합니다.

머리말

자료구조는 컴퓨터 과학의 근간을 구성하는 핵심 개념으로, 전통적으로 C/C++나 Java와 같은 언어를 통해 학습하는 것이 일반적이었습니다. 이들 언어는 메모리를 직접 제어하고 성능을 극대화할 수 있다는 장점이 있지만, 복잡한 문법과 저수준의 세부 구현에 대한 부담으로 학습자가 자료구조의 핵심 원리를 이해하기 전에 언어 자체의 장벽에 부딪히는 경우가 많았습니다. 이 책은 간결하고 직관적인 파이썬을 사용하여 자료구조를 설명함으로써, 독자가 복잡한 구문에 얽매이지 않고 알고리즘의 논리와 데이터의 구조적 본질에 집중할 수 있도록 돕습니다. 파이썬을 사용하면 아이디어를 빠르게 코드로 구현하고 테스트할 수 있어, 추상적인 개념을 더 쉽게 체득하고 응용력을 키울 수 있을 것입니다.

파이썬의 강점은 파이썬 관련 문서에서 언급되듯이 '배터리가 내장된(batteries included)' 철학에 기반한 강력한 내장 자료구조에 있습니다. 많은 프로그래머가 리스트, 딕셔너리, 집합과 같은 파이썬의 자료구조를 일상적으로 사용하지만, 내부 동작 원리나 성능 특성을 제대로 이해하지 못한 채 활용하는 경우가 많습니다. 이 책은 단순히 이론적인 자료구조를 나열하는 데 그치지 않고, 파이썬의 핵심 자료구조가 어떤 원리로 구현되었는지를 깊이 있게 탐구합니다. 독자는 이 책을 통해 어떤 상황에서 어떤 자료구조를 선택해야 하는지, 그리고 그 선택이 프로그램 성능에 어떤 영향을 미치는지 명확히 이해하게 될 것이며, 이는 여러분의 코드를 한 차원 높은 수준의 효율적이고 '파이썬스러운(Pythonic)' 코드로 만들어 줄 것입니다.

이 책에서 다루는 자료구조와 알고리즘 지식은 효율적인 코드 작성을 넘어, 현대 기술의 최전선에서 복잡한 문제들을 해결하는 기반이 됩니다. 오늘날 파이썬은 간결함과 강력한 생태계를 바탕으로 여러 첨단 분야의 핵심 언어로 자리 잡았으며, 특히 인공지능과 임베디드 시스템 분야에서 그 중요성은 더욱 커지고 있습니다. 예를 들어, 자율주행 자동차나 자율이동로봇이 최적의 경로를 탐색하는 데에는 그래프와 최단 경로 알고리즘이 사용되며, 방대한 센서 데이터를 실시간으로 처리하고 의사결정을 내리는 과정에는 효율적인 탐색 트리와 해시 테이블이 핵심적인 역할을 합니다. 이처럼 주어진 문제의 특성을 파악하고 최적의 자료구조를 선택하여 해결책을 설계하는 과정은 자연스럽게 컴퓨팅 사고력(Computational Thinking)을 길러줍니다. 복잡한 문제를 논리적으로 분해하고, 시간과 공간 효율성을 함께 고려하여 최선의 해결책을 찾아가는 훈련을 통해, 독자 여러분은 단순한 프로그래밍 기술을 넘어 진정한 문제 해결 능력과 논리적 사고력을 갖춘 인재로 성장하게 될 것입니다.

이 책의 구성은 다음과 같습니다.

1장에서는 알고리즘과 자료구조의 기본 개념에 관한 내용으로 알고리즘의 정의와 알고리즘의 효율성과 분석 방법, 차수 표기법을 설명합니다.

2장에서는 이 책을 이해하는 데 필요한 파이썬 기초 문법을 간결하게 다룹니다. 다른 프로그래밍 언어에 익숙하거나 파이썬 복습이 필요한 독자를 위한 내용으로, 이미 파이썬에 익숙한 독자는 이 장을 건너뛰고 바로 다음 장으로 넘어가도 무방합니다.

3장에서는 추상자료형(ADT)의 개념과 가장 일반적인 선형 자료구조인 선형리스트의 정의와 선형리스트에 필요한 연산들을 소개합니다. 선형리스트를 구현하기 위해 사용되는 배열 구조를 이해하고, 파이썬의 리스트 자료형의 특징을 자세하게 살펴봅니다. 사례 연구로서 기본 정렬 알고리즘인 선택 정렬, 버블 정렬, 삽입 정렬을 소개하고 시간복잡도를 분석합니다.

4장에서는 규칙이나 패턴에서 찾을 수 있는 재귀 관계와 재귀 관계를 재귀 함수로 구현하는 방법, 재귀 함수의 동작 원리와 시간복잡도와 공간복잡도를 분석하는 방법을 소개합니다. 그리고 재귀 함수의 단점을 분석하고 이를 해결하는 방법을 알아봅니다. 사례 연구로서 하노이 탑 문제와 이진 탐색 알고리즘을 소개합니다.

5장과 6장에서는 가장 기본적이며 중요한 자료구조인 스택과 큐의 개념과 동작 원리, 스택과 큐 연산의 종류를 소개하고, 이를 활용하여 문제를 해결하는 사례를 살펴봅니다. 그리고 스택과 큐를 구현하는 다양한 방식을 소개하고, 사례 연구로서 미로에서 길을 찾는 문제와 편집기 문제, 요셉 문제를 소개합니다.

7장에서는 자료구조를 구현하는 프로그래밍 도구로서 배열과 함께 사용되는 연결 구조를 소개하고, 배열 구조와 연결 구조의 장단점을 살펴봅니다. 그리고 선형 연결 구조인 연결리스트의 종류와 이를 구현하는 방법을 소개합니다. 사례 연구로 편집기 문제를 소개합니다.

8장에서는 자주 활용되는 중요한 연산인 정렬과 탐색에 관련된 알고리즘을 소개합니다. 효율적인 정렬 알고리즘인 합병 정렬, 퀵 정렬, 기수 정렬의 개념과 시간복잡도를 분석하고, 각 알고리즘의 특징을 살펴봅니다. 그리고 이진 탐색과 관련된 파이썬의 bisect 모듈과 퀵 선택 알고리즘을 소개합니다. 사례 연구로는 경시대회에 출제되었던 반딧불이 문제를 소개합니다.

9장과 10장에서는 비선형 자료구조로서 계층적인 자료를 효과적으로 표현할 수 있는 트리 구조를 소개합니다. 이진 트리의 표현과 연산, 일반적인 트리의 표현, 그리고 이진 탐색 트리와 힙, union-find 자료구조와 같은 중요한 트리 구조를 살펴봅니다. 사례 연구로서 최적 정렬 시

간, 수식 트리, 세그먼트 트리를 소개합니다.

11장과 12장에서는 가장 일반적인 형태의 자료를 표현할 수 있는 자료구조인 그래프를 소개합니다. 그래프의 정의 및 종류, 표현 방법, 깊이 우선 탐색과 너비 우선 탐색 등 기본적인 연산의 활용과 최소 신장 트리, 최단 경로 문제를 위한 알고리즘을 살펴봅니다. 사례 연구로서 방향 비순환 그래프에서의 위상 정렬과 임계 경로를 찾는 알고리즘을 소개합니다.

마지막 13장에서는 여러 가지 탐색 구조를 살펴봅니다. 균형 이진 탐색 트리인 AVL 트리와 레드-블랙 트리, 인덱싱 구조로 활용되는 B-트리의 정의와 동작 원리를 소개합니다. 그리고 매우 효율적인 성능을 제공하는 탐색 구조인 해싱 기법에 대해 살펴보고, 사례 연구로 파이썬에서 해싱 기반으로 구현된 자료구조인 딕셔너리(dict)와 집합(set)의 동작 원리를 알아봅니다.

이 책은 컴퓨터공학, 전자공학, 또는 소프트웨어 관련 학과나 전공에서 자료구조 강좌의 교재로 사용하거나 학원의 강의 교재 또는 독학으로 자료구조와 프로그래밍을 공부하고자 하는 일반인을 위한 자습서로 적합하게 구성되어 있습니다. 이 책에서는 다른 자료구조 책에서 흔히 볼 수 있는 이론적 정리와 수학적 증명을 거의 볼 수 없습니다. 자료구조를 컴퓨터 이론의 기초 과목으로 공부하는 경우는 정리와 증명이 필수적이지만, 이 책을 보는 독자의 대상을 넓히고 상대적으로 쉽게 자료구조에 접근할 수 있도록, 가급적 수학적인 서술은 생략하고 꼭 필요한 내용은 본문 중에 자연스럽게 서술하거나 연습문제로 돌려놓았습니다. 그리고 상세한 프로그램 소스를 책에 수록함으로써 자료구조를 막연하게 이해하고 있거나 프로그래밍 공부를 병행하고자 하는 독자에게 도움이 되도록 하였습니다.

이 책을 중간고사와 기말고사가 포함된 15주 한 학기용 자료구조 강의 교재로 활용할 경우, 한 주에 한 장씩 학습을 진행할 수 있도록 구성되어 있습니다. 이때, 2장 '파이썬 기초'는 필요에 따라 강의 일정에서 제외하거나 1장과 순서를 바꾸어도 됩니다. 또한, 이 책의 연습문제를 선별하여 강의에서 다루거나 국내외 온라인 채점 사이트를 활용하여 경시대회 기출 문제 등을 다루는 것도 추천합니다.

이 책이 독자 여러분에게 파이썬과 자료구조에 대한 이해를 높이는 소중한 길잡이가 되고, 뛰어난 프로그래머로 성장하는 밑거름이 되기를 바랍니다. 끝으로 이 책을 집필하기까지 여러모로 많은 조언을 해 주신 오이식님께 감사드립니다.

2025년 10월
저자 일동

목 차

제1장. 서론: 기본 개념 ····· 11
1.1 알고리즘 개요 12
1.2 알고리즘의 효율성 15
1.3 알고리즘의 성능 분석 18
1.4 차수 표기법 23
연습문제 32

제2장. 파이썬 기초 ····· 37
2.1 파이썬 개요 38
2.2 자료형과 연산자 40
2.3 제어 구조 45
2.4 컬렉션 자료형 51
2.5 함수와 모듈 58
2.6 클래스와 객체 64
연습문제 67

제3장. 선형리스트와 배열 ····· 69
3.1 자료구조와 추상자료형 70
3.2 선형리스트 72
3.3 배열 구조 74
3.4 파이썬의 순서 자료형 78
3.5 사례 연구: 리스트 정렬 85
연습문제 93

제4장. 재귀 알고리즘 ····· 99
4.1 재귀 관계 100
4.2 재귀 함수 104
4.3 재귀 함수의 성능 분석 110
4.4 사례 연구: 하노이 탑 115
4.5 사례 연구: 이진 탐색 118
연습문제 123

제5장. 스택 자료구조 ····· 129
5.1 스택의 개념 130
5.2 스택의 활용 134
5.3 스택의 구현 141
5.4 사례 연구: 미로에서 길 찾기 144
5.5 사례 연구: 편집기 150
연습문제 154

제6장. 큐 자료구조 ····· 159
6.1 큐의 개념 160
6.2 큐의 활용 162
6.3 큐의 구현과 deque 모듈 168
6.4 사례 연구: 미로에서 최단 경로 찾기 173
6.5 사례 연구: 요셉 문제 178
연습문제 183

제7장. 연결리스트 ····· 187
7.1 배열 구조와 연결 구조 188
7.2 단일 연결리스트 191
7.3 원형 연결리스트 198
7.4 이중 연결리스트 200
7.5 사례 연구: 편집기 208
연습문제 212

제8장. 정렬과 탐색 ····· 217
8.1 정렬 문제 218
8.2 합병 정렬 220
8.3 퀵 정렬 225
8.4 분배에 의한 정렬 232
8.5 탐색과 선택 235
8.6 사례 연구: 반딧불이 239
연습문제 243

제9장. 트리와 이진 트리 ····· 247
9.1 용어와 정의 248
9.2 이진 트리의 표현 254
9.3 이진 트리의 연산 258
9.4 일반적인 트리의 표현 267
9.5 사례 연구: 최적 정렬 시간 270
9.6 사례 연구: 수식 트리 272
연습문제 276

제10장. 트리 응용 ····· 283
10.1 이진 탐색 트리 284
10.2 힙 자료구조 290
10.3 분리된 집합을 위한 트리 표현 300
10.4 사례 연구: 세그먼트 트리 307
연습문제 314

제11장. 그래프 ····· 319
11.1 용어와 정의 320
11.2 그래프의 표현 326
11.3 그래프 탐색 331
11.4 그래프 탐색의 활용 336
11.5 사례 연구: 위상 정렬 340
연습문제 344

제12장. 그래프 응용 ····· 349
12.1 가중치 그래프 350
12.2 최소 신장 트리 352
12.3 최단 경로 트리 363
12.4 사례 연구: 작업 네트워크 371
연습문제 378

제13장. 탐색 구조 ····· 383
13.1 균형 이진 탐색 트리 384
13.2 B-트리 395
13.3 해싱 401
13.4 사례 연구: 파이썬에서의 해싱 407
연습문제 412

찾아보기 ····· 416

제1장

서론: 기본 개념

1.1 알고리즘 개요
1.2 알고리즘의 효율성
1.3 알고리즘의 성능 분석
1.4 차수 표기법
연습문제

학습 목표

- 알고리즘의 정의와 알고리즘이 갖추어야 할 필수 요건을 이해한다.
- 알고리즘의 효율성을 수행 시간과 메모리 사용량으로 평가하는 이유와 그 중요성을 이해한다.
- 프로그램의 '성능 측정'과 알고리즘의 '성능 분석'의 차이점을 이해한다.
- 알고리즘에서 '지배적 연산'을 식별하여 입력 크기에 대한 연산 수행 횟수를 계산하고, 시간복잡도 함수로 나타내는 방법을 이해한다.
- 차수 표기법의 종류와 의미를 이해하고, 차수 사이의 크기를 비교할 수 있다.

1.1 알고리즘 개요

자료구조와 알고리즘을 공부하는 목적은 주어진 문제를 효율적으로 해결하는 프로그램을 감별할 수 있는 안목과 그러한 프로그램을 개발할 수 있는 능력을 배양하는 데 있습니다. 프로그램의 효율성은 수행 시간과 메모리 사용량에 의해 결정됩니다. 프로그램의 수행 시간은 컴퓨터의 핵심 자원인 CPU를 차지하고 있는 시간을 의미하며, 이는 사용자의 대기 시간에 영향을 끼칩니다. 프로그램의 메모리 사용량은 프로그램의 실행에 필요한 메모리의 크기를 의미합니다. 특히 메모리 용량이 제한적인 임베디드 시스템이나 모바일기기에서는 프로그램의 성공적인 수행을 좌우하는 매우 중요한 요소가 됩니다.

주어진 문제를 해결하는 절차나 해법을 **알고리즘(algorithm)**이라고 합니다. 컴퓨터 프로그램은, 알고리즘을 바탕으로, 컴퓨터가 수행할 명령을 특정 프로그래밍 언어를 사용하여 구체적으로 나열한 것입니다. 반면에 알고리즘은 아무런 문법적 제약 없이 자유로운 형식으로 문제 해결 절차를 표현한 것을 의미합니다. 따라서 알고리즘은 프로그램을 추상화한 것이고, 반대로 프로그램은 알고리즘을 구현한 것이라고 말할 수 있습니다. 프로그램을 개발하는 과정에서 처음에는 문제 해결 절차를 알고리즘으로 작성하게 됩니다. 궁극적으로는 프로그램으로 구현되어야 하므로, 알고리즘을 작성할 때 다음의 요건을 충족하도록 유의해야 합니다.

① 알고리즘의 각 단계는 명확하고 모호하지 않아야 한다.
② 모든 명령은 컴퓨터가 수행할 수 있어야 한다.
③ 알고리즘은 유한한 단계를 수행한 후 반드시 종료되어야 한다.

예를 들어, 다음과 같은 명령이 사용되었다고 가정해 봅시다.

> 변수 A에 100의 약수를 배정하라.

이 명령은 “100의 모든 약수를 배정하라”는 것인지, 아니면 “약수 중 하나를 선택해서 배정하라”는 것인지 명확하지 않습니다. 따라서 위의 첫 번째 요건을 충족하지 못합니다. 알고리즘에는 “가장 큰 약수를 저장하라” 또는 “모든 약수를 리스트의 형태로 저장하라” 등과 같이 명확한 표현을 사용해야 합니다. 두 번째 요건은 당연한 내용이지만, 알고리즘을 작성하다 보면 이 요건을 충족하지 못하는 경우가 발생하기도 합니다. 예를 들어, 다음과 같은 명령을 고려해 봅시다.

> 5차 방정식의 모든 근을 구하여 크기 순서대로 출력하라.

위 명령이 어떤 일을 해야 하는지는 명확합니다. 그러나 수학자 아벨(Abel)에 의해 5차 이상의

일반적인 대수 방정식에서는 근의 공식이 존재하지 않는다는 것이 증명되었습니다. 따라서 이 명령은 컴퓨터가 수행할 수 있도록 일반적인 프로그램으로 구현하는 것이 불가능합니다. 그러므로 알고리즘을 작성할 때는 프로그램으로의 구현 가능성을 항상 염두에 두어야 합니다. 세 번째 요건은, 알고리즘이 모든 입력에 대해 무한 루프에 빠지지 않고 종료 시에 결과를 제공해야 한다는 것을 의미합니다. 알고리즘이 종료하지 않으면 결국 문제의 답을 제공할 수 없다는 점을 강조한 것입니다.

알고리즘을 컴퓨터 프로그램의 형식으로 직접 작성한다면 위에서 언급한 첫 번째(명확성)와 두 번째(수행 가능성) 요건을 자연스럽게 만족하게 됩니다. 그러나 프로그램은 컴퓨터가 수행해야 할 명령을 지나치게 상세하고 때로는 번거롭게 기술해야 하므로, 문제 해결의 핵심 아이디어를 명료하게 표현하는 데는 오히려 장애가 될 수 있습니다.

알고리즘을 표현하는 다른 방법으로 사람이 쓰는 **자연어(natural language)**를 이용하면 핵심 내용을 간결하고 이해하기 쉽게 나타낼 수 있다는 장점이 있습니다. 그러나 이 경우 위에서 언급한 첫 번째, 두 번째 요건을 충족하는지 항상 유의하고 점검해야 합니다. 또한, 자연어로 작성된 알고리즘은 나중에 프로그램으로 구현하는 과정에서 모호성으로 인해 어려움이나 불편을 겪을 수 있다는 단점도 있습니다.

이러한 단점을 보완하기 위해 널리 사용되는 알고리즘 표현 방법인 **가상코드(pseudo-code)**는 프로그래밍 언어와 자연어를 혼용하여 알고리즘을 나타냅니다. 즉 전체적인 구조는 프로그램과 유사하게 구성하고 세부적인 부분은 자연어로 표현하는 방식입니다. 가상코드로 작성된 알고리즘은 자연어 알고리즘보다 더 쉽게 프로그램으로 구현할 수 있습니다.

일반적으로 복잡하거나 어려운 문제를 해결할 때, 처음에는 자연어를 사용하여 문제 해결의 핵심 아이디어를 기술하거나 자유로운 형식으로 절차를 작성하고, 그 후에 가상 코드로 알고리즘을 좀 더 상세하게 작성하고, 마지막 단계에서 프로그램을 완전히 구현하는 방식을 따릅니다.

예를 들어, 알고리즘 1.1은 두 자연수의 최대공약수를 구하는 알고리즘을 자연어로 기술한 것이고, 알고리즘 1.2는 동일한 알고리즘을 가상코드를 사용하여 함수 형식으로 표현한 것입니다[1].

---

1) 이 알고리즘은 두 자연수의 최대공약수를 구하는 유클리드 호제법(Euclidean algorithm)을 표현하고 있습니다.

알고리즘 1.1: 최대공약수를 구하는 자연어 알고리즘

```
1 두 자연수를 입력받아 변수 one과 two에 배정한다.
2 two가 0이면 one을 출력하고 종료한다.
3 one을 two로 나눈 나머지를 다시 one에 배정한다.
4 one과 two의 값을 서로 바꾼다.
5 2번 명령부터 다시 수행한다.
```

알고리즘 1.2: 최대공약수를 구하는 가상코드

```
1 def gcd(one, two):    # 두 자연수 one과 two의 최대공약수를 반환한다.
2    while two > 0:
3       one을 two로 나눈 나머지를 다시 one에 배정한다.
4       one과 two의 값을 서로 바꾼다.
5    return one
```

이 알고리즘을 파이썬 프로그램으로 구현하면 프로그램 1.1과 같습니다.

프로그램 1.1: 최대공약수를 구하는 파이썬 프로그램 (반복 구조)

```
1 def gcd(one, two):
2    """두 자연수 one과 two의 최대공약수를 반환한다."""
3    while two > 0:
4       one, two = two, one % two
5    return one
```

이 알고리즘을 프로그램 1.2와 같이 재귀 함수(recursive function)로도 구현할 수 있습니다. 재귀 함수에 관한 자세한 내용은 4장을 참고하세요.

프로그램 1.2: 최대공약수를 구하는 파이썬 프로그램 (재귀 함수)

```
1 def gcd(one, two):
2    """두 자연수 one과 two의 최대공약수를 반환한다."""
3    if two == 0: return one
4    return gcd(two, one % two)
```

1.2 알고리즘의 효율성

어떤 프로그램이 좋은 프로그램일까요? 좋은 프로그램의 기준으로 다음 항목들을 고려할 수 있습니다.

- 알고리즘이 명확하고 이해하기 쉽다.
- 오류를 찾아 수정하는 디버깅이 쉽다.
- 프로그램의 일부를 재사용할 수 있다.
- 컴퓨터 자원을 효율적으로 활용한다.
- 속도가 빠르다.

이처럼 좋은 프로그램의 기준은 다양하지만, 자료구조와 알고리즘의 관점에서는 '얼마나 효율적인가'를 기준으로 평가합니다. 즉, 알고리즘의 수행 시간과 알고리즘의 수행에 필요한 메모리 사용량은 알고리즘의 효율성을 결정하는 가장 중요한 평가 척도입니다. 물론, 실제 프로그래밍 현장에서는 알고리즘의 효율성만큼이나 코드의 가독성과 유지보수의 용이성 또한 중요하게 여겨집니다. 아무리 빠른 알고리즘이라도 다른 사람이 이해할 수 없거나 수정하기 어렵다면 좋은 프로그램이라고 보기 어렵습니다. 이 책에서는 알고리즘의 효율성에 초점을 맞추어 학습하지만, 좋은 프로그램의 다른 기준들도 항상 염두에 두어야 합니다.

먼저 알고리즘의 효율성을 비교해 볼 수 있는 예를 살펴보겠습니다. 프로그램 1.3과 프로그램 1.4는 1부터 n까지의 합을 출력하는 프로그램입니다. 여기서 n은 입력으로 주어지는 값입니다. 프로그램 1.3은 반복문을 사용하여 합을 구하고, 프로그램 1.4는 등차급수의 합 공식을 이용하여 답을 계산합니다. 프로그램 1.3은 입력된 n의 값만큼 for 문이 반복됩니다.

프로그램 1.3: 반복문을 이용하여 1부터 n까지의 합을 계산하는 프로그램

```
1 n = int(input())
2 total = 0
3 for x in range(1, n+1):
4    total += x
5 print("sum =", total)
```

프로그램 1.4: 등차급수의 합 공식을 이용하여 합을 계산하는 프로그램

```
1 n = int(input())
2 print("sum = ", n*(n+1) // 2)
```

위 프로그램을 실행하는 컴퓨터의 성능이 1초에 for 문을 1억 번 반복시킬 수 있다고 가정해 봅시다. 입력값 n이 1억 이하면 1초 안에 곧바로 답이 출력됩니다. 그러나 n이 10억이면 답이 출력되는데 대략 10초가 걸리고, n이 100억이면 대략 100초가 소요됩니다. 반면에 프로그램 1.4는 입력의 크기와 상관없이 프로그램을 실행시키자마자 즉각 답을 출력합니다. 이 두 알고리즘 중에서 어느 쪽이 더 효율적인지는 누구나 쉽게 판별할 수 있을 것입니다.

다른 예시로, 소수(prime number)의 개수를 구하는 알고리즘을 살펴보겠습니다. 프로그램 1.5의 is_prime() 함수는 소수를 판별하는 일반적인 방법을 사용한 것으로, 정수 x에 대해 2와 $\sqrt{x}$ 사이의 정수 중에 x의 약수가 존재하지 않으면 소수로 판단합니다. 함수 count_primes_ normal(n)은 자연수 n보다 작거나 같은 소수의 개수를 반환합니다. 이를 위해, 2부터 n까지의 각 정수 x를 인자로 하여 is_prime(x) 함수를 호출합니다.

프로그램 1.5: 일반적인 소수 판별 방식으로 소수의 개수를 세는 프로그램

```
1  def is_prime(x):
2     """x가 소수이면 True, 아니면 False를 반환한다."""
3     m = int(x ** 0.5)                # x의 제곱근
4     for i in range(2, m+1):
5        if x % i == 0:                # x의 약수 발견
6           return False
7     return True
8
9  def count_primes_normal(n):
10    """n 이하의 소수 개수를 반환한다."""
11    count = 0
12    for x in range(2, n+1):
13       if is_prime(x):
14          count += 1
15    return count
```

프로그램 1.6의 count_primes_era(n) 함수는 고대 그리스의 유명한 수학자 에라토스테네스가 고안한 **에라토스테네스의 체(Sieve of Eratosthenes)**라고 불리는 방법을 구현한 것입니다. 먼저, 길이가 n+1이고 True로 초기화된 리스트 prime을 생성합니다. 그다음, prime에서 2를 제외한 2의 배수에 해당하는 첨자를 가진 장소에 False를 배정합니다. 이어서 3을 제외한 3의 배수에 해당하는 첨자를 가진 장소에 False를 배정합니다. 이러한 방식으로 n의 제곱근($\sqrt{n}$) 이하의 모든 소수에 대해 그 소수의 배수에 해당하는 첨자를 가진 장소에 False를 배정합니다. 그 결과, 2 이상의 소수에 해당하는 첨자를 가진 장소에는 초깃값 True가 그대로 남아 있고, 그 외 합성수에 해당하는 첨자를 가진 장소에는 False가 저장됩니다.

프로그램 1.6: 에라토스테네스의 체를 이용하여 소수의 개수를 세는 프로그램

```
1 def count_primes_era(n):
2     """에라토스테네스의 체를 사용하여 구한 소수의 개수를 반환한다."""
3     prime = [True] * (n+1)                  # True로 초기화된 리스트
4     m = int(n ** 0.5)                       # n의 제곱근
5     for i in range(2, m+1):
6         if prime[i]:                        # i가 소수인 경우
7             for j in range(2*i, n+1, i):    # i를 제외한 i의 모든 배수에
8                 prime[j] = False            # False 배정
9     return prime.count(True)-2      # True로 남아 있는 요소의 개수(0, 1 제외)
```

프로그램 1.5와 프로그램 1.6의 실제 실행 시간을 비교해 보면, 입력값 n이 큰 경우 프로그램 1.6이 훨씬 빠르게 답을 출력한다는 것을 확인할 수 있습니다. 프로그램 1.6이 빠르게 수행되는 이유는 prime 리스트에 소수가 될 수 없는 수(즉, 소수의 배수)의 정보를 저장함으로써 불필요한 연산의 수행을 최소화하였기 때문입니다. 반면에 프로그램 1.5는 입력의 크기와 상관없이 항상 몇 개의 변수만 사용하지만, 프로그램 1.6은 입력값에 비례하는 크기를 가진 메모리(크기가 n인 리스트)가 필요합니다. 파이썬의 경우 n이 1,000,000일 때는 약 8MB(메가바이트), 1,000,000,000일 때는 약 8GB(기가바이트) 크기의 메모리 공간이 필요합니다. 그러므로 수행 시간 면에서는 프로그램 1.6이 효율적이고 메모리 사용량 면에서는 프로그램 1.5가 효율적이라고 말할 수 있습니다.

여러 알고리즘 중에서 가장 빨리 답을 출력하는 알고리즘을 어떻게 식별할 수 있을까요? 한 가지 방법은 알고리즘을 프로그램으로 구현한 뒤 컴퓨터에 직접 실행시켜 정확한 실행 시간을 측정하는 것입니다. 이 방법을 프로그램의 **성능 측정(performance measurement)**이라고 부릅니다. 파이썬에서는 time 모듈을 이용하면 프로그램의 실행 시간을 정확하게 측정할 수 있습니다. time 모듈의 time() 함수는 현재 시각을 반환합니다. 실행 시간을 측정하고자 하는 프로그램 구역의 시작과 끝에서 time.time()을 호출하면, 두 시각 사이의 경과 시간이 해당 구역의 실행 시간이 됩니다. 프로그램 1.7은 이 방식을 사용하여 에라토스테네스의 체 프로그램의 실행 시간을 초 단위로 출력하는 예를 보여 줍니다.

프로그램 1.7: 에라토스테네스의 체 알고리즘의 실행 시간 측정

```
1 import time
2 n = int(input())
3 start = time.time()                    # 시작 시간 기록
4 print(count_primes_era(n))             # 함수 count_primes_era() 호출
5 stop = time.time()                     # 종료 시간 기록
6 print(f"time = {stop-start:.3f}")      # 소수점 셋째 자리까지 출력
```

1.3 알고리즘의 성능 분석

알고리즘의 수행 시간을 비교하기 위해 알고리즘을 일일이 프로그램으로 구현하여 실행시키는 것은 매우 번거롭고 어려운 일입니다. 더구나 프로그램의 실제 실행 시간은 사용하는 컴퓨터 시스템, 프로그래밍 언어, 컴파일러 등 환경적인 요인뿐만 아니라 프로그래머의 코딩 스타일에 따라서도 달라집니다. 따라서 프로그램의 실행 시간은 알고리즘의 수행 시간에 대한 절대적인 기준이 될 수 없습니다. 이런 이유로 알고리즘의 수행 시간을 특정 컴퓨터나 프로그래밍 언어와 상관없이 독립적으로 분석하는, **성능 분석(performance analysis)**이라고 부르는 방법을 도입합니다.

성능 분석에서는 알고리즘의 수행 시간을 실제 시간 단위의 값으로 표현하지 않고 알고리즘에 있는 연산의 수행 횟수로 대신합니다. 예를 들어, 입력 크기가 n일 때 2n 번의 덧셈을 수행하는 알고리즘의 수행 시간을 '2n'으로 나타냅니다. 프로그램에서 사용되는 연산의 종류는 매우 다양하므로, 분석의 편의를 위해 **기본 연산(primitive operation)**이라는 용어를 도입합니다. 기본 연산은 간단한 작업을 수행하는 명령 또는 연산을 지칭합니다. 엄밀하게 말하면, 기본 연산은 일정한 시간에 실행되는 하위 수준의 명령에 해당합니다. 대부분의 산술 연산, 비교 연산, 논리 연산, 한 개의 자료 원소에 대한 입출력 명령 등이 기본 연산에 포함됩니다. 이들 연산은 피연산자의 값에 상관없이 실행 시간이 거의 일정하다는 특성이 있습니다. 예를 들어, 덧셈 연산을 수행하는 두 식 '10+5'와 '123456789+100000'을 각각 계산할 때 걸리는 시간에는 차이가 없습니다.

그러나 지수 연산은 피연산자의 크기에 따라 실행 시간이 크게 달라질 수 있습니다. 예를 들어, 파이썬의 지수 연산자인 '**'를 사용하여 두 식 '10**5'와 '123456789**100000'을 계산할 때는 실행 시간의 차이가 매우 큽니다. 이를 확인하려면 프로그램 1.8을 실행해 보세요. 프로그램에서 피연산자의 값을 변경하여 실행하면 실행 시간도 변하는 것을 볼 수 있습니다. 이러한 특성 때문에 지수 연산은 기본 연산에 포함되지 않습니다[2].

프로그램 1.8: 지수 연산의 실행 시간

```
1 import time
2
3 start = time.time()                    # 시작 시간 기록
4 result = 10**5
```

2) 알고리즘 성능 분석에서 지수 연산 x^y는 기본 연산이 $\log_2 y$ 번 수행하는 것으로 처리합니다.

```
stop = time.time()                          # 종료 시간 기록
print("time1 =", round(stop-start, 3))      # 실행 시간 출력

start = time.time()                         # 시작 시간 기록
result = 123456789**100000
stop = time.time()                          # 종료 시간 기록
print("time2 =", round(stop-start, 3))      # 실행 시간 출력
```

알고리즘 성능 분석에서는 각 기본 연산이 수행된 횟수를 계산하여 이 숫자를 알고리즘 수행 시간의 척도로 사용합니다. 여기서 기본 연산들 사이의 미세한 실행 시간 차이는 무시합니다. 연산의 종류에 상관없이 기본 연산의 수행 횟수를 모두 더하면, 입력의 크기를 인수로 가지는 다항식으로 표현됩니다. 이 다항식을 **시간복잡도 함수(time complexity function)**라고 부릅니다. 여기서 입력 크기는 보통 리스트의 원소 개수처럼 입력 자료에 포함된 '자료 원소의 개수'를 의미하지만, 소수 판별 문제처럼 '입력값' 자체의 크기가 될 수도 있습니다.[3] 예를 들어, 프로그램 1.9를 분석해 보겠습니다.

프로그램 1.9: 알고리즘 성능 분석을 위한 프로그램 예

```
n = int(input())
i = 1
total = 0
while i <= n:
    total += i
    i += 1
print(total)
```

프로그램 1.9에서 배정 연산(=)의 수행 횟수는 3회, 비교 연산(<=)의 수행 횟수는 n+1회, 가산 연산(+=)의 수행 횟수는 2n회, 입출력 명령의 수행 횟수는 2회입니다. 연산의 종류에 상관없이 수행 횟수를 모두 더하면 3n+6이 되는데, 이 결과가 이 프로그램의 시간복잡도 함수입니다. 물론 이러한 분석만으로는 알고리즘의 수행 시간을 정밀하게 나타낼 수 없습니다. 알고리즘의 성능 분석은 정확한 실행 시간을 측정하려는 것이 아니라, 입력의 크기가 늘어남에 따라 알고리즘의 수행 시간이 대략 어떤 비율로 증가하는지를 알아내는 것이 목적입니다. 따라서 성능 분석의 목표는 기본 연산의 수행 횟수인 시간복잡도 함수의 차수입니다.

시간복잡도 함수의 차수 분석을 목적으로 할 때는 차수에 영향을 주는 기본 연산만을 선택한

3) 입력의 크기 또는 문제의 크기라는 용어는 사실 더 복잡한 개념으로 정의됩니다.

후 분석하면 됩니다. 앞으로 그런 연산을 **지배적 연산(dominating operation)**이라고 부르겠습니다. 지배적 연산은 보통 알고리즘의 반복 구조 안에 들어 있으며, 하나 이상의 기본 연산을 지배적 연산으로 둘 수 있습니다. 예를 들어 프로그램 1.9에서 지배적 연산은 `while` 문 안에 있는 '<=' 이나 '+='이 될 수 있습니다. '<=' 하나만 지배적 연산으로 두면 시간복잡도 함수는 n+1이 되고, '+='만 지배적 연산으로 두면 2n이 됩니다. 둘 다 지배적 연산으로 두면 3n+1로 계산됩니다. 이렇게 시간복잡도 함수는 분석에 포함되는 지배적 연산에 따라 다르게 나타날 수 있습니다. 그렇지만 시간복잡도 함수가 일차 함수라는 사실은 변하지 않습니다. 심지어 모든 기본 연산을 포함하고 연산들 사이의 미묘한 실행 시간 차이도 다 고려한다고 하더라도 시간복잡도 함수는 여전히 일차 함수입니다. 따라서 프로그램 1.9의 시간복잡도 함수를 T(n)이라고 할 때, T(n)을 다음과 같이 나타낼 수 있습니다.

$$T(n) = cn+d \qquad (c와\ d는\ 0보다\ 큰\ 상수)$$

위의 식에서 상수 c와 d는 분석에 참여하는 기본 연산의 포함 여부에 따라 값이 바뀌게 되지만, 함수의 차수 자체는 바뀌지 않습니다. 그러므로 알고리즘 성능 분석에서는 지배적 연산을 찾아서 그 수행 횟수만 분석해도 충분합니다.

차수 중심의 분석이 얼마나 유용한지 보여주는 예를 들어 보겠습니다. 네 종류의 알고리즘 A, B, C, D의 시간복잡도 함수를 각각 A(n), B(n), C(n), D(n)이라고 하고, 다음과 같은 식으로 표현된다고 합시다. A(n)은 1차 함수, B(n)과 C(n)은 2차 함수, D(n)은 3차 함수입니다.

$$A(n) = 40{,}000n + 5{,}000{,}000$$
$$B(n) = 2n^2$$
$$C(n) = 10n^2 + 200n + 1{,}000$$
$$D(n) = 3n^3$$

이 네 알고리즘을 구현한 프로그램을 1억(10^8) 개의 기본 연산을 1초에 수행하는 컴퓨터에서 실행시킨다고 가정해 봅시다. n = 100인 경우, 알고리즘 A는 약 9백만 개의 기본 연산을 수행하므로(40,000×100 + 5,000,000), 추정되는 실행 시간은 0.09초가 됩니다. 이런 방식으로 계산하여 구한 실행 시간이 표 1.1에 나와 있습니다. 표에서 볼 수 있듯이, 입력 크기 n이 매우 작은 경우에는 상수 요소가 알고리즘의 효율성에 큰 영향을 주지만, 현실적으로 체감하는 수행 시간의 차이는 미미합니다. 그렇지만, 입력의 크기가 커짐에 따라 차수가 다른 함수들 사이의 수행 시간은 시간 단위가 달라질 정도로 현저하게 큰 차이가 나므로, 시간복잡도 함수의 차수가 알고리즘의 효율성을 결정하는 가장 중요한 요인임을 알 수 있습니다. 반면에 알고리즘 B와 C의 비교에서 볼 수 있듯이 시간복잡도 함수에 있는 낮은 차수의 항은 수행 시간에 의미 있는

영향을 주지 못합니다. 따라서 시간복잡도 함수에서 가장 큰 차수에 대한 정보만 알고 있어도 충분하므로, 알고리즘의 성능 분석은 시간복잡도 함수의 차수 분석에 초점을 맞추게 됩니다.

표 1.1: 입력 크기에 따른 알고리즘의 수행 시간 (추정값)

입력의 크기 n	알고리즘의 수행 시간			
	A(n)	B(n)	C(n)	D(n)
100	0.09 초	0.0002 초	0.001 초	0.03 초
1,000	0.45 초	0.02 초	0.102 초	30 초
10,000	4.05 초	2 초	10 초	약 500 분
100,000	40.05 초	약 3.3 분	약 16.7 분	약 347 일
1,000,000	약 6.7 분	약 5.6 시간	약 28 시간	약 951 년

알고리즘의 수행 시간, 시간복잡도 함수, 시간복잡도는 모두 같은 개념으로 사용됩니다. 앞으로 시간복잡도 함수는 간단히 '시간복잡도'라고 언급합니다.

알고리즘 중에는 입력된 자료의 분포에 따라 수행 시간이 달라지는 경우가 있습니다. 그 예로서 프로그램 1.10에 나와 있는 **순차 탐색(sequential search)** 알고리즘을 살펴보겠습니다. 함수 seq_search(L, item)는 리스트 L에서 요소 item의 위치를 탐색합니다. 리스트에 item이 존재하면 item이 저장된 최초 위치인 첨자를 반환하고, 그렇지 않으면 None을 반환합니다.

프로그램 1.10: 순차 탐색 알고리즘

```
1 def seq_search(L, item):
2     """리스트 L에서 item의 최소 위치를 반환한다."""
3     for i in range(len(L)):
4         if L[i] == item:
5             return i
6     return None
```

리스트 L의 크기, 즉 len(L)을 n이라고 두겠습니다. 프로그램에서 if 문에 있는 '=='은 알고리즘의 지배적 연산입니다. 따라서 이 연산의 수행 횟수를 순차 탐색 알고리즘의 수행 시간으로 나타낼 수 있습니다. 그런데 알고리즘의 수행 시간은 리스트 L에 어떤 값들이 어떤 순서대로 저장되어 있느냐에 따라 달라집니다. 리스트의 맨 앞 요소가 item이면 '=='은 단 한 번만 수행되고 알고리즘이 종료됩니다. item이 저장된 최초 위치의 첨자가 k이면 '=='는 정확하게 k+1번

수행됩니다(0≤k<n). 만일 리스트에 item이 존재하지 않으면 이 연산은 n번 수행됩니다. 이처럼 알고리즘의 수행 시간이 입력 자료의 분포에 따라 달라지는 경우, 시간복잡도를 좀 더 구체적으로 구분하게 됩니다. 입력 자료의 분포가 알고리즘을 가장 빠르게 수행시키는 경우를 "**최상의 경우(bestcase)**"라고 하고, 이 경우의 시간복잡도를 **최상의 경우 시간복잡도**라고 부릅니다. 알고리즘의 수행을 가장 느리게 만드는 입력 자료의 분포를 "**최악의 경우(worstcase)**"라고 하고, 이 경우의 시간복잡도를 **최악의 경우 시간복잡도**라고 부릅니다. 프로그램 1.10의 순차 탐색 알고리즘은 최상의 경우 시간복잡도는 1이고, 최악의 경우 시간복잡도는 n입니다.

평균 시간복잡도(averagecase time complexity)는 입력 자료의 모든 분포에 대한 수행 시간의 평균을 말합니다. 평균 시간복잡도는 입력 자료에 대한 확률 분포를 알고 있어야만 분석할 수 있으므로 일반적으로는 정확하게 분석하기 어렵습니다. 순차 탐색 프로그램에서 리스트 L에 item이 반드시 존재하며 각 장소에 위치할 확률이 모두 같다고 가정할 때의 평균 시간복잡도를 Avg(n)이라고 하면 다음과 같이 계산됩니다.

$$Avg(n) = \frac{1+2+\cdots+n}{n} = \frac{n(n+1)}{2} \cdot \frac{1}{n} = \frac{n+1}{2}$$

최상의 경우 시간복잡도는 사실 큰 의미가 없습니다. 정상적으로는 최상의 경우가 거의 발생하지 않을 뿐만 아니라, 이것만으로는 알고리즘의 성질을 제대로 설명할 수 없기 때문입니다. 반면에 최악의 경우 시간복잡도는 매우 중요합니다. 최악의 경우 역시 발생할 빈도가 낮을 수 있지만, 알고리즘의 수행 시간에 대한 상한(upper bound)을 보장해 준다는 의미가 있습니다. 특히 원자로 제어, 항공기 통제 등 실시간(real-time)으로 처리해야 하는 분야에서는 수행 시간의 상한에 대한 제어는 시스템을 안정적으로 유지하기 위한 필수적 요소입니다. 평균 시간복잡도는 알고리즘의 수행 시간에 대한 **기대 시간(expected time)**을 알려준다는 측면에서 알고리즘의 성질을 가장 잘 나타내는 정보라고 볼 수 있습니다. 그러나 평균 시간복잡도를 계산하기가 어려울 뿐만 아니라 입력 자료에 대한 확률 분포를 알기가 어렵기 때문에 분석 결과가 정확하다고 말하기도 어렵습니다. 그래서 알고리즘의 성능 분석에서는 분석이 비교적 쉬우며 중요한 정보를 알려주는 최악의 경우 시간복잡도를 가장 많이 사용하게 됩니다.

시간복잡도와 비슷한 개념으로, **공간복잡도(space complexity)**는 알고리즘이 수행될 때 필요한 메모리 공간을 입력의 크기에 대한 함수로 나타낸 것입니다. 프로그램 1.3, 1.4, 1.5처럼 입력의 크기와 무관하게 일정한 개수의 변수만 사용하는 알고리즘의 공간복잡도는 상수 함수입니다. 프로그램 1.6의 경우 리스트 크기는 입력의 크기에 비례하기 때문에 공간복잡도는 1차 함수입니다. 알고리즘의 공간복잡도는 시간복잡도에 비해 대체로 분석하기가 쉽습니다.

1.4 차수 표기법

어떤 알고리즘의 시간복잡도가 다음 함수로 나타난다고 가정해 봅시다.

$$T(n) = 10n^2 + 200n + 1000$$

시간복잡도의 차수는 알고리즘의 수행 시간에 절대적인 영향을 주기 때문에, T(n)에서 2차 항의 계수와 낮은 차수의 항은 상대적으로 덜 중요하므로 무시될 수 있습니다. 물론 시간복잡도에서 상수의 값을 무조건 무시해도 된다는 의미는 아닙니다. 특히, 시간복잡도에서 차수가 가장 큰 항의 계수는 수행 시간에 상당한 영향을 줍니다. 똑같은 차수의 시간복잡도를 가진 두 알고리즘의 수행 시간 차이는 이 계수에 의해 결정됩니다. 이런 이유로 프로그램의 실행 시간에 매우 민감한 응용에서는 시간복잡도의 상수를 줄이기 위한 노력이 요구됩니다.

시간복잡도의 차수는 입력의 크기에 대한 수행 시간의 증가율을 결정합니다. 입력의 크기가 작은 구간에서는 표 1.1에서 볼 수 있듯이 차수가 작은 시간복잡도를 가진 알고리즘이 오히려 수행 시간이 더 길어질 수 있습니다. 예를 들어, 시간복잡도가 1차 함수인데, 1차 항의 계수가 10억(10^9)처럼 극단적으로 큰 값이라면, 다른 평범한 2차 함수에 비해 결코 효율적인 알고리즘이라고 말할 수 없을 것입니다. 그렇지만 대부분 경우에는 차수가 다른 시간복잡도를 가진 두 알고리즘의 효율성을 비교하는 데 있어서 상수는 무시될 수 있습니다.

시간복잡도의 증가율만 고려하여, 즉 함수의 차수에 초점을 맞추어 분석하는 방법을 **점근적 분석법(asymptotic analysis)**이라고 합니다. 이 책에서는 점근적 분석법을 사용하여 알고리즘의 효율성을 평가합니다. 따라서 어떤 두 알고리즘의 시간복잡도가 같은 차수를 갖는다면 두 알고리즘의 효율성은 점근적으로 동등한 것으로 취급할 것입니다.

차수 표기법(order notation)은 함수의 차수 정보를 효과적으로 표현할 수 있는 표기법입니다. 지금부터 차수 표기법으로 **O-표기법(빅오 표기법)**, **Ω-표기법(오메가 표기법)**, **Θ-표기법(세타 표기법)**을 차례로 소개합니다.

O-표기법의 수학적인 정의는 다음과 같습니다.

> 두 함수 f(n)과 g(n)이 다음을 만족하면 $f(n) = O(g(n))$이다: $n \geq k$인 모든 n에 대해서, $0 \leq f(n) \leq c \cdot g(n)$을 만족하는 0보다 큰 상수 c와 k가 존재한다.

O(g(n))은 "Big oh of g(n)"으로 읽으며, O-표기법이 차수 표기법에서 가장 많이 사용되기 때문에 "Order of g(n)"으로 읽기도 합니다. O-표기법은 함수의 상한(upper bound)을 나타내기 위한 표기법입니다. 위 정의에 있는 함수 f(n)과 g(n)에서 g(n)의 차수가 f(n)보다 크면, g(n)의 높은 증가율 때문에 n이 커짐에 따라 언젠가는 $f(n) \leq c \cdot g(n)$이 항상 성립하게 될 것입니다(그림 1.1 참조). 이 경우 상수 c는 임의의 양수이면 되고, k는 부등식이 성립하는 n의 가장 작은 값으로 두면 됩니다. 두 함수의 차수가 같으면 f(n)에서 차수가 가장 큰 항의 계수보다 더 큰 상수를 c로 선택하면 $f(n) \leq c \cdot g(n)$을 항상 만족하는 상수 k를 쉽게 찾을 수 있습니다.

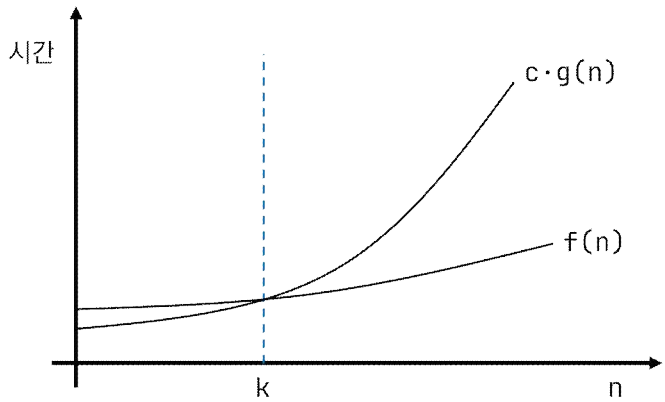


그림 1.1: O-표기법의 정의

유의해야 할 사항으로, "f(n) = O(g(n))"에서 '='는 등호의 개념이 아니라는 것입니다. 영어식 표현으로 "f(n) is O(g(n))"으로 해석하는 것이 적절합니다. 그래서 "O(g(n)) = f(n)"으로 작성하는 것은 옳은 표현이 아닙니다. 어떤 책에서는 O(g(n))을 g(n)보다 차수가 작거나 같은 함수들의 집합으로 정의하기도 합니다. 이런 경우는 "$f(n) \in g(n)$"으로 표현합니다. 이렇게 차수 표기법을 집합 개념으로 봐도 무방하지만, 여기서는 O(g(n))을 차수가 g(n)보다 작거나 같은 함수를 의미하는 표기법으로 정의합니다.

O-표기법을 좀 더 이해하기 위해 다음의 세 함수를 예로 들어 보겠습니다.

$A(n) = 40{,}000n + 5{,}000{,}000$
$B(n) = 2n^2$
$C(n) = 10n^2 + 200n + 1{,}000$

A(n) = O(n)이지만, 정의에 따르면 A(n) = O(n^2) 또는 A(n) = O(n^3)이라고 하더라도 틀린 표현이 아닙니다. 하지만 'O' 안에 들어가는 함수는 상한에 해당하는 함수 중 가장 근접한(tight) 함수를 선택합니다. 그래야 더 정확하고 도움이 되는 정보를 제공하게 됩니다. B(n)과 C(n)은 둘 다 2차 함수이므로, B(n) = O(C(n)) 또는 C(n) = O(B(n))으로 표현해도 됩니다. 또는 B(n) = O($5n^2 + 3n$), C(n) = O($25n^2 + 12n + 9$)로 표현해도 됩니다. 하지만 'O' 안에 들어가는 함수로는 차수 정보를 잘 나타내는 가장 간단한 함수를 선택하는 것이 좋습니다. 따라서 B(n) = O(n^2), C(n) = O(n^2)으로 작성하는 것이 가장 적절한 표현입니다.

Ω-표기법에서 Ω는 'Omega'라고 읽는 그리스 문자입니다. Ω(g(n))을 "Omega of g(n)"으로 읽습니다. 수학적인 정의는 다음과 같습니다.

> 두 함수 f(n)과 g(n)이 다음을 만족하면 f(n) = Ω(g(n))이다: n ≥ k인 모든 n에 대해서, f(n) ≥ c · g(n) ≥ 0을 만족하는 0보다 큰 상수 c와 k가 존재한다.

Ω-표기법은 함수의 하한(lower bound)을 나타내기 위한 표기법입니다. 위 정의에 있는 함수 f(n)과 g(n)에서 g(n)의 차수가 f(n)보다 작은 경우는 f(n)의 높은 증가율 때문에 n이 커짐에 따라 언젠가는 f(n) ≥ c · f(n)이 항상 성립하게 됩니다. f(n)과 g(n)의 차수가 같다면 f(n)에서 차수가 가장 큰 항의 계수보다 더 작은 상수 c를 선택하면 f(n) ≥ c · g(n)을 항상 만족하는 상수 k를 쉽게 찾을 수 있습니다.

앞의 세 함수를 예로 들면, A(n) = Ω(n)으로, B(n) = Ω(n^2)으로, C(n) = Ω(n^2)으로 나타낼 수 있습니다. B(n) = Ω(n)이라고 두더라도 틀린 것은 아니지만, 'Ω' 안에 들어가는 함수는 하한에 해당하는 함수 중 가장 근접한 함수를 선택하는 것이 좋습니다.

Θ-표기법은 영어로 'Theta'라고 읽고, Θ(f(n))을 "Theta of f(n)"으로 읽습니다. 수학적인 정의는 다음과 같습니다.

> 두 함수 f(n)과 g(n)이 다음을 만족하면 f(n) = Θ(g(n))이다: n ≥ k인 모든 n에 대해서, 0 ≤ c · g(n) ≤ f(n) ≤ d · g(n)을 만족하는 0보다 큰 상수 c, d, k가 존재한다.

Θ-표기법은 정확한 차수를 표현하기 위한 표기법입니다. f(n) = Θ(g(n))이 성립하려면 f(n) = O(g(n))인 동시에 f(n) = Ω(g(n))이어야 합니다. 앞의 세 함수에 대해서, A(n) = Θ(n)으로, B(n) = Θ(n^2)으로, C(n) = Θ(n^2)으로 나타낼 수 있습니다. 정의에 의해 B(n) ≠ Θ(n^3)이고 B(n) ≠ Θ(n)입니다.

다음은 차수 표기법에 대한 몇 가지 성질을 나타낸 문장입니다. (증명은 연습문제로 남겨둡니다.)

- $f(n) = O(g(n))$이고 $g(n) = O(h(n))$이면, $f(n) = O(h(n))$이다.
- $f(n) = \Omega(g(n))$이면 $g(n) = O(f(n))$이다. 그 역도 성립한다.
- $f(n) = \Theta(g(n))$이면 $g(n) = \Theta(f(n))$이다.
- $O(f(n)) \cdot \Theta(g(n)) = O(f(n) \cdot g(n))$
- $\Omega(f(n)) \cdot \Theta(g(n)) = \Omega(f(n) \cdot g(n))$

어떤 알고리즘의 시간복잡도가 $O(f(n))$으로 분석되었다면, 이 알고리즘의 정확한 시간복잡도는 $f(n)$보다 차수가 작을 수도 있고 같을 수도 있습니다. 따라서 이 알고리즘이 수행될 때 $f(n)$의 상수 배에 해당하는 수행 시간 안에는 수행이 완료된다는 것이 보장됩니다. 알고리즘의 시간복잡도가 $\Omega(f(n))$으로 분석되었다면, 이 알고리즘의 정확한 시간복잡도는 $f(n)$보다 차수가 크거나 같습니다. 따라서 이 알고리즘의 수행이 완료되기 위해서는 적어도 $f(n)$의 상수 배에 해당하는 수행 시간 이상을 기다려야 합니다. 알고리즘의 시간복잡도가 $\Theta(f(n))$으로 분석되었다면, 이 알고리즘의 정확한 시간복잡도는 $f(n)$과 차수가 같습니다. 따라서 이 알고리즘이 수행될 때 $f(n)$의 상수 배에 해당하는 시간을 기다리면 수행이 완료됩니다.

알고리즘의 시간복잡도를 정확하게 분석할 수 있으면 Θ-표기법을 사용하는 것이 가장 좋습니다. 알고리즘의 시간복잡도를 분석하기가 어려운 경우는 시간복잡도의 상한과 하한을 구해 각각 O-표기법과 Ω-표기법으로 그 사실을 나타내게 됩니다. 알고리즘의 시간복잡도가 입력의 특성(예, 최악의 경우)에 영향을 받는 경우, O-표기법을 사용하여 최악의 경우 시간복잡도를 표현할 수 있습니다. 이러한 이유로 O-표기법이 주로 사용되고 있으며, Θ-표기법을 사용할 수 있는 곳에서도 O-표기법을 사용하는 경우가 많습니다.

다음으로 알고리즘의 성능 분석에서 가장 많이 볼 수 있는 차수 표기법의 종류와 성질에 대해서 알아보겠습니다.

$O(1)$은 **상수 함수(constant function)**를 나타냅니다. 상수 함수는 "$f(n) = c$"의 형태로 주어지는 함수입니다. 여기서 상수 c는 1, 5, 20, 2^{10} 등 고정된 값입니다. 차수 표기법으로 나타낼 경우, $O(5)$, $O(20)$, $O(2^{10})$ 등 모두 가능하지만, 상수 중에서 가장 간단한 값인 1을 가지고 $O(1)$로 표현하는 것이 일반적입니다. 입력의 크기와 무관하게 프로그램이 일정한 횟수의 기본 연산을 수행하는 경우, "이 알고리즘은 상수 시간에 수행된다", 또는 "상수 시간 알고리즘이다"라고 말합니다.

$O(\log n)$은 **로그 함수(logarithm function)**를 나타냅니다. 로그 함수는 알고리즘 분석 과정에서 자주 만나는 함수입니다. 로그 함수는 매우 느리게 증가합니다. 이는 로그 함수와 지수 함수가 서로 역함수 관계에 있다는 사실로부터 알 수 있습니다. 즉, $b^y = x$이면 $y = \log_b x$입니다. 여기서 b를 로그 함수의 밑(base)이라고 부르는데 1보다 큰 상수입니다. 컴퓨터에서 다루어지는 로그 함수는 보통 밑이 2이기 때문에 이 책에서도 다른 언급이 없으면 항상 밑을 2라고 가정하고 주로 생략한 상태로 표시합니다. 즉, $\log n = \log_2 n$입니다. 그림 1.2에서 로그 함수를 그래프로 그린 모습을 볼 수 있습니다.

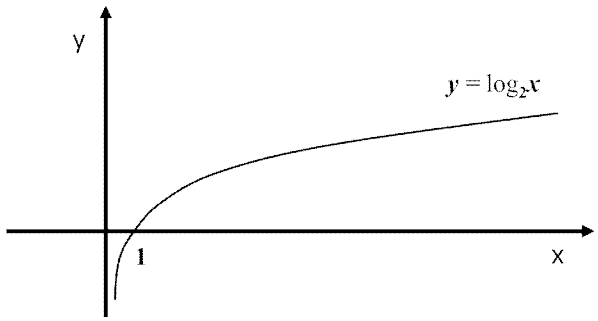


그림 1.2: 로그 함수

$O(n)$은 일차 함수인 **선형 함수(linear function)**를 나타냅니다. 파이썬의 리스트에서 순차 탐색을 수행하거나 가장 큰 원소를 찾는 등 리스트에 저장된 요소들을 최소한 한 번 이상 검색해야 하는 작업의 경우, 선형 함수는 작업의 목적을 달성할 수 있는 최상의 수행 시간입니다. 1차 항과 로그 항이 함께 있는 함수가 있을 때, 차수가 큰 항이 1차 항이므로 이 함수를 선형 함수라고 부를 수 있습니다. 예를 들어, 아래 함수는 선형 함수입니다.

$$5n + 2\log n + 10 = O(n)$$

$O(n \log n)$은 **로그선형 함수(log-linear function)**를 나타냅니다. 로그선형 함수를 편의상 "n-로그-n" 함수라고 부르는 경우가 많습니다. 이 함수는 선형 함수보다는 조금 더 빠르게 증가하고 이차 함수보다는 훨씬 느리게 증가합니다. 그래서 로그선형 함수를 시간복잡도로 가지는 알고리즘을 매우 효율적인 알고리즘으로 취급합니다.

$O(n^2)$은 **이차 함수(quadratic function)**, $O(n^3)$은 **삼차 함수(cubic function)**를 나타냅니다. 프로그램에서 이중 루프나 삼중 루프가 있으면 보통 시간복잡도가 이차 함수 또는 삼차 함수가 됩니다. k가 0보다 큰 수일 때, $O(n^k)$를 **다항식(polynomial)** 또는 **다항 함수**라고 부릅니다. k가 커질수록 증가율이 매우 높아지기 때문에 반대로 알고리즘의 효율성은 급격하게 하락합니

다. 그렇지만 다음에 소개할 지수 함수에 비하면 증가율이 낮아서 효율적인 알고리즘과 비효율적인 알고리즘을 구별하는 경계로 다항 함수를 언급하기도 합니다.

마지막으로 $O(k^n)$은 **지수 함수(exponential function)**를 나타냅니다. 지수 함수는 기하급수적으로 증가한다고 말할 때 언급되는 함수로서 증가율이 엄청나게 높습니다. 그래서 지수 함수를 시간복잡도로 가지는 알고리즘은 대부분 실용성이 떨어집니다. 답을 출력하는데 100년 또는 1,000년이라는 비현실적인 시간이 필요하다고 상상해 보세요. 지수 함수에서는 k의 값이 크면 차수가 더 큰 함수입니다. 즉 $O(3^n)$은 $O(2^n)$보다 차수가 큽니다. 반면에 로그 함수는 밑의 크기와 상관없이 차수가 같습니다. 예를 들어, $O(\log_2 n)$과 $O(\log_3 n)$은 같은 차수를 가진 것으로 취급합니다.

시간복잡도가 $O(1)$인 알고리즘을 상수 시간 알고리즘이라고 부르고, 차수에 따라 각각 로그 시간 알고리즘, 선형 시간 알고리즘, 로그선형 시간 알고리즘, 이차 시간 알고리즘, 삼차 시간 알고리즘, 다항 시간 알고리즘, 지수 시간 알고리즘 등으로 부릅니다.

표 1.2는 n의 값에 따른 $\log n$, n, $n\log n$, n^2, n^3, 2^n인 함수의 값을 보여줍니다. 이 표를 통해 차수의 크기에 따라 실제 함수의 값이 얼마나 차이가 나는지를 느낄 수 있습니다. 로그 함수는 매우 느리게 증가하여 마치 상수처럼 여겨도 될 수 있다는 것을 알 수 있고, 로그선형 함수의 증가율은 이차 함수보다는 훨씬 더 선형 함수에 가깝다는 것을 볼 수 있습니다. 또한 지수 함수의 엄청난 증가율도 확인할 수 있습니다.

표 1.2: 함수의 값 비교

함수					
$\log n$	n	$n\log n$	n^2	n^3	2^n
3.3	10	33	100	1000	1,024
4.9	30	147	900	27,000	1×10^9
5.6	50	282	2,500	125,000	1×10^{15}
6.6	100	664	10,000	1×10^6	1×10^{30}
10	1,000	9,966	1×10^6	1×10^9	1×10^{301}
13	10,000	132,877	1×10^8	1×10^{12}	-
17	100,000	1.7×10^6	1×10^{10}	1×10^{15}	-
20	1,000,000	2.0×10^7	1×10^{12}	1×10^{18}	-
23	10,000,000	2.3×10^8	1×10^{14}	1×10^{21}	-

그림 1.3에는 여러 함수의 증가율을 비교한 그래프가 나와 있습니다. 지수 함수 $O(2^n)$은 매우 급격하게 증가하는 반면, 로그 함수인 $O(\log n)$은 거의 수평에 가깝게 완만히 증가하는 것을 확인할 수 있습니다.

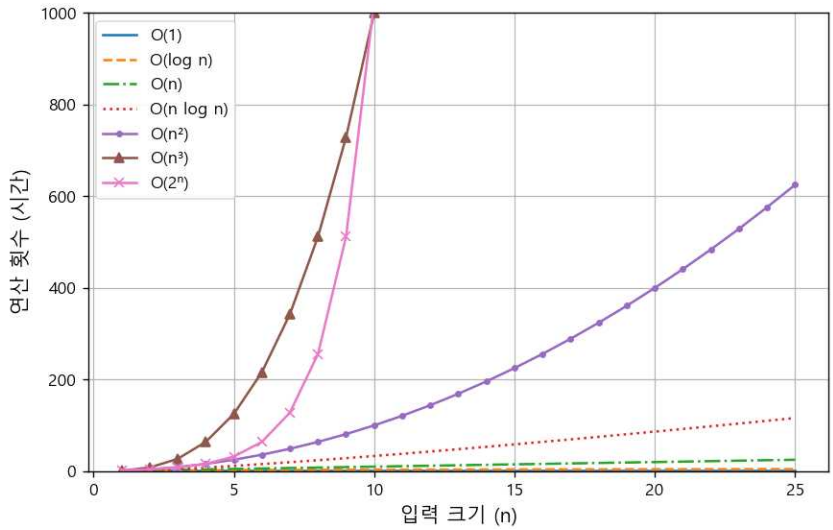


그림 1.3: 시간복잡도 함수의 증가율 비교

표 1.2에 나온 함수를 시간복잡도로 가진 알고리즘을 구현하여 1억(10^8) 개의 기본 연산을 1초에 수행하는 성능을 가진 컴퓨터에 수행시킨다고 가정해 봅시다. 알고리즘의 실행 시간은 기본 연산의 수행 횟수를 1억으로 나눈 값입니다. 로그선형 시간 알고리즘은 입력의 크기가 백만(10^6)인 경우에도 실행 시간이 1초도 걸리지 않습니다. 이차 시간 알고리즘은 입력의 크기가 10^4일 때 1초, 10^5일 때 100초, 10^6일 때는 10^4초(약 3시간)의 실행 시간이 소요됩니다. 삼차 시간 알고리즘은 입력의 크기가 10^3일 때 10초, 10^4일 때는 10^4초(약 3시간), 10^5일 때는 10^7초(약 116일)가 소요됩니다. 지수 시간 알고리즘은 입력의 크기가 30일 때 이미 10초가 소요되고, 그 이상이 되면 상상할 수 없는 비현실적인 실행 시간이 필요합니다.

만일 컴퓨터의 성능이 10배 향상되어 10억(10^9) 개의 기본 연산을 1초에 수행하는 컴퓨터에 수행시킨다고 하면, 위에서 언급한 각 실행 시간에서 10으로 나눈 값만큼의 실행 시간이 소요될 것입니다. 따라서 1초 안에 수행이 종료되기 위한 입력의 최대 크기는 이차 시간 알고리즘의 경우 약 3만이고, 삼차 시간 알고리즘의 경우는 약 1천(10^3), 2^n-시간 알고리즘의 경우는 30입니다. 로그선형 시간 알고리즘의 경우는 천만 크기의 입력에도 1초가 걸리지 않습니다. 이를 통해 컴퓨터의 성능을 높이는 것보다 시간복잡도의 차수를 개선하는 것이 실행 시간을 비약

적으로 개선하는 작업임을 알 수 있습니다. 이것이 차수 중심으로 알고리즘의 성능을 분석하는 이유입니다.

이제 프로그램 1.11에 나와 있는 각 함수의 시간복잡도를 분석하는 과정을 살펴보겠습니다.

프로그램 1.11: 다양한 시간복잡도 분석을 위한 프로그램 예

```
1  def program_A(n):     # 프로그램 A
2     countA = 0
3     for i in range(n):
4        for j in range(n):
5           for k in range(n):
6              countA += 1
7     return countA
8
9  def program_B(n):     # 프로그램 B
10    countB = 0
11    for i in range(n):
12       for j in range(i, n):
13          countB += 1
14    return countB
15
16 def program_C(n):     # 프로그램 C
17    countC = 0
18    for i in range(n):
19       j = n
20       while j >= 1:
21          countC += 1
22          j //= 2
23    return countC
24
25 def program_D(n):     # 프로그램 D
26    countD = 0
27    for i in range(1, n+1):
28       for j in range(i, n+1, i):
29          countD += 1
30    return countD
```

프로그램 1.11에서 매개변수 n의 값이 입력의 크기라고 합시다. 프로그램에서 반복문의 가장 안쪽에 있는 연산 '+='를 지배적 연산으로 둘 수 있습니다. 지배적 연산의 수행 횟수는 각 함수의 시간복잡도를 나타냅니다.

프로그램 A에서 3개의 for 문은 각각 n번씩 반복 수행되며 서로 종속적인 관계에 있으므로 "countA += 1"은 $n \times n \times n = n^3$ 번 수행됩니다. 따라서 프로그램 A의 시간복잡도는 3차 함수이고 $O(n^3)$으로 표기할 수 있습니다. 물론 정확한 차수 정보를 주기 위해서 $\Theta(n^3)$으로 표기해도 됩니다.

프로그램 B에서 첫 번째 for 문의 변수 i는 0부터 n-1까지의 값을 가집니다. 두 번째 for 문은 n-i번 반복 수행됩니다. 따라서 "countB += 1"의 수행 횟수는 $n + (n-1) + \cdots + 2 + 1 = n(n+1)/2$입니다. 그러므로 프로그램 B의 시간복잡도는 2차 함수이고 $O(n^2)$ 또는 $\Theta(n^2)$으로 표기할 수 있습니다.

프로그램 C에서 while 문의 변수 j는 n, n/2, n/4, … , 1의 값을 차례로 가집니다. 1이 될 때까지 반복되는 횟수를 k라고 하면, $n/2^k = 1$이므로 $k = \log_2 n$입니다. 첫 번째 for 문은 n번 반복되므로 "countC += 1"의 수행 횟수는 $n\log_2 n$입니다. 따라서 프로그램 C의 시간복잡도는 로그선형 함수이고 $O(n\log n)$ 또는 $\Theta(n\log n)$으로 표기할 수 있습니다.

마지막 프로그램 D에서 두 번째 for 문은 변수 j의 값이 i씩 증가하므로, 반복 횟수는 $\lfloor n/i \rfloor$입니다. 첫 번째 for 문에서 i는 1부터 n까지의 값을 가지므로, "countD += 1"의 수행 횟수는 $\lfloor n/1 \rfloor + \lfloor n/2 \rfloor + \cdots + \lfloor n/(n-1) \rfloor + \lfloor n/n \rfloor$입니다. 참고로, $n + n/2 + \cdots + n/n = n(1 + 1/2 + \cdots + 1/n) \approx n\log_e n$입니다. 따라서 프로그램 D의 시간복잡도는 로그선형 함수이고 $O(n\log n)$ 또는 $\Theta(n\log n)$으로 표기할 수 있습니다.

앞으로 정확한 차수에 대한 정보를 언급해야 하는 목적에서만 Θ-표기법을 사용하고, 그 외는 Θ-표기법을 사용할 수 있는 곳에서도 O-표기법으로 표기합니다. 그렇지만, O-표기법으로 표현된 차수는 단순한 상한이 아니라 가장 엄밀한(tight) 차수를 나타냅니다.

같은 방식으로 분석해 보면, 앞서 소개한 등차급수의 합을 구하는 프로그램 1.3과 프로그램 1.4의 시간복잡도는 각각 $O(n)$과 $O(1)$임을 알 수 있습니다. 또한, 소수의 개수를 구하는 프로그램 1.5의 시간복잡도는 $\sqrt{2}+\sqrt{3}+\cdots+\sqrt{n} = O(n\sqrt{n})$입니다. 프로그램 D와 수행 과정이 유사한 프로그램 1.6의 에라토스테네스의 체 알고리즘의 시간복잡도는 $O(n\log n)$으로 나타낼 수 있습니다[4]. 두 프로그램은 시간복잡도의 차수가 다르므로, 두 프로그램의 실제 실행 시간의 차이는 입력의 크기가 커질수록 점점 더 벌어집니다.

---

4) 에라토스테네스의 체 알고리즘을 정확하게 분석하면, 프로그램 D와는 달리 소수 p에 대해서만 n/p번 반복합니다. 즉, $n(1/2 + 1/3 + 1/5 + 1/7 \cdots)$. 따라서 정확하게 분석한 시간복잡도는 $O(n\log\log n)$입니다.

연습문제

1. 알고리즘의 요건 중에서 '명확성'은 만족하지만 '수행 가능성'은 충족하지 않은 명령의 예를 쓰시오.

2. 알고리즘 표현 방법으로 자연어, 가상코드, 프로그램의 차이점을 설명하시오.

3. $p(x)$를 차수가 n인 다항식이라고 하자. 즉, $p(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x^1 + a_0$. 입력으로 다항식의 계수 리스트 $(a_n, a_{n-1}, \cdots, a_1, a_0)$와 인수 x가 주어질 때, $p(x)$를 계산하는 파이썬 프로그램을 설계하고 시간복잡도를 분석하시오.

4. 다항식을 다음과 같은 방식으로 계산한다고 하자.

   $$p(x) = a_0 + x(a_1 + x(a_2 + x(a_3 + \cdots + x(a_{n-1} + xa_n) \cdots)))$$

   이 계산법을 **호너의 방법(Horner's method)**이라고 부른다. 이 방법으로 다항식을 계산하는 파이썬 프로그램을 작성하고 시간복잡도를 분석하시오.

5. 입력으로 들어온 값 n보다 작거나 같은 소수의 개수를 출력하는 프로그램 1.5와 프로그램 1.6을 여러분이 사용하는 컴퓨터에서 실행시켜 실행 시간을 측정하고 아래 표의 빈칸에 실행 시간을 소수점 셋째 자리까지 쓰시오.

입력의 크기 n	실행 시간	
	프로그램 1.5	프로그램 1.6
100		
1,000		
10,000		
50,000		
100,000		

6. 아래 표에 나온 각 시간복잡도를 가진 알고리즘을 10억 개의 연산을 1초에 수행하는 컴퓨터에 수행시켰을 때, 1초, 1분, 1시간, 1일, 1년이라는 시간이 걸리는 데이터의 최대 크기를 빈칸에 쓰시오.

시간복잡도	실행 시간				
	1초	1분	1시간	1일	1년
n					
$n\log n$					
n^2					
n^3					
2^n					

7. 시간복잡도가 $T(n)$인 알고리즘을 구현하여 어떤 컴퓨터에 수행시킨다고 하자. 입력의 크기가 100일 때 이 알고리즘의 수행 시간이 3초라면, 입력의 크기가 500일 때는 몇 초가 걸리는가? $T(n)$이 다음과 같다고 할 때, 각 경우에 대해 답을 구하시오.
   (1) $T(n) = 10n$
   (2) $T(n) = 5n^2$
   (3) $T(n) = n^3$

8. 컴퓨터 A는 컴퓨터 B보다 수행 속도가 100배 더 빠르다. 컴퓨터 B가 크기가 10^5인 입력 자료를 1시간에 처리할 수 있다고 하면, 컴퓨터 A는 크기가 최대 얼마인 입력 자료를 1시간에 처리할 수 있는가? 입력 자료를 처리하는 알고리즘의 시간복잡도가 다음과 같다고 할 때, 각 경우에 대해 답을 구하시오.
   (1) n
   (2) n^2
   (3) n^3
   (4) 2^n

9. $10n^2 + 5n + 100 = O(n^2)$임을 O-표기법의 수학적 정의를 이용하여 증명하시오. 즉, $n \geq k$인 모든 n에 대해서, $10n^2 + 5n + 100 \leq c \cdot n^2$을 만족하는 양의 정수 c와 k를 찾으시오.

10. 다음 함수 f()의 시간복잡도를 Θ-표기법으로 나타내시오. (참고, $1 + 1/2 + 1/3 + \cdots + 1/n \approx \log_e n$)

```
def f(n):
   x = 0
   for i in range(1, n+1):
      for j in range(n, 0, -i):
         x += 1
   return x
```

11. 다음 함수 g()의 시간복잡도를 Θ-표기법으로 나타내시오.

```
def g(n):
   x = 0
   for i in range(1, n+1):
      j = 1
      while j <= n:
         x += 1
         j *= 2
   return x
```

12. 다음 함수 h()의 시간복잡도를 Θ-표기법으로 나타내시오.

```
def h(n):
   x = 0
   for i in range(1, n):
      for j in range(i, n):
         for k in range(j+1):
            x += 1
   return x
```

13. 알고리즘 A의 시간복잡도는 $100n$이고, 알고리즘 B의 시간복잡도는 $2n^2$이다. 알고리즘 A가 알고리즘 B보다 더 느리게 수행하는 n의 범위를 구하시오.

14. 두 함수 $n\log n$과 $n^{0.1}$ 중에서 어느 쪽이 차수가 더 큰가? 수학적으로 증명하시오.

15. $n!$은 계승(factorial)이라고 부르는 함수로 다음과 같이 정의된다.

$$n! = n \times n-1 \times n-2 \times \cdots \times 1$$

$n!$과 2^n 중에서 어느 쪽이 차수가 더 큰가? 수학적으로 증명하시오.

16. 다음 함수들을 증가율이 낮은 것부터 높은 순서대로 나열하시오.

$$n!,\ 2^n,\ n\log n,\ n^2,\ 4^n,\ n^{1.5}$$

17. 차수 표기법에 대한 다음 문장이 옳은지 여부를 수학적으로 증명하시오.
    (1) $O(n^2) - O(n^2) = O(1)$
    (2) $1^2 + 2^2 + \cdots + n^2 = \Theta(n^3)$
    (3) $n! = O(n^n)$
    (4) $n^3 2^n + n^2 3^n = O(n^2 3^n)$

18. 차수 표기법에 대한 다음의 각 문장을 수학적으로 증명하시오.
    (1) $f(n) = O(g(n))$이고 $g(n) = O(h(n))$이면, $f(n) = O(h(n))$이다.
    (2) $f(n) = \Omega(g(n))$이면 $g(n) = O(f(n))$이다. 그 역도 성립한다.
    (3) $f(n) = \Theta(g(n))$이면 $g(n) = \Theta(f(n))$이다.
    (4) $O(f(n))\cdot\Theta(g(n)) = O(f(n)\cdot g(n))$
    (5) $\Omega(f(n)))\cdot\Theta(g(n)) = \Omega(f(n)\cdot g(n))$

제2장

파이썬 기초

2.1 파이썬 개요
2.2 자료형과 연산자
2.3 제어 구조
2.4 컬렉션 자료형
2.5 함수와 모듈
2.6 클래스와 객체
연습문제

학습 목표

- 파이썬의 변수 개념과 자료형, 연산, 조건문, 반복문 등의 기본 문법을 익힌다.
- 순서 자료형의 특성과 문자열, 리스트, 튜플을 이해하고, 이를 활용하는 프로그램을 작성할 수 있다.
- 딕셔너리와 집합 자료형을 이해하고, 이를 활용하는 프로그램을 작성할 수 있다.
- 사용자 정의 함수를 직접 작성할 수 있고, 주요 파이썬 내장 함수를 활용할 수 있다.
- 클래스의 개념을 이해하고, 클래스를 정의하고 사용하는 프로그램을 작성할 수 있다.

2.1 파이썬 개요

이 장은 다른 프로그래밍 언어에 익숙하거나 파이썬 복습이 필요한 독자가 이 책을 이해하는 데 필요한 파이썬 기초 문법을 간결하게 다룹니다. 이미 파이썬에 익숙한 독자라면 이 장을 건너뛰고 바로 다음 장으로 넘어가도 무방합니다. 이 장은 파이썬 문법에만 초점을 맞추고 있으므로, 파이썬 설치 및 실행 방법, 통합개발환경(IDE)에 대한 정보는 파이썬 공식 웹사이트(https://www.python.org) 또는 파이썬 입문서나 인터넷 자료를 참고해 주세요.

■ 기본 문법과 코드 블록

파이썬은 **가독성(readability)**을 가장 중요한 철학으로 삼고 있습니다. 이는 코드를 읽는 행위를 코드를 쓰는 행위만큼 중요하게 생각한다는 의미입니다. 파이썬 문법은 이러한 철학을 반영하여, 간결하고 명확하며 마치 자연어처럼 읽히도록 설계되었습니다.

다른 많은 프로그래밍 언어가 코드 블록을 구분하기 위해 중괄호({, })를 사용하는 반면, 파이썬은 **들여쓰기**를 사용합니다(프로그램 2.1 참조). 동일한 들여쓰기 수준을 가진 코드들은 하나의 블록으로 간주됩니다. 이는 코드를 자동으로 정렬시켜 가독성을 크게 높여주는 장점이 있습니다.

프로그램 2.1: 들여쓰기를 사용한 코드 블록 예시

```
1 if 5 > 3:
2     print("5는 3보다 큽니다.")      # 코드 블록의 시작
3     print("조건이 참입니다.")       # 코드 블록의 끝
4 print("이 문장은 블록 바깥에 있습니다.")
```

■ 변수와 동적 타입

변수(variable)는 데이터를 저장하고 관리하기 위한 이름표 역할을 합니다. 변수를 통해 프로그램에서 숫자, 문자열, 리스트 등 다양한 종류의 데이터를 메모리에 저장하고, 이 데이터에 쉽게 접근할 수 있습니다. 변수는 'age = 20'와 같은 **배정 연산(assignment operation)**에 의해 생성됩니다. 변수가 생성되어 사용되는 과정은 다음과 같습니다.

1. 객체 생성: 먼저 데이터(예, 20)를 위한 객체가 메모리에 생성된다.
2. 참조 배정: 변수를 위한 기억장소에 이름(예, age)을 부여하고, 이 장소에 새로 생성된 객체(예, 20)의 메모리 주소(참조값)가 저장된다. 즉 변수는 객체를 '참조'하게 된다.

3. 참조 연결: 이제 변수 age는 객체 20을 가리키고 있으므로, 이 이후 age를 호출하면 이 객체에 접근할 수 있다.

그림 2.1은 변수가 데이터 객체를 참조하는 모습을 보여줍니다.

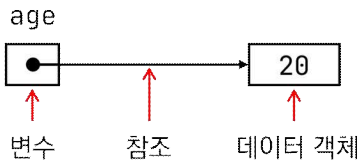


그림 2.1: 배정 연산 'age = 20'으로 생성되는 변수와 데이터 객체

다른 프로그래밍 언어에서는 변수를 위한 기억장소에 데이터를 직접 저장하기도 하지만, 파이썬은 숫자, 문자열, 리스트, 심지어 함수까지 모든 데이터를 객체로 취급하기 때문에, 이러한 **객체(object)**와 **참조(reference)**라는 개념을 사용하고 있습니다. 이 동작 원리에 의해, 다른 많은 언어와 달리, 파이썬은 변수의 자료형을 미리 선언할 필요가 없는 **동적 타입(dynamic typing)**을 지원합니다. 변수에 어떤 값을 배정하느냐에 따라 파이썬 인터프리터가 자동으로 자료형을 결정합니다(프로그램 2.2 참조). 이러한 유연함 덕분에 코드를 더 빠르고 직관적으로 작성할 수 있습니다.

프로그램 2.2: 변수에 값 배정 – 동적 타입의 예

```
1 age = 20            # age는 정수 객체 20을 참조한다.
2 age = "스무살"      # 이제 age는 문자열 객체 "스무살"을 참조한다.
```

프로그램 2.2에서 변수 age에 "스무살"이 배정되면, 기존의 정수 객체 20은 더 이상 age에 의해 참조되지 않으므로, 파이썬의 쓰레기 수집기(garbage collector)는 언젠가 이 객체를 메모리에서 해제합니다.

■ 표준 출력과 표준 입력

파이썬에서 가장 기본적인 표준 출력 함수는 `print()`입니다. 이 함수를 이용하여 문자열, 숫자, 변수 등 다양한 데이터를 화면(콘솔)에 출력할 수 있습니다. `print()` 함수를 호출할 때 여러 개의 인자(argument)를 쉼표(,)로 구분하여 나열하면, 출력되는 인자들 사이에 기본적으로 공백이 들어가고, 출력 후에는 자동으로 줄이 바뀝니다. 매개변수 sep에 문자열을 배정하고 호

출하면, 출력되는 인자들 사이에 공백 대신 이 문자열이 출력됩니다. 매개변수 end에 문자열을 배정하고 호출하면, 모든 인자가 출력된 후에 줄을 바꾸는 것 대신에 이 문자열이 출력됩니다. 프로그램 2.3에 print() 함수의 사용 예가 나와 있습니다.

프로그램 2.3: print() 함수의 사용 예

```
1 print("Hello, Python!")          # 출력 - Hello, Python!
2 print("이름:", "오이식")          # 출력 - 이름: 오이식
3 print("파이썬", "자료구조", sep="-", end="!")
4 print("쉽고", "재밌다.")          # 출력: 파이썬-자료구조!쉽고 재밌다.
```

input() 함수는 사용자로부터 데이터를 표준 입력으로 받을 때 사용됩니다. 사용자가 입력한 내용은 문자열로 반환되며, 사용자가 〈Enter〉 키를 누를 때까지 프로그램의 실행은 정지됩니다. input() 함수의 인자에 문자열을 주면, 사용자에게 보여줄 메시지(프롬프트)가 출력됩니다.

input() 함수는 사용자로부터 어떤 값을 입력받든 항상 문자열(str)로 반환(return)합니다. 따라서, 숫자 데이터를 입력받아 연산을 수행하려면, 자료형 변환 함수인 int()나 float() 함수를 사용하여 명시적으로 자료형을 변환해야 합니다. 자료형 변환 함수는 인자로 주어진 숫자 또는 문자열로 표현된 숫자를 해당 자료형 값으로 변환하여 반환해 줍니다. 프로그램 2.4에 input() 함수의 사용 예가 나와 있습니다.

프로그램 2.4: input() 함수의 사용 예

```
1 name = input("이름을 입력하세요: ")            # 사용자 이름 입력받기
2 print("Hello,", name)
3
4 x = int(input("첫 번째 숫자를 입력하세요: "))    # 숫자 입력받기
5 y = int(input("두 번째 숫자를 입력하세요: "))
6 print("두 숫자의 합은", x+y, "입니다.")
```

2.2 자료형과 연산자

■ 주석과 키워드

주석(comment)은 프로그램의 실행에 영향을 주지 않으면서 코드에 설명을 추가하기 위해 사용

합니다. 주석은 코드가 무엇을 하는지, 왜 그렇게 작성되었는지 등 개발자가 코드를 이해하는 데 필요한 정보를 제공합니다. 파이썬에서는 두 가지 방식으로 주석을 작성할 수 있습니다(프로그램 2.5 참조).

- 한 줄 주석: '#' 기호 뒤에 나오는 모든 내용은 주석으로 처리됩니다. 주로 코드 옆이나 윗줄에 간결한 설명을 달 때 사용합니다.
- 블록 주석: 여러 줄의 주석을 작성할 때는 따옴표 세 개(''' 또는 """)를 사용하여 블록 주석을 만듭니다. 보통 함수나 클래스에 대한 상세한 설명을 달거나, 여러 줄에 걸쳐 긴 설명을 작성할 때 유용합니다. 이렇게 작성된 블록 주석은 **독스트링(docstring)**이라고 불리며, 파이썬의 `help()` 함수나 자동 문서 생성 도구에 의해 자동으로 인식됩니다.

프로그램 2.5: 주석 사용 예

```
1 def add_numbers(a, b):
2     """이 함수는 두 수를 더한 뒤 결과를 반환합니다.
3     매개변수:
4         a (int): 첫 번째 숫자
5         b (int): 두 번째 숫자
6     """
7     return a + b    # 한 줄 주석은 #으로 시작합니다.
```

키워드(keyword)는 파이썬에서 특별한 의미를 가지며, 언어의 문법과 구조를 정의하는 데 사용되는 **예약어(reserved word)**입니다. 키워드는 변수 이름이나 함수 이름 등 다른 용도로 사용할 수 없습니다. 이들은 주로 제어 흐름을 관리하거나, 데이터 구조를 정의하거나, 특정 연산을 수행하는 데 사용됩니다. 파이썬 키워드는 다음과 같습니다.

```
False      async      del        from       lambda     return
None       await      elif       global     nonlocal   try
True       break      else       if         not        while
and        class      except     import     or         with
as         continue   finally    in         pass       yield
assert     def        for        is         raise
```

키워드를 변수명으로 사용하려고 시도하면 문법 오류가 발생합니다. `input()`이나 `print()`와 같은 내장 함수는 키워드가 아니기 때문에, 변수명으로 사용하더라도 오류가 발생하지 않지만, 이 경우 함수의 원래 기능을 더 이상 사용할 수 없으므로 유의해야 합니다.

■ 자료형

자료형(data type)은 데이터의 종류를 구분하기 위해 사용되는 용어입니다. 파이썬에서는 모든 데이터가 객체이므로, 자료형을 **class**라고 표시합니다. 표 2.1에는 파이썬에서 제공하는 내장 자료형의 종류와 **리터럴(literal)**, 즉 자료형 값의 예가 나와 있습니다.

표 2.1: 내장 자료형의 종류와 리터럴의 예

분류	자료형	리터럴의 예
수치 자료형	정수(int)	123, -50, 300000000000000000, 0b1010, 0x7F
	실수(float)	3.14, 0.14, 1.75E-5
	복소수(complex)	4+5j, 2.0-3.5j
논리형	부울(bool)	True, False
순서 자료형	문자열(str)	'hello', "Happy", '''Python Language'''
	리스트(list)	[], [1, 2, 3], ['a', 'b', 'c'], [12, 'kim', 4.2]
	튜플(tuple)	(), (1, 2, 3), ('a', 'b', 'c'), (12, 'kim', 4.2)
해시 자료형	딕셔너리(dict)	{}, {1:'a', 2:'b', 3:'c'}
	집합(set)	set(), {1, 2, 3}, {'kim', 'lee', 'park'}

파이썬에서 정수와 실수는 소수점의 존재 여부에 의해서 구분됩니다. 정수(int)는 소수점이 없는 숫자로서, 메모리 허용 한도 내에서 임의의 크기를 가질 수 있어 매우 큰 정수도 다룰 수 있습니다. 이진수, 8진수, 16진수로 정수를 표현하는 방법으로, 숫자 앞에 0b를 붙이면 이진수, 0o를 붙이면 8진수, 0x를 붙이면 16진수를 의미합니다.

실수(float)는 64비트 부동소수점 방식으로 표현됩니다. 표현 가능한 실수 범위는 대략 10^{-308} ~ 10^{308}입니다. 유효 자릿수는 약 15~17자리이므로, 실수 연산에서 발생할 수 있는 미세한 오차에 항상 유의해야 합니다.

논리형인 부울(bool)은 True 또는 False를 값으로 가지며, 조건문이나 반복문에서 코드의 흐름을 제어하는 데 사용됩니다. 논리값이 수식에서 사용되는 경우, True는 1로, False는 0으로 변환됩니다.

문자열(str)은 따옴표(', ") 또는 따옴표 세 개를 연속으로 나열한 삼중 따옴표로 둘러싸인 텍스트입니다. 문자열을 포함한 순서 자료형과 해시 자료형은 2.4절에서 자세하게 소개합니다.

■ 연산자

파이썬에서 제공하는 연산자는 산술 연산자, 배정 연산자, 비교 연산자, 논리 연산자, 비트 연산자, 그리고 기타 연산자로 구분할 수 있습니다.

산술 연산자(arithmetic operator)는 사칙연산과 기타 수치 연산을 수행하는 연산자로, +(덧셈), -(뺄셈), *(곱셈), /(나눗셈), //(정수 나눗셈), %(나머지), **(거듭제곱) 연산자가 있습니다. 연산자 /는 연산 결과가 항상 실수이고, //는 몫을 구해 줍니다. 예를 들어, 5/2의 결과는 2.5이고 5//2의 결과는 2입니다. 연산자 %는 나눗셈의 나머지를 구해 주고, **는 거듭제곱을 계산합니다. 예를 들어, 5%2의 결과는 1이고, 2**5의 결과는 32입니다.

배정 연산자(assignment operator)는 변수에 값을 배정하는 연산자로서, 연산자(=)의 오른쪽에 있는 값이나 수식의 결과를 왼쪽의 변수에 배정합니다. 'a, b = 1, 2'처럼 작성하면, 변수 a와 b에 각각 1과 2가 배정됩니다. 배정될 값을 항상 먼저 구하기 때문에, 명령 'x, y = y, x'는 x와 y의 값을 서로 바꾸게 됩니다. +=, -=, *= 등의 **복합 배정 연산자**는 산술 연산과 배정을 한 번에 수행합니다. 예를 들어, 'x += 1'은 'x = x+1'과 같습니다.

관계 연산자(relational operator)는 두 피연산자의 관계를 비교하는 연산자로, == (같다), != (다르다), > (크다), < (작다), >= (크거나 같다), <= (작거나 같다) 연산자가 있습니다. 관계 연산의 결과는 True 또는 False입니다. 수치 데이터 사이에 관계 연산자가 사용되면 숫자의 크기를 비교하고, 문자열, 리스트, 튜플과 같은 순서 데이터 사이에 사용되면 사전식 순서(lexicographic order)로 비교합니다. 정수와 문자열처럼 서로 비교할 수 없는 데이터 사이에 사용되면, 오류가 발생합니다. 파이썬에서 관계 연산자가 둘 이상 연속해서 있을 때, **연쇄 비교(chained comparisions)**라는 독특한 방식으로 수행됩니다. 예를 들어, 'a < b < c'는 '(a < b) and (b < c)'와 같은 방식으로 수행합니다.

논리 연산자(logical operator)는 논리식을 결합하는 연산자로, and (논리곱), or (논리합), not (논리 부정) 연산자가 있습니다. 'A and B'는 A와 B가 둘 다 참인 경우에만 식의 결과가 True이고, 하나라도 거짓이면 식의 결과는 False입니다. 'A or B'는 A와 B 둘 중의 하나라도 참이면 식의 결과는 True이고, 둘 다 거짓일 경우에만 False입니다. 'not A'는 A의 논리값을 반대로 바꿔줍니다. 즉, A가 참이면 False, A가 거짓이면 True가 됩니다.

비트 연산자(bitwise operator)는 정수형 데이터의 개별 비트(bit)에 대해 논리 연산을 수행하는 연산자로서, &, ^, ~, <<, >> 연산자가 있습니다. 비트 연산자는 일반적인 프로그래밍에서는 자주 사용되지 않지만, 성능이 중요하거나 하드웨어에 가까운 제어가 필요한 분야에서 활용됩니다. 비트 연산자에 대한 자세한 설명은 생략합니다.

기타 연산자로 조건 연산자인 if else, 멤버 연산자인 in, not in, 항등 연산자인 is, is not이 있습니다. 조건 연산자를 가진 식은 '값1 if 논리식 else 값2'의 형식으로 작성되며, 논리식이 True이면 값1, 아니면 값2가 식의 결과가 됩니다. 예를 들면, 다음 명령은 a가 0보다 크면 x에는 10을, 아니면 x에 20을 배정합니다.

```
x = 10 if a > 0 else 20
```

멤버 연산자는 어떤 값이 컬렉션 데이터에 포함되어 있는지를 검사하는 연산자로 2.4절에서 자세하게 다룹니다. 항등 연산자는 객체의 참조값(메모리 주소)을 비교하기 위한 것입니다. 관계 연산자의 '=='는 객체의 내용을 비교하는 것으로 항등 연산자와 구분됩니다. 프로그램 2.6에 그 예가 나와 있습니다.

프로그램 2.6: is 연산자와 == 연산자 비교

```
1 A = [1, 2, 3]       # 리스트 객체 생성
2 B = A               # 참조값 복사
3 C = [1, 2, 3]       # 새로운 리스트 객체 생성
4 print(A is B)       # 결과: True, A와 B는 동일한 참조값을 가지고 있음
5 print(A is C)       # 결과: False, A와 C의 참조값이 다름
6 print(A == C)       # 결과: True, A와 C는 동일한 내용으로 구성되어 있음
```

■ 연산자 우선순위와 결합법칙

연산자 우선순위(operator precedence)는 수식에 두 개 이상의 연산자가 있을 때, 어떤 연산을 먼저 수행해야 하는지를 결정하는 규칙입니다. 수학의 사칙연산 규칙과 유사합니다. 높은 우선순위의 연산자가 낮은 우선순위의 연산자보다 먼저 실행됩니다. 괄호 ()는 우선순위 규칙을 무시하고, 괄호 안의 내용을 가장 먼저 계산하게 만듭니다.

예를 들어, '2 + 3 * 4'라는 식에서 곱셈(*)은 덧셈(+)보다 우선순위가 높으므로, 3*4가 먼저 계산된 후 2와 더해져 결과는 14가 됩니다. 만약 덧셈을 먼저 계산하고 싶다면 '(2 + 3) * 4'와 같이 괄호를 사용해야 합니다.

연산자 결합법칙(operator associativity)는 동일한 우선순위를 가진 연산자들 사이에 어떤 연산을 먼저 수행해야 하는지를 결정하는 규칙입니다. 결합법칙은 **좌측 결합(left to right)**과 **우측 결합(right to left)** 두 가지로 나뉩니다. 좌측 결합은 왼쪽에서 오른쪽으로 가면서 연산을 수행합니다. 대부분의 이항 연산자(덧셈, 뺄셈, 곱셈, 나눗셈 등)가 여기에 속합니다. 예를 들면, '10 - 5 - 2'는 '(10 - 5) - 2'로 계산되어 결과는 3이 됩니다.

우측 결합은 오른쪽에서 왼쪽으로 가면서 연산을 수행합니다. 거듭제곱 연산자(**)와 배정 연산자(=)가 여기에 해당합니다. 예를 들면, '2 ** 3 ** 2'는 '2 ** (3 ** 2)'로 계산되어 2^9 = 512가 됩니다. 만약 왼쪽에서부터 계산했다면 2^3 ** 2 = 8^2 = 64가 되었을 것입니다. 다른 예로, 'a = b = c = 1'은 가장 오른쪽의 'c = 1'이 먼저 수행되어 변수 c에 1이 배정되고, 그 다음 'b = c'가 수행되어 변수 b에 c의 값 1이 배정됩니다. 마지막으로 'a = b'가 수행되면, 변수 a, b, c에 모두 1이 배정됩니다.

표 2.2는 주요 연산자들 사이의 우선순위와 결합법칙을 보여줍니다. 표에서 위에 위치할수록 우선순위가 높습니다.

표 2.2: 연산자 우선순위와 결합법칙

우선순위	연산자	결합법칙	비고
1	()	좌측 결합	괄호
2	**	우측 결합	거듭제곱
3	*, /, %, //	좌측 결합	곱셈, 나눗셈, 나머지
4	+, -	좌측 결합	덧셈, 뺄셈
5	==, !=, <, <=, >, >=	좌측 결합	비교 연산
6	not	좌측 결합	논리 부정
7	and	좌측 결합	논리곱
8	or	좌측 결합	논리합
9	=, +=, -=, *=, …	우측 결합	배정

2.3 제어 구조

제어 구조는 프로그램의 실행 흐름을 제어하여, 코드의 특정 부분을 선택적으로 실행하거나 반복 수행하게 합니다. 파이썬의 제어 구조는 조건문인 `if` 문, `if-else` 문, `if-elif-else` 문과 반복문인 `while` 문, `for` 문으로 구성됩니다. 파이썬의 제어 구조를 소개하기 전에 파이썬의 독특한 특성인 코드 블록의 개념을 이해해야 합니다.

■ 코드 블록

다른 프로그래밍 언어에서는 코드들의 그룹을 지정하기 위해 보통 중괄호({, })를 사용하는데, 파이썬은 특이하게 들여쓰기를 이용하여 그룹을 지정합니다. 그래서 파이썬에서는 프로그램을 작성할 때, 세로줄, 즉 열에 맞춰 문장을 작성하는 것이 매우 중요합니다.

파이썬에서 동일한 들여쓰기로 작성된 코드들의 그룹을 **코드 블록(code block)**이라고 부릅니다. **최상위 코드 블록(top-level code block)**은 프로그램의 첫 번째 열에서부터 명령이 시작하는 코드 블록입니다. 새로운 코드 블록은 주로 제어 구조, 함수, 클래스에 의해서 생성됩니다. 코드 블록 내부에 다른 코드 블록이 포함될 수 있는데, 이는 제어 구조 안에 다른 제어 구조가 들어갈 수 있다는 것을 의미합니다. 코드 블록의 끝은 처음으로 내어쓰기로 작성된 명령의 바로 앞까지입니다. 만일 내어쓰기로 작성된 명령이 없으면, 코드 블록의 끝은 프로그램의 끝입니다. 같은 코드 블록에 속하는 문장들의 들여쓰기 간격이 다르면 구문 오류가 발생합니다.

■ 조건문

조건문은 사용자의 반응이나 데이터의 내용에 따라 프로그램이 다른 동작을 수행하도록 해 주는 명령으로, 프로그램에 판단과 결정이라는 능력을 부여합니다. 파이썬의 조건문은 특정 조건이 참(True)일 때만 코드 블록을 실행하도록 하여, 프로그램의 실행 흐름을 제어합니다. 조건문에는 `if` 문, `if-else` 문, `if-elif-else` 문이 있습니다.

`if` 문은 가장 기본적인 조건문으로, 키워드 `if` 옆의 조건식이 참일 때만 다음 줄부터 시작하는 코드 블록을 실행합니다. 조건식이 거짓이면 해당 코드 블록을 건너뜁니다. 조건식 끝에는 반드시 콜론(:)을 붙여야 하며, 다음 줄의 코드 블록은 들여쓰기로 작성해야 합니다. 단, 코드 블록이 한 줄의 명령인 경우는 콜론 옆에 위치시킬 수 있습니다. 프로그램 2.7에는 `if` 문의 예가 나와 있습니다.

프로그램 2.7: `if` 문의 사용 예

```
1 score = int(input("점수를 입력하세요 : "))
2
3 if score >= 60:              # [사용 형식] if 조건식 :
4    print("축하합니다.")       # 코드 블록의 시작
5    print("합격입니다.")       # 코드 블록의 끝
6
7 if score < 60: print("불합격입니다.")    # 콜론 옆에 위치한 코드 블록
```

if-else 문은 조건식이 참일 때 수행되는 코드 블록과 거짓일 때 수행되는 코드 블록을 가진 조건문입니다. if와 else 사이에 참인 조건에서 수행되는 코드 블록이 위치하고, else 이후에 거짓인 조건에서 수행되는 코드 블록이 위치합니다. else는 if와 같은 열 위치에 위치해야 하고 끝에 콜론을 붙여야 합니다. 양자택일의 조건일 때, 두 개의 if 문을 사용하는 것보다 if-else 문을 사용하는 것이 프로그램을 더 읽기 쉽게 만들어 줍니다. 프로그램 2.8에는 프로그램 2.7의 두 if 문을 if-else 문으로 바꾼 프로그램이 나와 있습니다.

프로그램 2.8: if-else 문의 사용 예

```
1 score = int(input("점수를 입력하세요 : "))
2
3 if score >= 60:
4    print("축하합니다.")       # 참인 조건에서 실행되는 코드 블록
5    print("합격입니다.")
6 else:
7    print("불합격입니다.")     # 거짓인 조건에서 실행되는 코드 블록
```

if-elif-else 문은 다중 택일을 위한 조건문으로, 앞의 조건이 거짓일 때만 그다음 조건을 검사하는 구조입니다. 첫 번째 이후의 조건식은 elif 옆에 위치하고, elif는 중복해서 나올 수 있습니다. 참인 조건을 만나면 해당 코드 블록이 수행된 후, 남은 조건들은 모두 무시됩니다. 앞의 조건들이 모두 거짓일 때는 else 이후의 코드 블록이 수행됩니다. else는 생략될 수 있습니다. 프로그램 2.9에는 if-elif-else 문의 예가 나와 있습니다.

프로그램 2.9: if-elif-else 문의 사용 예

```
1 score = int(input("점수를 입력하세요 : "))
2
3 if score >= 90:
4    print("A 학점입니다.")
5 elif score >= 80:
6    print("B 학점입니다.")
7 elif score >= 70:
8    print("C 학점입니다.")
9 else:
10   print("F 학점입니다.")
```

조건식의 결과에 따라 다른 조건식을 검사하는 경우는 중첩된 조건문이 구성됩니다. 예를 들어, 윤년을 판별하는 조건문을 살펴보겠습니다. 연도가 400의 배수이면 윤년입니다. 그렇지 않은

경우, 4의 배수이면서 100의 배수가 아니면 윤년입니다. 그 외는 평년입니다. 프로그램 2.10에는 주어진 해가 윤년인지를 판별하는 기준을 중첩된 조건문을 사용하여 검사하는 프로그램이 나와 있습니다.

프로그램 2.10: 윤년 여부를 판별하는 중첩된 조건문

```
year = int(input("연도를 입력하세요 : "))

if year % 400 == 0:
   print("윤년입니다.")
else:
   if year % 4 == 0:
      if year % 100 != 0:
         print("윤년입니다.")
      else:
         print("평년입니다.")
   else:
      print("평년입니다.")
```

논리 연산자를 이용하면 중첩된 조건문을 간결하게 바꿀 수 있습니다. 프로그램 2.11에는 중첩되지 않은 조건문을 사용하여 윤년을 판별하는 프로그램이 나와 있습니다.

프로그램 2.11: 윤년 여부를 판별하는 조건문

```
year = int(input("연도를 입력하세요 : "))

if year % 400 == 0 or year % 4 == 0 and year % 100 != 0:
   print("윤년입니다.")
else:
   print("평년입니다.")
```

if 문 조건의 참과 거짓을 구분하는 값으로 True와 False 외에도 여러 값이 사용됩니다. 거짓으로 취급되는 값에는 False, 정수 0, 실수 0.0, 공백 문자열(""), 공백 리스트([]), None 등이 사용됩니다. None은 '값이 없음'을 나타내는 특별한 객체입니다. 거짓이 아닌 모든 값은 True로 취급됩니다.

■ 반복문

반복문은 특정 코드 블록을 반복해서 실행해 주는 명령으로, 동일한 작업을 빠르게 처리하여 작

업의 효율성을 높여줍니다. 파이썬에서 제공하는 반복문에는 for 문과 while 문이 있습니다. 이 두 반복문은 사용 목적에 따라 구분됩니다.

for 문은 리스트, 튜플, 문자열 등과 같은 **순회 가능한(iterable)** 객체의 각 요소에 대해 코드 블록을 반복 실행합니다. 미리 정해진 횟수만큼 반복하거나 컬렉션의 모든 요소를 처리할 때 유용합니다. for 문의 구조는 다음과 같습니다.

```
for 변수 in 순회_가능한_객체 :
    코드 블록
```

for와 in은 키워드이고, '변수'는 반복할 때마다 순회_가능한_객체의 요소를 하나씩 배정받는 변수입니다. 프로그램 2.12에 for 문의 예가 나와 있습니다.

프로그램 2.12: for 문의 사용 예

```
1 # 리스트 순회
2 fruits = ["apple", "banana", "cherry"]
3 for fruit in fruits:
4     print(fruit, end=' ')      # 출력: apple banana cherry
5 print()
6
7 # 문자열 순회
8 for char in "Python":
9     print(char, end=' ')       # 출력: P y t h o n
```

for 문과 함께 자주 사용되는 range() 함수는 지정된 범위에 속하는 정수들을 생성하는 순회 가능한 객체를 반환합니다. range() 함수는 다음과 같이 세 가지 방식으로 사용할 수 있습니다.

- range(stop) : 0부터 stop-1까지의 연속된 정수 범위 지정
- range(start, stop) : start부터 stop-1까지의 연속된 정수 범위 지정.
- range(start, stop, step) : start부터 step 간격으로 stop-1까지의 정수 범위 지정. 단, step이 음수면 start부터 step 간격으로 stop+1까지 범위 지정

range() 함수의 사용 예는 다음과 같습니다.

```
range(5)          ⇒  0, 1, 2, 3, 4
range(-1, 5)      ⇒  -1, 0, 1, 2, 3, 4
range(5, -3, -2)  ⇒  5, 3, 1, -1
```

range() 함수는 모든 숫자를 미리 생성하지 않고, 필요할 때마다 숫자를 하나씩 생성하는 **지연 평가(lazy evaluation)** 방식을 사용합니다. 그래서 range(1000000000)처럼 매우 큰 범위를 지정해도 메모리 낭비가 없습니다. 이러한 특성 때문에, 정해진 횟수만큼 반복하거나 첨자(index) 기반으로 반복을 수행할 때, 리스트나 튜플을 사용하는 것보다 range()를 사용하는 것이 더 효율적이고 권장되는 방식입니다. 프로그램 2.13에 range() 함수를 사용한 for 문의 예가 나와 있습니다.

프로그램 2.13: range() 함수를 사용한 for 문의 예

```
for x in range(3):
    print(x)            # 출력: 0, 1, 2

for x in range(1, 10, 2):
    print(x)            # 출력: 1, 3, 5, 7, 9
```

while 문은 주어진 조건식이 참인 동안 코드 블록을 반복해서 실행합니다. while 문은 반복 횟수가 명확하지 않고, 특정 조건이 충족될 때까지 반복해야 할 때 유용합니다. while 문의 구조는 if 문의 구조에 키워드 if 대신 while이 들어간 것과 같습니다. 프로그램 2.14에 while 문의 예가 나와 있습니다.

프로그램 2.14: while 문의 사용 예

```
count = 0
while count < 3:
    print(count)            # 출력: 0, 1, 2
    count += 1

letter = input()
while letter != 'q':        # 'q'가 입력될 때까지 입력된 문자열을 출력
    print(letter)
    letter = input()
```

while 문을 작성할 때, 영원히 반복되는 무한 루프(infinite loop)에 빠지는 상황을 만들지 않도록 유의해야 합니다.

■ 제어 구조의 내장 명령

조건문과 반복문에서 pass, break, continue는 특정한 목적으로 사용되는 명령입니다. pass는 파이썬에서 아무 동작도 수행하지 않는(do nothing) 명령입니다. 파이썬은 문법적으로

들여쓰기된 코드 블록이 필요하지만, 개발자가 당장 그 블록 안에 작성할 내용이 없을 때, 오류가 발생하는 것을 막기 위해 사용됩니다. pass는 코드를 구조화하거나, 나중에 구현할 부분을 미리 표시해 둘 때 유용하게 사용됩니다. 프로그램 2.15에 pass의 사용 예가 나와 있습니다.

프로그램 2.15: pass의 사용 예

```
def my_function():
    pass          # 아무것도 하지 않지만 문법 오류를 막음

x = 10
if x > 5:
    pass          # 5보다 클 때는 수행해야 할 작업이 없음
else:
    print("x는 5 이하입니다.")
```

break와 continue는 반복문 내에서 반복을 제어하기 위해 사용되는 명령입니다. break는 실행되는 즉시 반복을 중단하고 가장 가까운 반복문을 완전히 빠져나가고, continue가 실행되면 현재 반복을 건너뛰고 바로 다음 반복으로 넘어갑니다. 프로그램 2.16에 break와 continue의 사용 예가 나와 있습니다.

프로그램 2.16: break와 continue의 사용 예

```
for i in range(5):
    if i == 3:
        break             # i가 3일 때 반복문 종료
    print(i)              # 출력: 0, 1, 2

for i in range(5):
    if i == 3:
        continue          # i가 3일 때 바로 다음 반복으로 넘어감
    print(i)              # 출력: 0, 1, 2, 4
```

2.4 컬렉션 자료형

파이썬에서 **컬렉션(collection)** 또는 **컨테이너(container)** 자료형은 여러 개의 요소를 하나의 변수에 담아 관리하는 파이썬 자료구조를 지칭합니다. 이들은 데이터를 조직화하고, 효율적으로

접근하며, 특정한 연산을 수행합니다. 파이썬은 문자열, 리스트, 튜플, 딕셔너리, 집합 등 다양한 컬렉션 자료형을 제공하며, 각 자료형은 고유한 특징과 사용 목적을 가집니다. 컬렉션 자료형은 순서 자료형과 해시 자료형으로 크게 분류할 수 있습니다.

■ 순서 자료형

순서 자료형(sequence type)은 순서가 있는 요소들의 그룹인 순서 데이터를 다루는 자료형으로 첨자를 통해 각 요소에 접근할 수 있습니다. 첨자는 첫 번째 요소부터 0, 1, 2, … 순서로 부여되며, 역순으로는 -1, -2, -3 … 순으로 부여됩니다. 순서 자료형인 문자열, 리스트, 튜플은 표 2.3에 나와 있는 연산자와 내장 함수를 공통으로 사용할 수 있습니다. 표에서 s와 t는 순서 데이터, n은 정수, x는 요소를 나타냅니다. 연산의 결과로 항상 새로운 객체가 생성됩니다.

표 2.3: 순서 자료형의 연산자와 내장 함수

연산자/내장 함수	이름	동작	예 (s = 'Python')
s[i]	인덱싱	첨자가 i인 요소 반환	s[1] ⇒ 'y'
s[i:j]	슬라이싱	i부터 j-1까지 요소들 반환	s[1:3] ⇒ 'yt'
s + t	결합	s에 끝에 t의 요소를 추가	s+'Code' ⇒ 'PythonCode'
s * n	반복	s를 n번 반복하여 결합	s*2 ⇒ 'PythonPython'
x in s	멤버십	x가 s에 존재하는지 검사	'y' in s ⇒ True
len(s)	길이	s의 길이를 반환	len(s) ⇒ 6
print(*s)	언패킹	s의 요소들을 분리하여 출력	print(*s) ⇒ P y t h o n

슬라이싱 연산의 범위 [i:j]에서 시작 값 i가 생략되면 0으로, 마지막 값 j가 생략되면 len(s)로 처리됩니다. 둘 다 생략되면 전체 범위가 됩니다. 예를 들어, 표 2.3의 예에서 s[:2]는 'Py', s[2:]는 'thon', s[:]는 'Python'을 생성합니다.

■ 문자열

문자열(str)은 유니코드 문자의 시퀀스를 담는 불변(immutable) 자료형입니다. 일단 생성된 문자열의 내용은 변경할 수 없으며, 문자열에 대한 모든 수정 연산은 새로운 문자열 객체를 생성합니다. 문자열은 작은따옴표('), 큰따옴표("), 또는 삼중 따옴표(''', """)로 양쪽을 묶어서 생성할 수 있습니다. 문자열 내에 '\n', '\t', '\\' 등과 같은 이스케이프 시퀀스(escape sequence)가 포함되면, print() 함수를 사용하여 이 문자열을 출력할 때, 특정 명령이 수행됩니다. 예를 들면, '\n'은 새 줄(new line)을 생성하고, '\t'은 탭(tab) 문자, 즉 지정된 크

기의 빈칸을 출력하고, '\\'는 백슬래시(\)를 출력합니다.

f-문자열(f-string)은 파이썬 3.6부터 도입된 방법으로, 문자열 내부에 변수나 수식을 편리하게 포함할 수 있게 해 줍니다. 문자열 앞에 'f'를 붙이면 f-문자열이 되고, 문자열 내부에 포함할 변수나 리터럴을 {} 안에 넣어주면 됩니다. 소수점 자릿수를 지정하거나 출력 칸의 크기를 지정하는 등의 포맷을 위한 부가적인 기능도 가지고 있습니다. 프로그램 2.17에 f-문자열의 예가 나와 있습니다.

프로그램 2.17: f-문자열의 예

```
a = f"name={'kim'}, age={20}"    # 문자열, 숫자 포함
print(a)                         # 출력: name=kim, age=20
b = f"score={2.457:.2f}"         # 소수점 자릿수 지정 (반올림함)
print(b)                         # 출력: score=2.46
num = 123456789
c = f"금액 = {num:,}원"           # 긴 정수에 구분자 추가
print(c)                         # 출력: 금액 = 123,456,789원
```

문자열은 다양한 작업을 수행할 수 있는 풍부한 **메소드(method)**를 제공합니다. 메소드는 객체에 점(.)을 붙여서 호출하는 객체에 내장된 함수입니다. 프로그램 2.18에는 문자열 메소드 strip(), split(), join()을 사용한 예가 나와 있습니다.

프로그램 2.18: 문자열 메소드 strip(), split(), join()

```
msg= "   Hi, Kim   ".strip()     # 앞뒤에 붙은 공백 제거
print(msg)                       # 출력: 'Hi, Kim'
data = "12:02:30".split(':')     # 인자를 기준으로 분리하여 만든 리스트를 반환
print(data)                      # 출력: ['12', '02', '30']
data = " 1 2  3   4".split()     # 인자가 생략되면 공백을 기준으로 분리함
print(data)                      # 출력: ['1', '2', '3', '4']
names = ["kim", "lee", "park"]   # 문자열 리스트
result = ''.join(names)          # 문자열 요소들 합치기
print(result)                    # 출력: kimleepark
result = ', '.join(names)        # 문자열 사이에 ', '를 넣어 합치기
print(result)                    # 출력: kim, lee, park
```

■ 리스트

리스트(list)는 여러 요소를 쉼표(,)로 나열하여 대괄호([,])로 묶어 놓은 데이터입니다. 리스트의 가장 큰 특징은 **변경 가능(mutable)**하다는 점입니다. 즉, 일단 생성된 리스트의 항목들을 자

유롭게 추가, 삭제, 또는 변경할 수 있습니다. 프로그램 2.19는 리스트를 다루는 프로그램의 예를 보여줍니다.

프로그램 2.19: 리스트를 다루는 프로그램의 예

```
L = [7, 9, 1, 5, 10]
mix = [1.5, '사과', [9, 8]]
print(len(L), len(mix))      # 리스트 길이: 5, 3
print(*L)                    # 출력: 7 9 1 5 10 (언패킹)
print([L[2], mix[1]])        # 출력: 1 사과 (인덱싱 연산)
print(L[1:3], mix[:2])       # 출력: [9, 1] [1.5, '사과'] (슬라이싱 연산)
print([1, 2] + [3, 4])       # 출력: [1, 2, 3, 4] (결합 연산)
print([1, 2] * 3)            # 출력: [1, 2, 1, 2, 1, 2] (반복 연산)
L[2] = 4                     # 인덱싱을 이용한 요소 변경
print(L)                     # 출력: [7, 9, 4, 5, 10]
L[1:3] = [1, 2, 3]           # 슬라이싱을 이용한 요소 변경
print(L)                     # 출력: [7, 1, 2, 3, 5, 10]
```

리스트 내포(list comprehension)는 파이썬에서 리스트를 생성하는 간결하고 효율적인 방법입니다. 하나 이상의 반복문과 선택적인 조건문을 사용하여 코드를 한 줄로 작성함으로써, 코드를 깔끔하고 읽기 쉽게 만들어 줍니다. 프로그램 2.20에는 리스트 내포를 이용하여 다양한 리스트를 생성하는 프로그램 예가 나와 있습니다.

프로그램 2.20: 리스트 내포를 이용하여 리스트를 생성하는 프로그램의 예

```
A = [k*k for k in range(1, 10)]             # 10보다 작은 자연수의 제곱
print(A)     # 출력: [1, 4, 9, 16, 25, 36, 49, 64, 81]

B = [k*k for k in range(10) if k%2 == 1]    # 10보다 작은 홀수의 제곱
print(B)     # 출력: L = [1, 9, 25, 49, 81]

# 한 줄에 입력으로 들어온 정수들을 요소로 하는 리스트 생성
C = [int(x) for x in input().split()]       # 입력 예: 11 32 24 41
print(C)     # 출력: [11, 32, 24, 41]

D = [(x, y) for x in "ab" for y in [1, 2]]  # 모든 순서쌍 리스트 생성
print(D)     # 출력: [('a', 1), ('a', 2), ('b', 1), ('b', 2)]
```

리스트는 요소 삽입과 삭제, 복사, 검색과 정렬에 관련된 다양한 메소드를 제공합니다. 표 2.4에는 리스트의 주요 메소드를 기능별로 분류한 내용이 나와 있습니다.

표 2.4: 리스트의 주요 메소드 (L은 리스트 객체)

분류	메소드	설명
삽입, 추가	L.append(item)	L의 끝에 새 요소 item을 추가
	L.insert(k, item)	L의 첨자 k 위치에 새 요소 item을 삽입
	L.extend(obj)	L의 끝에 순회 가능한 객체 obj의 모든 요소를 추가
복사	L.copy()	L의 복사본을 반환 (L[:]과 동일)
삭제	L.remove(item)	L에서 item을 찾아 첫 번째 요소를 삭제
	L.pop(k)	L[k]를 삭제하고 반환, k가 없으면 마지막 요소 삭제
	del L[k]	L[k]를 삭제, L[i:j]는 L[i]~L[j-1]을 모두 삭제
	L.clear()	모든 요소를 삭제하고 공백 리스트로 만듦
검색, 정렬	L.index(item)	item의 최초 위치(첨자)를 반환
	L.count(item)	L에서 item의 개수를 반환
	L.sort()	L의 요소들을 사전식 순서로 정렬
	L.reverse()	요소들의 순서를 반대로 바꿈

■ 튜플

튜플(tuple)은 리스트와 유사하지만, 요소의 변경이 불가능한 불변 자료형입니다. 튜플은 소괄호 ()를 사용하여 생성하며, 요소들이 한 번 정해지면 변경할 수 없습니다. 튜플은 주로 변경되지 않아야 하는 데이터를 보호하거나 여러 개의 값을 한 번에 반환할 때 사용됩니다. 괄호 없이 요소들이 나열된 경우는 튜플 데이터에 해당합니다. 요소를 변경하지 않는 리스트의 연산과 메소드는 튜플에도 사용됩니다. 프로그램 2.21에 튜플을 다루는 프로그램의 예가 나와 있습니다.

프로그램 2.21: 튜플을 다루는 프로그램의 예

```
1  numbers = (2, 7, 14, 26, 14)        # 튜플 생성 - 소괄호 사용
2  names = "kim", "lee", "park"        # 튜플 생성 - 괄호 생략
3  print(numbers)                      # 출력: (2, 7, 14, 26, 14)
4  print(names)                        # 출력: ('kim', 'lee', 'park')
5  x, y, z = names                     # 언패킹(unpacking)
6  print(x, y, z)                      # 출력: kim lee park
7  A = (1)                             # 정수 1, 수식의 괄호로 인식
8  B = (1,)                            # 1개 요소를 가진 튜플
9  print(numbers[1:3] + (2, 3))        # 출력: (7, 14, 2, 3)
10 print(numbers.index(14))            # 출력: 2 (14의 최초 위치)
11 print(numbers.count(14))            # 출력: 2 (14의 개수)
```

■ 해시 자료형

해시 자료형(hash types)은 순서가 없는 요소들의 집합을 다루는 자료형으로, 해시 기반으로 구현되어 있어, 검색과 삽입, 삭제 연산이 매우 빠르게 수행됩니다[5]. 딕셔너리(dict)와 집합(set) 자료형이 여기에 해당합니다.

■ 딕셔너리

딕셔너리(dict)는 **키(key)**와 **값(value)**의 쌍으로 구성된 요소들의 집합을 다루는 자료형으로 변경 가능한(mutable) 자료형입니다. 중괄호 {}를 사용하여 생성하고, 각 요소는 콜론(:)으로 키와 값을 구분합니다. 딕셔너리의 가장 큰 특징은 "키를 통해 값에 빠르게 접근할 수 있다"는 점입니다.

딕셔너리의 모든 요소는 고유한 키를 가집니다. 이 키를 사용하면 연관된 값을 즉시 찾아낼 수 있습니다. 키는 변경 불가능한 객체(예: 숫자, 문자열, 튜플)만 될 수 있지만, 값은 어떤 자료형이든 가능하며 중복될 수도 있습니다. 다음은 3개의 요소를 가진 딕셔너리 객체를 생성하여 변수 person에 배정하는 명령입니다. 콜론 왼쪽의 'name', 'age', 'city'는 키이고, 오른쪽의 'kim', 20, 'seoul'은 각 키에 대응하는 값입니다.

```
person = { 'name': 'kim', 'age': 20, 'city': 'seoul' }
```

프로그램 2.22에는 딕셔너리에 새로운 요소의 삽입, 기존 요소의 값 검색 및 변경, 요소의 삭제, 포함 검사 등을 수행하는 프로그램의 예가 나와 있습니다.

프로그램 2.22: 딕셔너리를 다루는 프로그램의 예

```
person = { 'name': 'kim', 'age': 20, 'city': 'busan' }
print(f"이름: {person['name']}")    # 출력: 이름: kim (값에 접근)
person['job'] = 'student'           # 새로운 요소 삽입
person['age'] = 21                  # 기존 값 변경
del person['city']                  # 키를 가지고 요소 삭제
print(person)   # 출력: {'name': 'kim', 'age': 21, 'job': 'student'}
print(*person)                      # 출력: name age job (키만 출력)
if 'city' not in person:            # 키 비포함 검사 (not in)
   person['city'] = 'incheon'       # 새로운 요소 삽입
for key in person:                  # for 문에서 모든 키 검사
   print(person[key], end=' ')      # 출력: kim 21 student incheon
```

5) 1장에서 소개한 시간복잡도로 평균 $O(1)$ 시간에 수행됩니다.

딕셔너리는 요소를 다루기 위한 다양한 메소드를 제공합니다. 딕셔너리의 주요 메소드를 기능별로 분류한 내용이 표 2.5에 나와 있습니다.

표 2.5: 딕셔너리의 주요 메소드 (D는 딕셔너리 객체)

분류	메소드	설명
검색	D.get(key)	키 key에 해당하는 값 반환, 없으면 None 반환
	D.keys()	D의 모든 키를 담은 순회 가능한(iterable) 객체 반환
	D.values()	D의 모든 값을 담은 순회 가능한 객체 반환
	D.items()	D의 모든 (키, 값) 쌍을 튜플 형태로 반환
복사	D.copy()	D의 복사본을 반환
삭제	D.pop(key)	키가 key인 요소를 삭제하고 그 값을 반환
	del D[key]	키가 key인 요소를 삭제
	D.clear()	모든 요소를 삭제하고 공백 딕셔너리로 만듦

■ 집합

집합(set)은 중복을 허용하지 않는 고유한 요소들의 집합을 다루는 자료형으로 수학에서 다루는 집합과 같은 특성을 가집니다. 중괄호 {}를 사용하거나 set() 함수를 사용하여 집합 객체를 생성합니다. 공백 딕셔너리 {}와 구분하기 위하여 공백 set은 set()으로 표시합니다.

집합의 가장 큰 특징은 요소들의 순서가 정해져 있지 않아 순서 자료형처럼 인덱싱을 사용할 수 없고, 동일한 요소를 여러 개 포함하더라도 자동으로 중복된 요소를 제거한다는 점입니다. set은 요소를 추가하거나 제거할 수 있는 변경 가능한(mutable) 자료형입니다. 수학의 집합론에서 사용하는 연산을 제공하여, 두 개 이상의 집합 간의 관계를 효율적으로 처리할 수 있습니다. 프로그램 2.23에 set 객체를 다루는 연산과 메소드를 사용하는 프로그램 예가 나와 있습니다.

프로그램 2.23: 집합을 다루는 프로그램의 예

```
1 myset = {1, 6, 3, 8, 5, 1, 3, 8}    # set 객체 생성, 자동으로 중복을 제거함
2 print(myset)                        # 출력: {1, 3, 5, 6, 8}
3 myset.add(4)                        # 요소 4를 삽입
4 myset.remove(1)                     # 요소 1을 삭제, 1이 없으면 오류 발생
5 myset.discard(6)                    # 요소 6을 삭제, 6이 없어도 오류 미발생
6 print(myset)                        # 출력: {3, 4, 5, 8}
7
```

```
 8 set1 = {1, 2, 3}
 9 set2 = {3, 4, 5}
10 print(2 in set1, 3 not in set2)    # 출력: True, False (멤버 검사)
11 print(set1 < set2)                 # 출력: False (부분집합 검사)
12 print(set1 & set2)                 # 출력: {3} (교집합 연산)
13 print(set1 | set2)                 # 출력: {1, 2, 3, 4, 5} (합집합 연산)
14 print(set1 - set2)                 # 출력: {1, 2} (차집합 연산)
```

2.5 함수와 모듈

프로그램의 규모가 커지면 코드를 효율적으로 관리하고 재사용하는 것이 중요해집니다. 파이썬에서는 **함수(function)**와 **모듈(module)**, **패키지(package)**를 통해서 코드를 조직화하고 모듈화합니다. 간단히 설명하면, 함수는 코드의 가장 작은 재사용 단위, 모듈은 함수와 클래스를 담는 파일 단위, 패키지는 모듈들을 체계적으로 그룹화하는 폴더 단위입니다.

■ 사용자 정의 함수

함수는 특정 작업을 수행하는 코드 블록으로, 프로그램을 구성하는 기본 단위입니다. 함수를 사용하면 중복되는 코드를 줄이고, 프로그램의 가독성과 유지보수성을 높일 수 있습니다. 파이썬 인터프리터에 내장되어 별도의 `import` 과정 없이 바로 사용할 수 있는 함수를 **내장 함수(built-in function)**라고 부르고, 개발자가 직접 정의하고 작성하는 함수를 **사용자 정의 함수(user-defined function)**라고 부릅니다.

함수는 키워드 `def`를 사용하여 정의합니다. 필요에 따라 **매개변수(parameter)**를 통해 외부로부터 값을 받을 수 있고, `return`을 통해 결과를 반환할 수 있습니다. 함수의 구조는 **머리(header)**와 **몸체(body)**로 구성됩니다. 함수의 머리는 `def` 옆의 함수 이름과 괄호 `()`, 그리고 맨 끝에 붙이는 콜론(`:`)으로 구성됩니다. 매개변수는 괄호 안에 넣어주는데, 매개변수가 없더라도 빈 괄호는 반드시 있어야 합니다. 함수의 몸체는 코드 블록이며, 들여쓰기로 작성해야 합니다. 프로그램 2.24에 사용자 정의 함수의 예가 나와 있습니다.

프로그램 2.24: 사용자 정의 함수의 예

```
# 함수 정의
def greet(name):                                    # 함수 머리
    """주어진 이름을 가진 사람에게 인사하는 함수"""      # 함수 몸체
    return f"안녕하세요, {name}!"

message = greet("홍길동")          # 함수 호출
print(message)                     # 출력: 안녕하세요, 홍길동!
```

함수의 몸체에서 생성된 변수와 함수의 매개변수를 **지역 변수(local variable)**라고 부릅니다. 지역 변수는 해당 함수가 실행되는 동안에만 존재하며, 함수의 실행이 끝나면 메모리에서 사라집니다. 따라서 지역 변수를 함수의 외부에서 참조하면 오류가 발생합니다. 이와 비교하여, 최상위 코드 블록에서 정의된 변수는 **전역 변수(global variable)**입니다. 전역 변수는 함수를 포함하여 프로그램 전체에서 참조할 수 있습니다. 전역 변수와 지역 변수의 이름이 같으면, 지역 변수가 우선적으로 참조됩니다. 함수 내부에서 전역 변수의 값을 읽을 수 있지만, 배정 연산을 통해 전역 변수의 값을 변경하면, 같은 이름의 새로운 지역 변수가 되어 버리니 유의해야 합니다. 함수 내부에서 전역 변수의 값(참조값)을 변경하고 싶으면, `global` 선언을 해 주어야 합니다. 프로그램 2.25에 함수 내부에서 전역 변수를 사용하는 예가 나와 있습니다. 프로그램의 8줄의 `global` 선언으로 변수 y는 값을 변경하더라도 전역 변수로 남아 있습니다. 프로그램의 10줄에는 리스트 `name`에 요소를 추가하는 명령이 나와 있습니다. 리스트 전체가 아니라 요소만 변경되는 경우, `name`이 가진 참조값은 바뀌지 않기 때문에 `global` 선언이 필요하지 않습니다.

프로그램 2.25: 함수 내부에서 전역 변수를 사용하는 프로그램의 예

```
y = 20                      # y는 전역 변수
name = ['kim', 'lee']       # name은 전역 변수

def f(x):                   # 매개변수 x는 지역 변수
   print(x+y)               # 함수 내부에서 전역 변수 읽기

def g():
   global y                 # y를 전역 변수로 선언
   y = 30                   # 전역 변수 y의 값을 변경
   name.append('park')      # 리스트 name에 요소 추가

f(10)                       # 출력: 30
g()
print(y)                    # 출력: 30
print(*name)                # 출력: kim lee park
```

함수는 매개변수를 통해 외부로부터 값을 전달받습니다. 파이썬은 매개변수를 다양한 방식으로 정의하고 사용할 수 있어 함수를 더욱 유연하게 만들어 줍니다.

디폴트 인자(default argument)는 함수를 정의할 때 매개변수에 미리 기본값을 배정해 두는 방법입니다. 함수를 호출할 때 해당 인자에 값을 전달하지 않으면, 미리 설정된 기본값이 사용됩니다. 이 기능은 특정 인자가 자주 동일한 값을 가질 때 유용하며, 함수 호출을 더 간결하게 만들어 줍니다. 디폴트 인자는 항상 일반 인자의 뒤에 위치해야 합니다.

키워드 인자 (keyword argument)는 함수를 호출할 때 '**매개변수명=값**' 형태로 인자를 전달하는 방식입니다. 이 방식을 사용하면 인자의 순서에 상관없이 값을 전달할 수 있어, 함수 호출의 가독성을 높여줍니다. 특히 매개변수가 많은 함수에서 각 인자의 역할을 명확히 알 수 있게 해 줍니다. 키워드 인자는 항상 일반 인자의 뒤에 위치해야 합니다.

가변 인자(variable-length argument)는 함수가 받을 인자의 개수가 정해져 있지 않을 때 사용합니다. 매개변수 앞에 *를 붙이면 함수 호출 시 전달되는 인자들을 튜플로 묶어줍니다.

프로그램 2.26에 다양한 매개변수 전달 방식을 사용하는 프로그램의 예가 나와 있습니다. 프로그램의 11줄에서 `greet()` 함수를 호출할 때, 키워드 인자가 사용됩니다.

프로그램 2.26: 다양한 매개변수 전달 방식을 사용하는 프로그램의 예

```
1  def greet(name, msg='Hello'):          # 디폴트 인자를 가진 함수
2      print(f"안녕 {name}, {msg}")
3
4  def square_sum(*numbers):              # 가변 인자를 가진 함수
5      total = 0
6      for x in numbers:                  # numbers는 튜플
7          total += x*x
8      print(total)
9
10 greet('길동')                          # 출력: 안녕 길동, Hello
11 greet(msg="좋은 아침!", name="길동")    # 출력: 안녕 길동, 좋은 아침!
12 square_sum(1, 2)                       # 출력: 5
13 square_sum(1, 2, 3, 4, 5)              # 출력: 55
```

람다 함수(lambda function)는 `def` 없이 정의하는 한 줄짜리 익명의 함수로서 다음과 같은 형식으로 정의됩니다.

```
lambda 콤마로 나열된 매개변수들 : 식
```

람다 함수는 식의 결과를 반환하는 데, 람다 함수를 배정받은 변수는 보통의 함수 이름처럼 사용됩니다. 보통 sorted(), map(), filter()와 같은 내장 함수에서 임시적인 작은 함수를 즉석에서 정의할 때 사용됩니다. 프로그램 2.27에 람다 함수의 예가 나와 있습니다.

프로그램 2.27: 람다 함수의 예

```
1 is_even = lambda x: x % 2 == 0      # 람다 함수 (짝수 검사)
2 print(is_even(5))                   # 출력: False
3 print(is_even(8))                   # 출력: True
4
5 acc = lambda x, y=1: x + y          # 람다 함수 (디폴트 인수 사용)
6 print(acc(2, 3))                    # 출력: 5
7 print(acc(5))                       # 출력: 6
```

■ 파이썬 내장 함수

파이썬의 **내장 함수(built-in function)**는 print() 함수처럼 import 과정 없이 바로 사용할 수 있는 함수를 말합니다. 내장 함수들은 파이썬 인터프리터에 미리 정의되어 있으며, 개발자들이 프로그램을 작성할 때 자주 사용하는 기본적인 기능들을 제공합니다. 표 2.6에 주요 내장 함수의 종류와 기능이 설명되어 있습니다.

표 2.6: 파이썬의 주요 내장 함수

내장 함수	설명
eval(e)	문자열로 된 식 e의 계산 결과 반환. 예, eval('2+5') ⇒ 7
exec(e)	문자열로 된 문장 e를 수행함. 예, exec("print('hi')") ⇒ hi
abs(x)	숫자 x의 절대값 반환. 예, abs(-5) ⇒ 5
round(x, d)	숫자 x를 반올림하여 소수점 이하 d-자릿수까지 구한 값 반환 예, round(1.372, 1) ⇒ 1.4, round(3.2, 0) ⇒ 3.0
sum(s)	숫자 시퀀스 s의 모든 요소를 더한 값 반환, 예, sum([1, 2, 3]) ⇒ 6
max(s)	컬렉션 s의 최댓값 반환. 예, max([4, 2, 9, 1, 3] ⇒ 9
min(s)	컬렉션 s의 최솟값 반환. 예, min([4, 2, 9, 1, 3] ⇒ 1
chr(x)	유니코드 값 x에 해당하는 문자 반환. 예, chr(65) ⇒ 'A'
ord(c)	문자 c의 유니코드 값 반환. 예, ord('A') ⇒ 65
map(f, s)	순서 데이터 s의 요소 전체에 함수 f를 적용하여 만든 객체 반환
filter(f, s)	순서 데이터 s에서 함수 f의 조건을 만족하는 요소들로 만든 객체 반환
sorted(s)	순서 데이터 s의 요소들을 정렬한 새로운 리스트 객체 반환

■ 모듈

파이썬에서 **모듈(module)**은 함수, 클래스, 변수 등을 담고 있는 .py 확장자를 가진 파일입니다. 모듈을 사용하는 가장 큰 목적은 코드를 기능별로 분리하고, 다른 프로그램에서 재사용할 수 있도록 하는 것입니다. 또한, 함수나 변수의 이름이 겹쳐서 발생하는 충돌(collision)을 방지할 수 있습니다.

파이썬의 표준 라이브러리는 별도의 설치 과정 없이 바로 사용할 수 있는 풍부한 모듈들을 제공합니다. 이러한 모듈을 **표준 모듈** 또는 **내부 모듈**이라고 부릅니다. 모듈에 속한 기능을 사용하려면, import 명령을 사용하여 모듈을 불러와야 합니다. 모듈 내의 모든 기능을 불러오려면, "import **모듈명**" 형식으로 불러옵니다. 사용할 때는 "**모듈명.함수명()**"과 같은 형태로 호출합니다. 프로그램 2.28은 표준 모듈 time, math, sys를 불러와서 사용하는 예를 보여줍니다.

프로그램 2.28: 표준 모듈을 사용하는 프로그램의 예

```
1 import time, math, sys          # 여러 모듈을 한 번에 불러오기
2 print(time.ctime())             # 현재 시간 출력
3 print(math.pi)                  # 원주율 출력
4 sys.exit()                      # 프로그램 종료
```

모듈을 사용할 때 모듈명을 생략하고 싶으면, "from **모듈명** import *" 형식으로 불러오면 됩니다. 이때, '*'는 모듈 내의 모든 기능을 불러오고, 특정한 기능만 불러오고 싶으면, * 대신에 불러오고 싶은 기능의 이름을 넣어주면 됩니다. 모듈을 불러올 때, 모듈이 이름이 길거나 여러 번 사용될 경우 as 키워드를 사용하여 더 짧은 별칭(alias)을 부여할 수 있습니다. 프로그램 2.29는 다양한 방식으로 모듈을 불러오는 프로그램 예를 보여줍니다.

프로그램 2.29: 다양한 방식으로 모듈을 불러오는 프로그램의 예

```
1 from sys import *               # sys 모듈의 모든 기능을 불러옴
2 from math import pi, sqrt       # math 모듈에서 pi와 sqrt() 함수를 불러옴
3 import time as t                # time 모듈의 별칭으로 t를 설정
4 print(pi, sqrt(2))              # 원주율과 2의 제곱근 출력 (모듈명 생략)
5 print(t.ctime())                # 현재 시간 출력 (별칭 사용)
6 exit()                          # 프로그램 종료 (모듈명 생략)
```

__main__은 파이썬 스크립트가 직접 실행될 때 사용되는 특별한 모듈 이름입니다. 스크립트 파일이 맨 처음에 직접 실행되면, 모듈에서 자동으로 만들어지는 **특별 변수(special variable)**인 __name__ 변수에 "__main__"이 자동으로 배정됩니다. 이 특성을 활용하여 if __name__

== "__main__": 구문을 사용하면, 모듈이 직접 실행될 때만 동작하는 코드를 작성할 수 있습니다. 프로그램 2.30에 그 예가 나와 있습니다.

프로그램 2.30: __main__ 모듈인 경우에만 실행되는 코드를 가진 프로그램의 예

```
# my_module.py
def my_function():
    print("모듈의 함수가 실행되었습니다.")

if __name__ == "__main__":
    # 이 부분은 my_module.py가 직접 실행될 때만 동작합니다.
    print("__main__ 모듈로 실행되었습니다.")
    my_function()
```

파이썬의 표준 모듈의 수는 버전이 올라가면서 조금씩 늘어가고 있지만, 버전 3.12에서는 대략 200개 이상이라고 보면 됩니다. 그중에서 자주 사용되는 sys, math, time, random 모듈과 주요 기능을 간략히 살펴보겠습니다.

sys 모듈: 파이썬 프로그램이 실행되는 환경에 관련된 다양한 함수가 있습니다.

- sys.exit() : 프로그램을 강제 종료한다.
- sys.getsizeof(obj) : 객체 obj의 메모리 크기를 바이트 단위로 반환한다.
- sys.getrefcount(obj) : 객체 obj의 참조 개수를 반환한다.

math 모듈: 수학적 연산을 위한 함수와 상수를 제공합니다.

- math.sqrt(x) : 숫자 x의 제곱근($\sqrt{x}$)을 반환한다.
- math.pi : 원주율(π)의 근삿값인 실수를 반환한다.
- math.sin(x), math.cos(x), math.log(x) : 삼각 함수와 로그 함수 등

time 모듈: 시간과 관련된 기능을 제공합니다.

- time.time() : 현재 시각의 초 정보를 부동소수점 형태로 반환한다.
- time.sleep(t) : t초 동안 프로그램의 실행을 일시 중단한다.

random 모듈: 난수(random number)를 생성하는 기능을 제공합니다.

- random.random() : 0 이상 1 미만의 무작위 부동소수점 수를 반환한다.
- random.randint(a, b) : a 이상 b 이하의 무작위 정수를 반환한다.
- random.choice(s) : 순서 데이터 s의 요소 중 하나를 무작위로 뽑아서 반환한다.
- random.sample(s, k) : s의 요소 k개를 무작위로 뽑아서 만든 리스트를 반환한다.

2.6 클래스와 객체

객체 지향 프로그래밍(object-oriented programming)에서는 현실 세계의 사물이나 개념을 코드로 표현하기 위해 **객체(object)** 개념을 도입합니다. 모든 객체는 그 본질을 구성하는 기본 요소인 **속성(attribute)**과 **행위(behavior)**를 가집니다. 속성은 객체가 가지고 있는 데이터 또는 상태(state)를 의미하고, 행위는 객체가 수행하는 행동을 의미합니다.

클래스(class)는 객체를 정의하는 설계도 또는 청사진에 해당합니다. 따라서 객체를 클래스라는 설계도를 바탕으로 실제로 만들어진 실체 또는 **인스턴스(instance)**라고 부르기도 합니다. 클래스에서 객체의 속성은 '변수'로 표현되며, 이 변수를 **인스턴스 속성** 또는 **인스턴스 변수**라고 부릅니다. 객체의 행위는 '함수'로 표현되며, 이 함수를 **메소드(method)** 또는 **인스턴스 메소드**라고 부릅니다. 파이썬에서 클래스는 키워드 `class`를 사용하여 정의합니다. 프로그램 2.31에는 간단한 클래스를 정의하고 사용하는 프로그램의 예가 나와 있습니다.

프로그램 2.31: 클래스 정의와 사용 예

```
1  class Ball:                              # Ball 클래스 정의
2     def __init__(self, pos=(0,0)):       # 메소드(객체 생성자)
3        self.location = list(pos)         # 속성(인스턴스 변수) - 공 위치 좌표
4
5     def move(self, dx, dy):              # 메소드 정의
6        self.location[0] += dx            # 인스턴스 변수 참조
7        self.location[1] += dy            # 공의 위치를 (dx, dy) 만큼 이동
8
9  # Ball 객체(인스턴스) 생성과 사용
10 a = Ball()                    # Ball 객체 생성, a.location -> [0, 0]
11 a.move(10, 5)                 # 메소드 호출, a.location -> [10, 5]
12 a.move(20, 15)                # 메소드 호출, a.location -> [30, 20]
13 print(a.location)             # 출력: [30, 20] (속성 참조)
14 b = Ball((5, 10))             # Ball 객체 생성, b.location -> [5, 10]
15 b.location = [50, 50]         # 속성 값 변경
16 print(b.location)             # 출력: [50, 50]
```

클래스는 `__init__()`이라는 특별한 메소드를 통해 객체의 속성을 초기화합니다. 이 메소드는 객체가 생성될 때 자동으로 호출되는 생성자 함수입니다. `__init__()` 메소드를 정의하여 객체의 속성을 설정하고, 객체마다 고유한 상태를 가질 수 있도록 합니다.

모든 메소드는 첫 번째 매개변수로 self를 가집니다. self는, 파이썬이 메소드 호출 시 자동으로 전달하는, 현재 호출된 객체 자신에 대한 참조입니다. 이 self를 통해 객체의 속성과 메소드에 접근할 수 있습니다. 또한 클래스 내부에서 클래스의 다른 메소드를 호출할 때는 반드시 self.메소드명()의 형식을 사용해야 합니다. self를 생략하면 오류가 발생합니다.

파이썬 클래스의 속성은 **클래스 속성**과 **인스턴스 속성**으로 나뉩니다. **클래스 변수**라고 부르기도 하는 클래스 속성은 클래스의 메소드 밖에서 생성된 변수로서, 모든 인스턴스가 공통으로 공유해야 할 속성을 저장하기 위해 사용됩니다. 클래스 속성은 **클래스명.속성** 또는 **인스턴스명.속성** 형태로 접근할 수 있습니다. 반면에, __init__() 메소드 내에서 self를 통해 생성되는 변수인 인스턴스 속성은 각 인스턴스의 고유한 값을 가집니다. 프로그램 2.32는 클래스 속성을 정의하고 사용하는 프로그램의 예를 보여줍니다.

프로그램 2.32: 클래스 속성 정의와 사용 예

```
1  class Ball:                                  # Ball 클래스 정의
2     owner = "kim"                             # 클래스 속성
3
4     def __init__(self, pos=(0,0)):
5        self.location = list(pos)              # 인스턴스 속성
6
7     def move(self, dx, dy):
8        self.location[0] += dx
9        self.location[1] += dy
10
11 # Ball 객체(인스턴스) 생성과 사용
12 a = Ball()
13 b = Ball((5, 10))
14 print(a.owner, b.owner)       # 출력: kim kim (클래스 속성 참조)
15 Ball.owner = "lee"            # 클래스 속성 변경
16 print(a.owner, b.owner)       # 출력: lee lee (클래스 속성 참조)
```

파이썬에서 **내부 클래스(nested class)**는 하나의 클래스 안에 정의된 또 다른 클래스를 말합니다. 내부 클래스는 두 클래스가 서로 밀접하게 관련되어 있을 때 코드를 구조화하고 논리적인 **캡슐화(encapsulation)**를 달성하기 위해 사용됩니다.

내부 클래스는 **외부 클래스(outer class)**의 **이름 공간(namespace)**에 속합니다. 이 때문에 외부 클래스는 내부 클래스의 모든 기능에 접근할 수 있습니다. 하지만 내부 클래스 자체는 외부 클래스의 인스턴스 변수나 메소드에 접근할 수 없습니다. 즉, 내부 클래스는 외부 클래스의 일부

로 존재하지만, 그 기능은 독립적입니다. 내부 클래스의 인스턴스는 외부 클래스의 인스턴스를 통해 생성되거나, 클래스 이름 자체를 통해 직접 생성될 수도 있습니다. 프로그램 2.33은 내부 클래스를 정의하고 사용하는 프로그램 예를 보여줍니다.

프로그램 2.33: 내부 클래스 정의와 사용 예

```
1  class OutClass:                          # 외부 클래스 정의
2     class InClass:                        # 내부 클래스 정의
3        def __init__(self):                # 내부 클래스 생성자
4           print("내부 클래스 인스턴스 생성")
5
6        def in_method(self):               # 내부 클래스 메소드
7           print("내부 클래스 메소드 호출")
8
9     def __init__(self):                   # 외부 클래스 생성자
10       self.in_obj = self.InClass()       # 내부 클래스 인스턴스 생성
11
12    def out_method(self):                 # 외부 클래스 메소드
13       self.in_obj.in_method()            # 내부 클래스 메소드 호출
14
15 out_obj = OutClass()                     # 외부 클래스 인스턴스 생성
16 out_obj.out_method()                     # 외부 클래스 메소드 호출
17 new_in_obj = OutClass.InClass()         # 직접 내부 클래스 인스턴스 생성
18 new_in_obj.in_method()                  # 내부 클래스 메소드 호출
```

프로그램의 15줄에서 외부 클래스의 인스턴스를 생성하면, 9줄의 `__init__()` 함수는 내부 클래스의 인스턴스를 생성하여 그 참조값을 인스턴스 변수 `in_obj`에 저장합니다. 이 과정에서 "내부 클래스 인스턴스 생성"이 출력됩니다. 그다음, 16줄에서 `out_method()`를 호출하면, 13줄에서 내부 클래스의 `in_method()` 메소드가 호출되고, 결국 7줄에서 "내부 클래스 메소드 호출"이 출력됩니다. 프로그램의 17줄에서는 내부 클래스의 이름을 사용하여 직접 내부 클래스 인스턴스를 생성합니다.

이 장에서는 이 책에서 주로 사용되는 파이썬의 문법을 간결하게 정리하였습니다. 이 장에서 다루지 않은 세부적인 내용과 예외 처리, 파일 입출력, 클래스 상속 등 파이썬의 주요한 기능에 대해서는 인터넷 자료나 파이썬 도서를 참고해 주세요.

연습문제

1. 섭씨 온도를 입력받아, 화씨 온도로 변환하여 출력하는 프로그램을 작성하시오. (참고로 섭씨 온도 C를 화씨 온도 F로 변환하는 공식은 다음과 같다.)

$$F = \frac{9}{5}C + 32$$

2. 두 개의 정수 a와 b를 전달받아, a가 b의 배수이면 True, 그렇지 않으면 False를 반환하는 함수 is_multiple(a, b)을 작성하시오.

3. 한 개의 정수 k를 전달받아, 곱셈, 나눗셈, 나머지 연산을 사용하지 않으면서 k가 짝수이면 True, 그렇지 않으면 False를 반환하는 함수 is_even(k)를 작성하시오.

4. 입력으로 하나 이상의 정수로 구성된 리스트 data를 전달받아, 가장 작은 수와 가장 큰 수를 튜플로 묶어서 반환하는 함수 minmax(data)를 작성하시오. 단, 내장 함수 min()이나 max()를 사용하지 마시오.

5. 양의 정수 n을 전달받아, 0과 n 사이에 있는 모든 홀수의 제곱 합을 반환하는 함수 odd_square_sum(n)을 작성하시오.

6. 리스트 내포(comprehension)와 내장 함수 sum()을 사용하여 문제 5의 합을 계산하는 한 줄의 명령을 작성하시오.

7. 알파벳 소문자 26개를 직접 입력하지 않고, 리스트 내포를 사용하여 리스트 ['a', 'b', 'c', … , 'z']를 만드는 명령을 작성하시오.

8. 다음 숫자들을 생성하는 range() 함수를 작성하시오.

```
8, 6, 4, 2, 0, -2, -4, -6, -8
```

9. 양의 정수 n을 입력받아, 동전을 n번 던져서 앞면이 몇 번 나오는지 실험하는 시뮬레이션 프로그램을 작성하시오. 단, 프로그램을 실행시킬 때마다 출력되는 내용이 달라져야 한다. (힌트. random 모듈을 이용하라.)

10. random 모듈의 choice(s) 함수는 순서 데이터 s에서 무작위로 요소 하나를 뽑아서 반환한다. randint() 함수를 사용하여 choice() 함수와 똑같이 수행하는 명령을 작성하시오.

11. random 모듈의 shuffle(data) 함수는 리스트 data의 요소들 순서를 무작위로 섞어 준다. randint() 함수를 사용하여 shuffle() 함수와 똑같이 수행하는 명령을 작성하시오.

12. 컴퓨터가 뽑은 1과 100 사이의 정수를 사용자가 7번 만에 맞추는 게임 프로그램을 작성하시오. 사용자가 숫자를 입력하면 컴퓨터는 자신이 가진 숫자보다 큰지 작은지를 알려주고, 7번 만에 맞추지 못하면 실패했다는 내용을 출력한다.

13. 양의 정수 2보다 큰 값을 입력받아 이 숫자를 2로 반복해서 나누어 2보다 작은 값이 나올 때까지 몇 번 나누어야 하는지를 출력하는 프로그램을 작성하시오.

14. 입력으로 주어진 문자열 안에서 가장 긴 단어를 출력하는 프로그램을 작성하시오. 한 개의 단어는 공백이 아닌 문자들이 연속된 것을 말합니다. 가장 긴 단어가 여러 개 있는 경우는 가장 먼저 나온 단어를 출력한다. 예를 들어, 입력으로 문자열 "My favorite language is Python."이 들어오면, language를 출력한다.

15. 두 점의 좌표 (x, y)와 (a, b)를 나타내는 4개의 정수 x, y, a, b를 입력받아, 두 점 사이의 거리를 소수점 둘째 자리까지 출력하는 프로그램을 작성하시오. (참고로 두 점 사이의 거리를 구하는 공식은 다음과 같다.)

$$\sqrt{(x-a)^2+(y-b)^2}$$

16. 이름(name), 꽃잎 수(petals), 가격(price) 세 가지 속성을 가지는 클래스 Flower를 정의하시오. 단, name은 문자열, petals는 정수, price는 실수이며, 생성자에서 각 속성 값을 초기화하고, 속성 값을 설정하고 읽어오는 메소드를 포함해야 한다.

17. c, a, t, d, o, g 문자를 각각 한 번씩만 사용하여 만들 수 있는 모든 가능한 문자열을 출력하는 파이썬 프로그램을 작성하시오. (참고로 itertools 모듈의 permutations() 함수를 이용하면 모든 순열을 구할 수 있다.)

제3장

선형리스트와 배열

3.1 자료구조와 추상자료형
3.2 선형리스트
3.3 배열 구조
3.4 파이썬의 순서 자료형
3.5 사례 연구: 리스트 정렬
연습문제

학습 목표

- 추상자료형(ADT) 개념을 이해하고, 추상자료형의 도입하는 목적을 설명할 수 있다.
- 선형리스트의 개념을 이해하고, 선형리스트에 필요한 기본 연산을 정의할 수 있다.
- 배열 구조를 이해하고, 정적 배열과 동적 배열의 원리와 차이점을 설명할 수 있다.
- 파이썬 리스트의 구조를 이해하고, 주요 메소드의 시간복잡도를 설명할 수 있다.
- 기본 정렬 알고리즘의 동작 원리를 설명하고, 시간복잡도를 분석할 수 있다.
- 다양한 정렬 작업에서 파이썬의 sort() 메소드와 sorted() 함수를 활용할 수 있다.

3.1 자료구조와 추상자료형

프로그램 수행 과정에서 많은 부분이 데이터를 저장하고 검색하는 작업으로 이루어집니다. 따라서 프로그램의 수행 시간과 메모리 사용량은 데이터를 어떤 방식으로 저장하고 어떻게 활용하느냐에 따라 결정됩니다. 이때 **데이터의 표현(representation)과 연산(operation)**이라는 용어가 자주 사용됩니다. 데이터를 메모리에 저장하는 방식을 데이터 표현이라고 하고, 데이터를 조작하고 처리하는 절차를 그 데이터에 대한 연산이라고 합니다.

자료구조(data structures)는 데이터의 표현과 연산에 관한 내용을 다루며, 알고리즘과는 불가분의 관계에 있습니다. 저수준(low-level) 관점, 즉 매우 구체적이고 실제적인 관점에서 데이터 표현은 특정 데이터 값을 기억장소에 저장하는 방식을 의미합니다. 따라서 컴퓨터에 저장된 실수나 정수, 문자열 등이 여기에 해당합니다. 고수준(high-level) 관점에서 데이터 표현이란 "데이터 원소들 사이의 연관 관계", "데이터를 조직하고 구조화하는 방법" 등을 의미합니다. 데이터 원소들을 정렬 상태로 유지하거나 데이터 사이에 대칭 관계나 종속 관계 등을 설정하는 것이 여기에 해당합니다. 일반적으로 자료구조는 데이터의 '표현'과 그 데이터를 다루는 '연산'을 모두 포함하는 넓은 의미로 사용됩니다.

알고리즘은 자료구조를 정의하고 사용하는 절차라고 말할 수 있습니다. 데이터의 표현 방식에 따라 연산의 구현 내용이 달라지며, 그 결과로 알고리즘의 효율성도 달라질 수 있습니다. 결국 효율적인 알고리즘을 개발하기 위해서는 그에 따른 적절한 자료구조의 선택이 필수적입니다.

프로그래밍의 관점에서 볼 때, 데이터 표현을 프로그램에 그대로 이용하는 것은 유용한 면도 있지만 위험할 수도 있습니다. 추후 데이터의 표현 방식을 프로그램에서 수정해야 할 상황이 발생할 경우, 해당 표현을 사용하는 모든 코드를 찾아 변경해야 하기 때문입니다. 이는 번거로운 작업이기도 하지만 미처 변경하지 못한 코드로 인해 오류가 발생할 수도 있습니다. 이러한 문제점을 해결하면서 자료구조를 효과적으로 정의하고 사용하기 위해, 자료구조를 **명세(specification)**와 **구현(implementation)**으로 구분하는 것이 효과적입니다. 자료구조에서 다룰 데이터와 그 데이터를 처리하는 연산을 기능 면에서 정의하는 것을 '명세'라고 하고, 데이터의 표현 방식과 연산의 구현 방식을 구체적으로 결정하는 것을 '구현'이라고 말합니다.

자료구조의 명세에 해당하는 내용을 **추상자료형(abstract data type; ADT)**이라고 부릅니다. 예를 들어, 스택(stack) 자료구조의 추상자료형은 스택에서 다룰 데이터의 유형과 이 데이터에 접근하는 연산 집합을 정의합니다. 스택을 구현하기 위해 배열이나 연결리스트와 같은 프로그래밍 도구가 사용되지만, 이러한 구체적인 구현 사항은 스택 사용자에게 감춰집니다. 추상자료형

의 핵심은 사용자가 자료구조의 구체적인 구현 방식에 얽매이지 않고, 제공된 인터페이스만을 사용하여 더 높은 수준의 문제 해결에 집중하도록 돕는 것입니다.

객체지향 프로그래밍 언어(object oriented programming language)에서 제공하는 클래스(class)는 자료구조를 명세와 구현으로 분리해서 다룰 수 있게 해 줍니다. 클래스는 데이터와 이를 다루는 연산인 메소드(method)를 하나의 단위인 객체(object)로 묶어주는데, 이때 **캡슐화(data encapsulation)**와 **정보 은닉(information hiding)** 원리가 적용됩니다. 즉, 메소드의 구체적인 동작 방식은 클래스 내부에 감춰두고, 외부에는 메소드 이름과 기능 명세만을 공개하는 것입니다. 따라서 사용자는 클래스의 내부 구조를 전혀 몰라도, 공개된 명세를 통해 생성된 객체를 손쉽게 사용할 수 있습니다. 그러므로, 클래스는 자료구조의 명세와 구현이라는 개념을 프로그래밍 언어 안에서 구현한 도구라고 말할 수 있습니다.

프로그래밍 언어에 내장된 자료형은 추상자료형 개념으로 설계된 것으로 볼 수 있습니다. 특히 파이썬은 모든 자료형이 객체입니다. 여기서 클래스의 인스턴스인 객체는 데이터와 그 데이터를 처리하는 메소드를 포함하는 독립적인 단위입니다. 파이썬 프로그램을 작성할 때, 우리는 자료형이 어떻게 구현되어 있는지 알지 못하거나 알아야 할 필요성을 느끼지 못합니다. 이는 자료형이 본질적으로 추상자료형 개념으로 설계되었다는 것을 의미합니다.

넓게 보면 모든 자료형이 자료구조에 해당하지만, 일반적으로는 복수 개의 원소로 구성된 데이터를 다루는 자료형을 자료구조라고 부릅니다. 그래서 정수(`int`)와 실수(`float`)처럼 단일 값만 다루는 자료형은 자료구조라고 지칭하기보다 데이터 원소 또는 요소로 취급합니다. 이 책에서도 자료구조를 이러한 개념으로 사용합니다.

객체지향 프로그래밍 언어는 다양한 자료구조를 제공합니다. 이들 자료구조는 프로그래밍 언어에 내장된 형태(built-in type)로 제공되기도 하고, 모듈(module) 또는 라이브러리(library)의 형태로 제공되기도 합니다. 파이썬의 경우, 리스트(`list`), 튜플(`tuple`), 문자열(`str`), 집합(`set`), 사전(`dict`)이 내장된 자료구조 자료형입니다. 이 외에도 `collections` 모듈을 통해 `deque`, `OrderedDict`, `Counter` 등의 다양한 자료구조와 `heapq` 모듈, `bisect` 모듈 등 유용한 자료구조를 제공합니다.

자료구조는 선형 자료구조와 비선형 자료구조로 구분됩니다. 선형 자료구조는 데이터 원소들이 일렬로 나열된 것과 같은 구조를 의미합니다. 따라서 데이터에 대한 순차적인 접근이 가능하여 이전 원소(previous element)와 다음 원소(next element) 개념을 사용할 수 있습니다. 여기에는 선형리스트, 스택, 큐, 데크 등이 포함됩니다. 비선형 자료구조는 데이터 원소들이 비선형 방식으로 구조화된 형태를 의미합니다. 트리, 그래프, 힙, 이진 탐색 트리, 해싱 등이 비선형 자

료구조에 해당합니다. 파이썬의 자료형 중에서 순서 자료형(sequence type)에 속하는 리스트(list), 튜플(tuple), 문자열(str)은 선형 자료구조에 해당하고, 집합(set)과 사전(dict)은 비선형 자료구조에 해당합니다.

자료구조의 추상적 모델, 즉 추상자료형을 프로그램으로 구현하려면 궁극적으로 배열 또는 연결리스트가 사용되어야 합니다. 필요시에는 배열과 연결리스트를 결합한 하이브리드 구조를 사용할 수도 있습니다. 배열과 연결리스트는 그 자체로도 저수준(low-level)의 자료구조에 해당하지만, 추상자료형을 구현하는 프로그래밍 도구로서 중요한 역할을 담당합니다. 배열 기반 구현과 연결리스트 기반 구현은 효율성 면에서 큰 차이점이 있으므로, 이 두 기본 구조의 장단점을 이해하는 것은 자료구조 학습에서 핵심적인 부분입니다.

3.2 선형리스트

선형리스트(linear list)는 선형 구조의 가장 일반적인 형태로서, 수학의 집합 개념과는 달리, 원소들 사이에 순서가 유지됩니다. 그래서 선형리스트를 **순서리스트(ordered list)**라고 부르기도 합니다. 다음은 선형리스트의 예입니다.

- 알파벳 대문자 리스트 = (A, B, C, D, E, … , Z)
- 웹 브라우저 방문 기록
- 음원 서비스 실시간 음원 차트 TOP100

"알파벳 대문자 리스트"는 순서가 정해진 26개의 원소로 구성되며, 내용이 변하지 않는 정적인 리스트입니다. "웹 브라우저 방문 기록"은 사용자가 새로운 페이지를 방문할 때마다 기록이 추가되고, 특정 기록을 삭제할 때 목록에서 제거됩니다. 방문 기록의 순서는 일반적으로 방문 시각에 의해 결정됩니다. "음원 서비스 실시간 음원 차트 TOP100"은 음원 인기도를 실시간으로 집계하여 순위를 매기는 차트로, 순서가 빈번하게 변경되는 동적인 리스트입니다.

선형리스트를 일반적으로 표현하면 다음과 같습니다.

$$L = (a_0, a_1, a_2, \cdots, a_{n-1})$$

여기서 L은 선형리스트의 **이름**이고, 원소 a_i를 L의 **i번째 원소**, i를 a_i의 **순위(ranking)** 또는 **위치(position)**라고 부릅니다. 선형리스트의 **길이(length)**는 리스트에 포함된 원소의 개수입니다.

길이가 0인 리스트를 **공백 리스트(empty list)**라고 부르고 L = () 또는 L = ∅으로 표현합니다. 위 선형리스트 예에서 "알파벳 대문자 리스트"와 "음원 서비스 실시간 음원 차트 TOP100"의 길이는 각각 26과 100이고, "웹 브라우저의 방문 기록"은 길이가 가변적입니다.

이제, 선형리스트에 대해 수행될 수 있는 연산들을 살펴보겠습니다. "알파벳 대문자 리스트"에서 "10번째 문자는 무엇인가", "문자 'J'는 몇 번째 문자인가?"와 같은 질문이 나올 수 있습니다. 이러한 질문은 "실시간 음원 차트 TOP100"에서는 "순위가 10위인 노래는 무엇인가", "비비의 '밤양갱'의 순위는 얼마인가?"와 같은 종류의 질문으로 볼 수 있습니다. 선형리스트에서 자주 발생하는 질문들을 일반화된 명령으로 나열하면 다음과 같습니다.

- 리스트 L의 길이를 구하라.
- 리스트 L에 있는 원소들을 차례대로 방문하라.
- 리스트 L에서 특정 위치에 있는 원소를 검색하라.
- 리스트 L에서 주어진 원소의 순위를 구하라.
- 리스트 L의 특정 위치에 새로운 원소를 삽입하라.
- 리스트 L의 특정 위치에 있는 원소를 삭제하라.

위 명령들은 리스트 연산으로 표현될 수 있습니다. 마지막 두 연산의 경우, 수행 결과로 원소의 순위가 자동으로 변경됩니다. 리스트의 i번째 위치에 새로운 원소가 삽입되면, 이전의 i번째, i+1번째, …, n-1번째 원소는 각각 i+1번째, i+2번째, … , n번째 원소로 자동으로 바뀝니다. 마찬가지로, 리스트 L의 i번째 위치에 있는 원소가 삭제되면, 이전의 i+1번째, i+2번째, …, n-1번째 원소는 각각 i번째, i+1번째, … , n-2번째 원소로 자동으로 바뀝니다. 따라서 선형리스트 연산을 구현할 때 원소의 순위가 변경될 수 있음을 항상 유의해야 합니다.

선형리스트에 대한 연산으로, 위에 나열된 것 이외에도 리스트의 생성, 소멸, 분할, 정렬, 두 리스트 간의 동일성 검사, 병합 등도 포함될 수 있습니다.

프로그램에서 선형리스트를 구현할 때 배열을 이용하거나 연결리스트를 이용할 수 있습니다. 배열은 **첨자(index)**를 이용하여 메모리 공간을 참조하는 구조이므로, 첨자는 선형리스트에서의 순위와 자연스럽게 연관됩니다. 그래서 선형리스트를 배열과 동일한 개념으로 보는 경향도 많습니다. 그러나 배열 기반으로 구현된 선형리스트와 연결리스트 기반으로 구현된 선형리스트는 시간복잡도 면에서 차이가 있습니다. 예를 들어 특정 위치에 있는 원소를 검색하라는 명령은 배열 표현에서는 상수 시간에 수행되지만, 연결리스트 표현에서는 선형 시간이 소요됩니다. 반면에 삽입과 삭제 연산은 배열 표현에서는 원소들의 이동 작업 때문에 선형 시간이 소요되고, 연결리스트 표현에서는 링크의 변경만으로 해결되므로 상수 시간에 수행됩니다. 연결리스트에 관

한 상세한 내용은 7장에서 다룹니다.

리스트의 내용이 바뀌지 않고 주로 검색만 수행되는 정적 리스트는 배열 기반으로 구현하는 것이 효과적이며, 삽입과 삭제가 빈번한 동적 리스트는 연결리스트 기반으로 구현하는 것이 효과적입니다. 이처럼 구현 방식에 따라 시간복잡도가 달라지기 때문에, 추상자료형을 설계하는 단계부터 표현 방식을 고려해야 합니다. 프로그래밍 언어에서 제공하는 자료형 또는 라이브러리 클래스의 경우, C++은 STL(Standard Template Library)을 통해 배열 기반 클래스 `std::vector`와 연결리스트 기반 클래스 `std::list` 등을 제공하고, Java는 컬렉션 프레임워크(Collections Framework)를 통해 배열 기반의 `ArrayList`와 연결리스트 기반의 `LinkedList` 등을 제공합니다. 파이썬에서 제공하는 내장 자료형인 리스트, 튜플, 문자열은 배열 기반으로 구현되어 있습니다. 파이썬에는 순수한 형태의 연결리스트 자료형이 내장되어 있지 않지만, 대신 리스트의 양쪽 끝에서 원소를 빠르게 추가하고 삭제할 수 있는 기능을 가진 `collections.deque`가 표준 라이브러리로 제공됩니다.

3.3 배열 구조

파이썬은 기본적으로 배열 자료형을 직접 제공하지 않습니다. 파이썬 리스트는 배열 그 자체라기보다는 배열을 기반으로 하여 유연하고 사용하기 쉽게 만들어진 자료형입니다. 따라서 먼저 일반적으로 통용되는 배열의 개념을 소개하고, 이후에 파이썬 리스트와의 차이점을 살펴보겠습니다.

데이터의 저장과 검색 작업은 메모리(memory)라고 불리는 기억 공간에서 수행됩니다. 메모리는 컴퓨터 시스템의 주기억장치(main memory)를 지칭하며, RAM(random access memory)이라고 부르기도 합니다. 메모리에는 8비트, 즉 1바이트 단위의 기억장소마다 주소(address) 역할을 담당하는 고유 번호가 지정됩니다. 이 고유 번호를 **메모리 주소** 또는 **참조값(reference value)**이라고 부릅니다. 정수나 실수처럼 하나의 값을 저장하는 데 여러 바이트가 필요한 경우, 필요한 바이트 수만큼 묶어 하나의 기억장소 단위로 다루어집니다. 이러한 기억장소의 참조값은 첫 번째 바이트의 참조값으로 대표됩니다. 예를 들어, 그림 3.1에는 메모리의 일부분에 할당된 변수들이 표시되어 있습니다. 하단에 표시된 숫자가 각 바이트에 지정된 참조값이라고 가정합시다. 변수 `score`, `ch`, `pi`가 각각 4바이트, 2바이트, 8바이트의 크기의 기억장소에 할당된 경우, 이 세 변수의 참조값은 각각 첫 바이트의 참조값인 4080, 4084, 4086이 됩니다.

RAM이라는 용어에서 짐작할 수 있듯이, 참조값만 알면 시스템은 해당 기억장소에 곧바로 접근할 수 있습니다. 이는 차수 표기법으로 $O(1)$ 시간에 기억장소에 접근해서 값을 저장하거나 검색을 수행할 수 있다는 의미입니다.

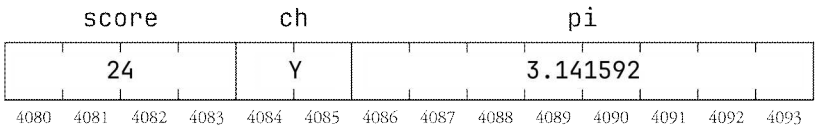


그림 3.1: 변수의 참조값

배열은 동일한 자료형으로 구성된 기억장소들의 집합체입니다. 배열을 위한 기억 공간은 논리적으로 메모리에서 연속적인 위치[6]에 할당되며, 첨자(index)를 이용하여 각 기억장소를 참조할 수 있습니다. **배열의 길이(length)**는 메모리에 할당된 기억장소의 개수입니다. 일반적으로 배열은 지정된 길이를 가지고 생성되며, 일단 생성된 후에는 길이를 변경할 수 없습니다. 이처럼 길이가 고정된 배열을 **정적 배열(static array)**이라고 부릅니다.

그림 3.2는 문자열 "선형리스트"가 배열에 저장된 모습을 보여줍니다. 각 문자가 유니코드(Unicode)로 표현되어 있다면, 2바이트 크기의 기억장소에 각 문자가 저장됩니다. 배열의 길이가 5이므로, 첨자 범위는 0부터 4까지이며 총 10바이트 크기의 연속된 메모리 공간에 문자열 데이터가 저장됩니다.

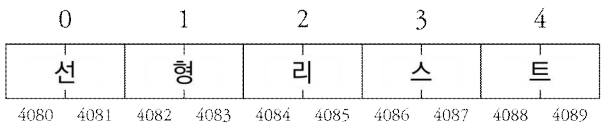


그림 3.2: 문자열 데이터가 저장된 배열의 예

배열의 가장 큰 특징은 첨자를 알면 해당 기억장소의 주소인 참조값을 간단한 연산으로 계산할 수 있다는 점입니다. 배열의 첫 번째 기억장소, 즉 첨자가 0인 기억장소의 참조값을 `s`라고 할 때, 첨자가 `i`인 기억장소의 참조값은 다음과 같이 계산됩니다. 식에서 `d`는 기억장소의 크기를 바이트 단위로 나타낸 값입니다.

첨자가 `i`인 기억장소의 참조값 = `s + i × d`

6) 물리적으로는 그렇지 않더라도 운영체제가 그러한 가상 메모리 환경을 만들어 줍니다.

그림 3.2의 배열에서 첫 번째 기억장소의 참조값이 4080이므로, 첨자가 i인 기억장소의 참조값은 다음과 같이 계산됩니다.

```
4080 + i × 2
```

만약 다음 예처럼 크기가 제각각인 원소들로 구성된 선형리스트는 배열 구조로 어떻게 표현할 수 있을까요?

```
("apple", "pear", "banana", "watermelon", "orange")
```

각 원소에 접근하는 연산을 상수 시간에 수행하기 위해서는 배열 구조가 필수적입니다. 배열은 동일한 크기의 기억장소들로 구성되어야 하는데, 원소의 크기가 서로 다르다면 배열의 기억장소에 직접 저장할 수 없습니다. 이런 경우에는 **참조 배열(reference array)**이라고 불리는 방식으로 표현합니다. 참조 배열의 각 기억장소에는 참조값이 저장되고, 이 참조값이 각 원소가 실제 저장된 기억장소를 가리키게 됩니다. 그림 3.3에 참조 배열의 예가 나와 있습니다.

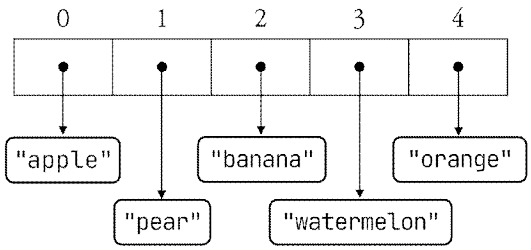


그림 3.3: 참조 배열의 예

앞서 정적 배열은 길이가 고정된 배열이라고 했습니다. 이와 비교하여 **동적 배열(dynamic array)**은 길이가 가변적인 배열입니다. 파이썬의 리스트를 포함하여 프로그래밍 언어에서 제공하는 배열 기반의 리스트는 길이가 자동으로 조정되는 동적 배열을 지원합니다. 동적 배열의 구현은 간단합니다. 현재 사용 중인 배열의 기억 용량이 부족해지면, ① 더 큰 용량을 가진 새로운 배열을 만들고, ② 기존 배열의 내용을 복사한 다음, ③ 새로운 배열을 이용하는 것입니다. 반대로 배열의 기억 용량이 과도하게 크면, 더 작은 용량을 가진 새로운 배열을 만들어 위와 같은 작업을 한 후에 이용하는 것입니다.

동적 배열에서 고려해야 할 주요 사항 중 하나는 파이썬 리스트의 `append()` 메소드처럼 리스트의 끝에 새로운 요소를 추가하는 연산의 수행 시간입니다. 배열 공간이 충분하면 이 연산은 상수 시간에 수행됩니다. 만일 총 n개의 요소를 추가하는 작업이 수행된다고 할 때, 처음부터

배열 공간이 충분하면 총 $O(n)$ 시간이 소요됩니다. 그런데, 처음에는 배열 공간을 작게 확보했다가 배열 공간이 부족해질 때마다 새로운 배열로 교체하는 작업을 수행한다면, 총 수행 시간은 어떻게 될까요? 새로운 배열로 교체할 때마다 기존 내용을 복사해야 하는데, 이 작업에는 선형 시간이 소요됩니다. 만일 새로운 배열을 매번 상수 크기인 c만큼 확충한다고 가정해 봅시다. 즉 배열이 교체될 때마다 그 길이가 c, 2c, 3c, … , mc로 변경된다고 합시다($m = \lceil n/c \rceil$). 총 n번의 추가 작업이 수행되는 동안, 추가 연산과 복사 작업이 반복되면서 총 $c + 2c + \cdots + mc$번의 연산이 수행됩니다. 연산의 총 횟수는, 다음의 식에 의해 $\Omega(n^2)$이 됩니다. 따라서 평균적으로 보면 연산 하나당 $\Omega(n)$ 시간이 소요되는 셈입니다.

$$c + 2c + \cdots + mc = c(1 + 2 + \cdots + m) = c\frac{m(m+1)}{2} \geq \frac{n^2}{2c} = \Omega(n^2)$$

이러한 비효율성은 배열의 크기를 항상 2배씩 확충시킨다면 개선될 수 있습니다. 이 경우는 배열의 길이가 1, 2, 4, 8, … , 2^k로 변경됩니다($n \leq 2^k < 2n$). 총 n번의 추가 작업이 수행되는 동안, 추가 연산 자체는 n번 수행되고, 중간의 복사 작업에 필요한 연산의 수행 횟수는 다음과 같습니다.

$$1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1 < 2n$$

따라서 작업에 필요한 연산의 전체 수행 횟수는 3n보다 작습니다. 추가 연산의 수행 횟수가 n이므로, 평균적으로 보면 추가 연산 하나당 상수 시간이 소요됩니다.

상각 시간복잡도(amortized time complexity)는 모든 작업이 완료된 후, 각 작업 당 소요된 평균 시간을 의미합니다. 상각 시간복잡도는 평균 시간복잡도와 비슷해 보이지만 개념이 다릅니다. 평균 시간복잡도는 작업의 횟수와 상관없이 한 작업 당 소요되는 평균 수행 시간을 말하지만, 상각 시간복잡도는 모든 작업이 수행된 후에 분석되는 평균 수행 시간입니다. 일반적으로 동일한 작업을 매우 빈번하게 수행해야 하는데, 대부분은 빠르게 동작하다가 가끔씩 느리게 동작하는 경우 상각 시간복잡도를 분석하게 됩니다. 따라서 동적 배열을 유지하기 위해 배열의 용량을 2배씩 증가시키는 방식의 상각 시간복잡도는 상수 시간입니다.

배열의 용량을 축소해야 하는 상황에서는, 요소의 개수가 현재 배열 용량의 1/4 미만으로 떨어질 때마다 배열 용량을 절반으로 줄이는 방식을 사용하면 상수 시간의 상각 시간복잡도를 가지면서 배열의 용량을 항상 요소 수의 4배 이하가 되도록 유지할 수 있습니다. 파이썬과 같은 프로그래밍 언어에서는 더욱 정교하게 설계된 방식을 통해 상수 시간의 상각 시간복잡도를 유지하면서 더 효과적인 동적 배열을 구현합니다.

3.4 파이썬의 순서 자료형

파이썬의 순서 자료형은 순서가 있는 요소들의 그룹을 위한 자료형으로 배열 기반으로 구현되어 있습니다. 리스트, 튜플, 문자열이 순서 자료형에 해당합니다. 정수와 실수를 포함하여 모든 자료형이 파이썬에서는 객체이기 때문에, 순서 자료형은 참조 배열 기반으로 구현되어 있습니다. 예를 들어, 정수 리스트인 [5, 22, 9, 10, 8, 4, 15]도 그림 3.4와 같이 참조 배열 구조로 구성됩니다.

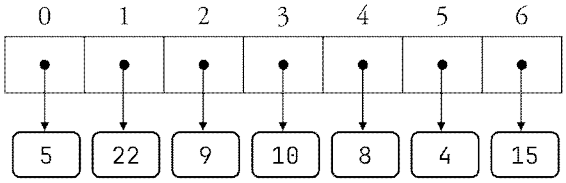


그림 3.4: 파이썬 리스트의 참조 배열 구조

리스트와 튜플은 다양한 자료형 요소를 가진 선형리스트를 표현한다는 점에서 유사합니다. 하지만 리스트는 요소 변경, 요소 삭제, 새로운 요소 삽입이 가능한 가변(mutable) 데이터인 반면에 튜플은 데이터를 변경할 수 없는 불변(immutable) 데이터라는 차이가 있습니다. 문자열도 튜플과 마찬가지로 불변(immutable) 데이터입니다. 따라서 한 번 생성된 후에는 그 길이가 바뀌지 않는 튜플과 문자열과는 달리, 길이가 가변적인 리스트는 동적 배열 기반으로 구현되어 있습니다. 리스트 연산 중에서 데이터를 변경하지 않는 연산은 튜플과 문자열에도 동일하게 수행되고, 데이터를 변경하는 연산은 튜플과 문자열에는 허용되지 않습니다.

데이터를 변경하지 않는 순서 자료형 연산의 종류와 차수 표기법으로 표시된 수행 시간이 표 3.1에 나와 있습니다. 표에서 L과 M은 순서 자료형 객체이고, n과 m은 각각의 길이, 그리고 item은 요소 데이터를 나타냅니다. 순서 자료형 객체의 길이 정보는 별도로 관리되므로 상수 시간에 수행됩니다. 인덱싱 연산은 임의 접근이 가능한 배열의 특성 덕분에 상수 시간에 수행됩니다. count(), index() 메소드와 in 연산자는 배열 공간을 왼쪽에서 오른쪽으로 순차적으로 탐색하여 결과를 도출합니다. 연산 "L.count(item)"은 L에 들어 있는 item의 개수를 파악하기 위해 모든 요소를 검사해야 하므로 $O(n)$ 시간이 소요됩니다. 두 연산 "L.index(item)"과 "item in L"은 item이 저장된 최초의 첨자 위치까지 탐색을 수행합니다. 최악의 경우는 모든 요소를 검사해야 합니다. 두 순서 자료형 객체 간의 비교 연산은 사전식 순서

(lexicographic order)로 수행되며, 비교 연산의 결과가 결정되는 최초의 위치까지 순차적으로 요소들을 비교하므로, 최악의 경우 길이가 짧은 쪽 객체의 모든 요소를 검사해야 합니다. 새로운 객체를 생성하는 슬라이싱 연산과 + 및 * 연산자는 새로 생성되는 객체에 포함되는 요소의 개수만큼 기본 연산이 수행됩니다.

표 3.1: 순서 자료형의 공통 연산과 수행 시간

연산	시간복잡도
`len(L)`	$O(1)$
`L[j]`	$O(1)$
`L.count(item)`	$O(n)$
`L.index(item)`	$O(k+1)$ (`L[k] = item` 또는 `k = n`)
`item in L`	$O(k+1)$ (`L[k] = item` 또는 `k = n`)
`L == M` (`!=`, `<`, `<=`, `>`, `>=`)	$O(k+1)$ (k는 L과 M에서 일치하지 않는 가장 왼쪽 첨자 또는 `k = min(n, m)`)
`L[j:k]`	$O(k-j+1)$
`L + M`	$O(n+m)$
`L * c`	$O(cn)$

표 3.2에는 리스트의 내용을 변경하는 연산과 그 시간복잡도가 나와 있습니다. 표에서 L과 M은 리스트 객체이고, n과 m은 각각의 길이, 그리고 item은 요소 데이터를 나타냅니다. 연산 "`L[j] = item`"은 리스트 L의 j번째 요소를 item으로 대체합니다. 이 연산은 다른 요소에는 아무런 영향을 주지 않기 때문에 상수 시간에 수행됩니다. `append()` 메소드는 새로운 배열로 교체하는 작업이 필요할 수 있으므로 최악의 경우 시간복잡도는 선형 시간이지만, 3.3절에서 설명한 것처럼 상각 시간복잡도는 상수 시간입니다. 이와 마찬가지로 요소의 삽입과 삭제에 관련된 모든 연산은 상각 시간복잡도로 분석됩니다. 전체 프로그램의 시간복잡도를 분석할 경우, 상각 시간복잡도를 최악의 경우 시간복잡도로 간주하고 분석하면 됩니다.

표 3.2: 리스트의 내용을 변경하는 연산과 수행 시간

연산	시간복잡도 (* 상각 시간복잡도)
`L[j] = item`	$O(1)$
`L.append(item)`	$O(1)$*
`L.insert(k, item)`	$O(n-k+1)$*
`L.pop()`	$O(1)$*

L.pop(k) del L[k]	O(n-k)*
L.remove(item)	O(n)*
L.extend(M) L += M	O(m)*
L.reverse()	O(n)
L.sort()	O(nlogn)

연산 "L.insert(k, item)"은 첨자 k인 위치에 item을 저장해야 하므로, k를 포함한 그 이후의 요소들을 한 칸씩 뒤로 이동시켜야 합니다(그림 3.5 참조). 그래서 수행 시간은 O(n-k+1)입니다. k가 0이면 최악의 경우가 발생하며, 이때의 시간복잡도는 O(n)입니다.

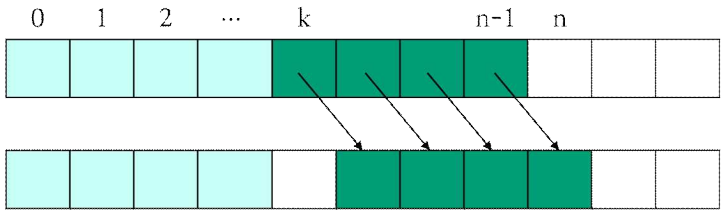


그림 3.5: insert(k, item)의 수행에서 요소들의 이동 작업

연산 "L.pop()"은 리스트 L의 마지막 요소를 제거합니다. 나머지 요소들은 원래 위치에 그대로 남아 있으므로 상수 시간에 수행됩니다. 연산 "L.pop(k)"은 첨자 k 위치의 요소를 삭제하기 위해 k 이후의 요소들을 한 칸씩 앞으로 이동시킵니다. 그래서 O(n-k) 시간이 소요됩니다. k가 0이면 최악의 경우가 발생하며, 이때의 시간복잡도는 O(n)입니다.

연산 "L.remove(item)"은 리스트 L에서 item이 처음 나타나는 위치를 찾아 해당 요소를 제거합니다. 그다음 삭제된 위치 이후의 요소들을 한 칸씩 앞으로 이동시킵니다. 그래서 시간복잡도가 O(n)입니다.

두 연산 "L.extend(M)"과 "L += M"은 리스트 M의 각 요소를 하나씩 리스트 L의 끝에 추가시키는 작업이므로, 상각 시간복잡도로 O(m) 시간이 소요됩니다. 이 두 연산을 사용하면 시스템은 최종 배열의 크기를 미리 알 수 있어 배열 교체 작업을 한 번만 하면 된다는 장점이 있습니다. 그래서 append() 메소드를 여러 번 호출하는 것에 비해 시간복잡도의 차수는 바뀌지 않지만, 숨겨진 상수 인자가 작아져 조금 더 효율적으로 수행됩니다.

리스트의 요소를 역순으로 배치하는 reverse() 메소드는 모든 요소를 한 번씩 참조해야 하므로 선형 시간이 소요됩니다. 최적화된 정렬 알고리즘으로 구현된 sort() 메소드는 $O(n\log n)$ 시간에 수행됩니다. 정렬 알고리즘의 종류와 동작 방식에 대해서는 추후 자세하게 설명합니다. 간단한 정렬 알고리즘은 3.5절에서 소개하고, 복잡하지만 효율적인 정렬 알고리즘은 8장에서 소개합니다.

행렬(matrix)은 현대 자연과학, 공학 및 사회과학의 여러 분야에서 활용되는 필수적인 수학적 도구입니다. 2차원 행렬은 숫자(원소)들이 직사각형 모양으로 배치되어 **행(row)**과 **열(column)**로 구분됩니다. 그림 3.6에 4×5 크기를 가진, 즉 4행 5열로 구성된 2차원 행렬의 예가 나와 있습니다.

	0	1	2	3	4
0	6	12	5	19	7
1	4	20	9	11	5
2	8	18	21	35	1
3	2	22	16	13	3

그림 3.6: 4×5 크기를 가진 2차원 행렬 M

행렬의 각 항(entry)은 행 번호와 열 번호를 사용하여 참조할 수 있습니다. 행렬 이름에 붙인 두 개의 첨자 중 첫 번째 첨자는 행 번호를, 두 번째 첨자는 열 번호를 나타냅니다. 그림 3.6의 행렬에서 M[2][3]의 값은 21이고 M[1][4]의 값은 5입니다.

프로그램에서 행렬 데이터를 저장하는 기억 공간을 2차원 배열이라고 부르고, 일반적으로 그림 3.6처럼 2차원 배열을 표시합니다. 파이썬에서는 리스트의 요소가 행렬의 각 행에 해당하는 "**리스트의 리스트(list of list)**"로 행렬 데이터를 표현합니다. 다음은 그림 3.6의 행렬 M을 파이썬 리스트로 표현한 것입니다.

```
M = [[6, 12, 5, 19, 7], [4, 20, 9, 11, 5], [8, 18, 21, 35, 1],
     [2, 22, 16, 13, 3]]
```

이 표현의 특징은 행렬에서 사용하는 방식으로 각 요소에 접근할 수 있다는 것입니다. 예를 들어, M[1]은 M의 두 번째 요소인데, 그 자체가 리스트이므로 다시 인덱싱을 수행할 수 있습니다. 따라서 M[1][4]와 같은 표현을 사용할 수 있습니다. 그림 3.7은 리스트 M을 위한 2차원 배열 구조를 보여줍니다. 그림에서 M은 4개의 요소로 구성된 참조 배열 구조를 나타내고, M의 각 요

소 또한 실제로는 참조 배열 구조이지만, 편의상 단순하게 1차원 배열 구조로 표시했습니다.

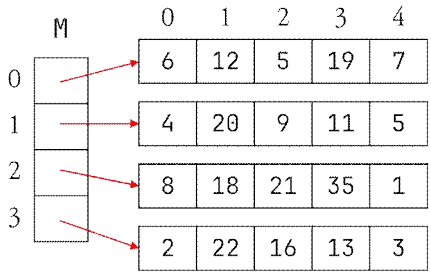


그림 3.7: 행렬 데이터를 위한 리스트의 리스트 구조

초기화된 1차원 리스트를 생성하는 것은 비교적 간단합니다. 다음 두 명령은 모두 0으로 초기화된 n개의 요소를 가진 리스트 L을 생성합니다.

```
L = [0] * n
L = [0 for _ in range(n)]
```

그러나 초기화된 2차원 리스트를 생성할 때는 유의해야 할 사항이 있습니다. 예를 들어, n개의 행과 m개의 열을 가진 0으로 초기화된 2차원 리스트를 생성하기 위해 다음과 같은 명령을 사용하면 의도하지 않은 결과를 초래하게 됩니다. 왜냐하면 순서 자료형 데이터에 대한 곱셈 연산은 참조값을 복사하기 때문입니다.

```
M = [[0] * m] * n
```

정수, 실수, 문자열, 튜플 등과 같이 불변 데이터에 대한 곱셈 연산은 참조값이 복사되더라도 아무런 문제가 발생하지 않지만, 리스트와 같은 가변 데이터에 대한 곱셈 연산은 신중하게 고려해야 합니다. 위에서 사용한 명령은 처음에 m개의 0으로 구성된 리스트를 만들고, 그 후 이 리스트의 참조값을 n개 복사하여 M의 요소로 만들어 줍니다. 예를 들어, n과 m이 각각 4와 5라고 하면, 그림 3.8과 같은 배열 구조가 만들어집니다.

이렇게 만들어진 리스트 M이 행렬 데이터를 유효하게 표현하지 못한다는 것을 프로그램 3.1에서 간단히 확인할 수 있습니다. 이 프로그램을 실행시키면 M[1][0], M[2][0], M[3][0]의 출력값으로 1이 나옵니다. 따라서 M이 독립적인 행들을 가진 2차원 배열 구조로 구성되어 있지 않음을 알 수 있습니다.

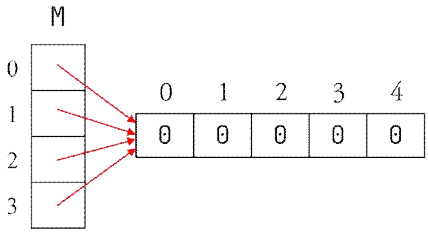


그림 3.8: 동일한 리스트를 참조하는 리스트

프로그램 3.1: 유효하지 않은 2차원 리스트

```
1 M = [[0]*5] * 4
2 M[0][0] = 1
3 print(M[0][0], M[1][0], M[2][0], M[3][0])
```

다음과 같은 **리스트 내포(list comprehension)**를 사용하면, 올바른 행렬 데이터, 즉 2차원 배열 구조를 생성할 수 있습니다.

```
M = [[0]*m for _ in range(n)]
```

이 명령은 for 반복이 수행될 때마다 새로운 리스트 객체를 생성하므로, 그림 3.7과 같은 정상적인 2차원 리스트 구조가 구성됩니다.

2차원 리스트를 다루는 과정에서 유의해야 할 두 번째 사항은 요소의 일부 또는 전부를 복사하는 경우입니다. 앞서 설명한 것처럼 정수, 실수, 문자열, 튜플 등과 같이 불변 데이터는 참조값이 복사되더라도 아무런 문제가 없지만, 리스트와 같은 가변 데이터를 복사할 때는 주의해야 합니다. 예를 들어, 프로그램 3.2를 살펴봅시다.

프로그램 3.2: 파이썬 리스트의 얕은 복사

```
1 L = [[0]*5 for j in range(4)]
2 M = L[:]      # 또는 M = L.copy()
3 L[0][0] = 1
4 print(M[0][0])
```

이 프로그램을 실행시키면 어떤 값이 출력될까요? 복사가 성공적으로 수행되었다면, M[0][0]의 값은 0이어야 하지만 출력값은 1입니다. 이렇게 되는 이유는 L의 요소인 내부 리스트에 대한 참조값이 그대로 M에 복사되었기 때문입니다. 즉, L과 M은 서로 다른 외부 리스트 객체이지만,

두 리스트가 가리키는 내부 리스트 객체는 동일합니다. 즉, 그림 3.9와 같은 배열 구조가 만들어지는 것입니다. 슬라이싱 연산이나 copy() 메소드는 리스트의 참조값을 복사한다는 점에 유의해야 합니다. 이렇게 참조값만을 복사하는 것을 **얕은 복사(shallow copy)**라고 부릅니다.

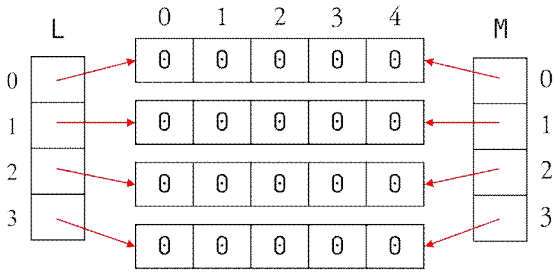


그림 3.9: 리스트의 얕은 복사

반면에, 참조값의 대상이 되는 데이터까지 완전하게 복사하는 것을 **깊은 복사(deep copy)**라고 부릅니다. 2차원 리스트에서 깊은 복사를 수행하기 위해서는 각 행을 독립적으로 복사하거나 copy 모듈의 deepcopy() 함수를 사용해야 합니다. 프로그램 3.3은 copy 모듈을 이용하여 2차원 리스트를 완전하게 복사하는 프로그램의 예를 보여줍니다.

프로그램 3.3: 파이썬 리스트의 깊은 복사

```
1 import copy
2 L = [[0]*5 for _ in range(4)]
3 M = copy.deepcopy(L)
```

배열의 요소가 1차원 배열이면 2차원 배열이라고 부르고, 배열 요소가 2차원 배열이면 3차원 배열이라고 부릅니다. 이처럼 배열의 차원은 얼마든지 높일 수 있습니다. 2차원 이상의 배열을 **다차원 배열(multidimensional array)**이라고 합니다. 2차원 배열 구조를 잘 이해하면 다차원 배열 구조도 같은 방식으로 이해할 수 있습니다. 또한 파이썬 리스트나 튜플을 이용하여 다차원 배열 구조도 어렵지 않게 구현할 수 있습니다.

참고로, 파이썬에서 배열을 효율적으로 다루기 위해 제공하는 모듈인 array와 numpy를 간단히 소개합니다. 내부 모듈인 array는 배열 요소들이 모두 동일한 자료형으로 구성되도록 제약을 두어 리스트보다 효율적으로 메모리 관리를 해 줍니다. array 모듈에 의해 생성되는 배열 객체의 요소는 문자, 정수, 실수 등 숫자형 데이터로 한정됩니다. 또한 정수의 경우만 보더라도 C 언어 스타일로 부호가 있는 정수와 부호가 없는 정수, 그리고 크기가 16비트, 32비트, 64비

트 정수 등으로 자료형을 구분합니다. 외부 모듈인 numpy는 빠른 수치 계산을 위해 제공되는 패키지로서 인공지능과 과학 계산 분야에서 주로 사용됩니다. numpy 배열은 C언어로 구현되어 최적화된 알고리즘과 효율적인 메모리 관리로 파이썬 리스트에 비해 월등한 성능을 보입니다. 단점으로는 외부 모듈로 제공되기 때문에 파이썬과는 별도로 설치 과정을 거쳐야 사용할 수 있습니다. 이 책에서는 array와 numpy 모듈을 이용하지 않습니다. 자료구조와 알고리즘의 개념을 이해하는 데 목적을 둔다면, 사용하기에 편리하고 시간복잡도 면에서는 동일한 파이썬 리스트를 이용하는 것이 더 효과적이기 때문입니다.

3.5 사례 연구: 리스트 정렬

정렬(sorting)은 특정한 기준에 따라 원소들을 순서대로 나열하는 작업입니다. 정렬은 많은 응용 프로그램에서 효율적인 수행을 위해 필수적으로 필요한 작업입니다. 정렬 문제는 비교적 간단하지만, 그 실용성 때문에 매우 중요하게 다루어져 왔습니다. 이에 따라 정렬 문제에 관한 수많은 이론 연구 결과와 다양한 정렬 알고리즘들이 발표되고 있습니다. 다양한 정렬 알고리즘을 학습하는 것은 알고리즘의 기본 원리와 동작 방식을 깊이 이해하고 나아가 새로운 알고리즘을 설계하는 데에 큰 도움이 됩니다.

이 절에서는 배열 기반으로 표현된 선형리스트에서 수행하는 작업의 사례로서 몇 개의 정렬 알고리즘을 소개합니다. 이 절에서는 비효율적이지만 간단한 정렬 알고리즘인 선택 정렬, 버블 정렬, 삽입 정렬을 차례로 소개하고, 마지막에는 파이썬에서 제공하는 정렬 함수의 사용법을 알아봅니다. 7장에서는 효율적이지만 다소 복잡한 정렬 알고리즘의 종류를 소개합니다.

정렬 알고리즘을 간단하면서 쉽게 설명하기 위해서, 정수 요소로 구성된 리스트에서 오름차순(ascending order)으로 정렬을 수행한다고 가정하겠습니다. 오름차순 정렬은 작은 값부터 시작하여 값이 증가하는 순서로 요소들을 배치하는 것을 의미합니다. 같은 값을 가진 요소가 존재하는 경우는 '오름차순'보다 '비내림차순(non-descending order)'이라고 하는 것이 더 명확하지만, 일반적으로 통용되는 '오름차순 정렬'이라는 용어를 사용하겠습니다.

선택 정렬(selection sort)은 매 단계에서 가장 작은 요소를 찾아 리스트의 맨 앞으로 보내는 과정을 반복하는 정렬 알고리즘입니다. 더 구체적으로 설명하면, 각 단계에서 현재 작업 구간에 있는 가장 작은 요소를 선택하여 작업 구간의 맨 앞에 있는 요소와 서로 위치를 교환합니다. 그

다음, 맨 앞 요소를 작업 구간에서 제외하여 작업 구간의 크기를 하나 줄입니다. 이런 과정을 반복하다가 작업 구간에 하나의 요소만 남으면 정렬 작업이 완료됩니다. 그림 3.10은 선택 정렬 과정의 예를 보여줍니다. 그림의 리스트에서 배경색이 흰 부분은 작업 구간을 나타내고, 원으로 표시된 곳은 현재 단계에서 가장 작은 요소를 나타냅니다.

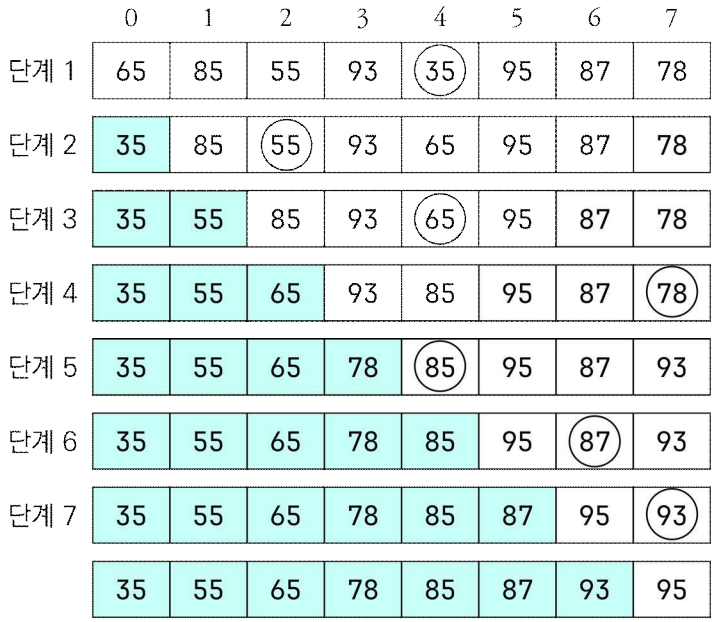


그림 3.10: 선택 정렬 알고리즘의 수행 과정

프로그램 3.4에 선택 정렬을 구현한 함수 `selection_sort(L)`가 나와 있습니다. 이 함수에서 반복문의 가장 안쪽에 있는 명령인 리스트의 두 요소를 비교하는 연산(`L[j] < L[min_pos]`)은 시간복잡도 분석을 위한 지배적 연산이 됩니다. 이 비교 연산은 변수 `start`의 값이 0부터 `n-2`까지 바뀔 때마다 차례대로 `n-1`, `n-2`, … , 1번 수행됩니다. 따라서 비교 연산의 총 수행 횟수는 `(n-1) + (n-2) + … + 1 = n(n-1)/2`이므로, 시간복잡도는 $O(n^2)$입니다. 선택 정렬은 각 단계에서 가장 큰 요소를 찾아 맨 끝의 요소와 교환하는 방식으로도 수행할 수 있습니다.

선택 정렬은 각 단계에서 단 한 번의 요소 교환을 수행하므로, 정렬 알고리즘 중에서 데이터 교환 횟수는 최소입니다. 따라서 비교 연산보다 요소 교환에 훨씬 많은 시간이 소요되는 특수한 환경에서는 선택 정렬이 시간복잡도의 차수와는 상관없이 가장 효율적인 정렬 알고리즘이 될 수 있습니다.

프로그램 3.4: 선택 정렬 알고리즘의 구현

```
1 def selection_sort(L):
2     """리스트 L의 요소들을 오름차순으로 정렬한다."""
3     n = len(L)                          # 요소의 개수
4     for start in range(n-1):            # start - 정렬 범위의 시작 위치
5         min_pos = start
6         for j in range(start+1, n):     # 가장 작은 요소 위치 탐색
7             if L[j] < L[min_pos]:       # 시간복잡도 분석을 위한 지배적 연산
8                 min_pos = j
9         L[min_pos], L[start] = L[start], L[min_pos]     # 요소 교환
```

버블 정렬(bubble sort)은 각 단계에서 두 개의 인접한 요소를 차례대로 비교하면서 더 큰 요소가 앞에 있으면 두 요소의 위치를 서로 교환합니다. 그 결과 각 단계에서 가장 큰 요소가 작업 구간의 맨 끝으로 이동합니다. 그다음, 맨 끝 요소를 작업 구간에서 제외하여 작업 구간의 크기를 하나 줄입니다. 이런 과정을 반복하여 작업 구간에 하나의 요소만 남으면 정렬 작업은 완료됩니다. 그림 3.11은 버블 정렬 과정의 예를 보여줍니다. 그림의 리스트에서 배경색이 흰 부분은 작업 구간을 나타냅니다. 그림에서 두 인접한 요소를 비교하는 모습을 나타낸 원호들이 비누거품처럼 보이기도 하여 버블이라는 이름을 가지게 되었습니다.

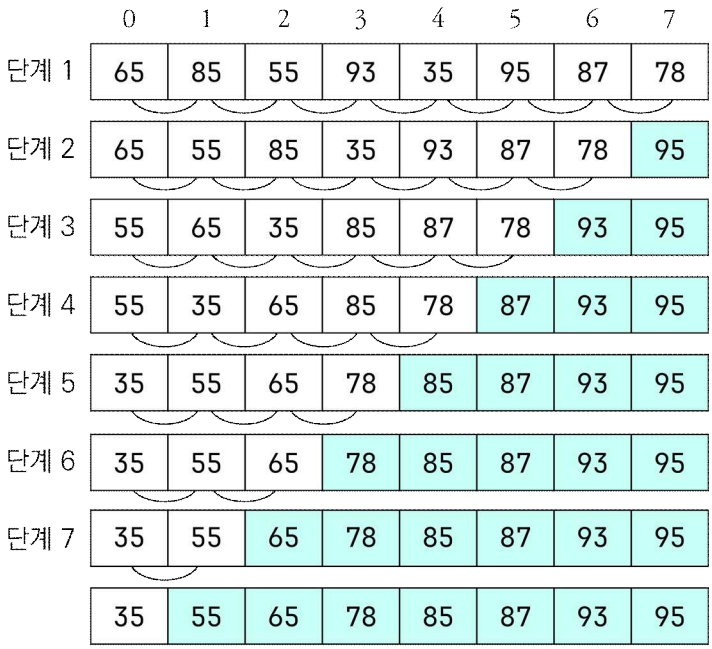


그림 3.11: 버블 정렬 알고리즘의 수행 과정

프로그램 3.5에는 버블 정렬을 구현한 함수 bubble_sort(L)가 나와 있습니다. 이 함수에서 반복문의 가장 안쪽에 있는 명령인 리스트의 두 요소를 비교하는 연산(L[j-1] > L[j])은 시간복잡도 분석을 위한 지배적 연산이 됩니다. 이 비교 연산은 변수 end의 값이 n부터 2까지 바뀔 때마다 차례대로 n-1, n-2, … , 1번 수행됩니다. 따라서 비교 연산의 총 수행 횟수는 (n-1) + (n-2) + … + 1 = n(n-1)/2이므로 시간복잡도는 $O(n^2)$입니다.

프로그램 3.5: 버블 정렬 알고리즘의 구현

```
1  def bubble_sort(L):
2     """리스트 L의 요소들을 오름차순으로 정렬한다."""
3     n = len(L)                              # 요소의 개수
4     for end in range(n, 1, -1):             # end - 작업 범위의 끝 + 1
5        for j in range(1, end):
6           if L[j-1] > L[j]:                # 지배적 연산
7              L[j-1], L[j] = L[j], L[j-1]   # 요소 교환
```

버블 정렬은 가장 간단하게 구현할 수 있다는 장점이 있지만, 요소의 교환 횟수가 가장 많이 발생하는 알고리즘입니다. 최악의 경우에는 비교 연산을 수행할 때마다 요소 교환이 이루어집니다. 선택 정렬과 마찬가지로 데이터의 분포와는 상관없이 비교 연산을 수행하므로, 최악의 경우 시간복잡도와 평균 시간복잡도가 동일합니다.

삽입 정렬(insertion sort)은 선택 정렬이나 버블 정렬보다 흥미로운 정렬 알고리즘입니다. 앞선 알고리즘과 마찬가지로 n개의 요소를 가진 리스트에 대해서 n-1 번의 단계가 수행됩니다. 정렬 과정에서 리스트의 앞쪽 구간이 차례대로 정렬된 상태로 변하게 됩니다. 리스트의 첫 번째 요소는 그 자체가 이미 정렬된 상태이므로, 두 번째 요소부터 고려합니다. 각 단계에서는 현재 고려하는 요소를 리스트의 앞쪽 구간에 삽입하여 정렬된 구간을 확장합니다. 앞쪽 구간에 요소를 삽입하는 작업은 오른쪽에서 왼쪽으로 순차적으로 탐색하면서 자신보다 작거나 같은 요소를 만날 때까지 자신보다 큰 요소를 한 칸 뒤로 이동시킵니다. 그림 3.12는 삽입 정렬 과정의 예를 보여줍니다. 그림의 리스트에서 배경색이 칠해진 부분은 정렬된 구간을 나타냅니다.

삽입 정렬을 구현한 함수 insert_sort(L)가 프로그램 3.6에 나와 있습니다. 이 함수의 지배적 연산으로, 앞의 두 알고리즘과 마찬가지로 두 요소를 비교하는 연산(L[j] > item)으로 둡니다. 삽입 정렬에서 이 비교 연산의 수행 횟수는 입력 분포에 따라 달라집니다. 그림 3.12의 예에서는 총 17(=1+2+1+4+1+3+5)번의 비교 연산이 수행됩니다. 그래서 삽입 정렬에서는 최악의 경우 시간복잡도를 고려해야 합니다. 최악의 경우는 각 단계에서 k번째 요소가 항상 리스트의 맨 앞으로 이동되는 경우입니다(1≤k≤n-1). 이 경우, for 문이 반복될 때마다 비교 연

	0	1	2	3	4	5	6	7
단계 1	65	85	55	93	35	95	87	78
단계 2	65	85	55	93	35	95	87	78
단계 3	55	65	85	93	35	95	87	78
단계 4	55	65	85	93	35	95	87	78
단계 5	35	55	65	85	93	95	87	78
단계 6	35	55	65	85	93	95	87	78
단계 7	35	55	65	85	87	93	95	78
	35	55	65	78	85	87	93	95

그림 3.12: 삽입 정렬 알고리즘의 수행 과정

산은 차례대로 1, 2, … , n-1번 수행됩니다. 따라서 비교 연산의 총 수행 횟수는 n(n-1)/2이므로, 최악의 경우 시간복잡도는 $O(n^2)$입니다. 이제 삽입 정렬의 평균 시간복잡도를 살펴볼까요? 각 단계에서 k번째 요소는 평균적으로 k/2개의 요소와 비교된다고 볼 수 있습니다. 그러면 비교 연산의 총 수행 횟수는 약 $n^2/4$이 되므로, 평균 시간복잡도도 $O(n^2)$입니다.

프로그램 3.6: 삽입 정렬 알고리즘의 구현

```
1  def insertion_sort(L):
2     """리스트 L의 요소들을 오름차순으로 정렬한다."""
3     for k in range(1, len(L)):          # n-1번 반복, n = len(L)
4        item = L[k]                      # 삽입할 요소
5        j = k-1                          # 탐색의 시작 위치
6        while j >= 0 and L[j] > item:    # 지배적 연산
7           L[j+1] = L[j]                 # item보다 큰 요소를 뒤로 이동
8           j -= 1
9        L[j+1] = item                    # item 삽입
```

삽입 정렬의 가장 큰 장점은 거의 정렬된 데이터에 대해서 매우 효율적으로 정렬 작업이 수행된다는 것입니다. 이미 정렬된 데이터가 입력으로 들어오는 경우, 삽입 정렬의 각 단계에서 비교

연산이 한 번씩만 수행되므로 총 수행 시간은 $O(n)$이 됩니다. 단지 k 개만 제외하고 요소들의 상대적인 순서가 유지된 데이터가 입력으로 들어오면, 삽입 정렬의 수행 시간은 $O(n + kn)$입니다. k가 상수이면 수행 시간은 $O(n)$이고, $k = O(\log n)$이면 수행 시간은 $O(n\log n)$이 됩니다. 이런 상황에서는 다른 어떤 정렬 알고리즘과 비교해도 효율적이라고 할 수 있습니다.

마지막으로, 파이썬에서 제공하는 정렬 함수의 사용법을 알아봅니다. 파이썬은 정렬 연산을 위해 내장 함수 sorted()와 리스트 메소드 sort()를 제공합니다. 두 함수 모두 로그선형 시간, 즉 $O(n\log n)$ 시간에 수행됩니다. sorted() 함수는 정렬 대상이 되는 객체를 첫 번째 매개변수로 받아 정렬된 결과를 리스트 형태로 반환합니다. 반면에 리스트 메소드 sort()는 리스트 데이터를 정렬된 상태로 변환해 줍니다. 이런 차이점을 제외하면 정렬 과정에서 고려해야 할 사항은 동일합니다. 그래서 sorted() 함수를 예로 들면서 설명하겠습니다. 추가적인 인수를 사용하지 않고 함수를 호출하면 오름차순으로 정렬을 수행합니다. 정렬 대상은 비교 가능한 자료형 요소들로 구성되어야 합니다. 만일, 정수와 문자열 또는 튜플과 같이 서로 비교할 수 없는 요소들이 정렬 대상에 포함되어 있으면 오류가 발생합니다. 정렬 기준은 숫자 데이터인 경우는 크기 순서이고, 순서 자료형 데이터인 경우는 사전식 순서입니다. 키워드 인수 reverse에 True를 배정하여 함수를 호출하면, 내림차순으로 정렬해 줍니다. 프로그램 3.7에 sorted() 함수의 사용 예가 나와 있습니다.

프로그램 3.7: sorted() 함수의 사용 예

```
1 A = [5, 2, 9, 1, 7]
2 B = ['kim', 'lee', 'park', 'choi']
3 print(sorted(A))                  # [1, 2, 5, 7, 9]
4 print(sorted(B, reverse=True))    # ['park', 'lee', 'kim', 'choi']
```

만약 사용자가 정렬 기준을 바꾸고 싶으면, 시스템에 정렬 기준을 알려주는 함수를 제시해야 합니다. 이 함수는 하나의 매개변수를 가지며, 이 매개변수에는 정렬 대상의 요소에 해당하는 데이터가 배정됩니다. 시스템은 이 함수가 반환해 주는 데이터를 가지고 정렬 순서를 결정하게 됩니다. sorted() 함수의 키워드 인수 key에 이 함수의 이름을 배정해 두면, 시스템은 정렬 과정에서 이 함수를 이용하게 됩니다. 예를 들어, 프로그램 3.8은 문자열 리스트를 사전식 순서가 아니라 문자열의 길이를 기준으로 정렬해 줍니다. 그 결과, 리스트 ['c', 'go', 'java', 'python']이 출력됩니다.

프로그램 3.8: 문자열의 길이를 기준으로 하여 정렬을 수행하는 예

```
1 old_list = ['java', 'c', 'python', 'go']
2 new_list = sorted(old_list, key=len)       # len() 함수를 이용
3 print(new_list)
```

프로그램 3.9는 정렬 기준을 제시하는 함수를 직접 정의하는 예를 보여줍니다. 정렬 대상인 리스트의 요소는 (학번, 이름, 점수)로 구성되는 튜플이라고 가정하겠습니다. score() 함수는 정렬 대상의 요소 자료형을 매개변수로 받아 세 번째 요소인 점수의 부호를 바꾸어 반환합니다. 시스템은 이 반환된 값을 가지고 사전식 순서로 수행하므로, 결국 점수의 역순으로 정렬이 수행됩니다. 그 결과, 리스트 [(12, '박창호', 97), (11, '홍길동', 95), (15, '김철수', 89)]가 출력됩니다.

프로그램 3.9: 정렬 기준을 제시하는 함수

```
1 def score(E):
2    return -E[2]    # 세 번째 요소의 부호를 바꾸어 리턴
3
4 score_list = [(11, '홍길동', 95), (12, '박창호', 97), (15, '김철수', 89)]
5 score_list = sorted(score_list, key=score)
6 print(score_list)
```

정렬 기준이 되는 함수는 일반적으로 매우 간단하므로, 람다(lambda) 함수를 활용할 수 있습니다. 프로그램 3.10은 람다 함수를 이용하여 sorted() 함수를 호출하는 예를 보여줍니다.

프로그램 3.10: lambda 함수를 활용하여 정렬을 수행하는 예

```
1 score_list = [(11, '홍길동', 95), (12, '박창호', 97), (15, '김철수', 89)]
2 score_list = sorted(score_list, key=lambda E: -E[2])
3 print(score_list)
```

프로그램 3.11은 두 개 이상의 기준을 가지고 정렬을 수행하는 프로그램의 예를 보여줍니다. 리스트 data의 요소는 (이름, 점수)로 구성된 튜플입니다. 정렬 순서는 점수의 내림차순을 기본으로 하고, 점수가 같은 경우에는 이름이 오름차순으로 배치되도록 나열합니다. 키워드 인수 key에 배정되는 람다 함수는 (-점수, 이름)으로 구성되는 튜플을 반환합니다. 시스템은 이 반환 값을 가지고 사전식 순서로 정렬하므로, 사용자가 원하는 방식으로 정렬이 수행됩니다. 그 결과, 리스트 [('cho', 5), ('kim', 5), ('hong', 4), ('lee', 4), ('park', 2)]가 출력됩니다.

프로그램 3.11: 두 개 이상의 기준을 가지고 정렬을 수행하는 예

```
data = [('kim', 5), ('lee', 4), ('park', 2), ('hong', 4), ('cho', 5)]
result = sorted(data, key=lambda E: (-E[1], E[0]))
print(result)
```

연습문제

1. 자료구조를 명세(specification)와 구현(implementation)으로 구분하는 것이 왜 유용한지 설명하시오.

2. 파이썬 `range()`, `map()`, `filter()` 등의 함수에 의해 생성되는 데이터는 필요할 때마다 요소를 계산하는 순서 데이터이다. 이러한 **지연 순서(lazy sequence)**의 특징과 동작 원리를 조사하고, 리스트, 튜플, 문자열과 같은 보통의 순서 데이터와 비교하여 지연 순서 데이터가 가지는 장점을 설명하시오.

3. `sys` 모듈의 내장 함수 `getsizeof()`를 이용하면 파이썬 리스트에 할당된 메모리 용량을 알 수 있다. 이 함수를 이용하여, 파이썬 리스트에 정수를 하나씩 추가하면서 메모리 용량이 어떻게 변화하는지 관찰하시오. 반대로, 리스트의 요소를 하나씩 제거하면서 메모리 용량이 변화하는 추이를 관찰하시오.

4. 파이썬 리스트의 메소드 `extend()`와 `append()`를 반복적으로 호출하여 동일한 리스트를 구성하는 데 소요되는 수행 시간을 비교하는 실험을 하시오.

5. 정렬된 두 파이썬 리스트 A와 B를 병합하여 하나의 정렬된 리스트를 반환하는 파이썬 함수 `merge(A, B)`를 작성하시오. 단, 이 함수의 시간복잡도는 $O(n)$이어야 한다. 여기서 `n = len(A) + len(B)` 이다.

6. 파이썬 명령 `L.remove(x)`는 리스트 L에서 값이 x인 첫 번째 요소를 삭제한다. 길이가 n인 리스트 L에서 값이 x인 모든 요소를 삭제하는 파이썬 함수 `removeall(L, x)`를 작성하시오. 단, 이 함수의 시간복잡도는 $O(n)$이어야 한다. `remove()` 메소드를 반복적으로 호출하는 방법은 효율적이지 않다는 것에 유의하라.

7. 데이터베이스에서 유용한 연산 중 하나는 **자연 조인(natural join)**이다. 데이터베이스를 객체의 순서쌍 리스트로 본다면, 데이터베이스 A와 B의 자연 조인은 A의 순서쌍 (x, y)와 B의 순서쌍 (y, z)를 결합한 순서쌍 (x, y, z)이다. n개의 순서쌍 리스트 A와 m개의 순서쌍 리스트 B의 자연 조인을 구하는 효율적인 알고리즘을 설계하고 시간복잡도를 분석하시오.

8. 길이가 n인 파이썬 리스트 L에 1과 n-1 사이의 정수가 들어 있다. 단, 하나의 정수만 두 군데 저장되어 있고, 나머지 정수들은 하나씩 저장되어 있다. L에 중복해서 들어 있는 숫자를 찾는 효율적인 알고리즘을 설계하고 시간복잡도를 분석하시오.

9. n×n 크기의 파이썬 2차원 리스트 L에 정수들이 들어 있다. L에 저장된 모든 숫자들의 합을 구하는 파이썬 명령을 작성하시오. 단, 파이썬 내장 함수 sum()과 리스트 내포(list comprehension)를 사용하여 한 줄의 코드로 작성하시오.

10. 다음 표에서 첫 행은 오름차순으로 정렬하기 위해 주어진 입력 데이터이고, 마지막 행은 정렬이 완료된 후의 결과이다. 중간의 행들은 선택 정렬, 버블 정렬, 삽입 정렬 알고리즘을 통하여 얻을 수 있는 중간 과정을 보여준다. 각 행에 대해 어떤 정렬 알고리즘의 중간 과정인지를 쓰시오.

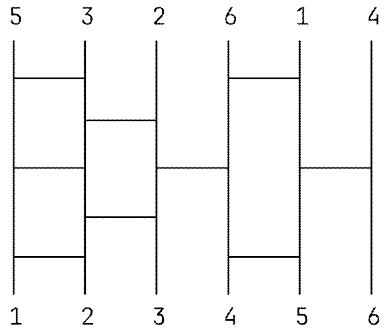
입력	21	41	15	25	21	37	45	9	23	18	36	4	11
①	9	15	21	21	25	37	41	45	23	18	36	4	11
②	15	21	21	25	37	9	23	18	36	4	11	41	45
③	4	9	11	15	18	37	45	41	23	21	36	21	25
결과	4	9	11	15	18	21	21	23	25	36	37	41	45

11. 버블 정렬의 수행 중간에 리스트가 정렬된 상태가 되는 경우, 불필요한 작업을 수행하지 않도록 하려고 한다. 정렬된 상태를 감지하게 되면 곧바로 정렬 작업을 종료시킬 수 있도록 프로그램 3.5의 버블 정렬 프로그램을 수정하시오.

12. 선택 정렬, 버블 정렬, 삽입 정렬, 파이썬 내장 정렬(sort, sorted)를 구현하여 실제 실행 시간을 비교하시오. 정렬 실험을 위한 데이터를 random 모듈을 이용하여 구성하고, 10부터 10^5까지 다양한 크기의 데이터를 가지고 실험하시오.

13. 튜플 (name, korean_score, english_socre)로 구성된 학생들의 성적 리스트 L이 주어진다. 리스트 L을 국어 점수(korean_score)를 기준으로 내림차순으로 정렬하되, 국어 점수가 같으면 영어 점수(english_score)를 기준으로 내림차순으로 정렬하는 파이썬 코드를 작성하시오. 단, 리스트의 sort() 메소드 또는 sorted() 함수와 람다 함수를 이용하라.

14. n개의 수직선으로 구성되는 사다리를 그리려고 한다. 사다리의 위쪽은 1부터 n까지의 정수가 임의의 순서로 쓰여 있고, 아래쪽은 1부터 n까지 정수가 순서대로 쓰여 있다(아래 그림 참조). 사다리에서 최소 개의 수평 선분을 그어, 위쪽 숫자와 아래쪽 숫자가 만나도록 하는 알고리즘을 설계하시오.



15. 두 행렬 A와 B의 덧셈은 두 행렬에서 같은 위치의 항끼리 더하는 연산이다. 따라서 행렬 덧셈이 성립하기 위해서는 두 행렬의 크기가 같아야 한다. 입력으로 n×m 크기의 두 행렬 데이터가 주어질 때, 두 행렬의 합 행렬을 출력하는 프로그램을 작성하시오.

16. 두 행렬 A와 B의 곱셈으로 얻은 행렬을 C라고 하자. 행렬 곱셈이 성립하기 위해서는 A의 열 크기와 B의 행 크기가 같아야 한다. 즉, A의 크기가 p×n이면 B의 크기는 n×q이어야 하고, 곱셈의 결과로 C의 크기는 p×q가 된다. C[i][j]는 A의 i번째 행 (a_0, a_1, … , a_{n-1})과 B의 j번째 열 (b_0, b_1, … , b_{n-1})을 아래와 같이 계산한 결과이다.

$$C[i][j] = a_0 \times b_0 + a_1 \times b_1 + \cdots + a_{n-1} \times b_{n-1}$$

입력으로 두 행렬 데이터가 주어질 때, 두 행렬의 곱 행렬을 출력하는 프로그램을 작성하시오.

17. 오름차순으로 정렬된 n개의 정수 리스트 L과 정수 target이 주어질 때, target = a+b인 되는 L의 두 요소 a와 b를 $O(n)$ 시간에 출력하는 프로그램을 작성하시오. 단, 그러한 a, b가 존재하지 않으면 -1을 출력하고, 여러 쌍이 존재하면 a*b가 가장 큰 a, b를 출력한다.

18. **마방진(magic square)**은 1부터 n^2까지의 정수로 구성된 n×n 크기의 행렬로서, 행의 합, 열의 합, 주대각선의 합이 모두 같다. 3×3 크기와 5×5 크기를 가진 마방진은 아래 그림과 같다.

8	1	6
3	5	7
4	9	2

17	14	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	18	21	3
11	18	25	2	9

홀수 n에 대한 n×n 크기의 마방진 M을 생성하는 다음의 자연어 알고리즘을 파이썬 프로그램으로 구현하시오. 행렬 M의 첨자는 0부터 시작한다.

```
1. n×n 크기의 빈 행렬 M을 생성한다.
2. 숫자 1을 첫 행의 가운데 칸, 즉 M[0][n/2]에 기입한다.
3. 2부터 n2까지의 각 숫자 k는 다음 규칙에 따라 기입한다.
   ① 숫자 k-1이 M[r][c]에 기입되어 있다고 하자.
   ② i와 j를 각각 r-1과 c+1로 둔다.
   ③ 만일 i < 0이면 i를 n-1로 둔다.
   ④ 만일 j와 n이 같으면 j를 0으로 둔다.
   ⑤ 만일 M[i][j]에 이미 숫자가 있으면, i와 j를 각각 r+1과 c로 둔다.
   ⑥ M[i][j]에 숫자 k를 기입한다.
```

19. [실습 문제] **모스 부호(Morse code)**는 점과 선으로 이루어진 전신 부호 체계로서 단음(점)과 장음(선)으로 불리는 짧고 긴 음조의 연속된 부호로 문자를 표현한다. 다음 표에는 알파벳의 문자에 대한 모스 부호가 나와 있다.

입력으로 들어온 문자열에 상응하는 모스 코드를 출력하는 프로그램을 작성하시오. 입력으로 길이가 100 이하인 문자열이 들어온다. 문자열은 알파벳 대소문자로 구성된 문장이고 대소문자는 구분하지 않는다. 문장의 단어들은 빈칸으로 구분된다. 프로그램은 입력 문자열의 각 단어에 해당하는 모스 코드를 한 줄에 하나씩 출력한다. 모스 부호의 단음은 마침표('.')로, 장음은 하이픈('-')으로 출력한다. 문자에 해당하는 코드들 사이에는 한 개의 빈칸을 둔다.

A	.-	J	.---	S	...
B	-...	K	-.-	T	-
C	-.-.	L	.-..	U	..-
D	-..	M	--	V	...-
E	.	N	-.	W	.--
F	..-.	O	---	X	-..-
G	--.	P	.--.	Y	-.--
H	....	Q	--.-	Z	--..
I	..	R	.-.		

입력의 예	출력의 예
ATTACK AT DAWN	.- - - .- -.-. -.- .- - -.. .- .-- -.

20. [실습 문제] 표 3.3의 모스 부호에 대해서 모스 코드를 입력으로 받아 이를 문자열로 변환하는 프로그램을 작성하시오. 입력으로 각 줄에 하나의 단어에 해당하는 모스 코드가 들어온다. 단어의 각 문자는 한 개의 빈칸으로 구분한다. 입력의 마지막 줄은 문자 '$'로 표시된다. 프로그램은 입력으로 들어오는 모스 코드에 해당하는 문자열을 한 줄에 출력한다.

입력의 예	출력의 예
.... . .-.. .--. -- . $	HELP ME

제4장

재귀 알고리즘

4.1 재귀 관계
4.2 재귀 함수
4.3 재귀 함수의 성능 분석
4.4 사례 연구: 하노이 탑
4.5 사례 연구: 이진 탐색
연습문제

학습 목표

- 규칙이나 패턴에서 재귀적 구조를 파악하고, 이를 재귀 관계로 표현할 수 있다.
- 재귀 관계가 주어지면, 이를 재귀 함수로 구현할 수 있다.
- 재귀 함수의 동작을 이해하고, 시간복잡도와 공간복잡도를 분석할 수 있다.
- 중복 호출, 재귀 깊이의 한계 등 재귀적 해법의 단점을 분석하여 이를 반복적 해법, 메모이제이션 등으로 해결할 수 있다.
- 하노이 탑 문제와 이진 탐색을 위한 재귀 알고리즘의 동작 원리를 이해하고, 이를 변형하거나 새로운 문제에 응용할 수 있다.

4.1 재귀 관계

다음 수열(sequence)을 살펴봅시다.

1 3 6 10 15 21 28 …

이 수열의 n번째 항을 S(n)이라고 할 때, S(1)=1, S(2)=3입니다. 그러면 S(100)은 어떤 수일까요?

n	1	2	3	4	5	6	7	…
S(n)	1	3	6	10	15	21	28	…

위의 수열을 잘 살펴보면, n 번째 항은 항상 n-1 번째 항에 n을 더한 것과 같습니다. 이 사실을 이용하면, S(n)을 정의하는 식을 다음과 같이 세울 수 있습니다.

$$S(n) = S(n-1) + n, \qquad n = 2, 3, 4, \cdots$$

위와 같이 수열의 각 항을 앞선 항 또는 항들에 대한 식으로 표현한 것을 **재귀 관계(recurrence relation)** 또는 **재귀 관계식**이라고 부릅니다. 같은 개념으로 **순환 관계**, **귀납 관계**, **점화식**, **재귀식** 등의 용어가 사용됩니다.

위 식은 n이 1일 경우 S(0)이 정의되어 있지 않으므로 유효하지 않습니다. 따라서 S(1)은 그 값을 직접 명시해 주어야 합니다. 이렇게 재귀 관계에서 정의되지 않은 항이 나타나지 않도록 해 주는 것을 **초기 조건(initial condition)**이라고 부릅니다. 위의 재귀 관계에서 초기 조건은 다음과 같습니다.

$$S(1) = 1$$

재귀 관계로부터 순환되지 않는 일반적인 식을 찾는 것을 "재귀 관계를 푼다"라고 말하고 이를 **재귀 관계의 해** 또는 **폐쇄형(closed form)**이라고 부릅니다. 위 재귀 관계의 해는 다음과 같습니다. 이 식은 바로 등차수열의 합 공식입니다.

$$S(n) = \frac{n(n+1)}{2}, \qquad n = 1, 2, 3, \cdots$$

재귀 관계의 초기 조건이 바뀌면 재귀 관계의 해가 달라집니다. 위 식에서 초기 조건을 다음과

같이 설정하면,

S(1) = 3

S(2)와 S(3) 등은 다음과 같이 바뀝니다.

S(2) = S(1) + 2 = 5
S(3) = S(2) + 3 = 8

따라서, 폐쇄형은 다음과 같이 변경됩니다.

$$S(n) = \frac{n(n+1)}{2} + 2, \qquad n = 1, 2, 3, \cdots$$

다음으로 솔방울, 해바라기 씨앗, 달팽이껍질, 나선 은하계 등 자연에서 발견되는 많은 회오리 모양에서 특징을 찾을 수 있는 **피보나치 수열(Fibonacci sequence)**을 살펴보겠습니다. 피보나치 수열은 다음과 같습니다.

0 1 1 2 3 5 8 13 21 34 …

피보나치 수열은 n 번째 항이 n-1 번째 항과 n-2 번째 항의 합으로 정의됩니다. 따라서 n 번째 항을 F(n)이라고 하면, F(n)을 다음과 같은 재귀 관계로 표현할 수 있습니다. 여기서는 관례에 따라 n을 0부터 시작합니다. 재귀 관계식에 직전의 두 개 항이 나타나므로, 두 개의 항이 초기 조건으로 주어집니다.

F(0) = 0
F(1) = 1
F(n) = F(n-1) + F(n-2), n = 2, 3, 4, …

피보나치 수열의 폐쇄형을 구하는 것은 쉽지 않습니다. 특성 방정식(characteristic equation)과 같은 수학적 기법을 사용하여 폐쇄형을 구하면 아래 식처럼 복잡한 형태가 나옵니다.

$$F(n) = \frac{1}{\sqrt{5}}\left\{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right\}, \quad n = 0, 1, 2, \cdots$$

주어진 재귀 관계의 n번째 항의 값을 구하는 문제에 대해 가장 효율적인 해법은 재귀 관계의 폐쇄형을 구해 답을 계산하는 것이지만, 재귀 관계의 폐쇄형을 항상 쉽게 구할 수 있는 것이 아닙

니다. 앞서 등차수열의 합 공식처럼 폐쇄형을 쉽게 구할 수 있는 경우도 있지만, 피보나치 수열처럼 매우 어려운 경우도 있으며, 심지어는 폐쇄형을 구하는 특별한 수학적 해법이 존재하지 않을 수도 있습니다.

폐쇄형을 이용하지 않고 재귀 관계의 n번째 항을 구하기 위해서는 재귀 관계를 전개해 가면서 풀어나가는 방법을 사용해야 합니다. 이 방법은 어떤 형태의 재귀 관계도 해결할 수 있다는 장점을 가지지만, 지루한 계산 과정을 반복적으로 수행해야 하는 단점이 있습니다. 또한 이 방법은 진행 순서에 따라 **하향식(top-down approach)**과 **상향식(bottom-up approach)**으로 구분됩니다. 하향식은 순환 과정을 초기 조건까지 역으로 전개한 후에 대입 과정을 거쳐서 답을 구하는 방식이고, 상향식은 초기 조건부터 시작하여 답을 찾아가는 방식입니다. 예를 들어, 맨 처음에 예를 든 수열(등차수열의 합 수열)에서 S(5)를 구하는 하향식 계산 과정은 다음과 같습니다. 처음에는 S(5)부터 시작하여 초기 조건에 이르기까지 식을 전개합니다.

```
S(5) = S(4) + 5
S(4) = S(3) + 4
S(3) = S(2) + 3
S(2) = S(1) + 2
S(1) = 1
```

초기 조건에 도달하면 다시 역순으로 대입 과정을 거쳐 답을 구합니다.

```
S(2) = 1 + 2 = 3
S(3) = 3 + 3 = 6
S(4) = 6 + 4 = 10
S(5) = 10 + 5 = 15
```

물론, 이 경우는 처음부터 역순으로 답을 구해나가는 상향식을 이용하는 것이 더 효율적입니다. 그런데, 재귀 관계가 다음과 같은 형태인 경우를 고려해 봅시다.

$$T(n) = 1$$
$$T(n) = 3T(\lfloor n/2 \rfloor) + 2, \qquad n \geq 2$$

위 재귀 관계의 경우는 상향식보다는 하향식이 더 효과적일 수 있습니다. 가령 T(15)를 구한다고 해 봅시다. 단순하게 상향식으로 답을 구하려면 T(1), T(2), T(3), … , T(15) 순으로 하나씩 식을 계산해야 합니다. 그러나 하향식의 경우는 다음과 같이 계산하게 됩니다.

```
T(15) = 3T(7) + 2
T(7) = 3T(3) + 2
T(3) = 3T(1) + 2
T(1) = 1
T(3) = 3 · 1 + 2 = 5
T(7) = 3 · 5 + 2 = 17
T(15) = 3 · 17 + 2 = 53
```

T(15)는 식을 전개하는 과정을 4번만 수행하면 됩니다. 인수의 값이 클수록 식을 계산해야 하는 횟수에서 차이가 크게 벌어집니다. T(1000)은 단지 10번만, T(10000)은 단지 14번만 식을 전개하면 됩니다. 따라서 하향식이 더 효율적인 경우도 있다는 것을 알 수 있습니다.

재귀 관계를 하향식 또는 상향식으로 풀어나가는 과정을 컴퓨터 프로그램으로 구현할 수 있습니다. 하향식은 곧 소개될 재귀 함수가 실행되는 모습과 유사합니다. 그래서 하향식을 **재귀적 해법(recursive solution)**이라고 부릅니다. 이에 반해 상향식은 반복문으로 프로그램이 작성되므로 **반복적 해법(iterative solution)**이라고 부릅니다.

재귀 관계는 수열뿐만 아니라 일반적인 문제의 해를 재귀적으로 표현하는 데에도 사용됩니다. 이 경우, 재귀 관계는 주어진 문제를 더 작은 문제의 해를 이용하여 해결하는 방법을 제시합니다. 여기서 "작은 문제"라는 표현은 문제 크기, 즉 문제에서 다루는 데이터의 크기가 작다는 것을 의미합니다. 데이터가 다수의 원소로 구성된 경우, "크기가 작은 데이터"는 더 적은 개수의 원소로 구성된 데이터를 말합니다. 데이터가 숫자인 경우는 크기가 작은 숫자, 문자열이라면 길이가 더 짧은 문자열이 크기가 작은 데이터로 취급됩니다. 예를 들어, 1장에서 소개했던 최대공약수 문제를 다시 살펴봅시다. 두 자연수 x와 y의 최대공약수를 gcd(x, y)라고 할 때, 다음과 같이 재귀 관계로 표현할 수 있습니다. 아래 식에서 mod는 나머지 연산자(modulo operator)를 나타냅니다.

```
gcd(x, y) = gcd(y, x mod y),      y > 0
gcd(x, 0) = x
```

위 재귀 관계에서 하향식으로 식을 전개할 때마다 인수 y의 값이 점점 작아지다가 y의 값이 0이 되면 초기 조건에 도달합니다. 32와 12의 최대공약수를 다음과 같이 하향식으로 구하면 최대공약수가 4임을 알 수 있습니다.

```
gcd(32, 12) = gcd(12, 8)
            = gcd(8, 4)
            = gcd(4, 0)
```

다음으로 주어진 문자열이 **회문(palindrome)**인지를 검사하는 문제를 고려해 봅시다. 회문은 거꾸로 읽어도 원래와 같은 문자열을 의미합니다. 문자열에서 영어 대소문자를 구분하지 않고 빈칸을 무시한다고 가정하면, 영어의 "Nurses run", "Was it a cat I saw"와 한글의 "다시 합창합시다", "소주 만 병만 주소" 등이 회문에 해당합니다. 회문 검사를 위한 재귀 관계는 다음과 같습니다. 문자열 `str`이 회문이면 `True`, 회문이 아니면 `False`인 재귀 관계를 `pal(str)`이라고 합시다. `str`은 입력으로 주어진 문자열에서 빈칸이 모두 삭제된 상태라고 가정할 때, `pal(str)`은 다음과 같이 정의됩니다.

`pal(str) = True,`	`str`의 길이가 1 이하인 경우
`pal(str) = False,`	`str`의 양 끝 문자가 다른 경우
`pal(str) = pal(substr),`	그 밖의 경우 (`substr`은 `str`에서 양 끝 문자를 뺀 문자열)

위 재귀 관계에서 하향식으로 식을 전개할 때마다 인수인 문자열의 길이가 점점 줄어들다가 0 또는 1이 되면 초기 조건에 도달합니다. 문자열 `"civic"`을 가지고 회문 여부를 검사하는 과정은 다음과 같습니다.

```
pal("civic") = pal("ivi")
             = pal("v")
             = True
```

재귀 관계는 재귀적으로 문제를 해결하는 수학적 해법입니다. 주어진 문제에서 재귀 관계를 찾아내기 위해서는 문제로부터 일반적인 규칙이나 패턴을 도출하는 과정이 필요합니다. 재귀 관계를 풀면 문제의 답을 구할 수 있으므로, 재귀 관계 자체를 컴퓨터 알고리즘으로 보아도 무방합니다. 그래서 재귀 관계의 일반적인 형태를 **재귀 알고리즘(recursive algorithm)**이라고 부릅니다. 더구나, 다음 절에서 설명하겠지만, 재귀 함수를 이용하면 재귀 관계의 구조를 그대로 컴퓨터 프로그램으로 구현할 수 있습니다. 즉 재귀 함수는 수학적 해법이 프로그램으로 표현되는 신기하면서도 강력한 프로그래밍 기법입니다.

4.2 재귀 함수

프로그램에서 **재귀 함수(recursive function)**는 자신을 호출하는 문장을 가진 함수를 말하며, 그

러한 문장을 **재귀 호출(recursive call)**이라고 부릅니다. 엄밀하게 말하면 재귀 호출은, 동일한 함수 코드를 사용하지만, 독립적인 메모리 공간을 가지는 실행 프레임(execution frame)을 생성시킵니다[7]. 마치 모습만 같은 별개의 함수를 호출한다고 볼 수 있습니다.

재귀 함수의 가장 큰 장점은 재귀 관계의 구조를 그대로 프로그램으로 표현할 수 있다는 것입니다. 재귀 관계의 초기 조건은 `if` 문으로 표현하고, 재귀 관계식은 재귀 호출을 포함한 문장으로 표현합니다. 예를 들어, 앞서 소개되었던 등차수열의 합을 구하는 다음의 재귀 관계를 프로그램 4.1과 같이 재귀 함수로 구현할 수 있습니다.

$$S(n) = 1, \qquad \text{if } n = 1$$
$$S(n) = S(n-1) + n, \qquad \text{if } n \geq 2$$

프로그램 4.1: 등차수열의 합을 구하는 재귀 함수

```
1 def rsum(n):
2     """1부터 n까지의 합을 반환하는 재귀 함수"""
3     if n == 1:                  # 종료 조건문
4         return 1
5     else:
6         return n + rsum(n-1)    # 재귀 호출문
```

함수 `rsum(n)`은 매개변수 `n`의 값에 따라 종료 조건문과 재귀 호출문으로 나뉩니다. 종료 조건문은 재귀 관계의 초기 조건과 같은 개념으로, 더 이상의 재귀 호출이 발생하지 않도록 해 줍니다. 함수는 호출될 때마다 매개변수 `n`의 값이 1씩 작아져서 결국 종료 조건에 도달합니다.

그림 4.1은 재귀 함수가 시스템 내부에서 어떻게 동작하는지를 보여줍니다. 맨 처음, 프로그램의 어느 곳에서 함수를 호출하는 명령 `rsum(3)`이 수행된다고 합시다. 시스템은 함수가 호출될 때마다 새로운 실행 프레임을 생성합니다. 처음에 호출된 `rsum(3)`은 `rsum(2)`를 호출하고, `rsum(2)`는 `rsum(1)`을 호출합니다. 마지막으로 호출된 `rsum(1)`은 더 이상 다른 함수를 호출하지 않고 종료 조건문에서 1을 반환합니다. 이 반환 값은 `rsum(2)`에 전달되고, `rsum(2)`는 이 값을 가지고 3(=1+2)을 계산한 후 `rsum(3)`으로 전달합니다. `rsum(3)`은 반환 값을 받아 6(=3+3)을 계산한 후에 최초의 호출문으로 돌려줍니다.

7) 실행 프레임은 4.3절에서 언급하는 시스템 스택의 활성화 레코드와 같은 개념입니다.

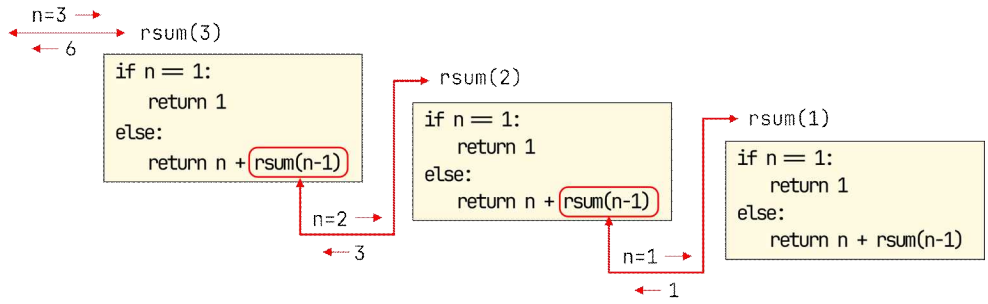


그림 4.1: 재귀 함수 rsum()의 수행 과정

이제 피보나치 수열에서 n번째 수 F(n)을 구하는 다음의 재귀 관계를 재귀 함수로 구현해 보겠습니다.

$$
\begin{aligned}
F(0) &= 0 \\
F(1) &= 1 \\
F(n) &= F(n-1) + F(n-2), \qquad n \geq 2
\end{aligned}
$$

이 재귀 관계는 두 개의 초기 조건을 가지고 있습니다. 이 두 조건을 재귀 함수의 종료 조건문으로 표현하고, 재귀 관계식을 재귀 호출문으로 구성합니다. 재귀 호출문은 재귀 관계식에서 등호 오른쪽에 있는 식을 그대로 return문에 적용하면 됩니다. 이렇게 구현한 재귀 함수가 프로그램 4.2에 나와 있습니다.

프로그램 4.2: 피보나치 수를 구하는 재귀 함수

```
1 def fib(n):
2     """n번째 피보나치 수를 반환 - 지수 시간이 소요되는 재귀 함수"""
3     if n <= 1:                          # 종료 조건
4         return n
5     else:
6         return fib(n-1) + fib(n-2)      # 재귀 호출문
```

이 재귀 함수는 재귀 호출을 두 번 수행하기 때문에 수행 과정이 다소 복잡합니다. 프로그램 4.1의 재귀 함수 rsum()의 경우는 연쇄적으로 수행되는 함수의 호출 및 반환이 선형으로 진행되는 반면, fib()의 경우는 그림 4.2에서 볼 수 있듯이 비선형으로 진행됩니다. 이것은 fib() 함수가 재귀 호출을 두 번 수행하기 때문입니다. 그림 4.2는 초기에 fib(5)가 호출될 때, fib() 함수가 재귀적으로 호출 및 반환되는 순서를 보여줍니다.

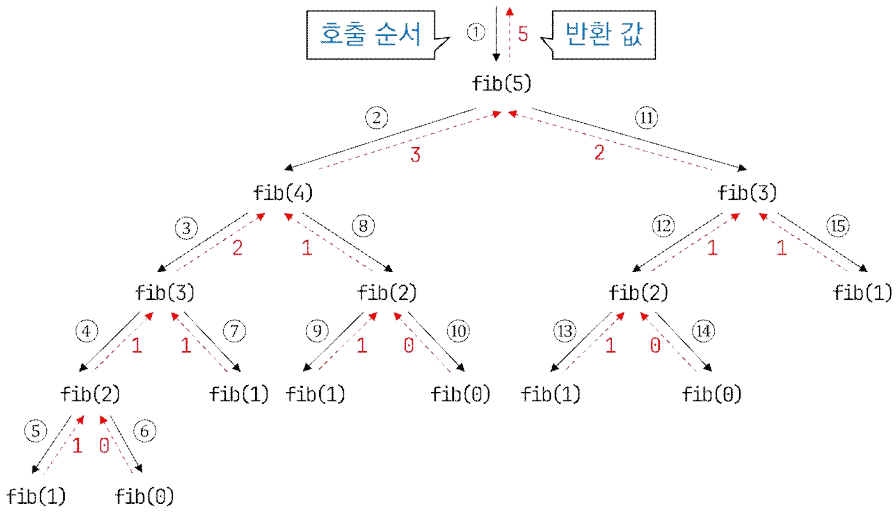


그림 4.2: 재귀 함수 `fib()`의 수행 과정

그림 4.2를 보면, 매개변수의 값이 같은 함수가 여러 곳에서 호출되는 것을 발견할 수 있습니다. 이러한 중복 호출은 동일한 작업을 독립적으로 다시 수행하게 하므로, 중복 호출이 많을수록 프로그램의 효율을 떨어뜨리게 됩니다. 상세한 분석 과정은 다음 절에서 소개하겠지만, 결과적으로 `fib()`의 시간복잡도는 지수 시간으로 매우 비효율적입니다. 이러한 비효율성은 프로그램을 반복 구조로 구현하거나 재귀 함수를 기술적으로 최적화하는 방법 또는 새로운 재귀 관계를 설계하여 해결할 수 있습니다. 프로그램 4.3에는 재귀 관계를 상향식으로 해결하는 반복적 해법을 구현한 함수 `iterfib()`가 나와 있습니다. 이 프로그램은 선형 시간에 수행됩니다.

프로그램 4.3: 피보나치 수를 구하는 반복 구조 프로그램

```
1  def iterfib(n):
2     """n번째 피보나치 수를 반환 - 선형 시간에 수행하는 반복 구조 프로그램"""
3     if n <= 1:                          # F(0) = 0, F(1) = 1
4        return n
5     curr, prev = 1, 0                   # curr = F(1), prev = F(0)
6     for i in range(2, n+1):             # F(2)부터 F(n)까지 차례로 구함
7        curr, prev = curr+prev, curr     # curr = F(i), prev = F(i-1)
8     return curr                         # F(n)
```

중복 호출이 많은 재귀 함수의 비효율성을 개선하는 방법으로 **메모이제이션(memoization)**이라고 부르는 기법이 있습니다. 이 기법은 동일한 계산을 반복해야 하는 환경에서, 이전에 계산했던 결과를 메모리에 저장해 두었다가 필요할 때마다 저장된 결과를 불러와서 사용함으로써 재

계산을 하지 않아도 되도록 해 줍니다. 프로그램 4.4에는 피보나치 수를 구하는 메모이제이션 기법을 도입한 재귀 함수 mfib(n, F)가 나와 있습니다. 이 함수는 파이썬 리스트 F에 계산된 피보나치 수를 저장합니다. 최초 호출 시에는 디폴트 인자를 가진 F를 빼고 호출합니다.

프로그램 4.4: 피보나치 수를 구하는 메모이제이션을 도입한 재귀 함수

```
1 def mfib(n, F = [0, 1]):
2     """n번째 피보나치 수를 반환 - 메모이제이션 기법으로 선형 시간에 수행하는 재귀 함수
3     """
4     if n < len(F):              # 종료 조건 - 계산 결과가 저장된 경우
5         return F[n]             # 저장된 피보나치 수 반환
6     else:
7         F.append(mfib(n-1, F) + mfib(n-2, F))  # 재귀 호출 후 계산 결과 저장
8         return F[n]                            # n번째 피보나치 수 반환
```

프로그램 4.5에는 피보나치 수열을 위한 새로운 재귀 함수 nfib()가 나와 있습니다. nfib(n)은 (F(n), F(n-1)), 즉 n 번째 피보나치 수와 n-1 번째 피보나치 수를 순서쌍으로 묶어 반환합니다. 이렇게 함으로써 재귀 호출을 한 번으로 줄일 수 있습니다. 맨 처음 nfib(5)가 호출되었을 때의 수행 과정을 그림 4.3에서 볼 수 있습니다. 프로그램의 종료 조건에서 n = 0인 경우에 (0, 0)을 반환하는 데, 이것은 (F(0), F(-1))을 의미하는 것이 아니라 단순히 기술적인 초깃값 설정이라고 보면 됩니다.

프로그램 4.5: 피보나치 수를 구하는 새로운 재귀 함수

```
1 def nfib(n):
2     """n번째와 n-1번째 피보나치 수를 반환 - 선형 시간에 수행하는 재귀 함수"""
3     if n <= 1:                  # 종료 조건
4         return (n, 0)           # (0, 0) 또는 (1, 0) 반환
5     else:
6         (a, b) = nfib(n-1)      # a = F(n-1), b = F(n-2)
7         return (a+b, a)         # (F(n), F(n-1))을 반환
```

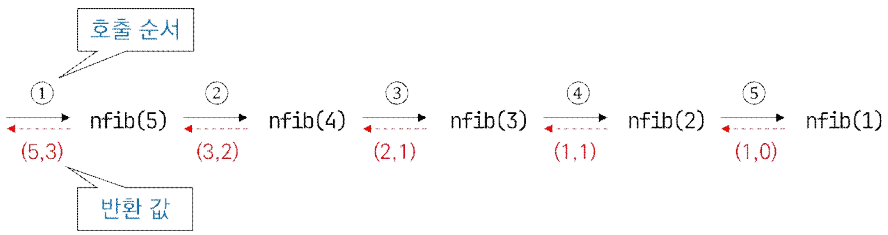


그림 4.3: 재귀 함수 nfib()의 수행 과정

다음으로, 앞 절에서 소개한 회문 검사를 수행하는 재귀 함수를 살펴보겠습니다. 검사를 위해 주어지는 문자열은 빈칸이 모두 삭제되어 있다고 가정합니다. 문자열 str의 양 끝 문자를 뺀 부문자열을 만들 때, "str[1:-1]"을 사용하면 코드는 간결하지만, 새로운 문자열이 매번 생성되어 수행 시간과 메모리 사용량에 큰 부담이 됩니다. 따라서 문자열의 시작과 끝을 가리키는 첨자를 사용하는 것이 효율적입니다. 프로그램 4.6의 함수 palindrom(str, low, high)는 문자열 str의 첨자 범위 [low, high]에 포함된 부문자열이 회문인지를 검사합니다.

프로그램 4.6: 회문 여부를 판별하는 재귀 함수

```
1  def palindrome(str, low, high):
2     """str[low:high+1]이 회문이면 True, 아니면 False를 반환하는 재귀 함수"""
3     if low >= high:                  # 종료 조건 - 문자가 없거나 하나만 있으면
4        return True                   # 회문으로 판정
5     if str[low] != str[high]:        # 양 끝 문자가 다르면
6        return False                  # 회문이 아닌 것으로 판정
7     return palindrome(str, low+1, high-1)    # 양 끝을 뺀 부문자열 검사
```

마지막으로 재귀 함수를 구현할 때 유의해야 할 사항을 소개합니다. 재귀 함수에 종료 조건문이 없거나 종료 조건에 도달하지 못하고 재귀 호출이 끝없이 수행되는 오류가 발생하는 경우가 있는데, 이를 **무한 재귀(infinite recursion)**라고 합니다. 프로그램 4.7은 무한 재귀에 빠진 프로그램의 예를 보여줍니다.

프로그램 4.7: 무한 재귀에 빠진 프로그램의 예

```
1  def wrong_rsum(n):    # 종료 조건문이 누락된 재귀 함수
2     return n + wrong_rsum(n)
3
4  def wrong(n):         # n이 짝수면 종료 조건에 도달하지 못함
5     if n == 1:
6        return 1
7     else:
8        return n + wrong(n-2)
```

무한 재귀에 빠지는 프로그램을 실행하면, 함수 정보를 가진 활성화 레코드가 메모리에 계속 누적되기 때문에, 결국 메모리 부족으로 인한 오류가 발생합니다. 프로그래밍 언어에 따라서는 컴퓨터 시스템이 다운되는 현상이 발생하기도 합니다. 파이썬 언어의 경우는 무한 재귀로 인해 발생하는 문제의 해결책으로 **최대 재귀 깊이(maximum recursion depth)**를 설정하여 연속적으로 활성화되는 함수의 최대 개수를 제한합니다. 이 깊이의 기본값은 일반적으로 1,000으로 설정되

어 있습니다. 이 한계에 도달하면 파이썬 인터프리터는 "RecursionError: maximum recursion depth exceeded"라는 런타임 오류(runtime error)를 발생시킵니다.

파이썬의 sys 모듈에 포함된 setrecursionlimit() 함수를 이용하면 최대 재귀 깊이를 변경할 수 있습니다. 프로그램 4.8은 최대 재귀 깊이를 10,000으로 변경하는 프로그램 코드를 보여줍니다.

프로그램 4.8: 최대 재귀 깊이를 변경하는 프로그램의 예

```
1 import sys
2 print(sys.getrecursionlimit())     # 현재 최대 재귀 깊이를 출력함
3 sys.setrecursionlimit(10000)      # 최대 재귀 깊이를 10000으로 변경함
```

sys.setrecursionlimit() 함수를 사용하여 재귀 깊이를 너무 크게 설정하면, 함수의 호출 정보와 지역 변수를 임시로 저장하는 메모리 영역인 시스템 스택(system stack) 공간이 고갈될 수 있습니다. 이 경우, '스택 오버플로'로 이어져서 프로그램이 강제 종료되거나 심지어 운영체제 수준에서 시스템 충돌(crash)을 유발할 수 있습니다. 시스템 스택의 크기는 운영체제마다 다르므로 최대 재귀 깊이를 늘여야 하는 경우, 신중하게 접근하는 것이 필요합니다. 참고로, 윈도우즈 운영체제에서 시스템 스택의 크기는 기본적으로 1MB(메가바이트)입니다.

4.3 재귀 함수의 성능 분석

1장에서 기본 연산의 수행 횟수를 사용하여 알고리즘의 성능을 분석하는 방법을 소개했습니다. 또한 알고리즘의 시간복잡도는 입력의 크기에 대한 함수의 형태로 나타나며, 이를 차수 표기법으로 표현한다고 했습니다. 재귀 알고리즘도 같은 방식으로 성능을 분석하지만, 재귀 호출로 인해 시스템 내부에서 함수가 자동으로 호출되는 과정까지 고려해야 합니다.

재귀 알고리즘의 특성상 수행 시간은 자연스럽게 재귀 관계로 표현됩니다. 예를 들어, 프로그램 4.1의 함수 rsum()의 수행 시간을 분석해 보겠습니다. rsum()의 코드에는 반복문이 없으므로, rsum() 함수의 실행 프레임 하나는 상수 시간에 수행됩니다. 그러므로, 전체 수행 시간은 재귀 호출이 몇 번 발생하느냐에 따라 결정됩니다. 맨 처음에 rsum(n)이 호출되면, rsum(n)은 rsum(n-1)을 호출하고, rsum(n-1)은 rsum(n-2)를 호출하는 식으로, rsum(1)이 호

출될 때까지 재귀적으로 호출됩니다. 따라서 총 n번의 함수 호출이 발생합니다. 그래서 수행 시간은 O(n)입니다. 함수 rsum()은 매우 간단해서 이렇게 분석해도 무방하지만, 다소 복잡한 재귀 알고리즘은 분석이 까다로울 수 있습니다. 재귀 알고리즘의 수행 시간이 재귀 관계로 표현된다는 것을 보이기 위해 rsum()의 경우도 재귀 관계로 표현해 보겠습니다. 맨 처음에 rsum(n)이 호출될 때 재귀적으로 호출되는 함수의 총 횟수를 T(n)이라고 하면, T(n)은 다음과 같이 표현됩니다.

$$T(1) = 1$$
$$T(n) = T(n-1) + 1, \qquad n \geq 2$$

위 재귀 관계를 풀면 폐쇄형이 다음과 같이 나옵니다.

$$T(n) = n$$

따라서 재귀 함수 rsum()의 시간복잡도는 O(n)입니다.

이제, 프로그램 4.2의 함수 fib()의 수행 시간을 분석해 봅시다. 함수 fib(n)이 호출될 때 시스템에 의해 연쇄적으로 호출되는 함수의 총 횟수는, fib(n-1)에 의해 재귀적으로 호출되는 함수의 횟수와 fib(n-2)에 의해 재귀적으로 호출되는 함수의 횟수를 더한 값에 fib(n) 자신에 해당하는 1을 더한 값과 같습니다. 따라서 호출되는 함수의 총 횟수를 T(n)이라고 하면, T(n)은 다음과 같이 표현됩니다.

$$T(0) = 1$$
$$T(1) = 1$$
$$T(n) = T(n-1) + T(n-2) + 1, \qquad n \geq 2$$

위 재귀 관계를 풀기 위해서는 다소 복잡한 과정을 거쳐야 합니다. 재귀 관계의 폐쇄형을 구하는 해법을 모르는 경우, 상한과 하한을 구하여 해당 함수의 차수를 추정해 볼 수 있습니다. 여기서는 T(n)의 정확한 차수를 계산하는 방법보다 간단한 유도 과정을 통해 상한과 하한을 구하는 방법을 제시합니다. 먼저 T(n)의 상한을 구하기 위해 T(n-2)를 더 큰 값인 T(n-1)로 치환합니다. 그러면 다음 재귀 관계를 얻을 수 있습니다.

$$T(n) = T(n-1) + T(n-2) + 1$$
$$\leq T(n-1) + T(n-1) + 1 = 2T(n-1) + 1$$

다음 재귀 관계 U(n)의 폐쇄형은 2^n-1입니다.[8)]

$$U(1) = 1$$
$$U(n) = 2U(n-1) + 1, \qquad n \geq 2$$

$T(n) \leq U(n)$이므로 $T(n) = O(2^n)$으로 나타낼 수 있습니다. 이제 $T(n)$의 하한을 구하기 위해 $T(n-1)$을 더 작은 값인 $T(n-2)$로 치환합니다. 그러면 다음 재귀 관계를 얻을 수 있습니다.

$$T(n) \geq T(n-2) + T(n-2) + 1 = 2T(n-2) + 1$$

다음의 재귀 관계 $L(n)$의 폐쇄형은 $2 \cdot 2^{n/2}-1$입니다.

$$L(0) = 1$$
$$L(1) = 1$$
$$L(n) = 2L(n-2) + 1, \qquad n \geq 2$$

$T(n) \geq L(n)$이므로 $T(n) = \Omega(2^{n/2})$으로 나타낼 수 있습니다. 따라서 재귀 함수 fib()의 시간복잡도는 $\Omega(2^{n/2})$이고 동시에 $O(2^n)$입니다. 그러므로 fib()의 시간복잡도는 지수 시간임을 알 수 있습니다[9].

프로그램 4.4의 재귀 함수 mfib(n, F)는 메모이제이션을 사용하므로, 각 피보나치 수는 단 한 번만 계산됩니다. 맨 처음 k를 가지고 mfib(k, F)가 호출되면, 재귀를 통해 k 번째 피보나치 수를 구하고 이 값을 F[k]에 저장합니다. 이후 mfib(k, F)가 다시 호출되면, 상수 시간에 F[k]를 반환합니다. 따라서 F[0]부터 F[n]까지 모든 피보나치 수를 계산하는 데 세 번 이상의 중복 호출은 발생하지 않으므로, 소요되는 총 수행 시간은 $O(n)$입니다. 프로그램 4.5의 재귀 함수 nfib(n)에는 한 번의 재귀 호출만 있으므로, 앞의 rsum() 함수와 마찬가지로 시간복잡도는 $O(n)$입니다.

다음으로, 프로그램 4.6의 회문 여부를 판별하는 재귀 함수 palindrome()의 시간복잡도를 분석해 봅니다. palindrome(str, low, high)에 의해 재귀적으로 호출되는 함수의 총 횟수를 $T(n)$이라고 합시다. 여기서 n = high-low+1. 최악의 경우 시간복잡도는 n이 0 또는 1이 될 때까지 재귀적으로 계속 수행하는 경우 발생합니다. 따라서 최악의 경우에 $T(n)$은 다음 재귀 관계로 표현됩니다.

---

8) 재귀 관계 $T(n)$은 비교적 간단한 유형이어서 폐쇄형을 쉽게 구할 수 있지만, 이 책에서는 폐쇄형 유도 과정에 대한 설명은 생략합니다. 유도 과정에 관심이 있는 독자는 이산 수학 관련 책이나 인터넷을 검색해 보세요.

9) 참고로 $T(n)$의 폐쇄형은 $\Theta(c^n)$입니다. 여기서 $c = (1+\sqrt{5})/2$.

$$T(0) = 1$$
$$T(1) = 1$$
$$T(n) = T(n-2) + 1, \quad n \geq 2$$

재귀 관계를 풀면, $T(n) = \lfloor n/2 \rfloor + 1$입니다. 따라서 시간복잡도는 $O(n)$입니다.

이제, 재귀 함수의 공간복잡도를 분석하는 방법에 대해 알아봅니다. 재귀 함수의 공간복잡도를 분석할 때는 함수 호출과 반환을 처리하기 위한 메모리 공간을 고려해야 합니다. 단일 함수가 실행하는 데 필요한 정보인 매개변수, 지역 변수, 복귀주소 등은 **활성화 레코드(activation record)**라고 불리는 시스템 자료구조에 포함됩니다. 활성화 레코드는 컴퓨터 시스템이 관리하는 특수한 메모리 영역인 **시스템 스택(system stack)**에 저장되며, 함수가 호출될 때 생성하고 함수가 반환되면 소멸합니다. 재귀 함수의 수행에서 종료 조건에 도달하기 전까지 시스템 스택에 동시에 적재되는 활성화 레코드의 최대 개수가 재귀 함수의 공간복잡도를 결정하는 요인이 됩니다.

프로그램 4.1의 함수 rsum()의 코드만 보면 상수 크기의 메모리 공간을 사용합니다. 따라서 재귀 호출로 인하여 누적되는 활성화 레코드의 최대 개수가 공간복잡도를 좌우합니다. 맨 처음에 rsum(n)이 호출되면, rsum(n)은 rsum(n-1)을 호출하고, rsum(n-1)은 rsum(n-2)를 호출하는 식으로 rsum(1)이 호출되어 반환을 시작할 때까지 생성된 활성화 레코드가 시스템 스택에 계속 쌓이게 됩니다. 그래서 시스템 스택에는 최대 n개의 활성화 레코드가 적재됩니다. rsum()의 활성화 레코드는 상수 크기의 메모리 공간을 사용하므로, 시스템 스택에서 활성화 레코드가 차지하고 있는 메모리 공간의 최대 크기, 즉 공간복잡도는 $O(n)$입니다. 재귀 함수의 공간복잡도도 재귀 관계로 표현될 수 있습니다. 맨 처음에 rsum(n)이 호출될 때 시스템 스택에 적재되는 활성화 레코드의 최대 개수를 C(n)이라고 하면, rsum(n)은 rsum(n-1)을 호출하므로 C(n)은 C(n-1)에 rsum(n) 자신의 것을 합한 것과 같습니다. 즉 다음과 같은 재귀 관계로 표현됩니다.

$$C(1) = 1$$
$$C(n) = C(n-1) + 1, \quad n \geq 2$$

위 순환 관계를 풀면 C(n) = n이므로 공간복잡도는 $O(n)$입니다.

이제, 프로그램 4.2의 함수 fib()의 공간복잡도를 분석해 봅시다. fib(n)이 호출될 때, 시스템 스택에 적재되는 활성화 레코드의 최대 개수를 C(n)이라고 합시다. fib(n)은 fib(n-1)과 fib(n-2)를 호출하지만 동시에 호출하는 것이 아니라 fib(n-1)의 수행이 끝나고 반환된 후에야 fib(n-2)가 호출됩니다. 따라서 C(n)은 C(n-1)과 C(n-2) 중의 큰 값에 1을

더한 것과 같습니다. C(n-1)이 C(n-2)보다 같거나 큰 값이 됨은 분명하므로 C(n)은 다음과 같습니다.

```
C(0) = 1
C(1) = 1
C(n) = max{C(n-1), C(n-2)} + 1,       n ≥ 2
     = C(n-1) + 1
```

위 순환 관계를 풀면 C(n) = n이고, 활성화 레코드의 크기는 $O(1)$이므로 공간복잡도는 $O(n)$입니다.

마지막으로 효율성 관점에서 재귀 함수와 반복 구조 프로그램을 비교해 보겠습니다. 같은 알고리즘을 구현하는 경우, 재귀 함수보다 더 효율적인 반복 구조 프로그램을 항상 고안할 수 있습니다. 재귀 함수는 함수 호출과 반환을 기반으로 수행되므로 시간적, 공간적 측면에서 오버헤드(overhead)가 발생합니다. 컴퓨터 시스템은 함수 호출이 발생하면 현재 함수의 정보와 상태를 저장하고 새로 호출된 함수를 위한 실행 환경을 설정해야 합니다. 반대로 함수가 반환되면 해당 함수의 스택 프레임을 제거하고 이전 함수 상태로 복원해야 합니다. 반복 구조 프로그램은 이러한 오버헤드를 가지지 않으므로 그만큼 더 빠르게 수행됩니다. 시간적 측면에서 볼 때, 시간복잡도의 차수가 같다고 하더라도 반복 구조 프로그램은 차수에 곱해지는 상수 인자에서 더 작은 값을 가집니다. 프로그램 4.2의 fib()처럼 중복 호출을 많이 발생시키는 재귀 함수의 경우에는 시간복잡도의 차수까지 큰 차이가 날 수 있습니다. 공간적인 측면에서 볼 때, 재귀 함수는 시스템 스택의 공간까지 고려해야 하므로 반복 구조 프로그램이 항상 더 효율적입니다. 더욱이 반복 구조 프로그램은 재귀 깊이에 대한 제한 사항을 고려하지 않아도 됩니다.

재귀 함수는 효율성에서 분명한 단점을 갖고 있음에도 불구하고, 단점을 상쇄하고도 남을 뚜렷한 장점도 가지고 있습니다. 수학적 해법인 재귀 관계의 구조를 그대로 프로그램으로 구현할 수 있어 코딩이 간편하다는 점, 그리고 그 결과 알고리즘이 이해하기 쉽고 알고리즘의 정확성을 증명하기도 용이하다는 장점입니다. 나중에 소개하는 하노이 탑 문제에서 보듯이, 반복 구조로 구현하기에는 너무 복잡하지만, 재귀 함수로는 매우 간결하게 구현되는 경우가 많습니다. 따라서, 동일한 차수의 시간복잡도를 가지면서 시스템 스택의 메모리 문제가 발생하지 않으면, 복잡한 반복 구조보다는 재귀 함수를 사용한 프로그램 구현이 코드의 가독성과 유지보수성 측면에서 더 권장되기도 합니다.

4.4 사례 연구: 하노이 탑

이 절에서는 재귀 알고리즘의 대표적인 사례로 널리 알려진 **하노이 탑(Towers of Hanoi)** 문제를 살펴봅니다. 하노이 탑 문제는 세 개의 막대 기둥과 크기가 서로 다른 원반을 이용하여 특정 규칙에 따라 원반을 옮기는 퍼즐 문제입니다.

■ 하노이 탑 문제

세 개의 막대 기둥 A, B, C와 크기가 서로 다른 n개의 원반이 주어져 있다. 처음에 모든 원반은 기둥 A에 꽂혀있는데, 가장 큰 원반이 맨 아래에 있고 위로 올라갈수록 원반의 크기가 작다. 목표는 다음의 세 규칙을 사용하여 모든 원반을 기둥 B로 옮기는 것이다.

1. 한 번에 하나의 원반만 옮길 수 있다.
2. 기둥의 맨 위에 있는 원반부터 옮길 수 있다.
3. 기둥에서 큰 원반이 작은 원반 위에 놓일 수 없다.

목표를 달성하기 위해, 원반을 어떤 순서로 옮겨야 하는지 그 순서를 나열하시오.

그림 4.4는 하노이 탑 문제에서 세 개의 기둥 A, B, C와 기둥 A에 4개의 원반이 꽂혀있는 모습을 보여줍니다.

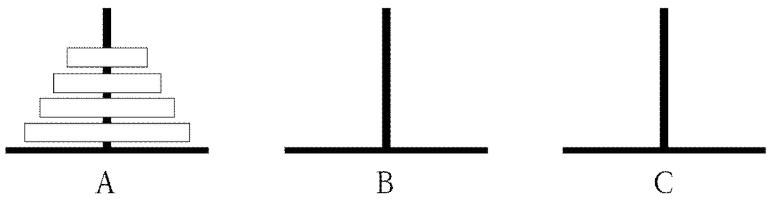


그림 4.4: 4개의 원반을 옮겨야 하는 하노이 탑 문제

n = 1이면, 단지 한 번의 이동만 해 주면 됩니다. 즉, 기둥 A에 있는 원반을 기둥 B로 옮기는 것입니다. 이 이동을 다음과 같이 표현한다고 합시다.

```
A -> B
```

n = 2일 때의 이동 과정은 다음과 같습니다. 기둥 A의 위에 놓인 원반을 일단 C로 옮기고, A에 남아 있는 원반을 B로 옮깁니다. 마지막으로 C의 원반을 B로 옮겨줍니다. 이 세 번의 이동으로

두 개의 원반을 목표 기둥인 B로 옮길 수 있습니다. 이 이동 순서는 다음과 같이 표현됩니다.

```
A -> C
A -> B
C -> B
```

n = 3인 경우에는 다음과 같이 총 7번의 원반 이동을 통해 목표 기둥으로 원반들을 모두 옮길 수 있습니다.

```
A -> B
A -> C
B -> C
A -> B
C -> A
C -> B
A -> B
```

총 7번의 이동 과정은 기둥 A에 위쪽 원반 두 개를 C로 옮기고(3번), A에 남은 가장 큰 원반을 B로 옮긴 후(1번), C에 있는 원반 두 개를 다시 B로 옮기는(3번) 과정으로 명확하게 나눌 수 있습니다. 즉 3개의 원반을 옮겨야 하는 문제의 해에는 비록 출발지와 목적지가 다르지만, 2개의 원반을 옮겨야 하는 문제의 해가 두 번 포함되어 있음을 볼 수 있습니다(그림 4.5 참조).

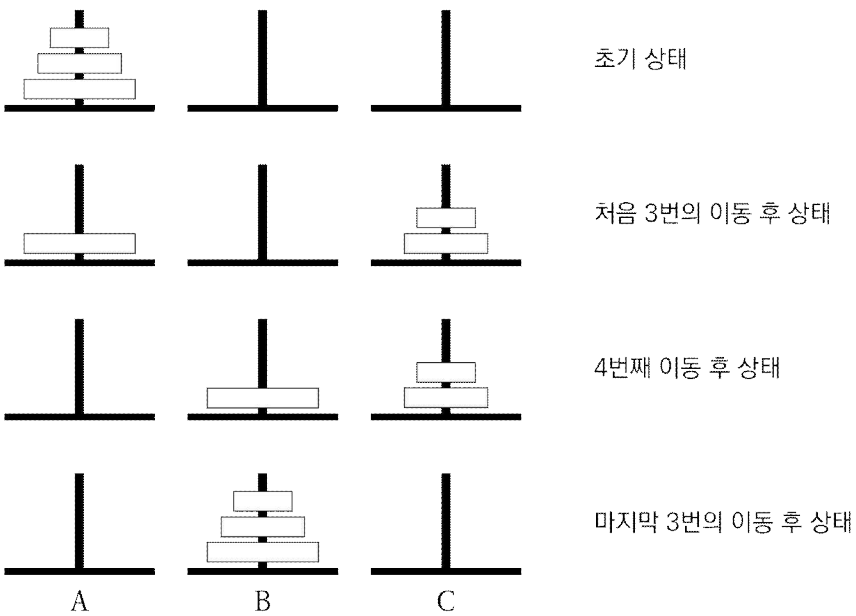


그림 4.5: 3개의 원반을 목표 기둥으로 옮기는 과정

이 사실을 이용하여 일반적인 하노이 탑 문제를 재귀적으로 해결하는 해법을 도출할 수 있습니다. 기둥 A에 n개의 원반이 꽂혀있을 때, A의 맨 밑에 있는 원반을 B로 옮기기 위해서는 미리 위쪽 n-1개의 원반을 C로 옮겨 놓아야 합니다. 그다음 가장 큰 원반을 B로 옮기고, 마지막으로 C에 있는 n-1개의 원반을 B로 옮기면 문제가 해결됩니다. 이 과정을 다음과 같이 정리할 수 있습니다.

1. 기둥 A에 있는 n-1개의 원반을 기둥 C로 옮긴다.
2. 기둥 A에 남아 있는 원반을 기둥 B로 옮긴다.
3. 기둥 C에 있는 n-1개의 원반을 기둥 B로 옮긴다.

결과적으로 n개의 원반을 옮겨야 하는 문제는, n-1개의 원반을 옮기는 문제의 해법을 두 번 적용하면 해결됩니다. 이렇게 주어진 문제를 문제 크기가 더 작은 문제의 해를 이용하여 해결하는 것은 재귀 관계의 일반적인 형태이고, 이를 재귀 알고리즘이라고 부른다고 하였습니다. 하노이 탑 문제를 위한 재귀 함수는 프로그램 4.9에 나와 있습니다. 함수 hanoi(n, src, dest, other)는 출발지 기둥 src에 꽂혀있는 n개의 원반을 목적지 기둥 dest로 기둥 other를 이용하여 옮기기 위한 원반 이동 순서를 출력합니다.

프로그램 4.9: 하노이 탑 문제를 위한 재귀 함수

```
1 def hanoi(n, src, dest, other):
2     """n개의 원반을 src에서 dest로 옮기는 절차를 출력한다."""
3     if n == 1:                          # 종료 조건
4         print(src, "->", dest)
5     else:
6         hanoi(n-1, src, other, dest)    # src의 n-1개 원반을 other로 옮김
7         print(src, "->", dest)          # src에 남아 있는 원반을 dest로 옮김
8         hanoi(n-1, other, dest, src)    # other의 n-1개 원반을 dest로 옮김
```

이제, hanoi() 함수의 시간복잡도를 분석해 봅니다. hanoi()의 시간복잡도는 피보나치 수를 구하는 재귀 함수 fib()와 비슷한 방식으로 구할 수 있습니다. 맨 처음에 원반의 개수가 n인 hanoi()가 호출될 때, 재귀적으로 호출되는 함수의 총 횟수 T(n)을 다음과 같은 재귀 관계로 표현할 수 있습니다.

$$T(1) = 1$$
$$T(n)) = 2T(n-1) + 1, \qquad n \geq 2$$

이 재귀 관계의 폐쇄형은 다음과 같습니다.

$$T(n) = 2^n-1$$

따라서 hanoi() 함수의 시간복잡도는 $O(2^n)$입니다. 함수가 호출될 때마다 원반 하나를 옮기는 작업이 수행되므로, 총 원반 이동 횟수는 2^n-1입니다. 그러므로 원반의 수가 늘어남에 따라 원반 이동 횟수는 기하급수적으로 증가한다는 것을 알 수 있습니다.

hanoi() 함수의 공간복잡도도 fib() 함수와 유사하게 분석할 수 있습니다. 단, hanoi() 함수는 fib() 함수와 달리 같은 크기를 가진 두 개의 함수를 호출합니다. 따라서 맨 처음에 원반의 개수가 n인 hanoi()가 호출될 때 시스템 스택에 적재되는 활성화 레코드의 최대 개수를 C(n)이라고 하면, C(n)은 다음과 같은 재귀 관계로 표현됩니다.

$$C(1) = 1$$
$$C(n) = C(n-1) + 1, \qquad n \geq 2$$

위 순환 관계를 풀면 C(n) = n이고, 활성화 레코드의 크기는 상수 크기이므로 공간복잡도는 $O(n)$입니다.

4.5 사례 연구: 이진 탐색

이 절에서는 정렬된 선형리스트에서 특정한 값을 가진 요소를 탐색하는 문제를 다룹니다. 파이썬의 순서 자료형 객체에서 탐색을 수행하는 연산자 in이나 메소드 index()는 순차적으로 탐색을 수행합니다. 따라서 최악의 경우 시간복잡도는 1차 함수, 즉 선형 시간에 수행됩니다. 요소들이 정렬된 상태로 배치되어 있다고 하더라도 이 시간복잡도는 바뀌지 않습니다. 이처럼 선형리스트의 요소들을 처음부터 끝까지 차례대로 하나씩 검사하는 방식을 **순차 탐색(sequential search)**이라고 합니다. 정렬된 상태가 아닌 선형리스트에서 탐색을 수행하려면 순차 탐색을 수행하는 것이 필수적입니다. 순차 탐색은 최악의 경우 모든 요소를 검사해야 하므로, 최악의 경우 시간복잡도는 선형 시간입니다.

선형리스트가 정렬되어 있고 배열 기반으로 표현되어 있다면, 순차 탐색보다 훨씬 효율적인 **이진 탐색(binary search)** 알고리즘을 활용할 수 있습니다. 이진 탐색은 비교적 간단하지만 가장 중요한 컴퓨터 알고리즘 중 하나로서, 효율적인 알고리즘을 설계할 때 핵심적인 역할을 담당하는 경우가 많습니다. 만약 선형리스트가 연결리스트로 표현된다면, 정렬된 상태라고 하더라도 이진 탐

색을 이용할 수 없습니다. 이진 탐색의 핵심은 특정 위치의 원소를 상수 시간에 접근할 수 있느냐에 달려있기 때문입니다.

이진 탐색 알고리즘은 우리가 국어사전 또는 영어사전에서 특정 단어를 찾는 방법과 유사합니다. 우리가 단어를 찾을 때 사전의 첫 페이지부터 일일이 살펴보지 않습니다. 단어가 있을 만한 페이지를 단번에 펼쳐보고, 단어가 그 페이지에 없으면 같은 방식으로 앞쪽이나 뒤쪽을 찾아봅니다. 이진 탐색 알고리즘을 설명하기에 앞서, 앞으로 배열 기반의 선형리스트를 편의상 '배열'이라고 부르고, 배열에서 찾고자 하는 값을 **탐색값(search value)**이라고 부르겠습니다. 이진 탐색은 배열에서 탐색 구간을 지정하는데, 탐색 구간에서 시작 위치와 끝 위치에 해당하는 배열 첨자를 각각 `low`와 `high`라고 합시다. 배열에 `n`개의 원소가 저장되어 있고 첨자가 0부터 시작한다면, 탐색을 시작할 때 `low`는 0, `high`는 `n-1`이 됩니다. 첫 번째 검사는 탐색 구간의 중앙 위치에 있는 원소와 탐색값을 비교하는 것입니다. 물론 탐색 구간에 원소가 하나도 없으면 탐색을 중단합니다. 그렇지 않으면, 중앙 위치 `mid`는 다음과 같이 계산됩니다.

$$mid = \lfloor (low + high) / 2 \rfloor$$

이 `mid` 위치에 있는 원소와 탐색값을 비교한 결과는 '같다', '작다', '크다' 중 하나가 되므로, 각 경우에 대해 다음과 같이 수행합니다.

1. `mid` 위치에 탐색값이 있으면 탐색 작업을 성공적으로 종료한다.
2. 탐색값이 `mid` 위치의 원소보다 작으면, 탐색 구간의 끝을 `mid-1`로 바꾸고 다시 검사한다.
3. 탐색값이 `mid` 위치의 원소보다 크면, 탐색 구간의 시작을 `mid+1`로 바꾸고 다시 검사한다.

위의 두 번째와 세 번째 경우, 탐색 구간은 절반으로 줄어듭니다. 즉, 한 번의 검사만으로 이진 탐색은 탐색 구간을 절반으로 줄입니다. 순차 탐색이 한 번의 검사로 탐색 구간의 크기를 하나 줄이는 것과 비교해 보면, 이진 탐색이 매우 효율적으로 작업을 수행한다는 것을 알 수 있습니다. 위에 표현된 탐색 작업은 재귀 관계의 일반적인 형태입니다. 초기 조건은 탐색 구간의 크기가 0인 경우입니다. 초기 조건이 아니면 유형별로 탐색을 종료하거나 탐색 구간을 축소하여 다시 작업을 시작합니다. 그림 4.6과 3.7에는 이진 탐색을 수행하는 과정이 나와 있습니다. 그림 4.6은 리스트에 탐색값이 존재하여 탐색이 성공하는 사례를 보여주고, 그림 4.7은 탐색이 실패하는 사례를 보여줍니다.

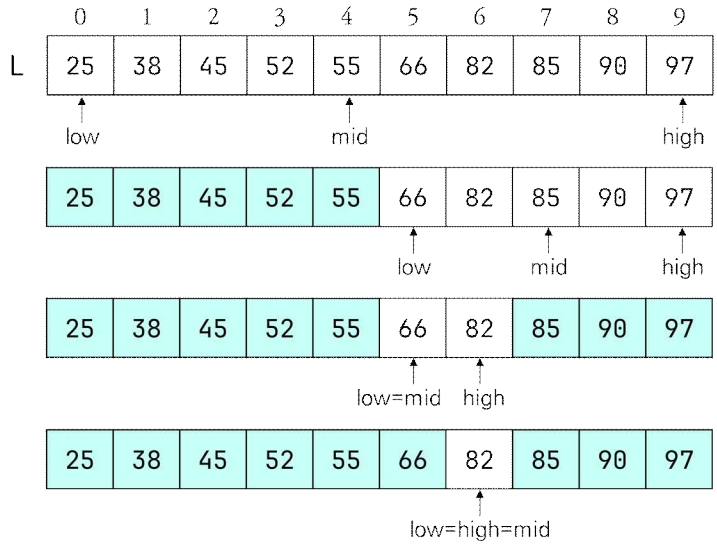


그림 4.6: 리스트 L에서 탐색값 82를 찾는 이진 탐색의 수행 예 (탐색 성공)



그림 4.7: 리스트 L에서 탐색값 50을 찾는 이진 탐색의 수행 예 (탐색 실패)

프로그램 4.10에 이진 탐색을 재귀 함수로 구현한 함수 bin_search(L, low, high, x)가 나와 있습니다. 이 함수는 리스트 L의 첨자 범위 [low, high] 구간에서 탐색값 x의 위치(첨자)를 반환하고, 탐색에 실패하면 None을 반환합니다. 맨 처음 호출 시에는 low에는 0, high에는 len(L)-1이 배정됩니다.

프로그램 4.10: 이진 탐색을 수행하는 재귀 함수

```
1  def bin_search(L, low, high, x):
2     """L에서 탐색값 x의 위치(첨자)를, 탐색에 실패하면 None을 반환한다."""
3     if low > high:                          # 종료 조건 - 공백인 탐색 구간
4        return None                          # 탐색 실패
5     else:
6        mid = (low + high) // 2              # 중앙 위치 계산
7        if x == L[mid]:                      # 탐색 성공
8           return mid
9        elif x < L[mid]:                     # 탐색값이 중앙값보다 작은 경우
10          return bin_search(L, low, mid-1, x)      # 왼쪽 구역 탐색
11       else:
12          return bin_search(L, mid+1, high, x)     # 오른쪽 구역 탐색
```

물론, 이진 탐색을 반복 구조로 구현할 수 있습니다. 프로그램 4.11에 반복 구조로 구현된 함수 bin_search_iter(L, x)가 나와 있습니다. 이 함수는 탐색 구간을 나타내는 변수 low와 high가 함수 내부에서 생성됩니다.

프로그램 4.11: 이진 탐색을 수행하는 반복 구조 프로그램

```
1  def bin_search_iter(L, x):
2     """L에서 탐색값 x의 위치(첨자)를, 탐색에 실패하면 None을 반환한다."""
3     low, high = 0, len(L)-1          # 탐색 구간의 초깃값 지정
4     while low <= high:               # 탐색 구간이 공백이 아니면 반복함
5        mid = (low + high) // 2       # 중앙 위치 계산
6        if x == L[mid]:               # 탐색 성공
7           return mid
8        elif x < L[mid]:              # 탐색값이 중앙값보다 작은 경우
9           high = mid-1               # 왼쪽 구역 탐색
10       else:
11          low = mid+1                # 오른쪽 구역 탐색
12    return None                      # 탐색 실패
```

이제 이진 탐색 알고리즘의 시간복잡도를 분석해 보겠습니다. 이진 탐색은 한 번의 비교 연산으로 탐색 구간을 절반으로 줄입니다. 따라서 최초로 호출된 프로그램 4.10의 bin_search() 함수에 의해 발생하는 함수 호출의 총 횟수를 T(n)이라고 하면, 재귀 관계 T(n)은 다음과 같습니다. 여기서 n은 리스트의 길이입니다.

$$T(1) = 1$$
$$T(n) = T(\lfloor n/2 \rfloor) + 1, \qquad n \geq 2$$

재귀 관계를 풀면, $T(n) = \lfloor \log n \rfloor + 1$입니다. 따라서 시간복잡도는 $O(\log n)$입니다. 프로그램 4.11의 bin_search_iter() 함수에서는 while 문의 반복 횟수가 시간복잡도의 차수를 결정하는 요인이 됩니다. 이 반복 횟수와 bin_search()의 재귀 호출 횟수는 동일하므로, bin_search_iter() 함수도 역시 로그 시간에 수행됩니다. 선형 함수와 로그 함수의 값을 비교한 1장의 표 1.2에서 보았듯이 로그 함수는 선형 함수에 비해 매우 느리게 증가하므로, 순차 탐색과 이진 탐색은 효율성 면에서 비교할 수 없을 정도로 차이가 큽니다. 따라서 정렬된 리스트에서의 탐색 작업은 이진 탐색이 필수적임을 알 수 있습니다.

다음으로 이진 탐색 알고리즘의 공간복잡도를 분석해 봅시다. 입력 데이터가 저장된 리스트 L은 제외하고 수행에 필요한 메모리 공간의 크기를 고려해 봅니다. 반복 구조 프로그램인 bin_search_iter()는 리스트 low, high, mid 등 몇 개의 변수만 사용하므로 공간복잡도는 $O(1)$입니다. 반면에 재귀 함수 bin_search()는 함수 호출마다 시스템 스택에 활성화 레코드가 쌓이고, 재귀 깊이는 log n에 비례하므로 공간복잡도는 $O(\log n)$입니다. 이러한 차이로 인해 실용적인 측면에서는 재귀 함수보다 반복 구조 프로그램이 더 선호됩니다.

연습문제

1. 다음의 급수에서 n번째 항까지의 합을 구하는 재귀 관계를 쓰시오.

$$1 + \frac{1}{2} - \frac{1}{3} + \frac{1}{4} - \frac{1}{5} + \cdots$$

2. **아커만 함수(Ackerman's function)**는 다음과 같이 정의되는 재귀 관계로서 함수의 증가율이 엄청나게 높다. 다음의 물음에 답하시오.

```
A(m, n) = n+1,                  if m = 0
A(m, n) = A(m-1, 1)             if m > 0, n = 0
A(m, n) = A(m-1, A(m, n-1))     if m > 0, n > 0
```

   (1) A(1, 2)의 값을 구하시오.
   (2) A(2, 2)의 값을 구하시오.

3. 양의 정수 n을 입력받아 n!을 출력하는 재귀 함수 factorial(n)을 쓰시오.

```
n! = n × (n-1) × ··· × 1
```

4. 정수로 구성된 파이썬 리스트 L의 모든 요소를 합산하여 반환하는 재귀 함수 rec_sum(L)을 쓰시오.

5. n개에서 k개를 뽑는 조합의 수 C(n, k)는 다음과 같이 재귀 관계로 나타낼 수 있다.

```
C(n, 0) = 1
C(n, n) = 1
C(n, k) = C(n-1, k-1) + C(n-1, k)
```

   C(n, k)의 값을 출력하는 재귀 함수를 작성하시오. 단, 중복 호출을 방지하기 위한 메모이제이션 기법을 사용하라.

6. x^n을 구하는 재귀 함수 power(x, n)의 시간복잡도와 공간복잡도를 구하시오.

```
def power(x, n):
   if  n == 1:
      return x
   temp = power(x, n//2)
   if  n % 2 == 0:
      return  temp * temp
   else:
      return  temp * temp * x
```

7. 파이썬은 메모이제이션을 시스템 차원에서 지원하는 도구를 제공한다. 모듈 functools의 lru_cache 데코레이터를 함수 정의 위에 추가하는 것만으로 기존의 코드를 바꾸지 않더라도 파이썬이 자동으로 함수의 호출 결과를 캐싱해 줌으로써 메모이제이션처럼 수행하게 해준다. 다음 프로그램을 직접 실행하여, 데코레이션이 없는 프로그램과 실행 시간을 비교해 보시오.

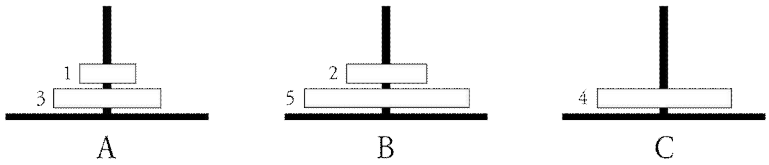
```
import functools

@functools.lru_cache
def fib(n):
   """n번째 피보나치 수를 반환 - 지수 시간이 소요되는 재귀 함수"""
   if n <= 1:
      return n
   else:
      return fib(n-1) + fib(n-2)
```

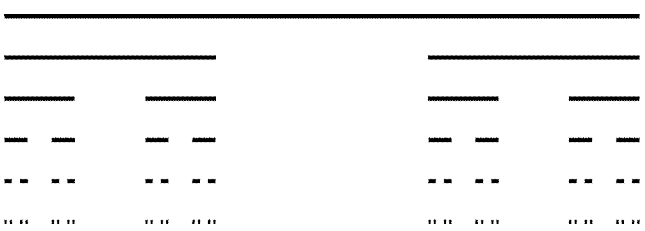
8. 회문(palindrome) 여부를 판별하는 다음 재귀 함수 pal()의 시간복잡도와 공간복잡도를 분석하고, 프로그램 4.6의 함수와 비교해 보시오.

```
def pal(str):
   """str이 회문이면 True, 아니면 False를 반환하는 재귀 함수"""
   if len(str) <= 1:        # 종료 조건 - 문자가 없거나 하나만 있으면
      return True           # 회문으로 판정
   if str[0] != str[-1]:    # 양 끝 문자가 다르면
      return False          # 회문이 아닌 것으로 판정
   return pal(str[1:-1])    # 양 끝을 뺀 부문자열 검사
```

9. 누군가 하노이 탑 문제를 풀다가 아래 그림처럼 중간 단계에서 포기하였다. 현재 상태에서 하노이 탑 문제를 해결하는 재귀 관계를 구하시오. 단, n개의 원반은 작은 것부터 차례로 1번부터 n번까지 번호로 구분한다. 각 원반이 꽂힌 기둥은 $P = (p_1, p_2, \cdots, p_n)$으로 표현된다. 여기서 p_i는 원반 i가 꽂혀있는 기둥이다. 그림에서 P = (A, B, A, C, B)이다.



10. **프랙탈(fractal)** 도형의 일종인 **칸토어 집합(Cantor set)**은 아래 그림처럼 선분을 3 등분하여 가운데 선분을 제거하는 작업을 무한히 반복하는 과정에서 만들어진다. 길이가 L인 선분에 대해 이 과정을 n번 반복하여 그리는 절차를 재귀적으로 서술하시오. (코드를 작성할 필요는 없음)



11. **바이토닉 수열(bitonic sequence)**은 숫자들이 처음에는 증가하다가 어느 지점부터는 감소하는 수열이다. 다음은 바이토닉 수열의 예이다.

[1, 3, 5, 7, 5, 3, 1]	(7까지 증가 후 감소)
[1, 2, 3, 4]	(계속 증가)
[4, 3, 2, 1]	(계속 감소)

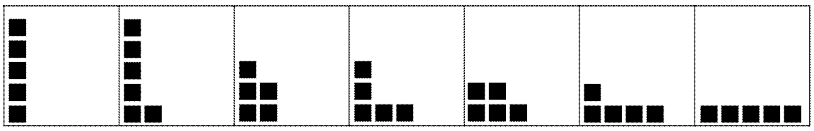
바이토닉 수열이 주어질 때, 이진 탐색을 변형하여 $O(\log n)$ 시간에 탐색값을 찾는 알고리즘을 설계하시오.

12. 주어진 집합에 대해서 **멱집합(power set)**은 모든 부분집합의 집합을 말한다. 예를 들어, 집합 {a, b, c}의 멱집합은 {{}, {a}, {b}, {c}, {a, b}, {a, c}, {b, c}, {a, b, c}}이다. 주어진 집합에 대한 멱집합을 출력하는 재귀 함수를 쓰시오.

13. 중복 값이 포함된 n개의 요소를 가진 정렬된 리스트 L에서 탐색값 x를 찾으려고 한다. 리스트 L에서 x가 처음 나타나는 위치(첨자)와 마지막으로 나타나는 위치(첨자)를 구하는 알고리즘을 설계하시오. 단, 알고리즘은 $O(\log n)$ 시간에 수행해야 한다.

14. 13번 문제에서 리스트 L에 x가 나타나는 개수를 구하는 알고리즘을 설계하시오. 단, 알고리즘은 $O(\log n)$ 시간에 수행해야 한다.

15. [실습 문제] 같은 크기의 정육면체 블록이 n개 주어져 있다. 블록들을 쌓아서 몇 개의 탑을 만들어 일렬로 나열하려고 한다. 이때 두 개 이상의 탑이 있으면, 반드시 오른쪽 탑의 높이는 왼쪽 탑의 높이보다 크지 않아야 한다. 예를 들어 n이 5이면, 아래 그림처럼 7가지 방법으로 블록 탑 그룹을 구성할 수 있다. 그림에서 가장 왼쪽은 높이가 5인 하나의 블록 탑만 만든 것이고, 가장 오른쪽은 높이가 1인 5개의 블록 탑을 만든 것이다. 블록의 개수 n이 주어질 때, 위의 방식으로 블록 탑 그룹을 구성하는 모든 방법의 수를 알아내는 프로그램을 작성하시오.



입력으로, 첫째 줄에 블록의 개수를 나타내는 정수 n이 주어진다. n은 1 이상 30 이하이다. 출력으로, n개의 블록을 사용하여 블록 탑 그룹을 구성하는 모든 방법의 수를 출력한다.

입력의 예	출력의 예
2	2

입력의 예	출력의 예
5	7

16. [실습 문제] 서로 다른 문자로 구성된 길이가 n인 문자열의 각 문자를 임의의 순서로 섞으면, n! 개의 문자열이 만들어진다. 예를 들어, 문자열 "dog"가 주어지면 총 6개의 문자열이 만들어지고, 이 문자열을 사전식 순서로 나열하면 다음과 같다. 여기서 문자열 "dgo"는 1번째, "dog"는 2번째, "gdo"는 3번째 문자열이다.

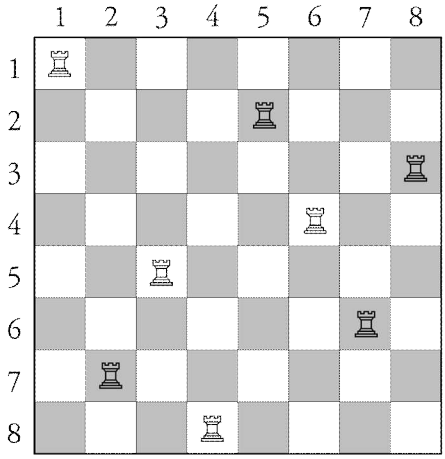
    dgo, dog, gdo, god, odg, ogd

    문자열 str과 정수 k가 주어질 때, str의 문자들로 만들어지는 문자열 중에서 사전식 순서로 k-번째인 문자열을 출력하는 프로그램을 작성하시오. 입력으로, 첫째 줄에 길이가 1 이상 10 이하이고 서로 다른 알파벳 소문자로 구성된 문자열 str이 주어지고, 둘째 줄에는 1 이상 10 이하의 정수 k가 주어진다. 출력으로, 문제에서 요구하는 k-번째 문자열을 출력한다.

입력의 예	출력의 예
dog 3	gdo

입력의 예	출력의 예
dog 6	ogd

17. [실습 문제] n-여왕 문제는 n×n 체스판에 n개의 여왕 말을 배치하는 문제이다. 이때, 어느 두 여왕 말도 같은 행, 같은 열, 같은 대각선에 놓여 있지 않아야 한다는 규칙을 지켜야 한다. 아래 그림은 8-여왕 문제에서 규칙을 지키면서 여왕 말을 배치한 체스판을 보여준다.



위 그림에서 8개의 여왕 말이 놓인 위치는 다음과 같다. 여기서 (i, j)는 i 행, j 열에 여왕 말이 배치된 것을 의미한다.

(1, 1), (2, 5), (3, 8), (4, 6), (5, 3), (6, 7), (7, 2), (8, 4)

그런데, 다음 위치에 여왕 말이 배치되어도 규칙을 만족한다.

(1, 4), (2, 1), (3, 5), (4, 8), (5, 2), (6, 7), (7, 3), (8, 6)

사실, 규칙을 만족하도록 배치하는 방법은 92가지가 있다.

n-여왕 문제의 규칙을 지키면서 여왕 말을 배치하는 모든 방법의 수를 출력하는 프로그램을 작성하시오. 입력으로 첫째 줄에 4 이상 20 이하의 정수 n이 주어진다. 출력으로, n-여왕 문제에서 여왕 말을 배치하는 모든 방법의 수를 출력한다.

입력의 예	출력의 예
4	2

입력의 예	출력의 예
8	92

제5장

스택 자료구조

5.1 스택의 개념
5.2 스택의 활용
5.3 스택의 구현
5.4 사례 연구: 미로에서 길 찾기
5.5 사례 연구: 편집기
연습문제

학습 목표

- 스택의 개념을 이해하고, 스택의 동작과 스택 연산의 종류와 기능을 설명할 수 있다.
- 자료구조를 설계하고 구현하는 과정을 설명할 수 있다.
- 스택을 활용하여 괄호 검사와 후위 수식의 계산하는 알고리즘을 설명할 수 있다.
- 파이썬 리스트와 collections.deque를 사용하여 Stack 클래스를 구현할 수 있다.
- 미로 탐색과 편집기 문제처럼 상태 관리가 필요한 상황에서 스택을 핵심 도구로 사용하여 효율적인 알고리즘을 설계하고 구현할 수 있다.

5.1 스택의 개념

스택(stack)이라는 단어는 물건들이 겹겹이 쌓여 있는 모습을 의미합니다. 그림 5.1의 예처럼, 뷔페식 식당에서 볼 수 있는 접시 더미나 두꺼운 책들이 쌓여 있는 모습이 스택의 전형적인 예입니다. 일반적인 경우, 접시 더미의 중간에서 접시를 빼내거나 끼워 넣는 행위를 하지 않을 것입니다. 왜냐하면 그렇게 할 경우, 잘못하면 접시 더미가 무너지는 사태가 발생할 수 있기 때문입니다. 그래서 접시가 필요하면 맨 위에 있는 접시부터 집어 들고, 깨끗한 빈 접시는 맨 위에 올려놓습니다.



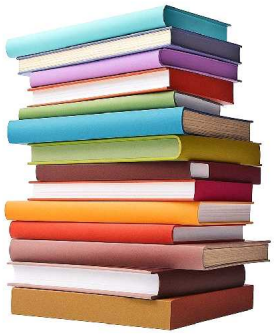


그림 5.1: 스택 구조의 예

자료구조로서의 스택은 데이터 원소가 순차적으로 저장되는 선형 구조에 해당하지만, 원소의 삽입과 삭제가 오직 한쪽 끝에서만 수행되어야 한다는 제약 조건을 가집니다. 그러므로 스택은 가장 나중에 삽입된 원소가 가장 먼저 삭제되는 방식으로 운영됩니다. 그래서 스택을 **후입선출(Last In First Out; LIFO)** 구조라고 부르기도 합니다. 알고리즘에서 스택을 대신하여 일반적인 선형리스트를 사용하더라도 스택의 모든 기능을 수행할 수 있지만, '스택'이라는 자료구조를 별도로 사용하는 이유는 "데이터를 한쪽 끝에서만 넣고 빼야 한다"는 제약을 의도적으로 적용하기 위해서입니다. 이러한 제약을 통해 얻는 이점은 다음과 같습니다.

- 스택을 사용하면 사용자의 명확한 의도를 담을 수 있어 단순하고 이해하기 쉬운 알고리즘을 구성할 수 있다.
- 프로그램 코드가 간결해지고 명확해져, 구현이 간편하며 유지보수가 쉬워진다.
- 후입선출에 기반한 특정한 순서로 데이터가 처리되어야 하는 상황에서 의도치 않은 오류를 방지할 수 있다.

스택에서 삽입과 삭제는 관습적으로 스택의 맨 위에서 발생한다고 말합니다. 그래서 스택에서 마지막에 저장된 원소의 위치를 **top**이라고 부르며 이 원소를 **top 원소**라고 합니다. 그림 5.2는 스택의 추상적인 모습을 보여줍니다. 그림의 스택에서 가장 먼저 삽입된 원소는 맨 밑에 있는 A이고 가장 나중에 삽입된 top 원소는 맨 위에 있는 D입니다. 스택에 저장된 원소가 하나도 없으면 그 스택을 **공백 스택(empty stack)**이라고 부릅니다. 스택에 새로운 원소를 삽입하는 연산은, 스택의 위쪽에서 삽입 작업이 수행되어, 그 원소를 새로운 top 원소로 만듭니다. 스택에서 원소를 삭제하는 연산은 기존의 top 원소를 스택에서 제거합니다. 이 연산 후에는 직전에 삽입되었던 원소가 새로운 top 원소가 됩니다. 스택에 원소를 삽입하는 연산을 보통 **push**라고 부르고, 스택에서 top 원소를 삭제하는 연산을 **pop**이라고 부릅니다.

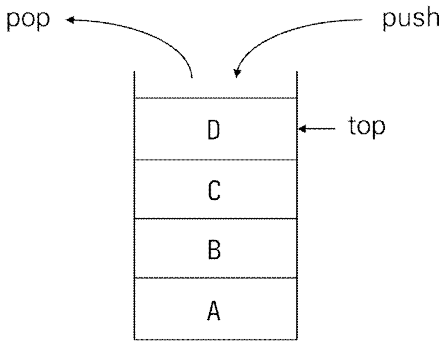


그림 5.2: 스택 구조

예를 들어, `push(item)`을 스택에 원소 `item`을 삽입하는 연산이라고 하고, `pop()`은 스택의 top 원소를 삭제하는 연산이라고 합시다. 그림 5.3은 공백 스택에서 다음과 같은 순서로 스택 연산들이 수행될 때 스택의 상태가 변화하는 모습을 보여줍니다.

```
push(A), push(B), pop(), push(C), push(D), pop()
```

앞서 언급한 접시 더미 외에도 스택의 후입선출 원리는 우리 주변에서 쉽게 찾아볼 수 있습니다. 몇 가지 사례를 들어보겠습니다.

- 텍스트 편집기의 '실행 취소(Undo)' 기능 : 연속적으로 '실행 취소' 명령을 사용하면 가장 최근에 수행한 작업부터 순서대로 되돌려진다.
- 웹 브라우저에서 '뒤로 가기' 버튼 : '뒤로 가기' 버튼을 누르면 가장 최근에 방문했던 페이지부터 역순으로 이동한다.

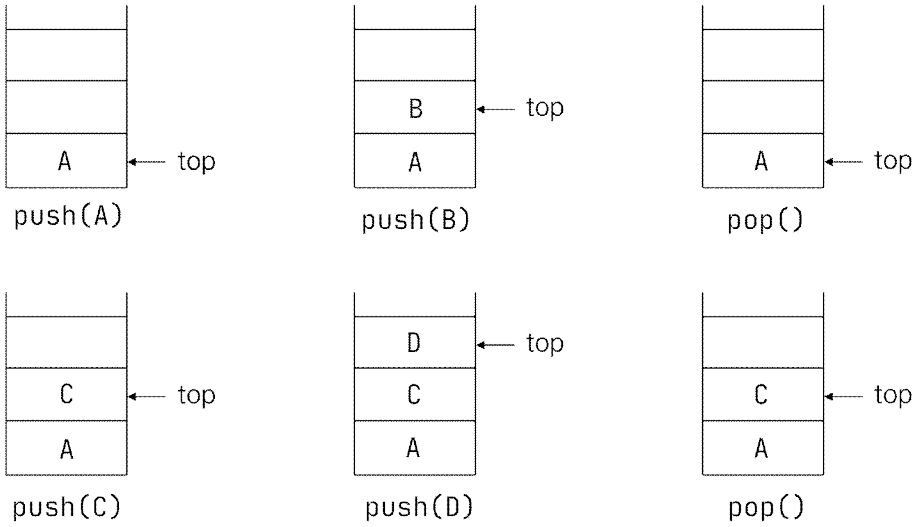


그림 5.3: 스택에서 삽입과 삭제 연산의 수행 예

- **프로그램 함수 호출** : 프로그램에서 함수가 호출될 때마다 해당 함수의 활성화 레코드가 시스템 스택에 쌓이고, 가장 마지막에 호출된 함수부터 실행을 끝내고 스택에서 제거된다.
- **막다른 골목 주차** : 막다른 골목에 주차된 차량은 나중에 들어온 차량부터 순서대로 빠져나가야 한다.

이처럼 스택의 후입선출 원리는 컴퓨터 과학뿐만 아니라 일상생활에서도 다양한 형태로 적용되고 있습니다.

이제, 스택의 동작 원리를 이해하는 데 도움이 되는 다음의 객차 재배치 문제를 살펴보겠습니다.

■ 객차 재배치 문제

기차역에는 객차의 순서를 재배열하기 위한 객차 창고가 있다. 입구 쪽 선로와 출구 쪽 선로의 중간에 객차 창고가 위치한다(그림 5.4 참조). 입구 쪽에서 들어오는 열차의 각 객차에는 재배치해야 할 순서에 해당하는 번호가 붙어 있다. 예를 들어, 입구 쪽에 4대의 객차가 (4, 1, 3, 2)의 순서로 연결되어 있으면, 출구로 나가는 객차의 순서는 (1, 2, 3, 4)이어야 한다. 객차 창고는 모든 객차가 들어갈 수 있을 정도로 충분히 긴 선로로 되어 있다. 입구 쪽 객차는 일단 객차 창고에 들어갔다가 출구 쪽으로 나가게 된다. 객차 창고에 여러 대의 객차가 들어가

있는 경우는 가장 나중에 들어간 객차가 가장 먼저 빠져나간다. 출구 쪽으로 나갈 때 객차의 앞뒤가 바뀌게 되어도 상관없다. 입구 쪽 객차를 객차 창고로 보내는 작업을 push라고 하고, 객차 창고의 객차를 출구로 보내는 작업을 pop이라고 하자. 객차의 재배치가 가능한 경우, 객차 재배치를 위한 작업의 순서를 나열하시오.

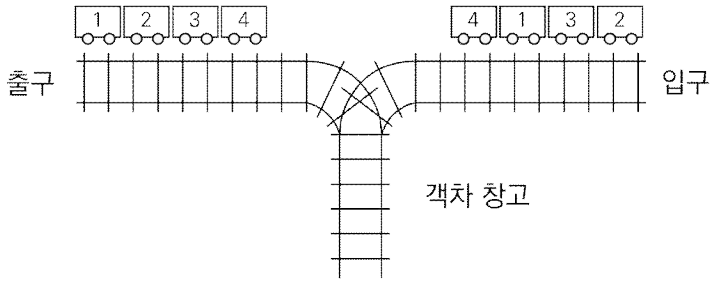


그림 5.4: 객차 재배치 문제

객차 창고에 들어간 객차들은 역순으로 객차 창고를 빠져나오므로 객차 창고는 스택 구조에 해당한다는 것을 알 수 있습니다. 입구 쪽에서 객차가 (4, 1, 3, 2) 순서로 들어오는 경우 객차 재배치를 위한 작업 순서는 다음과 같습니다.

push, push, pop, push, push, pop, pop, pop

처음 두 번의 push 작업으로 4번과 1번 객차가 차례로 객차 창고에 들어갑니다. 그다음 pop 작업이 수행되면, 객차 창고의 맨 위에 있는 1번 객차가 출구로 빠져나갑니다. 이어서 다시 두 번의 push 작업으로 3번과 2번 객차가 객차 창고에 들어가게 되어, 객차 창고에는 아래쪽부터 4번, 3번, 2번 순으로 객차들이 대기합니다. 마지막 세 번의 pop 작업으로 2번, 3번, 4번 순으로 객차가 빠져나와 결국 번호 순서대로 객차들이 출구 쪽으로 나가게 됩니다.

만약 객차가 (2, 3, 1, 4) 순서로 들어온다면 객차 재배치가 가능할까요? 1번 객차를 가장 먼저 출구로 내보내야 하므로, 그 앞의 2번, 3번 객차는 두 번의 push 작업으로 객차 창고에 대기시켜야 합니다. 그다음 2번 객차를 빼내어야 하는데, 더 나중에 들어온 3번 객차 때문에 2번 객차를 빼낼 수 없습니다. 그러므로 재배치 작업은 불가능합니다.

5.2 스택의 활용

자료구조를 설계하고 구현하는 과정은 보통 다음의 3단계를 거칩니다.

[단계 1] 추상자료형 명세: 프로그램에서 사용될 자료 객체의 유형과 그 자료 객체를 대상으로 수행되어야 할 연산들을 정의한다.
[단계 2] 추상자료형 설계: 자료 객체의 구체적인 표현 방식을 결정하고, 각 연산의 수행 과정을 컴퓨터 알고리즘으로 설계한다.
[단계 3] 프로그램 구현: 설계된 추상자료형을 컴퓨터 프로그램으로 구현한다.

앞으로 추상자료형 명세는 파이썬의 class 형식으로 표현합니다. 이 형식에서 추상자료형의 각 연산을 클래스 메소드의 머리(header)로 표현하고 주석을 사용하여 연산의 기능을 설명합니다. 이 클래스에서 각 메소드의 몸체(body)를 완성하면 추상자료형이 파이썬 프로그램으로 구현됩니다.

스택은 선형리스트에 비해 기능이 단순하므로 필요한 연산의 수는 많지 않습니다. 다음 추상자료형 5.1에는 스택에서 필요한 최소한의 연산만으로 정의한 추상자료형 명세가 나와 있습니다.

추상자료형 5.1: Stack

```
class Stack:
   def __init__(self):     # 공백 스택을 생성한다.
   def empty(self):        # 스택이 공백이면 True, 아니면 False를 반환한다.
   def push(self, item):   # 스택에 원소 item을 삽입한다.
   def pop(self):          # 스택의 top 원소를 삭제한 후 반환한다.
                           # 공백 스택에 대해 호출되면 None을 반환한다.
   def peek(self):         # 스택의 top 원소를 반환한다.
                           # 공백 스택에 대해 호출되면 None을 반환한다.
```

메소드 pop()과 peek()의 경우, 스택이 공백인 상태에서 호출되면 정상적으로 수행될 수 없으므로 오류가 발생해야 합니다. 이 오류는 대부분 스택의 공백 상태를 고려하지 않은 알고리즘의 잘못된 설계로 인해 발생합니다. 이 책에서는 이러한 오류에 대한 처리를 사용자에게 일임하기 위해 None을 반환하도록 정의합니다.[10] 메소드 push()의 경우, 스택을 정적 배열로 구현

10) 일반적으로는 None을 반환하지 않고, IndexError('pop from empty stack')처럼 명시적인 예외를 발생시키거나 오류 메시지를 출력하고 프로그램을 종료시키는 방식을 사용합니다.

하면 **스택 오버플로(stack overflow)**에 대한 대비를 해야 하지만, 동적 배열인 파이썬 리스트를 이용하면 스택 오버플로를 별도로 고려하지 않아도 됩니다. 나중에 스택의 구현에서 다시 소개되겠지만, 스택의 모든 연산은 상수 시간에 수행됩니다. 추상자료형 Stack의 구현 결과로 상수 시간에 수행되지 않는 연산이 존재한다면, 이는 스택을 부적절하게 구현한 것임을 의미합니다.

이제, 스택을 활용하여 문제를 해결하는 몇 가지 사례를 소개합니다. 먼저 괄호 검사 문제부터 살펴봅니다.

■ 괄호 검사 문제

세 가지 종류의 괄호, 즉 대괄호('[', ']'), 중괄호('{', '}'), 소괄호('(', ')')를 사용하는 텍스트가 있다. 괄호는 텍스트에서 특정 구간을 설정하며, 구간의 범위는 왼쪽 괄호부터 시작하여 오른쪽 괄호에서 끝난다. 괄호 검사 문제는 입력으로 주어지는 문자열에 포함된 괄호가 올바르게 사용되었는지를 검사하는 것이다. 다음의 세 가지 조건을 만족하면 괄호가 올바르게 사용되고 있는 것으로 판별한다.

1. 같은 종류의 괄호에서 왼쪽 괄호의 개수와 오른쪽 괄호의 개수는 같아야 한다.
2. 같은 종류의 괄호에서 왼쪽 괄호가 오른쪽 괄호보다 먼저 나와야 한다.
3. 괄호 사이에는 포함 관계만 존재해야 한다. 예를 들어, "([ )]"와 같이 종류가 다른 두 괄호의 범위 일부가 겹치지 않아야 한다.

괄호가 올바르게 사용되었으면 'VALID', 아니면 'WRONG'을 출력하는 프로그램을 작성하시오.

거의 모든 프로그래밍 언어의 소스 코드에는 위에서 언급한 세 가지 종류의 괄호가 사용됩니다. 소스 코드가 위의 세 가지 조건을 만족하지 않으면 컴파일 오류가 발생할 것입니다. 산술식이나 일반적인 텍스트에서도 괄호를 사용할 때, 위의 세 가지 조건을 만족해야 합니다. 다음은 괄호가 올바르게 사용된 텍스트 예입니다.

```
A * [ { ( B + C ) / D } - E ] + F
```

다음은 괄호가 잘못 사용된 텍스트 예입니다.

```
(A + (B * C)          # 왼쪽 괄호가 많음
A { ( B C ) D } }     # 오른쪽 괄호가 많음
[ A ( B ) } C { ]     # 오른쪽 중괄호가 왼쪽 중괄호보다 먼저 나와 있음
( A [ B C ) ]         # 소괄호와 대괄호의 범위가 겹침
```

조건 1과 2만 만족하는지 검사하는 것은 다음과 같이 간단하게 해결할 수 있습니다. 먼저, 괄호 종류마다 0으로 초기화된 변수를 생성합니다. 그다음, 입력으로 주어진 문자열의 각 문자를 차례로 조사하면서, 괄호 종류별로 왼쪽 괄호를 만나면 변수에 1 증가시키고, 오른쪽 괄호를 만나면 1 감소시킵니다. 마지막 문자까지 조사했을 때, 0이 아닌 변수가 있다면 조건 1을 만족하지 않음을 알 수 있습니다. 또한 작업 도중에 변수의 값이 음수가 되면 조건 2를 만족하지 않음을 알게 됩니다. 이 작업은 텍스트를 한번 읽는 동안 처리되므로 선형 시간에 수행됩니다. 그러나 이 작업만으로는 조건 3까지 처리할 수 없습니다.

조건 3이 만족하는지를 검사하기 위해서는 텍스트를 읽는 과정에서 발견한 왼쪽 괄호의 정보를 기록해 두어야 합니다. 그래야 오른쪽 괄호를 만났을 때 짝이 되는 왼쪽 괄호가 올바른 위치에 있는지를 판단할 수 있기 때문입니다. 괄호의 구간은 포함 관계만 성립하므로, 오른쪽 괄호의 짝이 되는 왼쪽 괄호는 항상 가장 최근에 발견된 괄호이어야 합니다. 따라서 가장 나중에 기록한 왼쪽 괄호를 먼저 검사해야 하는데, 이는 바로 스택의 성질에 해당하므로, 왼쪽 괄호들을 스택에 저장하는 것이 적절합니다. 알고리즘 5.1에는 스택을 사용하여 괄호 검사를 수행하는 알고리즘이 나와 있습니다.

알고리즘 5.1: 스택을 사용하여 괄호 검사를 수행하는 알고리즘

```
1  공백 스택 S를 생성한다.
2  for 입력 문자열에 있는 각 문자 c에 대해서 차례로 수행한다:
3     if c가 왼쪽 괄호이면: 스택 S에 삽입한다.
4     if c가 오른쪽 괄호이면:
5        if 스택 S가 공백이면: 'WRONG'을 출력하고 종료한다.
6        스택에서 왼쪽 괄호를 삭제한다.
7        if 삭제된 괄호와 c가 같은 종류가 아니면: 'WRONG'을 출력하고 종료한다.
8  if 스택이 공백이 아니면: 'WRONG'을 출력한다.
9  else: 'VALID'를 출력한다.
```

단계 5에서 스택이 공백이면 오른쪽 괄호와 짝이 되는 왼쪽 괄호가 없으므로, 조건 1 또는 2를 만족하지 않습니다. 단계 7에서, 스택에서 삭제된 왼쪽 괄호와 입력 문자열에 있는 오른쪽 괄호가 같은 종류가 아닌 경우, 조건 1 또는 2를 위배하거나 아니면 오른쪽 괄호의 짝이 스택의 더 안쪽에 있어야 하므로 조건 3을 위배합니다. 따라서 이 경우 알고리즘은 'WRONG'을 출력합니다. 마지막 괄호까지 조사한 후에도 스택에 괄호가 남아 있으면(단계 8), 조건 1을 위배하기 때문에 'WRONG'을 출력합니다. 마지막까지 이 모든 경우를 통과하면 괄호가 올바르게 사용된 것으로

판별되어 'VALID'를 출력합니다(단계 9). 이처럼 스택을 사용하면 스택의 단순한 특성이 반영되어 알고리즘이 간결하고 이해하기 쉬워집니다. 프로그램 5.1은 추상자료형 Stack을 사용하여 알고리즘 4.1을 파이썬으로 구현한 프로그램입니다. 프로그램에서 함수 is_valid(text)는 문자열 text에서 괄호가 올바르게 사용되었으면 True, 아니면 False를 반환합니다.

프로그램 5.1: 괄호를 검사하는 프로그램

```
1  def is_valid(text):
2     """문자열 text에서 괄호 사용이 유효하면 True, 아니면 False를 반환한다."""
3     S = Stack()                                  # 공백 스택 생성
4     for ch in text:
5        if ch in ['(', '{', '[']:                 # 왼쪽 괄호면 스택에 삽입
6           S.push(ch)
7        elif ch in [')', '}', ']']:               # 오른쪽 괄호인 경우
8           if S.empty():                          # 왼쪽 괄호가 부족한 경우
9              return False
10          chs = S.pop()                          # 스택의 top 괄호 삭제
11          if (ch== ')' and chs != '(') or \      # 괄호의 짝이 맞지 않는 경우
12             (ch== '}' and chs != '{') or \
13             (ch== ']' and chs != '['):
14             return False
15    return S.empty():                            # 마지막에 스택이 공백이어야 함
16
17 # main program
18 if is_valid(input()):
19    print('VALID')
20 else:
21    print('WRONG')
```

함수 is_valid()에서 for 문은 최대 n번 반복됩니다, 여기서 n은 문자열 text의 길이입니다. 스택 연산은 모두 상수 시간에 수행되므로, 이 함수의 수행 시간은 $O(n)$입니다.

다음으로, 산술식(arithmetic expression)의 표현 방법 중에서 후위 표기법으로 표현된 수식을 계산하는 문제를 살펴봅니다.

■ 후위 수식 계산

후위 수식의 계산 결과를 출력하는 프로그램을 작성하시오. 입력으로 들어오는 후위 수식은 다음의 조건을 따른다.

1. 피연산자는 한 자릿수 정수로 주어진다.
2. 연산자는 사칙 연산자(+, -, *, /)만 사용한다.
3. 나눗셈의 결과는 소수점 이하 숫자가 제거된 정수이다. (예, -1/5 = 0)
4. 각 토큰(연산자와 피연산자) 사이에는 빈칸이 하나 이상 들어간다.
5. 후위 수식은 항상 올바른 수식으로 입력된다.

수식을 표현하는 방법에는 **전위 표기법(prefix notation)**, **중위 표기법(infix notation)**, 그리고 **후위 표기법(postfix notation)**이 있습니다. 우리가 일상에서 보통 사용하는 수식은 중위 표기법으로 표현되어 있습니다. 이항 연산자, 즉 두 개의 피연산자를 가지는 연산자를 중심으로 표기법의 특징을 구분하면 다음과 같습니다. 중위 표기법은 연산자가 두 피연산자 사이에 들어간 형태이고, 전위 표기법은 연산자가 뒤에 두 개의 피연산자가 나오는 형태이며, 후위 표기법은 두 개의 피연산자 뒤에 연산자가 위치하는 형태입니다. 표 5.1은 각 표기법으로 표현된 수식의 예를 보여줍니다. 각 표기법으로 표현된 수식을 간략히 **중위 수식**, **전위 수식**, **후위 수식**으로 부릅니다.

표 5.1: 중위, 전위, 후위 표기법으로 표현된 수식의 예

중위 수식	전위 수식	후위 수식
a+b	+ab	ab+
a+b*c	+a*bc	abc*+
a*b+c	+*abc	ab*c+
a/(b-c+d)*(e-a)*c	**/a+-bcd-eac	abc-d+/ea-*c*

중위 표기법으로 표현된 아래 수식은 어떤 순서로 계산해야 할까요?

```
a + b * c
```

먼저 'b * c'를 계산합니다. 그 결과가 'd'라고 하면, 그다음 'a+d'를 계산합니다. 중위 수식은 연산자들 사이의 **우선순위(precedence)**를 알고 있어야 올바르게 계산할 수 있습니다. 우선순위에 대한 정보를 모르거나 순서를 바꾸어 계산하고 싶은 경우에는 괄호를 사용합니다. 그래서 위의 수식은 아래처럼 표현할 수 있습니다.

```
a + (b * c)
```

또한, 연산자의 종류에 따라서는 우선순위가 같다고 하더라도 역순으로 계산해야 하는지를 판단하기 위해 **결합법칙(associativity)**을 알아야 합니다. 이처럼 우리가 사용하고 있는 중위 표기법은 사실 꽤 복잡합니다.

후위 표기법으로 표현된 수식, 즉 후위 수식은 사람이 보기에는 다소 복잡해 보일 수 있지만, 오히려 더 쉽게 계산할 수 있습니다. 후위 수식에는 연산자 우선순위와 결합법칙이 없으며 괄호조차 필요하지 않습니다. 중위 수식을 후위 수식으로 변환하려면 다음의 절차대로 수행하면 됩니다.

① 우선순위와 결합법칙을 고려하지 않도록 수식에 괄호를 최대로 삽입한다.
② 연산자마다 자신을 포함하고 있는 가장 안쪽 괄호의 오른쪽 괄호 옆으로 연산자를 옮긴다.
③ 수식에서 모든 괄호를 제거한다.

위의 절차를 중위 수식에 적용한 예를 살펴보겠습니다.

```
a + b * c
   ⇒ (a + (b * c) )          # 괄호를 최대로 삽입한다.
   ⇒ (a   (b   c)*)+         # 연산자를 오른쪽 괄호 옆으로 옮긴다.
   ⇒ a b c * +               # 괄호를 제거한다.

a/(b-c+d)*(e-a)*c
   ⇒ (((a/((b-c)+d))*(e-a))*c)    # 괄호를 최대로 삽입한다.
   ⇒ (((a((bc)-d)+)/(ea)-)*c)*    # 연산자를 오른쪽 괄호 옆으로 옮긴다.
   ⇒ abc-d+/ea-*c*                # 괄호를 제거한다.
```

위 절차는 사람이 직접 손으로 중위 수식을 후위 수식으로 변환하고자 할 때 유용합니다. 참고로, 위 절차의 두 번째 단계에서 연산자를 왼쪽 괄호 옆으로 옮기면 전위 수식이 됩니다.

후위 수식은 수식을 읽으면서 바로 계산할 수 있습니다. 수식에서 연산자를 만나면 직전에 만난 두 피연산자를 가지고 계산하고 그 결과를 수식에 대입하는 방식으로 진행됩니다. 다음은 후위 수식을 계산하는 과정을 보여줍니다. 빨간색 숫자는 직전의 연산 결과를 표시한 것입니다.

```
3 8 2 / 5 3 - * +          # 연산자 /를 만나면, 8/2를 계산한다.
⇒ 3 4 5 3 - * +            # 연산자 -를 만나면, 5-3를 계산한다.
```

```
⇒ 3 4 2 * +          # 연산자 *를 만나면, 4*2를 계산한다.
⇒ 3 8 +              # 연산자 +를 만나면, 3+8을 계산한다.
⇒ 11                 # 수식의 결과
```

이 계산 과정에서 두 피연산자를 선택할 때 후입선출이라는 스택의 성질이 적용되었음을 알 수 있습니다. 표 5.2는 스택을 이용하여 후위 수식을 계산하는 과정을 보여줍니다. 수식의 각 토큰(연산자와 피연산자)을 차례대로 조사하면서, 피연산자를 만나면 스택에 삽입하고, 연산자를 만나면 스택에서 두 개의 피연산자를 빼내어 해당 연산자와 계산한 후 다시 스택에 삽입합니다. 수식의 계산 결과는 모든 토큰을 처리한 후에 스택에 남아 있는 값입니다.

표 5.2: 후위 수식 "3 8 2 / 5 3 - * +"의 수행 과정

토큰	스택의 내부				비고
	0	1	2	3	
3	3				피연산자 3을 스택에 삽입
8	3	8			피연산자 8을 스택에 삽입
2	3	8	2		피연산자 2를 스택에 삽입
/	3	4			8/2=4를 계산하여 삽입
5	3	4	5		피연산자 5를 스택에 삽입
3	3	4	5	3	피연산자 3을 스택에 삽입
-	3	4	2		5-3=2를 계산하여 삽입
*	3	8			4*2=8을 계산하여 삽입
+	11				3+8=11을 계산하여 삽입

프로그램 5.2는 추상자료형 Stack을 사용하여 후위 수식을 계산하는 프로그램입니다. 함수 eval_postfix(E)는 후위 수식 E의 계산 결과를 반환합니다.

프로그램 5.2: 후위 수식을 계산하는 프로그램

```
1  def eval_postfix(E):
2      """후위 수식 E의 계산 결과를 반환한다."""
3      S = Stack()                      # 공백 스택 생성
4      Tokens = E.split()               # 토큰들의 리스트 구성
5      for tk in Tokens:                # 각 토큰에 대해서 수행
6          if tk.isdigit():             # 숫자인 경우 스택에 삽입
7              S.push(int(tk))
8          else:                        # 연산자인 경우
```

```
9           y = S.pop()                 # 오른쪽 피연산자
10          x = S.pop()                 # 왼쪽 피연산자
11          # 계산 결과를 스택에 삽입
12          if tk == '+': S.push(x+y)
13          elif tk == '-': S.push(x-y)
14          elif tk == '*': S.push(x*y)
15          elif tk == '/': S.push(x//y)
16      return S.peek()         # 최종 계산 결과 반환
17
18  # main program
19  print(eval_postfix('2 3 * 4 +'))
20  print(eval_postfix('3 8 2 / 5 3 - * +'))
```

프로그램으로 중위 수식을 후위 수식으로 변환하는 과정도 스택을 활용할 수 있습니다. 이 과정에서는 스택에 피연산자가 아니라 연산자를 삽입합니다. 중위 수식을 읽어가면서 피연산자를 만나면 즉시 출력하고, 연산자를 만나면 우선순위와 결합법칙을 고려하여 스택에 연산자를 넣을 것인지 아니면 스택의 top에 있는 연산자를 빼낼 것인지를 결정합니다. 자세한 내용은 연습문제를 참고하세요.

5.3 스택의 구현

스택은 배열 구조인 파이썬 리스트를 이용하여 손쉽게 구현할 수 있습니다. 스택의 구현에서 모든 연산이 상수 시간에 수행되어야 한다는 점에 유의해야 합니다. 파이썬 리스트를 스택 공간으로 사용한다면, 스택에 삽입된 원소의 순서대로 리스트의 앞쪽부터 차례로 배치하면 됩니다. 즉, 스택의 top 원소가 리스트의 맨 끝에 위치하도록 배치하는 것입니다. 그림 5.5는 스택의 상태에 대응하는 파이썬 리스트의 모습을 보여줍니다.



그림 5.5: 스택과 파이썬 리스트

스택 S가 파이썬 리스트 L로 표현되어 있다면, 각 스택 연산과 대응하는 파이썬 리스트 연산은 다음과 같습니다.

```
S.push(e) ↔ L.append(e)
S.pop()   ↔ L.pop()
S.peek()  ↔ L[-1]
S.empty() ↔ len(L) == 0
```

실험용 프로그램을 작성하거나 프로그래밍 경시대회에서 프로그램을 작성하는 경우에는 파이썬 리스트를 직접 사용하는 것이 더 간편할 수 있습니다. 하지만 중요한 프로그램을 개발하는 과정에서는 추상자료형으로 표현된 스택을 사용하는 것이 권장됩니다. 스택을 명시적으로 사용함으로써 프로그램을 더욱 읽기 쉽게 만들 수 있고, 리스트의 임의 위치에서 요소의 삽입이나 삭제를 수행하는 등 추상화를 깨뜨리는 실수를 방지할 수 있습니다. 프로그램 5.3은 파이썬 리스트로 구현한 Stack 클래스를 보여줍니다.

프로그램 5.3: 파이썬 리스트를 사용하여 구현한 Stack 클래스

```
1  class Stack:
2     """파이썬 리스트를 사용하여 구현한 스택 클래스"""
3
4     def __init__(self):     # 공백 스택을 생성한다.
5        self._data = []
6
7     def empty(self):        # 스택이 공백이면 True, 아니면 False를 반환한다.
8        return len(self._data) == 0
9
10    def push(self, item):   # 스택에 원소 item을 삽입한다.
11       self._data.append(item)
12
13    def pop(self):          # 스택의 top 원소를 삭제한 후 반환한다.
14       if self.empty():
15          return None
16       return self._data.pop()
17
18    def peek(self):       # 스택의 top 원소를 반환한다.
19       if self.empty():
20          return None
21       return self._data[-1]
```

프로그램에서 스택 원소들을 저장할 공간은 self._data라는 이름을 가진 파이썬 리스트입니다. 여기서 밑줄('_')로 시작하는 이름은 클래스 외부에서는 직접 접근하지 말라는 의미로 사용됩니다. 파이썬 리스트의 append()와 pop() 메소드는 3장에서 언급했듯이 상각 시간복잡도로서 상수 시간에 수행되므로, 스택 연산 push()와 pop() 메소드도 동일한 수행 시간을 가집니다. 이는 대부분의 일반적인 호출은 상수 시간에 수행되지만, 때때로 (리스트 크기 재할당 작업 때문에) 선형 시간이 소요되는 경우가 발생할 수 있다는 것을 의미합니다. Stack 클래스의 나머지 메소드들은 모두 상수 시간에 수행됩니다.

메소드 push()와 pop()의 수행 시간이 상각 시간복잡도가 아닌 순수한 상수 시간에 수행되도록 구현하려면 7장에서 소개하는 연결리스트를 이용해야 합니다. 연결리스트를 직접 사용하여 스택을 구현할 수도 있지만, 파이썬에서 표준 모듈로 제공하는 collections 모듈의 deque 자료구조를 이용해도 됩니다. deque는 양쪽 끝에서만 삽입과 삭제가 가능한 선형리스트로, 연결리스트 기반으로 구현되어 있습니다. 따라서 deque를 이용하면 모든 연산이 순수하게 상수 시간에 수행되는 스택을 구현할 수 있습니다. 프로그램 5.4에 collections.deque를 이용하여 스택을 구현한 Stack 클래스가 나와 있습니다.

프로그램 5.4: collections.deque를 사용하여 구현한 Stack 클래스

```
1 from collections import deque
2
3 class Stack:
4    """collections.deque를 사용하여 구현한 스택 클래스"""
5
6    def __init__(self):     # 공백 스택을 생성한다.
7       self._data = deque()
8
9    # 나머지 함수들은 프로그램 5.3과 완전히 같으므로 생략합니다.
```

사실, 스택은 파이썬 리스트로 구현해도 충분히 효율적이므로, 굳이 연결리스트나 deque를 이용할 필요성은 거의 없습니다. 그래서 deque에 관한 추가적인 내용은 6장 큐 자료구조에서 자세하게 다룹니다.

5.4 사례 연구: 미로에서 길 찾기

그리스 로마 신화를 보면 영웅 테세우스가 크레타섬의 미궁에서 반인반수 괴물 미노타우로스를 퇴치하는 내용이 나옵니다. 테세우스는 크레타의 공주 아리아드네가 준 실타래를 이용하여 괴물을 처치한 후 미궁에서 무사히 빠져나오게 됩니다. 크레타섬의 미궁과 같은 미로(maze)에서는 두 가지 사항을 고려해야 합니다. 첫 번째는 미로의 입구에서부터 미로를 탐색하여 목적지를 찾는 것이고, 두 번째는 목적지에서 다시 입구로 되돌아와야 하는 것입니다. 따라서 목적지를 찾을 때 입구에서 목적지까지의 경로를 기록해 두어야만 그 정보를 이용하여 불필요한 시행착오 없이 곧바로 입구로 되돌아올 수 있습니다.

미로를 탐색할 때 가장 중요한 것은 같은 장소를 맴도는 것을 피하는 것입니다. 같은 장소를 맴돌지만 않는다면 시간이 다소 걸리더라도 언젠가는 목적지를 만나게 됩니다. 미로에서 길을 찾는 방법으로 '왼손 짚기'라는 방법이 있습니다. 이 방법은 미로 입구에서부터 왼손을 벽에서 떼지 않고 목적지를 만날 때까지 나아가는 것입니다. 이 방법은 미로를 탐색하는 효과적인 방법이긴 하지만 항상 목적지를 찾는다는 보장을 할 수는 없습니다. 그림 5.6 (a)의 미로에서는 왼손 짚기 방법으로 목적지를 찾을 수 있지만, (b)의 미로에서는 목적지를 찾을 수 없습니다. 왼손 짚기는 벽을 만지면서 나아가기 때문에 입구에서부터 연결된 벽들의 주변 장소는 빠짐없이 탐색될 수 있습니다. 따라서 왼손 짚기로서 목적지를 찾을 수 있는 미로는 모든 벽이 입구에서부터 연결되어 있어야 한다는 제약이 있습니다. 그림 5.6 (b)의 미로는 이 제약 조건을 만족하지 않기 때문에 이 방법으로 목적지를 찾을 수 없습니다.

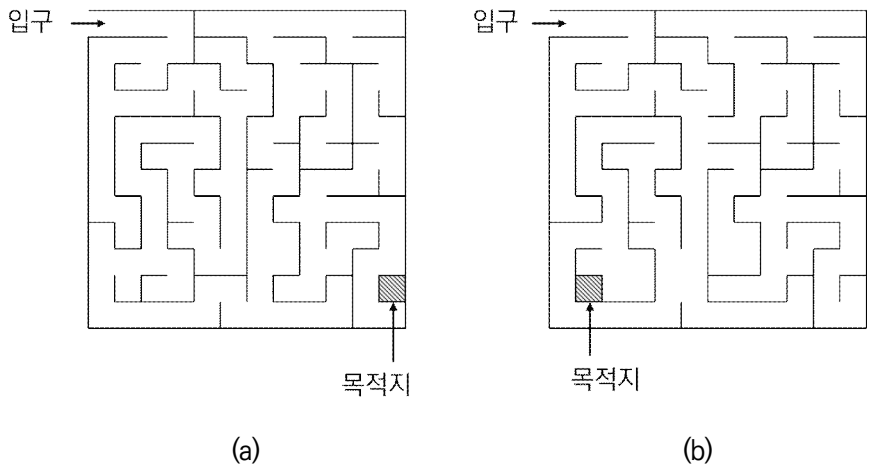
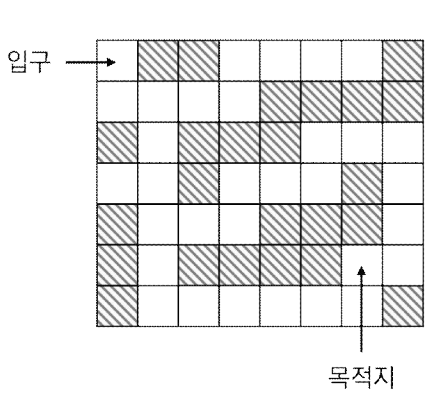


그림 5.6: 미로의 예

그림 5.6 (b)와 같은 형태의 미로에서는 테세우스가 사용한 실타래 방법이 효과적입니다. 실타래의 실 한쪽 끝을 입구에 묶어 둔 다음, 새로운 길을 찾아 나갈 때마다 실타래를 풀어주고 막다른 길에 도달하여 되돌아가야 할 때 실타래를 감아줍니다. 이때 같은 장소를 반복해서 맴돌지 않도록 미로에 한 번 지나간 길을 표시해 두어야 합니다. 미로를 탐색하는 동안 실타래는 항상 입구에서 현재 서 있는 장소까지의 경로 정보를 가지게 됩니다. 따라서 목적지를 찾은 후에 다시 입구로 되돌아가려면 실타래를 감아주기만 하면 됩니다.

지금부터 테세우스의 실타래 방법을 컴퓨터 알고리즘으로 표현해 보겠습니다. 먼저 미로를 표현하는 방법을 알아봅니다. 미로는 2차원 행렬로 표현하며, 행렬의 각 항에서 1은 벽 또는 장애물을, 0은 지나갈 수 있는 통로를 나타냅니다. 즉, 그림 5.7 (a)와 같은 형태의 미로를 그림 5.7 (b)의 행렬로 표현하는 것입니다.



(a)

	0	1	2	3	4	5	6	7
0	0	1	1	0	0	0	0	1
1	0	0	0	0	1	1	1	1
2	1	0	1	1	1	0	0	0
3	0	0	1	0	0	0	1	0
4	1	0	0	0	1	1	1	0
5	1	0	1	1	1	1	0	0
6	1	0	0	0	0	0	0	1

입구: (0, 0) 목적지: (5, 6)

(b)

그림 5.7: 미로의 표현

미로에서 각 위치는 행렬에서 행 번호와 열 번호의 쌍 (i, j)로 표시할 수 있습니다. 앞으로 이 쌍을 '좌표'라고 부릅니다. 미로에서 동서남북 네 방향 중 하나에 접해 있는 두 위치를 '서로 인접하다'고 말합니다. 서로 인접한 위치가 둘 다 통로이면 이동할 수 있습니다. 예를 들어 그림 5.7 (b)에서 좌표 (4, 1)의 인접 위치는 (4, 2), (4, 0), (5, 1), (3, 1)이 되고, (4, 0)의 위치는 벽이므로 나머지 세 곳만 이동 가능한 위치입니다. 미로에서의 길은 좌표들의 순서로 나타냅니다. 예를 들어, 다음의 좌표 리스트는 그림 5.7 (b)의 미로의 입구에서 목적지까지 연결되는 경로를 표현합니다.

(0, 0), (1, 0), (1, 1), (2, 1), (3, 1), (4, 1), (4, 2), (4, 3), (3, 3),
(3, 4), (3, 5), (2, 5), (2, 6), (2, 7), (3, 7), (4, 7), (5, 7), (5, 6)

알고리즘 5.2는 미로 입구에서 목적지까지 연결되는 경로를 찾는 알고리즘입니다. 이 알고리즘은 가상코드로 작성되어 있습니다. 알고리즘에서 미로를 표현한 행렬을 maze, 지나온 길의 표시를 위한 행렬을 mark라고 부릅니다. mark는 maze와 동일한 크기를 가지며 미리 0으로 초기화됩니다. mark에서 0은 해당 위치를 아직 방문하지 않았다는 표시입니다. 알고리즘 수행 도중에 어떤 위치 (i, j)를 방문하면 mark[i][j]에 1을 배정합니다.

알고리즘 5.2: 미로 문제를 위한 가상코드로 표현된 알고리즘

```
1  공백 스택 S를 생성한다.
2  입구 좌표 (x, y)를 스택 S에 삽입한다.
3  mark[x][y]에 1을 배정한다.
4  while 스택 S가 공백이 아니면:
5     스택 S의 top에 있는 위치를 (x, y)라고 하자.
6     if (x, y)가 목적지 좌표이면:
7        스택 S에 저장된 경로를 출력하고 종료한다.
8     for (x, y)의 각 인접 위치 (r, c)에 대해:
9        if (r, c)가 이동 가능한 새로운 위치이면:
10          mark[r][c]에 1을 배정한다.
11          스택 S에 (r, c)를 삽입한다.
12          for문을 벗어난다.
13    if 이동 가능한 새로운 위치를 발견하지 못하면:
14       스택 S에서 top 원소를 삭제한다.
15 미로에 길이 없음을 출력하고 종료한다.
```

알고리즘에서 스택은 테세우스의 실타래 역할을 담당합니다. 스택에는 입구에서부터 현재 위치까지 길의 좌표들이 차례로 저장됩니다. 새로운 위치로 전진할 때 그 새로운 현재 위치를 스택에 삽입합니다. 막다른 길에서 되돌아가야 할 때는 스택에서 top 위치를 삭제합니다. 그러면 현재 위치는 삭제 후의 top 위치로 바뀝니다. 스택에 새로운 위치를 삽입하는 것은 실타래의 실을 풀어주는 것을 의미하고, 스택에서의 삭제는 실타래를 감는 행동을 의미합니다. 만일, 미로에서 목적지까지의 길이 존재하지 않으면 결국 입구로 되돌아오게 됩니다. 따라서 입구에서도 이동 가능한 새로운 위치가 없으면 이 미로에는 길이 존재하지 않음을 알게 됩니다.

표 5.3은 그림 5.8의 미로에 대해서 알고리즘 5.2의 방식으로 입구에서 목적지까지 길을 찾는 과정을 나타낸 것입니다. 알고리즘 5.2의 for 문에서 현재 위치 (x, y)의 각 인접 위치를 찾을 때, 조사하는 방향의 순서는 별 의미가 없지만, 여기서는 북쪽부터 시계방향 순으로 즉, 북동-

남·서의 순서로 조사합니다. 현재 조사하고 있는 인접 위치가 이동 가능한 위치(즉, 통로이며 앞서 방문한 적이 없는 위치)이면, 그 위치가 새로운 현재 위치가 됩니다.

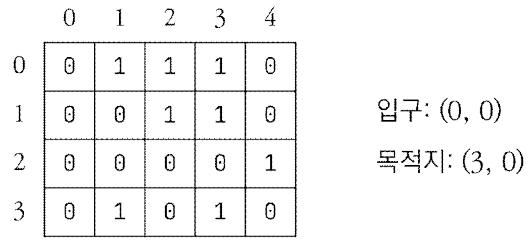


그림 5.8: 미로의 예

표 5.3: 그림 5.8의 미로에서 길을 찾는 과정

단계	스택						비 고
	0	1	2	3	4	5	
1	(0, 0)						입구 위치에서 시작
2	(0, 0)	(1, 0)					남쪽으로 전진
3	(0, 0)	(1, 0)	(1, 1)				동쪽으로 전진
4	(0, 0)	(1, 0)	(1, 1)	(2, 1)			남쪽으로 전진
5	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 2)		동쪽으로 전진
6	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 2)	(2, 3)	동쪽으로 전진
7	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 2)		서쪽으로 후퇴
8	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 2)	(3, 2)	남쪽으로 전진
9	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 2)		북쪽으로 후퇴
10	(0, 0)	(1, 0)	(1, 1)	(2, 1)			서쪽으로 후퇴
11	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 0)		서쪽으로 전진
12	(0, 0)	(1, 0)	(1, 1)	(2, 1)	(2, 0)	(3, 0)	남쪽으로 전진(목적지 도달)

미로에서의 현재 위치가 (x, y)일 때, 북쪽, 동쪽, 남쪽, 서쪽으로 이동 후의 좌표는 각각 (x-1, y), (x, y+1), (x+1, y), (x, y-1)이 됩니다(그림 5.9 참조). 각 인접 위치를 조사할 때, 미로의 좌표 범위를 벗어나지 않도록 유의해야 합니다. 미로의 크기를 m×n이라고 하면, 미로의 각 위치 (i, j)는 $0 \le i < m$, $0 \le j < n$을 만족해야 합니다.

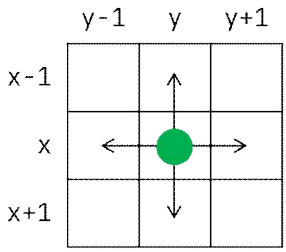


그림 5.9: 인접 위치

알고리즘 5.2에서 줄 번호 8부터 15까지의 코드를 구현할 때, 다음과 같이 각 방향에 대해 검사하는 if-elif-else 문을 사용할 수 있습니다.

```
# 각 방향에 대한 검사
if x > 0 and maze[x-1][y] == 0 and mark[x-1][y] == 0:        # 북쪽 조사
   mark[x-1][y] = 1
   S.push((x-1, y))
elif y < n-1 and maze[x][y+1] == 0 and mark[x][y+1] == 0:    # 동쪽 조사
   mark[x][y+1] = 1
   S.push((x, y+1))
elif x < m-1 and maze[x+1][y] == 0 and mark[x+1][y] == 0:    # 남쪽 조사
   mark[x+1][y] = 1
   S.push((x+1, y))
elif y > 0 and maze[x][y-1] == 0 and mark[x][y-1] == 0:      # 서쪽 조사
   mark[x][y-1] = 1
   S.push((x, y-1))
else:      # 이동 가능한 위치가 없으면 후퇴한다.
   S.pop()
```

인접한 방향의 개수에 대한 확장성을 고려하여 구현하는 경우, 편의를 위해서 북동남서의 각 방향에 대해 0, 1, 2, 3의 방향 코드를 부여할 수 있습니다. 방향 코드를 사용하면, 이동 가능한 각 위치에 대한 검사를 for 문에서 수행할 수 있습니다. 이때, 이동 후의 좌표를 계산하기 위해 리스트 move_row와 move_col을 다음과 같이 구성하고,

```
move_row = [-1, 0, 1, 0]     # 북, 동, 남, 서 방향의 행 첨자 변화
move_col = [0, 1, 0, -1]     # 북, 동, 남, 서 방향의 열 첨자 변화
```

방향 코드 idx로 이동한 후의 좌표 (r, c)는 다음과 같이 계산됩니다.

```
r = x + move_row[idx]
c = y + move_col[idx]
```

프로그램 5.4는 알고리즘 4.2를 구현한 파이썬 프로그램을 보여줍니다. 프로그램 5.5의 함수 find_path()는 미로 데이터 maze와 입구와 목적지 좌표를 가진 튜플 enter와 destination을 전달받아 경로를 찾습니다. 만일 경로가 존재하면 경로 정보를 담은 리스트를 반환합니다. 경로가 존재하지 않은 경우는 None을 반환합니다. 이 함수는 파이썬 리스트를 직접 스택으로 사용합니다.

프로그램 5.5: 미로 문제를 위한 파이썬 프로그램

```
def find_path(maze, enter, destination):
    """스택을 이용하여 미로에서 길을 찾는 프로그램:
    미로 maze의 입구 enter에서 목적지 destination까지 경로가 존재하면
    입구부터 목적지까지 경로 리스트를 반환하고, 없으면 None을 반환한다."""
    m, n = len(maze), len(maze[0])     # 미로의 행과 열의 크기
    mark = [[0]*n for _ in range(m)]   # mark 생성
    move_row = [-1, 0, 1, 0]
    move_col = [0, 1, 0, -1]
    stack = []                         # 공백 스택 생성
    stack.append(enter)                # 스택에 입구 좌표 삽입
    mark[enter[0]][enter[1]] = 1       # 입구 방문 기록

    while len(stack) > 0:              # 스택이 공백이 아니면 반복함
        x, y = stack[-1]               # 현재 위치 (x, y)
        if (x, y) == destination:      # 목적지 도달
            return stack               # 입구에서 목적지까지의 경로 반환
        found_next_move = False        # 이동 가능한 인접 위치 발견 여부
        for idx in range(4):           # 각 인접 위치 검사
            r = x + move_row[idx]      # 인접 위치 (r, c)
            c = y + move_col[idx]
            if not(0 <= r < m and 0 <= c < n):       # 미로 벗어남
                continue
            if maze[r][c] == 0 and mark[r][c] == 0:  # 이동 가능 위치
                stack.append((r, c))
                mark[r][c] = 1
                found_next_move = True
                break
        if not found_next_move:        # 이동 가능한 위치가 없으면
            stack.pop()                # 직전 위치로 후퇴함

    return None                        # 목적지까지 경로가 존재하지 않음
```

프로그램 5.5의 수행 시간을 살펴보겠습니다. 프로그램의 14줄에서 현재 위치가 목적지인가를 검사하는 비교 연산(`(x, y) == destination`)을 알고리즘 성능 분석을 위한 지배적 연산으로 둘 수 있습니다. `mark` 행렬로 인해 미로의 각 위치는 기껏해야 한 번씩 검사되므로, 이 비교 연산의 총 수행 횟수는 미로의 크기에 해당하는 `m×n` 이하입니다, 여기서 `m`과 `n`은 각각 미로의 행과 열의 크기입니다. 따라서 `find_path()`의 수행 시간은 O(`mn`)입니다.

5.5 사례 연구: 편집기

크로아티아 프로그램 경시대회에 출제되었던 편집기(editor) 문제[11]를 살펴보겠습니다.

영어 소문자만으로 구성된 한 줄짜리 텍스트가 주어져 있다. 커서(cursor)는 텍스트의 맨 앞(첫 번째 문자의 왼쪽), 맨 끝(마지막 문자의 오른쪽), 또는 인접한 두 문자 사이에 위치할 수 있다. 텍스트를 편집하기 위한 명령이 들어오면 차례대로 텍스트에 적용해야 한다. 편집 명령은 다음과 같다.

`L`	커서를 왼쪽으로 한 칸 옮긴다. (커서가 맨 앞에 있으면 무시함)
`D`	커서를 오른쪽으로 한 칸 옮긴다. (커서가 맨 끝에 있으면 무시함)
`B`	커서의 왼쪽 문자를 삭제한다. (커서가 맨 앞에 있으면 무시함)
`P $`	커서의 왼쪽에 문자 $를 삽입한다. (문자 $는 영어 소문자임)

편집 작업을 수행하기 전, 초기에는 텍스트의 맨 끝에 커서가 위치한다. 주어진 편집 명령을 모두 수행한 후에 텍스트의 상태를 출력하는 프로그램을 작성하시오.

입력 데이터의 첫 줄에는 텍스트가 주어진다. 텍스트의 길이는 100,000 이하이다. 둘째 줄에는 정수 `n`이 주어진다. `n`은 1 이상 500,000 이하이다. 그다음 `n`개의 줄에는 편집 명령이 한 줄에 하나씩 주어진다. 프로그램은 입력으로 주어진 편집 명령을 모두 수행한 후의 텍스트를 출력한다. 프로그램의 최대 수행 시간은 1초로 제한된다. 입력과 출력의 예는 다음과 같다.

---

11) 출처: Croatian Highschool Competitions in Informatics 2004의 National Competition #1 - Juniors 2번 문제

입력	출력
abcd 3 P x L P y	abcdyx

입력	출력
abc 9 L L L L L P x L B P y	yxabc

입력	출력
dmih 11 B B P x L B B B P y D D P z	yxz

위 문제를 해결하는 방법은 다양합니다. 텍스트를 파이썬 리스트에 저장한 후에 단순하게 편집 명령을 수행하는 프로그램은 "시간 초과(Time Limit Error)"라는 결과를 받게 될 것입니다. 편집 명령 중에 삭제 명령(B)과 삽입 명령(P)은 문자들의 이동 작업이 동반되기 때문에 최악의 경우 텍스트의 길이에 대한 선형 시간이 소요됩니다. 텍스트의 최대 길이가 100,000이고 편집 명령이 최대 500,000개 주어지므로, 배열 기반의 알고리즘은 최악의 경우 5×10^{10} 이상의 문자 이동 연산을 수행해야 합니다. 이 작업을 위해서는 수행 제한 시간인 1초를 훨씬 초과하는 시간이 필요합니다. 그러므로 이 문제를 해결하기 위해서는 자연스럽게 삽입과 삭제를 상수 시간에 수행하는 연결리스트 기반의 알고리즘을 고려하게 됩니다. 커서라는 존재 덕분에 추가적인 탐색 연산 없이 삽입과 삭제를 수행할 수 있으므로 연결리스트를 이용한 알고리즘은 모든 편집 명령을 상수 시간에 수행할 수 있습니다. 따라서 1초라는 제한 시간을 만족하는 프로그램을 구현할 수 있습니다. 연결리스트에 관한 내용은 7장에서 자세하게 다룹니다. 여기서는 스택을 이용하여도 이 문제를 효율적으로 해결할 수 있다는 것을 보여주려고 합니다.

편집 대상인 텍스트는 커서의 왼쪽 구간 `left_text`와 오른쪽 구간 `right_text`로 나눌 수 있습니다. 모든 편집 명령은 커서 위치에서 발생합니다. 'L' 명령이 수행되면, `left_text`의 오른쪽 끝 문자가 삭제되어 `right_text`의 왼쪽 끝에 추가됩니다. 'D' 명령이 수행되면, 반대로 `right_text`의 왼쪽 끝 문자가 삭제되어 `left_text`의 오른쪽 끝에 추가됩니다. 'B' 명령이 수행되면 `left_text`의 오른쪽 끝 문자가 삭제되고, 'P' 명령이 수행되면 `left_text`의 오른쪽 끝에 문자가 추가됩니다. 따라서 `left_text`와 `right_text`는 후입선출(LIFO)이라는 스택의 특성을 가지는 것을 볼 수 있습니다. 표 5.4는 입력 텍스트 "abcd"에 대해서, 6개의 편집 명령 (L, L, B, P x, D, P y)가 주어질 때, 편집 명령을 차례로 수행하는 과정을 보여

줍니다. 최종 생성된 텍스트는 left_text와 right_text를 합친 “axcyd”입니다.

표 5.4: 커서를 기준으로 분리된 두 텍스트에서 편집 명령을 수행하는 과정

편집 명령	left_text	right_text	비고
-	abcd		입력 텍스트
L	abc	d	커서 왼쪽 한 칸 이동
L	ab	cd	커서 왼쪽 한 칸 이동
B	a	cd	커서 왼쪽 문자 삭제
P x	ax	cd	커서 왼쪽에 x 삽입
D	axc	d	커서 오른쪽 한 칸 이동
P y	axcy	d	커서 왼쪽에 y 삽입

두 개의 스택을 이용하여 편집기 문제를 해결한 프로그램이 프로그램 5.5에 나와 있습니다. 프로그램에서 파이썬 리스트 left와 right를 스택으로 사용합니다. 리스트 left에는 텍스트의 왼쪽 구간을 저장하고, 리스트 right에는 텍스트의 오른쪽 구간을 역순으로 저장합니다.

프로그램 5.6: 편집기 문제를 위한 파이썬 프로그램

```
1  left = list(input())        # 텍스트의 왼쪽 구간, 초기에 커서가 맨 끝에 있음
2  right = []                  # 텍스트의 오른쪽 구간, 초기에 공백 리스트
3  n = int(input())            # 편집 명령의 개수
4
5  for _ in range(n):                  # 편집 작업 수행
6     op = input()                     # 편집 명령 입력
7     if op == 'L':                    # 커서 왼쪽 이동
8        if len(left) > 0:             # left가 공백이 아니면
9           right.append(left.pop())   # left에서 삭제한 문자를 right에 삽입
10    elif op == 'D':                  # 커서 오른쪽 이동
11       if len(right) > 0:            # right가 공백이 아니면
12          left.append(right.pop())   # right에서 삭제한 문자를 left에 삽입
13    elif op == 'B':                  # 커서 왼쪽 문자 삭제
14       if len(left) > 0:
15          left.pop()
16    else:                            # ‘P $’ 형태의 명령
17       x, y = op.split()             # 삽입 명령 토큰 분리
18       left.append(y)                # 커서의 왼쪽에 문자 삽입
19
20 right.reverse()                    # right를 원래 순서로 바꿈
21 print(''.join(left+right))         # left와 right를 이어 붙인 문자열 출력
```

표 5.4의 수행 과정을, 프로그램 5.5의 프로그램이 두 스택을 사용하여 수행하는 모습을 그림 5.10에서 볼 수 있습니다. 프로그램에서 `for` 블록 내의 모든 명령은 상수 시간에 수행되므로, `for` 문의 수행 시간은 편집 명령의 수 `n`에 대한 선형 시간입니다. 또한 프로그램의 마지막 줄에서 사용된 `join` 연산도 선형 시간에 수행됩니다. 그러므로, 최종적으로 생성되는 텍스트의 길이를 `m`이라고 할 때, 프로그램 5.5의 시간복잡도는 $O(n+m)$으로 분석됩니다.

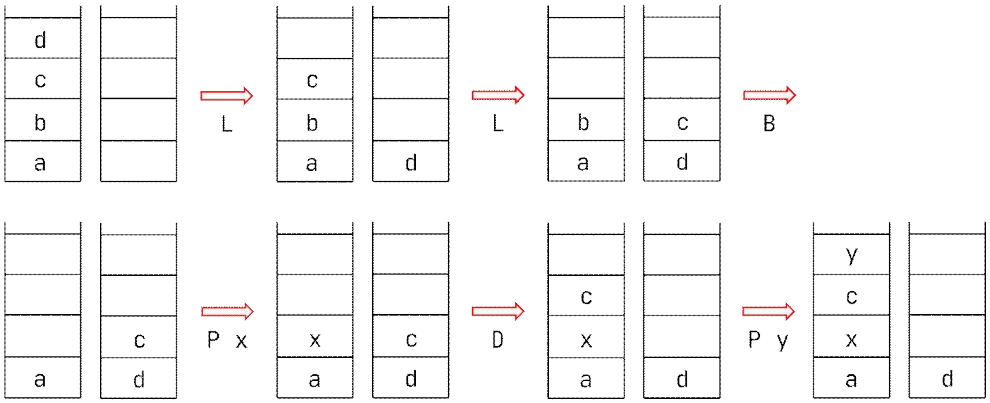


그림 5.10: 표 5.4의 수행 과정을 두 스택을 사용하여 수행함

연습문제

1. 알고리즘에서 일반적인 선형리스트를 사용할 수 있음에도 불구하고, 굳이 스택 자료구조를 사용하는 이유를 쓰시오.

2. 다음 스택 연산들이 공백 스택에서부터 순서대로 수행될 때, 각 연산이 완료된 후 스택의 상태를 가장 아래 원소부터 순서대로 쓰시오.

   `push(A), push(B), pop(), push(C), push(D), pop()`

3. 다음의 자료구조를 추가로 사용하여 스택 S에 들어있는 원소들을 역순으로 다시 스택 S에 저장하는 방법을 제시하시오.
   (1) 두 개의 스택
   (2) 한 개의 스택과 몇 개의 변수

4. 한 개의 스택과 몇 개의 변수만 사용하여 스택 S에 저장된 숫자들을 오름차순으로 다시 스택 S에 저장하는 알고리즘을 설계하고 시간복잡도를 분석하시오.

5. 스택에 들어있는 모든 원소를 제거하는 재귀 함수를 작성하시오.

6. 객차 재배치 문제에서 4대의 객차가 입국 쪽에서 들어올 때, 출구로 나가는 객차 순서를 (1, 2, 3, 4)로 만들 수 없는 입구 쪽 객차 순서를 모두 나열하시오.

7. 다음 중위 수식을 후위 수식으로 변환하시오.
   (1) `a + b * c`
   (2) `a * b + c`
   (3) `(a - b) * (c + d - e) / (f + g)`
   (4) `(a * (b + c) * d + e) + f`

8. 다음 후위 수식을 중위 수식으로 변환하시오.
   (1) a b + c d * -
   (2) a b c * c d - / +
   (3) a b c d e - * + /
   (4) a b c d + * + e f - +

9. 프로그램 5.3의 Stack 클래스의 구현에서 스택이 공백이면 pop()과 peek() 메소드는 None을 반환한다. 이 두 메소드가 None을 반환하는 것 대신에, 예외(exception)를 발생시키도록 수정하시오.

10. 프로그램 5..3의 Stack 클래스에 스택의 크기, 즉 저장된 원소의 개수를 반환하는 size() 메소드를 추가하시오.

11. 그림 5.8의 미로에 대해 이동 가능한 인접 위치를 찾을 때, 북서·남·동의 순서로 조사한다면, 미로에서 길을 찾는 과정이 어떻게 되는지를 표 5.3과 같은 형식으로 나타내시오.

12. 미로 문제에서 이동 방향을 동, 서, 남, 북 네 방향으로 한정하였다. 이를 북, 북동, 동, 남동, 남, 남서, 서, 북서의 여덟 방향으로 확장할 수 있도록 프로그램 5.4를 수정하시오.

13. 주어진 문자열이 회문(palindrome)인지 검사하는 문제에 대해, 재귀 함수가 아니라 스택을 직접 사용하는 프로그램을 작성하시오.

14. 4.4절의 하노이 탑 문제를 위한 재귀적 해법을 재귀 함수가 아닌 스택을 직접 관리하는 반복 구조 프로그램으로 구현하시오.

15. [실습 문제] 다음 왼쪽 그림과 같은 이진 영상 파일이 있다고 하자. 이 파일에는 여러 개의 물체에 대한 이미지가 이진화되어 있다. 이 파일에서 물체 부분은 1로 표현되고, 배경 부분은 0으로 표현된다. 이 영상에서 물체를 검출하여 몇 개의 물체가 존재하는지를 조사하려고 한다. 동서남북의 4방향으로 1이 인접해 있으면 같은 물체로 간주한다. 왼쪽 그림에는 모두 5개의 물체가 있다. 오른쪽 그림은 독립된 물체를 구분하기 위해 각 물체 부분에 동일한 번호로 표시한 것이다. 이진 영상 파일을 입력받아 물체를 검출하는 프로그램을 작성하시오.

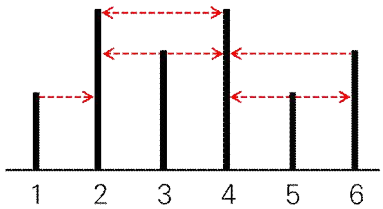
	0	1	2	3	4	5	6
0	0	1	1	0	1	0	1
1	0	1	0	0	1	1	1
2	1	1	1	0	1	0	0
3	0	1	0	0	0	1	0
4	1	0	0	0	1	1	1
5	1	0	1	1	0	1	0

	0	1	2	3	4	5	6
0		1	1		2		2
1		1			2	2	2
2	1	1	1		2		
3		1				3	
4	4				3	3	3
5	4		5	5		3	

입력으로 이진 영상의 행과 열의 크기 n, m이 주어진다, 1 ≤ n, m ≤ 1,000. 두 번째 줄부터 n 줄에 걸쳐 한 줄에 한 행씩 0과 1이 m개씩 주어진다. 각 숫자 사이에는 한 개의 빈칸이 있다. 출력으로 입력 영상에서 검출된 물체의 개수를 출력한다.

입력의 예	출력의 예
6 7 0 1 1 0 1 0 1 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 1 0 0 0 1 0 1 0 0 0 1 1 1 1 0 1 1 0 1 0	5

16. [실습 문제] n개의 탑이 일렬로 세워져 있다. 탑의 맨 위층에서 양방향으로 총을 쏘았을 때, 가장 많은 총알과 부딪힌 탑을 찾으려고 한다. 예를 들어 다음 그림처럼 6개의 탑이 주어져 있다고 하자. 2번 탑은 3개의 총알을, 4번 탑은 4개, 6번 탑은 1개의 총알과 부딪히게 되고, 1번과 5번 탑은 어떤 총알과도 부딪히지 않는다. 따라서 가장 많은 수의 총알과 부딪힌 탑은 4번 탑이다. 탑의 높이가 같은 경우는 서로 총알을 맞게 된다는 점에 유의하시오. 입력으로 주어진 n개의 탑의 높이가 주어질 때, 가장 많은 총알과 부딪힌 탑을 찾아서 총알을 맞은 횟수를 출력하는 프로그램을 작성하시오.



입력의 첫째 줄에 탑의 개수를 나타내는 정수 n이 주어진다. n은 2 이상 50,000 이하이다. 둘째 줄에는 탑들의 높이를 나타내는 n개의 정수가 빈칸을 사이에 두고 주어진다. 탑의 높이는 1 이상 1,000,000 이하이다.

입력의 예	출력의 예
6 2 4 3 4 2 3	4

입력의 예	출력의 예
5 1 2 3 4 5	1

17. [실습 문제] 다음은 스택을 이용하여 중위 수식을 후위 수식으로 변환하는 알고리즘이다. 단, 여기서 다루는 연산자는 이항 연산자이며 결합법칙이 왼쪽에서 오른쪽 순서라고 가정한다.

```
1. 공백 스택을 생성한다.
2. 중위 수식의 각 토큰을 왼쪽부터 차례대로 조사하면서 다음을 수행한다.
    ① 토큰이 피연산자이면, 즉시 출력한다.
    ② 토큰이 왼쪽 괄호이면, 스택에 삽입한다.
    ③ 토큰이 오른쪽 괄호이면, 스택에서 왼쪽 괄호가 나올 때까지 모든 연산자를 삭제하여
       출력한다.
    ④ 토큰이 연산자이면, 스택이 공백이거나 스택 top 연산자의 우선순위가 토큰보다 더
       높을 때까지 스택에서 연산자를 삭제하여 출력한 후, 토큰 연산자를 스택에 삽입한다.
       이때, 왼쪽 괄호는 스택에서 우선순위가 가장 높은 연산자로 간주한다.
3. 스택에 남아 있는 모든 연산자를 모두 삭제하여 출력한다.
```

위 알고리즘을 이용하여, 중위 수식을 후위 수식으로 변환하여 출력하는 프로그램을 작성하시오. 입력으로 들어오는 중위 수식은 다음의 조건을 따른다.

① 피연산자는 한 자릿수 정수로 주어진다.
② 연산자는 사칙 연산자(+, -, *, /)만 사용한다.
③ 모든 괄호는 소괄호이다.
④ 각 토큰(연산자, 괄호, 피연산자) 사이에는 빈칸이 없다.
⑤ 중위 수식은 항상 올바른 수식으로 입력된다.

변환된 후위 수식의 출력 시, 토큰 사이에 한 개의 빈칸을 넣어서 출력한다.

입력의 예	출력의 예
2*5+3	2 5 * 3 +

입력의 예	출력의 예
1/(2-3+4)*(5-6)*7	1 2 3 - 4 + / 5 6 - * 7 *

제6장

큐 자료구조

6.1 큐의 개념
6.2 큐의 활용
6.3 큐의 구현과 deque 모듈
6.4 사례 연구: 미로에서 최단 경로 찾기
6.5 사례 연구: 요셉 문제
연습문제

학습 목표

- 큐의 개념을 이해하고, 큐의 동작과 큐 연산의 종류와 기능을 설명할 수 있다.
- 원형 큐 방식을 이용하여 Queue 클래스를 구현할 수 있다.
- collections.deque를 사용하여 Queue 클래스를 구현할 수 있다.
- 큐를 활용하여 무선조정 자동차 문제와 부분합 구간 문제를 해결할 수 있다.
- 요셉 문제를 해결하는 다양한 해법을 이해하고, 큐 시뮬레이션 방식을 구현할 수 있다.
- 미로에서 최단 경로를 찾는 문제처럼 큐를 이용하여 해결해야 하는 문제의 특성을 파악할 수 있고, 문제 해결을 위해 큐를 이용한 알고리즘을 설계하고 구현할 수 있다.

6.1 큐의 개념

큐(queue)는 대기열을 의미합니다. 극장이나 경기장에서 표를 구매하기 위해 줄을 서 있는 사람들을 떠올려 보세요. 새로 도착한 사람은 줄의 맨 뒤에 서고, 가장 먼저 도착하여 맨 앞에 서 있는 사람은 가장 먼저 표를 구매하고 줄에서 빠져나갑니다. 이처럼 큐는 여러 개의 작업을 처리해야 할 때, 도착한 순서대로 하나씩 작업을 처리하기 위한 목적으로 이용됩니다. 이와 마찬가지로 물건들의 저장소로서의 큐는 가장 먼저 저장한 물건을 가장 먼저 빼내는 방식으로 운영되는 저장소를 의미합니다. 그래서 큐를 **선입선출(First In First Out; FIFO)** 구조라고 부르기도 합니다.

자료구조로서의 큐는 스택처럼 제한된 기능을 가진 선형리스트입니다. 큐에서의 삽입과 삭제는 스택과는 다르게 리스트의 양쪽 끝에서 발생하며, 삽입은 리스트의 뒤쪽(rear)에서, 삭제는 리스트의 앞쪽(front)에서만 수행합니다.

그림 6.1은 큐의 추상적인 모습을 보여줍니다. 큐의 맨 앞쪽, 즉 가장 먼저 큐에 들어온 원소의 위치를 **front**, 큐의 맨 끝, 즉 마지막으로 큐에 들어온 원소의 위치를 **rear**이라고 부릅니다. 큐에 원소를 삽입하는 연산을 보통 **enqueue**, front 위치의 원소(줄여서 **front 원소**)를 큐에서 빼내는 연산을 **dequeue**라고 부릅니다. 큐에 원소가 하나도 없으면 그 큐를 **공백 큐(empty queue)**라고 부릅니다.

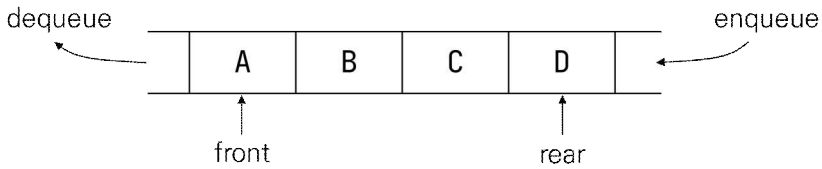


그림 6.1: 큐

예를 들어, `enqueue(item)`는 큐에 원소 `item`을 삽입하는 연산이고, `dequeue()`는 큐에서 front 원소를 삭제하여 반환하는 연산이라고 합시다. 그림 6.2는 공백 큐에서 다음과 같은 순서로 큐 연산들이 수행될 때 큐의 상태가 변화하는 모습을 보여줍니다.

`enqueue(A), enqueue(B), dequeue(), enqueue(C), enqueue(D), dequeue()`

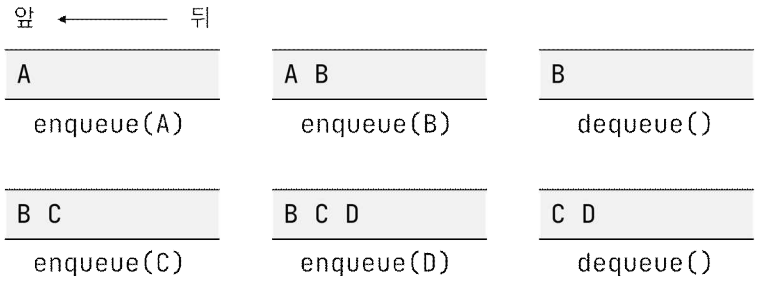


그림 6.2: 큐에서 삽입과 삭제 연산의 수행 예

큐 구조는 우리 일상에서 쉽게 찾아볼 수 있으며, 컴퓨터 시스템에서도 광범위하게 사용됩니다. 컴퓨터 운영체제는 CPU 스케줄링에서 큐를 사용하여 프로세스를 순차적으로 실행시키고, 입력 및 출력 관리를 위해 **데이터 버퍼**라고 불리는 큐를 이용합니다. 또한, 네트워크 프린터나 웹 서버와 같은 컴퓨팅 장치에서도 요청을 순차적으로 처리하기 위해 큐를 이용합니다. 나아가, 컴퓨터 알고리즘에서도 큐는 체계적인 탐색과 효율적인 문제 해결을 위해 폭넓게 활용됩니다.

이제, 큐의 동작 원리를 이해하는 데 도움이 되는 차량 재배치 문제를 살펴보겠습니다.

■ 차량 재배치 문제

경주용 차량을 경기장에 순서대로 입장시키기 위해서 경기장 옆에 그림 6.3과 같은 도로 구역이 마련되어 있다. 도로는 모두 1차선 일방 도로이다. K대의 차량이 입구 쪽에서 들어오는데, 차량은 1부터 K까지의 번호로 구분된다. 경기장 관리원은 이들 차량이 번호 순서대로 출구 쪽으로 나가도록 교통 정리를 해야 한다. 예를 들어, 입구 쪽에서 3대의 차량이 (2, 1, 3) 순서로 들어오면, 출구 쪽으로 (1, 2, 3) 순서로 내보내야 한다. 도로 구역에는 돌아가는 길이 있는데, 이 길에 들어간 차량은 일단 대기하고 있어야 한다. 입구 쪽 차량 한 대를 그대로 출구로 보내는 작업을 **pass**, 돌아가는 길로 보내는 작업을 **wait**, 돌아가는 길에서 대기하고 있는 차량 한 대를 출구로 보내는 작업을 **exit**이라고 하자. (2, 1, 3) 순서로 들어오는 경우, (**wait**, **pass**, **exit**, **pass**) 또는 (**wait**, **pass**, **wait**, **exit**, **exit**) 순서로 작업을 수행하면 된다. 차량 재배치가 가능한 경우, 차량 재배치를 위한 작업 순서를 나열하시오.

돌아가는 길에 들어간 차량은 순서대로 빠져나오므로 돌아가는 길은 큐에 해당한다는 것을 알 수 있습니다. 입구 쪽에서 차량이 (2, 3, 1, 4) 순서로 들어온다면, (**wait**, **wait**, **pass**, **exit**, **exit**, **pass**) 또는 (**wait**, **wait**, **pass**, **wait**, **exit**, **exit**, **exit**)의 순서로 작업을 진행하면 됩니다.

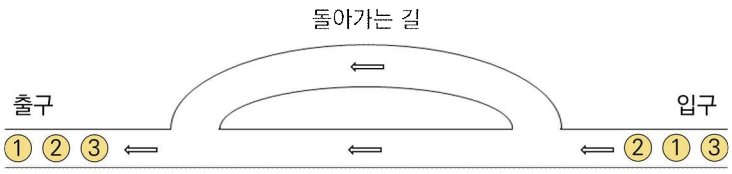


그림 6.3: 차량 재배치 문제

만약 객차가 (3, 2, 1, 4) 순서로 들어온다면 차량 재배치가 가능할까요? 1번 차량을 가장 먼저 출구로 내보내야 하므로, 그 앞의 3번, 2번 차량은 두 번의 wait 작업으로 돌아가는 길에 대기시켜야 합니다. 그다음 2번 차량을 빼내어야 하는데, 먼저 돌아가는 길에 들어온 3번 차량 때문에 2번 차량을 빼낼 수 없습니다. 그러므로 재배치 작업은 불가능합니다.

6.2 큐의 활용

큐도 스택과 마찬가지로 선형리스트에 비해 기능이 단순하므로 큐에서 필요한 연산의 수는 많지 않습니다. 다음 추상자료형 6.1에는 큐에서 필요한 최소한의 연산만으로 정의한 추상자료형 명세가 나와 있습니다.

추상자료형 6.1: Queue

```
class Queue:
   def __init__(self):        # 공백 큐를 생성한다.
   def empty(self):           # 큐가 공백이면 True, 아니면 False를 반환한다.
   def enqueue(self, item):   # 큐에 원소 item을 삽입한다.
   def dequeue(self):         # 큐의 front 원소를 삭제한 후 반환한다.
                              # 공백 큐에 대해 호출되면 None을 반환한다.
   def first(self):           # 큐의 front 원소를 반환한다.
                              # 공백 큐에 대해 호출되면 None을 반환한다.
```

메소드 dequeue()와 first()의 경우, 큐가 공백인 상태에서 호출되면 정상적으로 수행될 수 없으므로 오류가 발생해야 합니다. 이러한 오류는 대부분 알고리즘의 잘못된 설계로 인해 발생합니다. 이 경우, 예외를 발생시키거나 오류 메시지를 출력하고 프로그램을 종료하는 것이 일반

적이지만, 이 책에서는 오류에 대한 처리를 사용자에게 일임하기 위해 None을 반환하도록 정의합니다. 메소드 enqueue()의 경우, 큐를 구현할 때 정적 배열을 이용하면 **큐 오버플로(queue overflow)**에 대한 대비를 해야 합니다. 나중에 큐의 구현에서 다시 소개되겠지만, 큐의 모든 연산은 상수 시간에 수행됩니다. 추상자료형 Queue의 구현 결과로 상수 시간에 수행되지 않는 연산이 존재한다면, 이는 큐를 부적절하게 구현한 것임을 의미합니다.

이제, 큐를 활용하여 문제를 해결하는 몇 가지 사례를 소개합니다. 먼저, 다음 무선조종 자동차 문제부터 살펴봅니다.

■ 무선조종 자동차 문제

리모컨으로 조종하는 자동차(RC car)가 있다. 리모컨에는 두 개의 버튼 A와 B가 있는데, 버튼 A를 누르면 자동차가 a 미터 전진하고, 버튼 B를 누르면 b 미터 후진한다. 이제, 일직선을 따라 자동차를 이동시키려고 한다. 자동차를 목표 지점에 정확하게 위치시키기 위해 눌러야 할 버튼의 최소 횟수를 출력하는 프로그램을 작성하시오. 단, 자동차가 출발선 위치보다 뒤로 이동하거나 목표 지점을 넘어가지 않아야 한다. 만약 목표 지점에 자동차가 정차할 수 없으면 프로그램은 -1을 출력해야 한다.

예를 들어, 목표 지점이 9 미터이고 a=5, b=2라면, 버튼 A와 B를 각각 3번씩 총 6번 즉, (A, B, B, A, B, A) 또는 (A, B, A, B, B, A) 순으로 누르는 것이 답이다(그림 6.4 참조).

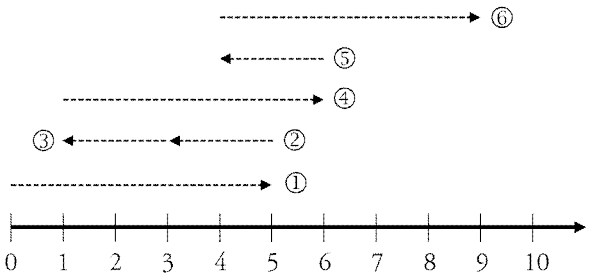


그림 6.4: (A, B, B, A, B, A) 순으로 자동차 이동

큐를 이용한 해법을 살펴보기 전에, 해의 존재 여부에 대해 알아봅니다. 자동차의 목표 지점을 t라고 하고, 목표 지점 t에 도달하기 위해 두 버튼 A와 B를 누르는 횟수를 각각 x와 y라고 합시다. 다섯 개의 변수 t, a, b, x, y에 대해서 다음의 식이 성립합니다.

```
t = ax - by
```

변수 t, a, b의 값이 주어질 때, 이 식은 선형 디오판토스 방정식에 해당합니다. a=5, b=2라면 5와 2는 서로 소(최대공약수가 1)이므로, 이론적으로 모든 정수 목표 지점 t에 대해 이 식을 만족하는 정수 해 x와 y를 찾을 수 있습니다[12]. 한편 a와 b의 값에 따라서는 자동차가 목표 지점에 정확하게 정차하지 못하는 경우도 발생합니다. 예를 들어, a와 b가 모두 짝수라면 거리가 홀수인 위치에 자동차를 위치시킬 수 없습니다.

이제, 큐를 이용하여 문제를 해결하는 방법을 알아보겠습니다. 큐를 사용한 시뮬레이션으로 버튼을 누르는 모든 동작에 대해 자동차가 이동된 상태를 구할 수 있습니다. 이 작업을 위해 공백 큐와 1차원 배열(파이썬 리스트)을 준비합니다.[13] 큐에는 버튼을 누를 때 도달하는 모든 위치 정보를 삽입하고, 배열에는 해당 위치에 도달하기 위한 최소 클릭 수를 저장합니다. 배열의 첨자는 이동 거리를 표현하고, 이를 위해 초기에 0으로 채워진 크기가 t+1인 배열을 생성합니다. 알고리즘 6.1에는 전진 거리 a와 후진 거리 b, 그리고 목표 지점 t가 주어질 때 최소 클릭 수를 구하는 알고리즘이 나와 있습니다.

알고리즘 6.1: 큐를 사용하여 최소 클릭 수를 구하는 알고리즘

```
1  # t: 목표 지점, a: 전진 거리, b: 후진 거리
2  공백 큐 Q를 생성한다.
3  리스트 D = [0]*(t+1)을 생성한다.          # 방문 여부 및 최소 클릭 수 기록
4  Q에 초기 위치 0을 삽입한다.
5  while Q가 공백이 아닌 경우:
6     Q에서 front 원소 p를 삭제한다.
7     if p == t: 최소 클릭 수 D[p]를 출력하고 종료한다.
8     if p + a <= t and D[p+a] == 0:      # p+a 위치로 전진 가능
9        Q에 p+a를 삽입하고, D[p+a]에는 D[p]+1을 저장한다.
10    if p - b > 0 and D[p-b] == 0:       # p-b 위치로 후진 가능
11       Q에 p-b를 삽입하고, D[p-b]에는 D[p]+1을 저장한다.
12 자동차가 목표 지점 t에 도달할 수 없음을 출력한다.
```

예를 들어, 목표 지점 t=9, 전진 거리 a=5, 후진 거리 b=2라고 할 때, 알고리즘 6.1의 수행 과정은 다음과 같습니다. 알고리즘의 수행 중 큐로부터 삭제한 위치에서 버튼을 눌렀을 때, 이동 가능 구간을 벗어나거나 이미 도달한 적이 있는 위치라면 해당 버튼을 누르지 않습니다.

---

12) 선형 디오판토스 방정식의 정수 해는 a와 b의 최대공약수가 t를 나눌 수 있을 때 존재합니다.
13) 공간복잡도를 고려한다면 배열보다는 해싱(파이썬 딕셔너리)을 이용하는 것이 더 효율적입니다. 해싱에 관한 내용은 13장에서 소개합니다.

① Q에서 0 삭제, 전진 버튼 클릭 ⇒ Q=(5), D[5]=1
② Q에서 5 삭제, 후진 버튼 클릭 ⇒ Q=(3), D[3]=2
③ Q에서 3 삭제, 전진 버튼, 후진 버튼 클릭 ⇒ Q=(8, 1), D[8]=3, D[1]=3
④ Q에서 8 삭제, 후진 버튼 클릭 ⇒ Q=(1, 6), D[6]=4
⑤ Q에서 1 삭제, 클릭 가능한 버튼 없음 ⇒ Q=(6)
⑥ Q에서 6 삭제, 후진 버튼 클릭 ⇒ Q=(4), D[4]=5
⑦ Q에서 4 삭제, 전진 버튼, 후진 버튼 클릭 ⇒ Q=(9, 2), D[9]=6, D[2]=6
⑧ Q에서 9 삭제 ⇒ 목표 지점 도달, 최소 클릭 수 6

큐를 이용한 탐색 과정에서 큐에 삽입되는 위치의 순서는 버튼 클릭 횟수가 작은 것부터 차례대로 증가하는 순서입니다. 따라서 각 위치에 자동차가 최초로 도달하는 순간의 버튼 클릭 수는 최소 횟수임이 보장됩니다. 알고리즘 6.1의 수행 시간은 최악의 경우 모든 위치가 단 한 번씩 큐에 삽입되므로 $O(t)$입니다. 알고리즘 6.1과 같이 배열(리스트 D)에 버튼 클릭 수를 저장하는 것 대신에, 큐에 (위치, 버튼 클릭 수) 쌍을 저장하면 탐색이 빠르게 종료되는 상황에는 기억 공간 면에서 더 효율적일 수 있습니다. 그러나 최악의 경우에는 큐에 삽입되는 원소의 수가 끝없이 늘어날 수 있다는 점과 자동차가 도달하지 못하는 위치를 판단하기 위해서는 방문 여부 기록을 위한 추가적인 메모리가 필요하다는 단점이 있습니다. 프로그램 6.1은 추상자료형 Queue를 사용하여 알고리즘 6.1을 파이썬으로 구현한 프로그램입니다.

프로그램 6.1: 무선조종 자동차 문제를 위한 프로그램

```
1  def min_clicks(t, a, b):
2     """목표 지점 t, 전진 거리 a, 후진 거리 b일 때,
3     출발점 0에서 목표 지점에 도달하면 최소 클릭 수, 아니면 -1을 반환한다."""
4     Q = Queue()                          # 공백 큐 생성
5     Q.enqueue(0)                         # 큐에 초기 위치 삽입
6     D = [0]*(t+1)                        # 0으로 초기화된 리스트 생성
7     while not Q.empty():                 # Q가 공백이 아니면 반복
8        p = Q.dequeue()                   # Q에서 front 원소 삭제
9        if p == t:                        # 목표 지점 도달
10          return D[p]                   # 최소 클릭수 반환
11       if p+a <= t and D[p+a] == 0:      # 전진 버튼 클릭
12          Q.enqueue(p+a)
13          D[p+a] = D[p]+1
14       if p-b > 0 and D[p-b] == 0:       # 후진 버튼 클릭
15          Q.enqueue(p-b)
16          D[p-b] = D[p]+1
17    return -1                            # 목표 지점 도달 불가능
```

다음으로 수열에서 부분합이 동일한 구간의 개수를 구하는 문제를 살펴봅니다.

■ 부분합 구간의 개수

0보다 큰 n개의 정수로 구성된 수열이 주어져 있다. **부분합**은 수열에서 연속된 구간에 놓인 숫자들의 합이다. 예를 들어, 다음과 같이 8개의 숫자로 구성된 수열을 고려하자.

(3, 1, 4, 2, 1, 3, 1, 6)

이 수열에서 구간 [0, 2]는 (3, 1, 4), 구간 [2, 5] = (4, 2, 1, 3), 구간 [6, 7] = (1, 6)을 나타낸다. 부분합은 구간에 포함된 값들의 합을 의미한다. 위 수열에서 구간 [0, 2]의 부분합은 8, 구간 [2, 5]의 부분합은 10, 구간 [6, 7]의 부분합은 7이다. 양수 K가 추가로 주어질 때, 수열에서 부분합이 K인 구간의 개수를 구하는 프로그램을 작성하시오.

이 문제를 해결하는 단순한 해법으로, 수열에 존재하는 모든 구간에 대해 각각 부분합을 구하여, 부분합이 K인 구간의 개수를 구하는 방법을 고려해 볼 수 있습니다. 구간 [s, t]에서 구간의 시작점 s가 될 수 있는 값은 0부터 n-1까지이고 끝점 t가 될 수 있는 값은 s부터 n-1까지입니다. 따라서 크기가 n인 수열에서 나올 수 있는 구간의 개수는 n(n+1)/2입니다. 그러므로 모든 구간의 부분합을 구하는 방식으로 문제를 해결하는 알고리즘의 시간복잡도는 $\Omega(n^2)$입니다. 프로그램 6.2는 모든 구간에 대한 부분합을 각각 독립적으로 구하는 프로그램으로, 수행 시간은 $O(n^3)$입니다. 반면에 프로그램 6.3은, $O(n^2)$ 시간에 수행되도록, 시작점이 동일한 구간에 대해 기존에 구한 부분합에 구간의 맨 끝 숫자만 더함으로써 효율적으로 부분합을 구하는 프로그램입니다.

프로그램 6.2: $O(n^3)$ 시간에 수행하는 프로그램

```
1  def num_interval(L, K):
2     """리스트 L에서 부분합이 K인 구간의 개수를 반환한다."""
3     count = 0                          # 부분합이 K인 구간의 개수
4     for s in range(len(L)):            # 부분합의 시작점 s
5        for t in range(s, len(L)):      # 부분합의 끝점 t
6           sub_sum = sum(L[s:t+1])     # 구간 [s, t]의 부분합, 선형 시간 소요
7           if sub_sum == K:            # 부분합의 값이 K인 구간 찾음
8              count += 1
9           elif sub_sum > K:           # 부분합이 K보다 크면, 현재 시작점에서는
10             break                    # 더 이상 탐색할 필요 없음
11    return count                       # 부분합이 K인 구간 개수 반환
```

프로그램 6.3: $O(n^2)$ 시간에 수행하는 프로그램

```
def num_interval(L, K):
    """리스트 L에서 부분합이 K인 구간의 개수를 반환한다."""
    count = 0                          # 부분합이 K인 구간의 개수
    for s in range(len(L)):
        sub_sum = 0                    # 부분합의 시작점 s
        for t in range(s, len(L)):     # 부분합의 끝점 t 갱신
            sub_sum += L[t]            # 부분합에 구간의 맨 끝 숫자 더함
            if sub_sum == K:           # 부분합의 값이 K인 구간 찾음
                count += 1
            elif sub_sum > K:          # 부분합이 K보다 크면, 현재 시작점에서는
                break                  # 더 이상 탐색할 필요 없음
    return count                       # 부분합이 K인 구간 개수 반환
```

이제, 큐를 사용하여 문제를 효율적으로 해결하는 전략을 소개합니다. 큐에는 부분합을 계산하려는 현재 구간에 속한 숫자들이 저장됩니다. 큐에 저장된 숫자들의 합이 K보다 크면 큐의 front 원소를 삭제하고, 그렇지 않으면 큐에 그다음 숫자를 삽입합니다. 큐에서 삽입 또는 삭제가 수행될 때 부분합이 K가 되면 구간의 개수를 1 증가시킵니다. 이처럼 큐를 이용하여 변화하는 연속된 구간을 표현하는 기법을 **슬라이딩 윈도우(sliding window)**라고 부릅니다. 프로그램 6.4는 이 전략을 추상자료형 Queue를 사용하여 구현한 프로그램입니다.

프로그램 6.4: 큐를 사용하여 구간의 개수를 구하는 프로그램

```
def num_interval(L, K):
    """리스트 L에서 부분합이 K인 구간의 개수를 반환한다."""
    Q = Queue()                        # 공백 큐 생성
    count = 0                          # 부분합이 K인 구간의 개수 기록
    sub_sum = 0                        # 큐에 저장된 숫자들의 합
    for x in L:
        Q.enqueue(x)                   # 큐에 그다음 숫자 삽입
        sub_sum += x                   # 부분합 갱신
        if sub_sum == K:               # 부분합이 K인 구간 찾음
            count += 1                 # 구간 개수 증가
        elif sub_sum > K:              # 부분합이 K 초과하면
            while sub_sum > K:         # 부분합이 K 이하가 될 때까지 반복
                sub_sum -= Q.dequeue() # 큐에서 원소 삭제와 부분합 갱신
            if sub_sum == K:           # 부분합이 K인 구간 찾음
                count += 1             # 구간 개수 증가
    return count                       # 부분합이 K인 구간 개수 반환
```

예를 들어, 리스트 L = [3, 1, 4, 2, 1, 3, 1, 6]에 대해 K=7인 경우, 프로그램 6.4의 함수 num_interval(L, K)의 수행 과정을 표 6.1에서 볼 수 있습니다.

표 6.1: 프로그램 6.4의 수행 과정, L = [3, 1, 4, 2, 1, 3, 1, 6], K = 7

단계	큐 이벤트	큐 내부	sub_sum	count	비고
1	L[0]=3 삽입	[3]	3	0	
2	L[1]=1 삽입	[3, 1]	4	0	
3	L[2]=4 삽입	[3, 1, 4]	8	0	부분합 초과
4	삭제	[1, 4]	5	0	
5	L[3]=2 삽입	[1, 4, 2]	7	1	
6	L[4]=1 삽입	[1, 4, 2, 1]	8	1	부분합 초과
7	삭제	[4, 2, 1]	7	2	
8	L[5]=3 삽입	[4, 2, 1, 3]	10	2	부분합 초과
9	삭제	[2, 1, 3]	6	2	
10	L[6]=1 삽입	[2, 1, 3, 1]	7	3	
11	L[7]=6 삽입	[2, 1, 3, 1, 6]	13	3	부분합 초과
12	삭제	[1, 3, 1, 6]	11	3	부분합 초과
13	삭제	[3, 1, 6]	10	3	부분합 초과
14	삭제	[1, 6]	7	4	

num_interval(L, K) 함수의 수행 과정에서 리스트 L의 각 숫자가 큐에 한 번씩만 삽입된 후 삭제되고, 삽입과 삭제는 상수 시간에 수행되므로 총 수행 시간은 O(n)입니다.

6.3 큐의 구현과 deque 모듈

스택의 경우에는 파이썬 리스트를 이용하여 간단하게 구현했습니다. 큐도 마찬가지로 파이썬 리스트를 이용하여 손쉽게 구현할 수 있을까요? 파이썬 리스트를 큐 공간으로 사용할 때, 큐에 들어온 순서대로 원소들을 리스트에 차례대로 배치하면, 큐의 front 원소는 리스트의 맨 앞에 위치하고 rear 원소는 리스트의 맨 끝에 위치합니다. 그러면, 큐에 새로 삽입되는 원소 e는 append(e) 메소드를 호출하여 리스트의 끝에 추가시키고, 큐에서 front 원소 삭제는 리스트의 맨 앞 요소를 삭제하는 명령으로 처리하면 됩니다. 이때, 리스트의 pop() 메소드가 아닌

pop(0) 메소드를 호출해야 합니다. 그런데, append(e) 메소드는 상수 시간에 수행되지만, pop(0) 메소드는 선형 시간이 소요됩니다. 앞서 큐의 연산이 상수 시간에 수행되지 않으면 올바르게 구현한 것이 아니라고 언급했었습니다. 결국 파이썬 리스트를 단순히 이용하는 방법은 올바른 큐의 구현이 아니므로, 다른 구현 전략을 고안해야 합니다.

큐의 삽입과 삭제 연산이 상수 시간에 수행되도록 만들기 위해, 리스트에서 front 위치를 유동적으로 두는 전략이 있습니다. 삭제 연산을 수행할 때 front 원소를 리스트에서 실제로 제거하는 대신, 삭제 후에 front 원소가 될 위치를 외부 변수에 저장하는 것입니다. front 원소의 위치(리스트의 첨자)를 저장하는 변수를 f라고 하면, 삭제가 수행될 때마다 f의 값을 1 증가시킵니다. 이 방법은 삽입과 삭제 연산 모두 상수 시간에 수행되도록 해 줍니다. 그림 6.5는 세 번의 큐 삭제 연산이 수행된 후의 리스트 모습을 보여줍니다. 리스트에서 변수 f가 참조하는 위치 이전 구간에는 비록 값이 저장되어 있더라도 큐에는 존재하지 않는 값으로 간주합니다.

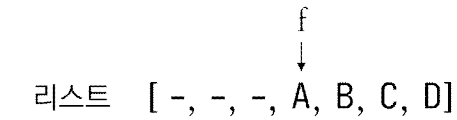


그림 6.5: 세 번의 큐 삭제 연산 후의 모습

이 전략에도 단점이 있습니다. 스택의 경우는 파이썬 리스트의 길이와 스택의 크기가 정확하게 일치하지만, 이 전략을 사용하면 파이썬 리스트의 길이는 삭제가 수행됨에 따라 큐의 크기보다 점점 더 커지게 됩니다. 변수 f가 참조하는 위치 앞쪽의 구간은 그대로 낭비되는 공간이기 때문에, 만약 어떤 알고리즘이 큐에 대한 삽입과 삭제를 빈번하게 수행한다면 메모리 사용의 효율성이 매우 나빠질 수 있습니다.

배열을 기반으로 큐를 표현하는 가장 효과적인 방법은 **원형 큐(circular queue)**라고 불리는 방식을 사용하는 것입니다. 이 경우, 배열은 크기가 고정된 정적 배열을 사용합니다. 원형 큐는 그림 6.6처럼 배열이 원형으로 연결되어 있다고 가정합니다. 즉, 배열의 크기를 M이라고 할 때, 원형 큐는 배열에서 첨자가 M-1인 장소와 첨자가 0인 장소는 인접한 것으로 간주합니다. 원형 큐 표현에서 삽입은 현재 rear 원소의 다음 위치에 새로운 큐 원소를 저장하고, 삭제는 현재 front 원소를 삭제한 후 그다음 위치를 front 원소의 위치로 두는 것입니다. 이렇게 운영하면 배열 공간을 낭비하지 않고 완전히 활용하게 됩니다. 그림 6.6은 M=10이고 front 원소의 위치가 7, rear 원소의 위치가 2인 원형 큐의 상태를 보여줍니다.

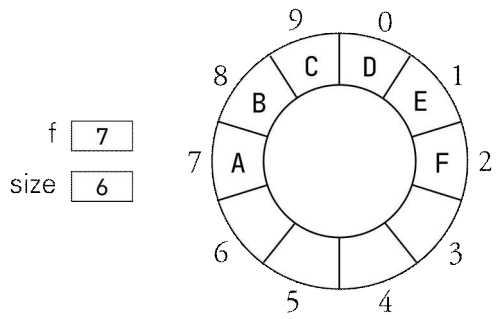


그림 6.6: 원형 큐의 예

원형 큐 표현에서는 항상 front 원소와 rear 원소의 위치를 참조할 수 있어야 합니다. 이를 위해 front 원소와 rear 원소의 위치를 저장하는 두 개의 변수를 사용할 수도 있지만, 이 방법은 큐가 꽉 찬 경우와 공백인 경우를 구분하기 위한 추가적인 전략이 필요합니다. 여기서는 두 개의 변수 `f`와 `size`를 사용하여 각각 front 원소의 위치와 큐 원소의 개수를 저장하는 방식을 사용합니다. `f`와 `size`의 값을 알면 rear 원소의 위치는 쉽게 계산됩니다.

원형 큐에서 다음 원소의 위치 계산은 나머지 연산(modulo operation; 파이썬의 `%` 연산)을 이용하면 편리합니다. 삭제 연산이 수행되어 front 원소의 위치 `f`를 갱신할 때 다음 파이썬 수식을 사용합니다.

```
f = (f + 1) % M
```

또한, 새로 삽입될 rear 원소의 위치는 다음 수식으로 계산합니다.

```
rear = (f + size) % M
```

예를 들어, `M=10`이고 `f=7`이면, `f+1`은 8이고 8을 10으로 나눈 나머지는 8이 됩니다. `f=9`인 경우는 `f+1`이 10이 되며 10을 10으로 나눈 나머지는 0이 되어 결국 배열의 맨 앞 위치를 참조하게 됩니다. `f=7`이고 `size=6`이면 `f+size`는 13이고 13을 10으로 나눈 나머지 3은 새로운 rear 원소의 위치가 됩니다.

원형 큐 표현은 정적 배열을 사용하므로, 큐를 생성할 때 미리 고정된 크기의 배열을 만들어야 합니다. 파이썬 리스트처럼 처음에는 디폴트 크기로 배열을 생성하고 메모리 공간이 부족해질 때마다 2배로 커진 배열을 생성하는 동적 배열 방식을 직접 구현할 수 있습니다. 이때 새로운 배열에 기존 큐 원소들을 복사하는 작업이 동반됩니다. 그러면 큐 연산은 상각 시간복잡도로 분석되어 상수 시간의 복잡도를 가지게 됩니다. 이렇게 구현하는 방식은 연습문제에서 다루고, 여

기서는 정적 배열을 기반으로 하여 좀 더 간단하게 구현하는 방식을 소개합니다. 공백 큐를 생성할 때, 사용자가 필요한 배열의 크기를 미리 요청하는 것입니다. 이러한 요청이 없으면 디폴트 크기를 사용합니다. 만일 배열 공간이 부족해지면, 즉 **큐 오버플로(queue overflow)**가 발생하면 None을 반환하여 사용자가 이에 대한 조치를 수행하도록 합니다. 이렇게 구현된 큐를 추상자료형 6.1의 Queue와 구분하기 위해 ArrayQueue라는 이름을 사용합니다. 프로그램 6.5에는 원형 큐 방식으로 구현한 ArrayQueue 클래스가 나와 있습니다.

프로그램 6.5: 원형 큐 방식으로 구현한 ArrayQueue 클래스

```
1  class ArrayQueue:
2     """원형 큐 방식으로 구현한 큐 클래스"""
3
4     def __init__(self, SIZE = 100):    # 공백 큐를 생성한다.
5        self._data = [None] * SIZE      # 큐 원소 저장을 위한 리스트
6        self._size = 0                  # 큐 원소의 개수 저장
7        self._front = 0                 # 리스트에서 front 원소의 위치 저장
8
9     def empty(self):        # 큐가 공백이면 True, 아니면 False를 반환한다.
10       return self._size == 0
11
12    def enqueue(self, item):        # 큐에 원소 item을 삽입한다.
13       if self._size == len(self._data):  # 큐 오버플로 발생
14          return None
15       rear = (self._front + self._size) % len(self._data)
16       self._data[rear] = item
17       self._size += 1
18
19    def dequeue(self):              # 큐의 front 원소를 삭제한 후 반환한다.
20       if self.empty():
21          return None
22       item = self._data[self._front]
23       self._front = (self._front + 1) % len(self._data)
24       self._size -= 1
25       return item
26
27    def first(self):                # 큐의 front 원소를 반환한다.
28       if self.empty():
29          return None
30       return self._data[self._front]
```

ArrayQueue 클래스의 모든 메소드는 상수 시간에 수행되지만, 큐 오버플로에 대한 처리가 필요하다는 단점이 있습니다. 큐 오버플로를 고려하지 않으면서 상각 시간복잡도가 아닌 순수한 시간복잡도로 상수 시간에 수행되도록 구현하려면 7장에서 소개하는 연결리스트를 이용해야 합니다. 연결리스트를 사용하여 직접 큐를 구현하는 내용은 7장에서 다루고 여기서는 파이썬의 deque 모듈을 이용하는 방법을 소개합니다.

■ deque

데크(deque)는 **양방향 큐(double-ended queue)**라고 부르는 자료구조로서, 양쪽 끝에서만 삽입과 삭제가 발생하는 선형리스트입니다. 데크는 스택과 큐의 특성을 모두 가지고 있으므로, 데크를 스택이나 큐로 활용할 수 있습니다. 파이썬에서 제공하는 collections 모듈에는 deque 클래스가 포함되어 있습니다. 이 클래스는 연결리스트 기반으로 구현되어 있어 양쪽 끝에서의 삽입과 삭제 연산이 모두 상수 시간에 수행됩니다. 표 6.2에는 collections.deque 클래스의 주요 연산과 메소드가 나와 있습니다. 여기에 나온 메소드는 모두 상수 시간에 수행됩니다.

표 6.2: collection.deque 클래스의 주요 연산과 메소드

연산/메소드	기능
D = deque()	공백 데크 생성 (from collections import deque 선언 필요)
D.append(item)	데크 D의 맨 끝에 item 삽입
D.appendleft(item)	데크 D의 맨 앞에 item 삽입
D.pop()	데크 D의 맨 끝 원소를 삭제하고 반환
D.popleft()	데크 D의 맨 앞 원소를 삭제하고 반환
D[0]	데크 D의 맨 앞 원소 반환
D[-1]	데크 D의 맨 끝 원소 반환
len(D)	데크 D의 크기 반환

큐 Q가 collections.deque D로 표현되어 있다면, 각 큐 연산과 대응되는 데크 메소드는 다음과 같습니다.

```
Q.enqueue(e) ↔ D.append(e)
Q.dequeue()  ↔ D.popleft()
Q.first()    ↔ D[0]
Q.empty()    ↔ len(D) == 0
```

실험용 프로그램을 작성하거나 프로그래밍 경시대회에서 프로그램을 작성하는 경우는 데크를 직접 사용하는 것이 더 간편할 수 있습니다. 하지만, 중요한 프로그램을 개발하는 과정에서는 추상자료형으로 표현된 큐를 사용하는 것이 권장됩니다. 프로그램 6.6에는 데크 기반으로 구현한 Queue 클래스가 나와 있습니다.

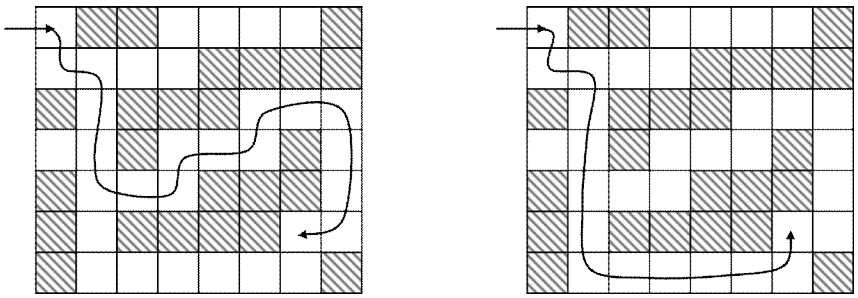
프로그램 6.6: collection.deque를 사용하여 구현한 Queue 클래스

```
1  from collections import deque
2
3  class Queue:
4     """collections.deque를 사용하여 구현한 큐 클래스"""
5
6     def __init__(self):              # 공백 큐를 생성한다.
7        self._data = deque()
8
9     def empty(self):                 # 큐가 공백이면 True를 반환한다.
10       return len(self._data) == 0
11
12    def enqueue(self, item):         # 큐에 원소 item을 삽입한다.
13       self._data.append(item)
14
15    def dequeue(self):               # 큐의 front 원소를 삭제한 후 반환한다.
16       if self.empty():
17          return None
18       return self._data.popleft()
19
20    def first(self):                 # 큐의 front 원소를 반환한다.
21       if self.empty():
22          return None
23       return self._data[0]
```

6.4 사례 연구: 미로에서 최단 경로 찾기

이 절에서는 미로의 입구부터 목적지까지 **최단 경로(shortest path)**를 구하는 알고리즘을 소개합니다. 5장의 사례 연구에서 다룬 미로에서 길 찾기는 미로 내부를 무작정 탐색하다가 목적지에 도달하면 탐색을 종료하는 방식입니다. 이렇게 찾은 길이 가장 짧은 길이 될 가능성은 거의

없습니다. 시간이 더 걸리더라도 미로를 완전히 분석하여 최단 경로를 구하고자 한다면 어떤 방법을 사용해야 할까요? 경로의 길이를 경로에 포함된 좌표의 개수라고 정의해 봅시다. 그림 6.7은 미로 입구에서 목적지까지의 두 경로를 보여줍니다. 그림 6.7 (a)에 표시된 경로는 모두 18개의 칸을 지나므로 길이가 18입니다. 그림 6.7 (b)에 표시된 경로의 길이는 14로서 이 미로에서 최단 경로에 해당합니다. 즉, 길이가 14보다 더 짧은 경로는 존재하지 않습니다.



(a) 경로의 길이 = 18 (b) 경로의 길이 = 14

그림 6.7: 경로의 길이

최단 경로를 찾기 위해서는 미로의 모든 장소를 탐색해야 합니다. 그래야 찾은 경로가 최단 경로가 되는지 알 수 있습니다. 그 결과로 미로의 모든 위치에 대해 입구에서 각 위치까지의 최단 경로가 구해집니다. 그림 6.8은 그림 6.7의 미로에서 통로에 해당하는 각 위치까지의 최단 경로 길이를 표시한 것입니다.

1			6	7	8	9	
2	3	4	5				
	4				12	13	14
6	5		9	10	11		15
	6	7	8				16
	7					14	15
	8	9	10	11	12	13	

그림 6.8: 최단 경로의 길이

먼저, 그림 6.8과 같이 최단 경로의 길이를 구하는 방법을 알아봅니다. 미로 행렬과 동일한 크기의 행렬 mark를 만들고, 이 행렬에 미로의 각 위치까지의 최단 경로 길이를 저장한다고 합

시다. 스택을 이용한 탐색에서는 현재 위치에서 이동할 수 있는 새로운 인접 위치가 발견되면 곧바로 그 위치로 이동합니다. 반면에 최단 경로를 구하기 위해서는 입구에서 가장 가까운 위치부터 차례대로 하나씩 모든 위치를 탐색해야 합니다. 이 작업을 위해 큐가 사용됩니다. 맨 처음에 행렬 mark를 0으로 초기화하고 입구 위치를 큐에 삽입합니다. 그다음, 매번 큐의 front 원소를 삭제한 후 이 위치로부터 이동할 수 있는 인접한 모든 위치들을 큐에 삽입합니다. 큐에서 삭제된 위치가 (x, y)이고 새로 삽입되는 위치가 (r, c)이면, 최단 경로의 길이가 1 증가하므로 mark[r][c]의 값은 mark[x][y]+1이 됩니다.

미로에서 최단 경로를 찾는 알고리즘이 알고리즘 6.2에 나와 있습니다. 알고리즘에서 탐색 도중에 목적지를 만나면 목적지까지의 최단 경로 길이를 알게 되므로 더 이상의 탐색은 필요하지 않습니다. 만약 큐가 공백이 될 때까지 목적지를 만나지 못하면 이 미로에는 길이 없음을 알게 됩니다.

알고리즘 6.2: 미로 문제에서 최단 경로를 찾는 알고리즘

```
1  공백 큐 Q를 생성한다.
2  0으로 초기화된 행렬 mark를 생성한다.
3  입구 좌표 (x, y)를 큐 Q에 삽입한다.
4  mark[x][y]에 1을 배정한다.
5  while 큐 Q가 공백이 아니면:
6     큐 Q에서 front 원소를 삭제하고, 그 위치를 (x, y)라고 하자.
7     if (x, y)가 목적지 좌표이면:
8        mark에 저장된 정보를 이용하여 최단 경로를 출력하고 종료한다.
9     for (x, y)의 각 인접 위치 (r, c)에 대해:
10       if (r, c)가 이동 가능한 새로운 위치이면:
11          mark[r][c] = mark[x][y] + 1
12          큐 Q에 (r, c)를 삽입한다.
13 미로에 길이 없음을 출력하고 종료한다.
```

표 6.3은 그림 6.9의 미로에 대해 알고리즘 6.2의 방식으로 입구에서 목적지까지 최단 경로를 찾는 과정을 나타낸 것입니다. 여기서도 현재 위치 (x, y)에서 각 인접 위치를 고려할 때, 북쪽부터 시계방향으로, 즉 북-동-남-서의 순으로 차례로 조사한다고 가정합니다.

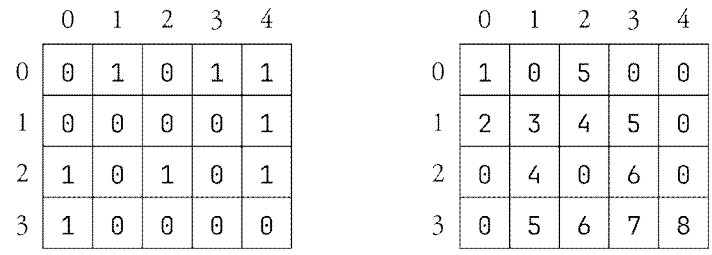


그림 6.9: 최단 경로를 찾는 과정

표 6.3: 그림 6.9의 미로에서 최단 경로를 찾는 과정(매 단계 front 원소가 삭제됨)

단계	큐 내부 (front → rear)	비 고
1	[]	초기 상태, 공백 큐 생성
2	[(0, 0)]	입구 위치 삽입, mark[0][0]=1
3	[(1, 0)]	남쪽 탐색, mark[1][0]=2
4	[(1, 1)]	동쪽 탐색, mark[1][1]=3
5	[(1, 2), (2, 1)]	동남 탐색, mark[1][2]=mark[2][1]=4
6	[(2, 1), (0, 2), (1, 3)]	북동 탐색, mark[0][2]=mark[1][3]=5
7	[(0, 2), (1, 3), (3, 1)]	남쪽 탐색, mark[3][1]=5
8	[(1, 3), (3, 1)]	새로운 인접 위치 없음
9	[(3, 1), (2, 3)]	남쪽 탐색, mark[2][3]=6
10	[(2, 3), (3, 2)]	동쪽 탐색, mark[3][2]=6
11	[(3, 2), (3, 3)]	mark[3][3]=7
12	[(3, 3)]	새로운 인접 위치 없음
13	[(3, 4)]	동쪽 탐색, mark[3][4]=8
14	[]	목적지 도달

최단 경로는 mark 행렬로부터 쉽게 추출할 수 있습니다. 그림 6.9의 미로에서 목적지 (3, 4)까지의 최단 경로의 길이는 8로 구해졌습니다. 그러면 mark[3][4]의 주변에는 반드시 길이가 7인 인접 위치가 존재하고, 길이가 7인 (3, 3)의 주변에는 반드시 길이가 6인 인접 위치가 존재합니다. 길이가 동일한 위치가 여러 군데 있으면 어느 곳을 선택하더라도 상관없습니다. 이런 방식으로 입구까지 역추적해 갈 수 있습니다. 입구까지 따라간 위치들의 역순이 바로 최단 경로에 해당합니다. 프로그램 6.7에는 행렬 mark로부터 구한 최단 경로 리스트를 반환하는 함

수 get_path()가 나와 있습니다. 이 함수의 수행 시간은 최단 경로의 길이에 비례하는 크기로서 최악의 경우는 $O(mn)$입니다.

프로그램 6.7: mark 행렬로부터 경로를 추출하는 프로그램

```
def get_path(mark, enter, destination):
    m, n = len(mark), len(mark[0])      # 미로의 행과 열의 크기
    x, y = destination                  # 현재 위치, 목적지에서 시작
    path = []                           # 최단 경로를 저장하는 리스트
    move_row = [-1, 0, 1, 0]            # 북, 동, 남, 서 방향의 행 첨자 변화
    move_col = [0, 1, 0, -1]            # 북, 동, 남, 서 방향의 열 첨자 변화

    while (x, y) != enter:              # 입구에 도착할 때까지 탐색
        path.append((x, y))             # 현재 위치 저장
        for idx in range(4):            # 각 인접 위치 검사
            r = x + move_row[idx]       # 인접 위치 (r, c)
            c = y + move_col[idx]
            if not(0 <= r < m and 0 <= c < n):  # 미로 영역 벗어남
                continue
            if mark[r][c] == mark[x][y] - 1:    # 이동 가능 위치
                x, y = r, c             # 현재 위치 갱신
                break

    path.append((x, y))                 # 입구 위치 저장
    path.reverse()                      # 입구부터 목적지까지의 경로 저장
    return path                         # 최단 경로 반환
```

프로그램 6.8은 알고리즘 6.2를 구현한 프로그램입니다. 함수 find_shortest_path()는 미로 데이터인 리스트 maze, 입구와 목적지 좌표를 가진 튜플 enter와 destination을 전달받아 최단 경로를 찾습니다. 만일 경로가 존재하면 프로그램 6.7의 함수 get_path()를 호출하여 최단 경로 리스트를 반환하고, 경로가 존재하지 않은 경우는 None을 반환합니다. 이 함수는 deque를 직접 큐로 사용합니다.

프로그램 6.8: 미로에서 최단 경로를 구하는 프로그램

```
from collections import deque

def find_shortest_path(maze, enter, destination):
    """미로에서 최단 경로를 찾는 프로그램:
    미로 maze의 입구 enter에서 목적지 destination까지 경로가 존재하면
    입구부터 목적지까지 최단 경로 리스트를 반환하고, 없으면 None을 반환한다."""
```

```
    m, n = len(maze), len(maze[0])      # 미로의 행과 열의 크기
    mark = [[0]*n for _ in range(m)]    # mark 행렬 생성
    move_row = [-1, 0, 1, 0]            # 북, 동, 남, 서 방향의 행 첨자 변화
    move_col = [0, 1, 0, -1]            # 북, 동, 남, 서 방향의 열 첨자 변화
    Q = deque()                         # deque를 이용한 큐 생성
    Q.append(enter)                     # 큐에 입구 위치 삽입
    mark[enter[0]][enter[1]] = 1        # 입구 방문 기록

    while len(Q) > 0:                   # 큐가 공백이 아니면 반복
        x, y = Q.popleft()              # 현재 위치 (x, y) (큐 삭제)
        if (x, y) == destination:       # 목적지 발견
            return get_path(mark, enter, destination)  # 최단 경로 반환
        for idx in range(4):            # 각 인접 위치 검사
            r = x + move_row[idx]       # 인접 위치 (r, c)
            c = y + move_col[idx]
            if not(0 <= r < m and 0 <= c < n):        # 미로 영역 벗어남
                continue
            if maze[r][c] == 0 and mark[r][c] == 0:  # 이동 가능 위치
                mark[r][c] = mark[x][y] + 1           # mark 갱신
                Q.append((r, c))                      # 큐에 삽입

    return None                                       # 경로 존재하지 않음
```

프로그램 6.8의 수행 시간을 살펴봅시다. 미로의 어떤 위치 (r, c)는 mark[r][c]가 0일 때만 큐에 삽입됩니다. 그 직후 mark의 값은 0이 아닌 값으로 바뀝니다. 따라서, 미로의 통로에 해당하는 각 위치는 기껏해야 한 번 큐에 삽입되고, 기껏해야 한 번 큐에서 삭제됩니다. 큐에 새로 삽입될 위치는 현재 위치에 인접한 위치이므로, 최대 4번 인접 위치 탐색이 수행됩니다. 그러므로, 전체 수행 시간은 미로의 크기 m×n에 해당하는 $O(mn)$입니다.

6.5 사례 연구: 요셉 문제

이번에는 1세기 유대-로마 전쟁의 역사적 일화에서 유래된 것으로, 수학과 컴퓨터 과학 분야에서 조합론적 퍼즐 문제로 널리 알려진 **요셉 문제(Josephus problem)**를 살펴봅니다.

유대인 병사 n명이 로마 병사로부터 도망가다가 동굴에 갇히게 되었다. 사람들은 포로로 잡히기보다는 죽음을 선택하기로 하였다. 자살하는 것은 어렵기 때문에, 다음과 같은 방식을 결정했다. 둥글게 서서 1번 위치부터 시작하여 차례대로 건너뛰어 k번째 위치에 있는 병사를 먼저 죽이고, 그다음 다시 k번째 떨어져 있는 병사를 죽이는 방식이다. 그러나 요셉은 그와 같은 죽음을 원치 않았기 때문에 자신이 최후로 남을 수 있는 위치에 서기로 하였다. 입력으로 n과 k가 주어질 때 요셉이 위치할 번호를 출력하는 프로그램을 작성하시오.

예를 들어, n=13이고 k=3이라고 하자(그림 6.10 참조). 둥글게 선 줄에서 사라지는 병사들의 순서는 다음과 같다. 따라서 최후로 살아남을 수 있는 위치는 13이다.

3, 6, 9, 12, 2, 7, 11, 4, 10, 5, 1, 8

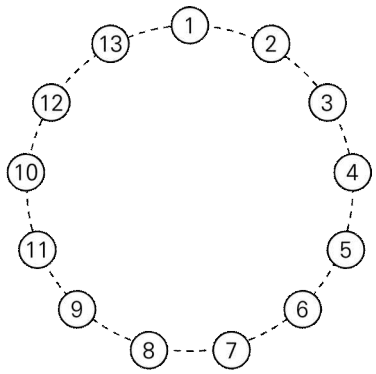


그림 6.10: 요셉 문제 n=13, k=3

요셉 문제를 해결하는 다양한 해법이 존재합니다. 먼저, 재귀 관계를 이용한 해법부터 소개하겠습니다. J(n, k)를 n명으로 시작하여 k번째 사람을 반복적으로 제거할 때 최후로 살아남을 수 있는 위치라고 합시다. 위치는 0부터가 아니라 1부터 시작한다는 점에 유의해야 합니다. 초기 조건으로 n=1이면 즉, 한 명만 남았다면 그 사람이 생존자이므로 1번 위치가 답입니다.

$$J(1, k) = 1$$

일반적인 경우, 첫 번째로 사라지는 사람의 위치는 k입니다. 이후에 그 위치부터 시작하여 남은 사람들의 위치 순서가 다시 조정됩니다. 즉, 남은 사람의 숫자는 n-1이며, 원래의 k+1번 위치는 1번 위치가 됩니다. 새롭게 조정된 위치에서 생존자의 위치 s가 구해지면, 원래 위치는 s에 k를 더한 값입니다. 이 값이 n보다 크면 n으로 나눈 나머지를 구해야 하며, 나머지가 0인 경우

에는 다시 n으로 바꾸어 원래 위치에 대응시켜야 합니다. J(n, k)의 재귀 관계를 간단하게 정리하기 위해, 원래 문제를 0-based 문제로 바꾸어 봅시다. 즉, 위치를 0부터 n-1까지로 두고 구한 재귀 관계 J'(n, k)는 다음과 같습니다.

$$J'(n, k) = (J'(n-1, k) + k) \pmod{n}$$

J(n, k)는 J'(n, k)의 값에 1을 더한 값이므로 다음의 식이 성립합니다.

$$J'(n, k) = J(n, k) - 1$$

이 식을 J'(n, k)의 재귀 관계에 대입하면 다음과 같이 정리됩니다.

$$J(n, k) - 1 = (J(n-1, k) - 1 + k) \pmod{n}$$
$$J(n, k) = ((J(n-1, k) + k-1) \pmod{n}) + 1$$

예를 들어, n=7, k=3이면, 위 재귀 관계를 이용하여 다음과 같이 답을 구할 수 있습니다. 다음의 식에서 나머지 연산자를 파이썬의 나머지 연산자 %로 표시합니다.

```
J(1, 3) = 1
J(2, 3) = ((J(1, 3) + 2) % 2) + 1 = ((1 + 2) % 2) + 1 = 2
J(3, 3) = ((J(2, 3) + 2) % 3) + 1 = ((2 + 2) % 3) + 1 = 2
J(4, 3) = ((J(3, 3) + 2) % 4) + 1 = ((2 + 2) % 4) + 1 = 1
J(5, 3) = ((J(4, 3) + 2) % 5) + 1 = ((1 + 2) % 5) + 1 = 4
J(6, 3) = ((J(5, 3) + 2) % 6) + 1 = ((4 + 2) % 6) + 1 = 1
J(7, 3) = ((J(6, 3) + 2) % 7) + 1 = ((1 + 2) % 7) + 1 = 4
```

이 재귀 관계를 반복적 방식 또는 재귀 함수로 구현한 프로그램의 시간복잡도는 $O(n)$입니다.

다음으로 큐를 활용한 시뮬레이션 방식으로 요셉 문제를 해결하는 방법을 알아봅니다. 공백 큐에 1부터 n까지 숫자를 차례대로 삽입한 다음, 매번 k개의 숫자를 큐에서 삭제합니다. 이때 마지막 k번째 숫자를 제외하고 삭제된 숫자들을 다시 큐에 순서대로 삽입합니다. 그러면 매 단계 k번째 숫자만 삭제됩니다. 이 과정을 단 한 개의 숫자만 큐에 남을 때까지 반복하면, 큐에 남은 최후의 숫자가 바로 최후에 살아남을 수 있는 위치가 됩니다. 예를 들어, n=7, k=3일 때의 큐 시뮬레이션 과정을 표 6.4에서 볼 수 있습니다.

표 6.4: n=7, k=3일 때의 큐 시뮬레이션 과정

단계	큐 내부 (front → rear)	비 고
1	[1, 2, 3, 4, 5, 6, 7]	초기 상태
2	[4, 5, 6, 7, 1, 2]	1, 2를 뒤로 보내고 3을 제거함
3	[7, 1, 2, 4, 5]	4, 5를 뒤로 보내고 6을 제거함
4	[4, 5, 7, 1]	7, 1를 뒤로 보내고 2를 제거함
5	[1, 4, 5]	4, 5를 뒤로 보내고 7을 제거함
6	[1, 4]	1, 4를 뒤로 보내고 5를 제거함
7	[4]	1, 4를 뒤로 보내고 1을 제거함

프로그램 6.9는 큐 시뮬레이션 방식으로 요셉 문제를 해결하는 프로그램으로, 큐 자료구조로 `collections.deque`를 사용합니다.

프로그램 6.9: 큐 시뮬레이션 방식으로 요셉 문제를 해결하는 프로그램

```
1  from collections import deque
2
3  def josephus(n, k):
4      """요셉 문제를 해결하는 프로그램:
5      n명으로 시작하여 k번째 사람을 반복적으로 제거할 때 최후로 살아남을 수 있는
6      위치를 반환한다."""
7      Q = deque()                          # deque를 이용하여 공백 큐 생성
8      for i in range(1, n+1):              # 1부터 n까지 큐에 삽입
9          Q.append(i)
10     while len(Q) > 1:                    # 마지막 한 명이 남을 때까지 반복
11         for _ in range(k-1):             # k-1번째 사람까지는 큐에 다시 삽입
12             Q.append(Q.popleft())
13         Q.popleft()                      # k번째 사람 제거
14     return Q[0]                          # 최후의 생존자 위치
```

함수 `josephus(n, k)`의 수행 시간은 $O(nk)$입니다. 그런데 k가 큐에 저장된 원소의 개수 n보다 큰 경우에는 동일한 큐 원소에 대해서 여러 번의 비효율적인 삭제 및 삽입 연산을 수행해야 합니다. 예를 들어, n=100, k=500인 경우, 첫 번째 원소를 제거하기 위해 큐의 모든 원소가 다섯 바퀴 돌 때까지, 원소 당 최대 5번씩 삭제 및 삽입 연산을 수행합니다($\lfloor 500/100 \rfloor = 5$). 큐에 남은 원소가 줄어들수록 이 비효율성은 더욱 커집니다. 큐에 두 개의 원소가 남았을 때는 동일한 원소를 250번이나 삭제/삽입하는 연산을 수행하게 됩니다. 이처럼 동일한 원소에

대한 비효율적인 반복 연산을 개선하려면, 나머지 연산을 활용하면 됩니다. 큐에서 k번째 원소를 찾기 위해, k-1번의 삭제/삽입 연산을 반복하는 대신, k-1을 현재 큐의 크기로 나눈 나머지 값만큼만 연산을 수행하면 됩니다. 예를 들어, 큐에 5개의 원소가 남았을 때 500번째 원소를 제거하는 것은 5번째 원소를 제거하는 것과 같습니다. 이 방법을 사용하면 큐를 여러 번 순회하는 불필요한 반복을 제거하고, 연산 횟수를 최소화하여 효율성을 높일 수 있습니다. 프로그램 6.10에는 이 방법으로 개선한 프로그램이 나와 있습니다.

프로그램 6.10: 큐 시뮬레이션 방식으로 요셉 문제를 해결하는 개선된 프로그램

```
1  from collections import deque
2
3  def josephus_up(n, k):
4     """요셉 문제를 해결하는 프로그램:
5     n명으로 시작하여 k번째 사람을 반복적으로 제거할 때 최후로 살아남을 수 있는
6     위치를 반환한다."""
7     Q = deque()                          # deque를 이용하여 공백 큐 생성
8     for i in range(1, n+1):              # 1부터 n까지 큐에 삽입
9        Q.append(i)
10    q_size = n                           # 현재 큐의 크기
11    while q_size > 1:                    # 마지막 한 명이 남을 때까지 반복
12       for _ in range((k-1) % q_size):   # k-1 번째 위치까지는 큐에 다시 삽입
13          Q.append(Q.popleft())
14       Q.popleft()                       # k번째 위치 제거
15       q_size -= 1                       # 큐 크기 갱신
16    return Q[0]                          # 최후의 생존자 위치
```

함수 josephus_up(n, k)의 최악의 경우 시간복잡도는 여전히 $O(nk)$이지만, 프로그램 6.9에 비해 항상 빠르게 수행됩니다. 큐를 활용한 알고리즘은 앞서 소개한 재귀 관계를 이용한 알고리즘보다는 수행 시간이 더 걸리지만, 사라지는 사람의 순서를 차례대로 구하는 등 시뮬레이션을 요구하는 문제에서는 효과적인 전략입니다.

참고로, collections.deque에 포함된 rotate(r) 메소드는 r이 양수면 데크를 r 단계 오른쪽으로 회전시키고, 음수면 r 단계 왼쪽으로 회전시킵니다. 큐에서 삭제 후 삽입을 반복적으로 수행해야 하는 상황에서 이 메소드를 이용하면, 반복문을 직접적으로 사용하지 않기 때문에, 수행 시간이 상당히 단축됩니다. 따라서 프로그램 6.10의 12-13줄의 for 문을 다음의 명령으로 치환하면 성능이 개선됩니다.

```
Q.rotate(-((k-1) % q_size))
```

연습문제

1. 다음 큐 연산들이 공백 큐에서부터 순서대로 수행될 때, 모든 연산이 수행된 후 최종 큐의 상태를 나타내시오.

   ```
   enqueue(A), enqueue(B), dequeue(), enqueue(C),
   enqueue(D), dequeue()
   ```

2. 몇 개의 변수와 한 개의 큐를 사용하여, 큐에 저장된 원소들을 역순으로 뒤집는 재귀 알고리즘을 설계하시오.

3. 몇 개의 변수와 다음 자료구조를 추가로 사용하여 큐 Q에 들어있는 원소들을 정렬하여 Q에 다시 저장하는 알고리즘을 제시하고, 그 시간복잡도를 분석하시오.
   (1) 두 개의 큐
   (2) 한 개의 큐

4. 두 개의 큐를 사용하여 스택을 구현하시오. 즉, push와 pop 연산을 큐의 enqueue와 dequeue 연산만으로 수행해야 한다.

5. 차량 재배치 문제에서 4대의 차량이 입국 쪽에서 들어올 때, 출구로 나가는 차량 순서를 (1, 2, 3, 4)로 만들 수 없는 입구 쪽 차량 순서를 모두 나열하시오.

6. 공백 큐에서 총 32개의 enqueue 연산과 10개의 first 연산, 15개의 dequeue 연산이 수행되었고, 이 중 5개의 empty 오류가 발생하여 None이 반환되었다면, 큐에 들어있는 원소는 몇 개인가?

7. 공백 데크(deque) D에서부터 다음 연산들이 순서대로 수행될 때, 반환되는 원소의 순서를 쓰고, 모든 연산이 수행된 후 데크의 최종 상태를 나타내시오. 데크 연산은 collections.deque 클래스의 연산으로 표시된다.

   ```
   D.appendleft(4), D.append(8), D.append(9), D.appendleft(5),
   D[-1],  D.popleft(), D.pop(), D.append(7), D[0], D[-1],
   D.append(6), D.popleft(), D.popleft()
   ```

8. collections.deque 클래스는 표 6.2에 소개된 메소드 외에도 extendleft(), extend(), rotate(), maxlen() 등이 메소드가 정의되어 있다. 이들의 기능과 시간 복잡도를 조사하시오.

9. **스크롤(scroll)**은 삭제는 양쪽 끝에서 발생하나 삽입은 한 쪽 끝에서만 발생하는 선형 구조로 입력 제한 데크라고도 불린다. 스크롤을 위한 추상자료형 Scroll을 설계하고 구현하시오.

10. **셸프(shelf)**는 삽입은 양쪽 끝에서 발생하나 삭제는 한 쪽 끝에서만 발생하는 선형 구조로 출력 제한 데크라고도 불린다. 셸프를 위한 추상자료형 Shelf를 설계하고 구현하시오.

11. 파이썬 딕셔너리를 이용하여 6.2절의 무선조종 자동차 문제를 해결하는 프로그램을 구현하시오.

12. 요셉 문제를 해결하는 프로그램 6.9, 프로그램 6.10을 직접 구현하여, 다양한 n과 k 값을 가지고 수행 시간을 비교하는 실험을 하시오. 그리고 프로그램 6.10의 12-13줄의 for 문을 deque 클래스의 rotate() 메소드로 치환하면 수행 시간이 얼마나 단축되는지 조사하시오.

13. k가 2일 때, 요셉 문제의 해를 찾는 마법같은 해법이 존재한다.

    ① n을 이진수로 표현한다.
    ② 이진수의 가장 왼쪽에 있는 1을 오른쪽 끝으로 옮긴다.
    ③ 다시 십진수로 변환한 값이 생존자의 위치이다.

    예를 들어, n=13, k=2일 때, n은 이진수 1101이고, 단계 2에 의해 1011이 된다. 1011은 십진수 11이므로, 11이 생존자의 위치이다. 이 해법이 항상 올바른 답을 준다는 것을 보이시오.

14. 프로그램 6.5의 ArrayQueue 클래스의 구현에서 큐가 공백이면 dequeue()와 first() 메소드는 None을 반환한다. 이 두 메소드가 None을 반환하는 것 대신에, 예외(exception)를 발생시키도록 수정하시오.

15. 파이썬 리스트를 사용하여 원형 큐를 구현할 때, 큐 오버플로가 발생하면, None을 반환하거나 예외를 발생시키는 것 대신에 동적 배열처럼 리스트 공간이 자동으로 확장되도록 추상자료형 ArrayQueue 클래스를 구현하시오.

16. [실습 문제] 종이 상자에 n 행 m 열로 귤들이 놓여 있다. 귤 중에는 상한 귤이 k개 있다. 1시간이 지나면 상한 귤에 인접한 정상 귤도 상해 버린다. 두 귤이 동서남북 방향으로 붙어 있으면 서로 인접하다라고 한다.

    예를 들어, 아래 그림은 4행 5열로 귤들이 담겨 있는 상자를 나타낸다. 상자의 각 칸에는 귤이 들어 있다. 그중에 두 개가 상한 귤이다. 0으로 표시된 곳은 정상 귤이, 1로 표시된 곳은 상한 귤이 들어 있는 것을 표시한다. 아래 그림은 한 시간마다 귤들의 상태가 어떻게 변하는지 보여준다. 1시간 후에는 정상 귤이 11개, 2시간 후에는 정상 귤이 3개가 남아 있다. 초기 귤 상자 상태가 주어질 때, t 시간 후에 귤들의 상태가 어떻게 변해 있는지를 알아내는 프로그램을 작성하시오.

0	0	0	0	0
0	1	0	0	0
0	0	0	0	1
0	0	0	0	0

〈초기 상태〉

0	1	0	0	0
1	1	1	0	1
0	1	0	1	1
0	0	0	0	1

〈1시간 후〉

1	1	1	0	1
1	1	1	1	1
1	1	1	1	1
0	1	0	1	1

〈2시간 후〉

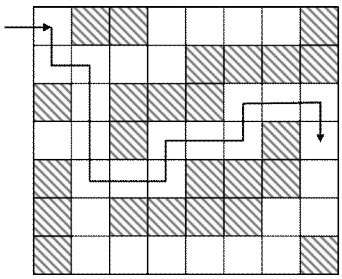
    입력의 첫 줄에 네 개의 정수 n, m, k, t가 주어진다. 네 정수 모두 1 이상 2,000 이하이고, k는 n×m보다 작거나 같다. 그다음 k개의 줄에는 각 줄에 상한 귤의 위치인 a행 b열을 나타내는 두 개의 정수 a와 b가 주어진다(1≤a≤n, 1≤b≤m). 행 번호와 열 번호는 1부터 시작한다. 출력으로 t 시간 후에 남아 있는 정상 귤의 개수를 출력한다.

입력의 예	출력의 예
4 5 2 1 2 2 3 5	11

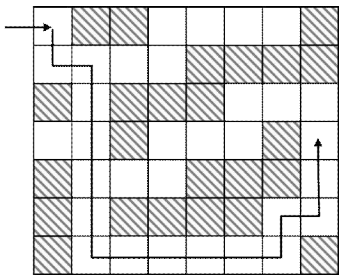
입력의 예	출력의 예
4 5 2 2 2 2 3 5	3

17. [실습 문제] 맨하튼과 같이 도로망이 수직 방향과 수평 방향으로만 되어 있는 도시에서 두 지점 사이에 자동차로 가장 빨리 갈 수 있는 경로를 찾으려고 한다. 자동차는 직선을 달릴 때는 최고 속도를 낼 수 있지만, 회전하게 되면 속도를 줄여야 한다. 그래서 A 지

점에서 B 지점으로 가장 빨리 가기 위한 경로는 방향 전환을 가장 적게 하는 경로로 볼 수 있다. 이 경로를 앞으로 **최적 경로**라고 부르자. 방향 전환은 항상 90도로 한다. 아래 왼쪽 그림의 지도에 표시된 경로는 두 지점 (0, 0)과 (3, 7) 사이의 길이가 14인 최단 경로이지만 8번의 방향 전환을 해야 하고, 오른쪽 그림의 지도에 표시된 경로는 경로의 길이는 16으로 더 길지만 방향 전환을 6번만 한다. 입력으로 도시의 지도와 두 지점의 좌표가 주어질 때 최적 경로를 구하는 프로그램을 작성하시오.



〈최단 경로〉



〈최적 경로〉

입력으로 첫 번째 줄에 지도의 행과 열의 크기 n과 m이 주어진다, $1 \le n, m \le 1000$. 두 번째 줄에는 4개의 정수 x_1, y_1, x_2, y_2가 주어진다, $0 \le x_1, x_2 \le n-1$, $0 \le y_1, y_2 \le m-1$. (x_1, y_1)은 출발 지점의 좌표이고 (x_2, y_2)는 도착 지점의 좌표이다. 세 번째 줄부터 n개의 줄에 걸쳐 지도 데이터가 들어온다. 각 줄에는 m개의 0과 1이 주어지고, 각 숫자 사이에는 한 개의 빈칸이 있다. 1은 지도에서 건물이나 장애물을 표시하고, 0은 도로를 표시한다. 출력으로 두 지점을 연결하는 최적 경로의 방향 전환 수를 출력한다.

입력의 예	출력의 예
7 8 0 0 3 7 0 1 1 0 0 0 0 1 0 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 1 1 1 0 1 0 1 1 1 1 0 0 1 0 0 0 0 0 0 1	6

제7장

연결리스트

7.1 배열 구조와 연결 구조
7.2 단일 연결리스트
7.3 원형 연결리스트
7.4 이중 연결리스트
7.5 사례 연구: 편집기
연습문제

학습 목표

- 연결 구조의 개념을 이해하고, 배열 구조와의 차이점을 설명할 수 있다.
- 파이썬 클래스를 이용하여 단일 연결리스트의 구축, 순회, 검색, 삽입, 삭제 연산을 직접 구현할 수 있다.
- 연결리스트를 사용하여 모든 연산이 상수 시간에 수행하는 스택과 큐를 구현할 수 있다.
- 원형 연결리스트와 이중 연결리스트의 구조를 이해하고, 각 구조가 유용하게 활용되는 문제의 특징을 설명할 수 있다.
- 이중 연결리스트를 사용하여 데크를 구현하고 편집기 문제를 해결할 수 있다.

7.1 배열 구조와 연결 구조

3장에서 **배열 구조**와 이를 기반으로 만들어진 파이썬 리스트에 대해 자세하게 살펴보았습니다. 이를 통해 파이썬 리스트가 추상자료형인 선형리스트를 구현하는 데 매우 편리한 도구라는 것을 알 수 있었습니다. 5장과 6장에서는 파이썬 리스트를 이용하여 선형리스트의 제한된 형태인 스택과 큐를 구현하는 방법도 알아보았습니다.

배열 구조는 메모리의 연속된 공간에 데이터를 저장함으로써 **첨자(index)**를 사용하여 배열 원소를 참조할 수 있습니다. 이러한 배열 구조의 특징은 간단한 주소 계산으로 배열 원소를 쉽고 빠르게 참조할 수 있도록 해 줍니다. 배열 기반 자료구조의 가장 큰 장점은 바로 "임의의 배열 원소에 대한 검색과 저장 연산을 상수 시간에 수행할 수 있다"는 것입니다.

한편, 다음과 같은 배열 구조의 단점도 주목할 필요가 있습니다.

1. 임의의 위치에 새로운 원소를 삽입하거나 기존 원소를 삭제하는 연산은 $O(n)$ 시간이 소요됩니다. 여기서 n은 배열에 저장된 원소의 개수입니다. 스택처럼 선형리스트의 끝에서 삽입과 삭제가 수행되는 경우는 $O(1)$ 시간에 수행되지만, 임의의 위치에서의 삽입/삭제 연산은 원소들의 이동 작업이 동반됩니다. 특히 배열의 맨 앞에서 삽입/삭제가 수행되면 모든 원소를 이동시켜야 합니다.

2. 고정 길이의 선형리스트를 표현할 때는 선형리스트의 길이와 일치하는 길이를 가진 정적 배열을 생성하여 메모리 공간을 매우 효율적으로 사용할 수 있습니다. 그러나 길이가 가변적인 선형리스트를 표현할 때는 선형리스트의 길이와 배열의 길이가 일치하지 않을 수 있습니다. 정적 배열의 경우에는 그 차이가 심각할 정도로 커질 수도 있습니다. 이런 상황에서는 메모리를 오히려 비효율적으로 사용하게 됩니다. 파이썬 리스트처럼 동적 배열을 사용하는 경우는 대부분 무시해도 될 정도이긴 하지만, 배열의 크기는 최악의 경우 실제 저장할 원소 개수의 2배가 될 수 있어 불필요한 메모리 낭비가 발생할 수 있습니다.

3. 파이썬 리스트와 같은 동적 배열을 사용하는 경우, 삽입 및 삭제 연산은 상각 시간복잡도(amortized time complexity)로 $O(1)$ 시간에 수행됩니다. 이는 동일한 연산의 수행 시 대부분은 즉각적인 응답을 해 주지만, 간혹 지연된 응답이 발생할 수 있다는 것을 의미합니다. 자율 주행 자동차, 항공관제, 의료 로봇 등 긴급 상황에서 즉각적인 조치가 이루어져야 하는 **실시간 시스템(real-time system)**에서는 상각 시간복잡도를 가진 프로그램이 시스템의 안정성을 저해할 수 있습니다.

이 장에서 소개하는 **연결 구조(linked structure)**는 앞에서 언급한 배열 구조의 단점을 해소할 수 있는 자료구조입니다. 나중에 다시 언급하겠지만, 배열 구조와 연결 구조는 장단점이 서로 반대로 나타납니다. 배열 구조의 단점만을 해소하기 위해 연결 구조를 이용한다면 오히려 배열 구조보다 더 나쁜 결과를 초래할 수도 있습니다. 그러므로 배열 구조와 연결 구조 중 어떤 방식을 선택할 것인가는 신중하게 결정해야 합니다. 이를 위해 연결 구조에 대한 특징을 잘 이해하는 것이 필요합니다.

배열 구조는 원소들을 메모리의 연속된 공간에 모아두지만, 연결 구조는 서로 연관이 없는 임의의 기억장소에 원소들을 저장합니다. 이렇게 흩어져 있는 원소들을 그룹화하기 위하여, 각 원소가 저장된 기억장소에 **링크(link)**라고 부르는 다른 원소의 위치 정보(객체 메모리 주소, 즉 참조값)를 함께 저장합니다. 연결 구조에서는 원소와 링크를 함께 저장하는 기억장소의 단위를 **노드(node)**라고 부릅니다. **연결리스트(linked list)**는 이러한 노드들이 선형으로 연결된 구조를 말합니다. 그림 7.1은 선형리스트 (a, b, c, d, e)가 연결리스트 방식으로 메모리에 저장된 가상의 형태를 간략하게 나타낸 그림입니다. 이 그림에서 노드는 **데이터 필드(data field)**와 **링크 필드(link field)**로 구성되어 있습니다. 데이터 필드에는 선형리스트의 원소가 저장되고, 링크 필드에는 그다음 노드의 위치 정보인 링크가 저장됩니다. 링크는 노드를 참조하는 값이기 때문에 그림에서 화살표로 표시되어 있습니다. 첫 번째 노드의 링크는 외부의 변수 head에 저장되고, 마지막 노드의 링크는 끝을 표시하는 값인 **널 링크(null link)**입니다.

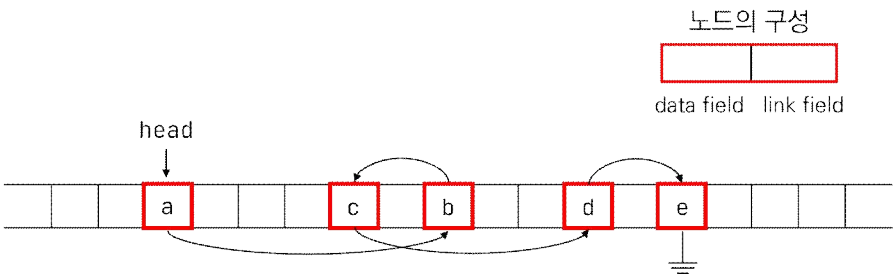


그림 7.1: 메모리에 저장된 연결리스트의 모습

그림 7.1처럼 노드에 링크가 하나만 있는 구조를 **단일 연결리스트(singly linked list)**라고 부릅니다. 단일 연결리스트에서 수행하는 작업에 관한 자세한 내용은 다음 절에서 소개합니다. 보통 단일 연결리스트를 그림으로 표시할 때 그림 7.2처럼 그립니다. 그림에서 노드의 데이터 필드와 링크 필드의 이름은 각각 `data`와 `next`이고, ∅은 널 링크를 나타냅니다. 외부 변수 head가 가리키는 노드를 **시작 노드** 또는 **헤드 노드**라고 부르고, 널 링크를 가진 노드를 **끝 노드** 또는 **꼬리 노드**라고 부릅니다.

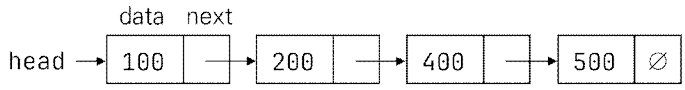


그림 7.2: 단일 연결리스트의 예

6장에서 연결리스트 기반으로 구현된 collections.deque를 소개했지만, 파이썬의 기본 문법은 연결리스트를 직접 지원하지 않습니다. 그래서 필요시에는 클래스를 이용하여 연결리스트를 직접 구현해야 합니다. 프로그램 7.1의 클래스 Node는 데이터 필드인 data와 링크 필드인 next로 구성됩니다. 링크 필드의 디폴트 값인 None은 널 링크를 나타냅니다.

프로그램 7.1: 연결리스트 노드를 위한 클래스

```
1 class Node:
2    def __init__(self, item=None, link=None):
3       self.data = item      # 자료 원소 저장
4       self.next = link      # 링크 저장
```

프로그램 7.2는 그림 7.2처럼 구성된 연결리스트를 생성하는 프로그램입니다. 함수 create_linkedlist(L)는 파이썬 리스트 L에 저장된 요소들로 연결리스트를 구성하고 시작 노드의 링크를 반환합니다. 이 함수는 연결리스트의 끝 노드부터 시작 노드까지 차례대로 노드를 하나씩 생성하여 그다음 노드와 연결합니다.

프로그램 7.2: 그림 7.2의 연결리스트를 생성하는 프로그램

```
1 def create_linkedlist(L):
2    """리스트 L의 요소들로 연결리스트를 생성하여 반환한다."""
3    q = None              # 다음 노드의 링크, 초깃값은 None
4    L.reverse()           # 리스트를 역순으로 변경
5    for x in L:
6       p = Node(x, q)     # p는 요소가 x이고 링크가 q인 노드를 참조한다.
7       q = p              # q는 새로 생성된 노드를 참조한다.
8    return q              # 시작 노드의 링크를 반환한다.
9
10 # main program
11 L = [100, 200, 400, 500]
12 head = create_linkedlist(L)
```

연결 구조의 특징은 임의의 위치에서 삽입과 삭제를 수행할 때 링크만 갱신해 주면 되기 때문에, 다른 원소들의 이동 작업이 필요하지 않다는 것입니다. 그래서 삽입과 삭제가 수행될 위치를 알기만 하면 상수 시간에 작업을 완료할 수 있습니다. 이 점이 연결 구조의 가장 큰 장점입니다. 연결 구조의 다른 장점으로는 노드의 개수와 데이터 원소의 개수가 항상 일치한다는 것입니다. 클래스의 객체로 표현되는 연결 구조의 노드는 필요할 때마다 동적으로 생성할 수 있고, 파이썬의 **자동 쓰레기 수집(automatic garbage collection)** 기능은 연결 구조에서 삭제되어 더 이상 참조가 없는 객체를 소멸시키고 할당된 기억장소를 시스템에 반납합니다. 물론 링크가 차지하는 공간은 배열 구조에는 없는 부가적인 공간이지만, 메모리를 심각한 수준으로 낭비할 가능성이 있는 정적 배열과 비교하면 메모리를 효과적으로 사용한다고 말할 수 있습니다.

연결 구조의 단점은 임의의 위치에 있는 원소를 검색하는 작업에 많은 시간이 필요하다는 것입니다. 배열 구조처럼 주소 계산을 하지 못하기 때문에, 연결 구조에서는 시작 노드부터 순차적으로 링크를 따라가야 합니다. 따라서 연결리스트에서 끝 노드를 찾으려면 시작 노드에서부터 시작하여 모든 노드를 한 번씩 방문해야 하므로, 검색 연산의 수행 시간은 $O(n)$입니다.

이처럼 배열 구조와 연결 구조는 장단점이 서로 반대이기 때문에, 어떤 추상자료형을 구현할 때 배열 구조를 기반으로 할 것인지, 연결 구조를 기반으로 할 것인지, 아니면 두 구조를 결합한 형태로 구현할 것인지를 신중하게 판단해야 합니다.

7.2 단일 연결리스트

단일 연결리스트(singly linked list)는 선형리스트를 표현하기 위한 연결 구조로서 노드에 링크가 하나만 있는 가장 간단한 연결 구조입니다(그림 7.2 참조). 앞으로 다른 언급이 없으면, 단일 연결리스트를 그냥 연결리스트라고 지칭합니다. 연결리스트에서 끝 노드는 널 링크를 가집니다. 연결리스트는 시작 노드의 링크만 알면 링크를 따라가면서 다른 모든 노드를 참조할 수 있습니다. 따라서 외부의 변수에 시작 노드의 링크를 저장하고, 프로그램의 수행 도중에 이 변수의 값이 바뀌지 않도록 주의해야 합니다.

연결리스트가 선형리스트 $L = (a_0, a_1, a_2, \cdots, a_{n-1})$을 표현하고 있을 때, 연결리스트에서 i-번째 노드는 데이터 필드에 a_i가 저장된 노드를 지칭하며, 이 노드는 시작 노드로부터 링크를 i번 따라갈 때 만납니다.

앞으로 변수 p가 어떤 노드의 링크를 가지고 있을 때, 그 노드를 'p가 가리키는 노드' 또는 'p가 참조하는 노드'라고 말하며, 간단히 '노드 p'라고 지칭하기도 합니다. 또한 변수 p가 가리키는 노드가 시작 노드인 연결리스트를 간단히 '연결리스트 p'라고 지칭합니다. 노드 p의 링크 필드가 노드 q를 가리킬 때, 간단히 '노드 p는 노드 q를 가리킨다'고 말합니다.

이제 연결리스트를 대상으로 수행하는 작업을 프로그램으로 구현하고 수행 시간을 분석해 보겠습니다. 다른 언급이 없으면 프로그램 7.1에서 정의한 클래스를 연결리스트 노드를 위한 클래스로 사용합니다. 먼저 연결리스트의 각 노드에 있는 원소를 차례대로 출력하는 작업입니다. 프로그램 7.3의 함수 print_all(head)은 변수 head가 가리키는 연결리스트에 저장된 데이터를 한 줄에 차례대로 출력합니다.

프로그램 7.3: 연결리스트의 모든 원소를 출력하는 프로그램

```
1  def print_all(head):
2      """연결리스트 head의 모든 원소를 출력한다."""
3      p = head
4      while p is not None:          # p가 널 링크를 가질 때까지 반복함
5          print(p.data, end=' ')    # 데이터 필드에 저장된 원소 출력
6          p = p.next                # p에 다음 노드의 링크를 배정함
7      print()
```

다음은 연결리스트에서 검색 연산을 수행하는 프로그램입니다. 프로그램 7.4의 함수 find(head, k)는 연결리스트 head에서 k-번째 원소를 찾아서 반환합니다. 여기서 k는 0 이상이며 연결리스트의 길이, 즉 노드의 개수보다 작으면 유효한 값입니다. 만일 k가 유효하지 않은 값이면 None을 반환합니다. 함수 index(head, value)는 연결리스트 head에서 value 값을 가진 원소의 위치, 즉 첨자를 반환합니다. 그러한 원소가 여러 개 있으면 가장 작은 첨자를 반환하고, 존재하지 않으면 None을 반환합니다. 이 두 함수는 모두 $O(n)$ 시간에 수행됩니다. 여기서 n은 노드의 개수입니다.

프로그램 7.4: 연결리스트에서 검색을 수행하는 프로그램

```
1  def find(head, k):
2      """연결리스트 head에서 k-번째 원소를 반환한다. (k >= 0)"""
3      if k < 0:                     # k가 음수면 None을 반환함
4          return None
5      p = head
6      for i in range(k):            # 링크를 k번 따라감
7          if p is None:             # 노드가 존재하지 않으면 None을 반환함
```

```
8           return None
9        p = p.next                # p에 다음 노드의 링크를 배정함
10    if p is None:                # k-번째 노드가 없으면 None을 반환함
11       return None
12    return p.data                # k-번째 원소 반환함
13
14 def index(head, value):
15    """연결리스트 head에서 value의 최초 위치인 첨자를 반환한다."""
16    i = 0                        # 첨자를 저장함
17    p = head
18    while p is not None:         # p가 널 링크를 가질 때까지 반복함
19       if p.data == value:       # value를 발견하면 원소의 첨자를 반환함
20          return i
21       p = p.next                # p에 다음 노드의 링크를 배정함
22       i += 1                    # 첨자 1 증가
23    return None                  # 존재하지 않으면 None을 반환함
```

다음으로 연결리스트에서 새로운 노드를 삽입하는 연산을 살펴봅니다. 새로운 노드의 삽입은 연결리스트의 맨 앞에 삽입하는 연산과 특정 노드의 바로 다음에 삽입하는 연산으로 구분됩니다. 연결리스트의 맨 앞에 노드를 삽입하면 시작 노드가 변경됩니다. 그러면 연결리스트를 가리키는 외부 변수의 값도 바꿔 주어야 합니다. 연결리스트의 맨 앞이 아닌 장소에 노드를 삽입하는 경우는 삽입할 노드를 가리킬 노드, 즉 삽입될 위치의 이전 노드를 미리 알고 있어야 합니다.

프로그램 7.5의 함수 add_front(head, new_node)는 연결리스트 head의 맨 앞에 노드 new_node를 삽입한 후에 새로운 시작 노드의 링크를 반환하고, 함수 add_node(prev, new_node)는 노드 prev의 뒤에 노드 new_node를 삽입합니다. 두 함수 모두 링크의 갱신만으로 삽입이 완료되므로 상수 시간에 수행됩니다(그림 7.3 참조).

프로그램 7.5: 연결리스트에서 삽입을 수행하는 프로그램

```
1  def add_front(head, new_node):
2     """연결리스트 head의 맨 앞에 노드 new_node를 삽입한다."""
3     new_node.next = head          # head를 new_node의 다음 노드로 설정
4     return new_node               # 새로운 시작 노드의 링크를 반환함
5
6  def add_node(prev, new_node):
7     """노드 prev의 뒤에 노드 new_node를 삽입한다."""
8     new_node.next = prev.next     # prev의 다음 노드를 new의 다음 노드로 설정
9     prev.next = new_node          # new_node를 prev의 다음 노드로 설정
```

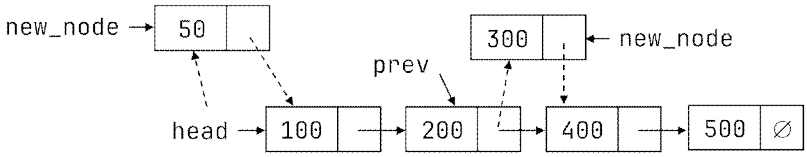


그림 7.3: 연결리스트에서 삽입 연산의 수행

다음으로 연결리스트에서 노드를 삭제하는 연산을 살펴봅니다. 노드의 삭제도 삽입과 마찬가지로 연결리스트의 시작 노드를 삭제하는 연산과 특정 노드의 바로 다음 노드를 삭제하는 연산으로 구분됩니다. 시작 노드를 삭제하면 두 번째 노드가 새로운 시작 노드가 됩니다. 따라서 새로운 시작 노드의 링크를 전달해 주어야 합니다. 시작 노드가 아닌 노드를 삭제하는 경우는 삭제할 노드 자체가 아니라 그 이전 노드의 링크를 알고 있어야 합니다. 그래야 이전 노드가 삭제될 노드의 다음 노드를 가리키도록 하여 삭제를 완성할 수 있습니다.

프로그램 7.6의 함수 `remove_front(head)`는 시작 노드 `head`가 가리키는 두 번째 노드의 링크를 반환하여 기존 시작 노드가 삭제되도록 합니다. `head`가 `None`이면 즉, 연결리스트가 공백이면, `None`을 반환합니다. 함수 `remove_node(prev)`는 노드 `prev`의 다음 노드를 삭제합니다. `prev`가 끝 노드를 가리키거나 `None`이면, 삭제할 노드가 없으므로 그대로 종료하고, 그렇지 않으면 노드 `prev`가 삭제될 노드의 다음 노드를 가리키게 합니다. 연결리스트에 있는 노드들이 가리키고 있지 않는 노드는 그 연결리스트에서 삭제된 것으로 보면 됩니다. 삭제 작업은 두 함수 모두 링크의 갱신만으로 완료되므로 상수 시간에 수행됩니다(그림 7.4 참조).

프로그램 7.6: 연결리스트에서 삭제를 수행하는 프로그램

```
1  def remove_front(head):
2     """연결리스트 head의 시작 노드를 삭제한다."""
3     if head is None:                # 연결리스트가 공백인 경우 None 반환
4        return None
5     return head.next                # 새로운 시작 노드의 링크를 반환함
6
7  def remove_node(prev):
8     """노드 prev의 다음 노드를 삭제한다."""
9     if prev is None or prev.next is None:
10       return                       # prev가 None이거나 끝 노드이면 종료함
11    prev.next = prev.next.next      # prev가 삭제될 노드의 다음 노드를 가리킴
```

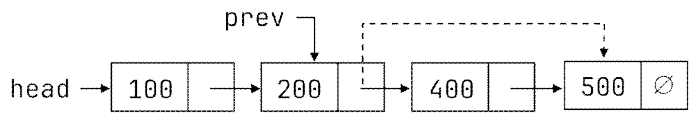


그림 7.4: 연결리스트에서 삭제 연산의 수행

삭제된 노드의 **참조 계수(reference count)**가 0이면, 이 노드는 파이썬의 자동 쓰레기 수집 기능에 의해 메모리에서 제거됩니다[14].

연결리스트를 다루는 프로그램에서 시작 노드와 끝 노드에서의 작업 그리고 연결리스트에 노드가 하나도 없는 공백 상태를 예외적으로 처리해야 할 경우가 많습니다. 이러한 예외 처리로 인해 프로그램이 길어질 뿐만 아니라 간혹 예외 처리의 누락으로 오류가 발생하기도 합니다. 이러한 상황을 줄이기 위한 목적으로 연결리스트에 **헤더 노드(header node)**를 추가하는 방법이 있습니다. 헤더 노드에는 자료 원소를 저장하지 않기 때문에 헤더 노드를 **더미 노드(dummy node)**라고 부르기도 합니다. 필요에 따라서는 헤더 노드에 자료 원소의 개수, 최댓값과 최솟값 등의 현재 구성된 연결리스트에 관한 정보를 저장할 수도 있습니다. 헤더 노드에 자료 원소의 개수를 저장한다면, 파이썬 리스트의 `len()` 함수처럼 상수 시간에 리스트의 길이를 반환할 수 있습니다. 일반적으로 연결리스트의 길이를 알기 위해서는, 모든 노드를 탐색하면서 카운트해야 하므로, 선형 시간이 소요됩니다. 헤더 노드가 연결리스트의 시작 노드로 고정되면 시작 노드가 삭제되거나 바뀌는 상황이 발생하지 않으므로, 프로그램 7.5와 프로그램 7.6의 `add_front()`, `remove_front()`와 같은 함수를 따로 둘 필요가 없습니다. 그만큼 코딩이 간편해지고 오류가 발생할 가능성도 줄어듭니다. 그림 7.5는 헤더 노드를 가진 연결리스트의 예를 보여줍니다. 그림에서 변수 `head`가 가리키는 노드가 헤더 노드입니다.

지금까지 살펴봤듯이 연결리스트의 특징은 특정 위치에서의 삽입과 삭제를 링크의 변경만으로 상수 시간에 수행할 수 있다는 점과 특정한 위치에 있는 원소를 검색하기 위해서는 시작 노드부터 링크를 따라가는 작업이 필요하므로 선형 시간이 소요된다는 점입니다. 또한 연결리스트는 동적 기억장소 할당 및 반납을 기반으로 하고 있어 메모리 공간을 효과적으로 활용할 수 있습니다. 그렇지만 프로그램을 구현하는 관점에서 볼 때, 연결리스트를 사용하는 것은 배열보다 구현이 다소 복잡하고 실수에 의한 오류가 발생할 가능성도 더 높습니다. 따라서 시간복잡도나 공간복잡도 면에서 배열보다 효율적인 경우에만 연결리스트를 이용하는 것이 일반적입니다.

---

14) 참조 계수는 어떤 객체를 참조하는 변수나 다른 객체의 수를 말합니다. 파이썬의 모든 객체는 내부적으로 자신의 참조 계수를 가지고 있습니다. 어떤 객체의 참조 계수가 0이 되면, 파이썬 인터프리터는 그 객체가 더 이상 프로그램에서 사용되지 않는다고 판단하고, 해당 객체가 차지하고 있던 메모리를 해제합니다.

이제, 연결리스트를 기반으로 하여 스택과 큐를 구현하는 방법을 차례로 알아봅니다.

■ 스택의 구현

스택은 배열을 기반으로 구현해도 삽입과 삭제 연산이 상수 시간에 수행됩니다. 다만, 파이썬 리스트와 같은 동적 배열을 사용하면, 3.3절에서 서술한 것과 같이 삽입 연산은 상각 시간복잡도(amortized time complexity)로 상수 시간에 수행됩니다. 이는 리스트 용량이 가득 찼을 때 크기를 확장하는 작업 때문에 일시적으로 시간이 더 걸릴 수도 있지만, 전체 연산의 평균 시간은 상수 시간이라는 의미입니다. 대부분의 응용에서 상각 시간복잡도는 문제가 되지 않지만, 연산 시간이 매우 민감한 실시간 시스템에서는 이러한 불확실성이 문제가 될 수 있습니다. 이 문제는 연결리스트를 사용하여 해결할 수 있습니다. 스택을 연결리스트로 표현할 때, 스택의 top 원소를 연결리스트의 시작 노드에 배정하는 것이 일반적입니다. 만일 연결리스트에 헤더 노드를 도입하면, 그림 7.5와 같이 변수 head는 헤더 노드를 가리키고, 스택의 top 원소는 헤더 노드가 가리키는 노드에 저장됩니다.

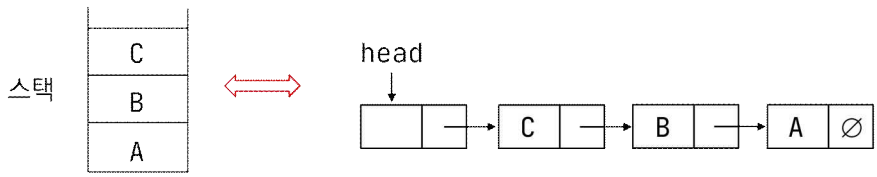


그림 7.5: 스택의 연결리스트 표현

프로그램 7.7은 헤더 노드를 가진 연결리스트를 사용하여 추상자료형 5.1의 Stack 클래스를 구현한 프로그램을 보여줍니다. 프로그램에서 연결리스트 노드 클래스를 내부 클래스(inner class)로 정의하고 있습니다. 이렇게 구현된 스택 연산들은 모두 상수 시간에 수행됩니다.

프로그램 7.7: 연결리스트를 사용하여 구현한 Stack 클래스

```
1  class Stack:
2     """연결리스트를 사용하여 구현한 스택 클래스"""
3     class _Node:                        # 노드 클래스 (내부 클래스로 정의)
4        def __init__(self, item=None, link=None):
5           self._data = item
6           self._next = link
7
8     def __init__(self):                 # 공백 스택 생성
9        self._head = self._Node()        # 헤더 노드 생성
```

```
10
11    def empty(self):                  # 공백이면 True, 아니면 False 반환
12       return self._head._next is None
13
14    def push(self, item):             # 스택에 원소 item을 삽입함
15       self._head._next = self._Node(item, self._head._next)
16
17    def pop(self):                    # 스택의 top 원소를 삭제한 후 반환
18       if self.empty():               # 스택이 공백이면 None을 반환
19          return None
20       item = self._head._next._data
21       self._head._next = self._head._next._next
22       return item
23
24    def peek(self):                   # 스택의 top 원소 반환
25       if self.empty():               # 스택이 공백이면 None을 반환
26          return None
27       return self._head._next._data
```

■ 큐의 구현

큐를 배열 기반으로 구현하기 위한 가장 적합한 방법은 원형 큐라고 불리는 방식입니다. 원형 큐는 원칙적으로 정적 배열을 기반으로 구현하기 때문에 큐 오버플로를 고려해야 합니다. 큐 오버플로가 발생할 때 새로운 배열을 생성하여 복사하는 경우는 수행 시간이 상각 시간복잡도로 분석됩니다. 이러한 배열 기반 구현의 한계는 연결리스트를 기반으로 구현하면 해소될 수 있습니다.

큐를 연결리스트로 표현할 때 큐의 front 원소는 연결리스트의 시작 노드에, 큐의 rear 원소는 끝 노드에 배정됩니다. 만일 헤더 노드를 가진 연결리스트로 표현한다면, 그림 7.6과 같이 front 원소는 헤더 노드가 가리키는 노드에 저장됩니다. 프로그램 7.8은 헤더 노드를 가진 연결리스트를 사용하여 추상자료형 6.1의 Queue 클래스를 구현한 프로그램을 보여줍니다. 이렇게 구현된 큐의 모든 연산은 상수 시간에 수행됩니다.

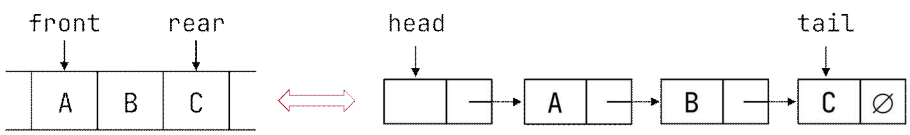


그림 7.6: 큐의 연결리스트 표현

프로그램 7.8: 연결리스트를 사용하여 구현한 Queue 클래스

```
class Queue:
   """연결리스트를 사용하여 구현한 큐 클래스"""
   class _Node:                          # 노드 클래스 (내부 클래스)
      def __init__(self, item=None, link=None):
         self._data = item
         self._next = link

   def __init__(self):                   # 공백 큐 생성
      self._head = self._Node()          # 헤더 노드 생성
      self._tail = self._head

   def empty(self):                      # 공백이면 True, 아니면 False 반환
      return self._head._next is None

   def enqueue(self, item):              # 큐에 원소 item 삽입
      new_node = self._Node(item)        # 새 노드 생성
      self._tail._next = new_node        # 끝 노드로 설정
      self._tail = new_node

   def dequeue(self):                    # 큐의 front 원소를 삭제한 후 반환
      if self.empty():                   # 큐가 공백이면 None을 반환
         return None
      item = self._head._next._data      # 반환할 원소
      self._head._next = self._head._next._next
      if self._head._next is None:       # 헤더 노드만 남으면
         self._tail = self._head         # tail도 헤더 노드를 가리킴
      return item

   def first(self):                      # 큐의 front 원소 반환
      if self.empty():                   # 큐가 공백이면 None을 반환
         return None
      return self._head._next._data
```

7.3 원형 연결리스트

원형 단일 연결리스트(circular singly linked list)는 앞 절에서 소개한 단일 연결리스트처럼 노드에 링크 필드가 하나만 있는 구조이지만, 그림 7.7과 같이 연결리스트의 끝 노드가 널 링크를

가지는 것 대신에 시작 노드를 가리킨다는 차이가 있습니다. 링크를 계속 따라가면 다시 처음으로 돌아갈 수 있으므로 원형 연결리스트라고 부릅니다. 앞으로 다른 언급이 없으면, 원형 단일 연결리스트를 간단히 원형 연결리스트라고 지칭합니다.

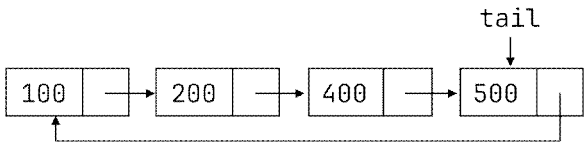


그림 7.7: 원형 연결리스트

원형 연결리스트에서는 끝 노드가 시작 노드의 링크를 가지고 있으므로 끝 노드의 링크만 알면 시작 노드의 링크도 바로 알 수 있는 특징이 있습니다. 그래서 원형 연결리스트에서는 전체를 대표하는 노드로서 끝 노드를 선택하기도 합니다.

원형 연결리스트는 단일 연결리스트와 마찬가지로 시작과 끝을 가진 선형리스트를 표현하기 위한 도구로 사용되기도 하지만, 시작과 끝에 대한 구분이 없는 데이터를 다루는 데 더욱 유용합니다. 6장에서 요셉 문제를 시뮬레이션 방식으로 해결하기 위해 큐를 이용하였습니다. 요셉 문제에서 병사들이 둥글게 서 있는 모습을 그림 7.8처럼 원형 연결리스트로 표현하는 것은 직관적입니다.

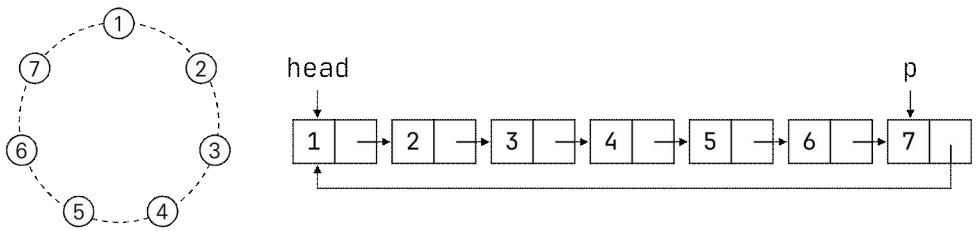


그림 7.8: 요셉 문제를 위한 원형 연결리스트의 예

원형 연결리스트를 이용하여 요셉 문제를 해결하는 프로그램이 프로그램 7.9에 나와 있습니다. 연결리스트 노드를 위한 클래스 `Node`는 프로그램 7.1에서 정의한 것과 같습니다. 수행 시간은 큐를 이용한 알고리즘과 마찬가지로 $O(nk)$입니다. 하지만, 큐 기반의 알고리즘이 k-번째 위치를 찾기 위해 연속적인 삭제와 삽입을 수행하는 반면, 원형 연결리스트에서는 노드 방문과 링크 갱신만으로 문제를 해결합니다.

프로그램 7.9: 원형 연결리스트를 이용하여 요셉 문제를 해결하는 프로그램

```
class Node:
   def __init__(self, item=None, link=None):
      self.data = item              # 자료 원소 저장
      self.next = link              # 링크 저장

def josephus(n, k):
   """요셉 문제를 해결하는 프로그램:
   n명으로 시작하여 k번째 사람을 반복적으로 제거할 때 최후로 살아남을 수 있는
   위치를 반환한다."""
   head = Node(1)                   # 시작 노드 생성
   head.next = head                 # 초기 원형 연결리스트
   tail = head                      # 끝 노드를 가리키는 변수

   for i in range(2, n+1):          # n개 노드의 원형 연결리스트 구성
      p = Node(i, head)             # 시작 노드를 가리키는 새 노드 생성
      tail.next = p                 # 현재 끝 노드가 새 노드를 가리킴
      tail = p                      # tail이 새 끝 노드를 가리킴

   # 시뮬레이션 수행
   while n > 1:                     # 마지막 한 명이 남을 때까지 반복
      for _ in range((k-1) % n):    # k-1 번째 노드로 이동
         p = p.next
      p.next = p.next.next          # k번째 노드 삭제
      n -= 1                        # 노드 수 1 감소

   return p.data                    # 최후의 생존자 위치 반환
```

7.4 이중 연결리스트

단일 연결리스트는 노드에 링크 필드가 하나만 있어 한 방향으로만 이동할 수 있습니다. 따라서 특정 노드의 다음 노드는 바로 찾을 수 있지만, 이전 노드를 찾기 위해서는 시작 노드부터 다시 탐색해야 하는 비효율성이 있습니다.

6장에서 소개한 데크(deque)와 같이 리스트의 양 끝에서 삽입과 삭제를 수행하는 자료구조를 구현한다고 해 봅시다. 배열로 구현하는 경우, 리스트의 맨 앞에서 삭제가 수행될 때 원소들의 이동 작업으로 인해 선형 시간이 소요됩니다. 단일 연결리스트로 구현하는 경우, 리스트의 맨

끝에서 삭제가 수행될 때 이전 노드의 링크를 알기 위해 시작 노드부터 탐색해야 하므로 선형 시간이 소요됩니다. 이처럼 리스트를 양방향으로 처리해야 하는 응용에서는 배열이나 단일 연결리스트를 사용해서는 효율적으로 수행할 수 없습니다. 이러한 경우에 이중 연결리스트를 사용하게 됩니다.

이중 연결리스트(doubly linked list)는 각 노드에 두 개의 링크 필드를 두고, 하나는 이전 노드의 링크를 다른 하나는 다음 노드의 링크를 가지는 연결 구조입니다. 프로그램 7.10의 클래스 DNode는 이중 연결리스트의 노드를 위한 클래스입니다. 이 클래스에서 링크 필드 prev에는 이전 노드의 링크, next에는 다음 노드의 링크가 저장됩니다. 앞으로 다른 언급이 없으면, DNode를 이중 연결리스트 노드를 위한 클래스로 사용합니다.

프로그램 7.10: 이중 연결리스트 노드를 위한 클래스

```
1 class DNode:
2    def __init__(self, item=None, next_link=None, prev_link=None):
3       self.data = item           # 자료 원소 저장
4       self.next = next_link      # 다음 노드의 링크 저장
5       self.prev = prev_link      # 이전 노드의 링크 저장
```

이중 연결리스트는 단일 연결리스트에 비해 링크 필드를 하나만 더 가지고 있지만, 프로그래밍의 어려움은 훨씬 더 증가합니다. 노드가 공백인 상황, next 링크가 None인 상황, prev 링크가 None인 상황, next와 prev가 둘 다 None인 상황을 모두 고려해야 오류 없이 성공적으로 프로그램을 완성할 수 있습니다.

이런 예외적인 상황을 줄이기 위해서 **더미 노드(dummy node)**를 사용하는 것이 좋습니다. 시작 노드와 끝 노드를 각각 더미 노드로 두는 방법도 있지만, 하나의 더미 노드를 사용하면서 원형으로 연결하는 구조를 만들어도 좋습니다. 이런 구조를 **헤더 노드를 가진 원형 이중 연결리스트**라고 부릅니다. 그림 7.9와 그림 7.10은 원형 이중 연결리스트의 예를 보여줍니다. 그림에서 변수 head가 가리키는 노드가 헤더 노드입니다.

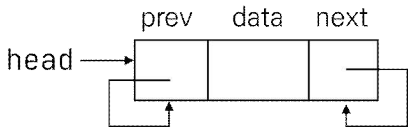


그림 7.9: 헤더 노드만 가진 원형 이중 연결리스트

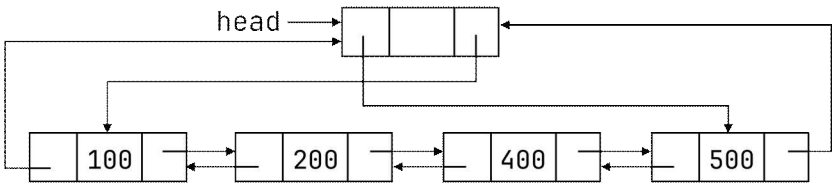


그림 7.10: 헤더 노드를 가진 원형 이중 연결리스트

원형 이중 연결리스트의 헤더 노드에서부터 next 링크를 계속 따라가면 다시 헤더 노드로 돌아오고, 헤더 노드에서부터 prev 링크를 계속 따라가면 다시 헤더 노드로 돌아옵니다. 그래서 원형 이중 연결리스트는 이중 고리를 가진 구조라고 말할 수 있습니다. 변수 p가 어떤 노드를 가리키고 있다면, 항상 다음 관계가 성립합니다.

```
p = p.prev.next = p.next.prev
```

또한 헤더 노드의 존재로 인해 연결리스트에서 삽입할 위치의 이전 노드와 다음 노드가 항상 존재합니다. 즉, 위 식에서 p.prev.next와 p.next.prev가 None이 되는 상황은 발생하지 않는다는 것입니다. 이는 코딩 시에 경계 조건(boundary conditions) 처리를 단순하게 해 주어 오류 발생 가능성을 줄여줍니다.

프로그램 7.11에는 그림 7.10의 헤더 노드를 가진 원형 이중 연결리스트를 생성하고, 정방향과 역방향으로 탐색하면서 원소들을 출력하는 프로그램이 나와 있습니다. 함수 create_doubly linkedlist(L)는 파이썬 리스트 L에 저장된 요소들로 헤더 노드를 가진 원형 이중 연결리스트를 구성하고 헤더 노드의 링크를 반환합니다. 이 함수는 단일 연결리스트의 생성과는 달리 L의 첫 번째 원소부터 차례대로 노드를 생성합니다.

프로그램 7.11: 헤더 노드를 가진 원형 이중 연결리스트를 생성하는 프로그램

```
1  def create_doublylinkedlist(L):
2      """리스트 L의 요소들로 원형 이중 연결리스트를 생성하여 반환한다."""
3      head = DNode()                        # 헤더 노드 생성
4      head.next = head.prev = head          # 헤더 노드 하나만 있는 구조
5      for x in L:
6          p = DNode(x, head, head.prev)     # 끝 노드로 삽입될 새 노드 생성
7          head.prev.next = p                # 이전 끝 노드가 새 노드를 가리킴
8          head.prev = p                     # 헤더 노드가 새 노드를 가리킴
9      return head                           # 헤더 노드 반환
10
```

```
# main program
L = [100, 200, 400, 500]
head = create_doublylinkedlist(L)

# 정방향으로 탐색하면서 원소 출력
p = head.next                          # 시작 노드 - 헤더 노드의 다음 노드
while p is not head:
    print(p.data, end=' ')
    p = p.next
print()

# 역방향으로 탐색하면서 원소 출력
p = head.prev                          # 끝 노드 - 헤더 노드의 이전 노드
while p is not head:
    print(p.data, end=' ')
    p = p.prev
print()
```

이제 헤더 노드를 가진 원형 이중 연결리스트에서 삽입과 삭제를 수행하는 과정을 차례로 알아보겠습니다. 프로그램 7.12의 함수 insert_after(p, new_node)는 노드 p의 다음 위치에 new_node를 삽입하고, 함수 insert_before(q, new_node)는 노드 q의 이전 위치에 new_node를 삽입합니다(그림 7.11 참조). 단일 연결리스트에서는 삽입을 위해서 반드시 삽입할 위치의 이전 노드를 알아야 하지만, 이중 연결리스트에서는 삽입할 위치만 알고 있어도 충분합니다. 이는 이중 연결리스트의 강력한 장점 중 하나입니다.

프로그램 7.12: 원형 이중 연결리스트에서 노드를 삽입하는 프로그램

```
def insert_after(p, new_node):     # 노드 p의 다음 위치에 new_node를 삽입
    new_node.next = p.next
    new_node.prev = p
    p.next.prev = new_node
    p.next = new_node

def insert_before(q, new_node):    # 노드 q의 이전 위치에 new_node를 삽입
    new_node.next = q
    new_node.prev = q.prev
    q.prev.next = new_node
    q.prev = new_node
```

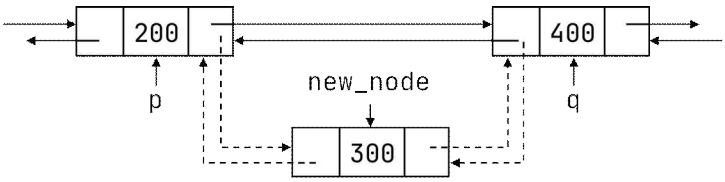


그림 7.11: 원형 이중 연결리스트에서 노드의 삽입

이중 연결리스트에서 링크를 갱신할 때 링크를 바꾸는 순서가 매우 중요합니다. 프로그램 7.12의 `insert_after(p, new_node)`는 링크를 바꾸는 4개의 명령을 수행합니다. 만일 수행 순서를 다음과 같이 변경한다면, 그림 7.12와 같이 엉뚱한 결과를 초래하게 됩니다.

```
# 잘못된 링크 설정의 예
def insertAfter(p, new_node):    # 노드 p의 다음 위치에 new_node를 삽입
   p.next.prev = new_node
   p.next = new_node
   new_node.next = p.next
   new_node.prev = p
```

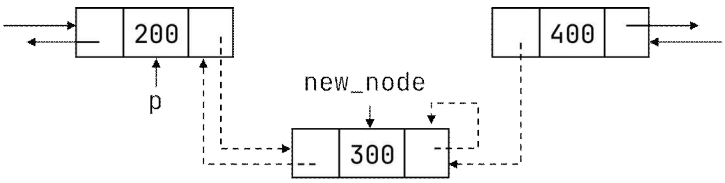


그림 7.12: 링크 설정 오류

이제, 원형 이중 연결리스트에서 임의의 위치에 있는 노드를 삭제한다고 해 봅시다. 물론 헤더 노드는 삭제되지 않으므로, 삭제될 노드는 헤더 노드가 아닙니다. 삭제될 노드의 이전 노드와 다음 노드가 서로를 가리키도록 단지 2개의 링크만 새로 설정해 주면 삭제가 완료됩니다(그림 7.13 참조). 프로그램 7.13의 함수 `delete(p)`는 노드 `p`를 삭제합니다. 삭제할 노드의 링크만 알고 있어도 삭제를 수행할 수 있다는 점은 단일 연결리스트에 비해 이중 연결리스트가 가지는 큰 장점입니다.

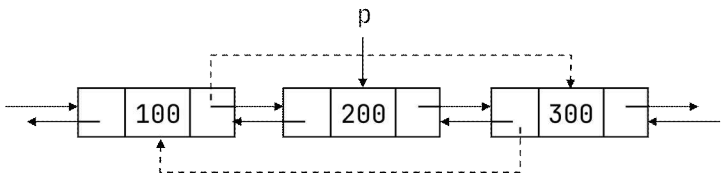


그림 7.13: 원형 이중 연결리스트에서 노드의 삭제

프로그램 7.13: 원형 이중 연결리스트에서 노드를 삭제하는 프로그램

```
1 def delete(head, p):          # 노드 p를 삭제
2    if p is not None and p is not head:    # 헤더 노드는 삭제되지 않음
3       p.prev.next = p.next    # p의 이전 노드가 p의 다음 노드를 가리킴
4       p.next.prev = p.prev    # p의 다음 노드가 p의 이전 노드를 가리킴
```

마지막으로 이중 연결리스트를 사용하여 데크를 직접 구현하는 프로그램 예를 살펴보겠습니다. 구현하려는 데크 자료구조 Deque의 명세는 추상자료형의 7.1에 나와 있습니다.

추상자료형 7.1: Deque

```
class Deque:
   def __init__(self):          # 공백 데크를 생성한다.
   def empty(self):             # 공백이면 True, 아니면 False를 반환한다.
   def __len__():               # 데크의 크기를 반환한다. (len() 함수 사용)
   def add_first(self, item):   # 데크의 맨 앞에 원소 item을 삽입한다.
   def add_last(self, item):    # 데크의 맨 끝에 원소 item을 삽입한다.
   def pop_first(self):         # 데크의 맨 앞 원소를 삭제한 후 반환한다.
                                # 공백이면 None을 반환한다.
   def pop_last(self):          # 데크의 맨 끝 원소를 삭제한 후 반환한다.
                                # 공백이면 None을 반환한다.
   def first(self):             # 데크의 맨 앞 원소를 반환한다.
                                # 공백이면 None을 반환한다.
   def last(self):              # 데크의 맨 끝 원소를 반환한다.
                                # 공백이면 None을 반환한다.
```

추상자료형 7.1의 클래스 Deque를 헤더 노드를 가진 원형 이중 연결리스트로 구현해 봅니다. 헤더 노드의 데이터 필드에는 데크에 저장된 원소의 개수를 저장하여, 데크의 크기를 상수 시간

에 구할 수 있도록 합니다. 프로그램 7.14에 클래스 Deque를 구현한 프로그램이 나와 있습니다. 데크 데이터에 대해 len() 함수를 사용할 수 있도록 클래스의 특별 함수인 __len__() 메소드가 정의되어 있습니다. 클래스 내부에서 pop_first()와 pop_last()에서만 호출되는 메소드인 _delete_node(self, p)는 노드 p를 연결리스트에서 삭제하고, p가 가진 원소를 반환합니다. Deque의 모든 메소드는 상수 시간에 수행됩니다.

프로그램 7.14: 이중 연결리스트를 사용하여 구현한 Deque 클래스

```
class Deque:
   """헤더 노드를 가진 원형 이중 연결리스트를 사용하여 구현한 데크 클래스"""

   class _DNode:                      # 노드 클래스 (내부 클래스)
      def __init__(self, item=None, next_link=None, prev_link=None):
         self._data = item            # 자료 원소 저장
         self._next = next_link       # 다음 노드의 링크 저장
         self._prev = prev_link       # 이전 노드의 링크 저장

   def __init__(self):                # 공백 데크 생성
      self._head = self._DNode(0)     # 헤더 노드에 데크 크기 저장
      self._head._next = self._head._prev = self._head

   def empty(self):                   # 공백이면 True, 아니면 False 반환
      return self._head._data == 0

   def __len__(self):                 # 데크 크기 반환 (len() 함수 사용)
      return self._head._data

   def add_first(self, item):         # 데크의 맨 앞에 원소 item 삽입
      new_node  = self._DNode(item, self._head._next, self._head)
      self._head._next._prev = new_node
      self._head._next = new_node
      self._head._data += 1

    def add_last(self, item):         # 데크의 맨 끝에 원소 item 삽입
       new_node = self._DNode(item, self._head, self._head._prev)
       self._head._prev._next = new_node
       self._head._prev = new_node
       self._head._data += 1

   def _delete_node(self, p):    # 노드 p 삭제/반환 (클래스 내부에서만 사용)
      if p is self._head:        # 헤더 노드 삭제 시도 시 예외 발생
         raise ValueError("헤더 노드를 삭제할 수 없습니다.")
```

```
35      p._prev._next = p._next
36      p._next._prev = p._prev
37      self._head._data -= 1
38      return p._data
39
40   def pop_first(self):          # 데크의 맨 앞 원소를 삭제한 후 반환
41      if self.empty():           # 공백이면 None 반환
42         return None
43      return self._delete_node(self._head._next)  # 삭제된 원소 반환
44
45   def pop_last(self):           # 데크의 맨 끝 원소를 삭제한 후 반환
46      if self.empty():           # 공백이면 None 반환
47         return None
48      return self._delete_node(self._head._prev)  # 삭제된 원소 반환
49
50   def first(self):              # 데크의 맨 앞 원소 반환
51      if self.empty():           # 공백이면 None 반환
52         return None
53      return self._head._next._data
54
55   def last(self):               # 데크의 맨 끝 원소 반환
56      if self.empty():
57         return None
58      return self._head._prev._data
```

프로그램 7.15는 클래스 Deque를 사용하는 예를 보여줍니다.

프로그램 7.15: 클래스 Deque를 사용하는 프로그램의 예

```
1  D = Deque()                          # 공백 데크 생성
2  for i in range(1, 4):
3      D.add_first(2*i)                 # 데크에 원소 삽입
4      D.add_last(2*i+1)                # D = [6, 4, 2, 3, 5, 7]
5
6  print('데크의 크기:', len(D))          # len() 함수 사용
7  print(D.first(), D.last())           # 맨 앞 원소와 맨 끝 원소 출력
8  print(D.pop_first(), D.pop_last())   # 양 끝 원소 삭제 후 출력
9
10 while not D.empty():                 # 공백이 될 때까지 원소 삭제 후 출력
11     print(D.pop_first(), end = ' ')  # 출력: 4 2 3 5
12 print()
```

7.5 사례 연구: 편집기

5장의 사례 연구에서 다뤘던 편집기 문제를, 연결리스트를 사용하여 해결하는 방법을 살펴보겠습니다. 편집기 문제는 다음과 같습니다.

영어 소문자만으로 구성된 한 줄짜리 텍스트가 주어져 있다. 커서(cursor)는 텍스트의 맨 앞(첫 번째 문자의 왼쪽), 맨 끝(마지막 문자의 오른쪽), 또는 인접한 두 문자 사이에 위치할 수 있다. 텍스트를 편집하기 위한 명령이 들어오면 차례대로 텍스트에 적용해야 한다. 편집 명령은 다음과 같다.

`L`	커서를 왼쪽으로 한 칸 옮긴다. (커서가 맨 앞에 있으면 무시함)
`D`	커서를 오른쪽으로 한 칸 옮긴다. (커서가 맨 끝에 있으면 무시함)
`B`	커서의 왼쪽 문자를 삭제한다. (커서가 맨 앞에 있으면 무시함)
`P $`	커서의 왼쪽에 문자 $를 삽입한다. (문자 $는 영어 소문자임)

편집 작업을 수행하기 전, 초기에는 텍스트의 맨 끝에 커서가 위치한다. 주어진 편집 명령을 모두 수행한 후에 텍스트의 상태를 출력하는 프로그램을 작성하시오.

입력 데이터의 첫 줄에는 텍스트가 주어진다. 텍스트의 길이는 100,000 이하이다. 둘째 줄에는 정수 n이 주어진다. n은 1 이상 500,000 이하이다. 그다음 n개의 줄에는 편집 명령이 한 줄에 하나씩 주어진다. 프로그램은 입력으로 주어진 편집 명령을 모두 수행한 후의 텍스트를 출력한다. 프로그램의 최대 수행 시간은 1초로 제한된다. 입력과 출력의 예는 다음과 같다.

입력	출력
abcd 3 P x L P y	abcdyx

입력	출력
abc 9 L L L L L P x L B P y	yxabc

입력	출력
dmih 11 B B P x L B B B P y D D P z	yxz

5장에서는 이 문제를 두 개의 스택을 사용하여 선형 시간에 해결하는 방법을 소개하였지만, 여기서는 연결리스트를 이용하여 문제를 해결하는 방법을 소개합니다. 텍스트 위의 커서는 양방향으로 이동해야 하므로, 효율성을 위해서 이중 연결리스트를 사용합니다. 텍스트의 각 문자는 연결리스트의 각 노드에 저장되며, 커서는 연결리스트의 노드를 가리키는 변수로 표현됩니다.

연결리스트에서는 위치가 지정된 노드의 삽입과 삭제 작업을 상수 시간에 수행하므로, 스택과 마찬가지로 텍스트의 길이와 편집 명령의 수에 대한 선형 시간에 편집기 문제를 해결할 수 있습니다. 프로그램 7.16에 연결리스트를 사용하여 편집기 문제를 해결하는 프로그램이 나와 있습니다. 프로그램에서 사용하는 연결리스트는 두 개의 더미 노드인 헤더 노드와 꼬리 노드를 가진 이중 연결리스트입니다. 프로그램에서 변수 cp는 커서 위치를 나타냅니다. 실제 커서 위치는 cp가 가리키는 노드와 이전 노드 사이에 해당합니다. 그림 7.14처럼 cp가 문자 'b'를 가진 노드를 가리키는 경우, 실제 커서는 문자 'a'와 'b' 사이에 있습니다. cp가 꼬리 노드(tail)를 가리키면 실제 커서는 텍스트의 맨 끝에 위치하고, cp의 이전 노드가 헤더 노드(head)이면 커서는 텍스트의 맨 앞에 위치함을 나타냅니다. 따라서 cp가 헤더 노드를 가리키는 경우는 발생하지 않습니다.

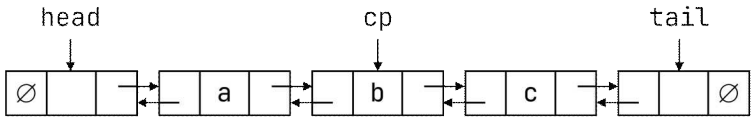


그림 7.14: 커서 위치의 표현

프로그램에는 이중 연결리스트에서 효율적으로 커서의 이동과 삽입 및 삭제를 수행하는 클래스 LinkedList를 정의합니다.

프로그램 7.16: 연결리스트를 이용하여 편집기 문제를 해결하는 프로그램

```
1  class DNode:
2     def __init__(self, letter=None):
3        self.data = letter
4        self.next = None
5        self.prev = None
6
7  class LinkedList:                          # 이중 연결리스트
8     def __init__(self):
9        self.head = DNode(None)             # 헤더 노드 생성
10       self.tail = DNode(None)             # 꼬리 노드 생성
```

```
11      self.head.next = self.tail          # 헤더 노드와 꼬리 노드 연결
12      self.tail.prev = self.head
13      self.cp = self.tail                 # 초기 커서 위치 설정
14
15   def move_left(self):                   # 커서 왼쪽 이동
16      if self.cp.prev is not self.head:   # 맨 앞이 아니면 이동
17         self.cp = self.cp.prev
18
19   def move_right(self):                  # 커서 오른쪽 이동
20      if self.cp is not self.tail:        # 맨 끝이 아니면 이동
21         self.cp = self.cp.next
22
23   def insert(self, letter):              # 커서 왼쪽에 letter 삽입
24      new_node = DNode(letter)
25      new_node.next = self.cp
26      new_node.prev = self.cp.prev
27      self.cp.prev.next = new_node
28      self.cp.prev = new_node
29
30   def delete(self):                      # 커서 왼쪽 문자 삭제
31      if self.cp.prev is not self.head:   # 맨 앞이 아니면 삭제
32         p = self.cp.prev
33         p.prev.next = self.cp
34         self.cp.prev = p.prev
35
36   def get_text(self):                    # 텍스트 반환
37      text = []
38      p = self.head.next
39      while p is not self.tail:
40         text.append(p.data)
41         p = p.next
42      return "".join(text)
43
44 # main program
45 text = input()                    # 입력 텍스트
46 n = int(input())                  # 편집 명령 개수
47 E = LinkedList()                  # 이중 연결리스트 생성
48
49 for x in text:                    # 초기 연결리스트 구성, 커서는 맨 끝에 위치
50    E.insert(x)
51
52 for _ in range(n):                # 편집 작업 수행
53    op = input()                   # 편집 명령 입력
```

```
54    if op[0] == 'L':
55       E.move_left()              # 커서 왼쪽 이동
56    elif op[0] == 'D':
57       E.move_right()             # 커서 오른쪽 이동
58    elif op[0] == 'B':
59       E.delete()                 # 커서 왼쪽 문자 삭제
60    elif op[0] == 'P':
61       E.insert(op.split()[1])    # 커서 왼쪽에 문자 입력
62
63 print(E.get_text())              # 편집 결과 출력
```

그림 7.15은 입력 텍스트 “abcd”에 대해서, 6개의 편집 명령 (L, L, B, P x, D, P y)가 주어질 때, 편집 명령을 차례로 수행하는 과정을 보여줍니다. 그림에서 연결리스트는 일반적인 리스트로 간략하게 표시됩니다.

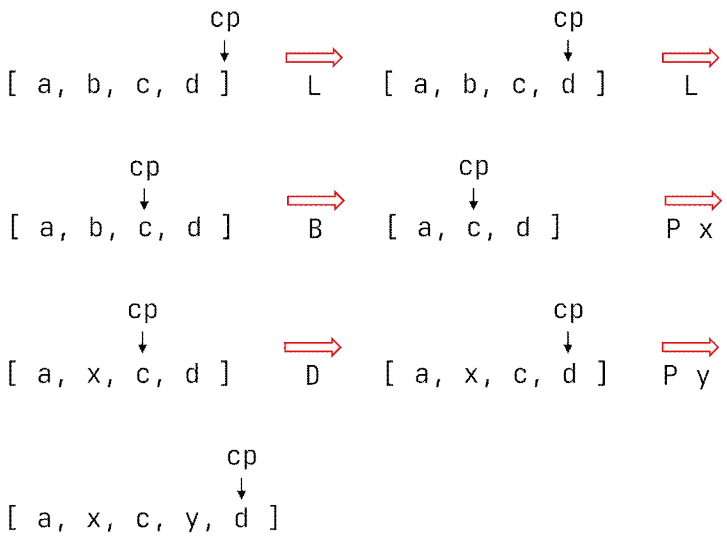


그림 7.15: 연결리스트에서 편집 명령을 수행하는 과정

연습문제

1. 배열로 표현하는 것보다 연결리스트로 표현하는 것이 더 효율적인 자료의 예를 들고 그 이유를 설명하시오.

2. 연결리스트로 표현하는 것보다 배열로 표현하는 것이 더 효율적인 자료의 예를 들고 그 이유를 설명하시오.

3. 단일 연결리스트의 길이, 즉 노드의 개수를 반환하는 함수 `get_length(head)`를 작성하시오. 단, `head`는 시작 노드의 링크를 가진 변수이다.

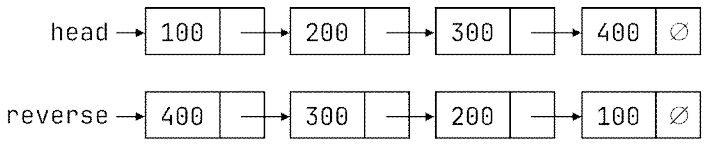
4. 원형 연결리스트의 길이, 즉 노드의 개수를 반환하는 함수 `get_size(tail)`를 작성하시오. 단, `tail`은 원형 연결리스트의 끝 노드의 링크를 가진 변수이다.

5. 단일 연결리스트에서 원소 `item`을 가진 모든 노드를 제거하는 함수 `remove_all(head, item)`을 작성하시오. 단, `head`는 시작 노드의 링크를 가진 변수이다.

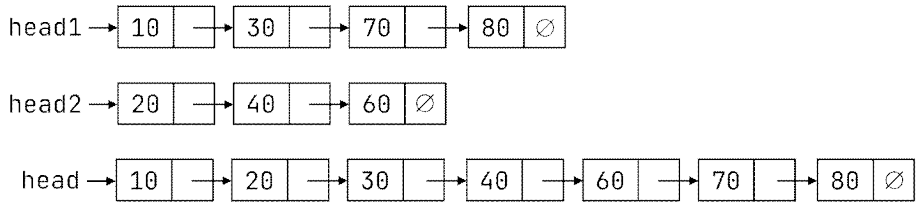
6. 단일 연결리스트에서 뒤에서부터 `k`번째 노드를 찾아 그 노드의 데이터를 반환하는 함수 `find_reverse(head, k)`를 작성하시오. 단, `head`는 시작 노드의 링크를 가진 변수이다.

7. 두 개의 단일 연결리스트 `head1`과 `head2`를 이어 붙인 연결리스트의 시작 노드의 링크를 반환하는 함수 `concat(head1, head2)`를 작성하시오. 연결리스트 `head1`의 끝 노드가 연결리스트 `head2`의 시작 노드를 가리키도록 연결한다. 단, `head1`이 `None`이면 결합된 연결리스트의 시작 노드는 `head2`이다.

8. 연결리스트에서 새로운 노드를 생성하지 않고 링크만 바꾸어서 아래 그림처럼 노드의 순서를 거꾸로 연결한 후에 변경된 연결리스트의 시작 노드의 링크를 반환하는 함수 `reverse_list(head)`를 작성하시오.



9. 두 개의 정렬된 단일 연결리스트를 합병하여 아래 그림처럼 하나의 정렬된 연결리스트를 만드는 함수 merge_lists(head1, head2)를 작성하시오. 단, 새로운 노드를 생성하지 않고 링크만 변경해야 하고, 합병된 연결리스트의 시작 노드를 반환한다.



10. 선택 정렬을 사용하여 단일 연결리스트를 정렬하는 selection_sort_list(head) 함수를 작성하시오. 단, 새로운 노드를 생성하지 않고 링크만 변경해야 하고, 정렬된 연결리스트의 시작 노드를 반환한다.

11. 버블 정렬을 사용하여 단일 연결리스트를 정렬하는 bubble_sort_list(head) 함수를 작성하시오. 단, 새로운 노드를 생성하지 않고 링크만 변경해야 하고, 정렬된 연결리스트의 시작 노드를 반환한다.

12. 삽입 정렬을 사용하여 단일 연결리스트를 정렬하는 insertion_sort_list(head) 함수를 작성하시오. 단, 새로운 노드를 생성하지 않고 링크만 변경해야 하고, 정렬된 연결리스트의 시작 노드를 반환한다. (힌트. 정렬된 구간은 링크를 거꾸로 연결하라.)

13. 프로그램 7.3은 반복문을 사용하여 단일 연결리스트에 저장된 모든 원소를 차례대로 출력한다. 재귀 함수를 사용하여 연결리스트의 모든 원소를 출력하는 함수 print_all_recursive(node)를 작성하시오.

14. n개의 노드를 가진 단일 연결리스트에 저장된 원소들을 역순으로 출력하는 재귀 함수 print_reverse_recursive(node)를 작성하시오. 단, 함수는 몇 개의 변수를 위한 공간 외에는 추가 메모리를 사용하지 않아야 하며, 전체 수행 시간은 $O(n)$이어야 한다.

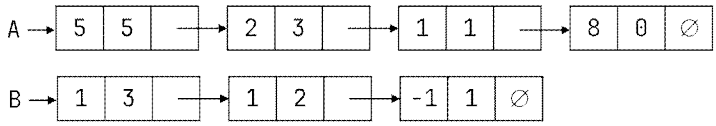
15. 헤더 노드를 가진 원형 이중 연결리스트에서 원소 item을 가진 모든 노드를 제거하는 함수 delete_all(head, item)을 작성하시오. 단, head는 헤더 노드의 링크를 가진 변수이다.

16. 두 개의 헤더 노드를 가진 원형 이중 연결리스트 head1과 head2를 이어 붙인 연결리스트의 헤더 노드의 링크를 반환하는 함수 concatenate(head1, head2)를 작성하시오. 단, head1과 head2는 각각 헤더 노드의 링크를 가진 변수이고, 결과로 만들어지는 연결리스트도 헤더 노드를 가진 원형 이중 연결리스트이다.

17. [실습 문제] 다항식은 계수와 지수로 구성되는 항들의 합이고 지수의 내림차순으로 항들이 나열된다. 예를 들어, 두 다항식 A(x)와 B(x)가 다음과 같다고 하자.

$$A(x) = 5x^5 + 2x^3 + x + 8$$
$$B(x) = x^3 + x^2 - x$$

다항식을 아래 그림처럼 단일 연결리스트로 표현한다고 하자. 연결리스트의 노드는 계수를 저장하는 coef 필드와 지수를 저장하는 expo 필드를 가진다.



두 다항식의 덧셈은 동일한 지수를 가진 항의 계수 합으로 구성된다. 예를 들어, A(x) + B(x)의 결과는 다음과 같다.

$$A(x)+B(x) = 5x^5 + 3x^3 + x^2 + 8$$

두 다항식 p1과 p2의 덧셈 결과로 만들어지는 연결리스트를 반환하는 함수 add_polynomial(p1, p2)를 작성하시오. p1과 p2는 각각 다항식을 표현한 연결리스트이다.

18. [실습 문제] 기차는 여러 개의 객차를 일렬로 연결하여 운행된다. 객차들은 1 이상의 정수로 구분되며, 객차의 양쪽에 다른 객차를 연결할 수 있다. 열차에서 객차 구성을 조작하는 명령은 다음과 같다.

FRONT x	열차의 맨 앞에 객차 x를 추가한다.
BACK x	열차의 맨 뒤에 객차 x를 추가한다.
REMOVE x	번호가 x인 객차를 열차에서 제거한다. (없으면 무시함)
SHOW	현재 객차 순서를 앞에서부터 순서대로 출력한다.

입력으로 첫 줄에 명령의 개수 n이 주어진다. n은 1 이상 100,000 이하이다. 둘째 줄부터 n개의 줄에는 각 줄에 명령이 하나씩 주어진다. 객차 번호 x는 1 이상 1,000,000 이하의 정수이다. 출력으로 SHOW 명령이 나올 때마다 열차의 맨 앞에서부터 마지막까지 객차 번호를 순서대로 한 줄에 출력한다. 객차 번호 사이에는 한 개의 빈칸이 들어간다. 객차가 하나도 없으면 0을 출력한다. 입력과 출력의 예는 다음과 같다.

입력의 예	출력의 예
8 BACK 101 BACK 102 FRONT 100 SHOW REMOVE 102 SHOW REMOVE 100 REMOVE 101 SHOW	100 101 102 100 101 0

제8장

정렬과 탐색

8.1 정렬 문제
8.2 합병 정렬
8.3 퀵 정렬
8.4 분배에 의한 정렬
8.5 탐색과 선택
8.6 사례 연구: 반딧불이
연습문제

학습 목표

- 정렬 알고리즘의 특성인 안정 정렬과 제자리 정렬의 개념을 이해하고, 주요 정렬 알고리즘이 어떤 특성을 갖는지 설명할 수 있다.
- 분할정복법에 기반한 정렬 알고리즘인 합병 정렬과 퀵 정렬의 동작 원리를 이해하고, 파이썬 프로그램으로 직접 구현할 수 있다.
- 분배를 기반으로 하는 버킷 정렬과 기수 정렬의 원리를 이해하고, 비교 기반의 정렬 알고리즘에 비해 더 효율적으로 수행되는 조건을 설명할 수 있다.
- 정렬된 리스트에서 상한과 하한을 찾는 파이썬의 bisect 모듈을 활용할 수 있다.
- 정렬되지 않은 리스트에서 k번째 원소를 찾는 퀵 선택 알고리즘을 구현할 수 있다.

8.1 정렬 문제

정렬(sorting)은 데이터를 특정한 순서, 예를 들어 오름차순이나 내림차순으로 배치하는 작업을 의미합니다. 정렬은 겉보기에는 단순해 보이지만, 데이터를 효율적으로 처리하는 데 핵심적인 역할을 하며, 다양한 알고리즘과 결합하여 복잡한 문제를 해결하는 데에도 활용됩니다. 특히 데이터베이스 검색, 그래프 알고리즘, 네트워크 최적화 등 여러 응용 분야에서 정렬은 필수적인 요소로 자리 잡고 있습니다. 이러한 중요성 때문에, 정렬 알고리즘에 관한 연구는 현재까지도 활발히 진행되고 있으며, 새로운 정렬 기법이 꾸준히 개발되고 있습니다.

정렬 알고리즘을 학습하는 또 다른 이유는 주어진 문제를 다양한 관점에서 접근하는 **알고리즘적 사고(algorithmic thinking)**의 훈련이 될 수 있다는 점입니다. 이를 통해 문제의 본질을 파악하여 상황에 따라 가장 적합한 전략을 선택하는 능력을 익히게 될 것입니다.

3장의 사례 연구에서 선택 정렬, 버블 정렬, 삽입 정렬을 살펴봤습니다. 이들 알고리즘은 모두 최악의 경우 $O(n^2)$의 시간복잡도를 가지지만 간단히 구현할 수 있다는 장점이 있습니다.[15] 또한 선택 정렬은 원소 교환의 횟수가 가장 적다는 특징이 있으며, 버블 정렬은 프로그램 구현 시에 코드 길이를 가장 짧게 작성할 수 있고, 삽입 정렬은 거의 정렬된 데이터에 대해서 다른 어떤 알고리즘보다 빠르게 정렬할 수 있다는 장점을 가지고 있습니다. 이렇게 기본적인 정렬 알고리즘도 나름대로 특징이나 장점을 가지고 있습니다.

가장 빠른 정렬 알고리즘은 $O(n\log n)$ 시간복잡도를 가지는 알고리즘입니다. 합병 정렬, 퀵 정렬, 힙 정렬 등이 여기에 속합니다. 합병 정렬과 퀵 정렬은 '분할정복법'이라는 알고리즘 설계 기법을 기반으로 하는 알고리즘이고, 힙 정렬은 이진 트리를 기반으로 설계된 알고리즘입니다. 합병 정렬과 퀵 정렬은 8.2절과 8.3절에서 소개하고, 힙 정렬은 10.2절에서 힙 자료구조와 함께 소개합니다.

파이썬 내장 함수인 `sorted()` 함수와 리스트 메소드인 `sort()`는 합병 정렬과 삽입 정렬의 조합으로 설계된 알고리즘인 'Timsort'를 기반으로 구현되었으며 최악의 경우 시간복잡도가 $O(n\log n)$입니다. 9장의 사례 연구에서는 '결정 트리'를 이용하여 원소 비교 기반의 알고리즘으로는 $O(n\log n)$보다 더 낮은 시간복잡도를 갖는 정렬 알고리즘은 존재할 수 없다는 것을 증명합니다. 그러므로 시간복잡도가 $O(n\log n)$인 정렬 알고리즘은 시간복잡도 면에서 **최적 알고리즘(optimal algorithm)**입니다.

---

15) 앞으로 다른 언급이 없으면 변수 n은 정렬 대상이 되는 원소의 개수를 나타냅니다.

8.4절에서는 다른 원소와의 비교 없이 원소 자체의 값을 이용하여 원소들을 분배하는 방식으로 정렬하는 버킷 정렬과 기수 정렬을 소개합니다. 이러한 분배에 의한 정렬 알고리즘은 특수한 환경에서는 어떤 비교 기반 정렬 알고리즘보다 빠르게 수행될 수 있습니다.

이 장에서 다루는 정렬 알고리즘은 원소들을 오름차순으로 정렬합니다. 원소들 사이의 비교를 위해 부등호 기호가 사용됩니다. 두 원소 a와 b에 대해, a < b이면 a가 b보다 정렬 순서에서 더 앞쪽에 위치한다는 의미입니다. 만약 원소가 여러 개의 요소로 구성되어 있다면, 비교 대상이 되는 특정 요소를 **키(key)**라고 부릅니다.

정렬 알고리즘의 특징을 설명하는 척도로 '안정 정렬'과 '제자리 정렬'이 있습니다. **안정 정렬(stable sort)**은 동일한 키를 가진 여러 원소가 있을 때 정렬 과정에서 원래의 상대적인 순서가 유지되는 정렬을 말합니다. 정렬 과정의 중간 또는 결과에서 이들의 상대적인 순서가 바뀌면 **불안정 정렬(unstable sort)**이라고 합니다. 버블 정렬과 삽입 정렬은 안정 정렬이고 선택 정렬은 불안정 정렬입니다. 안정 정렬이 좋은 알고리즘인가는 상황에 따라 다릅니다. 데이터의 정렬 기준이 여러 개일 때, 현재 기준으로 정렬하면서 이전 기준에 따른 순서를 유지하려는 상황에서는 안정 정렬이 유리합니다. 예를 들면, 학생 성적 데이터를 정렬하는 작업에서 먼저 '이름'의 오름차순으로 정렬한 후에 연이어 '시험 점수'의 내림차순으로 추가로 정렬할 때, 시험 점수가 같은 학생들 간에는 기존의 이름 순서가 그대로 유지되도록 하고자 하는 경우입니다. 그렇지만, 상대적인 순서가 중요하지 않거나 정렬 속도가 더 우선시되는 상황에서는 안정 정렬 여부를 고려할 필요가 없습니다. 빠른 정렬 알고리즘 중에서는 합병 정렬이 안정 정렬에 속하고, 퀵 정렬과 힙 정렬은 불안정 정렬에 해당합니다.

제자리 정렬(in-place sort)은 추가적인 기억 공간을 거의 사용하지 않고 정렬하는 알고리즘을 의미합니다. 구체적으로 정의하면, 데이터가 저장된 배열이나 리스트 공간 외에 $O(1)$ 크기의 추가 공간만을 사용하는 알고리즘을 제자리 정렬이라고 합니다. 선택 정렬, 버블 정렬, 삽입 정렬은 모두 제자리 정렬입니다. 제자리 정렬은 기억 공간 면에서 효율적이지만 수행 시간이 더 중요한 경우에는 제자리 정렬 여부보다는 시간복잡도를 우선적으로 고려해야 합니다. 빠른 정렬 알고리즘 중에서는 힙 정렬이 제자리 정렬에 해당하며, 합병 정렬과 퀵 정렬은 제자리 정렬이 아닙니다. 제자리 정렬을 엄밀하게 정의할 때는 $O(1)$의 추가 공간만 사용하는 것을 의미하지만, 문헌에 따라서는 $O(\log n)$ 크기의 추가 공간만 사용해도 제자리 정렬로 간주하기도 합니다. 이런 느슨한 정의에서는 퀵 정렬도 제자리 정렬에 속합니다.

정렬 알고리즘은 데이터를 처리하는 방식에 따라 '내부 정렬'과 '외부 정렬'로 구분할 수 있습니다. **내부 정렬(internal sort)**은 정렬 대상 데이터 전체가 메모리(RAM)에 올라와 있는 상태에서 모든 정렬 작업이 이루어지는 방식을 의미합니다. 이 책에서 다루는 모든 정렬 알고리즘은 내부

정렬에 해당합니다. 반면, **외부 정렬(external sort)**은 정렬해야 할 데이터의 용량이 너무 커서 한꺼번에 메모리에 올릴 수 없는 환경, 즉 하드 디스크나 SSD와 같은 보조 기억 장치에 대용량의 데이터가 저장되어 있을 때 사용하는 방식입니다. 이 경우, 데이터를 작은 블록으로 나누어 메모리로 읽어와 부분적으로 정렬한 후 다시 보조 기억 장치에 저장하는 과정을 반복합니다. 이렇게 부분적으로 정렬된 파일들을 나중에 합병(merge)하여 최종적으로 정렬된 결과를 얻게 됩니다. 대용량 데이터베이스나 빅데이터 처리 시스템에서 주로 합병 정렬을 기반으로 하는 외부 정렬 기법이 활용됩니다. 이 장에서는 내부 정렬에 해당하는 여러 알고리즘을 소개하는 데 초점을 맞추고 있어, 외부 정렬에 대해서는 더 이상 다루지 않습니다.

8.2 합병 정렬

합병 정렬과 퀵 정렬은 분할정복법 기반으로 설계된 알고리즘입니다. **분할정복법(divide and conquer)**은 입력 데이터를 쪼개어 작은 크기로 만들어 문제를 해결하려는 알고리즘 설계 전략입니다. 작은 크기로 분할된 입력 데이터를 대상으로 하는 문제를 원래 문제의 **부문제(sub problem)** 또는 크기가 작아진 문제라고 합니다. 분할정복법 알고리즘은 분할 단계, 정복 단계, 결합 단계로 구성되며, 각 단계의 역할은 다음과 같습니다.

1. **분할(divide)** : 입력 데이터의 크기가 충분히 작으면(예를 들어, 1 이하 또는 50 이하 등), 분할을 수행하지 않고 직접 문제를 풀어서 답을 구한다. 그렇지 않으면 입력 데이터를 두 개 이상의 서로 분리된 그룹으로 나눈다.
2. **정복(conquer)** : 분할된 각 데이터 그룹에 대해 재귀적으로 문제를 해결한다.
3. **결합(combine)** : 정복 단계에서 구한 부문제의 해를 이용하여 원래 문제의 답을 구한다.

분할 정복법 기반의 알고리즘은 주로 재귀 함수로 구현합니다. 알고리즘 8.1은 분할정복법 알고리즘의 전형적인 구조를 보여줍니다.

알고리즘 8.1: 분할정복법 알고리즘의 기본 구조

```
def dac(data):                          # 입력 데이터 data에 대한 재귀 함수
   if len(data) < k:                    # data의 크기가 충분히 작으면 (종료 조건)
      return direct_solve(data)         # 직접 문제를 풀어서 답을 반환
   else:
```

```
    D = partition(data)          # 분할 단계 - D는 분리된 데이터의 그룹
    S = [dac(d) for d in D]      # 정복 단계 - S는 부문제의 해의 그룹
    return combine(S)            # 결합 단계 - 원래 문제의 답을 반환
```

합병 정렬(merge sort)은 분할정복법과 재귀 함수에 기반을 둔 알고리즘으로 분할 단계, 정복 단계, 결합 단계에서 다음과 같이 수행합니다.

1. **분할 단계** : 정렬 대상인 리스트 S의 길이가 1 이하이면 이미 정렬된 상태이므로 그대로 작업을 종료한다. 그렇지 않으면, 중앙을 기준으로 S의 왼쪽 구간과 오른쪽 구간으로 구성되는 리스트 L과 R을 만든다. 이때, L과 R의 길이 차이는 1 이하이다.
2. **정복 단계** : 두 리스트 L과 R에 대해 각각 재귀적으로 정렬을 수행한다.
3. **결합 단계** : 정복 단계의 결과로 나온 두 개의 정렬된 리스트 L과 R을 합병하여 하나의 정렬된 리스트 S를 구한다.

예를 들어, 그림 8.1처럼 정렬하려는 리스트 S = (78, 35, 85, 95, 93, 55, 87, 65)가 주어지면, 분할 단계에서 정중앙을 기준으로 S를 분할하여 두 개의 리스트 L = (78, 35, 85, 95)와 R = (93, 55, 87, 65)를 만듭니다. 그다음 정복 단계에서 재귀적으로 정렬을 수행하면 L과 R은 정렬된 상태가 됩니다. 즉, L = (35, 78, 85, 95), R = (55, 65, 87, 93)이 됩니다. 마지막 결합 단계에서 두 정렬된 리스트 L과 R을 합병하여 하나의 정렬된 리스트 S = (35, 55, 65, 78, 85, 87, 93, 95)를 구합니다.

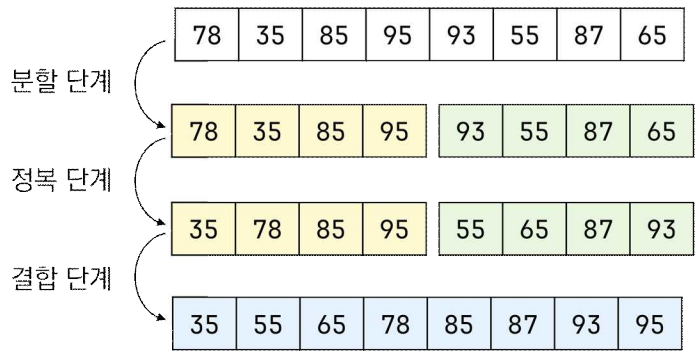


그림 8.1: 합병 정렬의 기본 전략

프로그램 8.1의 함수 merge_sort(S)는 합병 정렬 알고리즘으로 리스트 S를 정렬하는 재귀 함수입니다. 이 함수에서 호출하는 merge(S, L, R)은 프로그램 8.2에 나와 있습니다.

프로그램 8.1: 합병 정렬

```
1  def merge_sort(S):
2     """합병 정렬 알고리즘으로 리스트 S를 정렬한다."""
3     if len(S) <= 1:          # 길이가 1 이하이면 작업 종료 (종료 조건)
4        return
5     mid = len(S) // 2        # 중앙 위치 계산
6     L = S[:mid]              # S의 왼쪽 구간 (새로운 리스트 생성)
7     R = S[mid:]              # S의 오른쪽 구간 (새로운 리스트 생성)
8     merge_sort(L)            # 리스트 L의 정렬 (재귀 호출)
9     merge_sort(R)            # 리스트 R의 정렬 (재귀 호출)
10    merge(S, L, R)           # L과 R을 합병하여 정렬된 리스트 S를 구함
```

두 개의 정렬된 리스트를 하나의 정렬된 리스트로 결합하는 작업을 **합병(merge)** 또는 **병합**이라고 합니다. 합병은 두 리스트의 첫 번째 요소부터 각 리스트의 요소를 순차적으로 비교해 가면서 작은 요소부터 차례대로 저장하는 방식으로 수행합니다. 프로그램 8.2의 함수 merge(S, L, R)는 두 정렬된 리스트 L과 R을 합병하여 정렬된 리스트 S를 만듭니다. 합병된 결과가 저장될 리스트 S는 함수 호출 시에 인자로 전달된 원본 리스트로서, S의 길이는 두 리스트 L과 R의 길이 합과 같으므로 인덱싱을 사용하여 요소들을 저장합니다.

프로그램 8.2: 합병을 수행하는 프로그램

```
1  def merge(S, L, R):
2     """두 정렬된 리스트 L과 R을 합병하여 정렬된 리스트 S를 구축한다."""
3     i = j = 0
4     while i < len(L) and j < len(R):
5        if L[i] < R[j]:
6           S[i+j] = L[i]      # L[i]가 S의 다음 요소가 됨
7           i += 1
8        else:
9           S[i+j] = R[j]      # R[j]가 S의 다음 요소가 됨
10          j += 1
11
12    while i < len(L):         # L에 남아 있는 요소를 S에 배정함
13       S[i+j] = L[i]
14       i += 1
15
16    while j < len(R):         # R에 남아 있는 요소를 S에 배정함
17       S[i+j] = R[j]
18       j += 1
```

그림 8.2는 합병을 진행하는 과정의 예를 보여줍니다. 리스트 L과 R에서 각각 첨자 i와 j 위치의 두 요소를 비교하여 더 작은 쪽을 리스트 S의 다음 위치에 저장합니다.

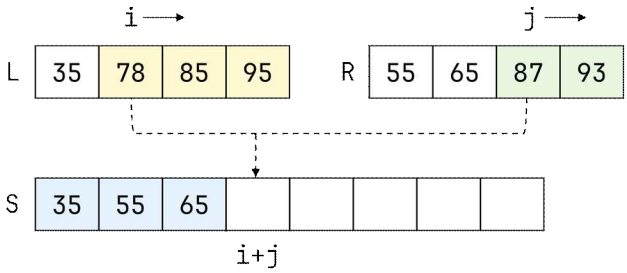


그림 8.2: 합병 진행 과정

그림 8.3은 합병 정렬의 전체 수행 과정의 예를 보여줍니다. 그림에서 표시된 번호는 재귀적으로 수행되는 합병 정렬의 작업 순서를 나타냅니다.

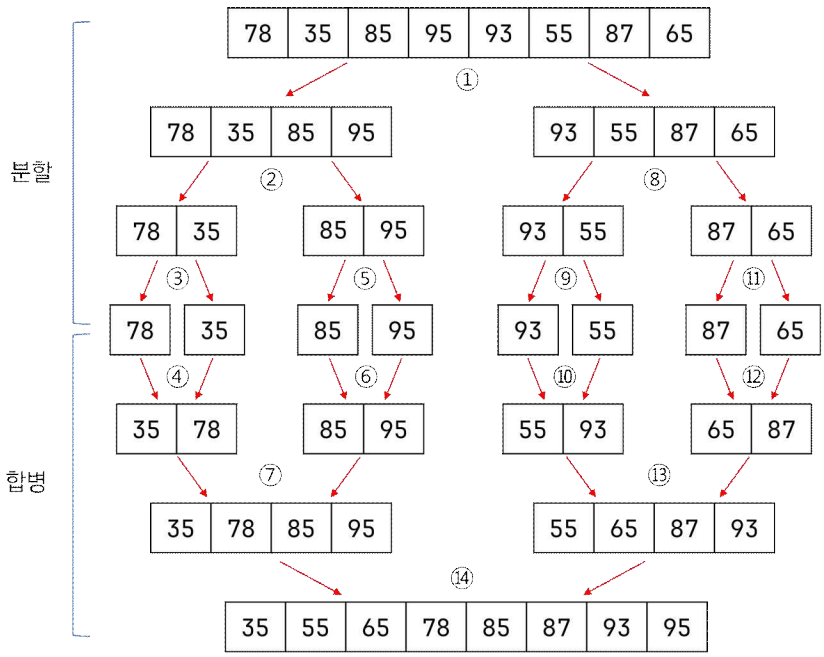


그림 8.3: 합병 정렬의 수행 과정

이제 합병 정렬의 시간복잡도를 알아보겠습니다. 먼저, 함수 merge(S, L, R)의 수행 시간을

분석해 봅시다. while 문은 한 번 반복될 때마다 하나의 요소가 S에 저장됩니다. L과 R 두 리스트의 길이의 합이 n이라고 할 때, S에는 총 n개의 요소가 저장되므로 while 문은 n번 반복됩니다. while 코드 블록은 상수 시간에 수행되니까 merge() 함수의 시간복잡도는 $O(n)$입니다.

다음으로, 함수 merge_sort(S)의 수행 시간을 분석해 봅니다. S의 길이가 n일 때 이 함수의 시간복잡도를 T(n)이라고 합시다. 합병 정렬은 분할 과정에서 크기 차이가 1 이하인 두 리스트로 분할하고, 합병 과정이 선형 시간에 수행되므로, T(n)을 다음과 같은 재귀 관계로 표현할 수 있습니다.

$$T(1) = d$$
$$T(n) = 2T(\lceil n/2 \rceil) + c \cdot n \qquad (n \geq 2)$$

여기서 c와 d는 상수입니다. 간편하게 계산하기 위해 $n = 2^k$(즉, $k = \log n$)라고 두면, 위 재귀 관계의 폐쇄형을 다음과 같이 구할 수 있습니다.

$$\begin{aligned} T(n) &= 2T(n/2) + c \cdot n \\ &= 2(2T(n/2^2) + c \cdot n/2) + c \cdot n = 2^2T(n/2^2) + 2c \cdot n \\ &= 2^2(2T(n/2^3) + c \cdot n/2^2) + 2c \cdot n = 2^3T(n/2^3) + 3c \cdot n \\ &= 2^kT(n/2^k) + k \cdot c \cdot n = 2^kT(1) + k \cdot c \cdot n \\ &= d \cdot n + c \cdot n\log n \end{aligned}$$

따라서, $T(n) = O(n\log n)$입니다. 합병 정렬은 데이터의 초기 분포 특성과는 무관하게 일관된 수행 시간을 보입니다. 즉, 데이터가 거의 정렬된 상태이거나 역순으로 정렬되었거나 무작위로 구성된 상태이거나 등에 상관없이 일정한 작업이 진행됩니다. 그러므로 합병 정렬의 시간복잡도는 최선의 경우, 최악의 경우, 그리고 평균의 경우 모두 $O(n\log n)$입니다.

이제, 합병 정렬에서 사용하는 메모리 공간을 살펴볼까요? 합병 정렬은 분할 단계에서 리스트를 거의 같은 크기로 나누기 때문에, 리스트의 길이가 1 이하가 될 때까지 연속적으로 발생하는 재귀 호출의 수, 즉 최대 재귀 깊이(recursion depth)는 $\log n$입니다. 이는 재귀 호출로 인해 시스템 스택에 누적되는 활성화 레코드의 최대 개수가 $\log n$임을 의미합니다. 그런데 분할 단계에서 분할된 데이터를 저장하기 위해 두 개의 새로운 리스트 L과 R을 추가로 생성합니다. 이 추가되는 두 리스트 길이의 합은 재귀 호출 시 전달되는 리스트 S의 길이와 같습니다. 맨 처음 호출 시에는 L과 R의 길이 합이 n, 그다음은 n/2, 이렇게 크기가 1 또는 0이 될 때까지 절반씩 줄어듭니다. 이 크기를 다 더하면 다음과 같이 계산됩니다.

$$n + n/2 + n/4 + \cdots + n/2^k = n(1 + 1/2 + 1/2^2 + \cdots + 1/2^k) < 2n$$

따라서 재귀 호출의 가장 깊은 단계에서 시스템 스택에 동시에 존재하는 임시 리스트들의 길이 합은 $O(n)$입니다. 그러므로, 입력 데이터가 저장된 리스트 외에 추가로 필요한 메모리 크기는 $O(n)$입니다. 이런 이유로 합병 정렬은 제자리 정렬이 아닙니다. 합병 정렬이 가장 빠른 정렬 알고리즘 중의 하나이지만, $O(n)$의 추가 메모리 공간이 필요하다는 점은 합병 정렬의 단점이라고 볼 수 있습니다.

마지막으로 합병 정렬이 안정 정렬인지를 알아보겠습니다. 합병 정렬은 분할 과정에서 요소들의 상대적인 순서를 바꾸지 않습니다. 합병 과정에서 L과 R에 키가 같은 요소가 있는 경우, 항상 L의 요소부터 S에 저장합니다. 또한 왼쪽부터 차례대로 합병을 진행하기 때문에, 같은 키를 가진 요소들의 상대적인 순서가 바뀌지 않습니다. 따라서 합병 정렬은 안정 정렬입니다.

8.3 퀵 정렬

퀵 정렬(quick sort)은 말 그대로 빠른 정렬 알고리즘으로 널리 알려져 있습니다. 퀵 정렬도 합병 정렬과 마찬가지로 분할정복법에 기반을 두고 있지만, 합병 정렬과는 달리 분할 단계에서 리스트를 균등하게 분할하지 않을 수 있다는 특징이 있습니다. 퀵 정렬에서 분할 단계, 정복 단계, 결합 단계의 역할은 다음과 같습니다.

1. **분할 단계** : 정렬 대상인 리스트 S의 길이가 1 이하이면 이미 정렬된 상태이므로 그대로 작업을 종료한다. 그렇지 않으면, S의 요소 중 하나를 **피벗(pivot)**으로 선택하여, 피벗보다 작은 요소들을 리스트 L에, 피벗보다 큰 요소들을 리스트 R에, 그리고 피벗과 키가 같은 요소들을 리스트 C에 저장한다.
2. **정복 단계** : L과 R을 각각 재귀적으로 정렬한다.
3. **결합 단계** : L과 C, 그리고 R을 차례로 이어 붙여 하나의 정렬된 리스트 S를 구성한다.

예를 들어, 그림 8.4처럼 정렬하려는 리스트 S = (65, 35, 65, 55, 85, 44, 12, 78)에서 피벗으로 65가 선택된다면, 분할 과정을 통해 리스트 L에는 65보다 작은 요소들을, C에는 65인 요소들을, R에는 65보다 큰 요소들을 배치합니다. 그 결과 L, C, R은 각각 (35, 55, 44, 12), (65, 65), (85, 78)이 됩니다. 정복 단계에서는 두 리스트 L과 R에 대해 각각 재귀적으로 정

렬합니다. 결합 단계에서 이들 세 리스트를 단순히 이어 붙이는 것만으로 전체 리스트는 정렬된 상태가 됩니다.

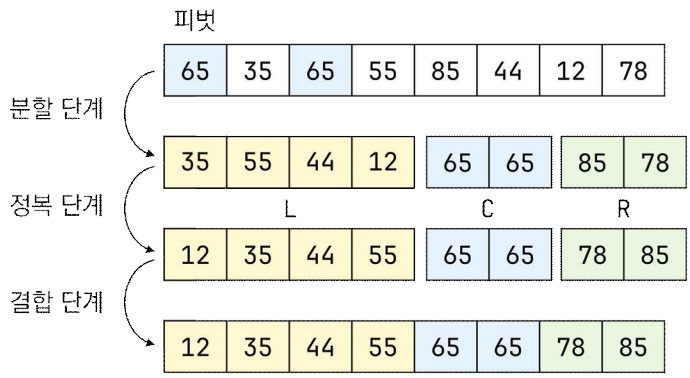


그림 8.4: 퀵 정렬의 기본 전략

프로그램 8.3의 함수 quick_sort(S)는 퀵 정렬의 기본 전략에 따라 리스트 S를 정렬하는 재귀 함수입니다.

프로그램 8.3: 퀵 정렬

```
1  def quick_sort(S):
2     """퀵 정렬 알고리즘으로 리스트 S를 정렬한다."""
3     if len(S) <= 1:              # 길이가 1 이하이면 작업 종료
4        return
5
6     L, C, R = [], [], []
7     pivot = S[0]                 # 첫 번째 요소를 피벗으로 선택
8     for x in S:                  # 리스트 분할
9        if x < pivot:
10          L.append(x)
11       elif x > pivot:
12          R.append(x)
13       else:
14          C.append(x)
15
16    quick_sort(L)                # L의 정렬
17    quick_sort(R)                # R의 정렬
18    S[:] = L + C + R             # 리스트 이어 붙임
```

S의 길이가 n일 때 quick_sort(S)의 수행 시간을 T(n)이라고 합시다. 최악의 경우 시간복잡도를 분석하기 위해, 리스트 분할 결과로 생성된 리스트 C의 길이가 1인 경우, 즉 피벗에 해당하는 요소가 유일하다고 가정하겠습니다. 그러면 리스트 L의 길이를 k라고 할 때, 리스트 R의 길이는 n-k-1이 됩니다. 분할은 선형 시간에 수행되므로, T(n)은 다음과 같이 재귀 관계로 표현됩니다.

$$T(1) = d$$
$$T(n) = T(k) + T(n-k-1) + c \cdot n \qquad (n \geq 2)$$

여기서 c와 d는 상수입니다.

퀵 정렬의 수행에서 가장 이상적인 상태는 분할 과정에서 리스트가 균등하게 분할되는 것입니다. 반대로 최악의 경우는 피벗으로 가장 작은 요소 또는 가장 큰 요소가 계속 선택되어 L 또는 R 중의 한쪽이 계속 비게 되는 상황이 발생하는 것입니다. 예를 들면 프로그램 8.3의 7줄처럼 피벗을 첫 번째 요소로 선택하는 경우, 이미 정렬되었거나 역순으로 정렬된 리스트를 정렬하려고 할 때 이러한 상황이 연속적으로 발생합니다. 이 경우, 분할 결과로 리스트 L과 R 중에 한쪽의 크기가 n-1이 됩니다. 따라서 퀵 정렬의 최악의 경우 시간복잡도 W(n)은 다음과 같습니다.

$$W(n) = W(n-1) + c \cdot n$$
$$= O(n^2)$$

이처럼 퀵 정렬의 최악의 경우 시간복잡도는 선택 정렬이나 삽입 정렬과 마찬가지로 $O(n^2)$이어서 효율적이라고 말할 수 없습니다. 분할 과정에서 리스트를 균등하게 나누기 위해서는 리스트의 중앙값(median)을 피벗으로 이용해야 합니다. 이 장의 8.5절에서 언급하는 '**중앙값의 중앙값(median of median)**'이라는 기법을 사용하여 리스트의 중앙값을 최악의 경우 $O(n)$ 시간에 찾을 수는 있지만, 시간복잡도에 붙는 상수 요소가 너무 커서 실용적이지 않습니다. 그 대안으로 피벗을 무작위로 선택하는 전략을 사용하면 최악의 경우가 연속적으로 발생할 가능성을 최소화할 수 있습니다. 예를 들어, 피벗을 다음과 같이 난수를 이용하여 선택하면 최악의 경우가 발생하는 것을 확률적으로 피할 수 있습니다.

```
pivot = random.choice(S)    # random 모듈을 이용하여 요소 선택
```

또는 난수를 이용하는 것 대신에 리스트의 맨 앞, 가운데, 맨 뒤에 있는 요소 중의 중앙값을 피벗으로 선택할 수도 있습니다.

퀵 정렬이 빠르다는 것은 이 정렬이 평균적으로 매우 빠른 수행 시간을 갖기 때문입니다. 리스

트의 모든 요소가 동일한 확률로 피벗이 될 수 있다면, 퀵 정렬의 평균 수행 시간 A(n)은 다음과 같이 구해집니다.

$$A(n) = \frac{1}{n}[(A(0)+A(n-1)) + (A(1)+A(n-2)) + \cdots + (A(n-1)+A(0))]$$
$$= \frac{2}{n}(A(1) + A(2) + \cdots + A(n-1))$$

이 식을 풀면, A(n)은 $O(n\log n)$이 됩니다.

매우 작은 개수의 요소들로 구성된 리스트를 정렬하는 경우는 3장에서 소개한 기본적인 정렬 알고리즘이 퀵 정렬이나 합병 정렬보다 더 빠르게 수행됩니다. 이는 입력의 크기가 작을 때의 수행 시간은 시간복잡도 함수에 붙는 상수 요소에 큰 영향을 받기 때문입니다. 입력의 크기가 대략 10 이하일 때는 기본적인 정렬 알고리즘을 이용하는 것이 대체로 유리하다는 것을 성능 측정을 통해 알 수 있습니다. 따라서 퀵 정렬의 성능을 개선하기 위해, 재귀 함수의 종료 조건(len(S) <= 1)을 특정 임계값으로 확장하여, 리스트의 크기가 작은 경우(예를 들어, len(S) <= 10)에는 재귀 호출을 하지 않고 직접 삽입 정렬 등으로 정렬하는 전략을 도입할 수 있습니다. 이러한 전략을 **하이브리드 정렬(hybrid sort)**이라고 부르는 데, 프로그래밍 언어에서 제공하는 정렬 라이브러리의 대부분은 이 전략을 채택하고 있을 정도로 실용적인 기법입니다.

이제, 퀵 정렬의 공간복잡도를 살펴보겠습니다. 먼저, 재귀 함수로 구현된 퀵 정렬의 수행 과정에서 필요한 시스템 스택의 메모리 공간에 대해 알아봅니다. 리스트의 길이가 n일 때의 최대 재귀 깊이를 C(n)이라고 합시다. 분할 단계에서 길이가 n-1인 리스트가 생성되는 최악의 경우가 계속 발생하면 C(n)은 다음과 같이 정의됩니다.

$$C(1) = 1$$
$$C(n) = C(n-1) + 1 \qquad (n \geq 2)$$

위 재귀 관계를 풀면 C(n) = n입니다. 즉, 재귀 호출로 인해 시스템 스택에 누적되는 활성화 레코드의 최대 개수가 n입니다. 게다가 프로그램 8.3은 분할 단계에서 분할된 데이터를 저장하기 위해 세 개의 리스트 L, C, R을 생성합니다. 재귀 호출 과정에서 길이가 n-1인 리스트를 생성하는 최악의 경우가 계속 발생한다면 추가되는 리스트의 총 크기는 $n + n-1 + \cdots + 1 = O(n^2)$입니다.

퀵 정렬의 분할 과정에서 생성되는 두 리스트 L과 R에 대해 호출 순서를 적절히 조절하면 최대 재귀 깊이를 줄일 수 있습니다. 분할 후에 두 리스트의 길이를 비교하여 작은 쪽을 먼저 재귀

호출한다면, 현재 정렬하려는 리스트의 길이는 항상 분할 전 리스트 길이의 절반 이하가 됩니다. 따라서 길이가 1이 될 때까지 연속적으로 발생하는 재귀 호출의 수는 logn을 넘지 않습니다. 이는 최대 재귀 깊이의 상한이 logn임을 의미합니다. 또한 이 전략을 사용하면, 정렬 과정에서 생성되는 리스트의 총 크기가 합병 정렬에서와 마찬가지로 $O(n)$이 됩니다.

프로그램 8.3은 퀵 정렬을 구현하는 가장 간단한 방법입니다. 그런데 이렇게 구현하면 분할 단계에서 생성되는 리스트로 인해 퀵 정렬은 제자리 정렬이 될 수 없습니다. 퀵 정렬의 구현 전략을 다르게 설계하면, 퀵 정렬이 느슨한 정의로서 제자리 정렬이 되도록 할 수 있습니다. 즉, $O(\log n)$ 크기의 추가 공간만을 사용하여 구현하는 것입니다. 이 전략은 분할 단계에서 추가 리스트를 생성하지 않고 분할된 요소들을 원래 리스트 내에서 재배치합니다. 이 구현 전략에서 분할 단계와 정복 단계의 역할은 다음과 같습니다. 정복 단계의 결과로 리스트는 정렬된 상태가 되기 때문에, 이 전략에서는 별도로 결합 단계가 필요하지 않습니다.

1. **분할 단계** : 정렬 대상인 리스트 S의 길이가 1 이하이면 이미 정렬된 상태이므로 그대로 작업을 종료한다. 그렇지 않으면, 리스트 요소 중 하나를 피벗으로 선택하여, 리스트의 왼쪽 구간에는 피벗보다 작은 요소들을 배치하고 오른쪽 구간에는 피벗보다 크거나 같은 요소들을 배치한다.
2. **정복 단계** : 피벗을 기준으로 리스트의 왼쪽 구간과 오른쪽 구간을 각각 재귀적으로 정렬한다. 두 구간 중에서 크기가 작은 구간부터 먼저 정렬한다.

예를 들어, 그림 8.5처럼 정렬하려는 리스트 S = (78, 35, 93, 55, 85, 44, 12, 65)에서 피벗으로 65가 선택된다고 하면, 분할 과정을 통해 리스트에서 65의 왼쪽에는 65보다 작은 요소들이, 오른쪽에는 65보다 크거나 같은 요소들이 배치됩니다. 그 결과가 (35, 55, 44, 12, 65, 93, 78, 85)라고 가정해 봅시다. 정복 단계에서는 (35, 55, 44, 12)와 (93, 78, 85) 두 구간에 대해 각각 재귀적으로 정렬합니다. 이미 피벗보다 작은 모든 요소는 리스트의 왼쪽 구간에, 피벗보다 큰 모든 요소는 오른쪽 구간에 배치된 상태이므로, 정복 단계를 거치고 나면 리스

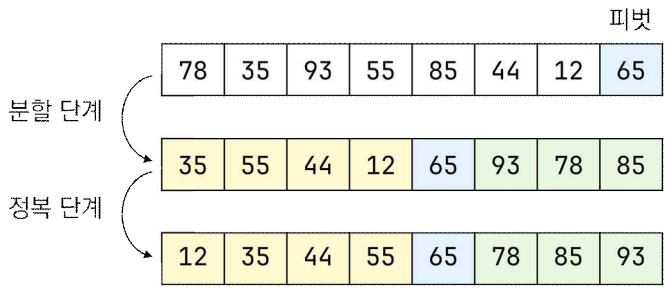


그림 8.5: 제자리 정렬을 위한 퀵 정렬의 기본 전략

트 전체가 정렬된 상태가 됩니다.

위에서 설명한 것처럼 리스트를 추가 생성하지 않으면 시스템 스택에 적재되는 활성화 레코드의 크기는 $O(1)$이므로, 분리된 두 구간 중에 크기가 작은 구간부터 먼저 정렬하면 시스템 스택을 $O(\log n)$ 크기로 유지할 수 있습니다.

재배치 전략으로 분할을 수행하는 알고리즘은 여러 가지가 있습니다. 그중에서 가장 간단한 방법으로 알려진 **로무토 분할(Lomuto partition)** 알고리즘을 구현한 프로그램이 프로그램 8.4에 나와 있습니다. 널리 사용되는 또 다른 분할 알고리즘은 연습문제에서 소개합니다.

프로그램 8.4: 퀵 정렬의 분할 프로그램

```
1  def partition(S, start, end):
2      """리스트 S의 start부터 end까지의 구간을 분할한다."""
3      pivot = S[end]                      # 맨 끝 요소를 피벗으로 선택
4      i = start-1                         # 피벗보다 작은 요소의 마지막 첨자 저장
5      for j in range(start, end):
6          if S[j] < pivot:                # 피벗보다 작으면
7              i += 1                      # 피벗보다 작은 요소 추가됨
8              S[i], S[j] = S[j], S[i]     # S[i]와 S[j]를 교환함
9
10     S[i+1], S[end] = S[end], S[i+1]     # S[i+1]과 S[end]를 교환함
11     return i+1                          # 피벗의 최종 위치(첨자) 반환
```

프로그램 8.4의 parition(S, start, end)는 리스트 S에서 첨자 구간 (start, end)에 대해 분할을 수행하고 분할의 기준이 되는 피벗의 위치(첨자)를 반환합니다. 먼저, 구간의 맨 끝 요소를 피벗으로 선택합니다. 구간의 왼쪽부터 차례로 조사하면서 피벗보다 작은 요소는 구간의 왼편에, 피벗보다 크거나 같은 요소는 오른편에 배치합니다. 변수 j는 현재 조사하려는 요소를 참조하고, 변수 i는 첨자 j의 왼편에서 피벗보다 작은 마지막 요소를 참조합니다. S[j]가 피벗보다 작으면 S[j]를 S[i+1]과 교환하여, 피벗보다 작은 요소들의 구역을 확장함으로써 변수 i가 항상 피벗보다 작은 요소 구역의 마지막 첨자를 가질 수 있도록 합니다. 모든 요소를 조사한 후에 마지막으로 S[end]와 S[i+1]을 교환하여 분할을 마무리합니다. 그림 8.6은 분할을 수행하는 예를 보여줍니다. 그림 8.6에서 ①은 초기 상태이고 ② ~ ⑤는 변수 j가 피벗보다 작은 요소를 찾은 상태입니다. 이때 S[i+1]과 S[j]가 교환되고 i는 1 증가됩니다. ⑥은 마지막 요소까지 처리된 상태이고, ⑦은 피벗과 S[i+1]을 교환하여 분할 작업이 끝난 상태를 나타냅니다.

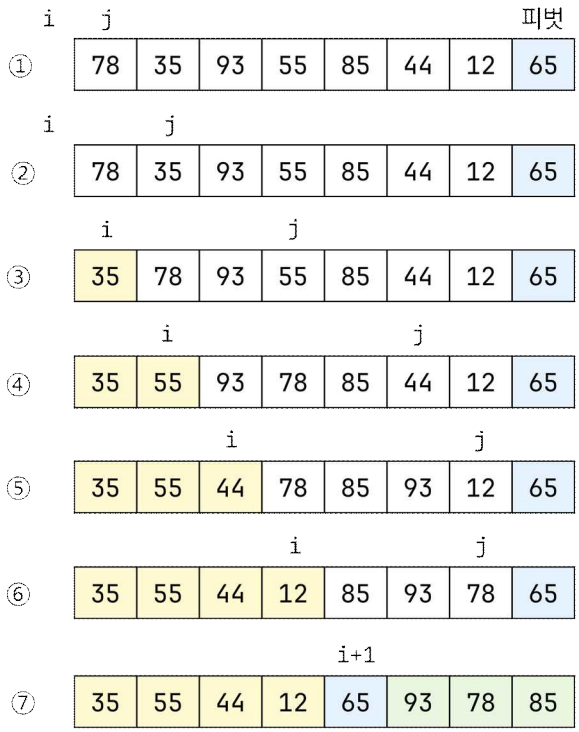


그림 8.6: 퀵 정렬에서 분할을 수행하는 예

함수 partition()은 구간 내의 요소를 피벗과 한 번씩 비교합니다. 따라서 함수는 구간의 길이에 대한 선형 시간에 수행됩니다. 프로그램 8.5는 partition()을 호출하는 퀵 정렬 프로그램입니다. quicksort(S, start, end)는 리스트 S에서 첨자 구간 (start, end)에 대해 정렬을 수행합니다. 맨 처음 호출은 quicksort(S, 0, len(S)-1)입니다.

프로그램 8.5: 제자리 정렬로 동작하는 퀵 정렬 알고리즘의 구현

```
1  def quicksort(S, start, end):
2     """퀵 정렬 알고리즘으로 리스트 S를 정렬한다."""
3     if start >= end:                    # 정렬 범위가 1 이하이면 작업 종료
4        return
5     pos = partition(S, start, end)      # 리스트 분할
6     if pos-start < end-pos:             # 왼쪽 구간의 크기가 더 작으면
7        quicksort(S, start, pos-1)       # 왼쪽 구간부터 정렬
8        quicksort(S, pos+1, end)
9     else:                               # 오른쪽 구간의 크기가 더 작으면
10       quicksort(S, pos+1, end)         # 오른쪽 구간부터 정렬
11       quicksort(S, start, pos-1)
```

퀵 정렬은 안정 정렬일까요? 퀵 정렬을 프로그램 8.3과 같이 구현하면 안정 정렬이 될 수 있습니다. 그러나 프로그램 8.4의 분할 알고리즘을 사용하는 퀵 정렬은 불안정 정렬입니다. 분할 과정에서 멀리 떨어진 두 요소를 교환하는 과정에서 정렬의 안정성을 깨뜨리는 상황이 발생할 수 있습니다. 예를 들어, 정렬 구간이 $(1, 5^1, 5^2, 2)$일 때 피벗이 2이면, 분할 과정의 마지막에서 2와 5^1이 교환되어 분할 결과는 $(1, 2, 5^2, 5^1)$이 됩니다. 일반적으로 퀵 정렬 알고리즘을 언급하는 경우는 프로그램 8.4와 프로그램 8.5로 구현된 제자리 정렬 버전을 의미합니다. 그래서 보통 퀵 정렬을 불안정 정렬이라고 합니다.

8.4 분배에 의한 정렬

8.1절에서 $O(n\log n)$의 시간복잡도보다 더 효율적인 정렬 알고리즘은 존재할 수 없다고 했습니다. 이 문장은 "원소끼리 비교하는 방식"으로 수행하는 알고리즘에 한정됩니다. 이 절에서 소개하는 알고리즘은 다른 원소와의 비교 없이 원소 자체의 값(정확하게는 정렬 기준이 되는 키의 값)만을 가지고 원소들을 분배(distribution)하는 방식으로 정렬합니다. 분배 기반의 알고리즘은 특수한 환경에서는 선형 시간에 수행하기도 합니다. 이 절에서 키에 대한 참조가 필요한 경우에 원소 e의 키를 key(e)로 표시합니다.

먼저 **버킷 정렬(bucket sort)**이라고 부르는 간단한 알고리즘부터 소개합니다. 버킷은 원소를 저장하는 장소입니다. 버킷 정렬에서 사용하는 버킷의 수는 키가 가지는 값의 종류에 의해 결정됩니다. 예를 들어, 키가 [0, M-1] 범위의 정수일 경우, M개의 버킷을 사용하여 정렬을 수행합니다. 정렬 과정은 입력 데이터의 각 원소를 순차적으로 해당 버킷에 저장한 후, 순서대로 버킷의 원소들을 이어 붙이는 것입니다. 프로그램 8.6에 버킷 정렬을 구현한 프로그램이 나와 있습니다.

프로그램 8.6: 버킷 정렬

```
1 def bucket_sort(S, M):
2     """키의 범위가 [0, M-1]인 요소들의 리스트 S를 정렬한다."""
3     B = [[] for _ in range(M)]        # M개의 공백 버킷 생성
4     for e in S:                       # S의 요소들을 해당 버킷에 저장
5         B[key(e)].append(e)           # key(e)는 e의 키를 의미함
6     S.clear()                         # S의 모든 요소 삭제 - 버킷 비움
7     for i in range(M):                # 버킷의 요소들을 S에 이어 붙임
8         S.extend(B[i])
```

함수 bucket_sort(S, M)은 0과 M-1 사이의 정수인 키를 가진 요소들의 리스트 S를 정렬합니다. 이 프로그램에는 기존의 정렬 알고리즘과 달리 두 요소를 직접 비교하는 연산이 없고, 단지 요소의 분배 과정을 통해 정렬을 수행합니다. bucket_sort()의 수행 시간은 $O(n+M)$입니다. M의 값이 적당히 작으면 bucket_sort()는 매우 효율적으로 수행되나, M의 값이 너무 크면 추가로 필요한 메모리 공간과 수행 시간의 증가로 오히려 기존의 비교 기반 알고리즘보다 비효율적으로 수행될 수 있습니다.

다음으로 버킷 정렬을 확장한 **기수 정렬(radix sort)**을 소개합니다. 기수 정렬에서는 키가 여러 개의 요소로 구분됩니다. 예를 들어, 키 k_i가 다음과 같이 d개의 요소로 구성되는 선형리스트로 표현된다고 합시다.

$$k_i = (k_i^0, k_i^1, \cdots, k_i^{d-1})$$

여기서, a < b이면 k_i^a는 k_i^b보다 **더 중요한 요소(more significant digit)**라고 하고, 특히 k_i^0는 **가장 중요한 요소(most significant digit; MSD)**, k_i^{d-1}은 **가장 덜 중요한 요소(least significant digit; LSD)**라고 합니다. 이때, 두 키의 순서는 키의 요소들에 대한 사전식 순서(lexicographic order) 방식으로 정해집니다. 즉, 두 키 k_i와 k_j에 대해 다음의 두 문장 ①과 ②는 동일합니다, 다시 말해 서로 필요충분조건이라고 말할 수 있습니다.

① $k_i < k_j$

② $k_i^0 < k_j^0$ 이거나,
$k_i^0 = k_j^0$ 이고 $k_i^1 < k_j^1$ 이거나,
⋮
$k_i^0 = k_j^0$ 이고 $k_i^1 = k_j^1$ 이고 ⋯ 이고 $k_i^{d-2} = k_j^{d-2}$ 이고 $k_i^{d-1} < k_j^{d-1}$ 이다.

예를 들어, 길이가 8인 문자열이 키로 사용된다고 하면, 키 "python"은 길이가 8인 문자들의 선형리스트 ('p', 'y', 't', 'h', 'o', 'n', '\0', '\0')로 표현됩니다. 여기서, 길이가 8보다 작으면, 널 문자('\0')로 채웁니다. 다른 예로, 8자리의 십진수 숫자가 키로 사용된다고 하면, 키 "53128"은 자릿수들의 선형리스트 (0, 0, 0, 5, 3, 1, 2, 8)로 표현될 수 있습니다. 키의 요소가 가질 수 있는 값 범위의 크기를 **기수(radix)**라고 부릅니다. 예를 들어, 키의 요소가 알파벳 소문자인 경우는 기수가 26이고, 키가 십진수인 경우는 기수가 10입니다.

기수 정렬은 원소들을 키의 각 요소에 대한 버킷 정렬을 수행하는 방식으로 진행됩니다. 키의 요소가 d개 있을 때, 기수 정렬은 d번의 **패스(pass)**로 구성됩니다. 패스는 키의 특정 요소에 대해서 분배와 수집을 수행하는 것을 말하고, d번의 패스가 수행되면 정렬이 완료됩니다. 키 요소

들을 분배하는 순서에 따라 MSD 방식과 LSD 방식이 있습니다. MSD 방식의 기수 정렬은 가장 중요한 키의 요소부터 차례로 분배하는 정렬이고, LSD 방식의 기수 정렬은 가장 덜 중요한 키의 요소부터 차례로 분배하는 정렬입니다. MSD 방식은 분배 결과로 만들어지는 부분 리스트들을 따로 정렬해야 하지만, LSD 방식은 독립된 부분 리스트를 유지할 필요가 없어 구현이 더 간단합니다.

예를 들어, 0과 M-1 사이의 십진수들의 리스트 S를 정렬한다고 합시다. 십진수의 각 자릿수를 키의 요소로 하는 기수 정렬을 이용한다면, 기수는 10이고 키의 요소 개수, 즉 패스의 수 d = $\lfloor \log_{10}(M-1) \rfloor + 1$입니다. 프로그램 8.7의 함수 radix_sort(S, M)은 LSD 방식으로 기수 정렬을 수행하는 프로그램입니다. 이 함수는 일의 자릿수부터 시작하여 가장 높은 자릿수까지 차례로 버킷 정렬을 수행합니다.

프로그램 8.7: 기수 정렬

```
1  def radix_sort(S, M):
2     """키의 범위가 [0, M-1]인 십진수들의 리스트 S를 정렬한다."""
3     x = 1          # 현재 처리할 자릿수 (1의 자리, 10의 자리, 100의 자리, ...)
4     while x < M:
5        B = [[] for _ in range(10)]   # 10개의 공백 버킷 생성
6        for e in S:                   # 분배 작업
7           B[(e//x)%10].append(e)     # 현재 자릿수에 해당하는 버킷에 저장
8        S.clear()                     # S의 모든 요소 삭제
9        for i in range(10):           # 수집 작업
10          S.extend(B[i])             # 버킷의 요소들을 이어 붙임
11       x *= 10                       # 다음 자릿수로 이동
```

radix_sort(S, M)에서 각 패스는 $O(n)$ 시간에 수행됩니다. 여기서 n은 리스트 S의 길이입니다. 패스의 수 d = $\lfloor \log_{10}(M-1) \rfloor + 1$이므로, 총 수행 시간은 $O(dn) = O(n \log M)$입니다. 표 8.1은 리스트 S = (304, 123, 212, 21, 321, 441, 343, 103)이고 M = 1000일 때 radix_sort(S, M)의 수행 과정을 보여줍니다. 패스의 수는 3이고, i번째 패스 후에는 리스트 S는 현재 자릿수 이하 부분이 정렬된 상태가 됩니다.

일반적으로 기수 정렬에서 리스트 S의 길이를 n, 기수를 R, 패스의 수를 d라고 하면, 시간복잡도는 $O(d(n+R))$이 됩니다. 만일, R이 상수이면 이 시간복잡도는 $O(dn)$이 되고, d도 상수이면 $O(n)$으로 선형 시간이 됩니다. 기수 정렬은 d와 R의 값이 적당히 작으면 매우 효율적으로 수행되나, 그렇지 않으면 기존의 비교 기반 알고리즘보다 수행 시간이 더 길어질 수 있습니다.

표 8.1: 프로그램 8.7의 수행 과정 (d = 3, 기수 = 10)

pass	x (현재 자릿수)	리스트 S (패스 후)							
		0	1	2	3	4	5	6	7
초기 상태		304	123	212	21	321	441	343	103
1	1	21	321	441	212	123	343	103	304
2	10	103	304	212	21	321	123	441	343
3	100	21	103	123	212	304	321	343	441

8.5 탐색과 선택

정렬과 마찬가지로 **탐색(searching)**은 컴퓨터 응용에서 매우 빈번하게 수행되는 작업입니다. 탐색의 대상인 데이터를 구분할 때, **정적 데이터(static data)**는 한 번 구성되면 더 이상 수정되지 않는 데이터를 말하고, **동적 데이터(dynamic data)**는 원소의 삽입과 삭제가 계속해서 발생하는 데이터를 말합니다. **반동적 데이터(semi-dynamic data)**는 새로운 원소만 추가되고 삭제는 발생하지 않는 데이터를 말합니다. 우리 주변에서 볼 수 있는 사전류와 전화번호부는 정적 데이터에 해당하고, 가입과 탈퇴가 자유로운 집단의 회원 명부는 동적 데이터에 해당합니다.

일반적으로 탐색을 위한 자료구조를 크게 세 가지로 분류할 수 있습니다. 첫 번째는 데이터를 선형리스트로 구성하는 방법이고, 두 번째는 트리 구조, 세 번째는 해싱 구조입니다. 선형리스트는 정적 데이터에 적합한 자료구조입니다. 선형리스트로 표현된 데이터에 대해 탐색, 삽입, 삭제라는 기본적인 탐색 연산 모두를 효율적으로 처리하기는 어렵습니다. 정렬된 배열은 탐색에는 효율적이지만 삽입과 삭제에는 비효율적이고, 연결리스트는 탐색에 비효율적이어서 보통 탐색이 동반되는 삽입과 삭제 역시 비효율적입니다. 그래서 동적 데이터와 반동적 데이터는 보통 트리 구조나 해싱으로 처리합니다. 이 절에서는 선형리스트로 표현된 데이터를 처리하는 탐색 작업에 대해 살펴보고, 탐색을 위한 트리 구조는 10장과 13장에서, 해싱은 13장에서 자세하게 다룹니다.

선형리스트로 구성된 데이터는 선형리스트의 표현 방식에 따라 배열 구조가 될 수도 있고 연결리스트 구조가 될 수도 있습니다. 또한 정렬된 리스트로 구성할 수도 있고 그렇지 않을 수도 있습니다. 선형리스트에서 탐색을 수행하는 기본적인 알고리즘에는 순차 탐색과 이진 탐색이 있습니다. **순차 탐색(sequential search)**은 리스트의 첫 번째 원소부터 시작하여 각 원소의 키

를 탐색 키와 차례로 비교하는 방법입니다. **탐색 키**는 탐색의 대상이 되는 키를 말합니다. 순차 탐색의 수행 시간은 $O(n)$입니다. 순차 탐색은 데이터가 배열이나 연결리스트로 표현되어 있거나 정렬되지 않은 상태에서도 수행할 수 있습니다. 데이터가 정렬된 배열 구조로 표현되어 있으면, 4장의 사례 연구에서 살펴봤듯이 이진 탐색을 사용할 수 있습니다. **이진 탐색(binary search)**은 $O(\log n)$ 시간에 수행되므로 순차 탐색에 비해 매우 효율적입니다.

파이썬에서 제공하는 bisect 모듈은 정렬된 파이썬 리스트에서 이진 탐색을 수행하는 함수를 제공합니다. 아래 설명에서 L은 정렬된 리스트이고 x는 삽입할 요소입니다. 리스트의 정렬 기준을 사용자가 직접 결정해야 하는 경우는 키워드 인수 key에 정렬 기준에 해당하는 함수를 배정합니다.

① bisect.bisect_left(L, x, key=None)
   x보다 크거나 같은 요소 중에서 가장 작은 요소의 최초 위치를 반환한다. x보다 크거나 같은 요소가 없으면 len(L)을 반환한다. 즉, L에서 반환된 위치 이전에는 x보다 작은 요소만(있다면) 존재한다.

② bisect.bisect_right(L, x, key=None) 또는
   bisect.bisect(L, x, key=None)
   x보다 큰 요소 중에서 가장 작은 요소의 최초 위치를 반환한다. x보다 큰 요소가 없으면 len(L)을 반환한다. 즉, L에서 반환된 위치 이후에는 x보다 큰 요소만(있다면) 존재한다.

③ bisect.insort_left(L, x, key=None)
   먼저, bisect_left()를 호출한 후 반환된 위치에 x를 삽입한다.

④ bisect.insort_right(L, x, key=None) 또는
   bisect.insort(L, x, key=None)
   먼저, bisect_right()를 호출한 후 반환된 위치에 x를 삽입한다.

이진 탐색을 수행하는 두 함수 bisect_left()와 bisect_right()는 $O(\log n)$ 시간에 수행되고, 삽입을 수행하는 두 함수 insort_left()와 insort_right()는 $O(n)$ 시간에 수행됩니다. 프로그램 8.8에는 bisect 모듈을 이용하는 프로그램 예가 나와 있습니다.

프로그램 8.8: bisect 모듈의 사용 예

```
1 from bisect import *
2
3 L = [5, 12, 18, 25, 25, 25, 30, 30, 32, 38]
4
```

```
print(bisect_left(L, 12))    # 1
print(bisect_left(L, 19))    # 3
print(bisect_left(L, 25))    # 3
print(bisect_left(L, 40))    # 10

print(bisect_right(L, 12))   # 2
print(bisect_right(L, 19))   # 3
print(bisect_right(L, 25))   # 6
print(bisect_right(L, 40))   # 10

insort_left(L, 30)
insort_right(L, 31)
print(L)          # [5, 12, 18, 25, 25, 25, 30, 30, 30, 31, 32, 38]
```

마지막 주제로, 정렬되지 않은 배열에서 특정한 순위를 가진 원소를 찾는 탐색에 대해 살펴봅니다. 이러한 탐색을 **선택(selection)**이라고 부릅니다. 배열에서 가장 큰 원소나 가장 작은 원소는 한 번의 순차 탐색으로 찾을 수 있습니다. 그래서 $O(n)$ 시간에 답을 구할 수 있습니다. 이제, k번째 작은 원소를 찾는 문제를 고려해 봅시다. 이 문제는 순차 탐색을 k번 수행하면 해결됩니다. 이 전략의 수행 시간은 $O(kn)$입니다. 만일 k가 매우 작은 값이면 이 알고리즘은 매우 빠르게 답을 찾을 것입니다. 반대로 k가 n에 가까운 매우 큰 값이면 순차 탐색을 역으로 수행하여 빠르게 답을 찾을 수도 있습니다. 가장 나쁜 상황은 k가 중간 순위에 가까운 경우입니다. 만일 $k = \lfloor n/2 \rfloor$이면, 이 전략의 수행 시간은 $O(n^2)$입니다. 이 경우는 배열을 정렬한 후에 k번째 원소를 찾는 방법보다 비효율적입니다. 미리 정렬을 수행하는 방식은, $O(n\log n)$ 시간을 들여 정렬한 이후에 상수 시간에 답을 찾습니다. 특정 순위의 원소를 찾는 질의(query)가 매우 많으면 미리 정렬을 수행하는 전략이 효과적입니다. 그러나 그런 질의가 한 번 또는 몇 번뿐이라면 정렬을 수행하는 것은 너무 비용이 큰 작업이라는 생각이 들 수 있습니다.

지금부터는 이 선택 문제를 선형 시간에 해결하는 전략에 대해 알아봅니다. 이 전략은 퀵 정렬과 유사한 **퀵 선택(quick select)**이라고 불리는 매우 실용적인 알고리즘입니다. 퀵 선택도 분할 정복법을 기반으로 설계되었습니다. 퀵 정렬과 마찬가지로 분할 단계에서 피벗을 기준으로 리스트를 분할합니다. 정복 단계에서 퀵 정렬은 두 부분 리스트에 대해 재귀적으로 수행하지만, 퀵 선택은 두 부분 리스트 중에서 k번째 작은 요소가 존재하는 리스트 하나에 대해서만 재귀적으로 수행하면 됩니다. 물론, 운이 좋으면 피벗이 k번째 작은 요소가 될 수도 있습니다.

분할 단계에서 세 개의 리스트 L, C, R이 생성된다고 합시다. L과 C의 길이가 각각 a와 b라고 하죠. 만일 k가 a보다 작으면, L을 대상으로 탐색을 수행하면 됩니다. 그렇지 않은 경우, k가

a+b보다 작으면, C에 존재하는 모든 요소는 키가 같으므로, 피벗이 k번째 작은 요소가 됩니다. 위의 경우가 아니면 k번째 요소는 R에 존재하고, 원래 리스트에서 k번째 순위는 R에서는 k-a-b번째 순위입니다. 그래서 R에서 k-a-b번째 요소를 찾기 위한 탐색을 수행하면 됩니다. 프로그램 8.9의 함수 quick_select(S, k)는 리스트 S에서 k번째 작은 요소를 반환합니다. 만일 k가 유효한 값이 아니면, 즉 0보다 작거나 len(S) 이상이면, -1을 반환합니다.

프로그램 8.9: 퀵 선택

```
def quick_select(S, k):
    """리스트 S에서 k번째 작은 요소를 반환한다. 0 <= k < len(S)."""
    if not (0 <= k < len(S)):      # k가 유효한 값이 아니면 -1을 반환
        return -1
    if len(S) == 1:                # 길이가 1이면 작업 종료
        return S[0]

    L, C, R = [], [], []
    pivot = random.choice(S)       # 피벗을 무작위로 선택함 (random 모듈)
    for x in S:                    # 리스트 분할
        if x < pivot:
            L.append(x)
        elif x > pivot:
            R.append(x)
        else:
            C.append(x)

    if k < len(L):                 # k번째 작은 요소는 L에 존재함
        return quick_select(L, k)  # 재귀 호출 (L에서 탐색)
    elif k < len(L)+len(C):        # k번째 작은 요소는 피벗임
        return pivot               # k번째 작은 요소 반환
    else:                          # k번째 작은 요소는 R에 존재함
        return quick_select(R, k-len(L)-len(C))    # 재귀 호출 (R에서 탐색)
```

퀵 정렬과 마찬가지로 퀵 선택도 최악의 경우 시간복잡도는 $O(n^2)$입니다. 분할 결과 한쪽 리스트의 길이가 n-1이 될 수 있기 때문입니다. 그러나 프로그램 8.9의 함수 quick_select()처럼 난수를 이용하여 피벗을 선택하면, 최악의 경우가 발생하는 것을 확률적으로 피할 수 있습니다. 이렇게 하면 퀵 선택의 기대 시간복잡도는 $O(n)$이 됩니다. 퀵 선택의 평균 시간복잡도 역시 $O(n)$으로 알려져 있습니다. 퀵 선택의 평균 시간복잡도를 분석하는 과정은 복잡하여 이 책에서는 다루지 않습니다.

참고로 선택 문제에 대해서 최악의 경우에도 $O(n)$ 시간에 수행하는 알고리즘이 존재합니다.

이 알고리즘도 퀵 선택과 유사합니다. 차이점은 피벗을 선택할 때 **중앙값의 중앙값(median of median)**이라는 기법을 사용하는 것입니다. 중앙값의 중앙값 기법으로 선택된 피벗을 가지고 분할을 수행하면, 부분 리스트들의 길이는 비교적 균등해집니다. 간단히 설명하면, 리스트의 요소들을 5 또는 7개씩 그룹을 지은 후에, 각 그룹에서 중앙값을 찾고, 그 중앙값들의 중앙값을 재귀적으로 찾습니다. 이렇게 찾은 값을 '중앙값의 중앙값'이라고 부르고, 이를 피벗으로 사용하여 분할하는 것입니다. 이 알고리즘은 이론적으로는 최악의 경우에도 O(n)을 보장하지만, 시간복잡도에 붙는 상수 요소가 너무 커서 무작위 피벗을 사용하는 퀵 선택보다 실용적이지는 않습니다. 알고리즘의 상세한 설명은 생략합니다.

8.6 사례 연구: 반딧불이

크로아티아 프로그램 경시대회에 출제되었던 BUDA라는 문제[16]를 살펴보겠습니다.

일본 반딧불이 한 마리가 장애물로 가득한 동굴로 날아왔다. 장애물은 석순(바닥에서 솟아오른 암석)과 종유석(천장에서 고드름처럼 매달려 있는 암석)이다. 동굴의 길이는 N 단위(N은 짝수)이고 높이는 H 단위이다. 첫 번째 장애물은 석순이고, 그다음에 종유석과 석순이 번갈아 가며 나타난다.

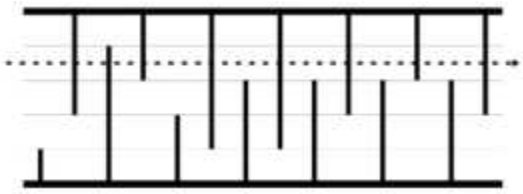
아래 그림은 길이가 14 단위, 높이가 5 단위로 이루어진 동굴의 예이다. 각 길이 단위마다 석순 또는 종유석이 있고, 높이는 5개의 구간으로 구분된다.



반딧불이는 장애물 주위를 날아다니는 유형이 아니라 동굴의 한쪽 끝에서 시작하여 동굴의 끝까지 일정한 높이(특정 구간의 중간)로 날아가면서 만나는 모든 장애물을 뚫고 나아간다. 위

16) 출처: Croatian Highschool Competitions in Informatics 2007의 Regional Competition - Juniors 4번 문제

예에서 밑에서 4번째 구간이 선택되면 아래 그림처럼 반딧불이가 8개의 장애물을 파괴한다.



반딧불이가 얼마나 많은 장애물을 파괴해야 하는지 알고 싶다. 반딧불이가 1번째 구간 또는 5번째 구간을 선택해서 날아가면 7개의 장애물을 파괴한다. 이 두 구간이 파괴해야 할 장애물의 수가 가장 작은 구간이다.

동굴의 길이와 높이, 각 장애물의 크기가 주어질 때, 반딧불이가 동굴 끝에 도달하기 위해 파괴해야 하는 장애물의 최소 개수와 그러한 구간의 개수를 출력하는 프로그램을 작성하시오.

입력 데이터의 첫 줄에는 동굴의 길이와 높이를 나타내는 두 정수 N과 H가 주어진다(2 ≤ N ≤ 200,000, 2 ≤ H ≤ 500,000). N은 항상 짝수이다. 그다음 N개의 줄에는 장애물의 크기가 순서대로 한 줄에 하나씩 주어진다. 장애물의 크기는 H보다 작은 양수이다. 프로그램은 반딧불이가 파괴해야 할 장애물의 개수와 그러한 구간의 개수를 빈칸으로 구분하여 한 줄에 출력한다. 프로그램의 최대 수행 시간은 1초로 제한된다. 입력과 출력의 예는 다음과 같다.

입력	출력
6 7 1 5 3 3 5 1	2 3

입력	출력
14 5 1 3 4 2 2 4 3 4 3 3 3 2 3 3	7 2

위 문제를 해결하는 단순한 해법은 H개의 각 높이 구간에 대해 독립적으로 순차 탐색으로 반딧불이가 뚫어야 하는 장애물을 찾는 것입니다. 구간마다 검사해야 할 장애물의 개수가 최대 200,000이고 구간의 개수가 최대 500,000이므로, 검사 횟수의 최댓값은 500,000 × 200,000 = 10^{11}입니다. 이 작업을 위해서는 수행 제한 시간인 1초를 훨씬 초과하는 시간이 필요합니다.

이 문제를 효율적으로 해결하기 위한 아이디어는 다음과 같습니다. 반딧불이가 크기가 X인 석순(또는 종유석)을 파괴할 경우, X보다 크거나 같은 모든 석순(또는 종유석)이 파괴됩니다. 장애물을 얼마나 파괴하느냐는 장애물의 상대적 순서와는 아무런 상관이 없습니다. 따라서 석순과 종유석을 따로 분류하여 미리 크기를 기준으로 정렬해 둔다면, 각 구간에 대해 이진 탐색을 이용할 수 있습니다. 이진 탐색으로 구간의 높이보다 큰 석순 중에 가장 작은 석순의 위치를 찾으면 정렬된 리스트에서 파괴될 석순의 수를 첨자 계산으로 구할 수 있습니다. 종유석 리스트에 대해서는 구간의 높이를 H에서 뺀 값을 가지고 이진 탐색을 수행하면 됩니다.

이렇게 이진 탐색을 이용하면, 각 구간에서 파괴할 장애물의 개수를 $O(\log N)$ 시간에 구할 수 있습니다. 그러면 총 수행 시간은 $O(H \log N)$이 됩니다. H와 N의 최댓값으로 계산해 보면, 500,000 × log200,000 < 10^7이므로, 1초의 제한 시간 안에 충분히 수행될 수 있습니다. 프로그램 8.10에 반딧불이 문제를 위한 프로그램이 나와 있습니다.

프로그램 8.10: 반딧불이 문제를 위한 파이썬 프로그램

```
1  from bisect import *
2
3  N, H = map(int, input().split())   # 동굴의 길이와 높이 입력
4
5  A, B = [], []                      # 석순과 종유석 리스트
6  for _ in range(N//2):              # 석순과 종유석 높이 입력
7     A.append(int(input()))          # 석순 입력
8     B.append(int(input()))          # 종류석 입력
9
10 A.sort()                           # 석순 높이의 오름차순 정렬
11 B.sort()                           # 종유석 높이의 오름차순 정렬
12
13 min_obs = N                        # 파괴할 장애물의 최소 개수
14 count = 1                          # 최소 개수의 장애물을 파괴하는 구간의 수
15 for y in range(H):                 # 각 높이 구간에 대해서
16    a = bisect_left(A, y+1)         # A에서 파괴되는 석순의 최초 위치
17    b = bisect_left(B, H-y)         # B에서 파괴되는 종유석의 최초 위치
18    num = N-a-b                     # 파괴되는 장애물의 수
19
```

```
20    if num < min_obs:                # 파괴되는 장애물의 수가 더 작은 구간 발견
21       min_obs = num
22       count = 1
23    elif num == min_obs:             # 최소 개수의 장애물을 파괴하는 구간 발견
24       count += 1
25
26 print(min_obs, count)                # 장애물 개수와 구간의 수 출력
```

연습문제

1. 안정 정렬이 왜 중요한지에 대한 예시로서 8.1절에서 소개했던 학생 성적 데이터를 정렬하는 예 외에 추가적인 예시를 들어보시오.

2. 선형리스트 L에서 L[i] > L[j]인 순서쌍 (i, j)를 **반전(inversion)**이라고 한다. 정수 리스트에서 반전의 총 개수를 세는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

3. 합병 정렬을 이용하여 O(nlogn) 시간에 2번에서 언급한 반전의 총 개수를 세는 알고리즘을 설계하시오.

4. 스택에 n개의 숫자가 들어 있다. 추가로 2개의 스택과 몇 개의 변수만 사용하여 정렬하는 알고리즘을 설계하시오.

5. 합병 정렬 알고리즘을 프로그램 8.1과 같은 재귀 함수가 아닌 반복 구조로 수행하는 프로그램을 작성하시오.

6. 연결리스트로 표현된 선형리스트를 합병 정렬을 사용하여 정렬하는 프로그램을 작성하시오. 단, 새로운 노드를 생성하지 않고 노드의 링크만 변경해야 한다.

7. 두 개의 정렬된 리스트 $(a_0, a_1, \cdots, a_{n-1})$과 $(b_0, b_1, \cdots, b_{n-1})$이 있을 때, 전체 2n 개의 원소 중에서 가장 큰 원소와 n번째로 큰 원소를 찾으려고 한다. 이 두 원소를 찾기 위한 효율적인 방법을 설명하시오.

8. 퀵 정렬의 수행 시간을 개선하기 위한 여러 가지 방법들을 적용한 프로그램을 구현하여, 원래의 퀵 정렬 프로그램에 비해 얼마만큼 개선되었는지를 실험·분석하시오.

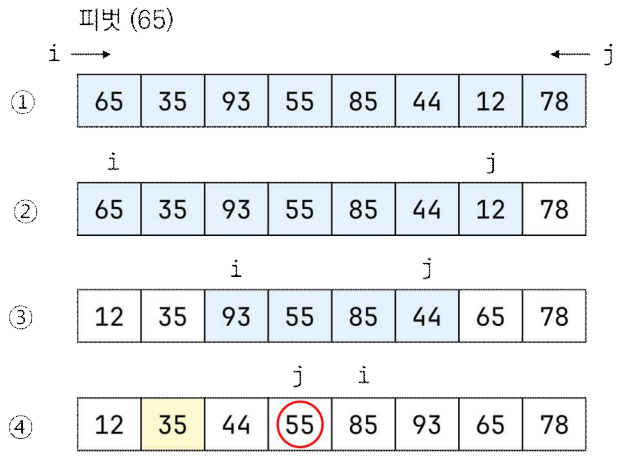
9. 정수 리스트 L과 정수 target이 주어질 때, 리스트의 두 요소의 합이 target이 되는 경우를 모두 찾는 프로그램을 작성하시오. 단, 시간복잡도는 O(nlogn) 이어야 한다. (힌트. 리스트를 먼저 정렬한다.)

10. 버킷 정렬은 안정 정렬인 이유를 설명하시오.

11. 0과 1로만 구성된 리스트를 제자리 정렬 방식으로 정렬하는 선형 시간 알고리즘을 설계하시오.

12. 퀵 정렬에서 **호어 분할(Hoare partition)**이라고 불리는 리스트 분할 알고리즘은 다음과 같다: 리스트 L에서 피벗을 선택한 후에 첨자를 저장하는 두 변수 i와 j를 사용하여 분할을 수행한다. 변수 i는 L의 왼쪽 끝에서 시작하여 L[i]가 피벗보다 크거나 같을 때까지 값을 증가시키고, 변수 j는 L의 오른쪽 끝부터 시작하여 L[j]가 피벗보다 작거나 같을 때까지 값을 감소시킨다. 두 변수가 멈춘 시점에서 i < j이면 L[i]와 L[j]를 서로 교환한다. 이 과정은 i >= j일 때까지 반복한다. 마지막 j의 위치를 기준으로 리스트를 L[:j+1]과 L[j+1:]로 분할한다.

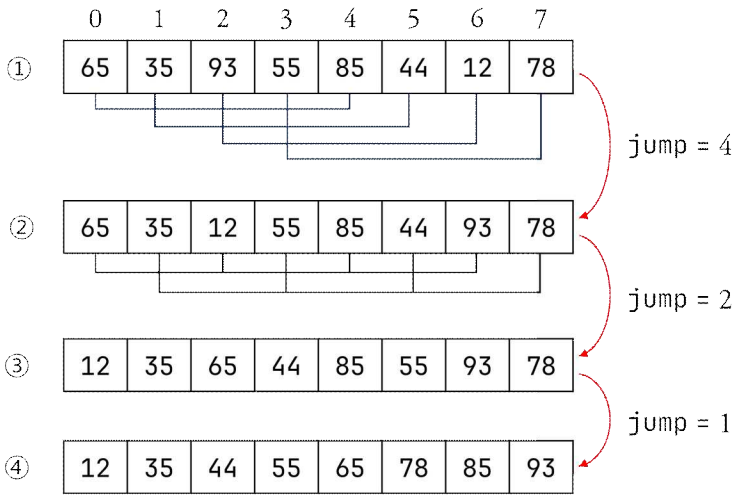
    예를 들면, 아래 그림에서 65를 피벗으로 선택했다고 가정하자. ① 초기 단계에서 변수 i와 j는 각각 리스트의 양 끝을 벗어난 위치로 지정한다. ② 피벗보다 같거나 큰 요소를 만날 때까지 변수 i를 증가시키면 i는 65의 위치가 되고, 피벗보다 같거나 작은 요소를 만날 때까지 변수 j를 감소시키면 j는 12의 위치가 된다. 이때, 65와 12의 위치를 교환한다. ③ 다음 i가 멈추는 위치는 93의 위치이고, j가 멈추는 위치는 44의 위치이다. 여기서 93과 44의 위치를 교환한다. ④ 다음 i가 멈추는 위치는 85의 위치이고, j가 멈추는 위치는 55의 위치이다. 이때 i >= j이므로, j의 위치를 분할 위치로 결정한다.



    호어 분할 알고리즘을 사용하여 퀵 정렬을 수행하는 프로그램을 구현하시오.

13. **쉘 정렬(shell sort)**는 삽입 정렬의 단점을 보완하기 위해 고안된 정렬 알고리즘이다. 쉘 정렬은 '점프(jump)'라는 개념을 도입하여, 점프 간격만큼 떨어져 있는 원소들 사이에서 삽입 정렬을 수행한다. 처음에는 점프 간격이 커서 멀리 떨어져 있는 원소들 사이에서 교환이 이루어지며, 점진적으로 점프 간격을 축소해 나간다. 최종적으로 점프 간격이 1이 되면 리스트 전체에 대해서 삽입 정렬을 수행하며 작업을 마친다.

예를 들어, 점프 간격이 4, 2, 1 순으로 구성된 쉘 정렬을 수행한다고 하자. 아래 그림에서 ①은 초기 단계로서 리스트에서 4칸 떨어져 있는 숫자 그룹 (65, 85), (35, 44), (93, 12), (55, 78)에 대해서 각각 삽입 정렬을 수행한다. 그 결과는 ②와 같다. 그다음, 점프 간격이 2가 되면, 2칸 떨어져 있는 숫자 그룹 (65, 12, 85, 93)과 (35, 55, 44, 78)에 대해서 각각 삽입 정렬을 수행한다. 그 결과, 리스트는 ③의 상태가 되고, 마지막으로 점프 간격 1, 즉 일반적인 삽입 정렬을 수행하면 ④처럼 정렬된 상태가 된다.



기존의 선택 정렬이 한 번의 비교로 반전 하나를 없애는 데 반해서, 쉘 정렬은 초기 과정에서는 한 번의 비교로 다수의 반전을 제거할 수 있어 매우 효율적이다. 최종 단계에서는 거의 정렬된 상태가 되어 있으므로, 마지막 삽입 정렬은 매우 빠르게 수행된다.

리스트의 길이가 n일 때, 점프 간격을 처음 n/2부터 시작하여 계속해서 2로 나누어 가면서 1이 될 때까지 수행하는, 즉 점프 간격 순서가 (n/2, n/4, ... , 1)인 쉘 정렬 알고리즘을 구현하고, 기존의 삽입 정렬, 합병 정렬, 퀵 정렬과 수행 시간을 비교하는 실험을 하시오.

14. 1 이상 1000 미만의 정수들이 단일 연결리스트의 각 노드에 저장되어 있다. 연결리스트에 저장된 숫자들을 오름차순으로 정렬하는 기수 정렬 알고리즘을 구현하시오. 단, 숫자들이 연결리스트에 저장된 상태로 정렬해야 하며, 새로운 노드를 생성하지 않아야 한다.

15. 오름차순으로 정렬된 정수 리스트 L과 정수 x가 주어질 때, L에 저장된 x의 개수를 구하는 효율적인 알고리즘을 설계하고, 그 시간복잡도를 분석하시오.

16. n개 정수로 구성된 리스트 L이 있다. L의 요소들을 순서대로 5개씩 묶어 총 $\lfloor n/5 \rfloor$ 개의 그룹을 구성한다. 각 그룹의 중앙값(median)들만 모아서 만든 리스트에서 구한 중앙값을 mm이라고 할 때, L의 요소 중에서 mm보다 작은 요소의 개수와 mm보다 큰 요소의 개수가 각각 최소 0.3n 이상 존재한다는 것을 증명하시오.

17. [실습 문제] 1부터 N까지 정수가 무작위로 저장된 리스트가 있다. 이 리스트를 오름차순, 즉 [1, 2, ... , N]으로 정렬하는 데 필요한 '교환' 연산의 최소 횟수를 구하는 프로그램을 작성하시오. **교환(swap)**은 리스트 내 두 요소의 위치를 서로 바꾸는 연산을 말한다.

    예를 들어, 리스트 L = [2, 3, 1]은 다음과 같이 처음에 L[0]과 L[2]를 교환하고, 그 다음 L[1]과 L[2]를 교환하는 2번의 교환으로 정렬된다.

    [2, 3, 1] ⇒ [1, 3, 2] ⇒ [1, 2, 3]

    입력으로, 첫 줄에 정수 N이 주어진다. N은 1 이상 100,000 이하이다. 둘째 줄에는 1부터 N까지의 정수로 이루어진 N개의 숫자가 들어온다. 출력으로 숫자들을 오름차순으로 정렬하는 데 필요한 교환 연산의 최소 횟수를 출력한다.

입력의 예	출력의 예
3 2 3 1	2

입력의 예	출력의 예
7 2 4 5 1 7 6 3	4

제9장

트리와 이진 트리

9.1 용어와 정의
9.2 이진 트리의 표현
9.3 이진 트리의 연산
9.4 일반적인 트리의 표현
9.5 사례 연구: 최적 정렬 시간
9.6 사례 연구: 수식 트리
연습문제

학습 목표

- 트리의 정의와 재귀적 성질을 이해하고, 루트, 노드, 에지, 부모, 자식, 높이, 레벨 등 기본 용어를 설명할 수 있다.
- 이진 트리의 배열 기반 표현과 연결 구조 기반 표현 방식을 이해하고, 각 표현에 적합한 이진 트리의 유형을 설명할 수 있다.
- 전위 순회, 중위 순회, 후위 순회, 레벨 순서 순회를 이해하고, 각 순회 알고리즘을 구현할 수 있으며, 이진 트리에 관련된 다양한 문제를 해결하는 데 활용할 수 있다.
- 일반적인 트리를 표현하는 여러 가지 방법을 비교 설명할 수 있다.
- 결정 트리를 이용하여 비교 기반 정렬 알고리즘의 하한을 증명하는 방법을 이해한다.

9.1 용어와 정의

트리(tree)는 자료를 계층적인 구조(hierarchical structure)로 표현하고자 할 때 사용하는 자료구조입니다. 가계도(족보), 계통도, 조직도, 컴퓨터 운영 체제의 파일 시스템 등은 본질적으로 계층적 성질을 가지고 있는 자료의 대표적인 사례입니다. 이러한 자료들을 선형 구조로는 효과적으로 표현하기 어렵습니다. 그림 9.1은 파일 시스템의 디렉터리(폴더) 구조의 일부를 트리 구조로 표현한 예이고, 그림 9.2는 파이썬 자료형의 계층 구조를 표현한 예입니다.

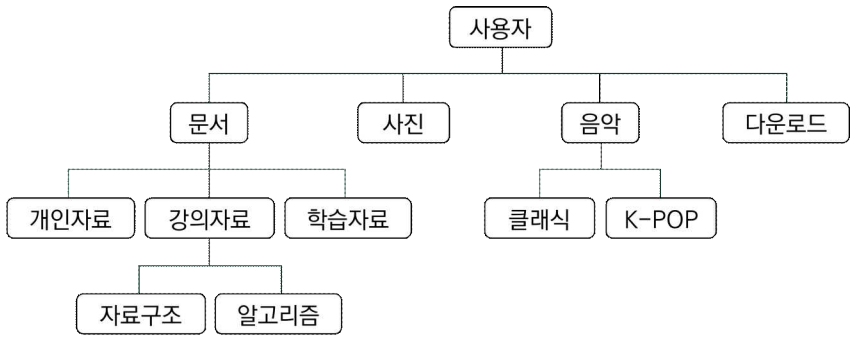


그림 9.1: 파일 시스템의 디렉터리 구조

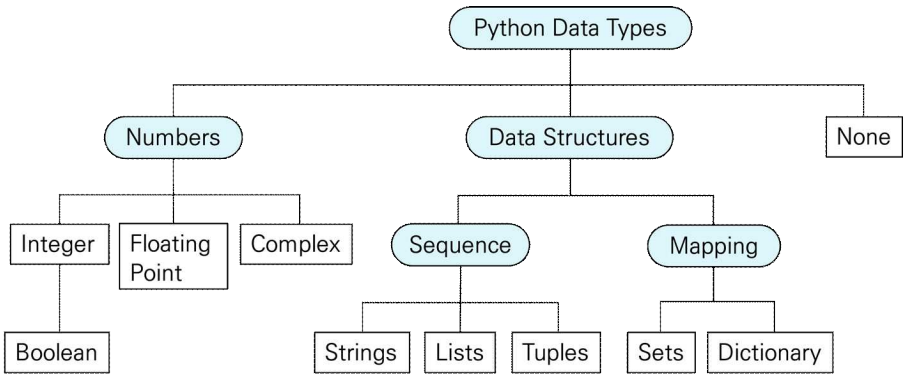


그림 9.2: 파이썬 자료형의 계층 구조

계층적인 구조를 도표로 표현하는 방법으로 그림 9.1과 9.2와 같은 트리 표현 외에도 그림 9.3처럼 블록 표현과 포함-관계 표현 등의 방법이 사용되기도 합니다.

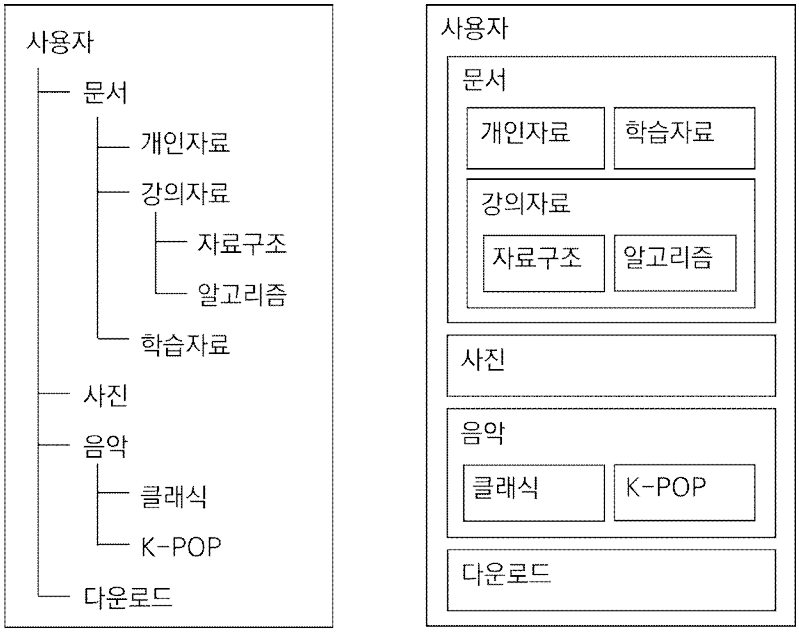


그림 9.3: 계층적인 구조의 표현 예

트리는 모양이 나무와 비슷하다고 하여 붙여진 이름입니다. 트리 구조를 표현할 때, 일반적으로 나무의 위아래를 거꾸로 배치한 모습처럼 나타냅니다. 따라서 나무의 가장 아래쪽에 뿌리가 있듯이 트리의 맨 위에도 **루트(root)**라고 불리는 객체가 있습니다. 트리에서 자료 원소에 대응하는 객체를 **노드(node)**라고 부릅니다. 그림 9.4에는 간략한 형태로 표현된 트리의 예가 나와 있습니다. 이 트리에서 원으로 표시된 부분이 노드이고, 루트는 맨 위에 있는 노드 A입니다.

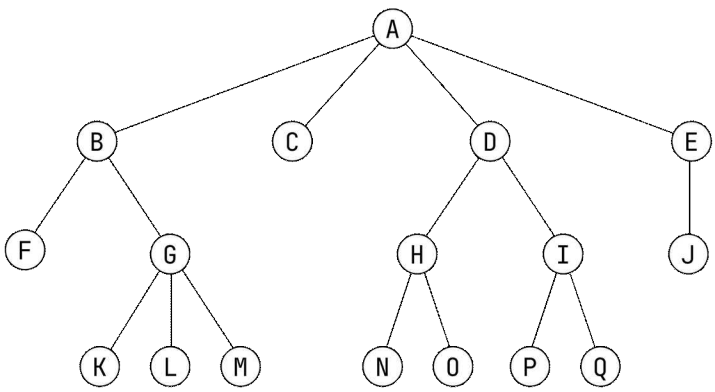


그림 9.4: 트리의 표현

트리를 형식적으로 정의하는 방법에는 여러 가지가 있습니다[17]. 그중에서 트리의 재귀적 특성을 이용한 정의는 다음과 같습니다.

1. 한 개 이상의 노드를 가진 트리에는 단 하나의 루트 노드가 존재하고,
2. 루트 노드를 제외한 나머지 노드들은 0개 이상의 서로 분리된 트리 T_0, T_1,… , T_{m-1} ($m \geq 0$)로 분할된다.

위의 정의에서 각 트리 T_i를 루트 노드의 **부트리(subtree)**라고 부릅니다. 그림 9.4의 트리에서 루트 노드 A를 제거하면, 노드 B, C, D, E를 각각 루트로 하는 4개의 부트리로 분리됩니다. 이 재귀적 성질은 트리를 다루는 알고리즘을 재귀적으로 설계할 수 있도록 해 줍니다.

이제 트리에서 사용되는 주요 용어에 대해 알아보겠습니다. 용어의 대부분은 가계도와 나무에서 유래된 것입니다. 두 노드 간의 직접적인 관계를 그림 9.4처럼 선으로 표시하는 데, 이 선을 **에지(edge)** 또는 **간선**이라고 부릅니다. 에지는 노드 사이에서 부모와 자식 관계를 나타냅니다. 에지로 직접 연결된 두 노드 중 위쪽의 노드를 **부모(parent)**, 아래쪽 노드를 **자식(child)**이라고 부릅니다. 즉, 노드 x의 부트리의 루트가 노드 y이면, x는 y의 부모이고 y는 x의 자식입니다. 가계도에서처럼 노드 사이의 관계를 형제, 조손, 사촌 등으로 부를 수 있습니다. 그림 9.4에서 노드 B, C, D, E는 부모가 같으므로 서로 형제 관계이고, A와 F는 조부모와 손주 관계, G와 H는 사촌 관계입니다.

트리에서 중복 없이 연속된 에지들로 연결된 선을 **경로(path)**라고 부릅니다. 어떤 노드의 **조상(ancestor)**은 그 노드와 루트 노드를 연결하는 경로에 놓인 노드들을 말하고, 어떤 노드의 **후손(descendant)**은 그 노드를 루트로 하는 부트리에 있는 노드들을 말합니다. 그림 9.4에서 노드 M의 조상은 M, G, B, A이고, 노드 B의 후손은 B, F, G, K, L, M입니다. 어떤 책에서는 노드의 조상과 후손을 정의할 때 노드 자신은 제외하기도 하지만, 이 책에서는 노드 자신도 포함하는 것으로 정의합니다.

노드의 차수(degree)는 그 노드의 자식 수이며, **트리의 차수**는 그 트리에 속한 노드의 차수 중에서 가장 큰 값입니다. 그림 9.4에서 노드 G의 차수는 3, 노드 J의 차수는 0이며, 트리의 차수는 4입니다. 차수가 0인 노드를 **잎 노드(leaf node)** 또는 **단말 노드(terminal node)**라고 부르고, 잎 노드가 아닌 노드를 **내부 노드(internal node)** 또는 **비단말 노드(nonterminal node)**라고 부릅니다. 그림 9.4의 트리에서 잎 노드를 모두 나열하면 C, F, J, K, L, M, N, O, P, Q입니다.

---

17) 보통 '트리'라고 부르는 자료구조는 루트를 가진 트리(rooted tree)를 의미합니다. 그래프의 한 종류로서 루트가 명시되지 않은 트리는 자유 트리(free tree)라고 합니다.

노드의 **레벨(level)** 또는 **깊이(depth)**는 해당 노드가 루트로부터 얼마나 멀리 떨어져 있는지를 나타내는 값으로, 루트에서 그 노드까지의 경로에 있는 에지의 수로 정의합니다. 루트의 레벨은는 0이며, 어떤 노드의 레벨은 부모 노드의 레벨에 1을 더한 값입니다[18]. 그림 9.4에서 노드 B의 레벨은 1, 노드 G의 레벨은 2, 노드 K의 레벨은 3입니다.

트리의 **높이(height)**는 그 트리에 속한 노드의 깊이 중에서 가장 큰 값으로 정의합니다. 따라서 그림 9.4의 트리는 높이가 3입니다. 루트 노드 하나만 있는 트리는 높이가 0이며, 노드가 하나도 없는 경우는 보통 -1로 정의합니다.

트리의 차수가 k 이하이면 그 트리를 **k진 트리(k-ary tree)** 또는 **k원 트리(k-way tree)**라고 부릅니다. 특히 k가 2인 이진 트리는 자료구조로서 가장 빈번하게 활용되기 때문에 매우 중요합니다. 이런 이유로, 앞으로 다룰 내용의 많은 부분이 이진 트리에 관한 것으로 구성됩니다.

이진 트리(binary tree)는 최대 두 개의 자식 노드를 가집니다. 두 자식 노드는 **왼쪽 자식**과 **오른쪽 자식**으로 구분합니다. 자식이 하나만 있는 경우에도 왼쪽 자식인지 오른쪽 자식인지를 구분합니다. 어떤 노드의 왼쪽 자식을 루트로 하는 부트리를 그 노드의 **왼쪽 부트리**, 오른쪽 자식을 루트로 하는 부트리를 **오른쪽 부트리**라고 부릅니다.

이진 트리를 효과적으로 다루기 위해서는 이진 트리의 특성을 이해하는 것이 필요합니다. 다음은 트리 및 이진 트리의 주요 특성 몇 가지를 나열한 것입니다.

① 이진 트리에서 깊이가 i인 노드의 최대 개수는 2^i이다($i \geq 0$).
② 높이가 h인 이진 트리가 가질 수 있는 노드의 최대 개수는 $2^{h+1}-1$이다($h \geq 0$).
③ 트리에서 노드 수는 항상 에지 수보다 하나가 많다.
④ 이진 트리에서 잎 노드 수는 항상 차수가 2인 노드의 개수보다 하나가 많다.

이진 트리에서 부모 노드는 최대 2개의 자식 노드를 가지므로, 레벨이 증가할 때마다 노드의 최대 개수는 2배씩 늘어납니다. 각 레벨의 최대 노드 수를 합하면 이진 트리가 가질 수 있는 노드의 최대 개수가 되므로 위의 ①, ②가 성립합니다.

이제 ③이 성립함을 살펴볼까요. 트리에서 루트를 제외한 모든 노드는 부모 노드가 하나씩 있습니다. 각 에지는 자식 노드와 그 부모 노드를 잇는 역할을 하므로, 에지 수는 루트를 제외한 노

18) 책에 따라서는 레벨을 1부터 시작하기도 합니다. 즉 루트의 레벨은 1이며, 어떤 노드의 레벨은 부모 노드의 레벨에 1을 더한 값으로 정의합니다. 그러나 이 책에서는 루트의 레벨을 0으로 정의하며, '레벨'과 '깊이'를 같은 의미로 사용합니다.

드 수와 같습니다. 따라서 ③이 항상 성립합니다.

마지막으로 ④가 성립한다는 것을 증명하겠습니다. 이진 트리에서 노드 수를 n이라고 하고, 에지 수를 m이라고 합시다. 그러면 ③에 의해 다음의 식이 성립합니다.

$$n = m + 1$$

잎 노드 수를 n_0, 자식이 하나만 있는 노드의 수를 n_1, 자식이 둘 다 있는 노드의 수를 n_2라고 하면, 노드 수 n은 다음과 같습니다.

$$n = n_0 + n_1 + n_2$$

그리고 에지 수 m을 각 노드와 그 자식 노드를 이어주는 에지의 수로 계산해 보면, 잎 노드는 자식이 없고, 자식이 하나만 있는 노드는 1개의 에지, 자식이 둘 다 있는 노드는 2개의 에지를 가지므로, 다음의 식이 성립합니다.

$$m = n_1 + 2n_2$$

이제, 위 식에 "n = m + 1"을 대입하면 다음과 같습니다.

$$n_0 + n_1 + n_2 = (n_1 + 2n_2) + 1$$

양변에서 n_1을 소거하고 n_2를 정리하면 다음의 식을 얻을 수 있습니다.

$$n_0 = n_2 + 1$$

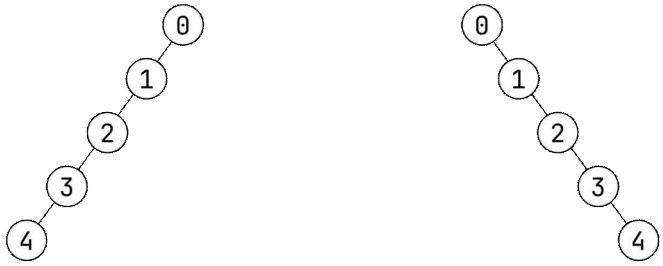
따라서, 위의 ④가 성립합니다.

트리나 이진 트리를 다루는 알고리즘의 효율성은 대부분 트리의 높이에 의해 결정됩니다. 따라서 트리의 노드 수와 높이와의 관계는 알고리즘의 설계와 분석에 있어서 중요한 역할을 합니다. 위에서 나열한 트리의 특성은 이러한 분석에 이용됩니다.

다음은 이진 트리의 특수한 형태로서 자주 언급되는 이진 트리의 종류인 경사 이진 트리, 정 이진 트리, 포화 이진 트리, 완전 이진 트리를 알아봅니다.

경사 이진 트리(skewed binary tree)는 왼쪽 경사 이진 트리와 오른쪽 경사 이진 트리로 구분됩니다. 모든 내부 노드가 왼쪽 자식만 가지고 있으면 왼쪽 경사 이진 트리, 오른쪽 자식만 가지고 있으면 오른쪽 경사 이진 트리라고 합니다(그림 9.5 참조). 경사 이진 트리는 각 레벨에 하

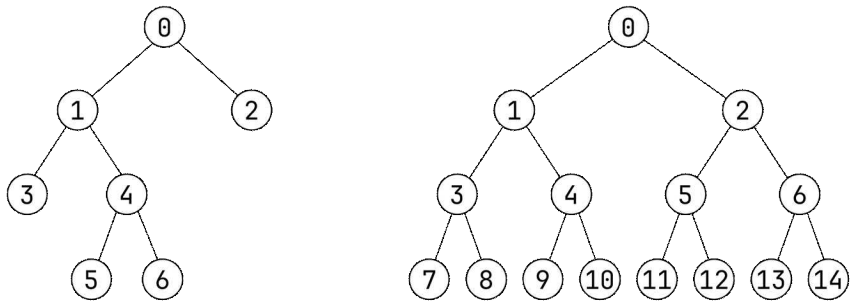
나의 노드만 존재하므로 트리의 높이는 노드 수에 1을 뺀 값과 같습니다. 따라서, 경사 이진 트리는 노드 수가 같은 이진 트리 중에서 높이가 가장 크기 때문에, 이진 트리에서 최악으로 구성된 구조의 예로서 주로 언급됩니다.



(a) 왼쪽 경사 이진 트리 (b) 오른쪽 경사 이진 트리

그림 9.5: 경사 이진 트리

전 이진 트리(full binary tree)는 잎 노드가 아닌 모든 노드가 2개의 자식을 가지는 이진 트리이고, **포화 이진 트리(perfect binary tree)**는 레벨마다 노드가 최대로 들어 있는 구조, 즉 레벨 i에 2^i 개의 노드가 있는 이진 트리입니다(그림 9.6 참조).



(a) 전 이진 트리 (b) 포화 이진 트리

그림 9.6: 전 이진 트리와 포화 이진 트리

전 이진 트리와 포화 이진 트리는 잎 노드를 제외한 모든 노드가 두 개의 자식을 가지고 있으므로, 잎 노드의 개수는 내부 노드의 개수보다 항상 하나가 많습니다. 높이가 h인 포화 이진 트리의 노드 수는 2^{h+1}-1이므로, 노드 수가 n인 포화 이진 트리의 높이는 log(n+1)-1 또는 $\lfloor \log n \rfloor$로 나타낼 수 있습니다.

완전 이진 트리(complete binary tree)는 높이가 h인 경우 레벨 0부터 h-1까지는 포화 이진 트리처럼 모든 레벨에 노드들이 꽉 차 있고, 마지막 레벨 h에서는 노드들이 왼쪽에서부터 연속적으로 채워져 있는 구조입니다(그림 9.7 참조).

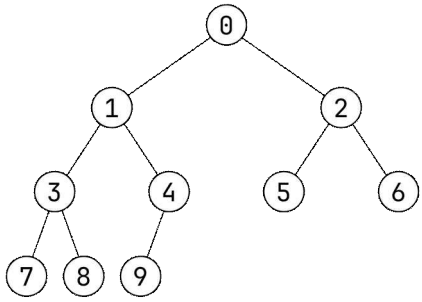


그림 9.7: 10개의 노드를 가진 완전 이진 트리

완전 이진 트리에는 자식이 하나만 있는 노드가 기껏해야 한 개뿐이며, 노드 수가 같은 이진 트리 중에서 높이가 가장 작습니다. 높이가 h인 완전 이진 트리의 노드 수를 n이라고 하면 다음의 부등식이 성립합니다.

$$2^h \leq n \leq 2^{h+1}-1$$

이 식에 의하면 n개의 노드를 가진 완전 이진 트리의 높이 h는 다음과 같습니다.

$$h = \lfloor \log n \rfloor$$

9.2 이진 트리의 표현

다른 자료구조와 마찬가지로 이진 트리를 표현하는 방법에도 배열 구조를 이용한 방법과 연결 구조를 이용한 방법이 있습니다. 이진 트리는 노드 집합과 에지 집합으로 정의됩니다. 에지는 노드들 사이에서 부모와 자식 관계를 나타내므로, 두 노드의 순서쌍으로 표현할 수 있습니다. 이진 트리를 표현하는 가장 단순한 방법은 노드들의 선형리스트와 에지들의 선형리스트를 사용하는 것입니다. 어떤 자료의 표현은 단지 그 자료의 정보를 보유하고 있는 것만이 전부가 아니라 그 자료를 효율적으로 처리할 수 있도록 구성되어야 합니다. 이진 트리를 두 개의 선형리스

트로 단순하게 표현하면 이진 트리에 필요한 연산을 효율적으로 수행할 수 없습니다. 예를 들어, 어떤 노드의 왼쪽 자식 노드를 검색한다든지 또는 차수를 조사하는 경우, 선형리스트 표현에서는 항상 모든 에지를 검사해야 하므로 비효율적입니다.

이진 트리는 본질적으로 비선형 구조이기 때문에, 이진 트리를 효과적으로 다루기 위해서는 비선형 구조 방식으로 표현하는 것이 필요합니다. 다만, 특수한 환경에서는 1차원 배열 구조로도 트리를 효과적으로 표현할 수 있습니다. 이 절에서는 이진 트리를 1차원 배열 구조와 비선형 연결 구조로 표현하는 방법을 차례로 살펴봅니다.

먼저, 이진 트리를 1차원 배열 구조를 사용하여 표현하는 방법을 살펴봅니다. 1차원 배열은 본질적으로 선형 구조이기 때문에, 일반적인 이진 트리를 1차원 배열로 표현하는 것은 적합하지 않지만, 완전 이진 트리와 같은 특수한 구조에서는 오히려 효과적입니다. 이진 트리를 배열로 표현하기 위해서는 트리의 노드와 배열 원소 사이에 적절한 대응 관계를 정해야 합니다.

노드 수가 n인 완전 이진 트리를 배열로 표현한다고 해 봅시다. 트리의 각 노드에 0부터 n-1까지 번호를 하나씩 부여하여, 각 노드에 부여된 번호를 첨자(index)로 하는 배열 원소에 해당 노드를 대응시킵니다. 번호는 다음과 같이 부여합니다. 먼저 루트 노드에 번호 0을 부여합니다. 그다음, 레벨이 증가하는 순서로, 같은 레벨에서는 왼쪽부터 오른쪽으로 가면서 만나는 노드에 차례대로 번호를 부여합니다. 번호 부여가 끝나면, 번호 i를 가진 노드를 배열의 첨자 i인 원소와 대응시킵니다. 그림 9.8은 완전 이진 트리에 대한 배열 표현의 예를 보여줍니다. 완전 이진 트리는 레벨이 증가할 때마다 노드 수는 2배씩 늘어납니다. 그래서 번호 i를 가진 노드의 왼쪽 자식의 번호는 2i+1, 오른쪽 자식의 번호는 2i+2가 됩니다.

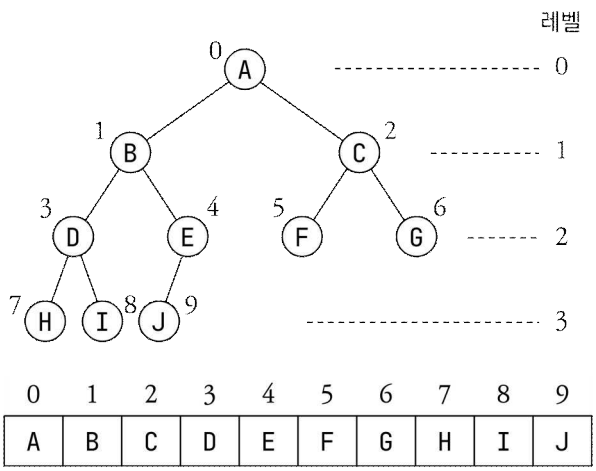


그림 9.8: 완전 이진 트리의 배열 표현

트리 자료구조에 대한 기초적인 연산은 어떤 노드의 부모, 왼쪽 자식, 오른쪽 자식을 검색하는 것입니다. 이 연산을 통해 트리에 대한 모든 정보를 획득할 수 있습니다. 완전 이진 트리의 배열 표현에서 이러한 연산은 배열 첨자를 이용한 간단한 수식으로 수행됩니다. 노드 수가 n인 완전 이진 트리를 1차원 배열 tree에 표현한다고 하고, tree[i]를 i번째 위치에 저장된 트리의 원소라고 합시다. 배열 첨자를 이용하여 트리의 기초적인 정보를 검색하는 방법은 다음과 같습니다.

- tree[0]은 루트이다.
- 2×i+1 ≥ n이면 tree[i]는 잎 노드이다.
- tree[i]의 부모는 tree[⌊(i-1)/2⌋]에 있다.
- tree[i]의 왼쪽 자식은 tree[2×i+1]에 있다.
- tree[i]의 오른쪽 자식은 tree[2×i+2]에 있다.

배열 표현법은 트리의 기초적인 연산을 상수 시간에 수행할 수 있어 매우 효율적입니다. 특히 완전 이진 트리의 경우는 노드 수를 길이로 하는 배열을 사용하므로 메모리 공간을 가장 효율적으로 활용하는 자료구조입니다.

이러한 배열 표현법을 일반적인 이진 트리에 적용하면 어떨까요? 이 경우에도 완전 이진 트리와 같은 방식으로 각 노드에 번호를 부여합니다. 한 가지 차이점은 존재하지 않은 노드에 대해서도 번호를 부여해야 하는 것입니다. 그래야 첨자 계산으로 부모와 자식을 검색할 수 있기 때문입니다. 존재하지 않은 노드의 번호에 대응하는 배열 원소에는 "존재하지 않음"을 나타내는 적절한 값(예, None)을 배정합니다. 이 방법은 이진 트리의 기초적인 연산 외에도 새로운 잎 노드의 삽입이나 기존 잎 노드의 삭제를 효율적으로 처리할 수 있다는 장점을 가집니다. 그러나 존재하지 않은 노드에 대해 배열 원소를 대응시킴으로써 메모리의 낭비 문제가 발생합니다. 최악의 경우, 노드 수가 n인 오른쪽 경사 이진 트리를 표현하기 위해서는 2^n-1개의 배열 원소가 필요합니다(그림 9.9 참조). 이런 극단적인 비효율성 때문에 일반적인 이진 트리를 배열로 표현하는 것은 현실적으로 받아들이기 어렵습니다. 따라서 완전 이진 트리로 항상 표현될 수 있는 자료에 대해서만 배열 표현법을 이용하고, 일반적인 이진 트리를 위해서는 이후에 소개하는 연결 구조 표현법을 사용하는 것이 좋습니다.

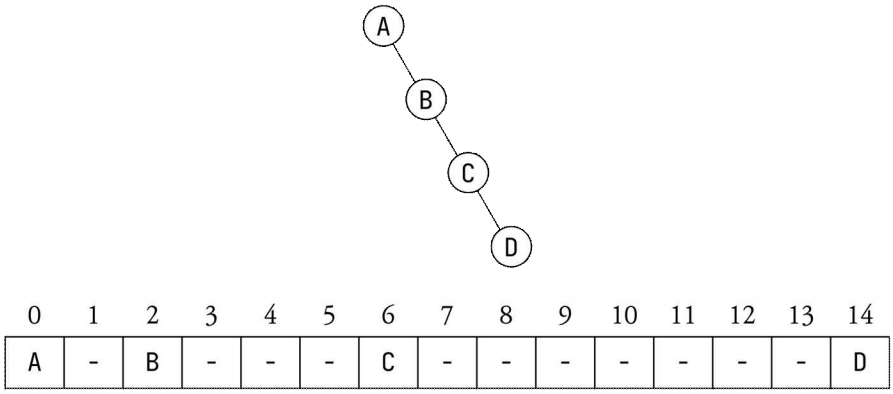


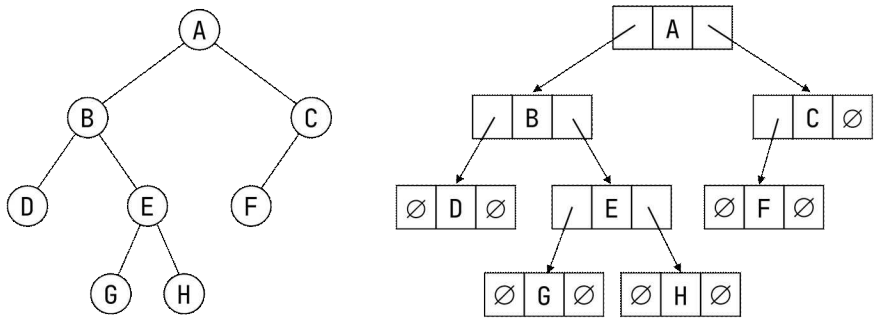
그림 9.9: 경사 이진 트리의 배열 표현

이제, 이진 트리를 비선형 연결 구조로 표현하는 방법을 살펴보겠습니다. 이진 트리의 연결 구조 표현은 이진 트리의 각 노드와 연결 구조의 노드를 일대일로 대응시켜 이진 트리의 개념적인 구조와 동일한 연결 구조를 구성하는 것입니다. 연결 구조의 노드를 위한 클래스는 보통 프로그램 9.1처럼 세 개의 필드, 자료 원소의 저장을 위한 데이터 필드와 왼쪽 자식을 가리키는 링크 필드, 오른쪽 자식을 가리키는 링크 필드로 구성됩니다.

프로그램 9.1: 이진 트리의 노드 클래스

```
1 class BinNode:
2    def __init__(self, item=None, left_child=None, right_child=None):
3       self.data = item              # 자료 원소
4       self.left = left_child        # 왼쪽 자식을 가리키는 링크
5       self.right = right_child      # 오른쪽 자식을 가리키는 링크
```

그림 9.10은 이진 트리의 연결 구조 표현을 보여줍니다. 프로그램 9.1의 클래스에는 부모 노드를 가리키는 링크가 없어 부모 노드의 위치를 쉽게 알아내기가 어렵지만, 대부분의 응용에서는 위의 구성으로도 충분합니다. 부모 노드를 빈번하게 참조하는 응용에서는 자식이 부모를 가리키는 링크를 추가하면 됩니다. 링크 필드에 표시된 ∅는 해당 링크 필드에 널 링크가 있음을 나타냅니다.



(a) 이진 트리의 예 (b) 연결 구조 표현

그림 9.10: 이진 트리의 연결 구조 표현

9.3 이진 트리의 연산

이 절에서는 이진 트리에서 수행되는 기본적인 연산인 순회 알고리즘과 순회 알고리즘을 이용한 응용에 대해 살펴봅니다.

■ 이진 트리 순회

이진 트리에 대한 작업을 수행할 때 각 노드에 배정된 자료 원소들을 한 번씩 처리해야 하는 경우가 종종 발생합니다. 이 작업을 위해서는 이진 트리의 각 노드를 한 번씩 방문해야 합니다. 특정한 순서로 노드들을 차례대로 방문하는 탐색 작업을 **순회(traversal)**라고 합니다. 노드를 방문한다는 것은 해당 노드의 자료 원소에 대해 어떤 작업을 수행하는 것을 의미합니다. 순회 도중에 노드의 방문 없이 그냥 경유하는 경우는 여러 번 나올 수 있지만, 노드의 방문은 한 번만 발생합니다.

이진 트리를 순회하는 방법에는 스택을 이용한 방법과 큐를 이용한 방법이 있습니다. 스택을 이용한 방법은 이진 트리가 가지고 있는 재귀적인 특성을 활용하는 것입니다. 순회는 다음의 세 가지 작업으로 구성됩니다.

- V - 노드를 방문한다
- L - 왼쪽 부트리를 순회한다
- R - 오른쪽 부트리를 순회한다

이 세 가지 작업의 수행 순서에 따라 순회 방식을 최대 6가지로 구분할 수 있습니다. 일반적으로 왼쪽 부트리를 오른쪽 부트리보다 항상 먼저 순회하는데, 이 경우 순회 방법은 다음 세 가지로 구분됩니다.

- 전위 순회 : vLR
- 중위 순회 : LvR
- 후위 순회 : LRv

전위 순회(preorder traversal)는 루트를 먼저 방문하고, 그다음 루트의 왼쪽 부트리와 오른쪽 부트리를 차례대로 순회합니다. **중위 순회(inorder traversal)**는 루트의 왼쪽 부트리를 먼저 순회하고 그다음 루트를 방문한 뒤 오른쪽 부트리를 순회합니다. **후위 순회(postorder traversal)**는 루트의 왼쪽 부트리와 오른쪽 부트리를 차례대로 순회한 후에 루트를 마지막으로 방문합니다.

스택을 이용한 순회는 재귀 함수를 이용하면 매우 간단하게 구현됩니다. 프로그램 9.2에는 연결 구조로 표현된 이진 트리에 대해서 전위 순회, 중위 순회, 후위 순회를 구현한 재귀 함수가 나와 있습니다.

프로그램 9.2: 전위 순회, 중위 순회, 후위 순회 프로그램

```
1  def visit(data):
2      """노드 방문 시 수행할 작업을 정의한다."""
3      print(data, end='')
4
5  def preorder(node):
6      """전위 순회 (vLR)"""
7      if node is None:            # 널 노드이면 퇴각함
8          return
9      visit(node.data)            # 노드 방문(자료 원소에 대한 작업)
10     preorder(node.left)         # 왼쪽 부트리를 순회
11     preorder(node.right)        # 오른쪽 부트리를 순회
12
13 def inorder(node):
14     """중위 순회 (LvR)"""
15     if node is None:            # 널 노드이면 퇴각함
16         return
17     inorder(node.left)          # 왼쪽 부트리를 순회
18     visit(node.data)            # 노드 방문(자료 원소에 대한 작업)
19     inorder(node.right)         # 오른쪽 부트리를 순회
20
```

```
21 def postorder(node):
22     """후위 순회 (LRv)"""
23     if node is None:            # 널 노드이면 퇴각함
24         return
25     postorder(node.left)        # 왼쪽 부트리를 순회
26     postorder(node.right)       # 오른쪽 부트리를 순회
27     visit(node.data)            # 노드 방문(자료 원소에 대한 작업)
```

프로그램 9.2의 세 함수 preorder(), inorder(), postorder()는 모두 트리의 각 노드에 대해 한 번씩 호출되기 때문에 전체 수행 시간은 $O(n)$입니다. 여기서 n은 이진 트리의 노드 수입니다. 이 재귀 함수의 공간복잡도를 알아볼까요? 이진 트리가 완전 이진 트리이면 최상의 경우로서 시스템 스택에 최대 $\lceil \log n \rceil$개의 함수 정보, 즉 활성화 레코드가 쌓이게 되고, 최악의 경우인 경사 이진 트리의 형태는 시스템 스택에 모두 n개의 활성화 레코드가 적재됩니다. 각 활성화 레코드의 크기는 $O(1)$이므로 전체 공간복잡도는 $O(n)$입니다.

전위 순회, 중위 순회, 후위 순회의 세 재귀 함수는 노드 방문을 제외하면 모두 왼쪽 부트리를 먼저 순회하고 그다음에 오른쪽 부트리를 순회하기 때문에 함수의 호출 및 반환 순서는 동일합니다. 그림 9.11에서 점선으로 표시된 선은 스택을 이용한 순회의 탐색 경로를 나타냅니다.

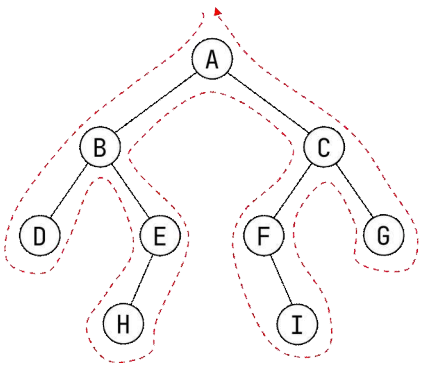


그림 9.11: 이진 트리 순회의 탐색 경로

표 9.1은 그림 9.11의 이진 트리에 대한 재귀 함수의 수행에서 이진 트리의 각 노드에 대한 호출 및 반환 순서를 보여줍니다(널 노드에 대한 호출 및 반환은 생략됨).

표 9.1: 그림 9.11의 이진 트리에 대한 순환 함수의 호출 및 반환 과정

단계	스택					비 고
	0	1	2	3	4	
1	A					A의 호출, 순회 시작
2	A	B				A의 왼쪽 자식 B의 호출
3	A	B	D			B의 왼쪽 자식 D의 호출
4	A	B				B의 왼쪽 자식 D로부터 반환
5	A	B	E			B의 오른쪽 자식 E의 호출
6	A	B	E	H		E의 왼쪽 자식 H의 호출
7	A	B	E			E의 왼쪽 자식 H로부터 반환
8	A	B				B의 오른쪽 자식 E로부터 반환
9	A					A의 왼쪽 자식 B로부터 반환
10	A	C				A의 오른쪽 자식 C의 호출
11	A	C	F			C의 왼쪽 자식 F의 호출
12	A	C	F	I		F의 오른쪽 자식 I의 호출
13	A	C	F			F의 오른쪽 자식 I로부터 반환
14	A	C				C의 왼쪽 자식 F로부터 반환
15	A	C	G			C의 오른쪽 자식 G의 호출
16	A	C				C의 오른쪽 자식 G로부터 반환
17	A					A의 오른쪽 자식 C로부터 반환
18						A로부터 반환, 순회 종료

재귀 함수의 수행 과정에서 전위 순회는 각 노드가 처음 호출될 때 방문하고, 중위 순회는 왼쪽 자식 노드로부터 반환되어 자신에게 돌아오는 시점에 방문하며, 후위 순회는 오른쪽 자식 노드로부터 반환되어 자신에게 돌아오는 시점에 방문합니다. 전위 순회, 중위 순회, 후위 순회에 따라 방문하는 노드 순서를 각각 **전위 순서(preorder sequence)**, **중위 순서(inorder sequence)**, **후위 순서(postorder sequence)**라고 부릅니다. 그림 9.11의 이진 트리에 대한 전위 순서, 후위 순서, 중위 순서는 다음과 같습니다.

- 전위 순서 : ABDEHCFIG
- 중위 순서 : DBHEAFICG
- 후위 순서 : DHEBIFGCA

큐를 이용한 순회 방법에는 **레벨 순위 순회(level order traversal)**가 있습니다. 레벨 순위 순회

는 이진 트리의 배열 표현에서 각 노드에 부여한 번호 순서대로 노드를 방문합니다. 즉 루트를 먼저 방문하고, 그다음 레벨이 증가하는 순서로 노드들을 방문하며 같은 레벨에서는 왼쪽에서 오른쪽 순서로 노드들을 방문합니다. 이를 위해, 초기 공백 큐에 루트를 삽입합니다. 그다음, 큐가 다시 공백이 될 때까지 큐의 front 노드를 꺼내와서 이 노드의 자식 노드를 차례로 큐에 삽입합니다. 노드 방문은 큐에 삽입할 때 또는 삭제할 때 수행합니다. 그림 9.11의 이진 트리에 대한 레벨 순위 순회의 순서는 "ABCDEFGHI"입니다. 프로그램 9.3에는 레벨 순위 순회를 구현한 함수가 나와 있습니다. 프로그램에서 큐 자료구조로 `collections.deque`를 사용합니다.

프로그램 9.3: 레벨 순위 순회

```
1  from collections import deque
2
3  def levelorder(root):
4     """큐를 이용하여 레벨 순서로 트리를 순회한다."""
5     if root is None:                  # 루트가 널 노드이면 작업 종료
6        return
7     queue = deque()                   # 공백 큐 생성
8     queue.append(root)                # 루트를 큐에 삽입
9     while len(queue) > 0:             # 큐가 공백될 때까지 반복
10       node = queue.popleft()         # 큐에서 front 노드를 꺼냄
11       visit(node.data)               # 노드 방문 (자료 원소에 대한 작업)
12       if node.left is not None:      # 왼쪽 자식이 존재하면 큐에 삽입
13          queue.append(node.left)
14       if node.right is not None:     # 오른쪽 자식이 존재하면 큐에 삽입
15          queue.append(node.right)
```

함수 `levelorder()`의 수행 시간은 큐에 삽입되는 노드의 개수에 비례합니다. 이진 트리의 각 노드는 한 번씩만 큐에 삽입된 후 삭제되므로 수행 시간은 $O(n)$입니다.

■ 이진 트리 크기, 높이, 복사

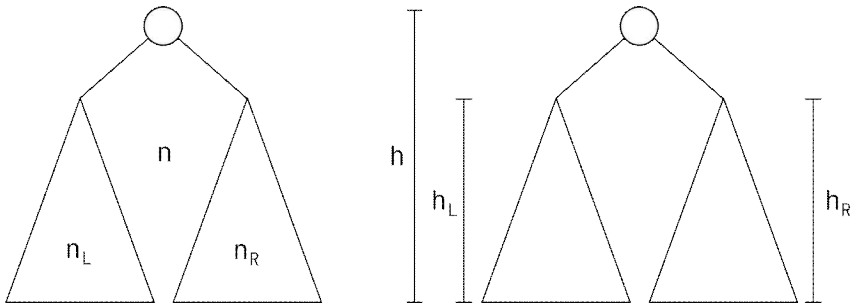
이진 트리의 재귀적인 성질을 이용한 간단한 재귀 알고리즘으로 이진 트리에 대한 기본적인 속성을 알아낼 수 있습니다. 먼저, 이진 트리의 노드 수와 높이를 구하는 문제를 살펴봅니다.

이진 트리의 노드들은 루트 노드와 왼쪽 부트리의 노드들, 그리고 오른쪽 부트리의 노드들로 구성됩니다. 따라서 이진 트리의 노드 수는 다음 재귀 관계로 표현됩니다(그림 9.12 (a) 참조)..

노드 수 = 1 + 왼쪽 부트리의 노드 수 + 오른쪽 부트리의 노드 수

이진 트리의 높이는 왼쪽 부트리의 높이와 오른쪽 부트리의 높이 중에서 큰 값에 1을 더한 값과 같습니다. 즉, 이진 트리의 높이는 다음 재귀 관계로 표현됩니다(그림 9.12 (b) 참조).

높이 = 1 + max(왼쪽 부트리의 높이, 오른쪽 부트리의 높이)



(a) 노드 수 $n = 1 + n_L + n_R$ (b) 높이 $h = 1 + \max(h_L, h_R)$

그림 9.12: 이진 트리의 크기와 높이

프로그램 9.4에는 이진 트리의 크기와 높이를 구하는 재귀 함수가 나와 있습니다. 두 함수 size()와 height()는 이진 트리의 순회 알고리즘과 수행 과정이 동일하므로 수행 시간도 역시 $O(n)$입니다.

프로그램 9.4: 이진 트리의 크기와 높이를 구하는 프로그램

```
1  def size(node):
2      """이진 트리의 노드 수를 구하는 재귀 함수"""
3      if node is None:          # 널 노드이면 0 반환
4          return 0
5      return 1 + size(node.left) + size(node.right)
6
7  def height(node):
8      """이진 트리의 높이를 구하는 재귀 함수"""
9      if node is None:          # 널 노드이면 -1 반환
10         return -1
11     return 1 + max(height(node.left), height(node.right))
```

다음으로 주어진 이진 트리와 동일한 이진 트리를 구축하는 문제를 살펴보겠습니다. 이진 트리의 재귀적 성질을 이용하여 이 문제의 해는 다음 재귀 관계로 표현됩니다.

① 루트 노드를 복사한다.
② 왼쪽 부트리를 복사한다.
③ 오른쪽 부트리를 복사한다.

프로그램 9.5에는 이진 트리를 복사하는 재귀 함수 copy_tree()가 나와 있습니다. 프로그램의 5줄에 나온 BinNode는 프로그램 9.1에서 정의한 이진 트리의 노드 클래스입니다. 이 함수도 이진 트리의 순회 알고리즘과 수행 과정이 동일하므로 $O(n)$ 시간에 수행됩니다.

프로그램 9.5: 이진 트리를 복사하는 프로그램

```
1 def copy_tree(node):
2     """이진 트리를 복사하는 재귀 함수"""
3     if node is None:                          # 널 노드이면 None 반환
4         return None
5     clon = BinNode(node.data)                 # 노드 생성 및 데이터 복사
6     clon.left = copy_tree(node.left)          # 왼쪽 부트리 복사
7     clon.right = copy_tree(node.right)        # 오른쪽 부트리 복사
8     return clon                               # 복사된 루트 노드의 링크 반환
```

■ 순회 순서를 이용한 이진 트리 구축

3개의 노드를 가진 이진 트리의 전위 순서가 다음과 같다고 합시다.

ABC

이 전위 순서를 가진 이진 트리의 중위 순서가 될 수 있는 순열은 어떤 형태일까요? 'ABC'의 모든 순열은 다음과 같이 6가지가 있습니다.

ABC ACB BAC BCA CAB CBA

이 중에서 중위 순서가 될 수 없는 것은 'CAB'입니다. 그 이유는 다음과 같습니다. 전위 순서로부터 맨 앞의 원소 A가 루트임을 알 수 있습니다. 그리고 중위 순서 'CAB'에서 A는 C와 B의 중간에 있으므로, C는 A의 왼쪽 자식이고 B는 A의 오른쪽 자식이 됩니다. 그러면 전위 순서에서 왼쪽 자식인 C는 오른쪽 자식인 B보다 먼저 나와야 합니다. 그런데 전위 순서 'ABC'는 B가 C보다 먼저 나와 있으므로 모순입니다. 나머지 순열에 해당하는 이진 트리는 그림 9.13에서 볼 수 있습니다.

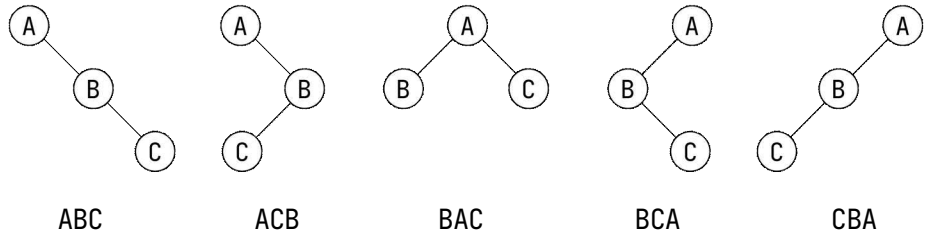


그림 9.13: 전위 순서가 ABC인 이진 트리의 종류와 중위 순서

전위 순서와 중위 순서가 주어진 경우, 이진 트리의 재귀적 성질을 이용하여 이들 순서에 해당하는 이진 트리를 구축할 수 있습니다. 전위 순서를 $(a_0, a_1, \cdots, a_{n-1})$, 중위 순서를 $(b_0, b_1, \cdots, b_{n-1})$이라고 할 때, 이진 트리를 구축하는 알고리즘은 다음과 같습니다.

1. 전위 순서의 맨 앞 원소 a_0를 가지는 루트 노드 R을 만든다.
2. 중위 순서에서 a_0와 일치하는 루트 원소 b_k를 찾는다. (즉, $b_k = a_0$)
3. 전위 부순서 $(a_1, a_2, \cdots, a_k)$와 중위 부순서 $(b_0, b_1, \cdots, b_{k-1})$을 가지고 R의 왼쪽 부트리를 재귀적으로 만든다.
4. 전위 부순서 $(a_{k+1}, a_{k+1}, \cdots, a_{n-1})$과 중위 부순서 $(b_{k+1}, b_{k+2}, \cdots, b_{n-1})$을 가지고 R의 오른쪽 부트리를 재귀적으로 만든다.

이 과정을 통해 전위 순서와 중위 순서로 이진 트리를 유일하게 정의할 수 있음을 알 수 있습니다. 프로그램 9.6에는 전위 순서와 중위 순서, 두 순서에 의해 정의되는 이진 트리를 구축하는 재귀 함수 make_binarytree(A, B)가 나와 있습니다. 이 함수는 리스트 A와 B에 각각 전위 순서와 중위 순서가 저장되어 있을 때, 이에 해당하는 이진 트리를 구축하여 그 루트 노드의 링크를 반환합니다.

프로그램 9.6: 전위 순서와 중위 순서로 정의되는 이진 트리를 구축하는 재귀 함수

```
1 def make_binarytree(A, B):
2     """전위 순서 A, 중위 순서 B로 정의되는 이진 트리를 구축하는 재귀 함수"""
3     if len(A) == 0:              # 공백 리스트이면 None을 반환
4         return None
5     root = BinNode(A[0])         # 노드 생성 및 루트 노드의 데이터 복사
6     k = B.index(A[0])            # 중위 순서에서 루트 원소의 위치 탐색
7     root.left = make_binarytree(A[1:k+1], B[:k])    # 왼쪽 부트리 생성
8     root.right = make_binarytree(A[k+1:], B[k+1:])  # 오른쪽 부트리 생성
9     return root                  # 루트 노드의 링크 반환
```

함수 make_binarytree()의 수행 시간을 살펴보겠습니다. 각 원소에 대해 한 번씩 호출이 발생하므로 이 함수의 총 호출 횟수는 n입니다. 각 호출 시 중위 순서에서 루트 원소의 위치를 찾기 위해 리스트의 index() 메소드를 호출합니다. 메소드 index()의 수행 시간은 리스트 길이에 대한 선형 시간입니다. 그다음, 왼쪽 부트리와 오른쪽 부트리를 생성하기 위해 리스트를 슬라이싱하여 재귀 호출을 두 번 수행합니다. 리스트의 슬라이싱은 리스트를 복사하기 때문에 선형 시간이 소요됩니다. 좀 더 효율적으로 수행하려면, 슬라이싱 대신에 동일한 리스트에서 첨자 범위를 이용하는 전략을 사용하면 됩니다. 다만, 슬라이싱을 통한 복사를 하지 않더라도 index() 메소드의 수행 시간이 선형 시간이므로 make_binarytree()의 수행 시간은 $O(n^2)$이 됩니다[19].

길이가 n인 전위 순서에 대해 중위 순서가 될 수 있는 순열의 개수는 노드 수가 n인 서로 다른 이진 트리의 개수와 같습니다. 길이가 n인 전위 순서에 대해 중위 순서가 될 수 있는 순열의 개수를 c_n이라고 합시다. $n = 0$ 또는 1이면 당연히 $c_n = 1$입니다. $n \geq 2$인 경우, 루트 원소의 위치가 i일 때, 왼쪽 부트리는 i개의 원소로 구성되고 오른쪽 부트리는 n-i-1개의 원소로 구성됩니다($0 \leq i < n$). 따라서 각 부트리에 대해 중위 순서가 될 수 있는 순열의 개수는 각각 c_i와 c_{n-i-1}입니다. 그러므로 c_n을 다음과 같이 재귀 관계식으로 표현할 수 있습니다.

$$c_n = 1 \qquad (n \leq 1)$$
$$c_n = c_0c_{n-1} + c_1c_{n-2} + \cdots + c_{n-1}c_0 \qquad (n \geq 2)$$

위 식을 풀면 c_n은 다음과 같습니다.[20]

$$c_n = \frac{1}{n+1} C(2n, n) \quad , C(m, k) = \frac{m!}{(m-k)!\,k!}$$

---

19) index() 메소드 대신에 중위 순서의 (원소, 첨자) 쌍을 미리 해싱 자료구조인 파이썬 딕셔너리(dict)에 저장해 두는 전략을 사용하면, 전체 수행 시간을 $O(n \log n)$으로 개선할 수 있습니다.

20) **카탈란 수(Catalan numbers)**라고 부르는 이 식은 생성 함수(generating function)를 이용하여 풀 수 있습니다. 자세한 내용은 이산 수학에 관한 책을 참조 바랍니다.

9.4 일반적인 트리의 표현

일반적인 트리는 매우 다양한 방법으로 표현될 수 있습니다. 먼저, 트리의 구조를 그대로 표현하는 다중 링크를 가진 연결 구조 표현을 소개하고, 두 번째로 이진 트리로 일반적인 트리를 표현하는 방법을 소개합니다. 그다음, 배열과 연결리스트를 혼합한 트리 표현법을 소개하고, 마지막으로 부모 포인터 배열 표현법을 소개합니다.

■ 다중 링크 표현법

다중 링크 표현법은 이진 트리의 연결 구조 표현을 그대로 확장한 것입니다. 즉, 일반적인 트리의 개념적인 구조와 동일하게 연결 구조를 구성합니다. 따라서 각 노드는 자식 노드를 가리키기 위해 트리의 차수, 즉 노드 차수 중 가장 큰 값만큼의 링크 필드를 가져야 합니다. 물론 필요한 경우 부모 노드를 가리키기 위한 링크 필드도 포함할 수 있습니다. 그림 9.14는 트리의 다중 링크 표현을 보여줍니다.

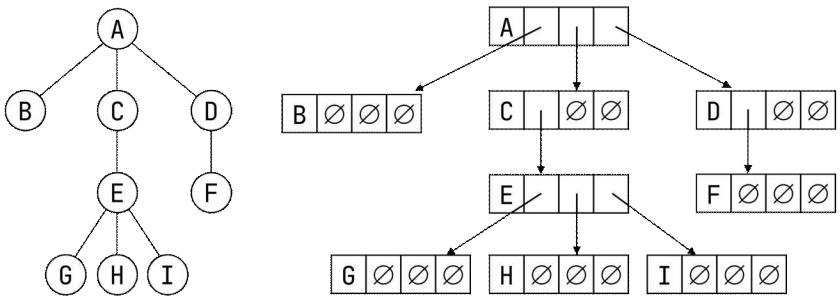


그림 9.14: 트리의 다중 링크 표현

트리의 다중 링크 표현은 링크 필드의 수가 매우 많습니다. 트리에서 노드의 수가 n이고 차수가 k라면, 부모 링크가 없는 경우 다중 링크 표현에서 링크 필드의 수는 k×n입니다. 이 k×n개의 링크 필드 중에서 실제로 자식 노드를 가리키는 링크 필드의 수는 루트 노드를 제외한 노드의 수와 같으므로 n-1입니다. 따라서 그 나머지인 k×n-(n-1)개의 링크 필드는 모두 널 링크를 가집니다. 만일 k가 2라면 전체의 1/2 이상이 널 링크이고, k가 3이면 전체의 2/3 이상이 널 링크입니다. 따라서 k가 커질수록 널 링크의 비율이 증가합니다. 널 링크는 이용 가치가 거의 없는 정보이기 때문에 널 링크에 필요한 메모리의 양이 많아진다는 것은 그만큼 메모리를 비효율적으로 사용한다는 것을 의미합니다. 그러므로 차수가 큰 트리를 다중 링크로 표현하는 것은 그다지 효율적인 표현 방법으로 볼 수 없습니다.

■ 이진 트리 표현법

이진 트리의 표현처럼 두 개의 링크 필드만 가진 노드를 사용하여 일반적인 트리를 표현하는 방법이 있습니다. 이 표현법을 소개하기 전에 몇 가지 용어를 정의합니다. 어떤 노드의 자식이 k개 있다고 하고, 차례대로 c_0, c_1, … , c_{k-1}로 표현된다고 합시다. 이때, c_0를 첫 번째 자식 또는 가장 왼쪽에 있는 자식이라고 부릅니다. 그리고 $i < k-1$일 때, c_i는 c_{i+1}의 왼쪽 형제라고 부르고, $i > 0$일 때, c_i는 c_{i-1}의 오른쪽 형제라고 부릅니다.

일반적인 트리를 표현하기 위해, 노드마다 가장 왼쪽에 있는 자식과 오른쪽 형제를 가리키는 두 개의 링크 필드를 둡니다. 이러한 표현을 **왼쪽 자식-오른쪽 형제(leftmost child-right sibling) 표현**이라고 부르기도 합니다. 그림 9.15는 왼쪽 자식-오른쪽 형제 표현의 예를 보여줍니다. 물론 여기서도 필요시에는 부모 노드를 위한 링크 필드를 추가할 수 있습니다.

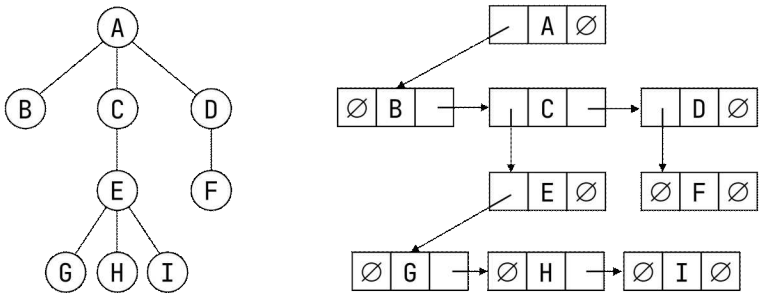


그림 9.15: 트리의 왼쪽 자식-오른쪽 형제 표현

왼쪽 자식-오른쪽 형제 표현은 기본적으로 2개의 링크 필드를 가지고 있으므로 이진 트리와 구조가 동일합니다. 따라서 가장 왼쪽의 자식과 오른쪽 형제를 가리키기 위한 두 링크 필드를 각각 이진 트리의 왼쪽 자식과 오른쪽 자식을 가리키는 링크 필드로 간주한다면, 일반적인 트리를 이진 트리로 변환하여 표현한 것이라고 볼 수 있습니다. 그래서 왼쪽 자식-오른쪽 형제 표현법을 이진 트리 표현법이라고도 합니다. 그림 9.16는 일반적인 트리를 나타내는 이진 트리 표현의 예를 보여줍니다.

이진 트리 표현법은 메모리를 효율적으로 사용한다는 점에서 장점이 있지만, 원래 트리에서의 부모나 자식을 찾는 데는 상대적으로 속도가 느리다는 단점이 있습니다.

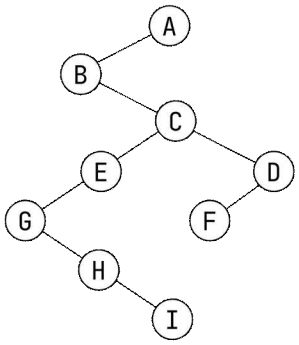


그림 9.16: 그림 9.15의 트리에 대한 이진 트리 표현

■ 자식 리스트-부모 포인터 표현법

자식 리스트-부모 포인터 표현법은 배열과 연결리스트를 결합한 표현입니다. 이 표현에서 사용하는 배열의 길이는 트리의 노드 수와 같습니다. 배열의 각 원소는 세 개의 요소로 구성됩니다. 이 세 요소는 트리 원소를 위한 요소, 배열에서 부모의 위치(첨자)를 위한 요소, 그리고 자식 원소들로 구성된 연결리스트를 가리키는 링크를 위한 요소입니다. 자식들을 위한 연결리스트의 노드는 배열에서 자식 원소의 위치(첨자)를 위한 데이터 필드와 오른쪽 형제 정보를 가진 노드를 가리키는 링크 필드로 구성됩니다. 그림 9.17은 자식 리스트-부모 포인터 표현의 예를 보여줍니다. 그림에서 루트 A는 부모가 없으므로 배열에서 부모 위치를 -1로 둡니다. 이 표현에서 연결리스트 대신에 동적 배열인 파이썬 리스트로 대체해도 됩니다. 또한, 13장에서 소개할 해싱이나 파이썬 딕셔너리를 이용하여 키(key)는 부모, 값(value)은 자식 리스트(또는 집합)인 요소들의 그룹으로 구성할 수도 있습니다.

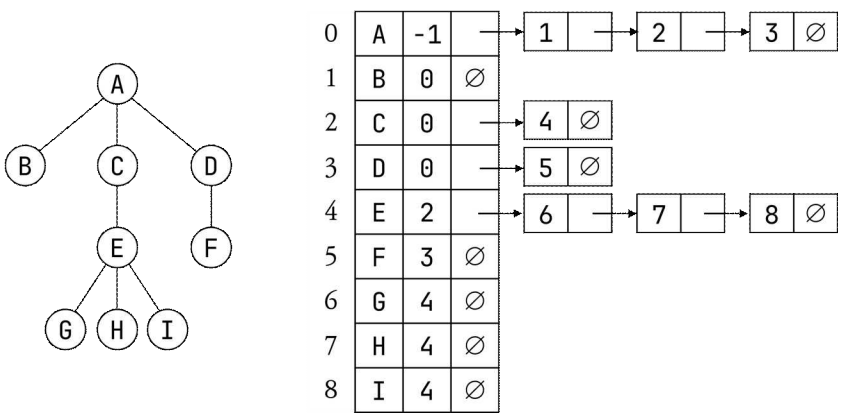


그림 9.17: 자식 리스트-부모 포인터 표현

자식 리스트-부모 포인터 표현은 부모와 자식을 빠르게 검색할 수 있다는 장점을 가지고 있으면서도 메모리를 효율적으로 사용하기 때문에, 일반적인 트리를 다루는 응용에서 많이 사용됩니다.

■ 부모 포인터 배열 표현법

부모 포인터 배열 표현은 자식 리스트-부모 포인터 표현에서 자식 리스트 부분을 제거한 형태입니다. 이 표현은 배열 첨자가 각 노드를 나타내고, 배열 원소의 값은 해당 노드의 부모 위치(첨자)를 나타냅니다. 단순한 표현 방법이어서 다양한 용도로는 사용할 수 없지만, 응용에 따라서는 매우 유용하게 사용됩니다. 부모 포인터 배열로 여러 개의 트리를 한 번에 표현할 수도 있습니다. -1을 가진 배열 원소의 첨자는 트리의 루트에 해당합니다. 그림 9.17은 두 개의 트리에 대한 부모 포인터 배열의 예를 보여줍니다.

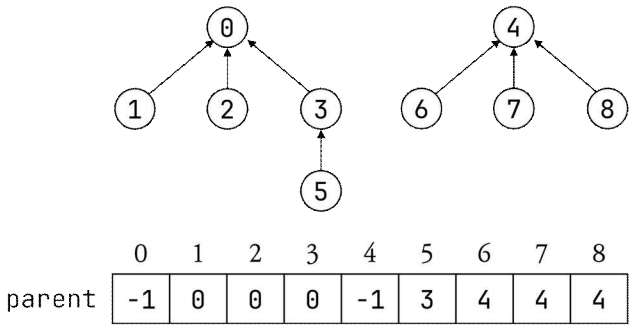


그림 9.18: 부모 포인터 배열

부모 포인터 배열은 메모리를 매우 효율적으로 사용한다는 점과 부모를 상수 시간에 찾을 수 있으며 간단히 구현될 수 있다는 장점을 가집니다. 단점으로는 자식을 찾기 위해서는 배열 전체를 탐색해야 한다는 점과 트리의 구조를 즉시 이해하기 어렵다는 점입니다. 보통 트리의 높이가 낮고 자식 개수가 많은 경우에 이 표현을 사용합니다.

9.5 사례 연구: 최적 정렬 시간

8장에서 합병 정렬의 최악의 경우 시간복잡도는 $O(n\log n)$이라고 하였습니다. 그러면 시간복

잡도가 $O(n\log n)$보다 더 낮은 차수를 갖는 정렬 알고리즘은 없을까요? 만약 그러한 알고리즘이 존재하지 않는다면 합병 정렬은 가장 효율적인 정렬 알고리즘이라고 부를 수 있을 것입니다.

대부분의 정렬 알고리즘은 기본적으로 두 요소의 비교를 통해서 정렬을 수행합니다. 이러한 비교 기반의 알고리즘을 결정 트리로 나타낼 수 있습니다. **결정 트리(decision tree)**는 논리식과 같은 의사 결정을 위한 객체를 내부 노드로 표현하고, 참과 거짓에 의해 분기되는 노드 사이의 연결 관계를 에지로 표현하는 이진 트리입니다. 결정 트리의 잎 노드는 최종적인 결정 결과를 나타냅니다. 그림 9.19는 세 개의 숫자 a, b, c를 정렬하는 결정 트리의 예를 보여줍니다. 이 결정 트리에서 잎 노드는 정렬된 순서를 나타냅니다.

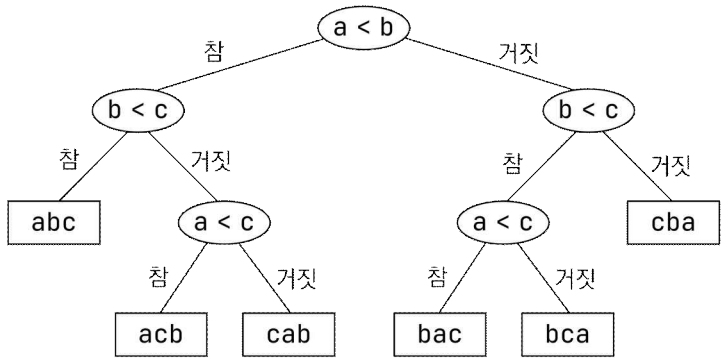


그림 9.19: 세 숫자의 정렬을 위한 결정 트리

그림 9.19의 결정 트리를 보면 세 숫자의 정렬 결과를 알기 위해서는 최소 2번의 비교가 필요하고, 어떤 경우에는 3번의 비교를 수행해야 합니다. 결국 높이가 k인 결정 트리가 있을 때, 이 결정 트리를 사용하면 최악의 경우 k번의 비교를 수행해야만 정렬 결과를 알 수 있습니다.

어떤 정렬 알고리즘이라도 두 요소를 비교하는 순서를 고려하여 결정 트리로 표현할 수 있습니다. 반대로 얘기하면 각 결정 트리는 그 자체가 바로 하나의 알고리즘과 같습니다. 따라서 어떤 정렬 알고리즘도 최악의 경우 가장 좋은 결정 트리의 높이보다 비교 연산을 더 적게 수행할 수 없습니다.

이제 n개의 원소를 정렬하는 결정 트리를 고려해 봅시다. n개의 원소를 정렬하는 모든 가능한 결과는 순열의 개수인 $n!$ 입니다. 따라서 결정 트리의 잎 노드의 개수는 $n!$ 이어야 합니다. m개의 잎 노드를 가진 이진 트리의 높이 k는, m개의 잎 노드를 가진 완전 이진 트리의 높이보다 작을 수 없으므로, $k \geq \lceil \log m \rceil$ 입니다. 그러므로, $n!$ 개의 잎 노드를 가진 결정 트리의 높이 k는 다음과 같습니다.

$$k \geq \lceil \log(n!) \rceil$$

n!은 다음과 같이 유도될 수 있습니다.

$$n! = n \times (n-1) \times (n-2) \times \cdots \times \lceil n/2 \rceil \times \cdots \times 1$$
$$\geq (n/2) \times (n/2) \times \cdots \times (n/2) = (n/2)^{n/2}$$

따라서, 결정 트리의 높이 k는 다음과 같이 유도됩니다.

$$k \geq \lceil \log(n!) \rceil \geq \log(n/2)^{n/2} = (n/2)\log(n/2)$$

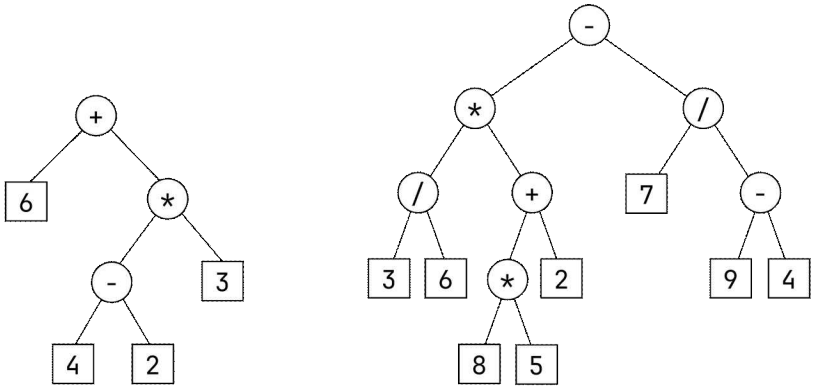
결국 n! 개의 잎 노드를 가진 결정 트리의 높이는 $\Omega(n\log n)$이므로, 가장 좋은 비교 기반 정렬 알고리즘의 최악의 경우 시간복잡도 역시 $\Omega(n\log n)$입니다. 이 말은 비교 기반의 어떤 정렬 알고리즘이라도 $\Omega(n\log n)$ 번의 비교 연산을 수행해야 한다는 것을 의미합니다.

이런 이유로 비교 기반 정렬의 **하한(lower bound)**은 $\Omega(n\log n)$이고, 이 하한을 시간복잡도로 가지는 알고리즘을 **최적 알고리즘(optimal algorithm)**이라고 부릅니다. 그러므로 합병 정렬은 최악의 경우에 대해 최적인 정렬 알고리즘입니다. 비슷한 방식으로 분석하면 평균의 경우도 정렬 문제의 하한이 $\Omega(n\log n)$임을 보일 수 있습니다. 따라서 퀵 정렬은 최악의 경우 시간복잡도는 $O(n^2)$이므로 최적이 아니지만, 평균의 경우에는 최적 정렬 알고리즘입니다.

9.6 사례 연구: 수식 트리

5장에서는 산술식을 표현하는 방법인 전위 표기법, 중위 표기법, 후위 표기법을 소개하였습니다. 우리가 보통 사용하는 수식은 중위 표기법이지만, 후위 표기법으로 표현된 수식은 연산자들 사이에 우선순위와 결합법칙을 고려할 필요가 없고 수식의 계산도 더 쉽게 수행할 수 있습니다.

수식은 선형 데이터이지만 효율적인 처리를 위해서 트리 구조로 나타내기도 합니다. 수식을 표현한 트리를 **수식 트리(expression tree)**라고 부릅니다. 수식 트리를 사용하면 수식의 계산이나 전위 수식, 중위 수식, 후위 수식과 같은 표기법 변환을 쉽게 처리할 수 있습니다. 그림 9.20에는 이항 연산자만 가진 수식을 표현한 수식 트리의 예가 나와 있습니다.



(a) 6+(4-2)*3 (b) 3/6*(8*5+2)-7/(9-4)

그림 9.20: 수식을 표현한 트리의 예

프로그램 9.7에는 후위 수식으로부터 수식 트리를 구축하고, 수식 트리로 표현된 수식을 계산하는 프로그램이 나와 있습니다. 수식 트리를 구축하는 과정은 5장에서 소개한 후위 수식을 계산하는 과정과 비슷합니다. 후위 수식을 왼쪽부터 읽으면서 피연산자를 만나면 트리 노드를 만들어 스택에 저장합니다. 연산자를 만나면 스택에서 꺼내 온 두 개의 피연산자 노드를 자식으로 하는 부모 노드를 만듭니다. 수식 트리의 계산 과정은 후위 순회처럼 탐색하면서 계산하는 것입니다. 함수 `make_tree(postfix)`는 후위 수식의 토큰(연산자, 피연산자)으로 구성된 리스트 `postfix`로부터 수식 트리를 구축하고 트리의 루트를 반환합니다. 재귀 함수 `evaluate(p)`는 맨 처음 수식 트리의 루트 노드를 가지고 호출되며, 파이썬 내장 함수 `eval()`을 이용하기 위해 수식의 계산 결과를 문자열로 반환합니다. 두 함수는 모두 수식의 토큰 개수에 대한 선형 시간에 수행됩니다. 프로그램에서 사용하는 `BinNode` 클래스는 프로그램 9.1에서 정의한 클래스입니다.

프로그램 9.7: 후위 수식으로부터 수식 트리를 구축하는 프로그램

```
1 def make_expr_tree(postfix):       # postfix : 후위 수식의 토큰 리스트
2    if not postfix:                          # 빈 수식인 경우 None 반환
3       return None
4    stack = []                               # 리스트를 이용한 스택 표현
5    for token in postfix:                    # 각 연산자 또는 피연산자에 대해
6       p = BinNode(token)                    # 트리 노드 생성
7       if token in ['+', '-', '*', '/']:     # token이 연산자이면
8          p.right = stack.pop()              # 오른쪽 자식으로 연결
9          p.left = stack.pop()               # 왼쪽 자식으로 연결
```

```
10      stack.append(p)                          # 스택에 트리 노드 삽입
11   return stack[0]                             # 루트 노드 반환
12
13 def evaluate(p):
14   if p is None:                               # 빈 수식이면 None 반환
15      return None
16   if p.left is None and p.right is None:      # 잎 노드이면
17      return p.data                            # 피연산자 반환
18   lval = evaluate(p.left)                     # 왼쪽 부트리 계산
19   rval = evaluate(p.right)                    # 오른쪽 부트리 계산
20   return str(eval(lval + p.data + rval))      # 계산 결과를 문자열로 반환
21
22 # 테스트
23 expr = "6 4 2 - 3 * +"                        # 6 + ((4 - 2) * 3)
24 root = make_expr_tree(expr.split())
25 print(evaluate(root))                         # 출력 결과: 12
```

프로그램 9.8에는 수식 트리로부터 전위 수식, 중위 수식, 후위 수식 표현을 생성하는 프로그램이 나와 있습니다. 수식 트리에 전위 순회를 수행하면 전위 표기법으로, 중위 순회를 수행하면 중위 표기법으로, 후위 순회를 수행하면 후위 표기법으로 수식이 표현됩니다. 함수 get_prefix(p), get_postfix(p), get_infix(p)는 맨 처음 수식 트리의 루트 노드를 가지고 호출하면, 수식을 토큰들의 리스트로 반환합니다. 함수 get_prefix()와 get_postfix()는 널 링크를 가지고 호출하면 공백 리스트를 반환합니다. 함수 get_infix()는 중위 표기법에서 연산자 우선순위를 고려하지 않아도 되도록, 각 연산에 대해 괄호로 포함하여 반환합니다. 피연산자에는 괄호를 붙이지 않기 위해 잎 노드에서는 괄호 없이 반환합니다.

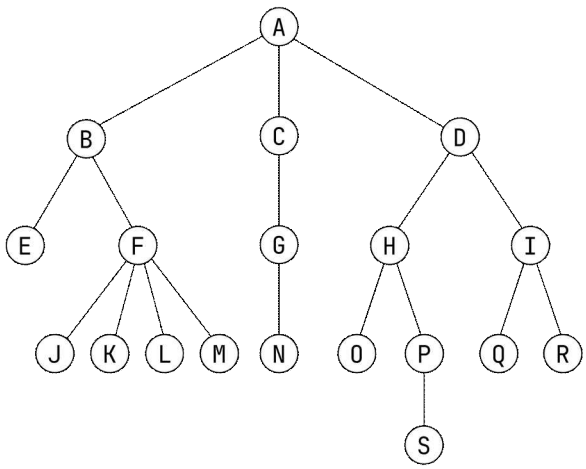
프로그램 9.8: 수식 트리로부터 전위, 중위, 후위 수식을 생성하는 프로그램

```
1  def get_prefix(p):                  # 전위 수식 생성
2     if p is None:                    # 공백 리스트 반환
3        return []
4     L = get_prefix(p.left)           # 왼쪽 부트리로부터 전위 수식 생성
5     R = get_prefix(p.right)          # 오른쪽 부트리로부터 전위 수식 생성
6     return [p.data] + L + R          # 전위 수식 반환
7
8  def get_postfix(p):                 # 후위 수식 생성
9     if p is None:                    # 공백 리스트 반환
10       return []
11    L = get_postfix(p.left)          # 왼쪽 부트리로부터 후위 수식 생성
```

```
12    R = get_postfix(p.right)      # 오른쪽 부트리로부터 후위 수식 생성
13    return L + R + [p.data]       # 후위 수식 반환
14
15 def get_infix(p):                # 중위 수식 생성
16    if p is None:                 # 공백 리스트 반환
17       return []
18    if p.left is None and p.right is None:
19       return [p.data]            # 잎 노드인 피연산자에는 괄호 없이 반환
20    L = infix(p.left)             # 왼쪽 부트리로부터 중위 수식 생성
21    R = infix(p.right)            # 오른쪽 부트리로부터 중위 수식 생성
22    return ['('] + L + [p.data] + R + [')']     # 중위 수식 반환
23
24 # 테스트
25 expr = "6 4 2 - 3 * +"           # 6 + ((4 - 2) * 3)
26 root = make_expr_tree(expr.split())  # 수식 트리 생성 - 프로그램 9.7
27 print(*get_prefix(root))         # 출력 결과 : + 6 * - 4 2 3
28 print(*get_postfix(root))        # 출력 결과 : 6 4 2 - 3 * +
29 print(*get_infix(root))          # 출력 결과 : ( 6 + ( ( 4 - 2 ) * 3 ) )
```

연습문제

1. 아래 그림의 트리에 대해서 다음 물음에 답하시오.
   (1) 노드 A의 차수는?
   (2) 이 트리의 차수는?
   (3) 이 트리의 높이는?
   (4) 잎 노드의 개수는?
   (5) 노드 F의 형제는?
   (6) 노드 J의 조상은?
   (7) 노드 S와 노드 R의 공통 조상은?



2. 100개의 노드를 가진 완전 이진 트리의 높이를 구하시오.

3. n개의 노드를 가진 완전 이진 트리의 잎 노드 개수를 n에 대한 식으로 나타내시오.

4. 이진 트리의 잎 노드의 개수를 세는 프로그램을 작성하시오.

5. 이진 트리의 너비(width)는 같은 깊이에 있는 노드의 최대 개수이다. 이진 트리의 너비를 찾는 프로그램을 작성하시오.

6. 두 이진 트리가 구조적으로 동일하고 노드 값도 같은지 판별하는 프로그램을 작성하시오.

7. 다음 수식에 대한 수식 트리로 그리시오.
   (1) A * (B + C - D) / (E + F) - G [중위 수식]
   (2) * A + B * C + D E [전위 수식]
   (3) A B C D - * E / + [후위 수식]

8. 4개의 노드를 가진 이진 트리의 종류를 모두 나열하시오.

9. 다음의 전위 순서와 중위 순서로 정의되는 이진 트리를 그리시오.
   (1) 전위 순서: ABCDEFG
   중위 순서: BAEDCFG

   (2) 전위 순서: ABCDEFG
   중위 순서: DCBFGEA

10. 다음의 후위 순서와 중위 순서로 정의되는 이진 트리를 그리시오.
   (1) 후위 순서: ABCDEFGHI
   중위 순서: ABECDIFGH

   (2) 후위 순서: ABCDEFGHI
   중위 순서: IACBHGDEF

11. 후위 순서와 중위 순서로 정의되는 이진 트리를 구축하는 프로그램을 작성하시오.

12. 반복 구조를 사용하여 전위 순회를 수행하는 프로그램을 작성하시오.

13. 반복 구조를 사용하여 후위 순회를 수행하는 프로그램을 작성하시오.

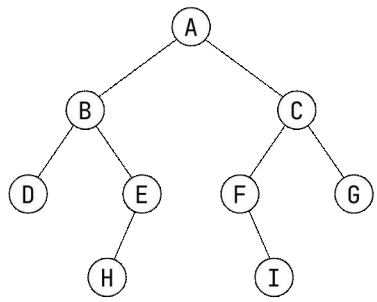
14. 문제 1의 트리를 이진 트리 표현 방식으로 바꾸시오.

15. 7개의 숫자를 정렬하기 위한 높이가 가장 작은 결정 트리를 그리시오.

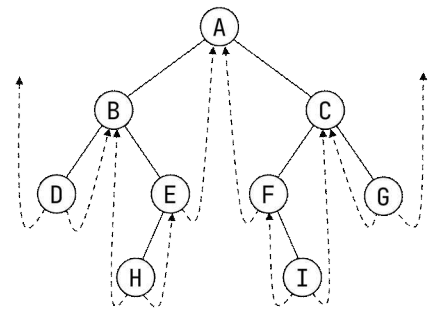
16. 이진 트리의 순회에서 노드 A 다음에 B, B 다음에 C가 방문된다면, A를 B의 **선행자(predecessor)**라고 하고, C를 B의 **후속자(successor)**라고 한다. 중위 순서에서 어떤 노드의 선행자와 후속자를 각각 그 노드의 중위 선행자와 중위 후속자라고 부른다. 변수 p가 이진 트리의 어떤 노드를 가리킬 때, 노드 p의 중위 후속자와 중위 선행자를 찾는 알고리즘을 기술하시오.

17. 16번과 같은 방식으로 전위 선행자, 전위 후속자, 후위 선행자, 후위 후속자를 정의할 수 있다. 변수 p가 이진 트리의 어떤 노드를 가리킬 때, 노드 p의 전위 선행자, 전위 후속자, 후위 선행자, 후위 후속자를 각각 찾는 알고리즘을 기술하시오.

18. **스레드 이진 트리(threaded binary tree)**는 이진 트리의 연결 구조 표현에서 널 링크 대신에 트리의 다른 노드를 가리키도록 만든 구조이다. 스레드 이진 트리를 위한 노드에는 2개의 논리 필드를 추가로 배정하여 노드의 왼쪽 링크와 오른쪽 링크가 각각 정상적인 자식을 가리키는 링크인지 스레드인지를 구분한다. 스레드 이진 트리를 구성하는 한 가지 방법은 왼쪽 널 링크 대신에 그 노드의 중위 선행자 노드를 가리키도록 하고, 오른쪽 널 링크 대신에 그 노드의 중위 후속자 노드를 가리키도록 하는 것이다. 아래 그림은 일반적인 이진 트리에 스레드를 표시한 스레드 이진 트리의 예를 보여준다. 그림에서 점선은 스레드 포인터를 나타낸다.



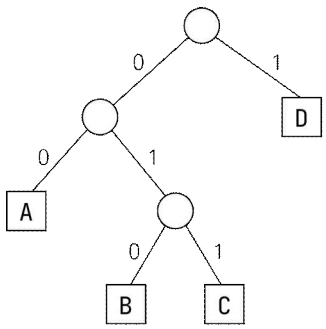
〈이진 트리〉



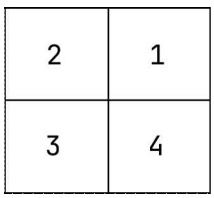
〈스레드 이진 트리〉

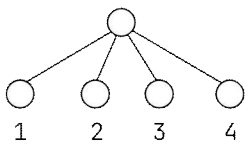
스레드 이진 트리에서 중위 순회를 수행할 때 순환 함수나 스택을 사용하지 않아도 된다. 스레드 이진 트리에서 중위 순회를 수행하는 알고리즘을 기술하시오.

19. 문자 집합에 대한 **인코딩 트리(encoding tree)**는 각 문자에 문자 코드를 부여할 수 있는 트리를 말한다. 인코딩 트리의 각 잎 노드에는 문자가 저장되고, 트리의 왼쪽 에지에는 0, 오른쪽 에지에는 1이 부여된다(아래 그림 참조). 잎 노드에 있는 문자의 코드는 루트에서부터 그 노드까지의 경로 상에 있는 0과 1의 순서이다. 그림에서 문자 'A'의 코드는 '00'이고, 문자 'B'의 코드는 '010'이다. 인코딩 트리가 주어질 때, 각 문자에 대한 문자 코드를 출력하는 프로그램을 작성하시오.

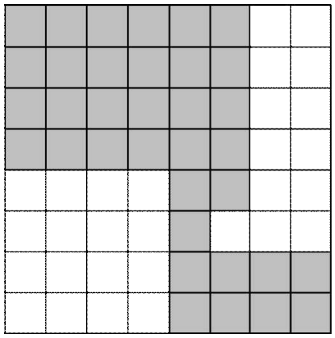


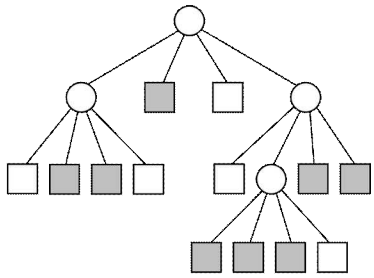
20. [실습 문제] **사진 트리(quad tree)**는 각 내부 노드가 4개의 자식을 가지는 트리로서 2차원 이진 영상(binary image)을 표현하기에 적합한 자료구조이다. 사진 트리의 각 노드는 이진 영상의 일정 영역에 대응한다. 루트는 전체 영역에 대응하고, 각 자식 노드는 부모 노드에 대응하는 영역에서 사등분된 각 영역에 대응된다(아래 그림 참조).





영상이 흑백으로 구성되어 있을 때, 흑색에 해당하는 물체를 사진 트리로 표현하는 방법은 다음과 같다. 먼저 루트 노드를 전체 영상 영역에 대응시킨다. 그다음, 각 노드에 대응하는 영역 전체가 물체에 포함되어 있거나, 또는 전혀 포함되어 있지 않으면 그 노드는 잎 노드로 둔다. 영역 중 일부분만 물체에 해당하는 경우는 4개의 자식 노드를 두고, 각 자식 노드를 사등분된 각 분할 영역에 대응시킨다. 따라서 각 노드에는 영역이 분할되는 경우, 영역이 물체에 완전히 포함되는 경우, 전혀 포함되지 않은 경우를 구분하는 정보를 저장해야 한다. 다음 그림은 이진 영상의 예와 그에 따른 사진 트리를 보여준다.





그림의 사진 트리에서 사각형으로 표시된 노드는 잎 노드로서 더 이상 분할이 필요 없는 영역에 대응한다. 검은 사각형은 물체에 완전히 포함된 경우를 나타내고, 흰 사각형은 전혀 포함되지 않은 경우를 나타낸다. 원으로 표시된 각 내부 노드는 일부분만이 물체에 포함되어 분할되어야 하는 경우를 나타낸다.

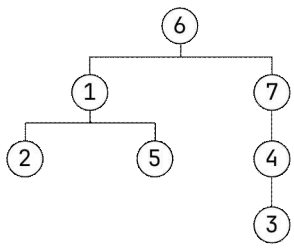
n개의 직사각형이 주어질 때, n개의 직사각형이 차지하는 영역의 크기를 구하는 프로그램을 작성하시오. 이때, 각 직사각형을 표현하는 사진 트리를 만들고, 사진 트리를 이용하여 직사각형의 합집합을 구한 후 이 합집합 영역의 크기를 구하라.

입력으로 첫 줄에 직사각형의 총 개수 n이 주어진다. 둘째 줄부터는 각 줄에 하나씩 직사각형의 정보 a, b, c, d가 들어 있다, 여기서 (a, b)와 (c, d)는 각각 직사각형의 왼쪽 아래 꼭짓점과 오른쪽 위 꼭짓점의 좌표이다. n은 각각 2 이상 1,000 이하의 정수이고, a, b, c, d는 1 이상 10,000 이하의 정수이다. 출력으로 직사각형들이 차지하는 영역의 크기를 출력한다.

입력의 예	출력의 예
2 20 20 100 70 50 50 70 100	4600

21. [실습 문제] A씨의 조상은 200년 전에 외국에서 이주해 왔다는 사실만 알려져 있고, A씨 가문의 내력에 대해서 구체적으로 남겨져 있는 자료가 없다. 그래서 A씨는 가문의 뿌리를 찾기로 하고, 수소문 끝에 친척들에 대한 여러 가지 기초 자료들을 모았다. 기초 자료는 'X는 Y의 부모이다' 또는 'X와 Y는 형제이다'로 구성되어 있다. A씨는 이 기초 자료를 통해 족보를 구성하고자 한다. 예를 들어, 다음 왼쪽의 기초 자료에 대한 족보는 오른쪽 그림과 같다.

1은 2의 부모이다.
2와 5는 형제이다.
4는 3의 부모이다.
6는 1의 부모이다.
1과 7은 형제이다.
7은 4의 부모이다.



이 기초 자료를 가지고 족보를 구성하여, 이를 통해 친척 두 사람의 촌수를 계산하는 프로그램을 작성하시오. 위의 예에서 2번과 3번 두 사람의 촌수는 5촌이다. 족보의 기초 자료는 족보를 완전히 구성할 수 있을 만큼 충실하다고 가정한다.

입력으로 첫 줄에 친척들의 인원수 n과 기초 자료의 개수 m이 주어진다. n, m은 각각 2 이상 10,000 이하의 정수이다. 둘째 줄부터 m개의 줄에는 세 정수 a b k가 주어지는데, a와 b는 친척들의 번호이고, k는 0 또는 1이다. k가 0이면 "a는 b의 부모이다"로 해석되고, k가 1이면 "a와 b는 형제이다"로 해석된다. 그다음 줄에는 촌수를 알고자 하는 친척 쌍의 개수 p가 주어지고, 그다음 p개의 각 줄에는 두 사람의 번호가 들어온다. p는 1 이상 100 이하의 정수이다. 출력으로, 입력의 마지막 p 줄에 주어진 각 두 사람의 촌수를 한 줄에 하나씩 출력한다. 입력과 출력의 예는 다음과 같다.

입력의 예	출력의 예
7 6 1 2 0 2 5 1 4 3 0 6 1 0 1 7 1 7 4 0 2 2 3 5 6	5 2

제10장

트리 응용

10.1 이진 탐색 트리
10.2 힙 자료구조
10.3 분리된 집합을 위한 트리 표현
10.4 사례 연구: 세그먼트 트리
연습문제

학습 목표

- 이진 탐색 트리의 정의와 개념을 이해하고, 탐색, 삽입, 삭제 연산의 수행 과정을 상세하게 설명할 수 있다.
- 우선순위 큐로서 힙의 정의와 개념을 이해하고, 힙에서의 삽입과 삭제 연산의 수행 과정을 설명할 수 있으며, 파이썬 heapq 모듈을 활용할 수 있다.
- 분리된 집합 모임을 위한 union-find 자료구조의 동작 원리와 성능을 이해하고, 이 자료구조를 이용하여 효율적으로 해결할 수 있는 문제의 특성을 설명할 수 있다.
- 세그먼트 트리의 구조를 이해하고, 트리 구축, 질의, 갱신 연산을 직접 구현할 수 있다.

10.1 이진 탐색 트리

이 절에서는 원소의 삽입과 삭제가 빈번하게 발생하는 동적 데이터(dynamic data)를 효율적으로 관리하기 위한 자료구조인 이진 탐색 트리를 살펴봅니다. 동적 데이터를 효율적으로 관리하려면 탐색뿐만 아니라 삽입과 삭제 연산도 효율적으로 수행할 수 있어야 합니다. 일반적인 선형 자료구조로는 탐색, 삽입, 삭제를 모두 효율적으로 처리할 수 없습니다. 이진 탐색 트리는 트리 구조를 이용하여 동적 데이터를 관리하는 가장 기본적인 자료구조입니다. 이진 탐색 트리보다 복잡하지만 좀 더 효율적으로 데이터를 관리하는 탐색 트리 구조는 13장에서 소개합니다.

이진 탐색 트리(binary search tree)는 각 노드가 자료 원소에 대응하는 이진 트리로서, 자료 원소에는 탐색 대상인 '키(key)'가 있습니다. 이진 탐색 트리로 표현하는 데이터에는 동일한 키를 가진 자료 원소가 없다고 가정합니다.[21] 공백이 아닌 이진 탐색 트리는 다음의 성질을 만족합니다.

① 루트 노드의 키는 왼쪽 부트리에 있는 어떤 키보다도 크다.
② 루트 노드의 키는 오른쪽 부트리에 있는 어떤 키보다도 작다.
③ 왼쪽 부트리와 오른쪽 부트리도 이진 탐색 트리이다.

그림 10.1은 이진 탐색 트리의 예를 보여줍니다. 그림에서 트리 노드에 표시된 값은 자료 원소의 키를 나타냅니다. 이진 트리의 어떤 구조도 위의 세 성질을 만족하면 이진 탐색 트리가 될 수 있습니다. 즉 완전 이진 트리나 경사 이진 트리도 이진 탐색 트리가 될 수 있습니다. 따라서

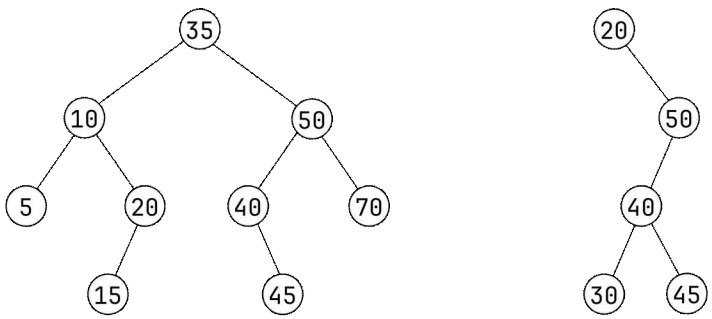


그림 10.1: 이진 탐색 트리의 예

21) 일반적으로 이진 탐색 트리에서는 중복된 키를 허용하지 않지만, 이를 허용하는 경우는 중복 횟수와 함께 같은 노드에 자료 원소를 저장하거나 항상 오른쪽 부트리에 저장하는 등 특정 규칙을 따릅니다.

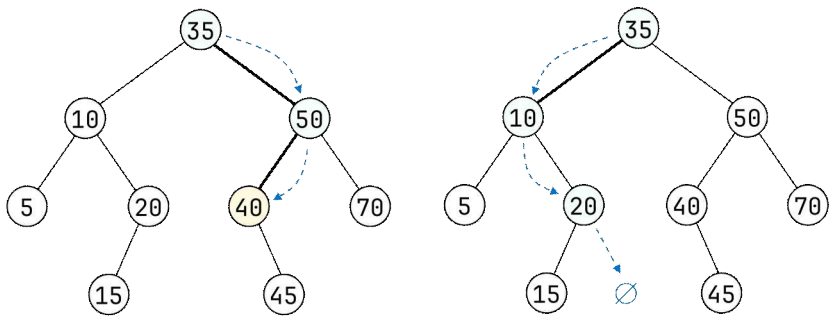
이진 탐색 트리의 높이는 최소 $\lfloor \log n \rfloor$부터 최대 n-1까지 될 수 있습니다. 또한 이진 탐색 트리의 성질로 인해 이진 탐색 트리에 대해 중위 순회를 수행하면 키들의 정렬된 순서로 노드들을 방문하게 됩니다.

이제, 이진 탐색 트리의 표현 방법과 탐색, 삽입, 삭제 연산의 구현 방식을 소개합니다. 이진 탐색 트리를 위한 노드 클래스는 프로그램 10.1에 나와 있습니다(프로그램 9.1의 BinNode 클래스와 동일합니다). 앞으로 설명의 편의를 위해 자료 원소가 키 자체라고 가정합니다.

프로그램 10.1: 이진 탐색 트리의 노드 클래스

```
1 class BinNode:
2    def __init__(self, item=None, left_child=None, right_child=None):
3       self.data = item              # 자료 원소
4       self.left = left_child        # 왼쪽 자식을 가리키는 링크
5       self.right = right_child      # 오른쪽 자식을 가리키는 링크
```

이진 탐색 트리에서의 탐색 과정은 다음과 같습니다. 탐색하려는 원소가 item이라고 가정하겠습니다. 먼저, 이진 탐색 트리의 루트 노드부터 조사합니다. 만일 노드의 원소가 item과 일치하면 탐색은 성공적으로 완료됩니다. 그렇지 않은 경우, 노드 원소보다 item이 더 작으면 왼쪽 자식으로, 더 크면 오른쪽 자식으로 내려갑니다. 이 과정은 item을 가진 노드를 찾거나 널 링크를 만나 탐색이 실패할 때까지 반복됩니다. 그림 10.2는 이진 탐색 트리에서 탐색을 수행하는 예를 보여줍니다.



(a) 원소 '40'의 탐색 (b) 원소 '25'의 탐색

그림 10.2: 이진 탐색 트리에서 탐색의 예

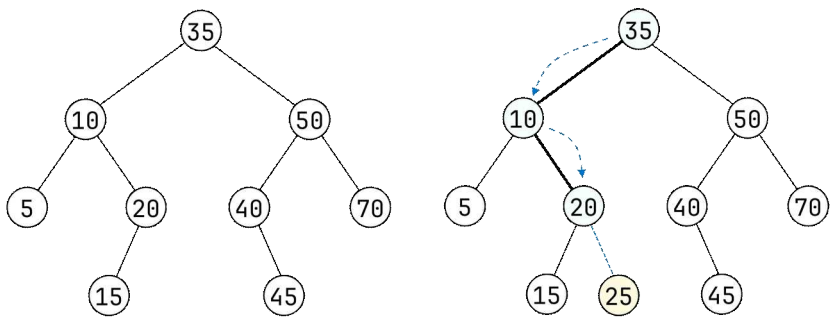
프로그램 10.2의 함수 search(root, item)은 이진 탐색 트리 root에서 원소 item을 가진 노드를 탐색합니다. 탐색이 성공하면 찾은 노드의 링크를 반환하고, 실패하면 None을 반환합

니다. 이 함수는 트리의 각 레벨에서 기껏해야 하나의 노드만 조사합니다. 그래서 수행 시간은 O(트리의 높이+1)입니다. 트리의 높이는 최대 n-1이 될 수 있으므로, 최악의 경우 수행 시간은 O(n)입니다.

프로그램 10.2: 이진 탐색 트리에서 탐색을 수행하는 프로그램

```
1  def search(root, item):
2     """이진 탐색 트리에서 item을 가진 노드의 링크를 반환한다."""
3     node = root                    # 루트 노드부터 탐색
4     while node is not None:        # 널 링크가 아니면 반복
5        if item == node.data:       # 탐색 성공
6           return node
7        if item < node.data:        # 왼쪽 부트리 탐색
8           node = node.left
9        else:                       # 오른쪽 부트리 탐색
10          node = node.right
11    return None                    # 탐색 실패
```

다음으로, 새로운 원소 item을 삽입하는 과정을 살펴봅니다. 삽입 과정은 탐색 과정과 매우 비슷합니다. 삽입에서는 item을 가진 노드가 존재하지 않으므로, 루트부터 시작하여 널 링크를 만날 때까지 자식 노드를 찾아 내려갑니다. 널 링크를 만나면 그 직전에 조사한 노드의 자식 노드를 만들어 item을 저장합니다. 이를 위해, 자식 노드의 링크와 부모 노드의 링크를 함께 추적하며 탐색합니다. 자식 노드 링크가 None이면 부모 노드 링크를 사용하여 새 노드를 삽입합니다. 이처럼 이진 탐색 트리에서 새로 삽입되는 노드는 항상 잎 노드가 됩니다. 그림 10.3은 삽입 과정의 예를 보여줍니다.



(a) 원래 트리 (b) 원소가 '25'인 노드의 삽입

그림 10.3: 이진 탐색 트리에서 삽입의 예

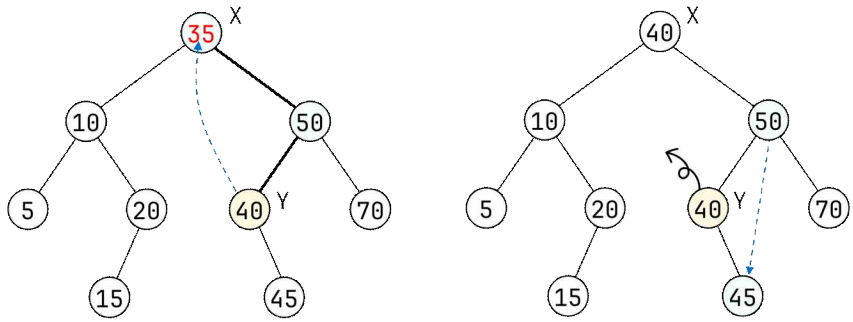
이진 탐색 트리가 공백이면 새 노드가 루트가 됩니다. 따라서 삽입 후에는 루트가 바뀔 수 있다는 점에 유의해야 합니다. 프로그램 10.3의 함수 insert(root, item)은 이진 탐색 트리 root에 원소 item을 가진 새로운 노드를 삽입한 후 루트 노드를 반환합니다. 프로그램에서 변수 parent와 node는 각각 부모 노드와 자식 노드를 참조합니다. 변수 node가 널 링크를 가지면, 삽입하려는 새 노드를 노드 parent의 자식으로 설정합니다. 이때, 노드 parent의 원소와 item을 비교하여 왼쪽 자식으로 설정할지 오른쪽 자식으로 설정할지를 결정합니다. 삽입 과정에서도 트리의 각 레벨에서 기껏해야 하나의 노드만 조사합니다. 그래서 수행 시간은 O(트리의 높이+1)이고, 따라서 최악의 경우 수행 시간은 O(n)입니다.

프로그램 10.3: 이진 탐색 트리에서 삽입을 수행하는 프로그램

```
1  def insert(root, item):
2     """이진 탐색 트리에 item을 가진 노드를 삽입하고, 루트 노드를 반환한다."""
3     if root is None:              # 공백이면, 새 노드를 루트로 반환
4        return BinNode(item)
5
6     parent = None                 # 초기에 None을 부모로 설정
7     node = root                   # 초기에 루트를 자식으로 설정
8     while node is not None:       # 널 링크를 만날 때까지 반복
9        parent = node              # 자식 레벨로 내려옴
10       if item < node.data:       # item이 작으면 왼쪽 부트리 탐색
11          node = node.left
12       else:                      # item이 크면 오른쪽 부트리 탐색
13          node = node.right
14
15    new_node = BinNode(item)      # 삽입할 새 노드 생성
16    if item < parent.data:        # 부모보다 키가 작으면, 왼쪽 자식으로 삽입
17       parent.left = new_node
18    else:                         # 그렇지 않으면, 오른쪽 자식으로 삽입
19       parent.right = new_node
20
21    return root                   # 루트 노드의 링크 반환
```

마지막으로 삭제를 수행하는 과정을 살펴봅니다. 삭제 과정은 삽입 과정보다 약간 복잡합니다. 이진 탐색 트리에서 노드 한 개를 삭제해야 할 경우, 이 노드가 잎 노드이거나 자식이 하나뿐인 노드라면 삭제 과정이 간단합니다. 이때는 단순히 삭제할 노드의 부모 노드가 이 노드의 유일한 자식 노드(존재한다면)를 직접 가리키도록 하면 됩니다. 그러나 자식이 둘 다 있는 노드를 삭제하려면 문제가 복잡해집니다. 이 노드를 삭제하면 연결해야 하는 자식 노드가 두 개 존재하지만, 이들을 연결해야 하는 부모 노드의 링크는 하나밖에 없기 때문입니다.

이 문제는 삭제될 노드의 원소를 대체할 원소를 찾는 전략으로 해결할 수 있습니다. 삭제될 노드를 X라고 합시다. 노드 X의 원소를 대체할 수 있는 원소는 오른쪽 부트리의 원소 중에서 가장 작은 원소이거나 왼쪽 부트리의 원소 중에서 가장 큰 원소이면 됩니다. 오른쪽 부트리에서 가장 작은 원소를 가진 노드를 X의 **중위 후속자(inorder successor)**라고 부릅니다. X의 중위 후속자를 Y라고 합시다. Y는 중위 순회를 수행할 때 X의 다음 순서로 방문하는 노드로서, X의 오른쪽 자식 노드에서 왼쪽 자식을 계속 따라갈 때 마지막으로 만나는 노드입니다. 노드 Y는 자식이 둘 다 없거나 오른쪽 자식만 있으므로, Y의 삭제는 Y의 부모가 Y의 자식(존재한다면)을 직접 가리키게 하면 됩니다. 따라서, 노드 Y의 원소를 노드 X로 옮긴 후 Y를 삭제하면 문제가 해결됩니다. 그림 10.4는 이진 탐색 트리에서 삭제를 수행하는 예를 보여줍니다.



(a) 중위 후속자 Y의 원소 '40'을 X로 옮김 (b) 중위 후속자 Y 삭제

그림 10.4: 이진 탐색 트리에서 원소가 '35'를 삭제하는 과정

프로그램 10.4의 함수 `delete(root, item)`은 이진 탐색 트리 `root`에서 원소 `item`을 삭제합니다. 프로그램은 다음과 같이 수행합니다.

① 처음에 원소 `item`을 가진 노드를 탐색한다. 그 결과로 두 변수 X와 `parent`는 각각 `item`을 가진 노드와 이 노드의 부모를 가리킨다.

② 다음으로, 노드 X(즉, 변수 X가 가리키는 노드)에 자식이 둘 다 있으면, 이 노드의 중위 후속자를 탐색한다. 그 결과로 두 변수 Y와 `parent`는 각각 노드 X의 중위 후속자와 그 부모를 가리킨다. 그 후, 중위 후속자 Y(즉, 변수 Y가 가리키는 노드)의 원소를 노드 X로 옮긴 다음, 변수 X에 Y를 배정하여 변수 X가 실제 삭제할 노드를 가리키도록 한다.

③ 마지막으로, 노드 X를 삭제한다. 이를 위해, 변수 `son`이 노드 X의 유일한 자식(존재하지 않으면 `None`)을 가리키도록 한 다음, 노드 `son`을 노드 `parent`의 자식으로 만들어 준다.

삭제 후에 이진 탐색 트리가 공백이 되거나 루트가 바뀔 수 있으므로, 루트 노드를 반환해야 합니다. 삭제 연산은 다소 복잡해 보이지만 삽입과 마찬가지로 트리의 각 레벨에서 단지 하나 이하의 노드만 조사하므로, 수행 시간은 O(트리의 높이+1)입니다. 따라서, 최악의 경우 수행 시간은 O(n)입니다.

프로그램 10.4: 이진 탐색 트리에서 삭제를 수행하는 프로그램

```
def delete(root, item):
    """이진 탐색 트리에 item을 가진 노드를 삭제하고, 루트 노드를 반환한다."""

    # ① item을 가진 노드 X를 탐색함
    parent = None                       # 초기에 None을 부모로 설정
    X = root                            # 초기에 루트를 자식으로 설정
    while X is not None and item != X.data:     # 삭제할 노드 X 탐색
        parent = X                      # 자식 레벨로 내려옴
        if item < X.data: X = X.left    # 왼쪽 부트리 탐색
        else: X = X.right               # 오른쪽 부트리 탐색

    if X is None:                       # 삭제할 노드가 없으면 작업 종료
        return root

    # ② 자식이 둘 다 있으면 중위 후속자 Y를 탐색함
    if X.left is not None and X.right is not None:
        parent = X
        Y = X.right
        while Y.left is not None:       # 중위 후속자 탐색
            parent = Y                  # 자식 레벨로 내려옴
            Y = Y.left                  # 왼쪽 자식 배정

        X.data = Y.data                 # 중위 후속자 Y의 원소를 X로 옮김
        X = Y                           # X가 실제 삭제할 노드를 가리키게 함

    # ③ 노드 X 제거 - 부모와 자식을 찾아 연결함(자식이 있으면)
    son = X.right if X.left is None else X.left    # 노드 X의 자식 배정
    if parent is None:                  # 부모가 없으면 son이 루트가 됨
        root = son
    else:                               # son을 parent의 자식으로 설정
        if X.data < parent.data: parent.left = son
        else: parent.right = son

    return root                         # 루트 노드의 링크 반환
```

이진 탐색 트리의 기본 연산인 탐색, 삽입, 삭제는 최악의 경우 시간복잡도가 $O(n)$입니다. 이진 탐색 트리를 구축할 때 원소의 값이 무작위로 들어오는 경우, 확률적으로 분석하면 트리의 기대 높이(expected height)가 $O(\log n)$으로 알려져 있습니다. 그러므로 탐색, 삽입, 삭제 연산의 기대 시간복잡도는 $O(\log n)$입니다. 기대 시간복잡도를 고려하면 이진 탐색 트리는 동적 데이터를 위한 유용한 자료구조로 사용될 수 있습니다. 그러나 최악의 경우에도 시간복잡도가 $O(\log n)$이 되도록 하려면 이진 탐색 트리의 높이를 항상 $O(\log n)$으로 유지할 수 있어야 합니다. 이러한 탐색 트리를 **높이 균형 탐색 트리(height-balanced search tree)**라고 부릅니다. 높이 균형 탐색 트리는 13장에서 소개합니다.

10.2 힙 자료구조

일반적인 큐는 먼저 삽입된 원소가 먼저 삭제되는 선입선출(FIFO) 구조이지만, **우선순위 큐(priority queue)**는 삽입된 순서와는 관계없이 우선순위가 가장 높은 원소부터 삭제되는 구조입니다. 우선순위 큐를 표현하기 위해 선형리스트를 사용한다면 어떨까요? 이 경우 우선순위에 따라 원소들을 정렬된 순서로 선형리스트에 저장하는 것을 고려해 볼 수 있습니다. 이렇게 하면 삭제는 선형리스트의 맨 앞이나 맨 끝의 고정된 위치에서 수행되므로 $O(1)$ 시간에 수행됩니다. 반면에 삽입을 수행하려면 먼저 삽입될 위치부터 찾아야 합니다. 그런데 길이가 n인 선형리스트의 연결리스트 표현은 삽입될 위치를 찾는 데 $O(n)$ 시간이 걸리고, 배열 표현은 삽입을 위한 원소들의 이동 작업을 수행하는 데 $O(n)$ 시간이 걸립니다. 따라서 어떤 표현이라도 삽입은 $O(n)$ 시간이 소요됩니다. 만약 우선순위와 상관없이 원소들을 선형리스트에 저장한다면, 삽입은 $O(1)$ 시간에 수행되지만, 삭제는 삭제될 원소를 찾기 위해 순차적 탐색을 수행해야 하므로 $O(n)$ 시간이 소요됩니다. 삽입과 삭제 두 연산 중 하나라도 $O(n)$ 시간에 수행된다면 효율적인 구현이라고 말할 수 없습니다.

우선순위 큐에 적합한 표현은 힙을 사용하는 것입니다. 앞으로 설명할 힙 자료구조를 이용하면 삽입 연산과 삭제 연산 모두 $O(\log n)$ 시간에 수행되므로, 선형리스트 표현에 비해 매우 효율적입니다. **힙(heap)**의 정의는 다음과 같습니다.

1. 완전 이진 트리이다.
2. 부모 노드의 우선순위는 자식 노드보다 높거나 같다. 즉, 모든 노드의 원소는 자기 자식의 원소보다 우선순위가 높거나 같다.

힙의 정의에 따르면 우선순위가 가장 높은 원소는 항상 트리의 루트에 위치합니다. 우선순위를 숫자로 표현할 때, **최대 힙(max heap)**은 부모 노드의 우선순위 값이 자식 노드보다 크거나 같은 힙을 말하며, **최소 힙(min heap)**은 부모 노드의 우선순위 값이 자식 노드보다 작거나 같은 힙을 말합니다.

힙은 완전 이진 트리이므로, 배열로 표현하는 것이 효율적입니다. 그림 10.5는 최대 힙과 배열 표현 예를 보여줍니다. 우선순위 값이 가장 큰 원소는 트리의 루트에 있으므로, 배열에서 첨자 0에 배정된 원소의 우선순위 값이 가장 큽니다.

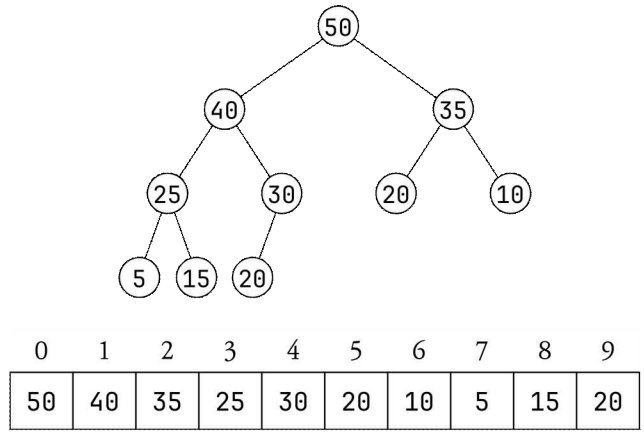


그림 10.5: 최대 힙과 배열 표현 예

이제 힙에서의 삽입 연산과 삭제 연산의 수행 과정을 살펴보겠습니다. 먼저, 크기가 n인 힙에 새로운 원소 item을 삽입한다고 가정해 봅시다. 삽입 연산이 수행되면, 완전 이진 트리에 새 원소를 위한 노드가 추가되어야 합니다. 이때, 완전 이진 트리의 구조상, 노드가 추가될 수 있는 위치는 이진 트리의 가장 아래 레벨에 놓인 가장 오른쪽 노드의 바로 옆입니다. 만일 그 레벨에 노드들이 꽉 차 있으면, 가장 왼쪽 노드의 왼쪽 자식 위치입니다. 배열 표현에서는 마지막 원소의 다음 위치, 즉 첨자 n인 장소입니다. 삽입 과정은 배열의 n번째 위치에서부터 시작하여 해당 노드의 조상 노드들을 아래에서부터 차례대로 조사하면서 부모 노드가 item보다 큰 원소를 가진 노드 X를 찾습니다. 노드 X가 발견될 때까지 탐색 과정에서 만나는 노드의 원소를 자식 노드로 옮깁니다. 노드 X가 발견되면 X에 item을 저장하고, 그렇지 않으면 루트 노드에 item을 저장합니다. 이 과정을 **상향식 힙 조정(heapify up)**이라고 합니다. 그림 10.6과 그림 10.7은 힙에서 삽입을 수행하는 상향식 힙 조정의 예를 보여줍니다.

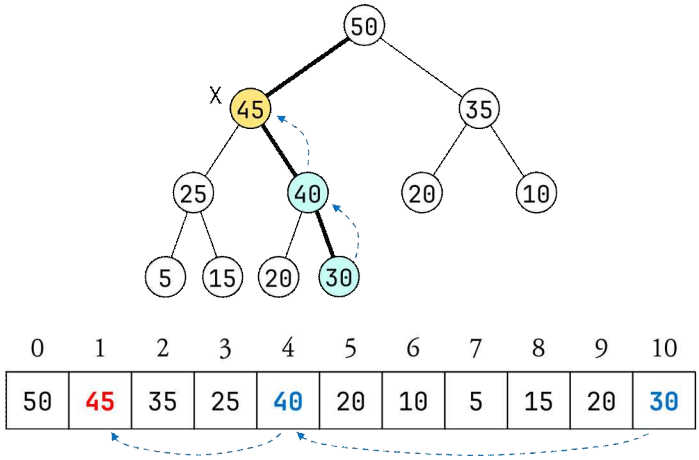


그림 10.6: 그림 10.5의 힙에 원소 '45'의 삽입

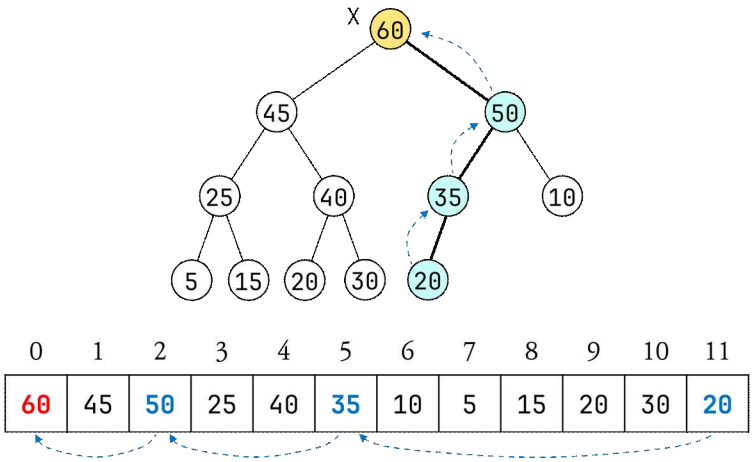


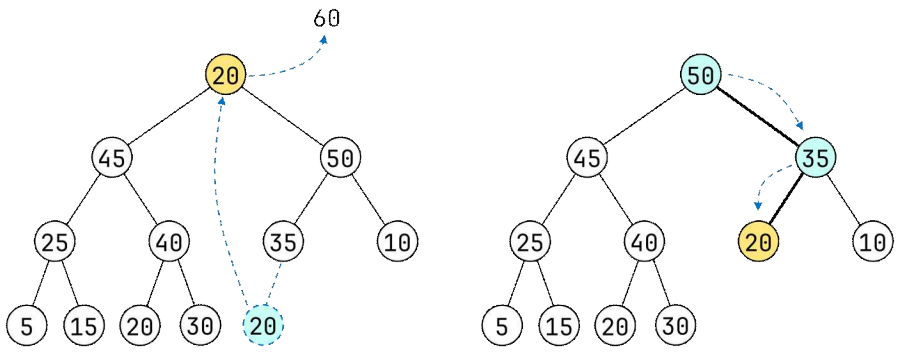
그림 10.7: 그림 10.6의 힙에 원소 '60'의 삽입

프로그램 10.5는 힙에서의 삽입 과정을 구현한 프로그램입니다. 함수 `insert_heap(H, item)`은 힙 정보를 가진 파이썬 리스트 `H`에 새로운 원소 `item`을 삽입하기 위해, `item`을 리스트 `H`의 맨 끝에 추가한 후 상향식 힙 조정을 수행합니다. 이 함수에서 `while` 문의 최대 반복 횟수는 힙의 높이입니다. 노드 수가 n인 완전 이진 트리의 높이는 $\lfloor \log n \rfloor$이므로, 이 함수의 수행 시간은 $O(\log n)$입니다. 시간복잡도를 엄밀하게 분석하면, 파이썬 리스트의 `append()` 메소드를 사용하기 때문에, 상각 시간복잡도로서 $O(\log n)$입니다.

프로그램 10.5: 힙에서 삽입을 수행하는 프로그램

```
1  def insert_heap(H, item):
2     """힙 H에 새 원소 item을 삽입한다."""
3     H.append(item)              # 새 원소를 리스트 맨 끝에 삽입
4     i = len(H)-1                # i는 새 원소가 삽입된 위치(첨자)
5     while i > 0:                # i가 루트가 아니면 반복함
6        dad_i = (i-1) // 2       # dad_i는 i의 부모 위치
7        if item <= H[dad_i]:     # 우선순위가 같거나 더 큰 부모를 만나면
8           break                 # while 문을 빠져나감
9        H[i] = H[dad_i]          # 부모 원소를 자식 위치로 옮김
10       i = dad_i                # 부모 레벨로 이동
11    H[i] = item                 # 새 원소 저장
```

다음으로 힙에서 삭제를 수행하는 과정을 소개합니다. 힙에서 삭제는 루트 노드에 있는 우선순위가 가장 높은 원소를 삭제하는 것입니다. 그런데, 루트 노드를 직접 트리로부터 삭제하기보다는 마지막 노드의 원소를 일단 루트로 옮긴 후에 그 노드를 삭제하는 것이 간편합니다. 이 경우, 힙의 조건을 만족시키기 위해 루트 노드에 옮겨진 원소가 자식 원소보다 크거나 자식이 없으면 바로 종료하고, 그렇지 않으면 자식 중 큰 원소와 위치를 서로 바꾼 후 자식 노드로 내려가서 동일한 작업을 부트리에 대해 수행합니다. 이 과정을 **하향식 힙 조정(heapify down)**이라고 합니다. 그림 10.8은 그림 10.7의 힙에서 삭제를 수행하는 하향식 힙 조정의 예를 보여줍니다.



(a) 루트 원소와 마지막 노드의 삭제 (b) 힙 조정

그림 10.8: 그림 10.7의 힙에 대한 삭제 과정

프로그램 10.6은 삭제를 수행하는 프로그램입니다. 함수 `delete_heap(H)`은 힙 H에서 우선순위가 가장 높은 루트 원소를 삭제하여 반환하고, 루트 노드에 힙의 마지막 원소를 배치한 후

하향식 힙 조정을 수행합니다. 힙이 공백인 상태에서 호출되면 None을 반환합니다. 이 함수에서 while 문의 최대 반복 횟수는 힙의 높이입니다. 따라서 삭제 과정도 $O(\log n)$ 시간에 수행됩니다.

프로그램 10.6: 힙에서 삭제를 수행하는 프로그램

```
1  def delete_heap(H):
2     """힙 H에서 루트 원소를 삭제한 후 반환한다."""
3     if len(H) == 0:              # 공백 상태에서 호출되면 None을 반환
4        return None
5     if len(H) == 1:              # 원소가 하나만 있는 경우, 원소 삭제 후 반환
6        return H.pop()           # 힙 조정이 필요 없음
7
8     del_item = H[0]              # 삭제 후 반환할 원소
9     item = H.pop()               # 루트로 옮겨질 리스트의 맨 끝 원소
10    i = 0                        # item을 저장할 위치
11    son_i = 1                    # i의 왼쪽 자식 위치
12    while son_i < len(H):        # i가 잎 노드 위치가 아니면 반복함
13       if son_i+1 < len(H) and H[son_i] < H[son_i+1]:
14          son_i += 1            # son_i에 우선순위가 더 큰 자식 위치 배정
15       if item >= H[son_i]:     # 자식보다 우선순위가 크거나 같으면 반복 중단
16          break
17       H[i] = H[son_i]          # 자식 원소를 부모 위치로 옮김
18       i = son_i                # 자식 레벨로 내려감
19       son_i = 2*i + 1          # son_i를 i의 왼쪽 자식 위치로 갱신
20
21    H[i] = item                 # i 위치에 item 저장
22    return del_item             # 삭제된 원소 반환
```

배열에 저장된 n개의 원소를 가지고 힙을 구축하는 것을 **힙 초기화**라고 부릅니다. 힙을 초기화하는 방법에는 두 가지가 있습니다. 그중의 하나는 공백 힙에서 시작하여 n개의 원소를 하나씩 차례대로 힙에 삽입하는 것입니다. 크기가 k인 힙에서 삽입을 수행할 때 $O(\log k)$ 시간이 걸립니다. 이 방법을 사용하여 힙을 초기화하는 데 걸리는 시간은 다음의 식에 의해 $O(n\log n)$ 입니다.

$$\log 1 + \log 2 + \cdots + \log n = \log(1\times 2\times\cdots\times n) = \log(n!) = \Theta(n\log n)$$

힙을 초기화하는 다른 방법은 일단 원소들로 완전 이진 트리를 구축한 후에 원소의 위치를 조정하여 힙의 성질을 만족시키는 것입니다. 이를 위해, 완전 이진 트리에서 마지막 잎 노드의 부모(즉, 마지막 내부 노드)에서부터 시작하여 루트까지 거슬러 올라가면서 각 노드를 루트로 하는 부트리를 힙으로 만들어 나갑니다. 이 과정에서 조정 대상인 각 부트리는 루트를 제외하면 이미

힙의 성질을 만족하고 있으므로, 해당 부트리의 루트 원소만 적절한 위치로 이동시키면 됩니다. 이 작업은 힙의 삭제 과정에서 루트 원소를 이동시키는 과정과 같은 방식으로 수행됩니다. 그림 10.9는 초기 완전 이진 트리에서 시작하여 점진적으로 힙을 만드는 과정을 보여줍니다.

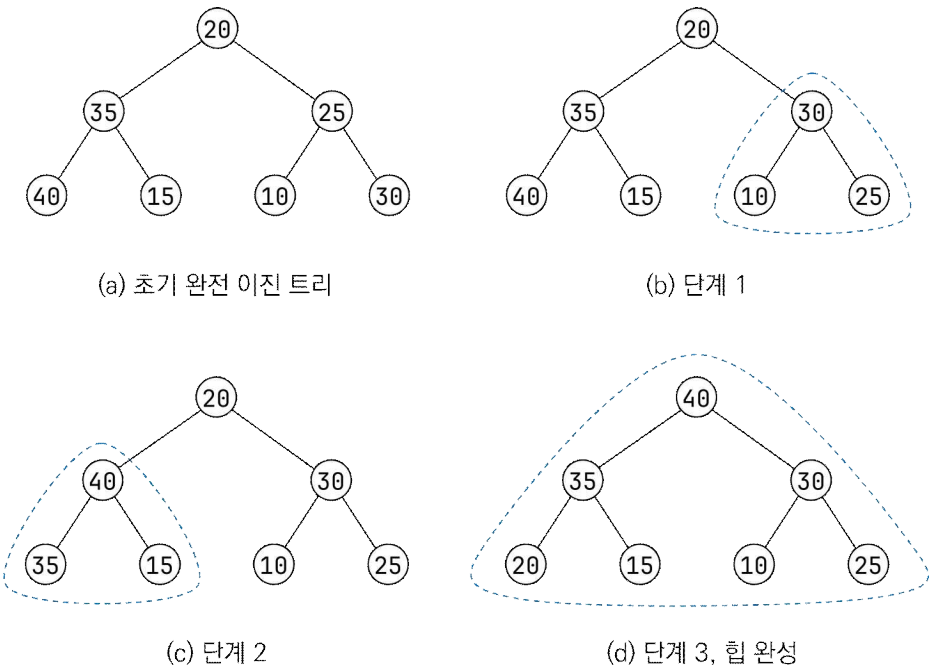


그림 10.9: 힙 초기화를 수행하는 과정의 예

이 방식으로 힙을 초기화하는 데 걸리는 시간을 살펴보겠습니다. 프로그램 10.6의 힙 삭제 알고리즘을 이용하면, 높이가 h인 부트리를 힙으로 조정하기 위해 수행하는 `while` 문의 반복 횟수는 최대 h입니다. 주어진 완전 이진 트리의 높이를 k라고 하면(즉, $2^k \le n < 2^{k+1}$), 레벨 `i`에 있는 노드를 루트로 하는 부트리의 높이는 `k-i`입니다. 레벨 `i`에 있는 노드의 최대 개수는 2^i이므로, 힙을 구축하는 데 필요한 `while` 문의 총 반복 횟수는 다음과 같습니다.[22)]

$$\sum_{i=0}^{k} 2^i(k-i) \;=\; \sum_{j=1}^{k} 2^{k-j} j \;=\; \sum_{j=1}^{k} \frac{2^k}{2^j} j \;\le\; n\sum_{j=1}^{k} \frac{j}{2^j} \;< 2n$$

22) 식의 마지막 부등식에 관한 내용은 아래 웹사이트를 참고하세요.
"https://math.stackexchange.com/questions/337937/why-sum-k-1-infty-frack2k-2?noredirect=1&lq=1"

따라서 이 방식으로 힙을 구축하는 데는 $O(n)$ 시간이 소요됩니다.

프로그램 10.7에는 힙 초기화를 수행하는 프로그램이 나와 있습니다. 힙의 삭제 과정과 힙 초기화 과정에서 공통으로 수행하는 하향식 힙 조정 작업은 함수 _fix_heap(H, root)에서 수행합니다. 이 함수는 힙 H에서 root를 루트로 하는 부트리를 힙으로 조정합니다. 이때 루트를 제외한 나머지 노드들은 힙의 성질을 만족하는 상태입니다. 함수 delete_heap(H)는 _fix_heap(H, 0)을 호출하여 힙에서 삭제를 수행하고 삭제된 원소를 반환합니다. 힙이 공백인 상태에서 delete_heap(H)이 호출되면 None을 반환합니다. 함수 init_heap(H)는 리스트 H에 저장된 요소들로 초기 힙을 구축해 줍니다. 구축 과정에서 H[i]가 루트인 부트리를 힙으로 조정하기 위해 _fixHeap(H, i)를 호출합니다. 함수 _fix_heap()의 수행 시간은 $O(\log n)$이므로, delete_heap()의 수행 시간도 $O(\log n)$입니다. 함수 init_heap()의 수행 시간은 앞서 분석한 대로 $O(n)$입니다.

프로그램 10.7: 힙 초기화와 삭제를 수행하는 프로그램

```
1  def _fix_heap(H, root):          # root 위치의 노드에 대해 하향식 힙 조정
2     item = H[root]                # 옮겨질 원소
3     i = root                      # i는 item을 저장할 위치
4     son_i = 2*i + 1               # son_i는 i의 왼쪽 자식 위치
5     while son_i < len(H):         # i가 잎 노드 위치가 아니면 반복함
6        if son_i+1 < len(H) and H[son_i] < H[son_i+1]:
7           son_i += 1             # son_i에 우선순위가 더 큰 자식 위치 배정
8        if item >= H[son_i]:       # 우선순위가 크거나 같으면 반복 중단
9           break
10       H[i] = H[son_i]            # 자식 원소를 부모 위치로 옮김
11       i = son_i                  # 자식 레벨로 내려감
12       son_i = 2*i + 1            # son_i를 i의 왼쪽 자식 위치로 갱신
13    H[i] = item                   # i 위치에 item 저장
14
15 def delete_heap(H):
16    """힙 H에서 루트 원소를 삭제한 후 반환한다."""
17    if len(H) == 0:               # 공백 상태에서 호출되면 None을 반환
18       return None
19    if len(H) == 1:               # 원소가 하나만 있는 경우, 원소 삭제 후 반환
20       return H.pop()             # 힙 조정이 필요 없음
21    del_item = H[0]               # 삭제 후 반환할 요소
22    H[0] = H.pop()                # 마지막 요소를 삭제하고 루트로 옮김
23    _fix_heap(H, 0)               # 루트 노드에 대해 힙 조정
24    return del_item               # 삭제된 원소 반환
25
```

```
26 def init_heap(H):
27    """리스트 H를 힙으로 초기화한다."""
28    for i in range((len(H)-1)//2, -1, -1):  # H[i]가 루트인 부트리를
29       _fix_heap(H, i)                      # 힙으로 조정
```

힙 자료구조를 이용하면 배열에 저장된 원소들을 정렬할 수 있습니다. 배열 원소들로 초기 힙을 구축한 후에 힙 삭제를 계속해서 수행하여, 힙에서 빠져나오는 원소들을 차례대로 저장하면 정렬된 상태로 얻을 수 있습니다. 이 정렬 방식을 **힙 정렬(heap sort)**이라고 부릅니다. 힙 초기화는 $O(n)$ 시간에 수행되고, 힙 삭제는 $O(\log n)$ 시간에 수행되므로, 정렬에 필요한 총 수행 시간은 $O(n\log n)$입니다. 그러므로, 힙 정렬도 합병 정렬과 마찬가지로 최악의 경우에도 $O(n\log n)$ 시간에 정렬하는 최적 정렬 알고리즘입니다. 정렬 알고리즘의 특성을 고려할 때, 힙 정렬은 불안정 정렬에 해당하지만, 배열 내부에서 정렬을 수행할 수 있어 제자리 정렬에 해당합니다. 힙 정렬을 제자리 정렬 방식으로 구현하려면, 배열에서 힙 영역과 정렬된 영역에 해당하는 구간을 구분해야 합니다. 힙에서 삭제된 원소들을 배열의 끝부분부터 차례로 저장해 나가면, 결국 배열은 정렬된 상태가 됩니다. 프로그램 10.8에는 힙 정렬을 제자리 정렬 방식으로 구현한 함수 heap_sort(S)가 나와 있습니다. 이 함수는 먼저 리스트 S를 힙으로 초기화한 다음, 힙의 루트 원소를 힙의 마지막 원소와 교환한 후에 힙을 재조정하는 작업을 계속해서 수행합니다. 이때, 프로그램 10.7의 _fix_heap(H, root) 대신에 _heapify(H, n, root)를 호출합니다. 이 함수는 리스트 H의 첨자 범위 0부터 n-1까지를 힙 영역으로 사용합니다.

프로그램 10.8: 제자리 정렬 방식으로 동작하는 힙 정렬

```
1  def _heapify(H, n, root):     # root 위치 노드에 대해 힙 조정, n은 힙의 크기
2     item = H[root]               # 옮겨질 원소
3     i = root                     # i는 item을 저장할 위치
4     son_i = 2*i + 1              # son_i는 i의 왼쪽 자식 위치
5     while son_i < n:             # i가 잎 노드 위치가 아니면 반복함
6        if son_i+1 < n and H[son_i] < H[son_i+1]:
7           son_i += 1             # son_i에 우선순위가 더 큰 자식 위치 배정
8        if item >= H[son_i]:       # 우선순위가 크거나 같으면 반복 중단
9           break
10       H[i] = H[son_i]            # 자식 원소를 부모 위치로 옮김
11       i = son_i                  # 자식 레벨로 내려감
12       son_i = 2*i + 1            # son_i를 i의 왼쪽 자식 위치로 갱신
13    H[i] = item                   # i 위치에 item 저장
14
15 def heap_sort(S):
16    """힙 정렬로 리스트 S를 정렬한다."""
```

```
17    for i in range((len(S)-1)//2, -1, -1):  # 힙 초기화
18       _heapify(S, len(S), i)
19
20    for i in range(len(S)-1, 0, -1):    # 힙의 크기가 1이 될 때까지 수행
21       S[i], S[0] = S[0], S[i]          # 맨 앞 원소와 맨 끝 원소를 교환
22       _heapify(S, i, 0)                # 힙 조정
```

힙 정렬의 장점은 우선순위가 높은 순서대로 k개의 원소를 선택하고자 할 때 나타납니다. 이 작업을 위해 합병 정렬이나 퀵 정렬을 사용하면 리스트 전체를 정렬해야 하므로 $O(n\log n)$ 시간이 소요되고, 순차 탐색을 이용하면 k번의 탐색을 수행하므로 $O(kn)$ 시간이 소요됩니다. 힙 정렬의 경우는, 프로그램 10.8의 함수 `heap_sort()`에서 힙 초기화 이후 `for` 문의 반복 횟수를 k로 변경시키기만 하면 됩니다. 따라서 이 작업을 위한 수행 시간은 $O(n+k\log n)$으로, k의 값에 따라서는 다른 정렬 알고리즘이나 순차 탐색을 이용하는 것보다 효율적입니다.

파이썬은 힙 자료구조를 위한 `heapq` 모듈을 제공합니다. `heapq` 모듈은 파이썬 리스트를 **최소 힙**으로 구성해 줍니다. `heapq` 모듈을 통해 제공되는 주요 함수는 표 10.1에 나와 있습니다.

표 10.1: `heapq` 모듈의 주요 함수와 연산

함수/연산	기능
`heapify(H)`	리스트 H를 최소 힙으로 변환함 (선형 시간) (`from heapq import *` 선언 필요)
`heappush(H, item)`	힙 H에 원소 item을 삽입함 (로그 시간)
`heappop(H)`	힙 H에서 가장 작은 요소를 삭제하고 반환함 (로그 시간) 힙이 공백이면 `IndexError` 발생
`H[0]`	힙 H에서 가장 작은 요소를 반환함 (상수 시간)
`heappushpop(H, item)`	힙 H에 item을 삽입한 후에 가장 작은 요소를 삭제하고 반환함 (로그 시간)
`heapreplace(H, item)`	힙 H에서 가장 작은 요소를 삭제한 후에 item을 삽입함. 삭제된 요소는 반환됨 (로그 시간)

프로그램 10.9에는 `heapq` 모듈을 이용하여 힙 정렬을 수행하는 함수 `heapsort(S)`가 나와 있습니다. 이 함수는 힙과 정렬된 리스트를 분리해서 유지해야 하므로, 제자리 정렬 방식은 아닙니다.

프로그램 10.9: heapq 모듈을 이용한 힙 정렬

```
from heapq import *

def heapsort(S):
    """heapq 모듈을 이용한 힙 정렬로 리스트 S를 정렬한다."""
    heapify(S)                                  # 힙 초기화
    S[:] = [heappop(S) for _ in range(len(S))]  # 정렬 작업
```

heapq 모듈은 sorted() 함수의 key 인수처럼 비교 기준을 사용자가 직접 지정해 주는 기능을 제공하지 않습니다. 따라서 비교 기준을 다르게 정하려면 리스트의 요소를 직접 변형해야 합니다. 프로그램 10.10에는 튜플을 이용하여 heapq를 최대 힙처럼 사용하는 예와 문자열의 길이를 기준으로 최소 힙을 구성하는 예가 나와 있습니다.

프로그램 10.10: heapq 모듈에서 비교 기준을 변경하는 예

```
from heapq import *

# 최대 힙 구성
L = [3, 6, 2, 1, 7, 4]
H = [(-x, x) for x in L]       # L의 요소 x를 튜플 (-x, x)로 변환
heapify(H)                     # 힙 초기화
while len(H) > 0:              # 힙 H가 공백이 될 때까지 반복
    print(heappop(H)[1])       # 7, 6, 4, 3, 2, 1 순으로 출력

# 문자열의 길이를 기준으로 최소 힙 구성
L = ["apple", "banana", "cherry", "kiwi"]
H = [(len(x), x) for x in L]   # L의 요소 x를 튜플 (len(x), x)로 변환
heapify(H)                     # 힙 초기화
while len(H) > 0:              # 힙 H가 공백이 될 때까지 반복
    print(heappop(H)[1])       # kiwi, apple, banana, cherry 순으로 출력
```

10.3 분리된 집합을 위한 트리 표현

이 절에서는 트리를 이용하여 집합을 표현하는 방법과 분리된 집합들의 모임에 대해서 union 연산과 find 연산을 효율적으로 수행하는 방법을 살펴봅니다. 여기서 다루는 집합들은 다음과 같은 단순한 특징이 있지만, 이러한 집합을 다루는 자료구조는 다양한 응용에서 매우 유용하게 활용될 수 있습니다.

1. n개의 원소를 가진 전체 집합이 정의되고, 원소들은 0부터 n-1까지의 정수로 구분된다.
2. 집합들은 서로 분리되어 있다. 즉 어떤 두 집합의 교집합은 항상 공집합이다.
3. 모든 집합의 합집합은 전체 집합이다.
4. 집합들은 이름이 아니라 임의로 정해지는 **대표 원소**에 의해 구분된다.
5. 이들 집합에 대해 두 가지 연산인 union과 find만 수행된다. union(i, j)는 대표 원소가 i와 j인 두 집합에 대해 합집합을 수행하는 연산이고, find(i)는 i가 속한 집합의 대표 원소를 알려주는 연산이다.

이러한 집합의 모임에 대해, 각 집합을 트리로 표현합니다. 예를 들어, 전체 집합이 {0, 1, 2, … , 9}인 세 집합 {0, 3, 8}, {1, 2, 4, 5, 6}, {7, 9}는 그림 10.10처럼 세 개의 트리로 표현될 수 있습니다.

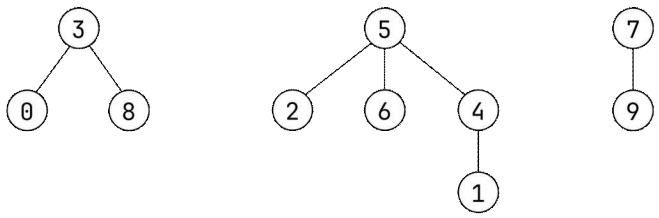


그림 10.10: 분리된 집합의 트리 표현

트리로 표현된 각 집합의 대표 원소는 해당 트리의 루트 원소입니다. 따라서 위의 세 집합을 대표 원소가 각각 3인 집합, 5인 집합, 7인 집합으로 구분합니다. 위의 예에 대해, find(0), find(1), find(7)의 값은 각각 3, 5, 7로 정해집니다. 집합을 트리로 표현하면 union 연산이 매우 효율적으로 수행될 수 있습니다. 위의 예에서 union(3, 5)는 루트가 3인 트리와 5인 트리를 하나로 합치는 연산에 해당하는데, 단순히 5를 3의 부모로 설정하면 됩니다(그림 10.11 참조).

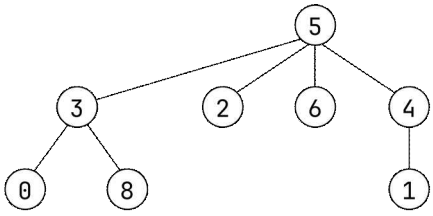


그림 10.11: union(3, 5)의 수행

union 연산과 find 연산은 부모에 대한 정보만 필요하므로, 트리를 부모 포인터 배열로 표현하는 것이 효과적입니다. 부모 포인터 배열을 사용하면 모든 트리를 하나의 배열로 표현할 수 있습니다. 그림 10.12는 그림 10.10의 트리들을 부모 포인터 배열로 나타낸 것입니다.

	0	1	2	3	4	5	6	7	8	9
parent	3	4	5	-1	5	-1	5	-1	3	7

그림 10.12: 부모 포인터 배열 표현

부모 포인터 배열에서 사용하는 배열 이름을 parent라고 하면, union(i, j)와 find(i)를 다음과 같이 간단히 구현할 수 있습니다. 이 두 함수 중에서 union()은 상수 시간에 수행되고, find(i)는 원소 i의 레벨에 비례하는 수행 시간을 가집니다.

```
def union(i, j):
   parent[i] = j

def find(i):
   while parent[i] >= 0:
      i = parent[i]
   return i
```

union-find 자료구조를 사용하는 응용에서는, 보통 초기 상태로서 {i}로 구성된 n개의 집합이 주어집니다(0≤i≤n-1). 그리고 알고리즘의 수행 과정에서 union 연산과 find 연산이 반복적으로 호출되며, 일반적으로 $O(n)$ 번의 union 연산과 $O(n)$ 번의 find 연산이 호출되는 상황에서 전체 수행 시간을 분석하게 됩니다. 예를 들어, 각각 n번의 union()과 find()가 다음과 같은 순서로 수행된다고 가정해 봅시다.

union(0, 1), find(0), union(1, 2), find(0), … , union(n-2, n-1), find(0)

그러면 매번 union()이 수행될 때마다 트리의 높이가 하나씩 높아져서 마지막 union(n-2, n-1)의 수행 후에는 높이가 n-1인 트리가 됩니다(그림 10.13 참조).

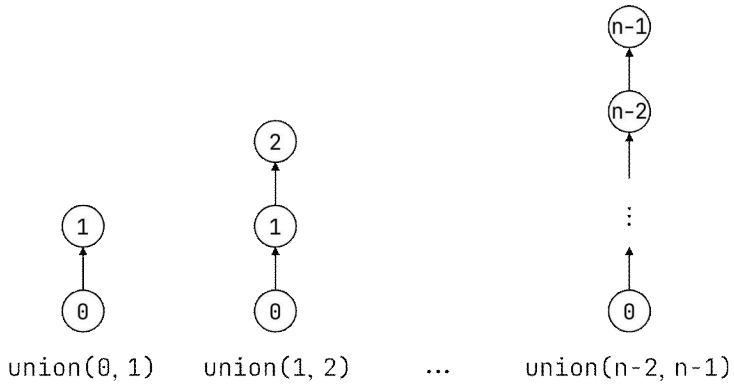


그림 10.13: union() 함수의 수행 후 트리의 형태

union()은 O(1) 시간에 수행되므로 union()의 총 수행 시간은 O(n)입니다. find()의 수행 시간은 for 문의 총 반복 횟수에 비례하며, 노드 0의 레벨이 점점 커지기 때문에 find(0)의 총 반복 횟수는 다음과 같습니다.

$$1 + 2 + \cdots + n-1 = n(n-1)/2$$

따라서 find()의 총 수행 시간은 $O(n^2)$입니다. 이러한 수행 순서는 union()과 find()가 반복적으로 수행되는 상황에서 발생할 수 있는 최악의 경우에 해당합니다. 그러므로 각각 O(n)번씩 union()과 find()가 수행되는 상황에서 최악의 경우 시간복잡도는 $O(n^2)$입니다.

이제, union 연산과 find 연산의 성능을 개선하는 방법을 살펴보겠습니다. 집합을 나타내는 트리의 높이가 성능에 직접적인 영향을 주므로, 트리의 높이를 최소화하는 것이 중요합니다. 이를 위해 union 연산과 find 연산에서 높이를 줄이는 전략을 도입합니다. 먼저, union 연산의 개선 방법을 살펴보겠습니다. union 연산은 두 개의 트리를 하나로 합병합니다. 두 트리 T_1과 T_2의 높이가 각각 h_1과 h_2이고 $h_1 > h_2$라고 가정합시다. 두 트리가 합병될 때, T_1의 루트가 합병된 트리의 루트가 되면 합병된 트리의 높이는 h_1이 되고, T_2의 루트가 합병된 트리의 루트가 되면 합병된 트리의 높이는 $h_1 + 1$이 됩니다. 따라서 두 트리를 합칠 때는 높이가 더 큰 트리의 루트가 새로운 트리의 루트가 되도록 구성하는 것이 유리합니다. 하지만 union 연산 과정에서 현재 트리의 높이를 빠르게 알아내기는 어렵습니다. 대신, 노드 수가 많은 트리는 그 높이가 클 가능성이 높으므로, 두 트리를 합칠 때 노드 수가 많은 트리의 루트가 합병된 트

리의 루트가 되도록 구성하면 비슷하게 효율적인 결과를 얻을 수 있습니다. 이렇게 수행하는 union 연산을 **가중치 규칙(weighting rule)**을 적용한 union 연산이라고 부릅니다. 트리의 노드 수를 빠르게 알아내기 위해서는 트리를 표현하는 부모 포인터 배열을 약간 수정하면 됩니다. 배열 parent에서 루트 원소 i에 대한 parent[i]의 값은 -1이지만, 수정된 표현에서는 루트 원소 i에 대해 parent[i]에 '-(트리의 노드 수)'를 배정하도록 합니다. 따라서 음수 값으로 루트임을 나타내고, 그 절댓값으로 노드 개수를 알 수 있습니다. 다음은 가중치 규칙을 적용한 union() 함수입니다.

```
def union(i, j):
   temp = parent[i] + parent[j]     # -(노드 수의 합)
   if parent[i] > parent[j]:        # 트리 j의 노드가 많음
      parent[i] = j                 # j가 i의 부모가 됨
      parent[j] = temp
   else:                            # 트리 i의 노드가 많음
      parent[j] = i                 # i가 j의 부모가 됨
      parent[i] = temp
```

개선된 union()의 수행 시간도 역시 $O(1)$입니다. find 연산은 그대로 사용하고 union 연산만 개선된 것을 사용하더라도, 각각 $O(n)$ 번씩 union()과 find()가 수행되는 상황에서 발생하는 최악의 경우 시간복잡도는 $O(n\log n)$으로 개선됩니다. 이는 가중치 규칙을 적용한 union 연산의 결과로 생성되는 n개의 노드를 가진 트리의 높이가 항상 $\lfloor \log n \rfloor + 1$ 이하로 유지되기 때문입니다(증명은 연습문제로 돌림).

다음으로 find 연산의 개선 전략을 소개합니다. 이 전략은 find(i)를 수행할 때마다 노드 i의 조상들을 모두 루트의 자식으로 직접 연결해 주는 것입니다. 그러면 이후의 이 노드의 조상에 대한 find 연산은 상수 시간에 수행될 수 있습니다. 이렇게 개선된 find 연산을 **경로 압축 규칙(path collapsing rule)**을 적용한 find 연산이라고 부릅니다. 다음은 경로 압축 규칙을 적용한 find() 함수입니다.

```
def find(i):
   root = i
   while parent[root] >= 0:   # 루트 탐색
      root = parent[root]
   while i != root:           # i의 조상을 root의 자식으로 둠
      parent[i] = root
      i = parent[i]
   return root
```

개선된 find(i)도 원소 i의 레벨에 비례하는 수행 시간을 가집니다. 가중치 규칙을 적용한 union 연산과 경로 압축 규칙을 적용한 find 연산을 함께 사용하면, union()과 find()의 반복적인 수행에서 발생하는 최악의 경우 시간복잡도가 $O(n\log^* n)$으로 개선됩니다. 여기서 $\log^* n$은 n에다 계속 log를 취할 때, 값이 1보다 작아지거나 같아질 될 때까지 취한 log의 횟수를 의미합니다. 예를 들어, log16 = 4, log4 = 2, log2 = 1이므로 $\log^* 16 = 3$입니다. 또한 log(65536) = 16이므로 $\log^*(65536) = 4$이고, $\log^*(2^{56636}) = 5$입니다. 따라서 $\log^* n$은 상수로 취급해도 무방할 정도로 매우 느리게 증가하는 함수입니다. 좀 더 엄밀하게 분석한 결과로, m번의 union 연산과 n≥m번의 find 연산이 수행될 때, 최악의 경우 시간복잡도가 $\Theta(n\alpha(n, m))$이라는 연구 결과가 나와 있습니다. 함수 $\alpha(n, m)$은 **아커만 함수(Ackermann's function)**[23]와 역함수 관계에 있는 함수로서 $\log^* n$보다도 더 느리게 증가하는 함수입니다.

지금까지 소개한 분리된 집합 모임을 위한 union과 find 연산을 편리하게 활용할 수 있도록 클래스 UFSet을 정의합니다. 프로그램 10.11에 클래스 UFSet을 구현한 프로그램이 나와 있습니다.

프로그램 10.11: UFSet 클래스

```
1  class UFSet:
2     """분리된 집합 모임을 위한 union-find 클래스"""
3
4     def __init__(self, n):
5        self.parent = [-1]*n    # 집합 모임 ({0}, {1}, ... , {n-1}) 생성
6
7     def union(self, i, j):     # i와 j를 대표 원소로 하는 집합의 합집합
8        temp = self.parent[i] + self.parent[j]  # -(노드 수의 합)
9        if self.parent[i] > self.parent[j]:     # 트리 j의 노드가 많음
10          self.parent[i] = j                   # j가 i의 부모가 됨
11          self.parent[j] = temp
12       else:                                   # 트리 i의 노드가 많음
13          self.parent[j] = i                   # i가 j의 부모가 됨
14          self.parent[i] = temp
15
16    def find(self, i):         # i가 포함된 집합의 대표 원소 반환
17       root = i
18       while self.parent[root] >= 0:   # 루트 탐색
19          root = self.parent[root]
20       while i != root:                # i의 조상을 root의 자식으로 둠
21          next_item = self.parent[i]
```

23) 참고: https://en.wikipedia.org/wiki/Ackermann_function

```
22          self.parent[i] = root
23          i = next_item
24        return root
25
26    def root(self, i):          # i가 대표 원소이면 True, 아니면 False 반환
27        return self.parent[i] < 0
```

분리된 집합 모임을 위한 union-find 자료구조는 주어진 집합의 원소들을 **동치 관계(equivalence relation)**에 따라 **동치 클래스(equivalence class)**로 나누는 문제를 효율적으로 해결할 수 있습니다. 동치 관계는 **반사적(reflexive)**, **대칭적(symmetric)**, **이행적(transitive)** 성질을 가진 관계를 말합니다. 즉, 집합 S에 대해 어떤 관계를 기호 '≡'로 표시할 때, S의 원소에 대해 다음의 세 성질이 성립하면 이 관계를 동치 관계라고 부릅니다.

1. 반사적 성질 : a ≡ a
2. 대칭적 성질 : a ≡ b이면 b ≡ a
3. 이행적 성질 : a ≡ b이고 b ≡ c이면 a ≡ c

동치 관계는 집합 S를 동치 클래스들로 나눕니다. S의 두 원소 a와 b에 대해 a ≡ b이면 a와 b는 같은 동치 클래스에 속하며, 그 역도 성립합니다. 예를 들어, 집합 S = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}에 대해 동치 관계에 있는 원소의 쌍들이 다음과 같이 주어지면,

0 ≡ 3, 1 ≡ 4, 2 ≡ 5, 3 ≡ 8, 4 ≡ 6, 5 ≡ 6, 7 ≡ 9

동치 관계에 따라 S는 다음과 같이 세 개의 동치 클래스들로 나누어집니다.

{0, 3, 8}, {1, 2, 4, 5, 6}, {7, 9}

다음의 문제들을 동치 클래스를 구하는 문제의 유형으로 볼 수 있습니다. 그러므로, UFSet 클래스를 이용하면 이들 문제를 효율적으로 해결할 수 있습니다.

1. 친구 그룹 문제 : 사람들 사이의 친구 관계가 주어질 때, 친구의 친구는 모두 같은 그룹에 속한다고 하면 최종적으로 구성되는 친구 그룹의 수는 몇 개인가?
2. 네트워크 연결 문제 : N개의 컴퓨터가 있고 일부가 서로 연결되어 있을 때, 특정 두 컴퓨터가 같은 네트워크에 있는지 판별하라.
3. 그래프에서 사이클 판별 문제[24] : 주어진 그래프에서 사이클이 발생하는지를 판별하라.

24) 12장의 최소 신장 트리를 구하는 크루스칼 알고리즘을 참고하세요.

4. 언어/국가 등 그룹 분류 문제 : 사람마다 사용할 수 있는 언어 목록이 주어질 때, 언어를 통해 서로 소통이 가능한 그룹의 수를 구하라.

위 문제 중에서 친구 그룹 문제를 UFSet 클래스를 사용하여 해결하는 방식을 살펴보겠습니다. 예를 들어, 10명의 사람에 대해 친구인 쌍들이 다음과 같다고 합시다.

(0, 3), (1, 4), (2, 5), (3, 8), (4, 6), (5, 6), (7, 9)

1번은 4번과 친구이고, 4번은 6번과 친구이므로 1, 4, 6번은 서로 같은 그룹에 속합니다. 또한 5번과 6번이 친구이고 2번은 5번과 친구이므로, 결국, 1, 2, 4, 5, 6번은 결국 같은 그룹에 속하게 됩니다. 같은 방식으로 고려하면 다음과 같이 3개의 그룹이 생기는 것을 알 수 있습니다.

(0, 3, 8), (1, 2, 4, 5, 6), (7, 9)

지금부터 UFSet 클래스를 사용하여 이 문제를 해결해 보겠습니다. 알고리즘의 개요는 다음과 같습니다. 친구들의 그룹을 집합으로 보고, 초기에 한 명씩만 포함된 집합들을 구성합니다. 그 다음, 각 친구 쌍 (i, j)에 대해 i가 속한 집합과 j가 속한 집합에 대해 합집합을 구성합니다. 최종 결과로 나오는 각 집합은 친구 그룹에 해당합니다. 표 10.2는 수행 과정을 단계별로 보여줍니다.

표 10.2: 친구 쌍 자료로부터 각 그룹을 구하는 과정

단계	친구 쌍	집 합
0	초기 상태	{0}, {1}, {2}, {3}, {4}, {5}, {6}, {7}, {8}, {9}
1	(0, 3)	{0, 3}, {1}, {2}, {4}, {5}, {6}, {7}, {8}, {9}
2	(1, 4)	{0, 3}, {1, 4}, {2}, {5}, {6}, {7}, {8}, {9}
3	(2, 5)	{0, 3}, {1, 4}, {2, 5}, {6}, {7}, {8}, {9}
4	(3, 8)	{0, 3, 8}, {1, 4}, {2, 5}, {6}, {7}, {9}
5	(4, 6)	{0, 3, 8}, {1, 4, 6}, {2, 5}, {7}, {9}
6	(5, 6)	{0, 3, 8}, {1, 2, 4, 5, 6}, {7}, {9}
7	(7, 9)	{0, 3, 8}, {1, 2, 4, 5, 6}, {7, 9}

프로그램 10.12의 함수 groups()는 이 과정을 구현한 프로그램입니다. groups(n, S)는 n명의 사람으로 구성된 친구 리스트 S에 대한 친구 그룹 수를 반환합니다. 함수 groups()의 수행 시간은 $\Theta(n\alpha(n, m))$입니다, 여기서 m은 리스트 S의 길이입니다.

프로그램 10.12: 친구 그룹 수를 구하는 프로그램

```
def groups(n, S):
    """n명의 친구 리스트 S에 대한 친구 그룹 수를 반환한다."""
    C = UFSet(n)            # n개의 원소를 가진 UFSet 객체 생성
    for p in S:             # 합집합을 구하는 과정
        C.union(C.find(p[0]), C.find(p[1]))
    count = 0
    for i in range(n):      # 그룹의 수 계산
        if C.root(i):
            count += 1
    return count            # 그룹의 수 반환
```

10.4 사례 연구: 세그먼트 트리

세그먼트 트리를 소개하기 전에 다음 문제를 먼저 풀어봅시다.

■ 온라인 점수판

프로그램 경시 대회에서 대회가 진행되는 동안 각 경시생이 얻은 점수를 온라인으로 제공하고 있다. 점수판에는 1번 경시생부터 차례대로 점수가 나열된다. 당연하게도 경시생의 점수는 빈번하게 갱신된다. 점수판에는 현재 최고점과 최저점, 그리고 점수의 합계뿐만 아니라 경시생의 번호 구간을 지정하면, 그 구간에서의 최고점, 최저점, 점수 합을 표시해 준다. 예를 들어, 12명의 경시생이 참여한 대회에서 현재 점수가 다음과 같다고 가정하자.

경시생 번호	1	2	3	4	5	6	7	8	9	10	11	12
점수	8	1	5	4	9	7	8	2	0	5	7	2

경시생 번호 구간을 [3, 9]로 지정하면, 7개의 점수 (5, 4, 9, 7, 8, 2, 0) 중에서 최고점은 9, 최저점은 0, 합계는 35이다. 특정 구간을 지정하여 경시 정보를 알아보려는 사용자가 많으므로, 가능한 한 빠르게 결과를 표시할 수 있어야 한다. 이 작업을 위한 프로그램을 작성하시오.

위 문제에 대한 단순한 해법은 특정 구간에 대해 순차 탐색을 사용하여 답을 구하는 것입니다. 경시생의 수가 n이고 결과를 구해야 할 질의 구간의 수가 m이라면, 단순한 해법의 시간복잡도는 $O(m \cdot n)$입니다. 반면, 이제 소개할 세그먼트 트리라는 자료구조를 활용하면 이 문제를 $O(m \log n)$ 시간에 해결할 수 있습니다.

세그먼트 트리(segment tree)는 선형리스트, 특히 배열의 특정 구간에서 최댓값, 최솟값, 합계 등을 빠르게 구하는 데 활용되는 이진 트리입니다. 위 문제를 세그먼트 트리를 활용하여 해결한다면, 세그먼트 트리를 구축하는데 $O(n)$, 질의에 대한 답을 구하는데 $O(\log n)$, 점수가 바뀌어 세그먼트 트리를 갱신하는데 $O(\log n)$의 시간복잡도를 가집니다.

먼저, 세그먼트 트리의 구조에 대해서 살펴보겠습니다. 세그먼트 트리는 기본적으로 트리의 모든 레벨마다 노드가 꽉 차 있는 포화 이진 트리(perfect binary tree)를 가정합니다. 세그먼트 트리의 잎 노드는 자료 원소에 대응하고, 내부 노드는 그 노드를 루트로 하는 부트리에 속한 자료 원소들(즉, 잎 노드들의 구간)에 대한 정보, 즉 최댓값, 최솟값, 합계 등의 값을 나타냅니다. 세그먼트 트리는 포화 이진 트리를 기반으로 하므로, 효율적인 배열 표현법으로 구현됩니다. 자료 원소의 개수가 n이면, 잎 노드의 개수 m은 n보다 크거나 같은 수 중에서 가장 작은 2의 거듭제곱입니다. 즉, $m = 2^k$, $2^{k-1} < n \leq 2^k$. 그리고 내부 노드의 수는 m-1이므로, 이진 트리의 총 노드 개수는 2m-1입니다. 따라서 2m-1 크기의 배열이 필요합니다. 예를 들어, n = 12이면, $m = 2^4 = 16$이고, 그러면 총 노드 개수는 2×16-1 = 31입니다. 만일 $n = 2^{k-1}+1$이면, $m = 2^k$이며, 따라서 n = m/2+1, 즉 m = 2(n-1)이므로, 총 노드 개수는 2m-1 = 2(2(n-1))-1 = 4n-5 < 4n입니다. 그러므로, 세그먼트 트리의 최대 노드 개수는 대략 2n에서 4n 사이가 됩니다. 일반적으로 세그먼트 트리의 표현을 위해 노드 개수의 상한인 4n 크기의 배열을 만들고 0 또는 None으로 초기화합니다. 세그먼트 트리의 잎 노드의 레벨이 k이므로, 트리의 높이는 $O(\log n)$입니다.

이제, 세그먼트 트리의 구축, 세그먼트 트리에서 질의와 자료 원소의 갱신을 수행하는 과정을 차례로 살펴보겠습니다.

세그먼트 트리는 다음과 같은 재귀적인 수행을 통해 구축됩니다.

① 배열을 같은 크기의 두 개 구간으로 분리한다.
② 각 구간에 대해 필요한 연산(합계, 최댓값 등)을 계산한다.
③ 계산 결과를 노드에 저장한다.

프로그램 10.13은 구간 합을 요구하는 질의에 답하기 위한 세그먼트 트리를 구축하는 프로그

램입니다. 프로그램에서 자료 원소는 리스트 D에 저장되어 있고, 세그먼트 트리는 리스트 T로 표현합니다. 재귀 함수 build(i, start, end, D, T)는 자료 원소가 저장된 리스트 D에서 첨자 범위 start부터 end까지의 요소들로 부트리 T[i]를 구축합니다. 이 함수에 의해 구축된 세그먼트 트리의 모습은 그림 10.14에 나와 있습니다. 세그먼트 트리의 실제적인 구조는 포화 이진 트리가 아니라 마지막 레벨의 노드가 군데군데 빠져 있는 형태의 트리입니다. 자료 원소의 수가 정확하게 2의 거듭제곱이라면 포화 이진 트리의 구조를 갖게 됩니다. 그림에서 회색으로 칠해진 원은 트리에 존재하지 않은 노드를 나타냅니다. 이런 노드는 리스트 T에서 공간을 차지하고 있지만 실제 연산에는 사용되지 않습니다. 리스트 T의 길이는 4n이고, 각 요소를 기껏해야 한 번씩 방문하면서 트리를 구성하므로, 함수 build()의 수행 시간은 O(n)입니다.

프로그램 10.13: 세그먼트 트리를 구축하는 프로그램

```
1  def build(i, start, end, D, T):
2     """구간 합을 구하기 위한 세그먼트 트리를 구축한다."""
3     if start == end:                          # 잎 노드인 경우
4        T[i] = D[start]
5     else:
6        mid = (start + end) // 2               # 범위의 중앙 위치
7        build(2*i+1, start, mid, D, T)         # 왼쪽 부트리 생성
8        build(2*i+2, mid+1, end, D, T)         # 오른쪽 부트리 생성
9        T[i] = T[2*i+1] + T[2*i+2]             # 합 계산
10
11 # 세그먼트 트리 T의 구축 테스트
12 D = [8, 1, 5, 4, 9, 7, 8, 2, 0, 5, 7, 2]    # 자료 원소
13 T = [None] * (4*len(D))                      # None으로 초기화된 리스트
14 build(0, 0, len(D)-1, D, T)                  # 세그먼트 트리 구축
```

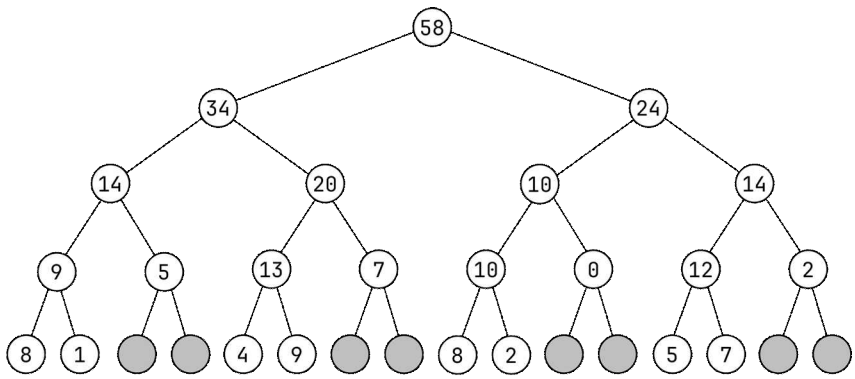


그림 10.14: 구간의 합 질의를 위한 세그먼트 트리의 예

자료 원소의 리스트 D = [1, 3, 5, 7, 9, 11]을 가지고 프로그램 10.13의 함수 build()를 호출하면, 그림 10.15와 같은 세그먼트 트리가 구축됩니다. 그림에서 노드의 위에 표시된 빨간색 번호는 해당 노드가 위치한 리스트 T의 첨자를 나타내고, 노드 아래에 표시된 범위 [a, b]는 해당 노드가 참조하는 D의 구간을 나타냅니다. 루트 노드는 D의 전체 구간을 참조하고, 잎 노드는 개별 자료 원소를 참조합니다.

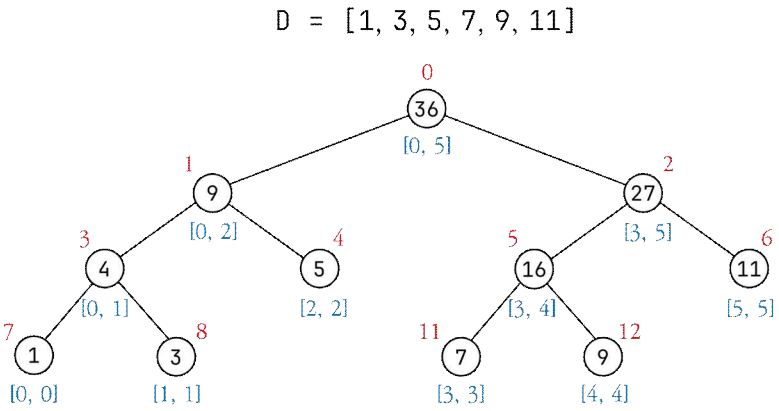


그림 10.15: 세그먼트 트리에서 노드의 구간 정보

다음으로, 세그먼트 트리에서 질의를 수행하는 과정을 소개합니다. 질의는 자료 원소 리스트의 특정 구간에서 합계나 최댓값, 최솟값 등을 요구하는 것입니다. 여기서는 합계를 요구하는 질의를 수행합니다. 프로그램 10.13의 build() 함수에 의해 구축된 세그먼트 트리에서 주어진 질의에 대한 답을 찾는 프로그램이 프로그램 10.14에 나와 있습니다. 재귀 함수 query(i, start, end, left, right, T)에서 T는 세그먼트 트리가 저장된 리스트이고, i는 현재 방문한 노드의 첨자를, start와 end는 현재 노드가 담당하는 구간을, left와 right는 합계를 구하고자 하는 질의 구간을 나타냅니다. 함수 query()를 최초로 호출할 때, i는 루트에 해당하는 0, start와 end는 0과 len(D)-1입니다. 함수 query()는 질의 구간과 노드 구간을 비교하여 다음과 같이 수행합니다.

1. 노드 구간과 질의 구간이 완전히 분리된 경우 : 구간 합에 포함될 자료 원소가 없으므로 0을 반환한다. (경우 1)
2. 노드 구간이 질의 구간에 포함되는 경우 : 노드 구간에 있는 모든 자료 원소를 더해야 하므로 현재 노드의 값을 반환한다. (경우 2)
3. 노드 구간이 질의 구간을 포함하거나 부분적으로 교차하는 경우 : 현재 노드의 왼쪽 부트리와 오른쪽 부트리를 각각 재귀적으로 탐색하여 얻은 값을 더해서 반환한다. (경우 3)

프로그램 10.14: 세그먼트 트리에서 질의를 수행하는 프로그램

```
def query(i, start, end, left, right, T):
    """질의 구간 [left, right]의 합을 반환한다."""
    # 노드 구간과 질의 구간이 분리된 경우 (경우 1)
    if right < start or end < left:
        return 0
    # 노드 구간이 질의 구간에 포함되는 경우 (경우 2)
    if left <= start and end <= right:
        return T[i]
    # 노드 구간이 질의 구간을 포함하거나 부분적으로 교차하는 경우 (경우 3)
    mid = (start + end) // 2                # 범위의 중앙 위치
    leftsum = query(2*i+1, start, mid, left, right, T)   # 왼쪽 부트리
    rightsum = query(2*i+2, mid+1, end, left, right, T)  # 오른쪽 부트리
    return leftsum + rightsum               # 합계 반환

# 테스트
D = [1, 3, 5, 7, 9, 11]                     # 자료 원소
T = [None] * (4*len(D))                     # None으로 초기화된 리스트
build(0, 0, len(D)-1, D, T)                 # 세그먼트 트리 구축
answer = query(0, 0, len(D)-1, 2, 4, T)  # 질의 구간 [2, 4]
```

예를 들어, D = [1, 3, 5, 7, 9, 11]이고 질의 구간이 [2, 4]일 때, query() 함수의 수행 과정을 표 10.3과 그림 10.16에서 살펴볼 수 있습니다. 표 10.3에서 호출 순서는 해당 노드를 방문하는 순서이고, 그림 10.16에서 에지에 표시된 'a/b'는 각각 해당 노드의 방문 순서와 그 노드에서의 반환 값을 나타냅니다.

표 10.3: 프로그램 10.14의 함수 query()의 수행 과정 (질의 구간 [2, 4])

호출 순서	노드 첨자	노드 구간 [start, end]	반환 값	비고
1	0	[0, 5]	21	(경우 3) 좌, 우 부트리 탐색
2	1	[0, 2]	5	(경우 3) 좌, 우 부트리 탐색
3	3	[0, 1]	0	(경우 1) 0을 반환
4	4	[2, 2]	5	(경우 2) 5를 반환
5	2	[3, 5]	16	(경우 3) 좌, 우 부트리 탐색
6	5	[3, 4]	16	(경우 2) 16을 반환
7	6	[5, 5]	0	(경우 1) 0을 반환

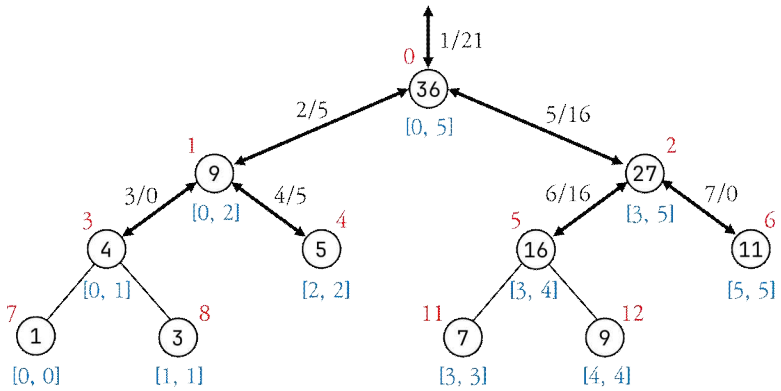


그림 10.16: 프로그램 10.14의 수행 과정 (질의 구간 [2, 4])

함수 query()의 수행에서 세그먼트 트리의 많은 노드를 방문하는 것처럼 보이지만, 각 레벨에서 기껏해야 4개의 노드만 방문합니다. 그중의 2개는 (경우 1) 또는 (경우 2)에 해당하는 노드이고, 나머지 2개는 (경우 3)에 해당합니다. (경우 1) 또는 (경우 2)에 해당하는 노드는 부트리를 탐색하지 않으므로, 모든 레벨에서 최대 4개의 노드만 방문합니다. 따라서 함수 query()의 수행 시간은 $O(\log n)$입니다.

마지막으로, 세그먼트 트리에서 자료 원소의 값을 갱신하는 과정을 소개합니다. 프로그램 10.15에 자료 원소의 갱신을 수행하는 프로그램이 나와 있습니다. 재귀 함수 update(i, start, end, idx, value, T)에서 T는 세그먼트 트리가 저장된 리스트이고, i는 현재 방문한 노드의 첨자를, start와 end는 현재 노드가 담당하는 구간을, idx는 갱신할 자료 원소의 원래 첨자, value는 변경할 값을 나타냅니다. 재귀 함수 update()는 연속적으로 재귀 호출하여 도달한 잎 노드에서 자료 원소의 값을 갱신한 후에, 함수의 복귀 과정을 통해 해당 잎 노드의 조상 노드를 차례로 방문하면서 노드의 값을 갱신합니다.

프로그램 10.15: 세그먼트 트리에서 자료 원소의 갱신을 수행하는 프로그램

```
1 def update(i, start, end, idx, value, T):
2     """ D[idx]의 값이 value로 갱신된 세그먼트 트리를 구축한다."""
3     if start == end:                    # 잎 노드이면 자료 원소의 값을 갱신함
4         T[i] = value
5         return
6
7     mid = (start + end) // 2            # 범위의 중앙 위치
8     if start <= idx <= mid:             # 왼쪽 부트리에 갱신할 노드가 있는 경우
```

```
9       update(2*i+1, start, mid, idx, value, T)
10    else:                              # 오른쪽 부트리에 갱신할 노드가 있는 경우
11       update(2*i+2, mid+1, end, idx, value, T)
12    T[i] = T[2*i+1] + T[2*i+2]         # 구간 합 갱신
13
14 # 테스트
15 D = [1, 3, 5, 7, 9, 11]              # 자료 원소
16 T = [None] * (4*len(D))              # None으로 초기화된 리스트
17 build(0, 0, len(D)-1, D, T)          # 세그먼트 트리 구축
18 D[1] = 8                             # D[1]의 값을 8로 변경함
19 update(0, 0, len(D)-1, 1, 8, T)      # 자료 원소 갱신을 위한 호출
```

예를 들어, D = [1, 3, 5, 7, 9, 11]이고 D[1]을 8로 갱신할 경우, update() 함수는 재귀 호출을 통해 T[0], T[1], T[3], T[8]을 차례로 방문한 다음, 잎 노드에 해당하는 T[8]의 값을 8로 갱신합니다. 그런 다음, 복귀 과정을 통해, T[3]의 값을 1+8=9로 갱신하고, T[1]의 값을 9+5=14로 갱신하고, 마지막 T[0]의 값을 14+27=41로 갱신합니다. 그림 10.17은 잎 노드까지 도달 후에 복귀 과정을 통해 구간 합이 갱신되는 과정을 보여줍니다.

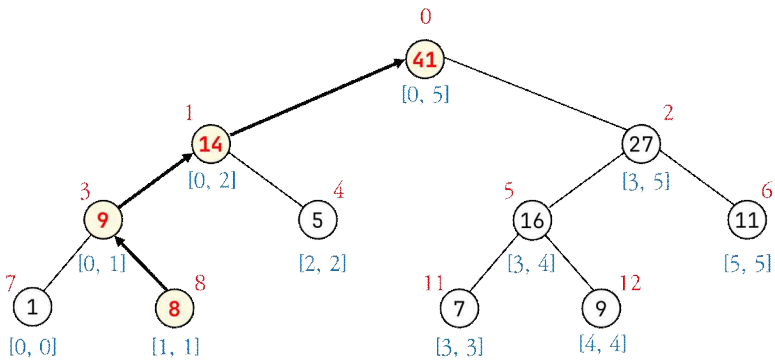


그림 10.17: 프로그램 10.15의 수행에서 update()의 복귀 과정

함수 update()의 재귀 호출과 복귀 과정에서 세그먼트 트리의 레벨당 한 개 노드만 방문하므로, 수행 시간은 $O(\log n)$입니다.

연습문제

1. 공백 이진 탐색 트리에서 다음의 순서로 원소들이 차례대로 삽입될 때, 최종 만들어지는 이진 탐색 트리의 상태를 그리시오.

   40, 80, 70, 10, 60, 20, 90, 50, 30

2. 1번 문제에서 구해진 이진 탐색 트리에서 원소 40과 50을 차례로 삭제한 이후의 이진 탐색 트리의 상태를 그리시오.

3. 이진 탐색 트리에서 키의 범위가 주어질 때, 해당 범위의 키를 가진 원소들을 모두 출력하는 함수 range_search_bst(root, a, b)를 작성하시오. 이 함수는 이진 탐색 트리 root에서 키의 범위가 a와 b 사이인 원소들을 모두 출력한다.

4. 이진 탐색 트리에서 탐색, 삽입, 삭제를 수행하는 함수(프로그램 10.2, 10.3, 10.4)를 각각 재귀 함수로 구현하시오.

5. 이진 트리 root가 이진 탐색 트리의 속성을 만족하는지 판별하는 함수 is_bst(root)를 작성하시오.

6. 이진 탐색 트리 root와 정수 k가 주어질 때, 트리에서 k번째로 작은 값을 반환하는 함수 find_kth_smallest(root, k)를 작성하시오. 단, k는 1 이상 노드 수 이하이다.

7. 오름차순으로 정렬된 정수 리스트 L이 주어질 때, L의 요소들로 구성되면서 트리의 높이가 최소인 이진 탐색 트리를 구축하는 함수를 작성하시오.

8. 공백 최대 힙에 다음의 순서로 원소들이 차례대로 삽입될 때(프로그램 10.5의 insert_heap()의 수행), 최종 만들어진 힙의 상태를 그리시오.

   40, 80, 70, 10, 60, 20, 90, 50, 30

9. 8번 문제에 주어진 순서의 원소들을 가지고 초기 힙을 만든다고 할 때(프로그램 10.7의 init_heap()의 수행), 최종 만들어진 초기 힙의 상태를 그리시오.

10. 9번 문제에서 구해진 초기 힙에서 세 번의 삭제가 수행된 후의 힙의 상태를 그리시오.

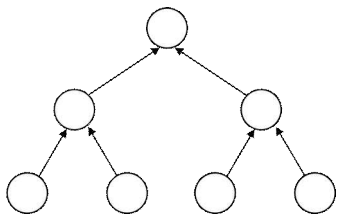
11. 정수 리스트 L이 주어질 때, 이 리스트가 최대 힙의 속성을 만족하는지 판별하는 함수 is_max_heap(L)을 작성하시오.

12. 0부터 9까지 10개의 정수로 초기화된 UFSet 객체에 대해 다음 union 연산을 순서대로 적용했을 때의 parent 배열의 최종 상태를 그리시오. 단, union 연산에 가중치 규칙이 적용된다고 가정한다.

    union(0, 1), union(2, 3), union(4, 5), union(0, 2),
    union(6, 7), union(8, 9), union(4, 6), union(0, 4)

13. 분리된 집합을 위한 트리 표현에서 가중치 규칙을 적용한 union 연산을 사용할 때, n개의 노드를 가진 트리의 높이가 항상 $\lfloor \log n \rfloor + 1$ 이하가 됨을 증명하시오.

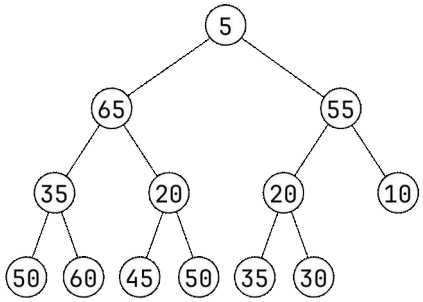
14. 가중치 규칙을 적용한 union 연산으로는 다음과 같은 트리가 구성될 수 없음을 증명하시오.



15. 세그먼트 트리의 구축, 질의, 갱신을 수행하는 프로그램 10.13, 10.14, 10.15는 구간의 합을 구하기 위한 프로그램이다. 이들 프로그램을 구간의 최솟값(range minimum)을 구하기 위한 코드로 수정하시오.

16. **최소-최대 힙(min-max heap)**은 최소 힙과 최대 힙을 한꺼번에 표현할 수 있는 자료구조로서 원소의 삽입 연산뿐만 아니라 키가 최소 또는 최대인 원소의 삭제를 각각 $O(\log n)$ 시간에 수행할 수 있다. 최소-최대힙은 보통의 힙과 같이 완전 이진 트리로 구성된다. 최소-최대 힙의 짝수 레벨에 있는 노드의 키는 자신의 부트리에 있는 노드들의 키보다 작거나 같고(최소 힙), 홀수 레벨에 있는 노드의 키는 자신의 부트리에 있는 노드들의 키보다 크거나 같다(최대 힙). 레벨이 0인 루트 노드에는 전체 힙의 최솟값을 가지고 있다.

다음 그림은 최소-최대 힙의 예이다. 인터넷에서 최소-최대 힙을 검색하여 학습하고, 다음 물음에 답하시오.



(1) 위 그림의 최소-최대 힙에서 원소 '3'이 삽입된 후의 모습을 그리시오.
(2) 위 그림의 최소-최대 힙에서 원소 '70'이 삽입된 후의 모습을 그리시오.
(3) 위 그림의 최소-최대 힙에서 최소 키를 가진 원소를 삭제한 후의 모습을 그리시오.
(4) 위 그림의 최소-최대 힙에서 최대 키를 가진 원소를 삭제한 후의 모습을 그리시오.

17. [실습 문제] n개의 정수가 순서대로 입력된다. 매번 새로운 정수가 입력될 때마다, 현재까지 입력된 모든 정수들의 중앙값(median)을 실시간으로 출력하는 프로그램을 작성하시오. 중앙값은 숫자들을 정렬했을 때 정중앙에 위치하는 수이다. 중앙값이 두 개이면, 그 중 작은 수를 출력한다.

예를 들어, 1, 5, 2, 4가 차례로 입력되면, 1, 1, 2, 2를 출력해야 한다. 즉, [1]의 중앙값은 1, [1, 5]의 중앙값은 1, [1, 5, 2]의 중앙값은 2, [1, 5, 2, 4]의 중앙값은 2이다.

입력으로 첫 줄에 정수의 개수 n이 주어진다. n은 1 이상 100,000 이하이다. 둘째 줄부터 n개의 정수가 한 줄에 하나씩 주어진다. 각 정수는 -10^9과 10^9 사이의 값이다. 출력으로, 총 n 줄에 걸쳐, 각 단계에서 현재까지 입력된 정수들의 중앙값을 출력한다.

입력의 예	출력의 예
4 1 5 2 4	1 1 2 2

18. [실습 문제] 10.4절의 온라인 점수판 문제를 해결하는 프로그램을 작성하시오.

프로그램 경시 대회에서 대회가 진행되는 동안 각 경시생이 얻은 점수를 온라인으로 제공하고 있다. 점수판에는 1번 경시생부터 차례대로 점수가 나열된다. 당연하게도 경시생의 점수는 빈번하게 갱신된다. 점수판에는 현재 최고점과 최저점, 그리고 점수의 합계뿐만 아니라 경시생의 번호 구간을 지정하면, 그 구간에서의 최고점, 최저점, 점수 합을 표시해 준다.

경시생 번호	1	2	3	4	5	6	7	8	9	10	11	12
점수	8	1	5	4	9	7	8	2	0	5	7	2

경시생 번호 구간을 [3, 9]로 지정하면, 7개의 점수 (5, 4, 9, 7, 8, 2, 0) 중에서 최고점은 9, 최저점은 0, 합계는 35이다. 특정 구간을 지정하여 경시 정보를 알아보려는 사용자가 많으므로, 가능한 한 빠르게 결과를 표시할 수 있어야 한다. 이 작업을 위한 프로그램을 작성하시오.

입력으로, 첫 줄에 경시생의 수 n과 질의 개수 m이 주어진다. n과 m은 1 이상 100,000 이하의 정수이다. 경시생은 1부터 n까지의 번호로 구분된다. 둘째 줄에는 n개의 점수가 공백으로 구분되어 주어진다. i번째 점수는 i번 경시생의 초기 점수이고, 각 점수는 0 이상 10,000 이하의 정수이다. 셋째 줄부터 m개의 줄에 걸쳐 한 줄에 하나씩 질의가 주어진다. 질의는 두 가지 유형으로 다음과 같은 형식으로 되어있다.

1 i x : i번 경시생의 점수를 x로 변경하는 갱신 질의
2 a b : a번 경시생부터 b번 경시생까지 구간의 통계 질의 (a ≤ b)

출력으로, 각 “2 a b” 유형의 질의에 대해, 해당 구간의 최고점, 최저점, 합계를 한 줄에 출력한다.

입력의 예	출력의 예
5 3 10 20 30 40 50 2 1 3 1 2 25 2 1 3	30 10 60 30 10 65

제11장

그래프

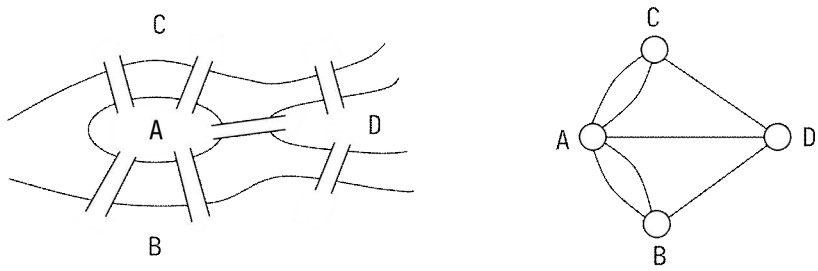
11.1 용어와 정의
11.2 그래프의 표현
11.3 그래프 탐색
11.4 그래프 탐색의 활용
11.5 사례 연구: 위상 정렬
연습문제

학습 목표

- 그래프의 정의와 개념을 이해하고, 정점, 에지, 경로, 사이클, 차수 등 기본 용어와 다양한 그래프의 종류와 특성을 설명할 수 있다.
- 그래프를 표현하는 인접 행렬과 인접 리스트 방식을 이해하고, 각 표현법의 정의와 특징을 비교 설명할 수 있다.
- 깊이 우선 탐색과 너비 우선 탐색의 원리를 이해하고, 각각 재귀 함수와 큐를 이용하여 프로그램으로 구현할 수 있다.
- 그래프 탐색을 이용하여 연결 요소와 신장 트리를 찾거나 최단 경로를 구할 수 있다.
- 방향 비순환 그래프에서 위상 정렬을 수행하는 알고리즘을 설명할 수 있다.

11.1 용어와 정의

그래프(graph)는 그림 11.1 (b)와 같이 **정점(vertex)** 또는 **노드(node)**라고 불리는 원과 **에지(edge)** 또는 **간선**이라고 불리는 선으로 구성된 객체입니다. 그래프는 현실 세계의 사물이나 현상을 단순하게 표현하기 위한 수학적 모델링 도구로서 널리 활용됩니다. 그래프의 개념은 옛날부터 일상생활에서도 다양하게 사용되었지만, 그래프가 수학적 구조로 이론적인 체계를 갖추게 된 것은 18세기의 수학자 오일러(Euler)에 의해서입니다. 1736년에 발표한 논문에서 "쾨니히스베르크(Königsberg) 다리 문제"를 해결하기 위해 오일러가 그래프를 사용한 것으로 알려져 있습니다. 구 프러시아의 쾨니히스베르크라는 지역을 가로지르는 강에는 그림 11.1 (a)와 같이 두 개의 섬과 7개의 다리가 놓여 있었습니다.



(a) 쾨니히스베르크 다리들 (b) 그래프

그림 11.1: 쾨니히스베르크의 다리들과 그래프 모델

그 지역 사람들은 다리를 단 한 번씩만 지나면서 7개의 다리를 모두 건널 수 있는 방법을 찾아보려 했지만, 십수 년에 걸쳐 한 사람도 성공하지 못했다고 합니다. 그래서 당시의 위대한 수학자인 오일러에게 도움을 청하게 되었고, 오일러는 그 문제를 받자마자 "불가능하다"라고 단언했다고 전해집니다. 오일러는 불가능하다는 사실을 증명하기 위해, 강에 의해 분리된 각 지역을 정점으로, 각 다리를 에지로 하는 그림 11.1 (b)의 그래프를 구성했습니다. 쾨니히스베르크 다리 문제는 이 그래프에서는 어떤 한 정점에서 출발하여 모든 에지를 한 번씩만 지나서 다시 출발점으로 되돌아오는 경로를 찾는 문제가 됩니다. 오일러는 그래프의 모든 정점에서 짝수 개의 에지가 붙어 있는 경우에만 이러한 경로가 있음을 증명했습니다. 또한 홀수 개의 에지가 붙어 있는 정점이 단 두 개만 있는 경우에, 그 두 정점이 각각 출발점과 도착점이 되면서 모든 에지를 한 번씩 지나는 경로가 존재함을 증명했습니다. 오일러의 증명은 그래프의 단순성을 잘 활용한 결과였습니다.

그래프의 단순한 구조는 문제의 핵심을 잘 파악할 수 있도록 해 주어 문제 해결에 도움을 줍니다. 이런 이유로 현실 세계의 현상이나 사물을 그래프로 모델링한 후 문제를 해결하려는 시도는 다양한 응용 분야에서 광범위하게 나타납니다. 또한 그래프는 추상적 모델이므로, 다른 응용 분야에서 발생하는 문제들이 그래프로 모델링된 후에는 동일한 문제가 되는 경우가 많습니다.

컴퓨터를 이용하여 문제를 해결하려는 경우에는 그래프가 자료구조로서 역할을 합니다. 그래프는 가장 일반적인 구조를 가진 자료구조이기 때문에, 선형리스트와 트리도 그래프의 일종으로 취급됩니다. 그래프로 표현하고자 하는 자료에 대해, 그래프의 정점과 에지는 각 자료 원소에 대응합니다. 특히, 에지는 정점에 대응하는 자료 원소들 사이의 관계를 나타냅니다.

이제 그래프의 정의와 주요 용어를 알아보겠습니다. 그래프 G를 수학적으로 정의할 경우, 정점 집합 V와 에지 집합 E를 가지고 정의합니다. 즉, G = (V, E)입니다. 일반적으로, 각 정점은 자료 원소에 부여된 번호로 표시됩니다. 따라서 정점 집합 V는 자료 원소에 부여된 정점 번호들의 집합입니다. 그래프를 그림으로 표현할 경우, 동일한 그래프를 다양한 모습으로 그릴 수 있습니다. 예를 들어, 그림 11.2에 그려진 세 그래프는 모두 같은 그래프입니다.

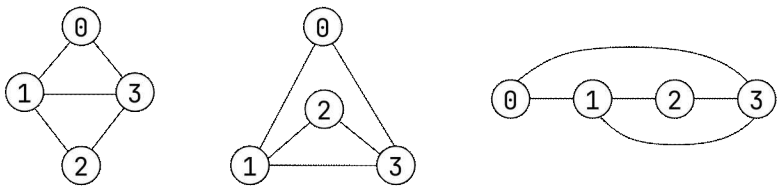


그림 11.2: 다른 모습으로 그려진 그래프

두 정점을 잇는 선으로 표시되는 에지는 양방향 또는 단방향 속성을 가집니다. 양방향 속성을 가진 에지는 방향을 표시하지 않으므로 **무향 에지(undirected edge)**라고 부릅니다. 두 정점 v와 w를 연결하는 무향 에지는 정점 쌍 (v, w)로 표시되며, 정점의 순서에는 무관하므로 (v, w) = (w, v)입니다. 모든 에지가 무향 에지인 그래프를 **무향 그래프(undirected graph)**라고 부릅니다. 그림 11.2에 그려진 그래프는 정점 집합이 {0, 1, 2, 3}이고 에지 집합이 {(0, 1), (0, 3), (1, 2), (1, 3), (2, 3)}인 무향 그래프입니다. 무향 그래프의 경우, 특별히 '방향 없음'을 강조하는 것이 아니라면 '무향 에지'는 단순히 '에지', '무향 그래프'는 단순히 '그래프'라고 지칭합니다.

화살표로 표시하는 단방향 속성을 가진 에지는 **방향 에지(directed edge)**라고 부릅니다. 정점 v에서 정점 w로의 방향을 가진 방향 에지는 정점의 순서쌍 ⟨v, w⟩로 표시되며, 따라서 ⟨v, w⟩

≠ ⟨w, v⟩입니다. 방향 에지 ⟨v, w⟩에서 v를 **꼬리(tail)**, w를 **머리(head)**라고 부릅니다. 방향 에지를 가진 그래프는 무향 그래프와 구분하기 위해 항상 '방향'을 붙여서 **방향 그래프(directed graph 줄여서 digraph)**라고 부릅니다. 그림 11.3에 그려진 두 그래프는 동일한 그래프로서 정점 집합이 {0, 1, 2, 3}이고 에지 집합이 {⟨0, 1⟩, ⟨0, 3⟩, ⟨1, 2⟩, ⟨1, 3⟩, ⟨2, 3⟩, ⟨3, 1⟩}인 방향 그래프입니다.

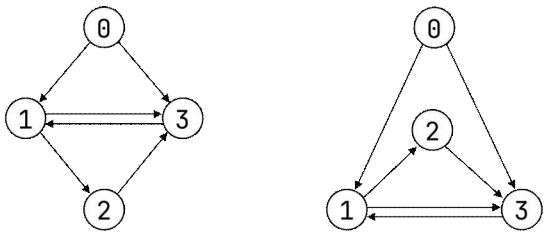


그림 11.3: 방향 에지를 가진 그래프

양 끝이 같은 정점인 에지, 즉 (v, v) 또는 ⟨v, v⟩와 같은 에지를 **셀프 루프(self loop)**라고 부르고, 두 정점을 연결하는 에지가 두 개 이상 존재하는 그래프를 **다중 그래프(multigraph)**라고 부릅니다. 따라서 그림 11.1 (b)는 다중 그래프에 해당합니다. **단순 그래프(simple graph)**는 셀프 루프가 없으며 다중 그래프가 아닌 그래프를 말합니다. 보통 그래프라고 부를 때는 무향이며 동시에 단순한 그래프를 의미합니다.

그래프에서 에지 (v, w)가 존재하면, "정점 v와 w는 서로 **인접하다(adjacent)**"라고 말합니다. 그래프의 두 에지 (v, w)와 (x, y)에 대해서, w = x이면 "두 에지는 서로 인접하다"고 말합니다. 방향 그래프의 방향 에지 ⟨v, w⟩에 대해서, "정점 v는 나가는 방향으로 정점 w에 인접하다"라고 말하거나(v is adjacent to w), "정점 w는 들어오는 방향으로 정점 v에 인접하다"라고 말합니다(w is adjacent from v). 에지 e = (v, w)에 대해서, "e는 v와 w에 부속되어 있다"고 말합니다(e is incident on v and w).[25]

어떤 정점의 **차수(degree)**는 그 정점에 부속된 에지의 개수입니다. 따라서 정점의 차수는 그 정점에 인접한 정점의 수와 같습니다. 그림 11.4의 그래프 G_1에서 정점 0의 차수는 4이고 정점 1의 차수는 1입니다. 모든 에지는 양 끝점인 두 정점에 부속되기 때문에, 그래프의 모든 정점

25) incidence라는 단어는 사건, 발생, 입사 등의 의미가 있습니다. 그래프에서 '에지는 정점에서 시작하고 정점에서 끝난다'는 일종의 사건으로 보거나 점과 선이 만나는 '입사'의 개념이 확장되어 incidence라는 단어가 도입된 것으로 보입니다. 이 책에서는, 한글 용어로서, 비교적 직관적이고 이해에 도움이 되는 '부속되다' 또는 '붙어있다'를 사용합니다.

에 대한 차수의 합은 에지 개수의 두 배가 됩니다. 방향 그래프에서 어떤 정점의 **전입 차수(in-degree)**는 그 정점이 머리가 되는 방향 에지의 개수이고, **전출 차수(out-degree)**는 그 정점이 꼬리가 되는 방향 에지의 개수입니다. 그림 11.4의 그래프 G_2에서 정점 2의 전입 차수는 1이고 전출 차수는 2입니다. 방향 그래프에서 모든 정점의 전입 차수의 합과 모든 정점의 전출 차수의 합은 같으며 그 값은 에지 개수에 해당합니다.

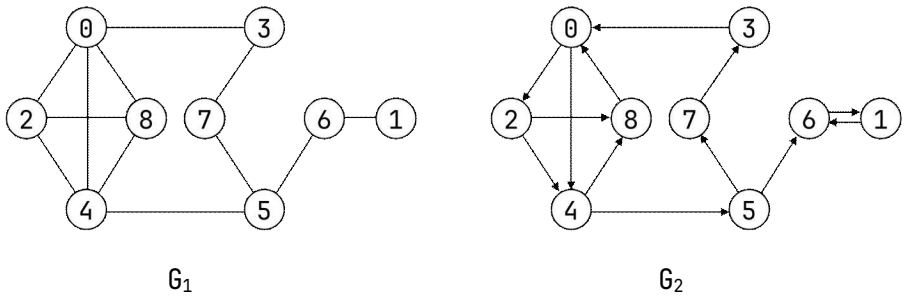


그림 11.4: 그래프의 예

그래프에서 두 정점 v와 w 사이의 **경로(path)**는 v에서부터 w까지의 연속된 에지들로 연결된 선을 말하며, 정점들의 순서 리스트 ($v = u_0, u_1, \cdots, u_{n-1} = w$)로 표시합니다. 그림 11.4의 그래프 G_1에서 순서 리스트 (0, 2, 4, 5, 6, 1)은 정점 0과 정점 1 사이의 경로입니다. **경로의 길이**는 경로를 구성하는 에지의 개수입니다. 그래서 경로 (0, 2, 4, 5, 6, 1)은 길이가 5인 경로입니다. 특히 경로에 놓인 정점들이 모두 다른 경로를 **단순 경로(simple path)**라고 부릅니다. 경로 (0, 2, 4, 5, 6, 1)은 단순 경로이지만 경로 (0, 4, 8, 2, 4, 5, 6, 1)은 단순 경로가 아닙니다. 출발점과 도착점이 같다는 점만 예외로 하는 단순 경로를 **사이클(cycle)**이라고 부릅니다. 그림 11.4의 그래프 G_1에서 (0, 2, 4, 8, 0)은 길이가 4인 사이클이고, (0, 2, 8, 4, 5, 7, 3, 0)은 길이가 7인 사이클입니다. 방향 그래프에서 방향을 고려한 경로 및 사이클을 각각 **방향 경로(directed path)**와 **방향 사이클(directed cycle)**이라고 부릅니다. 그림 11.4의 그래프 G_2에서 ⟨0, 2, 4, 5, 6, 1⟩은 정점 0과 정점 1 사이의 방향 경로이지만, ⟨0, 3, 7, 5, 6, 1⟩은 방향 경로가 아닙니다.

어떤 그래프의 정점 집합과 에지 집합에 대해 각각 부분 집합으로 구성된 그래프를 그 그래프의 **부그래프(subgraph)**라고 부릅니다. 다음 그래프 G = (V, E)는 그림 11.4의 그래프 G_1의 부그래프입니다.

$$V = \{0, 2, 4, 8\}$$
$$E = \{(0, 2), (2, 4), (4, 8), (8, 0)\}$$

그래프의 두 정점 v와 w 사이에 경로가 존재하면 "두 정점은 연결되어 있다"고 말합니다. 그래프 G의 모든 정점이 서로 연결되어 있으면, G를 **연결 그래프(connected graph)**라고 합니다. 최대로 연결된 부그래프, 즉 더 이상 정점을 추가하면 연결된 상태가 유지되지 않는 부그래프를 **연결 요소(connected component)**라고 합니다. 그림 11.5는 9개의 정점을 가진 그래프가 3개의 연결 요소로 구성된 예를 보여줍니다.

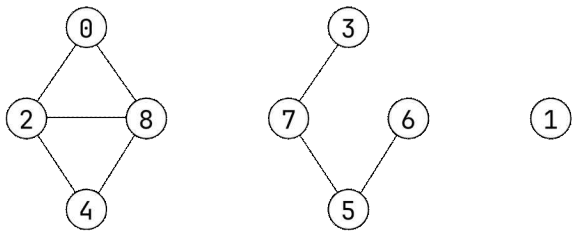


그림 11.5: 3개의 연결 요소로 구성된 그래프

방향 그래프 G에서 어떤 두 정점 v와 w를 선택하더라도 v에서 w로 향하는 방향 경로와 w에서 v로 향하는 방향 경로가 존재하면, "G는 강하게 연결되어 있다"고 말합니다. 그리고 그런 그래프를 **강 연결 방향 그래프(strongly connected digraph)**라고 부릅니다. 이와 비교하여 방향을 고려하지 않은 상태에서 연결된 그래프는 "약하게 연결되어 있다"고 말합니다. 방향 그래프에서 최대로 강하게 연결된 부그래프를 **강 연결 요소(strongly connected component)**라고 부릅니다. 예를 들어, 그림 11.4의 G_2는 약하게 연결된 상태이고, 두 개의 강 연결 요소가 있습니다. 그림 11.6에서 점선으로 둘러싸인 부분이 강 연결 요소에 해당합니다.

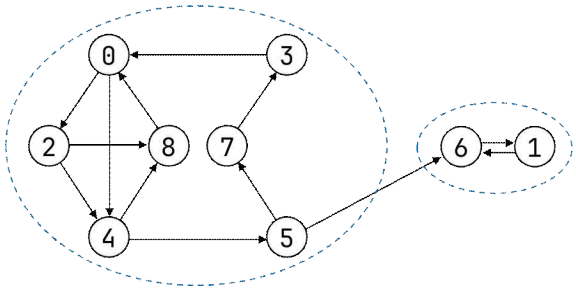
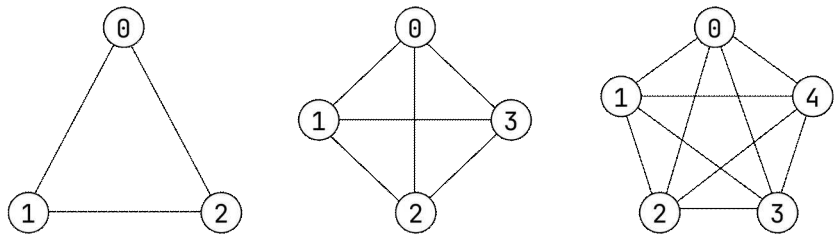


그림 11.6: 그림 11.4의 그래프 G_2의 강 연결 요소

이제, 특수한 구조를 가진 그래프의 유형에 대해 살펴보겠습니다. 그래프의 유형은 다양하지만, 여기서는 완전 그래프, 평면 그래프, 트리에 대해서 알아봅니다.

완전 그래프(complete graph)는 모든 정점 쌍 사이에 에지가 존재하는 단순 그래프입니다. 따라서 완전 그래프는 정점 수가 같은 단순 그래프 중에서 에지 수가 가장 많은 그래프입니다. 그림 11.7은 정점 수가 각각 3, 4, 5인 완전 그래프를 보여줍니다. 정점 수가 n인 완전 그래프의 에지 수는 정점 쌍의 개수에 해당하는 n(n-1)/2입니다.



G_1 G_2 G_3

그림 11.7: 완전 그래프의 예

평면 그래프(planar graph)는 그래프를 평면에 그릴 때 에지를 교차하지 않고 그릴 수 있는 그래프입니다. 그림 11.7의 G_1과 G_2는 평면 그래프이며, G_3는 평면 그래프가 아닙니다. G_2의 경우 그림에서는 에지 (0, 2)와 에지 (1, 3)은 서로 교차하고 있으나 둘 중의 하나를 바깥으로 돌려서 그린다면 교차하지 않고 그릴 수 있습니다. 평면 그래프가 아닌 G_3은 어떤 방법을 사용하더라도 에지의 교차 없이는 그릴 수 없습니다. 에지 수가 많으면 에지의 교차 없이 그리기 어렵기 때문에, 평면 그래프의 에지 수는 비교적 적습니다. n개의 정점과 m개의 에지를 가진 평면 그래프에 대해 다음의 식이 성립합니다. 이 식을 평면 그래프에 대한 **오일러 공식(Euler formula)**이라고 부릅니다.

$$m \leq 3n - 6$$

트리(tree)는 사이클이 없는 연결 그래프입니다. 정점 수가 n인 트리의 에지 수는 항상 n-1입니다. 트리에서 어떤 에지 하나만 제거하더라도 연결성이 깨지며, 어떤 위치에라도 에지 하나를 추가하면 사이클이 형성됩니다. 즉, 트리는 최소 개의 에지로서 모든 정점을 연결하는 구조라고 말할 수 있습니다. 또한, 트리의 두 정점을 연결하는 경로는 유일하다는 특성이 있습니다. 그래프의 유형 중 하나로 언급되는 '트리'는 9장에서 정의했던 '루트가 있는 트리(rooted tree)'와 달리 루트를 반드시 정의할 필요가 없습니다. 이처럼 루트가 없는 트리를 **자유 트리(free tree)**라고 부르기도 합니다. 한편, 9장과 10장에서 다루었던 루트가 있는 트리를 그래프의 관점에서 볼 때는 방향 그래프의 일종으로 간주할 수 있습니다. 그림 11.8에는 루트가 있는 트리와 자유 트리의 예가 나와 있습니다.

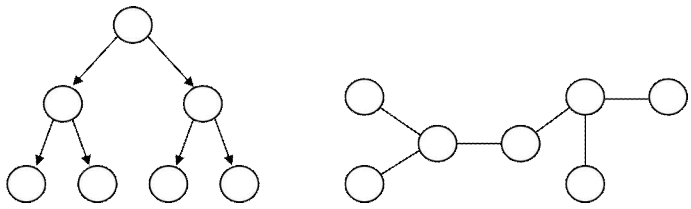


그림 11.8: 트리의 예

11.2 그래프의 표현

그래프를 대상으로 수행하는 다음의 기본적인 연산을 고려해 봅시다.

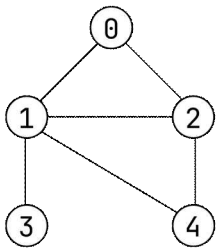
① 정점 쌍 (v, w)는 에지인가? 즉, 두 정점 v와 w는 서로 인접한가?
② 에지 (v, w)를 추가하라.
③ 에지 (v, w)를 삭제하라.
④ 정점 v에 인접한 정점 리스트를 생성하라.
⑤ 모든 에지를 출력하라.

그래프 G = (V, E)에서 정점 수를 n, 에지 수를 m이라고 합시다, 즉 n = |V|, m = |E|. 그리고 정점 집합 V = {0, 1, 2, ... , n-1}이라고 합시다. 에지 집합 E를 에지들의 선형리스트(배열 또는 연결리스트)로 표현하는 경우, 연산 ②는 상수 시간에 수행되지만, 선형리스트를 순차적으로 탐색해야 하는 나머지 연산들은 모두 $O(m)$ 시간에 수행됩니다. 연산 ⑤를 제외하고, 기본 연산이 $O(m)$ 시간에 수행되는 것은 그다지 효율적이지 않아 보입니다.[26] 따라서 그래프를 효율적으로 다룰 수 있는 표현 방식이 필요합니다. 그래프의 표현 방식 중에서 인접 행렬과 인접 리스트가 가장 널리 사용됩니다. 지금부터 무향 그래프와 방향 그래프를 표현하는 인접 행렬과 인접 리스트를 차례로 살펴봅니다.

인접 행렬(adjacency matrix)은 정점들 사이의 인접성 관계를 이용하여 행렬, 즉 2차원 배열로

26) 13장에서 소개하는 해싱 기반의 자료구조인 파이썬 set에 에지 집합을 저장하면, 연산 ①, ②, ③은 평균적으로 상수 시간에 수행되고, 연산 ④는 평균적으로 $O(n)$, 연산 ⑤는 $O(m)$ 시간에 수행될 수 있습니다. 이 장에서는 해싱 기반이 아닌 방식으로 그래프를 표현하는 것을 다룹니다.

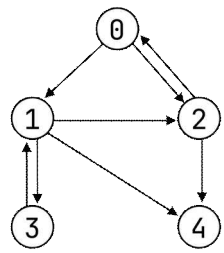
그래프를 표현하는 방법입니다. 정점 수가 n인 그래프의 인접 행렬은 n×n 크기의 2차원 행렬입니다. 이 행렬을 M이라고 합시다. 행렬 M의 각 첨자는 그래프의 각 정점에 대응합니다. 무향 그래프의 인접 행렬 M에서 M[i][j]에는 정점 i와 정점 j의 인접 여부 정보가 저장됩니다. 정점 i와 정점 j가 서로 인접하면, 즉 정점 쌍 (i, j)가 에지이면 M[i][j] = 1, 그렇지 않으면 M[i][j] = 0입니다. 단순·무향 그래프의 인접 행렬은 항상 M[i][i] = 0이고 M[i][j] = M[j][i]이므로, 인접 행렬은 주대각선을 기준으로 대칭입니다. 인접 행렬의 첨자가 i인 행 또는 열에 있는 1의 개수는 정점 i의 차수에 해당합니다. 따라서 인접 행렬에 있는 1의 개수는 에지 수의 두 배입니다. 그림 11.9는 무향 그래프에 대한 인접 행렬의 예를 보여줍니다.



	0	1	2	3	4
0	0	1	1	0	0
1	1	0	1	1	1
2	1	1	0	0	1
3	0	1	0	0	0
4	0	1	1	0	0

그림 11.9: 무향 그래프의 인접 행렬

방향 그래프에 대한 인접 행렬은, 방향 에지 ⟨i, j⟩가 존재하면 M[i][j] = 1이고, 그렇지 않으면 M[i][j] = 0입니다. 방향 에지는 ⟨i, j⟩ ≠ ⟨j, i⟩이므로, M[i][j] ≠ M[j][i]입니다. 방향 그래프의 인접 행렬에서 첨자가 i인 행에 있는 1의 개수는 정점 i의 전출 차수이고, 첨자가 i인 열에 있는 1의 개수는 정점 i의 전입 차수입니다. 따라서 인접 행렬에 있는 1의 총 개수는 에지 수와 같습니다. 그림 11.10은 방향 그래프에 대한 인접 행렬의 예를 보여줍니다.



	0	1	2	3	4
0	0	1	1	0	0
1	0	0	1	1	1
2	1	0	0	0	1
3	0	1	0	0	0
4	0	0	0	0	0

그림 11.10: 방향 그래프의 인접 행렬

그래프가 인접 행렬로 표현되면, 앞서 언급한 그래프 연산 ①, ②, ③은 상수 시간에 수행되고,

연산 ④는 행렬의 한 행 또는 한 열을 조사해야 하므로 $O(n)$, 연산 ⑤는 행렬 전체를 조사해야 하므로 $O(n^2)$에 수행됩니다.

인접 리스트(adjacency list)는 각 정점에 대해 그 정점에 인접한 정점들의 선형리스트로 그래프를 표현하는 방법입니다. 이 선형리스트는 연결리스트 또는 파이썬 리스트로 구현할 수 있습니다[27]. 연결리스트로 구현하는 경우는 각 연결리스트를 참조하는 링크 배열을 둡니다. 연결리스트의 각 노드는 정점 번호를 저장하는 데이터 필드와 노드의 링크를 저장하는 링크 필드로 구성됩니다. 그림 11.11은 무향 그래프에 대한 인접 리스트를 연결리스트와 파이썬 리스트로 나타낸 예를 보여줍니다.

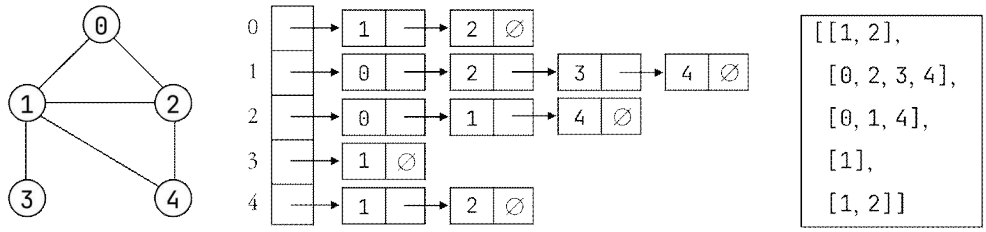


그림 11.11: 무향 그래프의 인접 리스트 (연결리스트와 파이썬 리스트)

방향 그래프에 대한 인접 리스트는 정점마다 두 개의 선형리스트로 구성됩니다. 하나는 그 정점에서 나가는 방향으로 인접한 정점들의 선형리스트이고, 다른 하나는 그 정점으로 들어오는 방향으로 인접한 정점들의 선형리스트입니다. 일반적으로, 나가는 방향으로 인접한 선형리스트들의 그룹을 인접 리스트라고 부르고, 들어오는 방향으로 인접한 선형리스트들의 그룹을 **역 인접 리스트(inverse adjacency list)**라고 합니다. 그림 11.12는 방향 그래프에 대한 인접 리스트와 역 인접 리스트를 파이썬 리스트로 나타낸 예를 보여줍니다.

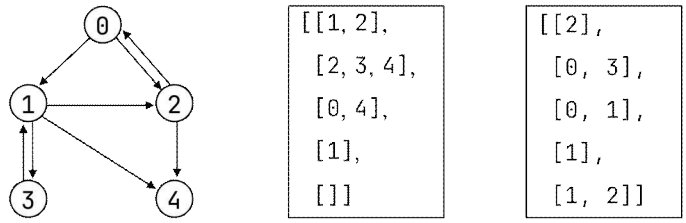


그림 11.12: 방향 그래프의 인접 리스트와 역 인접 리스트

27) 인접 리스트의 표현에서 각 선형리스트를 파이썬 `set`으로 구현하거나, 인접 리스트 자체를 정점이 키(key)이고 선형리스트가 값(value)인 요소들로 구성된 파이썬 딕셔너리(`dict`)로 구현할 수도 있습니다.

무향 그래프에서 모든 정점의 차수 합은 에지 수의 2배이므로, 인접 리스트에 존재하는 연결리스트의 노드 개수 역시 에지 수의 2배입니다. 방향 그래프에서 전출 차수의 합과 전입 차수의 합은 각각 에지 수와 같으므로, 인접 리스트와 역 인접 리스트에 존재하는 노드 수도 각각 에지 수와 같습니다. 그러므로 인접 리스트를 위한 기억 공간의 크기는 $O(n+m)$입니다. 표 11.1은 무향 그래프가 각각 에지 리스트, 인접 행렬, 인접 리스트로 표현된 경우, 기본 연산의 수행 시간을 비교한 표입니다. 표에서 d_v와 d_w은 각각 정점 v와 w의 차수입니다.

표 11.1: 그래프 표현 방식에 따른 기본 연산의 수행 시간 비교

연산	에지 리스트	인접 행렬	인접 리스트
(v, w)가 에지인가?	$O(m)$	$O(1)$	$O(\min(d_v, d_w))$
에지 (v, w) 삽입	$O(1)$	$O(1)$	$O(1)$
에지 (v, w) 삭제	$O(m)$	$O(1)$	$O(d_v+d_w)$
정점 v에 인접한 정점 리스트	$O(m)$	$O(n)$	$O(d_v)$
모든 에지 출력	$O(m)$	$O(n^2)$	$O(m)$

인접 행렬을 위한 기억 공간의 크기가 $O(n^2)$이므로, 인접 행렬을 사용하는 알고리즘은 시간복잡도의 하한이 $\Omega(n^2)$입니다. 에지 수 $m = O(n^2)$인 그래프를 대상으로 할 경우에는 인접 행렬과 인접 리스트 중 어떤 표현 방식을 사용하더라도 알고리즘의 수행 시간은 큰 차이가 없습니다. 이 경우는 에지의 삽입과 삭제 등이 상수 시간에 수행되고 코딩도 간편한 인접 행렬을 이용하는 것이 일반적입니다.

그러나 평면 그래프나 트리처럼 에지 수가 적은 그래프, 예를 들어 $m = O(n)$인 그래프의 경우는 인접 리스트를 사용하면 $O(n+m)$ 크기의 메모리 공간만을 사용하므로, 시간복잡도의 하한도 $\Omega(n)$으로 낮아집니다. 따라서 인접 행렬을 사용하는 것에 비해 더 효율적인 알고리즘을 개발할 가능성이 있습니다. 또한 정점들의 모든 차수의 합은 에지 수의 2배이므로, 기본 연산만을 사용하는 알고리즘의 수행 시간은 에지 수에 비례합니다. 따라서 에지 수가 적은 그래프의 경우는 알고리즘의 효율성을 위해서 인접 리스트를 사용하는 것이 필요합니다.

보통 에지 수 m이 $O(n^2)$에 가까운 그래프를 **밀집 그래프(dense graph)**라고 부르고, m이 $O(n)$에 가까운 그래프는 **희소 그래프(sparse graph)**라고 부릅니다. 따라서 밀집 그래프를 대상으로 할 때는 인접 행렬이 효과적이고, 희소 그래프를 대상으로 할 때는 인접 리스트가 효율적입니다.

그래프 데이터에 대한 입력이 다음과 같은 형식으로 주어진다고 합시다: 첫째 줄과 둘째 줄에 각각 정점 수 n과 에지 수 m이 주어지고, 그다음 줄부터 m개의 줄에는 각 줄에 하나씩 에지 (v, w)를 나타내는 두 정점 v와 w가 주어집니다. 예를 들어, 그림 11.11의 그래프를 위한 입력은 다음과 같습니다.

5	정점 수
6	에지 수
0 1	에지 1
0 2	에지 2
1 2	에지 3
1 3	에지 4
1 4	에지 5
2 4	에지 6

프로그램 11.1과 프로그램 11.2에는 위 형식으로 주어진 그래프 데이터를 읽어서 각각 인접 행렬과 인접 리스트를 구축하는 프로그램이 나와 있습니다. 인접 리스트는 파이썬 리스트를 이용하여 구축합니다.

프로그램 11.1: 무향 그래프의 인접 행렬을 구축하는 프로그램

```
1 def create_adj_matrix():           # 무향 그래프의 인접 행렬을 구축한 후 반환
2    n = int(input())                    # 정점 수
3    m = int(input())                    # 에지 수
4    M = [[0]*n for _ in range(n)]       # 인접 행렬을 위한 2차원 리스트
5    for _ in range(m):                  # m개의 에지를 입력받음
6       v, w = map(int, input().split())  # 에지 (v, w) 입력
7       M[v][w] = M[w][v] = 1             # 양방향으로 1 설정
8    return M                            # 인접 행렬 반환
```

프로그램 11.2: 무향 그래프의 인접 리스트를 구축하는 프로그램

```
1 def create_adj_list():           # 무향 그래프의 인접 리스트를 구축한 후 반환
2    n = int(input())                    # 정점 수
3    m = int(input())                    # 에지 수
4    G = [[] for _ in range(n)]          # 인접 리스트를 위한 2차원 리스트
5    for _ in range(m):                  # m개의 에지를 입력받음
6       v, w = map(int, input().split())  # 에지 (v, w) 입력
7       G[v].append(w)                    # v에 대한 리스트에 w 추가
8       G[w].append(v)                    # w에 대한 리스트에 v 추가
9    return G                            # 인접 리스트 반환
```

11.3 그래프 탐색

그래프를 대상으로 수행하는 작업에는 각 정점이나 에지를 체계적으로 방문해야 하는 경우가 빈번히 발생합니다. 따라서 이진 트리에서의 순회 알고리즘과 유사하게 그래프의 정점과 에지를 한 번씩 방문하는 탐색 전략이 필요합니다.

그래프 탐색 알고리즘으로 가장 많이 사용되는 것은 **깊이 우선 탐색(depth first search; DFS)**과 **너비 우선 탐색(breadth first search; BFS)**입니다. 깊이 우선 탐색은 스택 기반의 탐색 전략으로 5장에서 다룬 미로에서 길 찾기와 이진 트리에서의 전위 순회와 유사합니다. 너비 우선 탐색은 큐 기반의 탐색 전략으로 이진 트리에서의 레벨 순위 순회와 유사합니다. 그래프 탐색 알고리즘은 무향 그래프와 방향 그래프 모두에 적용되며, 기본 알고리즘 구조는 거의 똑같습니다. 또한 그래프가 인접 행렬로 표현되어 있으면 $O(n^2)$ 시간에, 인접 리스트로 표현되어 있으면 $O(n+m)$ 시간에 수행됩니다.

그래프 탐색에서 어떤 정점이 이미 방문한 정점인지를 판별해야 합니다. 이를 위해 정점 방문 여부를 기록하기 위한 배열(파이썬 리스트)을 사용합니다. 이 배열의 이름을 `visited`라고 합시다. `visited[i]`가 `True`이면 정점 `i`는 이미 방문한 정점이고, `visited[i]`가 `False`이면 정점 `i`는 아직 방문하지 않은 상태라고 정의합니다. 초깃값으로 `visited` 배열의 모든 요소에는 `False`가 저장됩니다.

■ 깊이 우선 탐색

깊이 우선 탐색은 그래프의 구조를 파악하거나 그래프의 연결성을 확인하는 데 유용한 알고리즘입니다. 이는 스택의 특성을 활용하여 하나의 경로를 따라 계속 탐색하다가 더 이상 나아갈 수 없으면 되돌아가서 다른 경로를 탐색하는 방식입니다. 마치 미로에서 막다른 길에 도달하면 이전에 왔던 길로 되돌아가는 방식과 같습니다. 탐색의 시작 정점 `v`가 주어질 때 깊이 우선 탐색은 다음과 같이 수행합니다.

① 현재 정점 `v`를 방문한다.
② `v`에 인접한 정점 중에서 아직 방문하지 않은 정점 `w`를 선택하여 단계 ①부터 다시 수행한다.
③ 만일 `w`가 존재하지 않으면, 이전 정점 `u`로 '퇴각(backtrack)'하여 단계 ②부터 다시 수행한다. 정점 `u`는 `v`를 방문하기 위해 거쳐온 에지 (`u`, `v`)의 출발점이다.
④ 만일 `u`가 존재하지 않으면(즉, 모든 경로를 탐색한 후에 시작 정점으로 되돌아왔을 때), 탐색을 종료한다.

예를 들어, 그림 11.13 (a)의 그래프를 정점 0을 시작 정점으로 하여 깊이 우선 탐색을 수행하는 과정이 그림 11.13과 표 11.2에 나와 있습니다. 탐색 과정에서 인접 정점을 선택할 때 항상 번호가 작은 정점부터 조사한다고 가정합니다.

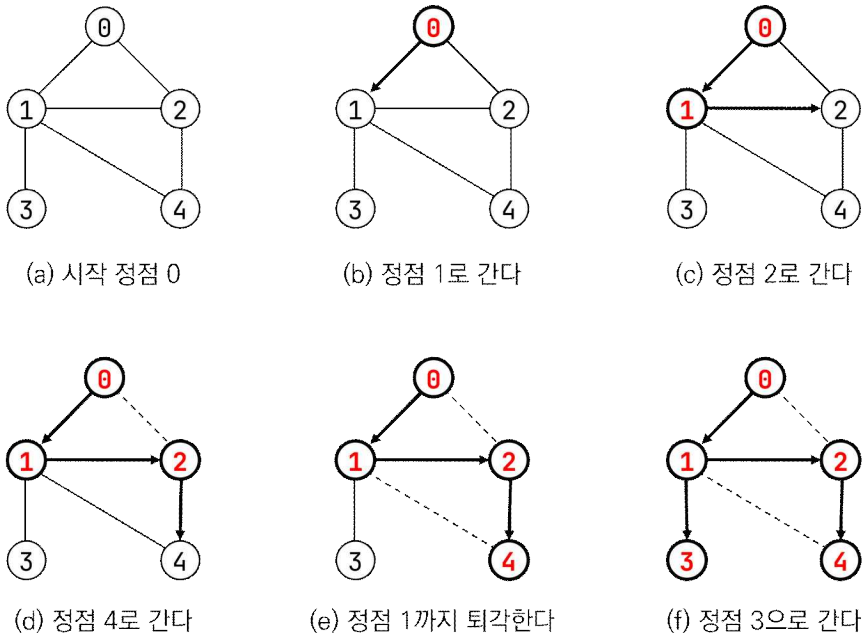


그림 11.13: 깊이 우선 탐색의 수행 과정

표 11.2: 그림 11.13 (a)의 그래프에 대한 깊이 우선 탐색의 수행 과정

현재 정점 v	방문하지 않은 인접한 정점	선택 정점 w	이전 정점 u	비고
0	1, 2	1	없음	그림 11.13 (b)
1	2, 3, 4	2	0	그림 11.13 (c)
2	4	4	1	그림 11.13 (d)
4	없음	없음	2	정점 2로 퇴각한다
2	없음	없음	1	정점 1로 퇴각한다
1	3	3	0	그림 11.13 (f)
3	없음	없음	1	정점 1로 퇴각한다
1	없음	없음	0	정점 0으로 퇴각한다
0	없음	없음	없음	탐색을 종료한다

그림 11.13에서 점선으로 표시된 선은 이미 방문한 정점을 연결하는 에지를 나타냅니다. 그림 11.13 (e)는 현재 정점 4에서는 더 이상 방문할 새로운 정점이 없으므로 정점 2로 퇴각하고, 다시 정점 1까지 퇴각한 상태를 나타냅니다. 그림 11.13 (f)에서 정점 3을 방문하고 나면 모든 정점을 방문한 상태가 되므로 계속해서 정점 1과 정점 0으로 퇴각한 후에 탐색이 종료됩니다.

깊이 우선 탐색 알고리즘은 스택을 직접 사용하여 구현할 수도 있지만 보통은 간편하게 재귀 함수로 구현합니다. 프로그램 11.3과 프로그램 11.4에는 그래프가 각각 인접 행렬과 인접 리스트로 표현되어 있을 때의 깊이 우선 탐색을 수행하는 재귀 함수 dfs()가 나와 있습니다. 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색은 인접 행렬의 모든 요소를 한 번씩 조사하므로 $O(n^2)$ 시간에 수행되고, 인접 리스트의 경우는 각 정점에 인접한 정점들만 조사하므로 $O(n+m)$ 시간에 수행됩니다.

프로그램 11.3: 인접 행렬로 표현된 그래프에 대한 깊이 우선 탐색

```
# M : 2차원 리스트로 구현된 인접 행렬 (전역 변수)
# visited : 정점 방문 여부를 기록하는 리스트, False로 초기화됨 (전역 변수)
def dfs(v):
   visit(v)                                  # 정점 v에 특정 작업을 수행
   visited[v] = True                         # 정점 v를 방문한 것으로 기록
   for w in range(len(M)):                   # 모든 정점 w에 대해서
      if M[v][w] == 1 and not visited[w]:    # 방문하지 않은 인접한 정점
         dfs(w)                              # w를 가지고 호출함
```

프로그램 11.4: 인접 리스트로 표현된 그래프에 대한 깊이 우선 탐색

```
# G : 파이썬 리스트로 구현된 인접 리스트 (전역 변수)
# visited : 정점 방문 여부를 기록하는 리스트, False로 초기화됨 (전역 변수)
def dfs(v):
   visit(v)                    # 정점 v에 특정 작업을 수행
   visited[v] = True           # 정점 v를 방문한 것으로 기록
   for w in G[v]:              # v에 인접한 정점 w에 대해서
      if not visited[w]:       # 방문하지 않은 정점이면
         dfs(w)                # w를 가지고 호출함
```

■ 너비 우선 탐색

너비 우선 탐색도 기본적인 그래프 탐색 알고리즘으로서 다양한 문제 해결에 활용됩니다. 너비 우선 탐색은 마치 트리에서 레벨 순서로 순회하듯이, 큐를 이용하여 시작 정점에 가까운 정점부터 차례대로 탐색합니다. 탐색의 시작 정점 v가 주어질 때 너비 우선 탐색은 공백 큐를 생성하

여 다음과 같이 수행합니다.

① 시작 정점 v를 방문하고 큐에 삽입한다.
② 큐가 공백이 될 때까지 단계 ③과 ④를 반복한다.
③ 큐에서 맨 앞 정점 w를 꺼낸다.
④ w에 인접한 정점 중에서 아직 방문하지 않은 정점들을 차례로 방문하고 큐에 삽입한다.

예를 들어, 그림 11.14 (a)의 그래프를 정점 0을 시작 정점으로 하여 너비 우선 탐색을 수행하는 과정이 그림 11.14와 표 11.3에 나와 있습니다. 탐색 과정에서 인접 정점을 선택할 때 항상 번호가 작은 정점부터 조사한다고 가정합니다.

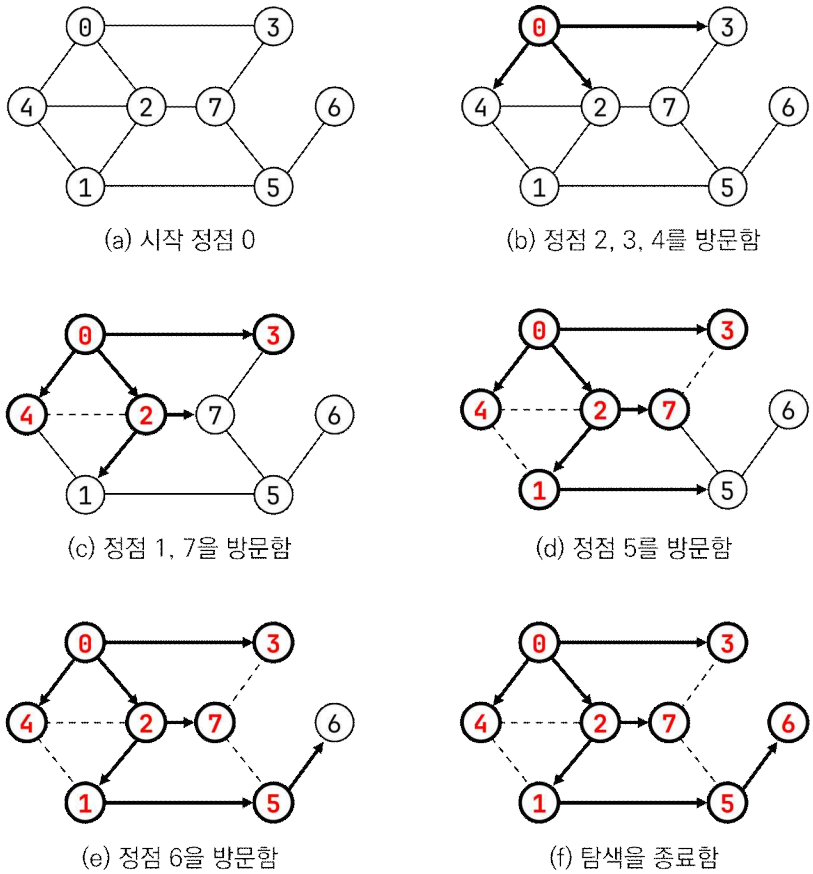


그림 11.14: 너비 우선 탐색의 수행 과정

표 11.3: 그림 11.14 (a)의 그래프에 대한 너비 우선 탐색의 수행 과정

큐 상태	큐에서 삭제할 정점	큐에 삽입할 정점	비고
[0]	0	2, 3, 4	그림 11.14 (b)
[2, 3, 4]	2	1, 7	그림 11.14 (c)
[3, 4, 1, 7]	3	없음	
[4, 1, 7]	4	없음	
[1, 7]	1	5	그림 11.14 (d)
[7, 5]	7	없음	
[5]	5	6	그림 11.14 (e)
[6]	6	없음	그림 11.14 (f)
[]	없음	없음	탐색을 종료한다

프로그램 11.5에는 그래프가 파이썬 리스트로 구현된 인접 리스트로 표현되어 있을 때의 너비 우선 탐색을 수행하는 함수 bfs()가 나와 있습니다. 프로그램에서 collections.deque를 큐 자료구조로 사용합니다.

프로그램 11.5: 인접 리스트로 표현된 그래프에 대한 너비 우선 탐색

```
1  # G : 파이썬 리스트로 구현된 인접 리스트 (전역 변수)
2  # visited : 정점 방문 여부를 기록하는 리스트, False로 초기화됨 (전역 변수)
3  from collections import deque
4
5  def bfs(v):
6     queue = deque()                # 공백 큐 생성
7     visit(v)                       # 정점 v에 특정 작업을 수행
8     visited[v] = True              # 정점 v를 방문한 것으로 기록
9     queue.append(v)                # 시작 정점 v를 큐에 삽입
10    while len(queue) > 0:          # 큐가 공백이 될 때까지 반복
11       v = queue.popleft()         # 큐에서 정점 v 삭제
12       for w in G[v]:              # v에 인접한 정점 w에 대해서
13          if not visited[w]:       # 방문하지 않은 정점이면
14             visit(w)              # w에 특정 작업을 수행
15             visited[w] = True     # w를 방문한 것으로 기록
16             queue.append(w)       # w를 큐에 삽입
```

그래프가 인접 행렬 M으로 표현된 경우, 프로그램 11.5의 for 문(12~13줄)을 다음과 같이 바꾸면 됩니다.

```
for w in range(n):                          # 모든 정점 w에 대해서
  if M[v][w] == 1 and not visited[w]:       # 방문하지 않은 인접한 정점이면
```

깊이 우선 탐색과 마찬가지로 인접 행렬로 표현된 그래프에 대한 너비 우선 탐색은 인접 행렬의 모든 요소를 한 번씩 조사하므로 $O(n^2)$ 시간에 수행되고, 인접 리스트의 경우는 각 정점에 인접한 정점들만 조사하므로 $O(n+m)$ 시간에 수행됩니다.

11.4 그래프 탐색의 활용

그래프 탐색 알고리즘은 다양한 그래프 문제를 해결하는 데 활용됩니다. 깊이 우선 탐색(DFS)은 하나의 경로를 최대한 따라가면서 탐색하기 때문에, 경로 탐색, 사이클 탐지, 연결 요소 찾기 등의 문제 해결에 유용합니다. 반면에 너비 우선 탐색(BFS)은 시작 정점에서 가까운 정점부터 탐색하기 때문에, 최단 경로 찾기와 계층 구조 탐색 등의 문제 해결에 유용합니다. 이 절에서는 연결 요소, 신장 트리, 최단 경로 트리에 관련된 문제와 그래프 탐색을 이용한 해법을 소개합니다.

■ 연결 요소

연결 그래프는 모든 정점이 서로 연결된 그래프이고, 연결 요소는 최대로 연결된 부그래프입니다. 따라서 연결 요소가 하나만 있는 그래프는 연결 그래프입니다. 그래프에서 연결 요소를 찾는 문제는 실세계의 다양한 분야에서 발생할 수 있습니다. 몇 가지 예시를 들면 다음과 같습니다.

- 소셜 네트워크에서 친구 관계인 사용자들의 그룹을 구하라.
- 컴퓨터 네트워크에서 장애가 발생했을 때, 서로 통신이 가능한 컴퓨터들의 그룹을 찾아라.
- 전력, 철도, 도로, 상하수도와 같은 국가 기간망에서 고립된 지역을 식별하라.
- 이미지에서 동일한 속성(색상, 질감)을 가진 연속된 픽셀들의 그룹을 찾아라.

깊이 우선 탐색과 너비 우선 탐색은 에지를 통해 탐색을 수행하기 때문에, 시작 정점과 연결된 모든 정점을 방문합니다. 따라서 탐색이 종료된 후에 방문한 정점들은 시작 정점이 포함된 연결 요소를 구성합니다. 그래프에서 모든 연결 요소를 찾기 위해서는 그래프의 모든 정점에 대해 한

번씩 탐색을 수행해야 합니다. 프로그램 11.6의 함수 all_components()는 그래프의 모든 연결 요소를 찾아서 한 줄에 하나씩 연결 요소에 포함된 정점들을 오름차순으로 출력합니다. 이 함수는 앞서 구현한 깊이 우선 탐색을 수행하는 재귀 함수 dfs() 또는 너비 우선 탐색을 수행하는 함수 bfs()를 호출합니다. dfs() 또는 bfs()에서 호출하는 함수인 visit(v)는 정점 v를 현재 연결 요소에 포함하는 작업을 수행합니다.

프로그램 11.6: 모든 연결 요소를 출력하는 프로그램

```
# M 또는 G : 인접 행렬 또는 인접 리스트 (전역 변수)
# visited : 정점 방문 여부를 기록하는 리스트, False로 초기화됨 (전역 변수)
# CC : 연결 요소에 포함되는 정점들을 저장하는 리스트 (전역 변수)
def visit(v):                   # dfs() 또는 bfs()에서 호출
   CC.append(v)                 # v를 연결 요소에 추가

def all_components(n):          # 모든 연결 요소를 출력, n은 정점의 수
   for v in range(n):           # 각 정점 v에 대해서
      if not visited[v]:        # v를 아직 방문하지 않았으면 수행
         CC.clear()             # 리스트를 공백 상태로 설정함
         dfs(v)                 # bfs()를 호출해도 됨
         print(*sorted(CC))     # 현재 연결 요소를 정렬하여 출력
```

프로그램 11.6의 수행에서 인접 행렬로 표현된 그래프는 인접 행렬의 모든 요소를 한 번씩 조사하므로 $O(n^2)$ 시간에 수행되고, 인접 리스트의 경우는 각 정점에 인접한 정점들만 조사하므로 $O(n+m)$ 시간에 수행됩니다.

■ 신장 트리

그래프의 **신장 트리(spanning tree)**는 모든 정점을 포함하는 트리 구조의 부그래프입니다. 트리는 사이클이 없는 연결 그래프이므로, 주어진 그래프가 연결 그래프여야만 신장 트리가 존재합니다. 신장 트리는 최소 비용으로 전체를 연결하는 네트워크를 구성하려는 응용에서 사용됩니다. 예를 들어, 여러 대의 PC를 서로 연결하여 근거리 통신망(LAN)을 구축하는 작업에서 서로 연결할 PC 쌍의 개수를 줄이면 비용이 절감됩니다. 신장 트리는 최소 개수의 에지를 사용하여 그래프의 모든 정점을 연결하는 구조이기 때문에, PC를 정점으로, PC들 사이에 연결이 가능한 쌍을 에지로 만든 그래프에서 신장 트리를 찾는 것이 LAN 구축 비용을 줄이는 해법이 됩니다. 이와 같이 신장 트리는 전기 배선, 수도관 연결 등 모든 종류의 네트워크에서 최소한의 연결만으로 네트워크를 구성하려는 응용에 활용됩니다.

일반적인 그래프에서 찾을 수 있는 신장 트리의 종류는 매우 많습니다. 그림 11.15는 그림 11.13의 그래프에서 찾을 수 있는 모든 신장 트리를 나열한 것입니다.

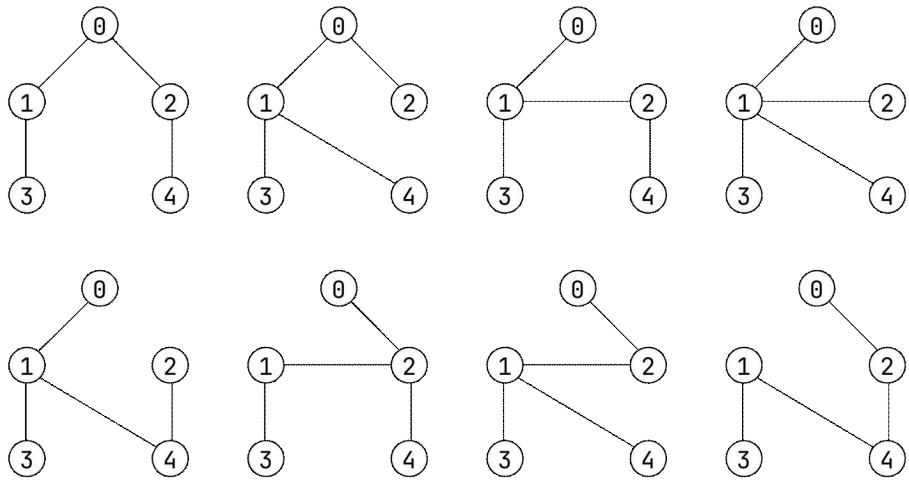
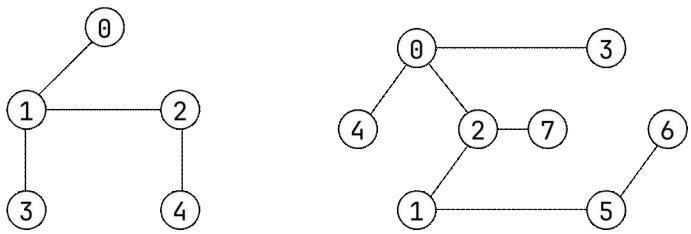


그림 11.15: 그림 11.13 그래프의 모든 신장 트리

주어진 그래프의 신장 트리 중 하나를 찾는 문제는 어려운 문제처럼 보이지만, 그래프 탐색 알고리즘을 이용하면 간단히 해결됩니다. 그래프 탐색 알고리즘에서 새로운 정점을 방문할 때 사용한 에지들을 모두 모으면, 즉 정점 v에서 새로 방문할 정점 w를 찾아갈 때 사용한 에지 (v, w)를 모두 모으면 신장 트리가 됩니다. 깊이 우선 탐색을 수행하면서 모은 에지들로 구성된 신장 트리를 **깊이 우선 신장 트리(depth first spanning tree)**라고 부르고, 너비 우선 탐색으로 모은 에지들로 구성된 신장 트리를 **너비 우선 신장 트리(breadth first spanning tree)**라고 부릅니다. 그림 11.16은 표 11.2와 표 11.3의 탐색 순서에 의해 만들어진 깊이 우선 신장 트리와 너비 우선 신장 트리를 보여줍니다.



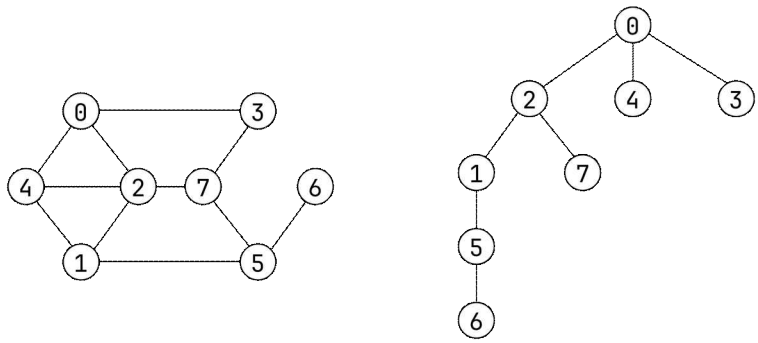
(a) 깊이 우선 신장 트리 (b) 너비 우선 신장 트리

그림 11.16: 표 11.2와 표 11.3의 탐색 순서에 의해 생성된 신장 트리

결론적으로, 주어진 그래프에서 하나의 신장 트리를 찾는 작업은 그래프가 인접 행렬로 표현된 경우는 $O(n^2)$ 시간, 인접 리스트로 표현된 경우는 $O(n+m)$ 시간에 수행됩니다.

■ 최단 경로

경로를 구성하는 에지의 개수를 경로의 길이로 정의하는 비가중치 그래프[28]에서의 최단 경로는 너비 우선 탐색을 이용하여 구할 수 있습니다. 너비 우선 탐색은 시작 정점에서 가까운 정점부터 차례로 방문하기 때문에, 정점을 방문하는 시점에서 그 정점까지의 최단 경로를 알 수 있습니다. 따라서 경로의 출발점을 탐색의 시작 정점으로 하여 너비 우선 탐색을 수행하면 출발점에서 모든 다른 정점까지의 최단 경로를 구할 수 있습니다. 모든 최단 경로를 한꺼번에 나타내면 출발점을 루트로 하는 트리로 표현됩니다. 이 트리를 **최단 경로 트리(shortest path tree)**라고 부릅니다. 그림 11.17은 그래프와 그 그래프에서 정점 0을 출발점으로 하는 최단 경로 트리의 예를 보여줍니다.



(a) 그래프 예 (출발점 0) (b) 최단 경로 트리

그림 11.17: 최단 경로 트리의 예

너비 우선 탐색에서 정점 방문 순서가 일치하면, 최단 경로 트리와 너비 우선 신장 트리는 동일한 구조로 구성됩니다. 표 11.3의 탐색 순서로 수행한 탐색의 결과로 그림 11.16의 너비 우선 신장 트리와 그림 11.17의 최단 경로 트리는 구조가 동일합니다. 최단 경로 트리는 루트가 있는 트리이고 경로 정보를 구하는 것이 목적이기 때문에, 부모 포인터 배열로 간편하게 표현하면 됩니다. 그림 11.17의 최단 경로 트리는 다음과 같이 부모 포인터 배열로 표현됩니다.

---

28) 11장에 소개된 그래프는 모두 비가중치 그래프이고, 가중치 그래프(weighted graph)는 에지에 가중치가 배정된 그래프로서 12장에서 다룹니다.

0	1	2	3	4	5	6	7
-1	2	0	0	0	1	5	2

부모 포인터 배열 이름이 parent라고 할 때, 최단 경로 트리 정보를 parent 배열에 저장하는 프로그램은 프로그램 11.5에서 함수 visit(v)을 호출하는 명령을 아래 명령으로 바꾸기만 하면 됩니다.

```
parent[w] = v          # 정점 v를 정점 w의 부모로 설정함
```

수정된 함수 bfs()를 호출하기 전에, parent를 -1로 초기화된 길이가 n인 리스트로 만들어 주는 것이 필요합니다.

프로그램 11.7의 함수 get_path(v)는 목적지 정점이 v인 최단 경로를 담은 리스트를 반환합니다. 이 함수는 최단 경로 트리 정보가 저장된 부모 포인터 배열 parent로부터 최단 경로를 찾습니다. 종착점 v부터 루트를 만날 때까지 계속 부모를 찾아가면서 만나는 정점들을 리스트 path에 삽입합니다. 마지막으로, 리스트 path를 역순으로 만들어, 출발점부터 종착점까지 정점들이 순서대로 배치된 리스트를 반환합니다.

프로그램 11.7: 최단 경로 트리를 찾는 프로그램

```
1 def get_path(v):                  # 종착점이 v인 최단 경로를 구하는 함수
2     path = [v]                    # 종착점을 삽입
3     while parent[v] != -1:        # v가 루트가 될 때까지 반복
4         v = parent[v]             # 부모 노드로 올라감
5         path.append(v)            # path에 v를 추가함
6     path.reverse()                # 역순으로 배치
7     return path                   # 최단 경로 리스트를 반환
```

11.5 사례 연구: 위상 정렬

대규모 프로젝트의 수행 계획을 세운다고 가정해 봅시다. 프로젝트는 많은 소규모 작업으로 구성되고 각 작업 사이에는 선행 관계가 정해져 있습니다. 프로젝트 수행 계획표를 그래프로 표현한다면, 각 작업은 그래프의 정점으로 표현되고 작업 사이의 선행 관계는 방향 에지로 표현됩니

다. 이 방향 그래프에는 방향 사이클이 존재하지 않습니다. 방향 사이클이 존재하면 작업 사이에 선행 관계가 성립하지 않기 때문입니다. 이러한 방향 사이클이 없는 방향 그래프를 **방향 비순환 그래프(directed acyclic graph; DAG)**라고 부르며, 그래프의 특수한 유형으로 자주 나타납니다. 방향 비순환 그래프는 방향 사이클이 없으므로, 전입 차수가 0인 정점과 전출 차수가 0인 정점이 각각 반드시 하나 이상 존재합니다. 그림 11.18은 방향 비순환 그래프의 예를 보여줍니다.

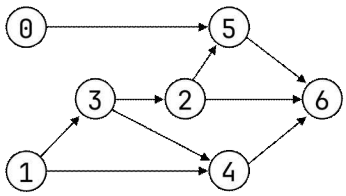


그림 11.18: 방향 비순환 그래프의 예

방향 비순환 그래프의 각 정점을 선행 조건을 유지하면서 순차적으로 나열한 순서를 **위상 순서(topological sequence)**라고 부르며, 위상 순서를 구하는 절차를 **위상 정렬(topological sort)**이라고 합니다. 위상 순서는 여러 개가 존재할 수 있습니다. 예를 들어, 다음의 각 순서는 모두 그림 11.18의 그래프에 대한 유효한 위상 순서입니다.

- `0, 1, 3, 2, 4, 5, 6`
- `1, 0, 3, 4, 2, 5, 6`
- `1, 3, 4, 2, 0, 5, 6`

방향 비순환 그래프에 대한 위상 순서는 전입 차수가 0인 정점부터 나열되어야 하므로, 위상 정렬 과정을 다음과 같이 간략하게 나타낼 수 있습니다.

> 전입 차수가 0인 정점 v를 찾아 출력하고, v와 v에 부속된 모든 에지를 삭제한다. 이 작업을 모든 정점이 출력될 때까지 반복한다.

위상 정렬을 효율적으로 구현하기 위해, 초기에 각 정점의 현재 전입 차수를 저장한 배열 `indeg`를 생성하고, 전입 차수가 0인 모든 정점을 스택(또는 큐)에 저장합니다. 그다음 과정은 스택이 공백이 될 때까지 다음을 반복하는 것입니다.

> 스택(또는 큐)에서 삭제한 정점 v를 위상 순서에 추가한다. 그리고 v에 인접한 각 정점 w에 대해 `indeg[w]`를 1 감소시키고, `indeg[w]`가 0이면 w를 스택에 저장한다.

그림 11.19에는 그림 11.18의 그래프를 대상으로 스택을 사용하여 위상 정렬을 수행하는 과정을 보여줍니다. 위상 순서는 정점이 스택에 삽입되는 순서 또는 스택에서 삭제되는 순서입니다. 스택에 삽입되는 순서는 (0, 1, 3, 2, 4, 5, 6)이며, 스택에서 삭제되는 순서는 (1, 3, 4, 2, 0, 5, 6)입니다.

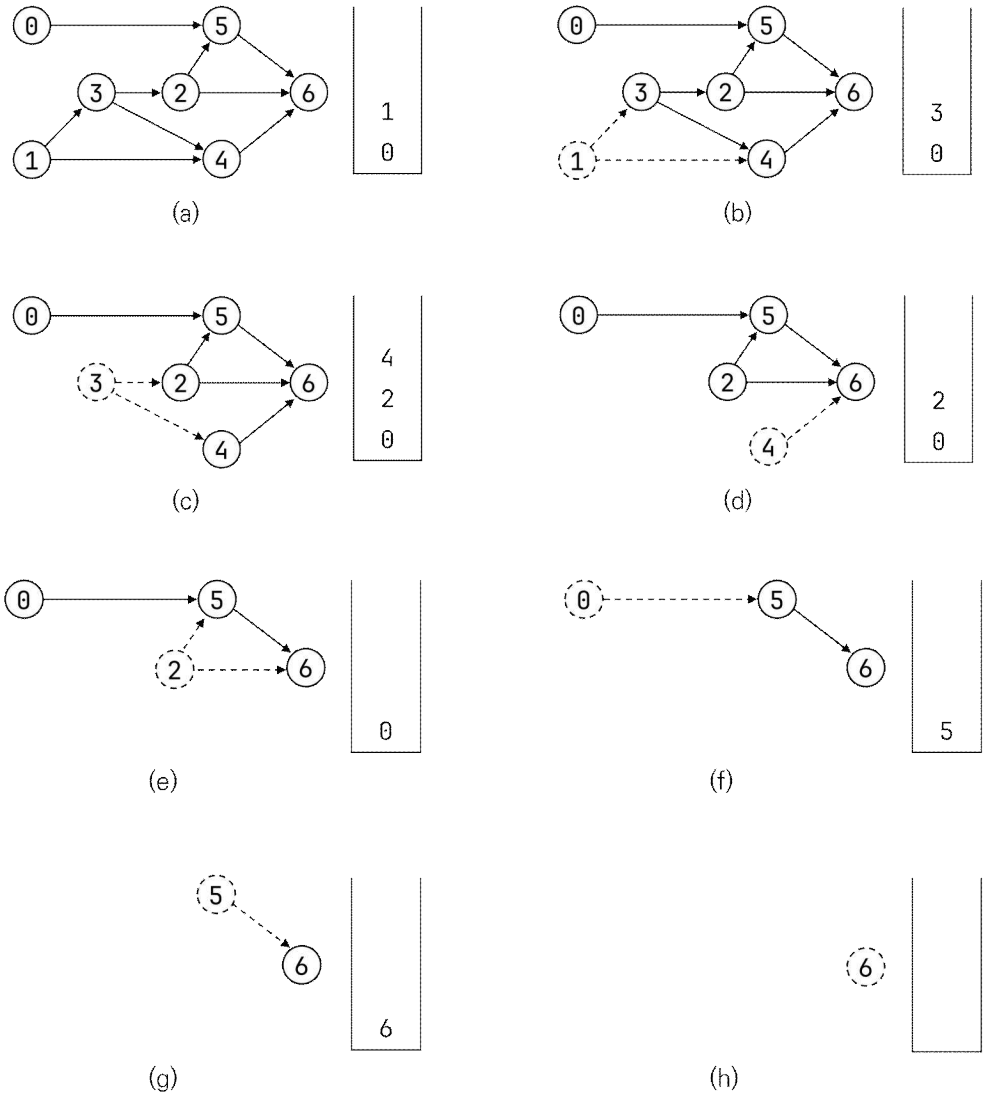


그림 11.19: 위상 정렬 과정 : (a) 전입 차수가 0인 정점 0과 1을 스택에 삽입, (b) 정점 1을 삭제하고 스택에 3을 삽입, (c) 정점 3을 삭제하고 스택에 2와 4를 삽입, (d) 정점 4 삭제, (e) 정점 2 삭제 (f) 정점 0을 삭제하고 스택에 5를 삽입, (g) 정점 5를 삭제하고 스택에 6을 삽입, (h) 6을 삭제하고 작업을 종료함

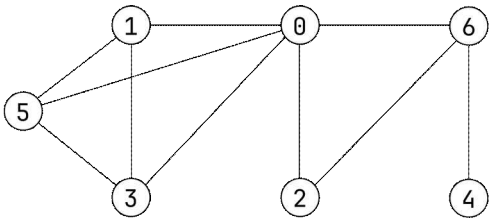
프로그램 11.8에는 방향 비순환 그래프에 대해서 위상 정렬을 반환하는 함수 topological_sort()가 나와 있습니다. 프로그램에서 방향 그래프가 인접 리스트로만 표현되어 있다고 가정합니다(즉, 역 인접 리스트를 사용하지 않습니다). 프로그램의 수행 과정에서 초기에 전입 차수를 구하기 위해 각 에지를 한 번씩 조사하고, 위상 정렬 과정에서 각 에지가 한 번씩 조사됩니다. 각 정점은 위상 정렬 과정에서 한 번씩 스택에 삽입된 후 삭제됩니다. 따라서 프로그램의 시간복잡도는 $O(n+m)$입니다. 만일 그래프가 인접 행렬로 표현되면 위상 정렬은 $O(n^2)$ 시간에 수행됩니다.

프로그램 11.8: 위상 정렬을 수행하는 프로그램

```
1  def topological_sort(G):
2     """인접 리스트 G로 표현된 방향 비순환 그래프에서 구한 위상 순서를 반환한다."""
3     n = len(G)                  # 정점의 개수
4     toseq = []                  # 위상 순서를 저장하는 리스트
5     indeg = [0] * n             # 전입 차수를 저장하는 리스트
6     stack = []                  # 공백 스택 생성 (리스트 이용)
7
8     for v in range(n):          # 각 정점의 전입 차수를 구함
9        for w in G[v]:
10          indeg[w] += 1
11
12    for v in range(n):          # 전입 차수가 0인 정점을 스택에 삽입
13       if indeg[v] == 0:
14          stack.append(v)
15
16    while len(stack) > 0:       # 스택이 공백이 될 때까지 반복
17       v = stack.pop()          # 스택의 top 정점 v 삭제
18       toseq.append(v)          # 위상 순서에 정점 v 추가
19       for w in G[v]:              # 방향 에지 <v, w>에 대해
20          indeg[w] -= 1         # 전입 차수 1 감소
21          if indeg[w] == 0:     # 전입 차수가 0이면 스택에 삽입
22             stack.append(w)
23
24    return toseq                # 위상 순서 반환
```

연습문제

1. 아래 그림의 그래프에 대해서 다음 물음에 답하시오.
   (1) 인접 행렬로 표현하시오.
   (2) 인접 리스트로 표현하시오.
   (3) 각 에지에 방향을 주어 강 연결 방향 그래프로 만드시오.
   (4) 이 그래프는 평면 그래프인가?



2. 6개의 정점을 가진 완전 그래프를 그리시오.

3. 7개 정점의 차수가 다음과 같은 단순 그래프를 그리시오.

   (4, 4, 3, 2, 2, 1, 0)

4. 12개의 정점과 3개의 연결 요소를 가진 단순 그래프가 가질 수 있는 에지의 최대 개수는 얼마인가?

5. 문제 1의 그래프에 대해서 다음 물음에 답하시오.
   (1) 깊이 우선 탐색을 정점 0에서부터 시작할 때, 정점들의 방문 순서를 쓰시오.
   (2) 너비 우선 탐색을 정점 0에서부터 시작할 때, 정점들의 방문 순서를 쓰시오.
   (3) 정점 0을 출발점으로 하는 최단 경로 트리를 그리시오.

6. 깊이 우선 탐색을 재귀 함수가 아닌 명시적인 스택을 이용하는 반복문으로 구성된 프로그램을 구현하시오.

7. 그래프에서 두 정점 사이의 **거리(distance)**는 그 두 정점 사이의 최단 경로의 길이를 말하고, **지름(diameter)**은 가장 멀리 떨어져 있는 두 정점 사이의 거리를 말한다. 문제 1에 있는 그래프의 지름을 구하시오.

8. 연결 그래프의 어떤 정점과 그에 부속된 에지들을 제거할 때 남아 있는 그래프가 연결되지 않은 상태가 되면 그 정점을 **절단 정점(cut vertex)**라고 부른다. 문제 1의 그래프에서 절단 정점을 모두 나열하시오.

9. 주어진 연결 그래프에서 모든 절단 정점을 찾는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

10. 트리 T에 대한 다음의 문장들이 모두 동치임을 증명하시오.
    ① T는 사이클이 없는 연결 그래프이다.
    ② T는 에지의 개수가 정점의 개수보다 하나 적은 연결 그래프이다.
    ③ T는 에지가 하나라도 제거되면 연결되지 않은 그래프가 된다.
    ④ T는 에지가 하나 추가되면 항상 사이클이 발생하는 그래프이다.
    ⑤ T는 임의의 두 정점 사이의 단순 경로가 하나뿐인 그래프이다.

11. 2개 이상의 정점을 가진 모든 트리에는 차수가 1인 정점이 적어도 두 개 이상 있음을 증명하시오.

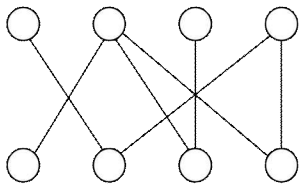
12. 그래프에 사이클이 존재하는지를 판별하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

13. 방향 그래프에 방향 사이클이 존재하는지를 판별하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

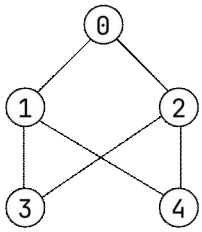
14. 방향 그래프와 두 정점 `start`와 `end`가 주어질 때, `start`에서 `end`로 가는 방향 경로가 존재하는지를 판별하는 프로그램을 작성하시오.

15. 연결 그래프에서 임의의 정점과 그에 부속된 에지들을 제거하더라도 남아 있는 그래프가 연결된 상태가 되면 **이중 연결 그래프(biconnected graph)**라고 부른다. 주어진 그래프가 이중 연결 그래프인지를 판별하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

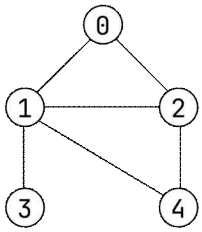
16. 그래프의 모든 정점을 두 개의 그룹 A와 B로 나눌 때, 모든 에지가 그룹 A와 B 사이에만 존재하고, 같은 그룹 내의 정점 사이에는 하나도 없는 경우, 이 그래프를 **이분 그래프(bipartite graph)**라고 한다(아래 그림 참고). 주어진 그래프가 이분 그래프인지 판별하는 알고리즘을 설계하시오.



〈이분 그래프〉



〈이분 그래프〉



〈이분 그래프 아님〉

17. [실습 문제] n개의 섬과 섬들 사이를 연결하는 m개의 다리가 있다. 섬들은 1부터 n까지 번호로 구분되고, 다리는 양방향으로 건널 수 있다. 다리는 항상 다른 두 섬을 연결하고, 두 섬 사이에 다리는 기껏해야 하나가 있다. 임의의 섬에서 출발하여 4개 이상의 다리를 건너서 다시 출발 섬으로 돌아오는 경로를 찾으려고 한다. 이때, 경로에 있는 다리는 모두 달라야 하며, 경로의 시작인 출발 섬을 제외하면 경로가 지나는 섬들은 모두 달라야 한다. 그러한 경로가 존재하는지 판별하는 프로그램을 작성하시오.

입력으로, 첫째 줄에 섬의 수 n과 다리의 수 m이 주어진다. n은 1 이상 1,000 이하이고, m은 1 이상 5,000 이하이다. 그다음 m개의 줄에는 각 줄에 각 다리가 연결하는 두 섬의 번호가 들어온다. 출력으로, 적어도 4개 이상의 다리를 건너서 출발 섬으로 돌아오는 경로가 존재하면 YES, 존재하지 않으면 NO를 출력한다.

입력의 예	출력의 예
5 6 1 2 2 3 3 4 4 5 5 3 1 5	YES

입력의 예	출력의 예
5 6 1 2 1 3 2 3 3 4 3 5 4 5	NO

왼쪽의 입출력 예는 다섯 개의 다리를 건너는 경로 (1, 2, 3, 4, 5, 1)이 존재하고, 오른쪽 예는 조건을 만족하는 경로의 최대 길이가 3이다.

18. [실습 문제] 시작 단어, 목표 단어, 그리고 단어 목록이 주어질 때, 한 번에 한 글자만 바꿔서 시작 단어에서 목표 단어로 변환하는 가장 짧은 경로의 길이를 구하는 프로그램을 작성하시오. 각 변환 단계의 단어는 반드시 단어 목록에 존재해야 합니다. 예를 들어, 단어 목록이 다음과 같다고 하자.

    ```
    bat, cat, cog, dog, dot, fog, hit, hot
    ```

    시작 단어가 hit이고 목표 단어가 cog일 때, 목표 단어로 변환하는 가장 짧은 경로는 다음과 같다. 경로의 길이는 변환 횟수로 정의된다.

    ```
    hit → hot → dot → dog → cog
    ```

    입력으로, 단어 목록의 길이를 나타내는 정수 n이 주어진다. n은 3 이상 50 이하이다. 그다음 n개의 줄에는 각 줄에 단어 목록에 포함될 단어가 들어온다. 단어는 모두 알파벳 소문자로만 구성되고, 길이는 모두 같으며 3 이상 10 이하이다. 단어 목록에 중복된 단어는 없다. 그다음 줄에는 시작 단어, 마지막 줄에는 목표 단어가 들어온다. 시작 단어와 목표 단어는 서로 다르며, 둘 다 단어 목록에 속해 있다. 출력으로, 시작 단어에서 목표 단어로 변환하는 가장 짧은 경로의 길이를 출력한다. 만일, 목표 단어로 변환이 불가능하면 0을 출력한다.

입력의 예	출력의 예
8 bat cat cog dog dot fog hit hot hit cog	4

입력의 예	출력의 예
8 bat cat cog dog dot fog hit hot bat dog	0

제12장

그래프 응용

12.1 가중치 그래프
12.2 최소 신장 트리
12.3 최단 경로 문제
12.4 사례 연구 : 작업 네트워크
연습문제

학습 목표

- 가중치 그래프의 개념을 이해하고, 가중치 그래프 표현 방식인 가중치 행렬과 가중치 인접 리스트를 비교 설명할 수 있다.
- 최소 신장 트리의 개념을 이해하고, 프림 알고리즘과 크루스칼 알고리즘의 동작 원리를 비교 설명할 수 있으며, 직접 프로그램으로 구현할 수 있다.
- 최단 경로를 구하는 다익스트라 알고리즘과 플로이드-와샬 알고리즘의 동작 원리를 설명할 수 있으며, 직접 프로그램으로 구현할 수 있다.
- 작업 네트워크에서 프로젝트 일정에 영향을 주는 임계 경로와 중대 작업을 구할 수 있다.

12.1 가중치 그래프

많은 응용에서는 그래프의 에지에 **가중치(weight)**를 부여합니다. 예를 들어, 고속도로망을 그래프로 표현하는 경우, 각 도시는 정점으로, 도시 사이의 고속도로는 에지로 표시됩니다. 이때 고속도로의 길이 또는 통과 시간 등을 에지에 표시할 수 있는데, 이러한 값을 에지의 가중치라고 하며, 에지에 가중치가 부여된 그래프를 **가중치 그래프(weighted graph)**라고 부릅니다. 그림 12.1은 가중치 무향 그래프와 가중치 방향 그래프의 예를 보여줍니다. 비가중치 그래프에서는 경로의 길이를 에지의 개수로 정의하지만, 가중치 그래프에서는 경로의 길이를 에지에 부여된 가중치의 합으로 정의합니다. 따라서 그림 12.1의 가중치 그래프에서 경로 (0, 1, 2, 3)의 길이는 70 = 10 + 20 + 40입니다.

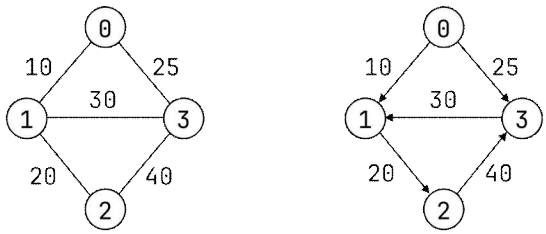


그림 12.1: 가중치 그래프의 예

가중치 그래프를 표현할 때 인접 행렬을 **가중치 행렬(weight matrix)**로 대체할 수 있습니다. 가중치 행렬은 각 정점 쌍에 대해 가중치를 표시한 행렬입니다. 에지 (v, w)의 가중치가 두 정점 v와 w 사이에서 발생하는 이득(profit)을 나타내는 경우도 간혹 있지만, 비용(cost)을 의미하는 경우가 대부분입니다. 가중치를 비용으로 보는 경우, 가중치 행렬을 **비용 행렬(cost matrix)**이라고 부르기도 합니다. 비용의 관점에서 가중치 행렬의 표현은 다음과 같습니다. 가중치 행렬을 M이라고 하고, 에지 (i, j)의 가중치를 cost(i, j)라고 합시다. 그래프에서 에지 (i, j)가 존재하면 M[i][j] = cost(i, j)이고, 그렇지 않으면 M[i][j] = ∞입니다. 무한대(∞) 가중치는 에지가 존재하지 않는다는 것을 의미합니다. 무향 에지는 (i, j) = (j, i)이므로, M[i][j] = M[j][i]입니다. 셀프 루프가 없는 단순 그래프의 가중치 행렬은 주대각선 위치에 있는 M[i][i]에 가중치 0이 배정됩니다. 가중치 0은 비용이 전혀 들지 않는다는 것을 의미합니다. 가중치 방향 그래프의 가중치 행렬에서 M[i][j]는 에지 ⟨i, j⟩의 가중치입니다. 방향 에지는 ⟨i, j⟩ ≠ ⟨j, i⟩이므로, M[i][j]와 M[j][i]는 값이 다를 수 있습니다. 그림 12.2는 그림 12.1의 가중치 그래프에 대한 가중치 행렬을 보여줍니다.

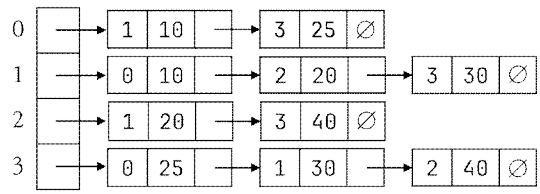
	0	1	2	3
0	0	10	∞	25
1	10	0	20	30
2	∞	20	0	40
3	25	30	40	0

	0	1	2	3
0	0	10	∞	25
1	∞	0	20	∞
2	∞	∞	0	40
3	∞	30	∞	0

그림 12.2: 그림 12.1의 가중치 그래프에 대한 가중치 행렬

가중치 행렬을 파이썬 리스트로 구현할 때, ∞가 들어가는 위치에 파이썬에서 무한대를 의미하는 값인 `float('inf')`를 넣거나, 아니면 `None`을 사용할 수 있습니다.

가중치 그래프를 인접 리스트로 표현하는 경우, 각 정점 v에 대한 선형리스트의 요소는 v에 인접한 정점 w와 cost(v, w)로 구성됩니다. 그림 12.3에는 그림 12.1의 가중치 무향 그래프에 대해, 선형리스트를 연결리스트로 구현한 예와 파이썬 리스트로 구현한 예가 나와 있습니다.




```
[[(1, 10), (3, 25)],
 [(0, 10), (2, 20), (3, 30)],
 [(1, 20), (3, 40)],
 [(0, 25), (1, 30), (2, 40)]]
```

그림 12.3: 그림 12.1의 가중치 무향 그래프에 대한 가중치 인접 리스트

가중치 무향 그래프 데이터에 대한 입력이 다음과 같은 형식으로 주어진다고 합시다: 첫 줄과 둘째 줄에 각각 정점 수 n과 에지 수 m이 주어지고, 그다음 줄부터 m개의 줄에는 각 줄에 하나씩 에지 (v, w)와 가중치 c를 나타내는 세 정수 v, w, c가 주어집니다. 예를 들어, 그림 12.1의 가중치 무향 그래프를 위한 입력은 다음과 같습니다.

```
4          정점 수
5          에지 수
0 1 10     에지 1
0 3 25     에지 2
1 2 20     에지 3
1 3 30     에지 4
2 3 40     에지 5
```

프로그램 12.1과 프로그램 12.2에는 위 형식으로 주어진 가중치 무향 그래프 데이터를 읽어서 각각 가중치 행렬과 가중치 인접 리스트를 구축하는 프로그램이 나와 있습니다. 가중치 인접 리스트는 파이썬 리스트를 이용하여 구축합니다.

프로그램 12.1: 가중치 무향 그래프의 가중치 행렬을 구축하는 프로그램

```
def create_cost_matrix():                          # 가중치 행렬을 구축하는 함수
   n = int(input())                                # 정점 수
   m = int(input())                                # 에지 수
   M = [[float('inf')]*n for _ in range(n)]        # 가중치 행렬의 초기화
   for v in range(n):                              # 주대각선은 0으로 설정
      M[v][v] = 0
   for _ in range(m):                              # m개 에지 입력
      v, w, c = map(int, input().split())          # 에지 (v, w)와 가중치 c
      M[v][w] = M[w][v] = c                        # 에지 가중치 설정
   return M                                        # 가중치 행렬 반환
```

프로그램 12.2: 가중치 무향 그래프의 인접 리스트를 구축하는 프로그램

```
def create_weighted_adj_list():          # 가중치 인접 리스트를 구축하는 함수
   n = int(input())                      # 정점 수
   m = int(input())                      # 에지 수
   G = [[] for _ in range(n)]            # 인접 리스트를 위한 2차원 리스트
   for _ in range(m):                    # m개 에지 입력
      v, w, c = map(int, input().split())     # 에지 (v, w)와 가중치 c
      G[v].append((w, c))                # v에 대한 리스트에 (w, c) 추가
      G[w].append((v, c))                # w에 대한 리스트에 (v, c) 추가
   return G                              # 가중치 인접 리스트 반환
```

12.2 최소 신장 트리

신장 트리는 그래프의 모든 정점을 연결하는 트리 형태의 부그래프입니다. 트리는 최소 개수의 에지로서 전체를 연결하는 구조이기 때문에, 도로망, 전화망, 상하수도망 등의 응용에서 최소의 비용으로 전체를 연결하는 네트워크를 구축하고자 할 때 신장 트리를 찾게 됩니다. 그런데 이러한 응용에서는 직접 연결해야 할 구간의 개수뿐만 아니라 어떤 구간을 선택하느냐에 따라 비용이 달라집니다. 그래서 이들 응용에서는 비용까지 고려할 수 있는 가중치 그래프를 사용합니다.

예를 들어, 그림 12.4의 가중치 그래프가 고속도로망을 구축하려는 목적으로 생성되었다고 가정해 봅시다. 그래프의 정점은 도시에 대응하고 에지는 고속도로를 설치하려고 하는 구간의 후보를 나타냅니다. 에지의 가중치는 해당 구간에 도로를 설치할 때 필요한 예측 비용입니다. 에지가 표시되어 있지 않은 곳은 해당 구간의 공사가 불가능하거나 공사 비용이 너무 많아 고려할 필요가 없다는 것을 의미합니다.

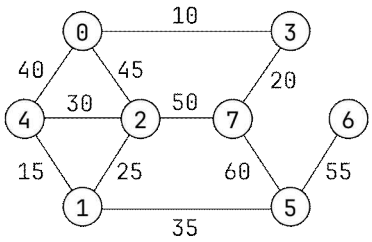
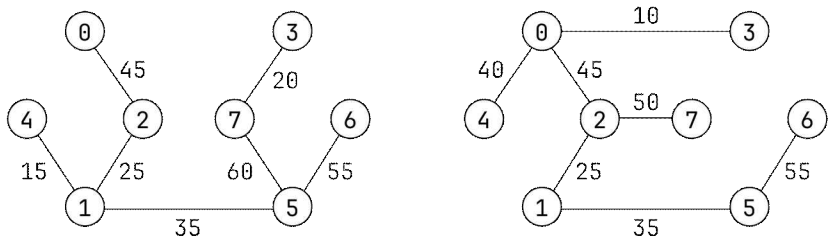


그림 12.4: 가중치 그래프의 예

문제는 최소한의 예산을 사용하여 모든 도시를 연결하는 고속도로망을 구축하는 것입니다. 공사 비용을 줄이기 위해서는 최소 개의 공사 구간만으로 모든 도시를 연결하는 신장 트리를 찾아야 합니다. 그런데 어떤 에지를 선택하느냐에 따라 신장 트리의 가중치 합이 달라집니다. 그림 12.5 (a)와 같이 깊이 우선 신장 트리를 구성하면 가중치 합이 255가 되고, 그림 12.5 (b)와 같이 너비 우선 신장 트리를 구성하면 가중치 합은 260이 됩니다. 이 외에도 신장 트리는 매우 많이 발견될 수 있습니다. 이처럼 다양한 신장 트리 중에서 가중치 합이 최소가 되는 신장 트리를 어떻게 찾을 수 있을까요?



(a) 깊이 우선 신장 트리 (b) 너비 우선 신장 트리

그림 12.5: 그림 12.4의 가중치 그래프에 대한 신장 트리

가중치 그래프의 신장 트리 중에서 가중치 합이 최소인 신장 트리를 **최소 신장 트리(minimum spanning tree, MST)**라고 부릅니다. 최소 신장 트리를 깊이 우선 탐색이나 너비 우선 탐색과

같은 그래프 탐색만으로는 찾을 수 없습니다. 최소 신장 트리를 찾는 문제는 네트워크 설계, 회로 연결, 군사 통신망 설계 등 다양하면서도 중요한 응용 분야를 포함하고 있어 20세기 초부터 활발히 연구되었습니다. 1926년에 보루프카 알고리즘(Borůvka's algorithm)이 최초로 발표되었고, 1956년에 크루스칼 알고리즘(Kruskal's algorithm), 1957년에 프림 알고리즘(Prim's algorithm)이 차례로 발표되었습니다. 이 절에서는 널리 알려진 프림 알고리즘과 크루스칼 알고리즘에 관하여 자세하게 알아봅니다. 신장 트리는 무향 그래프에서 정의되기 때문에, 여기서 다루는 가중치 그래프는 무향 그래프입니다. 앞으로 그래프의 정점 수와 에지 수를 각각 n과 m으로 나타냅니다.

■ 프림 알고리즘

알고리즘을 설명하기 전에, 프림 알고리즘의 이론적 배경을 제공하는, **절단 속성(cut property)**이라고 불리는 다음의 문장을 살펴봅시다.

> 그래프 G가 연결된 가중치 그래프이고, G의 정점들을 분리하여 만든 두 집합을 각각 A와 B라고 하자. 이때 A와 B에 끝점이 하나씩 있는 에지 중에서 가중치가 최소인 에지를 e라고 하면, e를 포함한 최소 신장 트리가 반드시 존재한다.

위 문장이 옳다는 것을 증명해 보겠습니다. 최소 신장 트리에는 분리된 두 정점 집합 A와 B를 연결하는 에지 중 하나가 반드시 포함되어야 합니다. 그런데 에지 e가 아닌 에지 f가 포함된 최소 신장 트리 T가 있다고 가정해 봅시다(그림 12.6 참조). 트리의 성질에 의해, T에 e를 추가하면 사이클이 형성됩니다. 트리의 모든 정점은 이 사이클에 연결되어 있으므로, T에서 f를 제거하고 e를 추가해도 여전히 신장 트리가 됩니다. 이 신장 트리를 S라고 합시다. 그런데 e의 가중치는 f의 가중치보다 작거나 같으므로, S의 가중치 합은 T의 가중치 합보다 작거나 같습니다. 따라서 T가 최소 신장 트리라면 e와 f의 가중치가 같아야 하고, 결과적으로 S도 최소 신장 트리가 됩니다.

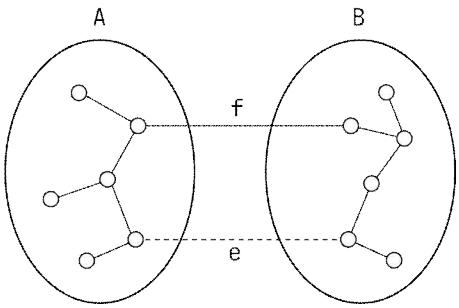
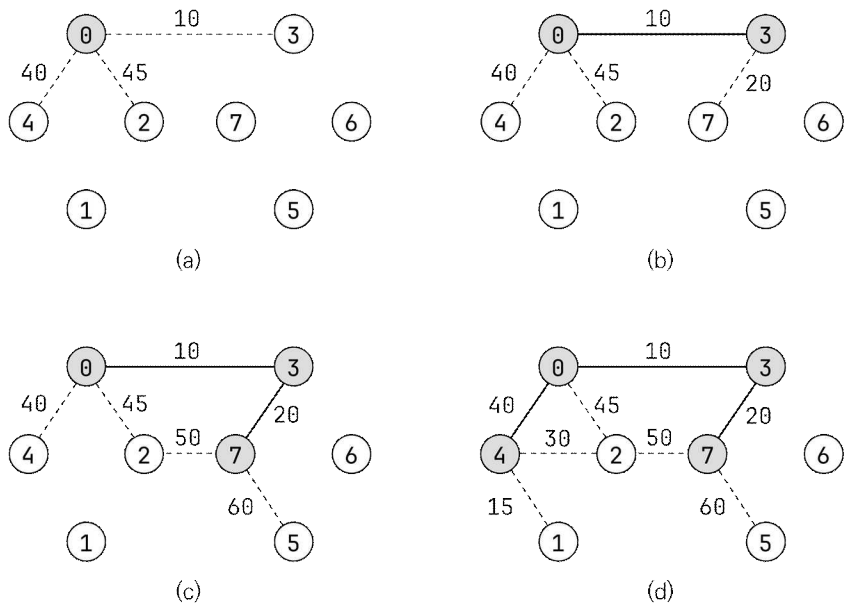


그림 12.6: e를 포함하는 최소 신장 트리의 존재

프림 알고리즘은 하나의 정점에서 시작하여 신장 트리를 점진적으로 구축해 나가는 방식으로 수행됩니다. 이 과정을 통해서 구축된 트리에 속하는 정점과 에지를 각각 **트리 정점(tree vertex)**과 **트리 에지(tree edge)**라고 부릅니다. 초기에는 맨 처음 선택한 정점을 트리 정점으로 설정합니다. 알고리즘은 여러 단계로 구성되는데 각 단계에서 다음의 원칙에 따라 정점과 에지를 하나씩 선택합니다.

> 트리 정점과 비트리 정점을 연결하는 에지 중에서 가중치가 최소인 에지와 그 에지의 끝점인 비트리 정점을 선택한다.

여기서 선택된 정점과 에지가 각각 새로운 트리 정점과 트리 에지가 됩니다. 모든 정점이 트리 정점이 되면 알고리즘은 종료됩니다. 신장 트리는 연결 그래프에서만 존재하기 때문에, 수행 도중에 트리 정점과 비트리 정점을 연결하는 에지가 존재하지 않으면 최소 신장 트리는 존재하지 않음을 알게 됩니다. 반면에 모든 정점이 트리 정점이 된 경우, 트리 에지들이 최소 신장 트리를 구성한다는 것은 앞에서 소개한 이론적 배경을 통해 알 수 있습니다. 그림 12.7에는 그림 12.4의 그래프에 대해 최소 신장 트리를 구하는 과정이 나와 있습니다. 그림에서 칠해진 정점은 트리 정점이고, 점선으로 표시된 에지는 트리 정점과 비트리 정점을 연결하는 에지(즉, 선택 고려 대상인 에지), 그리고 실선으로 표시된 에지는 현재까지 선택된 트리 에지입니다. 최종 구해진 최소 신장 트리의 가중치 합은 200입니다.



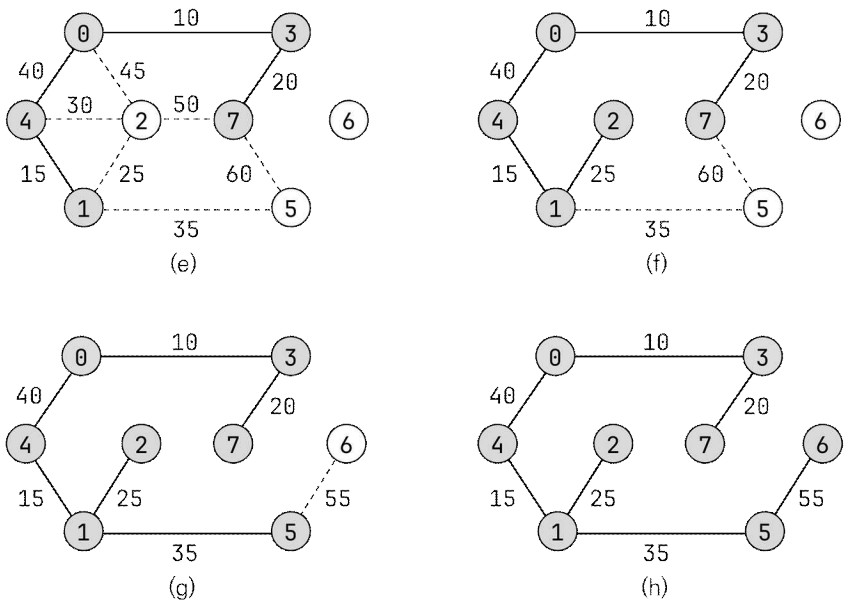


그림 12.7: 그림 12.4의 그래프에 대한 프림 알고리즘의 수행 과정

이제, 프림 알고리즘을 어떻게 구현할 것인가를 생각해 봅시다. 프림 알고리즘처럼 컴퓨터 사용 이전 시대에 고안된 고전적인 알고리즘은 효율성을 고려하여 구체적인 구현 전략을 세워야 합니다. 알고리즘에서 고려해야 할 핵심 사항은 트리 정점과 비트리 정점을 연결하는 에지 중에서 가중치가 최소인 에지를 효율적으로 찾는 작업입니다.

그래프가 가중치 행렬로 표현된 경우부터 살펴보겠습니다. 이 경우, 그래프의 모든 에지를 대상으로 양 끝점이 트리 정점과 비트리 정점에 해당하는지를 검사하고, 그러한 에지 중에서 가중치가 최소인 에지를 찾는 전략은 $O(n^2)$ 시간에 수행됩니다. 따라서 n-1개의 최소 신장 트리 에지를 모두 찾기 위해서는 $O(n^3)$ 시간이 소요됩니다.

프림 알고리즘을 더 효율적으로 구현하는 방법은, 각 비트리 정점 v에 대해 "트리 정점을 연결하는 에지 중 가장 가중치가 작은 에지 (v, w)"를 미리 관리하는 것입니다. 이 정점 w를 v에서 가장 가까운 트리 정점이라고 부르고, 각 비트리 정점에 대해 가장 가까운 트리 정점에 관한 정보를 배열 nearest에 저장한다고 합시다. 즉, 비트리 정점 v에 대해 nearest[v]는 v에 가장 가까운 트리 정점입니다. 따라서, 트리 정점과 비트리 정점을 연결하는 에지 중에서 가중치가 최소인 에지는 바로 각 비트리 정점 v와 nearest[v]를 연결하는 에지 중에서 가중치가 가장 작은 에지입니다.

이 방법을 사용하면 트리 에지를 선택하는데 단지 $O(n)$ 시간이면 충분합니다. 물론 새로운 트리 정점 w가 선택되면, 각 비트리 정점에 대해서 nearest 정보를 갱신해야 합니다. 이 작업은 각 비트리 정점 v에 대해서 에지 (v, w)의 가중치가 현재의 에지 (v, nearest[v])의 가중치보다 작으면 nearest[v]를 w로 치환하는 것입니다. 이 갱신 작업도 $O(n)$ 시간에 수행됩니다. 따라서 이 구현 전략을 사용하면 프림 알고리즘은 총 $O(n^2)$ 시간에 수행됩니다. 표 12.1은 그림 12.4의 그래프에 대해 정점 0에서 시작하여 각 단계에서 트리 에지와 트리 정점을 찾는 과정을 보여줍니다. 표에서 정점 v가 트리 정점이 된 이후에는 nearest[v]의 값을 표시하지 않습니다.

표 12.1: 그림 12.4의 그래프에 대한 프림 알고리즘의 수행 과정

단계	선택된 트리 정점	선택된 트리 에지	nearest							
			0	1	2	3	4	5	6	7
1	0	-		0	0	0	0	0	0	0
2	3	(3, 0)		0	0		0	0	0	3
3	7	(7, 3)		0	0		0	7	0	
4	4	(4, 0)		4	4			7	0	
5	1	(1, 4)			1			1	0	
6	2	(2, 1)						1	0	
7	5	(5, 1)							5	
8	6	(6, 5)								

프로그램 12.3은 프림 알고리즘을 구현한 프로그램입니다. 프로그램에서 가중치 행렬로 그래프를 표현하고 float('inf')로 무한대를 나타냅니다. 함수 prim()은 최소 신장 트리의 에지 집합을 파이썬 리스트에 저장하여 반환하고, 최소 신장 트리가 존재하지 않으면 None을 반환합니다. 수행 과정에서 트리 정점과 비트리 정점을 구분하기 위하여 리스트 selected를 사용합니다. 정점 v가 트리 정점이면 selected[v] = True이고, 아니면 False입니다. 함수 prim()의 수행 시간은 $O(n^2)$입니다.

프로그램 12.3: 가중치 행렬 기반의 프림 알고리즘의 구현

```
1  def prim(M):
2      """프림 알고리즘의 구현:
3      가중치 행렬 M으로 표현된 가중치 그래프에서 최소 신장 트리의 에지 리스트를
4      반환한다."""
5      n = len(M)                              # 정점 수
```

```
    mst = []                                # 최소 신장 트리의 에지 리스트
    nearest = [0] * n                       # 가장 가까운 정점 리스트
    selected = [False] * n                  # 트리 정점이면 True, 아니면 False
    selected[0] = True                      # 시작 정점 0을 트리 정점으로 설정

    while len(mst) < n-1:                   # n-1개의 에지를 선택할 때까지 반복
        w, mincost = -1, float('inf')       # 선택할 비트리 정점 w와 가중치
        for v in range(n):                  # 가중치가 최소인 에지 탐색
            if not selected[v] and M[v][nearest[v]] < mincost:
                w, mincost = v, M[v][nearest[v]]
        if w == -1:                         # MST가 존재하지 않으면 None 반환
            return None
        selected[w] = True                  # w를 트리 정점으로 설정
        mst.append((nearest[w], w, M[nearest[w]][w]))   # 트리 에지 추가
        for v in range(n):                  # nearest 갱신
            if not selected[v] and M[v][w] < M[v][nearest[v]]:
                nearest[v] = w

    return mst                              # 최소 신장 트리의 에지 리스트 반환
```

평면 그래프처럼 에지 개수가 상대적으로 적은 가중치 그래프가 인접 리스트로 표현된 경우는 우선순위 큐인 힙을 사용하여 효율적으로 구현할 수 있습니다. 힙에는 에지 정보 (c, v, w)를 삽입합니다, 여기서 c는 에지 (v, w)의 가중치이고 v는 트리 정점, w는 비트리 정점입니다. 이 힙은 에지 가중치에 대한 최소 힙으로 동작합니다. 알고리즘은 다음과 같이 수행됩니다.

맨 처음, 시작 정점에 부속된 각 에지를 힙에 삽입합니다. 그다음, 힙에서 가중치가 가장 작은 에지를 하나씩 삭제합니다. 만약 삭제된 에지의 양 끝점이 트리 정점이면, 해당 에지는 무시합니다. 그렇지 않고 양 끝점이 각각 트리 정점과 비트리 정점이면, 이 에지는 새로운 트리 에지이므로, 이 에지를 최소 신장 트리의 에지로 추가하고 해당 비트리 정점을 트리 정점으로 전환합니다. 그리고 이 새로운 트리 정점에 부속된 모든 비트리 에지들을 힙에 삽입합니다. 이 과정을 n-1개의 트리 에지가 선택될 때까지 반복합니다.

프로그램 12.4에는 인접 리스트로 표현된 가중치 그래프에서 최소 신장 트리를 구하는 함수 prim_heap()이 나와 있습니다. 이 함수의 수행 과정에서 모든 에지가 한 번씩 힙에 삽입되고 삭제될 수 있으므로, 수행 시간은 $O(m\log m)$입니다[29]. 신장 트리가 존재하지 않은 경우, 즉 m < n인 경우까지 고려한다면, 시간복잡도를 $O(n + m\log m)$으로 나타내는 것이 더 정확합니다.

29) $m = O(n^2)$이고 $\log n^2 = 2\log n$ 이므로, $O(m\log m)$은 $O(m\log n)$으로 표현할 수 있습니다.

프로그램 12.4: 인접 리스트 기반의 프림 알고리즘의 구현

```
import heapq

def prim_heap(G):
    """프림 알고리즘의 구현:
    인접 리스트 G로 표현된 가중치 그래프의 최소 신장 트리를 구해 에지 리스트를
    반환한다. G[v] = [(인접 정점, 가중치), ...]"""
    n = len(G)                              # 정점 수
    mst = []                                # 최소 신장 트리의 에지 리스트
    H = []                                  # 공백 최소 힙
    selected = [False] * n                  # 트리 정점이면 True, 아니면 False
    selected[0] = True                      # 시작 정점 0을 트리 정점으로 설정
    for w, c in G[0]:                       # 각 인접 정점 w와 가중치 c에 대해
        heapq.heappush(H, (c, 0, w))        # 시작 정점 0에 부속된 에지 삽입

    while len(mst) < n-1:                   # n-1개의 에지를 선택할 때까지 반복
        if len(H) == 0:                     # MST가 존재하지 않으면 None 반환
            return None
        c, v, w = heapq.heappop(H)          # 힙 H에서 에지 삭제
        if selected[w]:                     # w가 트리 정점이면 무시
            continue
        selected[w] = True                  # w를 트리 정점으로 설정
        mst.append((v, w, c))               # (v, w, c)를 트리 에지로 추가
        for u, c in G[w]:                   # w의 각 인접 정점 u와 가중치 c
            if not selected[u]:             # 비트리 에지를 힙에 삽입
                heapq.heappush(H, (c, w, u))

    return mst                              # 최소 신장 트리의 에지 리스트 반환
```

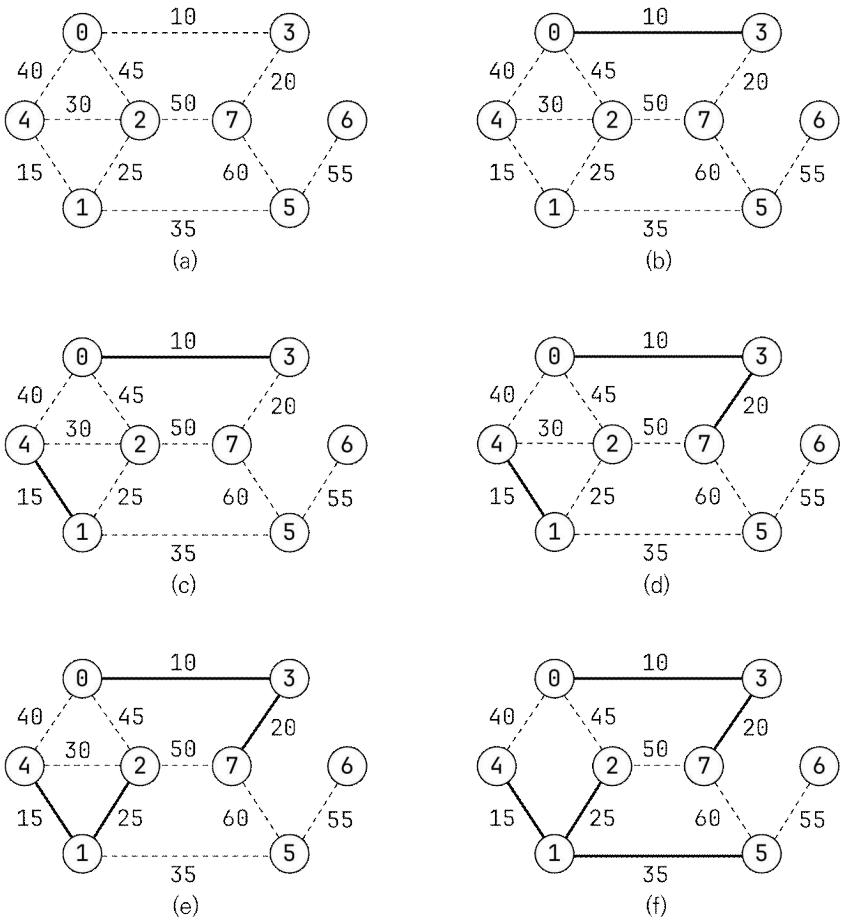
■ 크루스칼 알고리즘

프림 알고리즘이 정점 중심의 알고리즘인 것에 비해 크루스칼 알고리즘은 에지 중심의 알고리즘입니다. 크루스칼 알고리즘은 에지 집합에서 가중치가 작은 에지부터 차례대로 고려하면서 최소 신장 트리에 포함될 에지를 찾는 방식으로 진행됩니다. 알고리즘의 수행 도중에 여러 개의 연결 요소가 유지되고, 모든 정점이 하나의 연결 요소에 포함되면 알고리즘을 종료합니다. 여기서 각 연결 요소는 트리입니다. 연결 요소들의 모임은 트리들의 모임이므로, 이 모임을 **숲(forest)**이라고 부르기도 합니다. 최종적으로 모든 정점이 포함된 하나의 연결 요소가 바로 최소 신장 트리입니다.

맨 처음에는 각 정점이 독립된 연결 요소로서 초기화됩니다. 에지 집합에서 선택한 에지의 양 끝점이 같은 연결 요소에 속하면, 이 에지는 기존에 선택된 트리 에지들과 함께 사이클을 발생

시키게 되므로 무시합니다. 양 끝점이 다른 연결 요소에 속하는 에지는 최소 신장 트리에 포함될 에지로 선택되고 두 연결 요소는 하나로 합병됩니다. 이 에지는 분리된 두 연결 요소를 연결하는 에지 중에서 가중치가 최소인 에지이므로, 프림 알고리즘의 이론적 배경으로 소개한 문장(절단 속성)에 의해서 이 에지가 최소 신장 트리의 에지임이 증명됩니다.

그림 12.8에는 그림 12.4의 그래프에 대해 크루스칼 알고리즘으로 최소 신장 트리를 구하는 과정이 나와 있습니다. 그림에서 점선으로 표시된 에지는 에지 집합에서 선택 고려 대상으로 남아 있는 에지이고, 실선으로 표시된 에지는 최소 신장 트리에 포함된 에지입니다. 최종적으로 구해진 최소 신장 트리의 가중치 합은 200입니다.



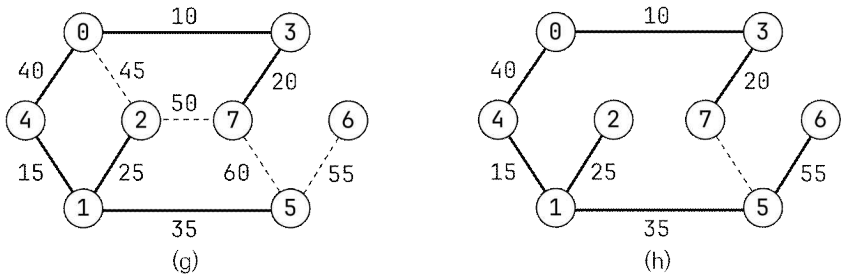


그림 12.8: 그림 12.4의 그래프에 대한 크루스칼 알고리즘의 수행 과정

크루스칼 알고리즘의 구현에서 효율성을 위해 고려해야 할 작업은 두 가지가 있습니다. 하나는 에지 집합에서 가중치가 최소인 에지를 선택하는 것이고, 다른 하나는 현재 고려하는 에지의 양 끝점이 같은 연결 요소에 있는지를 판별하는 것입니다.

첫 번째 작업은 에지 집합을 미리 정렬해 두고 가중치가 작은 에지부터 하나씩 고려하면 효율적으로 처리됩니다. 이 경우 정렬 작업에 $O(m \log m)$ 시간이 소요되고, 그 후에는 상수 시간에 그다음 가중치가 작은 에지를 선택할 수 있습니다. 또 다른 방법은 힙(우선순위 큐)을 사용하는 것입니다. 에지 집합으로 초기 힙을 만드는 데 걸리는 시간은 $O(m)$이고, 그 후에는 힙에서 에지를 하나씩 삭제하는 데 하나당 $O(\log m)$ 시간이 소요됩니다. 따라서 두 방법 모두 이 작업을 수행하는데 걸리는 전체 시간은 $O(m \log m)$입니다. 다만, 입력에 따라서는 모든 에지를 고려하지 않아도 최소 신장 트리를 구할 수 있으므로, 힙을 사용하는 방법이 에지 전체를 정렬하는 방법에 비해 수행 시간에서 약간 유리할 수 있습니다.

다음으로, 현재 고려하는 에지의 양 끝점이 같은 연결 요소에 있는지를 판별하기 위해서 연결 요소를 집합 개념으로 보는 전략을 도입합니다. 맨 처음에 각 정점을 독립된 집합으로 두고, 두 연결 요소가 합병되면 두 집합을 하나로 합칩니다. 이 작업에는 에지의 양 끝점이 같은 집합에 속하지를 검사하는 연산과 두 집합을 하나로 합치는 연산이 포함됩니다. 각 정점은 하나의 집합에만 속하기 때문에, 10장에서 소개한 '분리된 집합을 위한 union-find 자료구조'를 사용하여 이 작업을 효율적으로 수행할 수 있습니다. 현재 고려하는 에지에 대해 양 끝점을 대상으로 2번의 find 연산을 수행하고, 양 끝점이 다른 집합에 속하면 한 번의 union 연산을 수행합니다. 처음에 n개의 집합이 주어지고, 최대 2m번의 find 연산과 n-1번의 union 연산이 수행되므로, 이 자료구조를 이용하면 거의 선형 시간과 다름없는 $O(m\alpha(m, n))$ 시간에 모든 작업을 완료할 수 있습니다.

프로그램 12.5에는 인접 리스트로 표현된 가중치 그래프를 대상으로 크루스칼 알고리즘을 구현

한 함수 kruskal()이 나와 있습니다. 이 함수는 파이썬의 heapq 모듈과 10장에서 정의한 UFSet 클래스를 이용합니다. 프로그램에서 힙을 관리하는 작업에 가장 많은 시간이 소요되므로, kruskal() 함수의 수행 시간은 $O(m\log m)$입니다.

프로그램 12.5: 인접 리스트 기반의 크루스칼 알고리즘의 구현

```
import heapq

def kruskal(G):
    """크루스칼 알고리즘의 구현:
    인접 리스트 G로 표현된 가중치 그래프의 최소 신장 트리를 구해 에지 리스트를
    반환한다. G[v] = [(인접 정점, 가중치), ...]"""
    n = len(G)                              # 정점 수
    mst = []                                # 최소 신장 트리의 에지 리스트
    C = UFSet(n)                            # n개의 원소를 가진 UFSet 객체 생성
    H = []                                  # 최소 힙으로 사용할 리스트
    for v in range(n):                      # H에 모든 에지를 삽입
        for w, c in G[v]:                   # v의 인접 정점 w와 가중치 c
            if v < w:                       # 중복 방지를 위한 조건
                H.append((c, v, w))         # 가중치 기준으로 힙에 삽입

    heapq.heapify(H)                        # H를 힙으로 초기화
    while len(mst) < n-1:                   # n-1개의 에지를 선택할 때까지 반복
        if len(H) == 0:                     # 힙이 공백이면 None을 반환
            return None
        c, v, w = heapq.heappop(H)          # 힙 H에서 에지 삭제
        if C.find(v) != C.find(w):          # 양 끝점이 다른 집합에 속하면
            mst.append((v, w, c))           # (v, w, c)를 트리 에지로 추가
            C.union(C.find(v), C.find(w))   # 합집합 수행

    return mst                              # 최소 신장 트리의 에지 리스트 반환
```

거의 모든 정점 쌍에 에지가 존재하는 **밀집 그래프(dense graph)**는 에지 수가 $\Theta(n^2)$이므로, 밀집 그래프를 대상으로 하는 작업은 인접 행렬로 표현하여 정점 중심으로 수행하는 것이 일반적으로 효율적입니다. 그에 반해 에지 수가 상대적으로 적은 **희소 그래프(sparse graph)**는 인접 리스트로 표현하고 에지 중심으로 수행하는 것이 훨씬 효율적인 경우가 많습니다. 최소 신장 트리를 구하려는 가중치 그래프가 밀집 그래프라면, 가중치 행렬 기반의 프림 알고리즘은 $O(n^2)$ 시간에 수행되지만, 인접 리스트 기반의 프림 알고리즘과 크루스칼 알고리즘은 정점 수를 기준으로 시간복잡도를 나타내면 $O(n^2\log n)$이 되어 상대적으로 비효율적입니다. 반면에 희소 그래프의 경우, 특히 $m = O(n)$이면, 가중치 행렬 기반의 프림 알고리즘은 여전히 $O(n^2)$ 시간

에 수행되지만, 다른 두 방식은 $O(n\log n)$ 시간에 수행되므로 훨씬 효율적입니다.

프림 알고리즘과 크루스칼 알고리즘처럼, 여러 단계에 걸쳐 수행하는 과정의 각 단계에서 '최적의 선택' 원칙을 정해 전체적인 최적해를 찾는 알고리즘 설계 기법을 **욕심쟁이법(greedy method)**이라고 합니다. 이때 적용되는 원칙을 **욕심쟁이 원칙**이라고 말합니다. 프림 알고리즘의 욕심쟁이 원칙은 "트리 정점과 비트리 정점 사이를 연결하는 에지 중 가중치가 최소인 에지를 선택한다"이고, 크루스칼 알고리즘의 욕심쟁이 원칙은 "현재 남아 있는 에지 집합에서 가중치가 최소인 에지를 뽑아서, 그 에지가 앞서 선택한 에지들과 함께 사이클을 형성하면 버리고 아니면 선택한다"입니다. 욕심쟁이 알고리즘은 비교적 이해하기 쉽고 간단하지만, 모든 문제에 욕심쟁이법이 통하는 것은 아니기 때문에, 욕심쟁이 원칙으로 항상 해답을 찾을 수 있다는 사실을 수학적으로 증명하는 과정이 필요합니다.

12.3 최단 경로 문제

비가중치 그래프에서는 최단 경로를 너비 우선 탐색으로 구하지만, 경로의 길이를 에지의 가중치 합으로 정의하는 가중치 그래프에서는 최단 경로를 같은 방법으로 구할 수 없습니다. 예를 들어, 그림 12.10의 가중치 방향 그래프에 대해 정점 0에서 정점 2로 가는 방향 경로의 경우 에지 두 개를 거쳐서 가는 경로 〈0, 1, 2〉의 길이는 55(=35+20)입니다. 그런데 에지 세 개를 거쳐서 가는 경로 〈0, 4, 3, 2〉의 길이는 50(=20+20+10)으로 더 짧습니다. 너비 우선 탐색은 에지 수에 기반하여 탐색을 수행하기 때문에, 가중치 그래프에서는 최단 경로를 정확히 구하지 못합니다.

가중치 그래프에서 출발점을 루트로 하는 **최단 경로 트리(shortest path tree)**는, 루트와 다른 모든 정점 사이의 경로가 그래프에서 최단 경로가 되는 트리입니다. 그림 12.9는 정점 0을 출발점으로 하는 최단 경로 트리의 예를 보여줍니다. 그림에서 최단 경로 트리의 각 정점에 표시된 값은 루트에서 해당 정점까지의 최단 경로 길이입니다.

가중치 그래프에서의 최단 경로는 최소 신장 트리와 마찬가지로 다양한 분야에서 응용되기 때문에, 최단 경로를 찾는 알고리즘은 1950년대부터 연구되기 시작하여 최근까지도 새로운 알고리즘이 계속 발표되고 있습니다. 가장 널리 알려진 것으로는, 1959년에 발표된 **다익스트라 알고리즘(Dijkstra algorithm)**과 1962년에 발표된 **플로이드-와샬 알고리즘(Floyd-Warshall algorithm)**

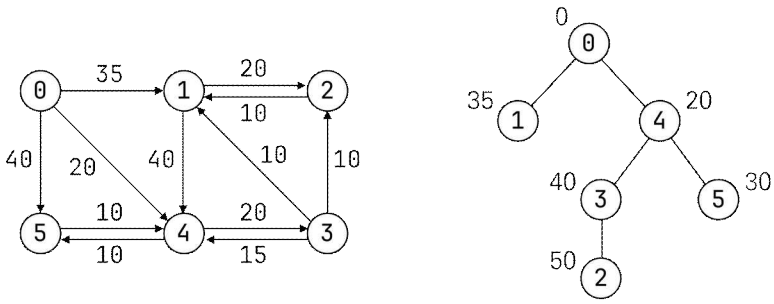


그림 12.9: 가중치 방향 그래프와 최단 경로 트리

이 있습니다. 이 절에서는 이 두 알고리즘에 대해 차례로 소개합니다. 방향 그래프에 대한 최단 경로 알고리즘은 무방향 그래프에서도 거의 똑같이 적용되기 때문에, 여기서 다루는 가중치 그래프는 방향 그래프라고 가정하고, n과 m을 각각 그래프의 정점 수와 에지 수라고 하겠습니다.

■ 다익스트라의 최단 경로 알고리즘

다익스트라 알고리즘은 하나의 출발점에서 다른 모든 정점까지의 최단 경로를 찾는 알고리즘입니다. 출발점에서 어느 한 정점까지의 경로가 최단 경로라는 사실을 확정하기 위해서는 출발점에 더 가까운 모든 정점까지의 최단 경로가 먼저 결정되어 있어야 합니다. 따라서 두 정점 사이의 최단 경로를 탐색하는 알고리즘도 다익스트라 알고리즘처럼 '단일 출발점 모든 도착점(single source all destinations)' 방식으로 최단 경로를 구하게 됩니다.

다익스트라 알고리즘은 가중치가 음수인 에지를 가진 그래프에서는 올바른 결과를 보장하지 못합니다. 따라서 음수 가중치가 없는 그래프에 대해서만 다익스트라 알고리즘을 사용해야 합니다. 다익스트라 알고리즘은 출발점에서 가장 가까운 정점부터 차례대로 최단 경로를 확정해 나갑니다. 출발점에 인접한 정점 중에서 가중치가 최소인 에지로 연결된 정점은 출발점에서 가장 가까운 정점이고, 그 가중치가 그 정점까지의 최단 경로의 길이입니다. 왜냐하면 음수 가중치 에지가 없는 그래프에서의 경로 길이는 항상 정점을 경유할수록 증가하거나 유지되기 때문입니다.

다익스트라 알고리즘은 수행 과정에서 '집합 S'와 '배열 `distance`'를 사용합니다. 집합 S는 최단 경로의 길이, 즉 최단 거리(shortest distance)가 알려진 정점들의 집합이고, 배열 `distance`는 S에 속하지 않은 각 정점 v에 대해 출발점에서 S에 속한 정점들만을 거쳐 v로 도착하는 최단 경로의 길이입니다. 식으로 나타내면 다음과 같습니다.

```
distance[v] = min(distance[u] + cost(u, v) | u ∈ S, v ∈ V-S)
```

여기서 V는 정점 집합이고 cost(u, v)는 에지 ⟨u, v⟩의 가중치입니다. 알고리즘의 초기에 집합 S에는 출발점 src만 포함되고, 각 정점 v에 대한 distance[v]는 cost(src, v)입니다. 알고리즘은 여러 단계로 구성되며, 각 단계에서 최단 경로의 길이가 알려지는 정점을 선택합니다. 이때 다음과 같은 욕심쟁이 원칙이 적용됩니다.

S에 속하지 않은 정점 중에서 distance의 값이 가장 작은 정점을 선택한다.

이 원칙에 의해 선택된 정점 v를 집합 S에 추가하고, S에 속하지 않은 각 정점 w에 대한 distance의 값을 다음과 같이 갱신합니다.

```
distance[w] = min(distance[w], distance[v] + cost(v, w))
```

즉, S에 속하지 않은 각 정점 w에 대해, 정점 v를 경유하는 새로운 경로의 길이와 기존의 distance[w] 중에서 더 작은 값을 distance[w]로 설정합니다.

알고리즘은 모든 정점이 S에 포함될 때까지 각 단계에서 이 욕심쟁이 원칙을 적용합니다. 이 원칙으로 구한 distance[w]가 출발점에서 정점 w까지의 최단 거리라는 사실을 어떻게 증명할 수 있을까요? 집합 S에 속한 정점은 이미 최단 거리가 확정되었으며, 정점 w보다 출발점에 더 가까운 정점입니다. 그림 12.10은 이 상황을 그림으로 보여줍니다. 증명을 위해서 집합 S에 속하지 않은 정점 중에서 distance의 값이 가장 작은 정점이 w이지만, distance[w]가 최단 거리가 아니라고 가정해 봅시다. 그러면 S에 속하지 않은 다른 정점 u를 지나서 돌아가는 더 짧은 경로가 존재해야 합니다. 그런데 distance[w] ≤ distance[u]이며, u에서 w로 가는 경로에 음수 가중치를 가진 에지는 없습니다. 따라서 경로의 길이가 작아지는 경우는 발생하지 않으므로, distance[w]가 최단 거리임이 증명됩니다.

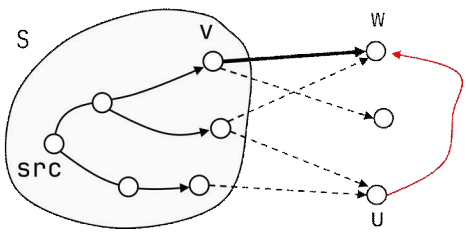


그림 12.10: 최단 경로의 증명

표 12.2와 그림 12.11은 그림 12.9의 그래프에 대해 다익스트라 알고리즘의 수행 과정을 보여줍니다. 그림 12.11에서 칠해진 정점은 현 단계에서 집합 S에 포함된 정점이고, 점선은 새로 선택될 에지의 후보, 굵은 선은 에지 후보 중에서 선택된 에지를 나타냅니다.

표 12.2: 그림 12.9의 그래프에 대한 다익스트라 알고리즘의 수행 과정

단계	추가된 S의 정점	선택된 에지	distance					
			0	1	2	3	4	5
1	0	⟨0, 4⟩	0	35	∞	∞	20	40
2	4	⟨4, 5⟩	0	35	∞	40	20	30
3	5	⟨0, 1⟩	0	35	∞	40	20	30
4	1	⟨4, 3⟩	0	35	55	40	20	30
5	3	⟨3, 2⟩	0	35	50	40	20	30
6	2	-	0	35	50	40	20	30

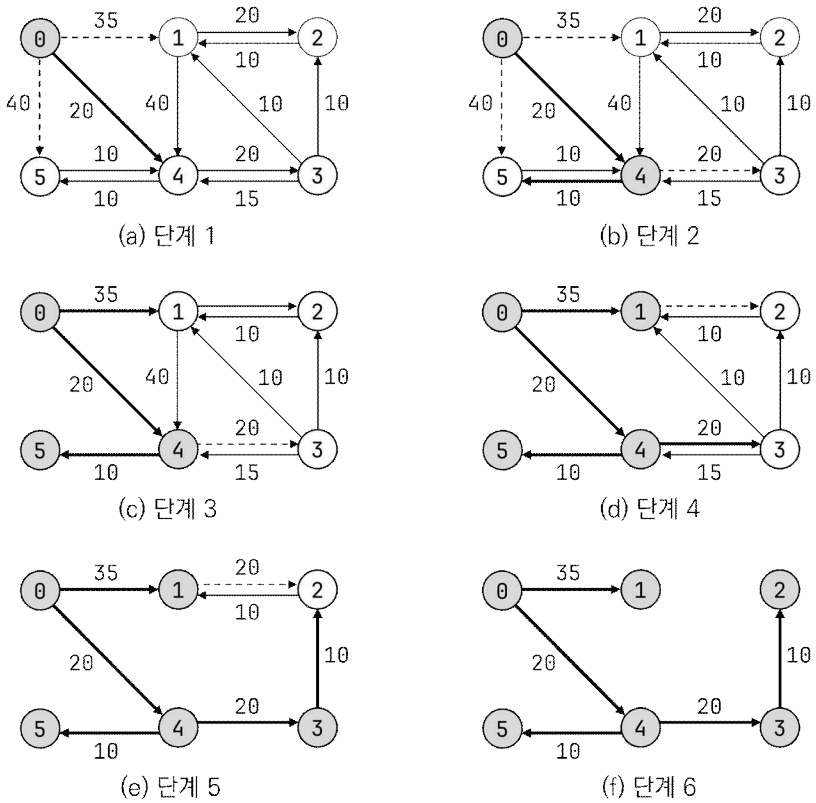


그림 12.11: 그림 12.9의 그래프에 대한 다익스트라 알고리즘의 수행 과정

다익스트라 알고리즘을 이용하여 최단 경로 자체를 추적하고자 할 때는 부모 포인터 배열을 사용하여 출발점을 루트로 하는 최단 경로 트리를 구축하면 됩니다. 알고리즘의 수행 과정에서 선택된 에지들이 최단 경로 트리를 구성합니다. 구체적으로 설명하면, 알고리즘의 초기에는 출발점을

제외한 모든 정점이 출발점의 자식으로 초기화됩니다. 그런 다음 각 단계에서 distance를 갱신할 때, distance[w]의 값이 distance[v] + cost(v, w)로 갱신되면 정점 w의 부모로 v를 설정합니다. 그 이유는 에지 ⟨v, w⟩가 최단 경로를 구성하는 에지들의 후보이기 때문입니다.

프로그램 12.6은 다익스트라 알고리즘을 구현한 프로그램입니다. 함수 dijkstra(M, src)는 가중치 행렬 M으로 표현된 그래프에서 출발점이 src인 최단 거리와 최단 경로 트리를 반환합니다. 프로그램의 수행 과정에서 출발점에서 각 정점까지의 최단 거리를 배열 dist에 저장하고, 배열 found를 사용하여 집합 S에 포함되는 정점을 구분합니다. 정점 v가 S에 속하면 found[v]의 값은 True이고 아니면 False입니다. 함수 dijkstra()의 수행 시간은 $O(n^2)$입니다. 11번째 줄의 for 문은 n-1번이 아니라 n-2번 반복합니다. 출발점 src은 미리 선택되었고 마지막 남은 정점의 선택 여부와는 상관없이 배열 dist와 parent는 더 이상 갱신되지 않으므로, n-2번의 반복으로 모든 정점의 최단 거리가 구해지기 때문입니다.

프로그램 12.6: 가중치 행렬 기반의 다익스트라 알고리즘의 구현

```
1  def dijkstra(M, src):
2     """최단 경로를 구하는 다익스트라 알고리즘의 구현:
3     가중치 행렬 M으로 표현된 가중치 그래프에서 출발점이 src인 최단 거리 리스트와
4     최단 경로 트리를 저장한 부모 포인터 배열을 반환한다."""
5     n = len(M)                              # 정점 수
6     found = [False] * n                     # 최단 거리가 구해진 정점 집합 S
7     found[src] = True                       # 출발점을 집합 S에 포함
8     dist = M[src].copy()                    # 최단 거리 리스트 dist 초기화
9     parent = [src] * n                      # 부모 포인터 배열 초기화
10    parent[src] = -1                        # 출발점 src를 루트로 설정
11
12    for _ in range(n-2):                    # n-2번 반복
13       v = -1                               # 선택할 정점을 저장, 없으면 -1
14       minlength = float('inf')             # 최단 거리, 초깃값 무한대
15       for w in range(n):                   # dist에서 최단 거리 검색
16          if not found[w] and dist[w] < minlength:
17             v, minlength = w, dist[w]
18       if v == -1:                          # 선택한 정점이 없으면
19          break                             # 최단 거리는 더 이상 바뀌지 않음
20       found[v] = True                      # 정점 v의 최단 거리 구해짐
21       for w in range(n):                   # dist 갱신
22          if not found[w] and dist[w] > dist[v] + M[v][w]:
23             dist[w] = dist[v] + M[v][w]
24             parent[w] = v                 # 최단 경로 트리 갱신
25
26    return dist, parent           # 최단 거리 리스트와 최단 경로 트리 반환
```

함수 dijkstra()에서 최단 경로 트리를 제외하고 최단 거리만 구하고자 할 때는 9, 10, 25줄을 삭제하고, 마지막 줄에서 dist만 반환하면 됩니다.

가중치 그래프가 인접 리스트로 표현된 경우는 최소 힙을 사용하여 $O(n + m\log m)$ 시간에 다익스트라 알고리즘이 수행하도록 구현할 수 있습니다. 이 구현은 희소 그래프가 입력으로 들어오는 경우 가중치 행렬을 사용할 때보다 훨씬 효율적입니다. 최소 힙에는 (현재까지의 거리, 정점) 쌍이 저장됩니다. 알고리즘은 다음과 같이 수행됩니다.

1. 최초에 출발점 src에 대해, (0, src) 쌍을 힙에 삽입한다.
2. 힙이 공백이 될 때까지 다음을 반복한다.
   ① 힙에서 '현재까지의 거리'가 가장 짧은 원소 (length, v)를 삭제한다.
   ② 만약 거리 length가 dist[v]보다 크거나 같으면, 정점 v는 이미 최단 거리가 구해진 정점이므로 무시한다(이전에 더 짧은 경로의 거리가 힙에 삽입된 후 먼저 처리되었음).
   ③ 그렇지 않으면, v에 인접한 각 정점 w에 대해, v를 경유하는 더 짧은 경로가 발견되면, (새로운 거리, w) 쌍을 힙에 삽입하고, dist[w]를 갱신하고, parent 배열에서 w의 부모를 v로 설정한다.

프로그램 12.7에 위 알고리즘을 구현한 프로그램이 나와 있습니다. 힙에는 최대 에지 개수만큼의 원소가 저장되므로, 전체 수행 시간은 $O(n + m\log m)$입니다[30].

프로그램 12.7: 인접 리스트 기반의 다익스트라 알고리즘의 구현

```
1  import heapq
2
3  def dijkstra_heap(G, src):
4      """최단 경로를 구하는 다익스트라 알고리즘의 구현:
5      인접 리스트 G로 표현된 가중치 그래프에서 출발점이 src인 최단 거리 리스트와
6      최단 경로 트리를 반환한다. G[v] = [(인접 정점, 가중치), ...]"""
7      n = len(G)                          # 정점 수
8      dist = [float('inf')]*n             # 최단 거리 리스트 dist 초기화
9      dist[src] = 0                       # src의 최단 거리
10     parent = [src] * n                  # 부모 포인터 배열 초기화
11     parent[src] = -1                    # src를 루트로 설정
12
13     H = [(0, src)]                      # 힙 H에 출발점 삽입
```

30) 참고로, 고급 우선순위 큐인 피보나치 힙(Fibonacci heap)을 사용하면 다익스트라 알고리즘을 $O(m + n\log n)$ 시간에 구현할 수 있습니다.

```
    while len(H) > 0:                       # 공백 힙이 될 때까지 반복
        length, v = heapq.heappop(H)        # (거리, 정점) 쌍을 힙에서 삭제
        if length > dist[v]:                # v의 최단 거리가 이미 구해진 경우
            continue                        # 무시함
        for w, c in G[v]:                   # v의 인접 정점 w와 가중치 c
            length = dist[v] + c            # 정점 v를 지나는 경로의 길이
            if length < dist[w]:            # 더 짧은 경로 발견
                dist[w] = length            # 최단 거리 변경
                heapq.heappush(H, (length, w))  # (거리, 정점) 쌍을 힙에 삽입
                parent[w] = v               # w의 부모를 v로 설정

    return dist, parent                     # 최단 거리와 최단 경로 트리 반환
```

■ 모든 정점 쌍의 최단 거리를 구하는 플로이드-와샬 알고리즘

플로이드-와샬 알고리즘은 가중치 행렬을 기반으로 동작하는 알고리즘으로 모든 정점 쌍 사이의 최단 거리를 구해줍니다. 다익스트라 알고리즘을 사용하여 모든 정점 쌍 사이의 최단 거리를 구하려면, 그래프의 각 정점을 출발점으로 하여 다익스트라 알고리즘을 n번 반복하면 됩니다. 그러므로, 수행 시간은 가중치 행렬 기반인 경우는 $O(n^3)$, 인접 리스트 기반인 경우는 $O(n^2 + n \cdot mlogm)$이 됩니다. 플로이드-와샬 알고리즘은 가중치 행렬 기반으로 동작하는 알고리즘으로, 다익스트라 알고리즘을 이용하는 것과 똑같이 시간복잡도가 $O(n^3)$이지만, 알고리즘이 비교적 간단하면서도 음수 가중치를 가진 에지가 존재하는 그래프에 대해서도 제한적으로 최단 거리를 구할 수 있다는 장점이 있습니다. 구체적으로, 무향 그래프인 경우는 음수 가중치를 가진 에지가 존재하면 플로이드-와샬 알고리즘을 적용할 수 없지만, 방향 그래프인 경우는 에지들의 가중치 합이 음수인 방향 사이클만 존재하지 않으면 이 알고리즘을 적용할 수 있습니다.

플로이드-와샬 알고리즘은 여러 단계로 구성되며, 각 단계에서 모든 정점 쌍에 대해 특정 제약 조건을 가진 최단 거리를 구합니다. 초기에는 매우 강력한 제약 조건을 가진 상태에서 최단 거리를 구하고, 단계적으로 제약 조건을 조금씩 완화하면서 최단 거리를 갱신해 나갑니다. 마지막에는 제약 조건이 완전히 풀리게 되고 원래 문제의 최단 경로 해가 구해집니다[31].

k-최단 경로를, 경로의 중간에 정점 번호가 k보다 큰 정점을 지나지 않는 경로 중에서 길이가 가장 짧은 경로라고 합시다. 그리고 A^k[v][w]를 정점 v에서 정점 w까지 k-최단 경로의 길이라고 정의합니다. k = -1이면 중간에 경유할 수 있는 정점이 없으므로 A^{-1}[v][w]는 cost(v,

31) 플로이드-와샬 알고리즘처럼 설계되는 알고리즘의 유형을 '동적계획법(dynamic programming)'이라고 부릅니다.

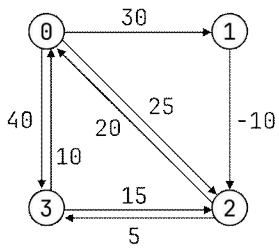
w)와 같습니다. k = n-1이면 모든 제약 조건이 풀린 상태이므로 $A^{n-1}[v][w]$는 최종적으로 구하고자 하는 v에서 w까지 최단 거리가 됩니다.

플로이드-와샬 알고리즘은 A^{-1}부터 시작하여 단계적으로 A^0, A^1, … , A^{n-1}을 구합니다. 현재 A^{k-1}까지 구해져 있다고 가정하고 A^k를 구하려고 한다고 합시다. 정점 v에서 w까지 k-최단 경로는 중간에 정점 k를 경유하거나 또는 경유하지 않거나 둘 중 하나에 해당합니다.

1. 만일 중간에 k를 경유하지 않는다면 v에서 w까지의 k-최단 경로는 (k-1)-최단 경로와 같습니다. 따라서 $A^k[v][w] = A^{k-1}[v][w]$입니다.
2. 그렇지 않으면, k-최단 경로는 정점 k를 지나는데, 이 경우 v에서 w까지의 k-최단 경로는 v에서 k까지의 k-최단 경로와 k에서 w까지의 k-최단 경로를 합친 것과 같습니다. 즉, $A^k[v][w] = A^k[v][k] + A^k[k][w]$입니다. 그런데 v에서 k까지의 k-최단 경로는 중간에 다시 정점 k를 지나지 않으므로 v에서 k까지의 (k-1)-최단 경로와 같습니다, 마찬가지로 k에서 w까지의 k-최단 경로도 (k-1)-최단 경로와 같습니다. 따라서 $A^k[v][w] = A^{k-1}[v][k] + A^{k-1}[k][w]$입니다. 위의 내용을 정리하면 다음과 같습니다.

$$A^k[v][w] = \min(A^{k-1}[v][w], A^{k-1}[v][k] + A^{k-1}[k][w]), k \geq 0$$

그림 12.12는 위의 식에 따라 단계적으로 모든 정점 쌍에 대한 최단 경로의 길이를 구하는 과정을 보여줍니다.



A^{-1}	0	1	2	3
0	0	30	25	40
1	∞	0	-10	∞
2	20	∞	0	5
3	10	∞	15	0

A^0	0	1	2	3
0	0	30	25	40
1	∞	0	-10	∞
2	20	50	0	5
3	10	40	15	0

A^1	0	1	2	3
0	0	30	20	40
1	∞	0	-10	∞
2	20	50	0	5
3	10	40	15	0

A^2	0	1	2	3
0	0	30	20	25
1	10	0	-10	-5
2	20	50	0	5
3	10	40	15	0

A^3	0	1	2	3
0	0	30	20	25
1	5	0	-10	-5
2	15	45	0	5
3	10	40	15	0

그림 12.12: 플로이드-와샬 알고리즘의 수행 과정

프로그램 12.8에 플로이드-와샬 알고리즘을 구현한 함수 floyd(M)이 나와 있습니다. 프로그램에서 A^{-1}부터 A^{n-1}까지 최단 거리를 저장하는 배열을 단계별로 따로 두지 않아도 됩니다. A^k는 A^{k-1}로부터 만들어지므로, 그 이전의 행렬은 필요하지 않습니다. 또한 두 행렬에서 k번째 행과 k번째 열의 값은 변하지 않으므로, 2차원 리스트 A를 하나만 두고 값을 덮어쓰더라도 정보가 훼손되지 않습니다. 함수 floyd()에는 각각 n번씩 반복되는 3개의 for 문이 중첩되어 있어 수행 시간은 $O(n^3)$입니다.

프로그램 12.8: 플로이드-와샬 알고리즘의 구현

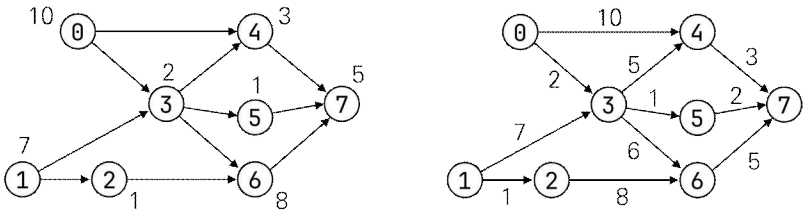
```
def floyd(M):
    """플로이드-와샬 알고리즘의 구현:
    가중치 행렬 M으로 표현된 가중치 그래프에서 모든 정점 쌍의 최단 거리를 반환한다."""
    n = len(M)                      # 정점 수
    A = [x.copy() for x in M]       # A의 초기화, M을 복사함
    for k in range(n):              # 최단 거리를 단계별로 구함
        for v in range(n):
            for w in range(n):
                A[v][w] = min(A[v][w], A[v][k]+A[k][w])
    return A                        # 최단 경로 행렬 반환
```

12.4 사례 연구: 작업 네트워크

11장의 사례 연구에서, 프로젝트의 수행 계획을 방향 비순환 그래프(DAG)로 표현하고 작업 사이의 선행 관계를 유지하면서 순차적으로 작업을 나열하는 위상 정렬에 대해 살펴봤습니다. 이 절에서는 좀 더 일반화된 **작업 네트워크(activity network)**를 소개합니다. 작업 네트워크는 프로젝트 내의 모든 **작업(activity)**과 작업 사이의 **종속성(dependency)**을 시각적인 그래프로 표현한 것이며, 본질적으로 방향 비순환 그래프입니다.

작업 네트워크를 그래프로 표현하는 방식에는 두 가지가 있습니다. 하나는 **정점 중심 작업 네트워크(Activity On Vertex Network)**, 줄여서 **AOV 네트워크**라고 부릅니다. 이 방식은 정점이 각 작업을 나타내고 방향 에지는 작업 사이의 선행 관계를 나타냅니다. 다른 하나는 **에지 중심 작업 네트워크(Activity On Edge Network)**, 줄여서 **AOE 네트워크** 또는 **AOA(Activity On Arrow) 네트워크**라고 부릅니다. 이 방식은 방향 에지가 각 작업을 나타내고 정점은 작업의 시작과 끝을 의

미하는 이벤트를 나타냅니다. 그림 12.13에 AOV 네트워크와 AOE 네트워크의 예가 나와 있습니다. 그림에는 각 작업을 완료하기 위한 소요 시간이 표시되어 있습니다. AOV 네트워크에서는 정점에 소요 시간이 표시되고, AOE 네트워크에서는 에지에 소요 시간이 표시됩니다.



AOV 네트워크 AOE 네트워크

그림 12.13: 작업 네트워크의 예

AOV 네트워크는 정점이 작업을 나타내므로, 작업 정보를 정점에 직접 저장하기 편리하며, 위상 정렬과 같은 알고리즘을 적용하기에 직관적이라는 장점이 있습니다. 이러한 이유로 대부분의 현대 프로젝트 관리 시스템에서는 AOV 네트워크 방식이 주로 채택됩니다. 반면, AOE 네트워크는 에지가 작업을 나타내므로, 병렬로 진행되는 작업을 시각적으로 구분하기에 편리하고, 각 작업의 시작 및 종료 이벤트를 명확히 표현할 수 있습니다.

작업 네트워크가 구축되면, 프로젝트를 다양한 측면에서 분석하고 최적화할 수 있습니다. 특히 각 작업의 소요 시간이 주어졌을 때, 프로젝트가 가장 빠르게 완료될 수 있는 시점(최소 완료 시간)을 계산하는 일은 매우 중요한 분석 요소입니다. 또한 작업 중에서 소요 시간을 맞추지 못할 경우, 프로젝트의 전체 일정에 영향을 주는 작업을 찾아내는 일도 중요합니다. 작업 네트워크의 방향 경로에 포함된 작업은 순차적으로 수행되어야 하므로, 프로젝트의 완료 시간은 그 경로에 포함된 작업의 소요 시간 합보다 작을 수 없습니다. 그러므로, 프로젝트의 가장 빠른 완료 시간을 알기 위해서는 모든 방향 경로 중에서 소요 시간의 합이 가장 큰 경로를 찾아야 합니다. 소요 시간의 합은 경로의 길이에 해당하므로, 이 경로는 **최장 방향 경로(longest directed path)**입니다. 그림 12.14의 AOV 네트워크에서 최장 방향 경로는 〈0, 3, 6, 8〉로 길이가 25이고, AOE 네트워크에서의 최장 경로는 〈1, 3, 6, 5〉로 길이가 18입니다. 일반적인 그래프에서 최장 경로를 찾는 것은 다항 시간에 풀 수 없는 매우 어려운 문제이지만[32], 방향 비순환 그래프에서는 최장 방향 경로를 효율적으로 찾을 수 있습니다. AOE 네트워크에서 가중치에 음수가

32) 일반적인 그래프에서 최장 경로를 찾는 것은 NP-hard에 해당하는 문제입니다. NP-hard는, 수학적으로 증명되지는 않았지만, 다항 시간에 해를 찾을 수 없는 문제의 유형으로 알려져 있습니다.

포함되어 있지 않다면, 모든 가중치를 음수로 바꾸고 최단 경로 알고리즘을 적용하면 원래 AOE 네트워크에서의 최장 경로가 구해집니다.

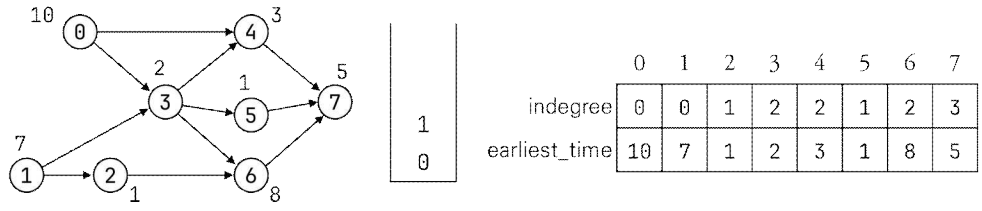
이제, AOV 네트워크에서 프로젝트를 완료하는데 걸리는 가장 빠른 시간을 구하는 효율적인 알고리즘을 소개합니다. 이 알고리즘은 11장 사례 연구에서 소개한 위상 정렬을 구하는 알고리즘을 기반으로 합니다. 알고리즘에서 관리하는 데이터는 다음과 같습니다.

`indegree[i]` : 작업 `i`의 선행 작업의 수 (방향 그래프의 전입 차수)
`work_time[i]` : 작업 `i`의 소요 시간 (입력으로 주어짐)
`earliest_time[i]` : 작업 `i`를 완료하는데 걸리는 가장 빠른 시간

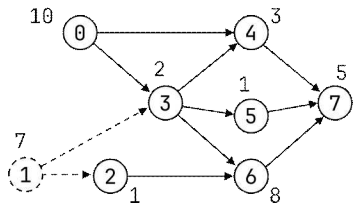
알고리즘은 초기에 각 정점(작업)의 전입 차수를 저장한 배열 `indegree`를 생성하고, 전입 차수가 0인 모든 정점을 스택(또는 큐)에 저장합니다. 그다음, 배열 `earliest_time`을 생성하여 `work_time`의 값으로 초기화합니다. 그런 다음, 스택(또는 큐)가 공백이 될 때까지 다음을 반복합니다.

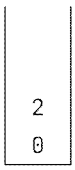
스택(큐)에서 삭제한 정점 `v`에 인접한 각 정점 `w`에 대해,
1. `indegree[w]`를 1 감소시키고, `indegree[w]`가 0이면 `w`를 스택(또는 큐)에 저장한다.
2. `earliest_time[w]`을 다음 값으로 갱신한다.
   `max(earliest_time[w], earliest_time[v] + work_time[w])`

알고리즘의 종료 후, 배열 `earliest_time`에는 각 작업의 가장 빠른 완료 시간이 저장됩니다. 그림 12.14의 (a) ~ (h)는 그림 12.13의 AOV 네트워크를 대상으로 스택을 사용하여 알고리즘을 수행하는 과정을 보여줍니다.



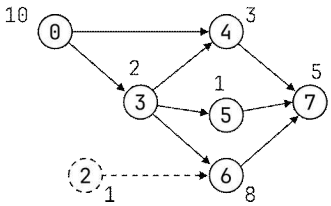
(a) 초기 상태: 스택에 0과 1 삽입





	0	1	2	3	4	5	6	7
indegree	0	0	0	1	2	1	2	3
earliest_time	10	7	8	9	3	1	8	5

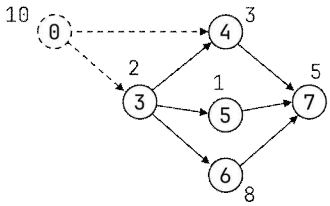
(b) 정점 1 삭제: 정점 2, 3의 indegree와 earliest_time 갱신, 스택에 정점 2 삽입

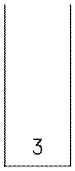




	0	1	2	3	4	5	6	7
indegree	0	0	0	1	2	1	1	3
earliest_time	10	7	8	9	3	1	16	5

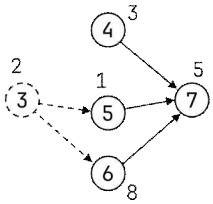
(c) 정점 2 삭제: 정점 6의 indegree와 earliest_time 갱신

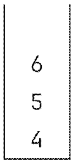




	0	1	2	3	4	5	6	7
indegree	0	0	0	0	1	1	1	3
earliest_time	10	7	8	12	13	1	16	5

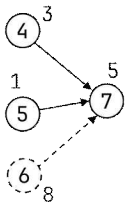
(d) 정점 0 삭제: 정점 3, 4의 indegree와 earliest_time 갱신, 스택에 정점 3 삽입





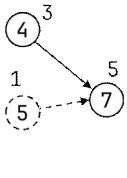
	0	1	2	3	4	5	6	7
indegree	0	0	0	0	0	0	0	3
earliest_time	10	7	8	12	15	13	20	5

(e) 정점 3 삭제: 정점 4, 5, 6의 indegree와 earliest_time 갱신, 스택에 정점 4, 5, 6 삽입



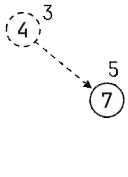
	0	1	2	3	4	5	6	7
indegree	0	0	0	0	0	0	0	2
earliest_time	10	7	8	12	15	13	20	25

(f) 정점 6 삭제: 정점 7의 indegree와 earliest_time 갱신



	0	1	2	3	4	5	6	7
indegree	0	0	0	0	0	0	0	1
earliest_time	10	7	8	12	15	13	20	25

(g) 정점 5 삭제: 정점 7의 indegree와 earliest_time 갱신



	0	1	2	3	4	5	6	7
indegree	0	0	0	0	0	0	0	0
earliest_time	10	7	8	12	15	13	20	25

(h) 정점 4 삭제: 정점 7의 indegree와 earliest_time 갱신



	0	1	2	3	4	5	6	7
indegree	0	0	0	0	0	0	0	0
earliest_time	10	7	8	12	15	13	20	25

(i) 정점 7 삭제: 알고리즘 종료

그림 12.14: 프로젝트의 가장 빠른 완료 시간을 구하는 과정

프로그램 12.9에는 인접 그래프 G로 표현된 AOV 네트워크에서 프로젝트의 가장 빠른 완료 시간을 구하는 함수 `get_earliest_time(G, work_time)`이 나와 있습니다. 프로그램은 11장의 위상 정렬 알고리즘에서 `earliest_time`을 갱신하는 코드만 추가된 형태이므로, 시간복잡도는 $O(n+m)$입니다.

프로그램 12.9: AOV 네트워크에서 가장 빠른 완료 시간을 구하는 프로그램

```
def get_earliest_time(G, work_time):
    """AOV 네트워크에서의 가장 빠른 완료 시간:
    인접 리스트 G로 표현된 방향 비순환 그래프에서 각 작업의 가장 빠른 완료 시간을
    저장한 리스트 earliest_time을 반환한다. G[v] = [인접 정점, ...],
    work_time[v] : 정점 v에 해당하는 작업의 소요 시간"""
    n = len(G)                              # 정점 수
    earliest_time = work_time.copy()        # earlist_time 초기화
    stack = []                              # 공백 스택 생성, 리스트 이용
    indegree = [0] * n                      # 전입 차수를 저장하는 리스트

    for v in range(n):                      # 각 정점의 전입 차수를 구함
        for w in G[v]:
            indegree[w] += 1

    for v in range(n):                      # 전입 차수가 0이면 스택에 삽입
        if indegree[v] == 0:
            stack.append(v)

    while len(stack) > 0:                   # 스택이 공백이면 종료
        v = stack.pop()                     # 스택의 top 정점 v 삭제
        for w in G[v]:                      # 방향 에지 <v, w>에 대해
            t = earliest_time[v] + work_time[w]
            if t > earliest_time[w]:        # early_time 갱신
                earliest_time[w] = t
            indegree[w] -= 1                # 전입 차수 1 감소
            if indegree[w] == 0:            # 전입 차수가 0이면 스택에 삽입
                stack.append(w)

    return earliest_time                    # 최단 완료 시간 반환
```

프로젝트의 가장 빠른 완료 시간을 알면, 프로젝트의 완료 시간에 영향을 주지 않으면서 각 작업의 완료를 최대로 미룰 수 있는 **가장 느린 완료 시간(latest finish time)**을 알 수 있습니다. 가장 느린 완료 시간을 저장하는 리스트를 `latest_time`이라고 할 때, `latest_time[i]`에는 작업 `i`의 가장 느린 완료 시간이 저장됩니다. 가장 느린 완료 시간은 가장 빠른 완료 시간을 구하는 과정과 유사하게 구해집니다. 먼저, 그래프의 모든 에지를 역방향으로 뒤집어 역방향 그래프를 생성합니다. 그리고 이 역방향 그래프에 대해 위상 정렬을 수행하면서 각 작업의 `latest_time`을 계산합니다. 수행 과정에서 `latest_time[v]`이 이미 구해진 정점 `v`에 인접한 정점 `w`에 대해, `latest_time[w]`을 다음과 같이 갱신합니다.

```
min(latest_time[w], latest_time[v] - work_time[v])
```

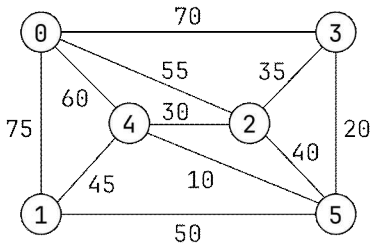
그림 12.13의 AOV 네트워크에서 각 작업의 가장 빠른 완료 시간과 가장 느린 완료 시간을 구한 결과가 아래에 나와 있습니다.

	0	1	2	3	4	5	6	7
earliest_time	10	7	8	12	15	13	20	25
latest_time	10	10	12	12	20	20	20	25

이 결과에 따르면, 작업 0, 2, 3, 6, 7은 가장 빠른 완료 시간과 가장 느린 완료 시간이 같습니다. 이러한 작업은 여유 시간이 없는 작업입니다. 이들 중 하나라도 계획보다 작업 시간이 늘어나면 전체 프로젝트의 완료 시간도 늘어나게 됩니다. 이렇게 프로젝트의 전체 일정에 영향을 주는 작업을 **중대 작업(critical activity)**이라고 부르고, 중대 작업들로 구성되는 방향 경로를 **임계 경로(critical path)**라고 합니다.

연습문제

1. 아래 그림의 그래프에 대해서 다음 물음에 답하시오.
   (1) 가중치 행렬로 표현하시오.
   (2) 최소 비용 신장 트리를 구하시오.



2. 프림 알고리즘과 크루스칼 알고리즘이 서로 다른 최소 비용 신장 트리를 구하는 그래프의 예를 그리시오.

3. 어떤 그래프의 모든 에지의 가중치가 서로 다르다면, 이 그래프의 최소 신장 트리(MST)는 유일함을 증명하시오.

4. 문제 1의 그래프에서 출발점이 정점 3인 최단 경로 트리를 그리시오.

5. 최단 경로를 구하는 다익스트라 알고리즘이 틀리게 동작하는 음수 가중치를 가진 방향 그래프의 예를 그리시오.

6. 다익스트라 알고리즘을 이용해 그래프의 신장 트리를 구하는 알고리즘을 설계하시오.

7. 다익스트라 알고리즘과 프림 알고리즘은 힙을 사용할 때 전체적으로 매우 유사하다. 두 알고리즘의 핵심적인 차이점(힙에 저장하는 값, 거리 갱신 방식 등)을 비교 설명하시오.

8. 음수 가중치를 포함한 그래프에서 '음수 가중치 사이클'이 존재할 경우, 플로이드-와샬 알고리즘이 왜 최단 경로를 정의할 수 없는지 설명하시오.

9. 가중치 그래프에서 두 정점 사이의 **거리(distance)**는 그 두 정점 사이의 최단 경로의 길이를 말하고, **지름(diameter)**은 가장 멀리 떨어져 있는 두 정점 사이의 거리를 말한다. 문제 1의 그래프에 대해 지름을 구하시오.

10. 가중치 그래프에서 지름을 구하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

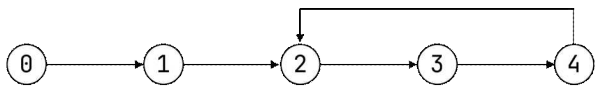
11. 연결 그래프의 **센터(center)**는 정점들의 부분 집합으로서 다음과 같이 정의된다. 정점 v에서 가장 멀리 떨어져 있는 정점까지의 거리를 max_dist(v)라고 하자. 그러면 센터에 속하는 정점 c의 max_dist(c)는 c가 아닌 정점 v의 max_dist(v)보다 항상 작거나 같다. 문제 1의 그래프에 대해 센터를 구하시오.

12. 가중치 그래프의 센터를 구하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

13. 연결 그래프의 **중앙(median)**은 정점들의 부분 집합으로서 다음과 같이 정의된다. 중앙에 속하는 정점 v에서 다른 모든 정점까지의 거리의 합은 v가 아닌 정점에서 다른 모든 정점까지의 거리의 합보다 항상 작거나 같다. 문제 1의 그래프에 대해 중앙을 구하시오.

14. 주어진 그래프의 중앙을 구하는 알고리즘을 설계하고, 시간복잡도를 분석하시오.

15. 그래프에 대한 **이행적 폐쇄(transitive closure)** 행렬 A^+는 다음과 같이 정의된다. 정점 v에서 정점 w로의 길이가 1 이상인 경로가 존재하는 경우 A^+[v][w] = 1이고, 그렇지 않으면 A^+[v][w] = 0이다. 아래 그림의 방향 그래프에 대한 이행적 폐쇄 행렬을 구하시오.



16. 플로이드-와샬 알고리즘을 이용해 그래프의 이행적 폐쇄 행렬을 구하려고 한다. 이행적 폐쇄 행렬을 구할 수 있도록 프로그램 12.8을 수정하시오.

17. 그래프에 대한 **반사 이행적 폐쇄(reflexive transitive closure)** 행렬 A^*는 다음과 같이 정의된다. 정점 v에서 정점 w로의 길이가 0 이상인 방향 경로가 존재하는 경우 A^*[v][w] = 1이고, 그렇지 않으면 A^*[v][w] = 0이다. 문제 15의 방향 그래프에 대한 반사 이행적 폐쇄 행렬을 구하시오.

18. 다익스트라 알고리즘을 사용하면 부모 포인트 배열 parent를 이용하여 최단 경로 자체를 구할 수 있다. 플로이드-와샬 알고리즘을 사용하였을 때도 두 정점 사이의 최단 경로를 구하고자 한다. 이를 위해 프로그램 12.8을 수정하여 아래 프로그램처럼 2차원 리스트 mid를 구하는 코드를 추가한다. mid[v][w]에는 두 정점 v와 w 사이의 최단 경로의 중간에 놓여 있는 정점 중 하나가 저장된다. 만일 v와 w 사이의 최단 경로가 에지 (v, w)이면 mid[v][w] = -1이다.

```
def floyd(M):
   n = len(M)                              # 정점 수
   A = [x.copy() for x in M]               # A의 초기화, M을 복사함
   mid = [[-1]*n for _ in range(n)]        # mid의 초기화
   for k in range(n):                      # 최단 거리를 단계별로 구함
      for v in range(n):
         for w in range(n):
            A[v][w] = min(A[v][w], A[v][k]+A[k][w])
            mid[v][w] = k                 # k는 v와 w 사이의 최단 경로에 위치함
   return A, mid                           # 최단 경로 행렬과 mid 반환
```

이 배열을 사용하여 두 정점 v와 w 사이의 최단 경로 자체를 구하는 방법을 다음과 같이 재귀적으로 표현할 수 있다.

> v와 w 사이의 최단 경로는 v와 mid[v][w] 사이의 최단 경로와 mid[v][w]와 w 사이의 최단 경로로 구성된다.

위의 방법을 이용하여 두 정점 v와 w 사이의 최단 경로 자체를 구하는 프로그램을 작성하시오.

19. 음수 가중치를 허용하는 단일 출발점 최단 경로 알고리즘인 **벨만-포드 알고리즘(Bellman-Ford algorithm)**을 조사하여 다익스트라 알고리즘과 비교하고 장단점을 설명하시오.

20. 최소 신장 트리를 구하는 알고리즘으로, 프림 알고리즘과 크루스칼 알고리즘 외에도 **보루프카 알고리즘(Borůvka's algorithm)**이 있다. 이 알고리즘의 특징과 시간복잡도를 조사하시오. 보루프카 알고리즘은 **솔린 알고리즘(Sollin's algorithm)**이라고도 불린다.

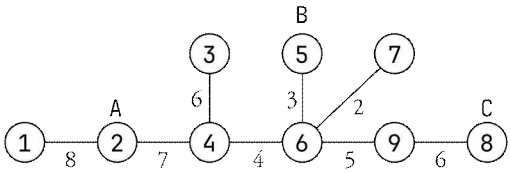
21. AOV 네트워크에서 가장 빠른 완료 시간과 가장 느린 완료 시간을 구하는 프로그램을 작성하시오.

22. [실습 문제] n×n 크기의 격자 형태로 구성된 2차원 동굴 지도가 있다. 각 칸에는 그 칸을 탐사하는 동안 소모되는 에너지 손실량이 기록되어 있다. 출발점 (0, 0)에서 종착점 (n-1, n-1)까지 이동하는 경로에서 총 에너지 손실량을 최소로 하는 경로를 찾는 프로그램을 작성하시오. 이동은 현재 칸에서 상, 하, 좌, 우 네 방향으로 인접한 칸으로만 이동할 수 있다.

입력으로, 첫 번째 줄에는 동굴 지도의 크기 n×n을 나타내는 정수 n이 주어진다. n은 1 이상 125 이하이다. 그다음 n개의 줄에는 각 줄에 지도의 각 행에 해당하는 정보가 순서대로 들어온다. 이 정보는 각 칸의 에너지 손실량을 나타내는 n개의 양의 정수이다. 에너지 손실량은 1 이상 9 이하이다. 출력으로, 출발점 (0, 0)에서 종착점 (n-1, n-1)까지의 총 에너지 손실량 중 최솟값을 나타내는 정수를 출력한다.

입력의 예	출력의 예
3 5 5 4 3 9 1 3 2 7	20

23. [실습 문제] 창업을 희망하는 K는 대학 졸업 후 브런치 카페를 차리려고 준비하고 있다. K가 거주하는 지역의 도로망은 아래 그림처럼 트리 형태로 구성되어 있다. 트리의 정점은 카페 설치 후보지이고, 에지는 도로를 나타내며, 선 옆에 표시된 값은 도로의 길이이다.



후보지 중에는 아파트 단지가 위치한 곳이 있다. 위 그림에는 A, B, C로 표시된 아파트 단지 3곳이 후보지 2, 5, 8에 위치한다. K는 경쟁력과 수익성을 고려할 때 다음의 조건을 충족하는 후보지가 최적지라고 생각한다.

최적지 p는 다른 후보지 q와 비교할 때, 아파트 단지 중에 적어도 한 곳과는 거리가 더 가까워야 한다. 즉, d(x, y)를 후보지 x와 y 사이의 최단 거리라고 할 때, 최적지 p는

다른 후보지 q에 대해 d(p, z) < d(q, z)인 아파트 단지가 위치한 후보지 z가 존재해야 한다.

위 그림에서 커피전문점 최적지는 2, 4, 5, 6, 8, 9이다. 예를 들어, 후보지 6은 후보지 5보다 A와 C에 더 가깝고, 나머지 후보지보다는 B에 더 가까우므로 최적지에 포함된다. 후보지 7은 후보지 6과 비교할 때 아파트 단지 A, B, C의 어떤 한 곳도 더 가까이 있지 않기 때문에 최적지가 될 수 없다. 도로망과 아파트 단지에 대한 정보가 주어질 때, 최적지의 개수를 출력하는 프로그램을 작성하시오.

입력으로, 첫 줄에 후보지 개수 n과 아파트 단지의 개수 h가 주어진다. n은 1 이상 10^5 이하이고, h는 1 이상 n 이하이다. 후보지는 1부터 n까지 번호로 구분된다. 그다음 n-1개의 줄에는 각 줄에 세 정수 i, j, w가 주어진다. i와 j는 서로 인접한 후보지이고 w는 i와 j 사이 도로의 길이이다. 마지막 줄에는 아파트 단지가 위치한 후보지의 번호를 나타내는 h개의 정수가 한 줄에 들어온다. 출력으로, 최적지의 개수를 출력한다.

입력의 예	출력의 예
9 3 1 2 8 2 4 7 4 3 6 4 6 4 5 6 3 6 7 2 6 9 5 9 8 6 2 5 8	6

입력의 예	출력의 예
4 4 1 2 1 1 3 1 1 4 1 2 4 1 3	4

제13장

탐색 구조

13.1 균형 이진 탐색 트리
13.2 B-트리
13.3 해싱
13.4 사례 연구: 파이썬에서의 해싱
연습문제

학습 목표

- 일반적인 이진 탐색 트리의 단점인 최악의 경우 시간복잡도를 개선하는 높이 균형 탐색 트리의 개념을 이해하고 이를 설명할 수 있다.
- AVL 트리와 레드-블랙 트리의 정의와 탐색 및 삽입 연산을 자세하게 설명할 수 있다.
- B-트리의 정의와 특징, 2-3-4 트리와 레드-블랙 트리와의 관계, 2-3 트리에서 탐색 및 삽입 연산을 자세하게 설명할 수 있다.
- 해싱의 개념, 해시 테이블과 해시 함수, 개방 주소법, 해시 체인법을 설명할 수 있다.
- 해싱 기반 자료형인 dict와 set의 동작 원리를 이해하고, 이를 문제 해결에 활용할 수 있다.

13.1 균형 이진 탐색 트리

동적 파일의 효율적인 표현을 위해 10.1절에서 소개한 이진 탐색 트리를 이용할 수 있습니다. 이진 탐색 트리는 일반적인 이진 트리 구조이며 각 노드에 탐색의 기준이 되는 키가 있습니다. 이 키는 왼쪽 부트리에 있는 모든 키보다 크고 오른쪽 부트리에 있는 모든 키보다 작습니다. 이진 탐색 트리에서 탐색, 삽입, 삭제의 세 연산은, 트리의 높이를 h라고 할 때, 모두 $O(h)$ 시간에 수행됩니다. 노드 수가 n인 이진 탐색 트리의 높이는 무작위로 트리가 만들어지는 경우 확률적으로 $O(\log n)$이지만 일반적으로는 $O(n)$입니다. 최악의 경우, 이진 탐색 트리는 높이가 n-1인 경사 이진 트리가 될 수 있습니다. 예를 들어, 이미 정렬된 상태의 자료를 순서대로 이진 탐색 트리에 삽입하면 경사 이진 트리가 됩니다.

최악의 경우에도 탐색, 삽입, 삭제 세 연산이 모두 $O(\log n)$ 시간에 수행하기 위해서는 트리의 높이 자체가 $O(\log n)$으로 유지되어야 합니다. 높이를 이렇게 유지하는 탐색 트리를 **높이 균형 탐색 트리(height balanced search tree)**라고 부릅니다. 높이 균형 탐색 트리에는 여러 가지 종류가 있습니다. 그중에는 이진 트리도 있고 일반적인 트리 형태도 있습니다. 높이 균형 탐색 트리를 구축하는 알고리즘은 매우 복잡하여 프로그램을 직접 구현하기가 어렵기 때문에, 보통은 프로그래밍 언어에서 제공하는 라이브러리를 이용하게 됩니다. Java는 `TreeSet`, `TreeMap`이라는 이름의 라이브러리를 제공하고, C++은 `stl::set`, `stl:map`, `stl:multiset`, `stl:multimap` 등의 라이브러리를 제공합니다. 그러나 파이썬은 기본 탐색 구조로 평균 성능이 더 뛰어난 해싱 기반의 `dict`와 `set`을 채택했기 때문에, 아쉽게도 표준 라이브러리에 높이 균형 탐색 트리가 포함되어 있지 않습니다[33].

13.3절에서 소개하는 해싱과 비교할 때, 높이 균형 탐색 트리의 특징이자 장점은 정렬과 관련된 연산을 효율적으로 수행할 수 있다는 점입니다. 즉 자료 원소들을 정렬하거나, 특정 자료 원소의 다음 원소 또는 이전 원소, 그리고 상한과 하한을 빠르게 찾을 수 있습니다.

이 절에서는 프로그래밍 언어에서 라이브러리를 구현할 때 주로 사용되는 높이 균형 이진 탐색 트리인 AVL 트리와 레드-블랙 트리에 대해서 알아봅니다.

■ AVL 트리

1962년 Adelson-Velskii와 Kandis에 의해 고안되어 그들의 이름을 따서 명명된 **AVL 트리**는

33) 설치 과정이 필요한 외부 모듈로는 AVL 트리를 구현한 `PyAVLTree`와 레드-블랙 트리를 구현한 `rb-tree` 등을 이용할 수 있습니다.

트리의 높이가 항상 $O(\log n)$으로 유지되는 이진 탐색 트리입니다. 따라서 AVL 트리에서는 탐색뿐만 아니라 삽입과 삭제 연산도 $O(\log n)$ 시간에 수행됩니다.

AVL 트리의 각 노드에는 왼쪽 부트리의 높이에서 오른쪽 부트리의 높이를 뺀 값인 **균형 인자(balance factor)**가 부여됩니다. 모든 노드에서 균형 인자는 -1, 0, +1 중의 하나라는 AVL 트리의 핵심적인 성질이 트리의 높이를 항상 $O(\log n)$으로 유지해 줍니다. 더 정확하게 얘기하면, AVL 트리의 최대 높이는 $1.44 \times \log n$을 초과하지 않는다는 것이 보장됩니다. 그림 13.1은 AVL 트리의 예를 보여줍니다. 그림에서 노드 옆에 균형 인자가 표시되어 있습니다.

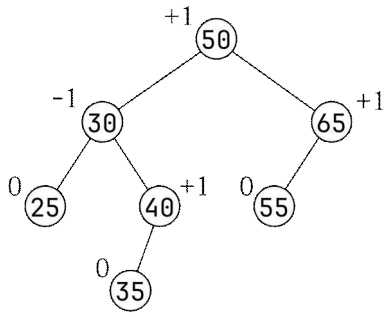


그림 13.1: AVL 트리와 균형 인자

AVL 트리에서의 삽입 및 삭제는 기본적으로 이진 탐색 트리에서의 삽입 및 삭제와 같은 방식으로 수행됩니다. 삽입 및 삭제 시에 AVL 트리에서 높이의 불균형이 발생하면, 트리의 일부 또는 전체에 대한 회전 변환을 수행하여 높이의 균형을 맞춥니다. 회전 변환을 위한 조건은 다음의 4가지 유형으로 구분됩니다.

- LL : 인접한 두 노드의 균형 인자가 +2, +1로 구성된 경우
- RR : 인접한 두 노드의 균형 인자가 -2, -1로 구성된 경우
- LR : 인접한 두 노드의 균형 인자가 +2, -1로 구성된 경우
- RL : 인접한 두 노드의 균형 인자가 -2, +1로 구성된 경우

LL 유형이 발생하는 상황과 높이 균형을 맞추기 위한 변환을 그림 13.2에서 볼 수 있습니다. 그림 13.2 (a)의 상태에서 노드 B의 왼쪽 부트리에 새로운 노드가 삽입되면, (b)처럼 노드 A의 균형 인자가 +2로 불균형이 발생합니다. 이때, (c)처럼 노드 A를 노드 B의 오른쪽 자식으로 두고, 노드 B의 오른쪽 부트리(그림의 β)를 노드 A의 왼쪽 부트리로 옮기면 높이의 균형이 맞추어집니다. 이와 같이 (b) 상태에서 (c) 상태로 변환하는 것을 **LL 회전 변환**이라고 부릅니다.

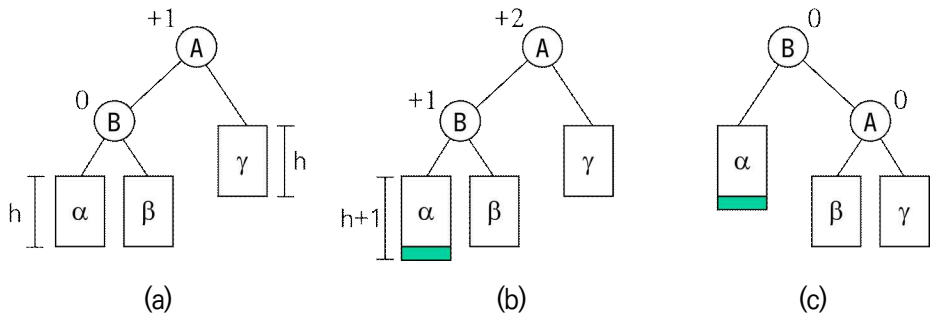


그림 13.2: LL 유형과 LL 회전 변환

RR 유형은 LL 유형과 좌우 대칭적입니다(그림 13.3 참조). 그림 13.3의 (b) 상태에서 (c) 상태로 변환하는 것을 **RR 회전 변환**이라고 부릅니다.

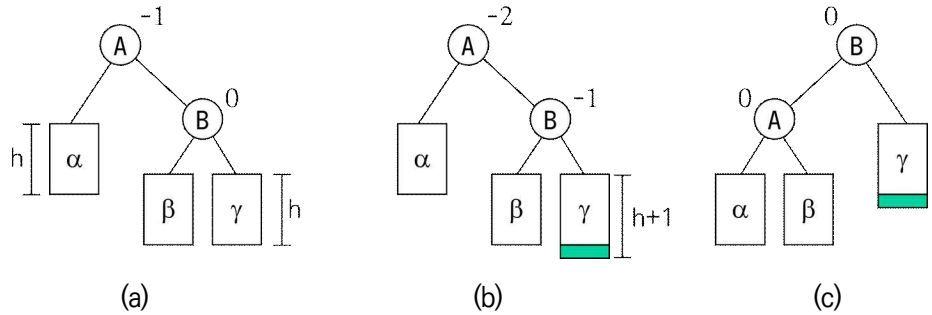


그림 13.3: RR 유형과 RR 회전 변환

LR 유형과 RL 유형은 각각 그림 13.4와 그림 13.5의 상태에서 발생합니다. 그림 13.4 (a)에서 노드 C의 왼쪽 부트리나 오른쪽 부트리에 새로운 노드가 삽입되면 (b)처럼 노드 A의 균형 인자가 +2로 불균형이 발생합니다. 이때 노드 A의 자식 노드 B의 균형 인자는 -1이므로 LR 유형이 됩니다. 그림 13.5 (a)에서 노드 C의 왼쪽 부트리나 오른쪽 부트리에 새로운 노드가 삽입되면 (b)처럼 노드 A의 균형 인자가 -2로 불균형이 발생합니다. 이때 노드 A의 자식 노드 B의 균형 인자는 +1이므로 RL 유형입니다.

LR 회전 변환은 그림 13.4의 (b) 상태에서 그림 13.6의 (a)처럼 노드 A와 노드 B를 각각 노드 C의 오른쪽 자식과 왼쪽 자식으로 두는 변환을 말합니다. **RL 회전 변환**은 그림 13.5의 (b) 상태에서 그림 13.6의 (b)처럼 노드 A와 노드 B를 각각 노드 C의 왼쪽 자식과 오른쪽 자식으로 두는 변환을 말합니다. 두 변환 모두 수행 결과로 높이의 균형이 맞추어집니다.

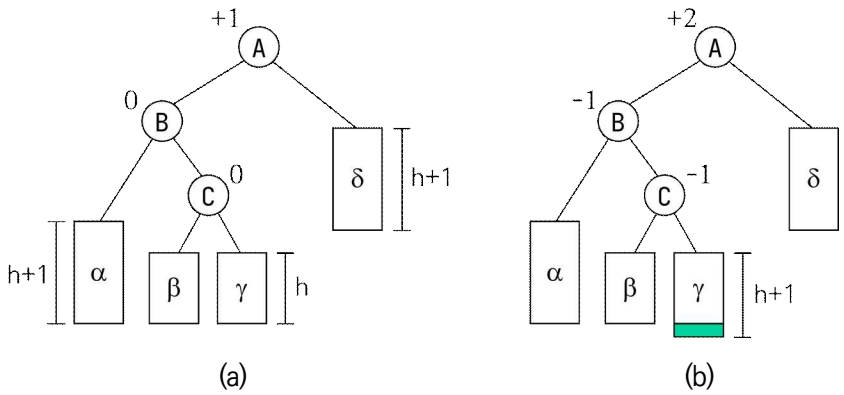


그림 13.4: LR 유형의 예

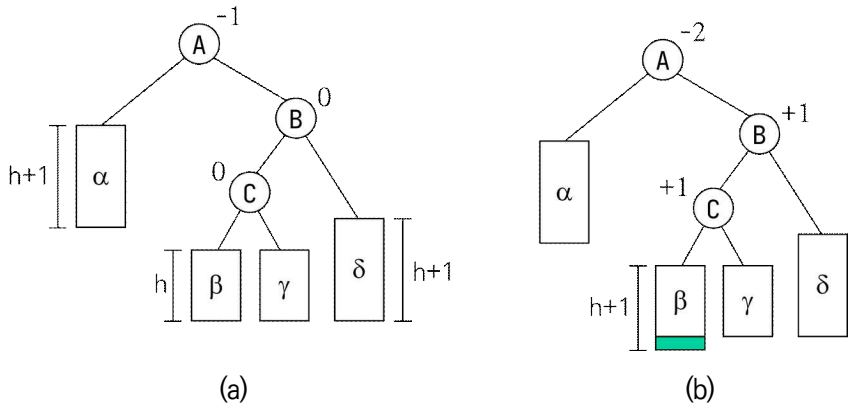


그림 13.5: RL 유형의 예

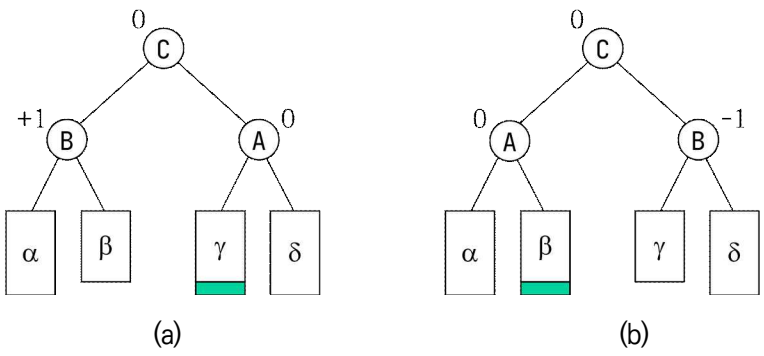
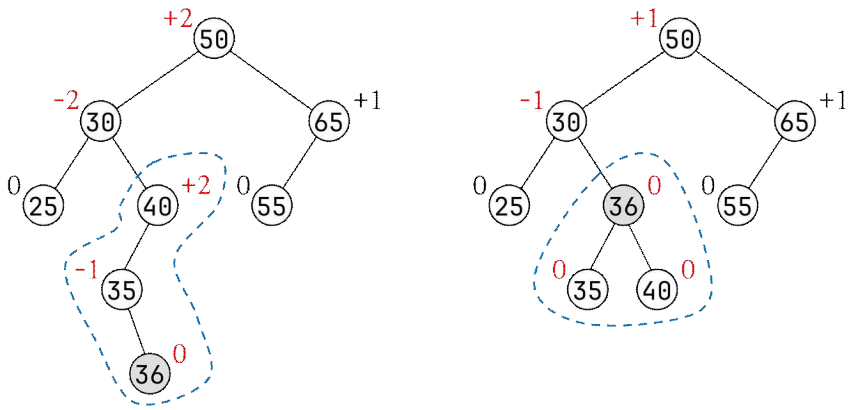


그림 13.6: LR 회전과 RL 회전 변환의 예

AVL 트리에서 새로운 노드를 삽입하는 과정은 다음과 같습니다. 먼저 이진 탐색 트리에서와 같은 방식으로 삽입을 수행합니다. 그다음, 삽입된 노드로부터 그 조상들을 차례로 검사하면서 노드의 균형 인자를 갱신합니다. 만일 갱신된 균형 인자에 이상이 없으면 그대로 삽입 과정을 종료합니다. 그렇지 않으면 균형 인자가 +2 또는 -2가 되는 첫 번째 노드와 그의 자식 노드의 균형 인자를 조사하여 LL, RR, LR, RL 유형 중 어디에 해당하는지 파악하고, 해당 유형에 대해 상응하는 회전 변환을 수행하면 삽입 과정이 종료됩니다. 이때 한 번의 회전 변환만으로 트리 전체의 균형을 맞출 수 있습니다. 회전 변환의 수행 시간은 상수 시간이기 때문에 삽입 과정의 수행 시간은 결국 트리의 높이에 좌우됩니다. 그러므로 삽입 연산은 $O(\log n)$ 시간에 수행됩니다.

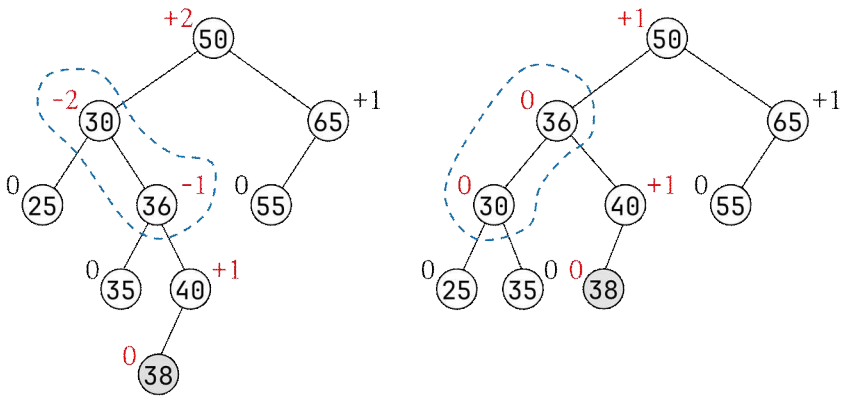
예를 들어, 그림 13.7은 그림 13.1의 AVL 트리에 노드 36의 삽입 과정을 보여주고, 그림 13.8은 그림 13.7 (b)의 AVL 트리에 노드 38의 삽입 과정을 보여줍니다. 이 그림에서 볼 수 있듯이 회전에 관련된 노드 이외에는 균형 인자의 변동이 전혀 없습니다.



(a) 단순 삽입 결과 (b) LR 회전 변환 후 결과

그림 13.7: 그림 13.1의 AVL 트리에서 노드 36의 삽입 과정

AVL 트리에서의 삭제 과정은 삽입 과정에 비해 약간 복잡합니다. 여기서는 개념 위주로 설명하며 자세한 내용은 생략합니다. 먼저 이진 탐색 트리에서와 같은 방식으로 삭제를 수행합니다. 그다음, 삭제된 노드 위치에서부터 그 조상들을 차례로 검사하면서 갱신된 균형 인자에 이상이 없는지 살펴봅니다. 균형 인자에 이상이 없으면 그대로 삭제 과정을 종료합니다. 그렇지 않으면, 균형 인자가 +2 또는 -2인 첫 번째 노드와 그 노드의 자식 노드를 통해 4가지 불균형 유형 중 해당하는 유형을 찾고, 그 유형에 상응하는 회전 변환을 수행합니다. 삭제 과정은 삽입 과정과는 달리 단 한 번의 회전 변환으로 균형이 맞추어지지 않을 수 있습니다. 따라서 조



(a) 단순 삽입 결과 (b) RR 회전 변환 후 결과

그림 13.8: 그림 13.7 (b)의 AVL 트리에서 노드 38의 삽입 과정

상 노드를 계속해서 따라가면서 균형 인자에 이상이 없는지 검사해야 합니다. 삭제 과정에서는 최대 $O(\log n)$번의 회전 변환이 필요할 수 있지만, 전체 시간복잡도는 $O(\log n)$입니다.

■ 레드-블랙 트리

레드-블랙 트리(red-black tree)는 1978년 Guibas와 Sedgewick에 의해서 설계된 높이 균형 이진 탐색 트리입니다. AVL 트리는 좌우 부트리의 높이 차이를 최대 1로 설정하여 엄격하게 균형을 유지하기 때문에, 삭제 연산 시에 빈번한 회전 변환이 발생할 수 있습니다. 이에 반해, 레드-블랙 트리는 약간 느슨한 균형 조건을 사용하여 회전 변환이 발생하는 횟수를 줄이면서도 트리의 높이를 $O(\log n)$으로 유지합니다.

레드-블랙 트리는 각 노드에 Red 또는 Black으로 색상이 지정되며 다음의 규칙을 만족합니다.

① 루트 노드는 Black이다.
② 모든 외부 노드는 Black이다. 여기서 외부 노드는 데이터를 가지지 않은 가상의 노드로서 NIL 노드라고 부른다.
③ Red 노드의 자식은 항상 Black이다. 즉, Red 노드가 연속해서 나올 수 없다.
④ 루트에서 외부 노드까지의 모든 경로는 같은 개수의 Black 노드를 가진다.

그림 13.9에는 레드-블랙 트리의 예가 나와 있습니다. 그림에서 흰 노드는 Red 노드, 칠해진 노드는 Black 노드를 나타냅니다. NIL에 해당하는 외부 노드는 사각형으로 표시되어 있습니다. NIL 노드의 도입으로 이진 탐색 트리의 모든 노드가 두 개의 자식을 갖게 되어, 레드-블랙 트리

의 규칙을 쉽게 설명하고 이해하는 데 도움이 됩니다. 그림 13.10 (a)의 트리는 레드-블랙 트리이지만, 그림 13.10 (b)의 트리는 레드-블랙 트리가 아닙니다. 그림 13.10 (c)처럼 NIL 노드를 표시함으로써 그림 13.10 (b)의 트리가 규칙 ④를 위반한다는 것을 쉽게 알 수 있습니다.

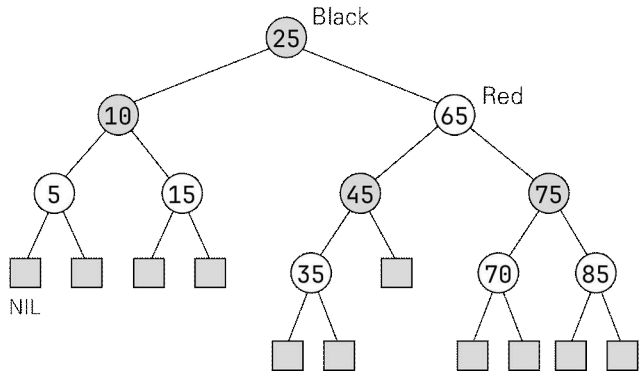
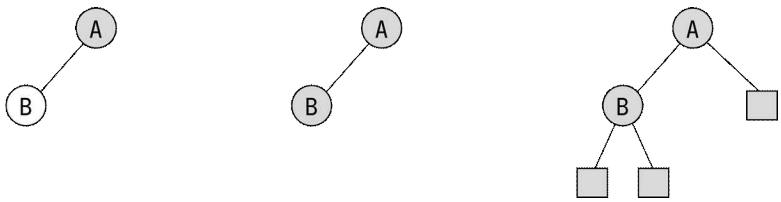


그림 13.9: 레드-블랙 트리의 예



(a) 레드-블랙 트리 (b) 비 레드-블랙 트리 (c) 규칙 ④의 위반

그림 13.10: 레드-블랙 트리에서 NIL 노드 도입

위의 규칙을 만족하는 레드-블랙 트리의 최대 높이는 2log(n+1)을 초과하지 않는다는 것이 보장됩니다. 레드-블랙 트리도 높이의 균형을 유지하기 위해 AVL 트리와 유사한 회전 변환을 수행합니다.

이제, 레드-블랙 트리에서의 삽입 과정을 살펴봅니다. 삽입 과정의 첫 단계는 이진 탐색 트리에서와 같은 방식으로 삽입을 수행하는 것입니다. 그러면 삽입될 노드는 이진 탐색 트리의 잎 노드(NIL 노드가 아님)가 되고, 이 노드를 Red 노드로 지정합니다. 만일, 이 노드가 루트이면 Black으로 색상을 변경한 후 삽입 과정을 종료하고, 부모가 Black 노드인 경우는 그대로 삽입 과정을 종료합니다. 그렇지 않은 경우, 즉 부모가 Red 노드이면, Red 노드가 연속해서 나올 수 없다는 규칙을 위반하므로, 이를 해결하기 위한 처리 방안이 필요합니다.

새로 삽입된 노드를 NN(New Node), 그의 부모 노드를 PN(Parent Node), 조부모 노드를 GN(Grandparent Node), 부모의 형제, 즉, 삼촌 노드를 UN(Uncle Node)이라고 합시다. 현재 NN과 PN은 둘 다 Red 색상으로 지정되어 있습니다. 규칙을 위반하는 상황을 8가지 유형으로 구분할 수 있는데, PN이 GN의 왼쪽 자식인 경우와 오른쪽 자식인 경우에 대해 각각 4가지 유형으로 구분합니다.

① PN이 GN의 왼쪽 자식인 경우
- LLr : NN이 PN의 왼쪽 자식이고 UN이 Red 노드인 경우 (그림 13.11 참조)
- LRr : NN이 PN의 오른쪽 자식이고 UN이 Red 노드인 경우 (그림 13.12 참조)
- LLb : NN이 PN의 왼쪽 자식이고 UN이 Black 노드인 경우 (그림 13.13 참조)
- LRb : NN이 PN의 오른쪽 자식이고 UN이 Black 노드인 경우 (그림 13.14 참조)

② PN이 GN의 오른쪽 자식인 경우
- RRr : NN이 PN의 오른쪽 자식이고 UN이 Red 노드인 경우
- RLr : NN이 PN의 왼쪽 자식이고 UN이 Red 노드인 경우
- RRb : NN이 PN의 오른쪽 자식이고 UN이 Black 노드인 경우
- RLb : NN이 PN의 왼쪽 자식이고 UN이 Black 노드인 경우

RRr, RLr, RRb, RLb 유형은 각각 LLr, LRr, LLb, LRb 유형과 좌우 대칭적입니다. 예를 들어, RRr은 GN의 오른쪽 자식이 PN이고 PN의 오른쪽 자식이 NN이며, UN이 Red 노드임을 의미하고, RLb는 GN의 오른쪽 자식이 PN이고 PN의 왼쪽 자식이 NN이며, UN이 Black 노드임을 의미합니다.

각 유형에 대해서 색상 변경이나 회전 변환을 통해 위반 상황을 해결합니다. LLr 유형과 LRr 유형은 회전 변환 없이 노드의 색상만 변경합니다. 그림 13.11과 그림 13.12에 나와 있듯이 PN을 Black으로, GN을 Red로 색상을 변경합니다. 이 경우, 루트에서 외부 노드까지 경로에 있는 Black 노드의 수는 변동이 없습니다. 그렇지만, GN이 Red 노드로 변경되었기 때문에, GN의 부모가 Red 노드이면 트리의 위쪽으로 올라가서 재조정을 해야 합니다. RRr 유형과 RLr 유형은 각각 LLr 유형과 LRr 유형의 과정을 좌우 대칭적으로 수행하면 됩니다.

LLb 유형과 LRb 유형의 경우는 회전 변환과 색상 변경을 동시에 수행합니다. LLb 유형에 대한 회전 변환은 AVL 트리의 LL 회전과 같습니다. 그림 13.13에 나와 있듯이, LLb 유형은 PN을 GN 위치로 이동시켜서 해결합니다. 이를 위해 GN을 PN의 오른쪽 자식으로 두고, PN의 기존 오른쪽 부트리(그림의 γ)는 GN의 왼쪽 부트리로 옮깁니다. 그런 다음, PN을 Black 노드로, GN을 Red 노드로 색상을 변경하면, 레드-블랙 트리의 모든 규칙을 만족하게 됩니다.

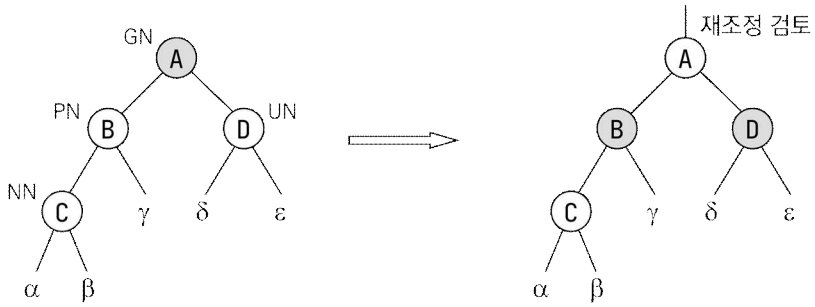


그림 13.11: LLr 유형에 대한 색상 변경

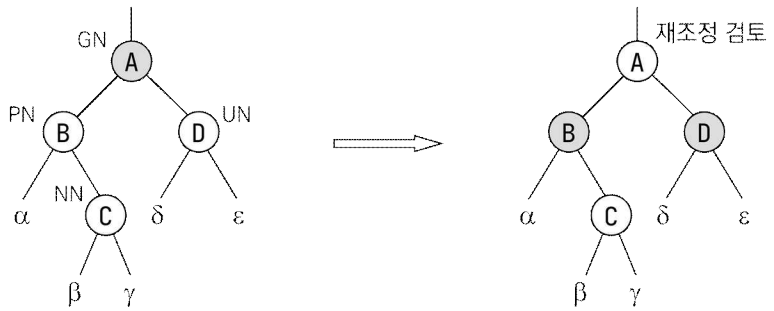


그림 13.12: LRr 유형에 대한 색상 변경

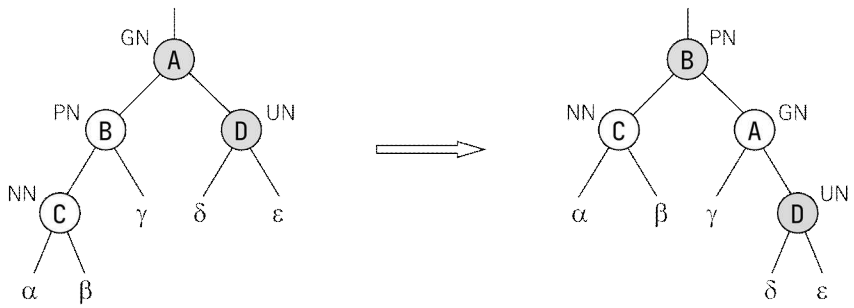


그림 13.13: LLb 유형에 대한 회전 변환 및 색상 변경

LRb 유형에 대한 회전 변환은 AVL 트리의 LR 회전과 같습니다. 그림 13.14에 나와 있듯이, LRb 유형은 NN을 GN 위치로 이동시켜서 해결합니다. 이를 위해 PN은 NN의 왼쪽 자식으로, GN은 NN의 오른쪽 자식으로 둡니다. NN의 기존 왼쪽 부트리(그림의 β)는 PN의 오른쪽 부트리로, NN의 기존 오른쪽 부트리(그림의 γ)는 GN의 왼쪽 부트리로 옮깁니다. 그런 다음, NN을 Black 노드로, GN을 Red 노드로 색상을 변경하면, 레드-블랙 트리의 모든 규칙을 만족하게

됩니다.

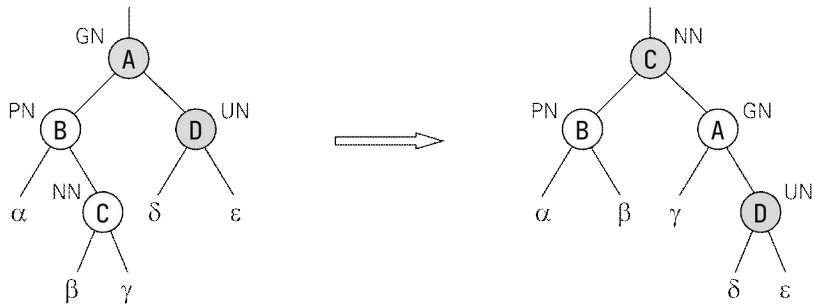
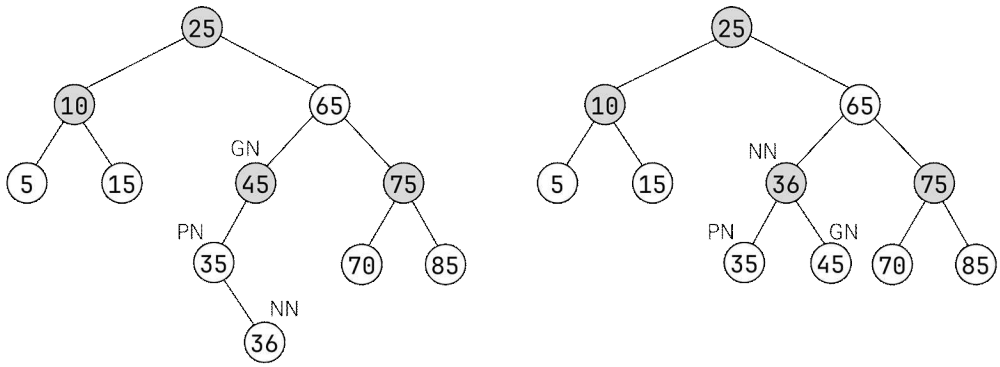


그림 13.14: LRb 유형에 대한 회전 변환 및 색상 변경

LLb 유형과 LRb 유형은 변환 후 부트리의 루트가 Black 노드이므로, 추가의 변환 없이 삽입 과정을 그대로 종료합니다. RRb 유형과 RLb 유형은 각각 LLb 유형과 LRb 유형의 과정을 좌우 대칭적으로 수행하면 됩니다. 각 노드에 대한 색상 변경 및 회전 변환은 상수 시간에 수행되므로, 삽입 과정은 $O(\log n)$ 시간에 수행됩니다.

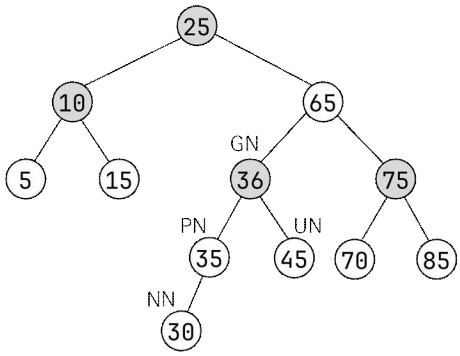
예를 들어, 그림 13.15는 그림 13.9의 레드-블랙 트리에 노드 36을 삽입하는 과정을 보여줍니다. 그림에서 NIL 노드는 생략되어 있습니다. 그림 13.15 (a)는 노드 36을 단순하게 삽입한 모습입니다. 이때, 새로 삽입된 노드 36의 부모 노드가 Red 노드이므로 규칙을 위배합니다. 조부모 노드와 부모 노드, 그리고 NIL 노드인 삼촌 노드를 고려할 때, 규칙 위반 유형은 LRb 유형에 해당합니다. 따라서 LRb 유형에 대한 회전 변환을 수행하면 그림 13.15(b)와 같이 레드-블랙 트리의 모든 규칙을 만족하게 됩니다.



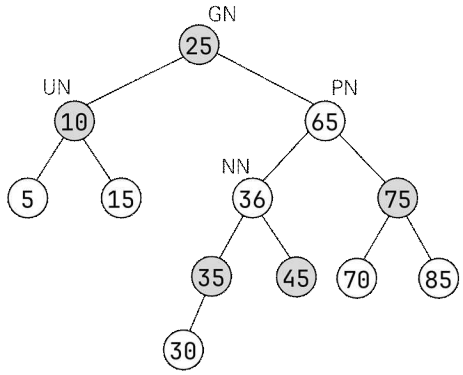
(a) 단순 삽입 결과 (LRb 유형) (b) LRb 유형의 색상 및 회전 변환

그림 13.15: 그림 13.9의 레드-블랙 트리에서 노드 36의 삽입 과정

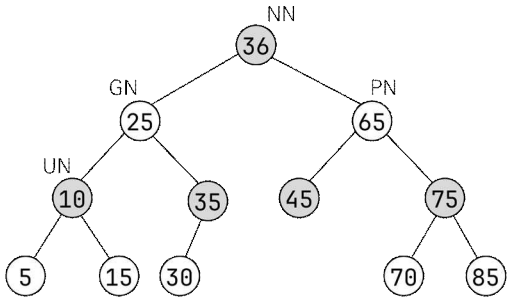
그림 13.16은 그림 13.15(b)의 레드-블랙 트리에 노드 30을 삽입하는 과정을 보여줍니다. 그림 13.16 (a)는 노드 30을 단순하게 삽입한 모습인데, 새 노드 30의 부모가 Red 노드이므로 규칙을 위배합니다. 규칙 위반 유형인 LLr 유형에 대한 색상 변환을 수행하면 그림 13.16 (b)와 같은 모습이 되는데, 노드 36이 Red 노드가 되고, 그 부모인 노드 65도 Red 노드이므로, 다시 RLb 유형에 해당하는 규칙 위배 상황이 발생합니다. 그림 13.16 (c)는 RLb 유형에 대한 색상 변경 및 회전 변환을 수행한 후의 모습입니다. 이제, 모든 규칙을 만족하게 되어 변환 작업이 종료됩니다.



(a) 단순 삽입 결과 (LLr 유형)



(b) LLr 유형의 색상 변경 결과 (RLb 유형)



(c) RLb 유형의 색상 및 회전 변환

그림 13.16: 그림 13.15 (b)의 레드-블랙 트리에서 노드 30의 삽입 과정

레드-블랙 트리에서 삭제 과정은 삽입 과정에 비해 훨씬 복잡하지만, 삽입 과정과 마찬가지로 $O(\log n)$ 시간에 수행됩니다. 삭제 과정에 대한 설명은 생략합니다.

13.2 B-트리

B-트리는 Bayer와 McCreight가 1970년에 발표한 탐색 트리로서 B-트리와 B-트리의 변형된 형태는 데이터베이스의 내부적 표현을 구현하기 위한 인덱싱 구조로서 널리 활용되고 있습니다. 앞서 살펴봤던 AVL 트리와 레드-블랙 트리는 주기억장치(RAM)에 모든 데이터가 저장되어 있다고 가정할 때 매우 효율적인 자료구조입니다. 만약, 데이터가 매우 방대하여 하드 디스크나 SSD와 같은 보조기억장치에 저장해야 하는 경우, 탐색 등의 연산을 위해 보조기억장치에 여러 번 접근해야 합니다. 최악의 경우는 트리의 높이에 해당하는 접근이 필요할 수 있습니다. 보조기억장치는 주기억장치에 비해 접근 속도가 매우 느리므로, 가능한 접근 횟수를 줄이는 노력이 필요합니다. 노드 수 n인 이진 트리의 높이는 적어도 $\lfloor \log n \rfloor$이므로, 데이터의 양이 수억, 수십억의 크기이면 logn의 디스크 접근에도 상당한 시간이 소요됩니다. 트리의 높이를 줄이기 위해서는 트리의 차수가 커야 하는데, B-트리는 차수가 크면서도 균형이 잘 잡힌 구조입니다. 이런 이유로 B-트리는 디스크 기반의 대용량 데이터 저장 및 검색에 최적화된 자료구조라고 알려져 있습니다.

이제, B-트리의 정의를 알아봅니다. 차수가 m인 B-트리는 다음 규칙을 만족하는 탐색 트리입니다.

① 잎 노드가 아닌 루트는 두 개 이상의 자식을 가진다.
② 루트와 잎 노드를 제외한 모든 노드는 $\lceil m/2 \rceil$개 이상의 자식을 가진다.
③ 모든 잎 노드의 레벨은 같다.

이진 트리 중에는 포화 이진 트리의 형태만 위의 규칙을 충족하기 때문에, B-트리의 차수는 3 이상이어야 합니다. 각 노드는 자식의 개수보다 하나 적은 개수의 키를 가지고 있습니다. 따라서 규칙 ②를 만족하려면, 루트와 잎 노드를 제외한 노드는 $\lceil m/2 \rceil - 1$ 이상 m-1 이하 개수의 키를 가지고 있어야 합니다. 키들은 노드 안에서 오름차순으로 정렬된 상태로 저장됩니다.

예를 들어, 그림 13.17은 차수가 4인 B-트리의 예를 보여줍니다. 차수가 4인 B-트리에서 잎 노드가 아닌 각 노드는 2 이상 4 이하 개수의 자식을 가지며, 이에 따라 각 노드에는 1 이상 3 이하 개수의 키가 있습니다. 네 개의 자식을 가진 노드에는 세 개의 키 a, b, c가 있습니다(a < b < c). 이때, 첫 번째 자식이 루트인 부트리에 있는 모든 키는 a보다 작고, 두 번째 자식이 루트인 부트리에 있는 모든 키는 a와 b 사이의 값이고, 세 번째 자식이 루트인 부트리에 있는 모든 키는 b와 c 사이이고, 마지막 네 번째 자식이 루트인 부트리에 있는 모든 키는 c보다 큽니다.

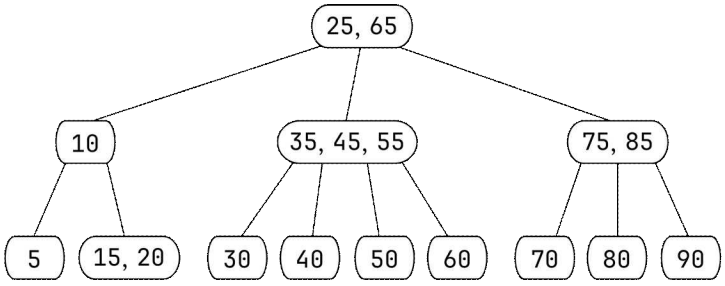


그림 13.17: 차수가 4인 B-트리의 예

n개의 키를 가진 B-트리의 최대 높이는 모든 노드가 최소 개수의 자식을 가질 때 구해집니다. 바로 루트 노드의 자식 수는 2이고, 나머지 내부 노드는 $\lceil m/2 \rceil$ 개의 자식을 가지는 경우입니다. 이 경우, 키의 수 n과 높이 h는 다음과 같은 관계를 만족합니다.

$$n \geq 2\lceil m/2 \rceil^{h} - 1$$

따라서 높이 h는 다음의 조건을 만족합니다.

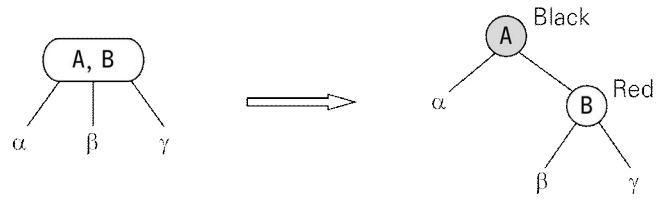
$$h \leq \log_t \frac{n+1}{2}, \qquad t = \lceil m/2 \rceil$$

이 식은 B-트리의 높이가 $O(\log_m n)$임을 의미합니다.

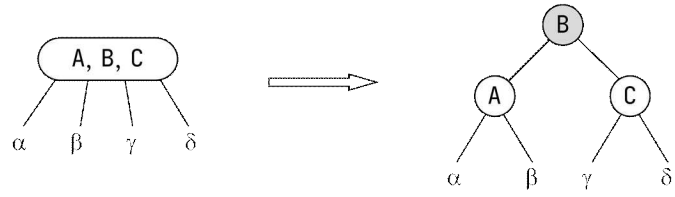
B-트리에서의 탐색 과정은 이진 탐색 트리와 유사하지만, 각 노드에서 여러 키 중 하나를 선택하여 다음 경로를 결정한다는 점이 다릅니다. 따라서 B-트리에서 탐색을 수행하는 데 걸리는 시간은 방문한 노드 수와 방문한 노드에 들어 있는 키 수에 의해 결정됩니다. B-트리의 차수가 클수록 트리의 높이가 낮아져서 탐색 시 방문할 노드의 수가 줄어들지만, 노드가 보유하는 키의 개수는 증가합니다. 각 노드에서의 탐색은 순차 탐색 또는 이진 탐색을 이용할 수 있습니다. 따라서 노드에서의 탐색에는 $O(m)$ 또는 $O(\log m)$ 시간이 소요됩니다. 따라서, h가 트리의 높이일 때, 탐색에 소요되는 전체 시간은 $O(hm)$ 또는 $O(h\log m)$입니다. B-트리가 보조기억장치에 저장된 환경에서는 디스크 I/O가 주된 작업이어서, 노드에서의 탐색 시간은 무시해도 되므로, 트리의 차수는 수백 또는 수천에 이를 정도의 큰 값으로 설정됩니다. 만약, B-트리 전체가 메모리에 저장된다면, B-트리의 효율성을 높이기 위해서 트리의 차수를 적절하게 결정해야 합니다. 이 경우 차수의 적절한 값은 B-트리를 사용하는 환경에 따라 달라지지만, 일반적으로 4에서 32 사이의 값을 가지는 경우가 많습니다.

차수가 작은 B-트리는 AVL 트리나 레드-블랙 트리와 같은 용도로 활용될 수 있습니다. 특히

차수가 3인 B-트리를 **2-3 트리**라고 부르고, 차수가 4인 B-트리를 **2-3-4 트리**라고 합니다. 사실, 13.1절에서 소개한 레드-블랙 트리는 차수가 4인 B-트리, 즉 2-3-4 트리를 이진 트리로 나타낸 것입니다. 2-3-4 트리를 레드-블랙 트리로 변환할 때, 2-3-4 트리의 1-노드(키가 1개 있는 노드)는 Black 노드가 되고, 2-노드(키가 2개 있는 노드)와 3-노드(키가 3개 있는 노드)는 분리되면서 Red 노드가 생성됩니다. 2-노드와 3-노드가 각각 2개와 3개의 노드로 분리되는 방식은 그림 13.18에서 볼 수 있습니다. 분리된 노드의 부모는 Black 노드가 되고, 자식 노드는 Red 노드가 됩니다.



(a) 2-노드의 변환



(b) 3-노드의 변환

그림 13.18: 2-3-4 트리의 2-노드와 3-노드의 변환

그림 13.19에는 그림 13.17의 2-3-4 트리가 레드-블랙 트리로 변환된 모습이 나와 있습니다. 그림에서 NIL 노드는 생략되어 있습니다.

이런 방식으로 변환되면, 2-3-4 트리의 모든 잎 노드는 레벨이 같으므로, 레드-블랙 트리의 루트에서 잎 노드 사이의 모든 경로에서 블랙 노드의 개수는 같습니다. 반면에 Red 노드의 부모와 자식 노드는 항상 Black 노드이므로, Red 노드가 연속해서 나올 수 없습니다.

이제 B-트리에서의 삽입과 삭제 과정을 살펴보겠습니다. 이해를 돕기 위해서 차수가 3인 B-트리, 즉 2-3 트리를 가지고 설명합니다. 일반적인 B-트리에 대한 삽입과 삭제 과정은 2-3트리에 대한 알고리즘을 확장하면 됩니다.

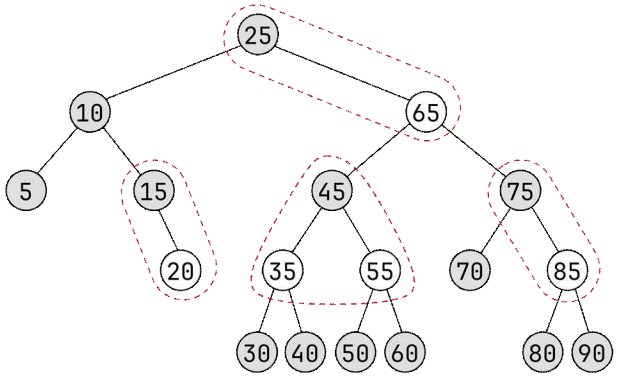


그림 13.19: 그림 13.17의 2-3-4 트리가 변환된 레드-블랙 트리

2-3 트리에서 삽입 과정을 간단히 설명하면 다음과 같습니다. 삽입할 키를 item이라고 하겠습니다. 먼저, 탐색 과정을 통해 도달할 수 있는 잎 노드 T를 찾습니다. 노드 X에 키가 한 개만 있으면, item을 이 노드에 추가하고 작업을 끝냅니다. 노드 X에 키가 두 개 있으면, 노드 X를 두 노드 X와 Y로 분리합니다. 그다음, 원래 X에 있던 두 키와 item 중에서 제일 작은 키는 X에, 제일 큰 키는 Y에 저장하고, 가운데 키 mid는 부모 노드로 올려보냅니다. 이 상황에서 다음의 세 가지 유형이 고려됩니다.

[유형 1] 부모 노드가 없는 경우

새로운 부모 노드를 만들어 키 mid를 저장하고, 노드 X와 Y를 자식으로 설정한 후에 작업을 종료한다. 이 부모 노드는 루트 노드가 되고, 따라서 B-트리의 높이가 1 증가한다.

[유형 2] 부모 노드에 키가 하나만 있는 경우

부모 노드 P에 키 mid를 저장하고, 새로 만들어진 노드 Y를 자식으로 설정한 후에 작업을 종료한다.

[유형 3] 부모 노드에 키가 두 개 있는 경우

부모 노드 P를 두 노드 P와 Q로 분리하고, P의 원래 두 자식과 두 노드 X와 Y를 키의 크기 순서에 맞춰 각각 P와 Q의 두 자식으로 설정합니다. 그다음, 원래 P에 있던 두 키와 mid 중에서 제일 작은 키는 P에, 제일 큰 키는 Q에 저장하고, 가운데 키는 부모 노드로 올려서 재귀적으로 수행합니다.

예를 들어, 그림 13.20에 2-3 트리의 예가 나와 있습니다. 이 트리에서 키가 5인 원소를 삽입하면, 키 10을 가진 잎 노드에 키 5를 추가합니다. 그다음, 키가 18이 삽입되면 두 키 15와

17을 가진 잎 노드를 찾게 되어, 이 노드를 분리하게 됩니다. 키의 순서가 15, 17, 18이므로, 분리된 두 노드에 각각 15와 18을 저장하고, 가운데 키 17을 부모로 올립니다. 이 부모 노드는 키가 하나만 있으므로 [유형 2]에 해당하여, 그림 13.21과 같이 부모 노드는 두 키 12와 17을 갖고, 세 자식 노드를 가리키게 됩니다.

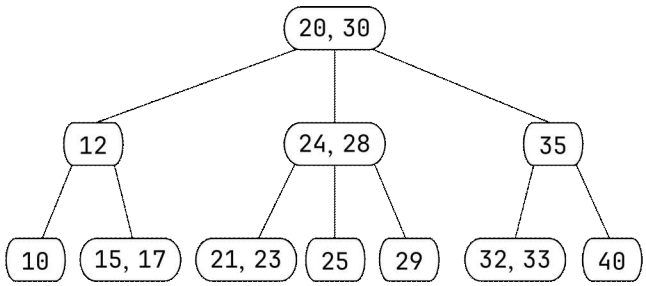


그림 13.20: 2-3 트리의 예

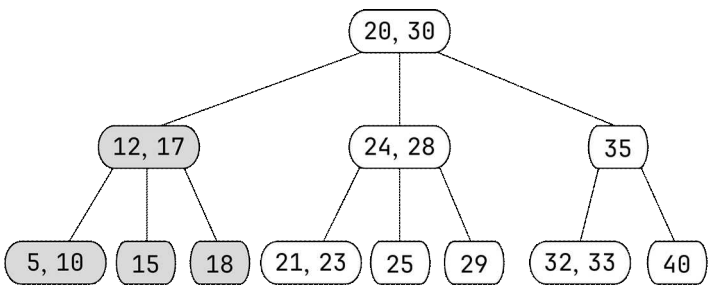


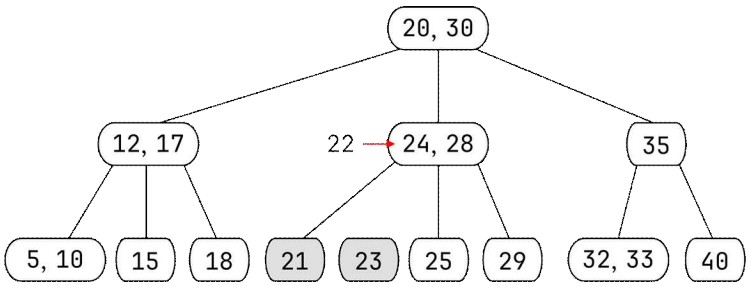
그림 13.21: 두 키 5와 18이 차례로 삽입된 후의 2-3 트리

그다음, 키 22가 삽입되면 두 키 21과 23을 가진 잎 노드를 찾게 되어, 이 노드를 분리하게 됩니다. 키의 순서가 21, 22, 23이므로, 분리된 두 노드에 각각 21과 23을 저장하고, 가운데 키 22를 부모로 올립니다. 이 부모 노드는 두 키 24와 28을 가지고 있으므로 [유형 3]에 해당합니다(그림 13.22 (a) 참조).

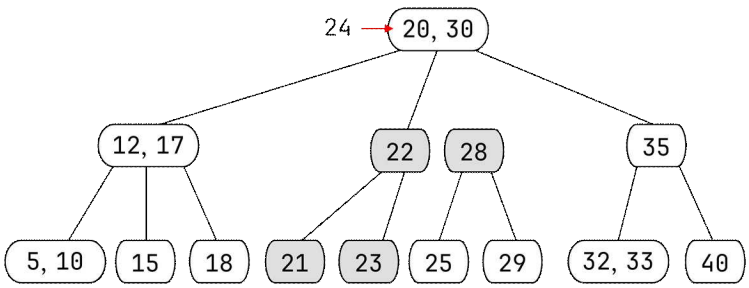
다시 분리된 두 노드에 각각 22와 28을 저장하고, 가운데 키 24를 부모로 올립니다. 이때, 키 22를 가진 노드와 키 28을 가진 노드는 각각 두 개의 자식 노드를 가리키도록 합니다. 키 24를 가져야 할 부모 노드는 두 키 20과 30을 가지고 있어 다시 [유형 3]에 해당합니다(그림 13.22 (b) 참조).

또다시 분리된 두 노드에 각각 20과 30을 저장하고, 가운데 키 24를 부모로 올립니다. 이때,

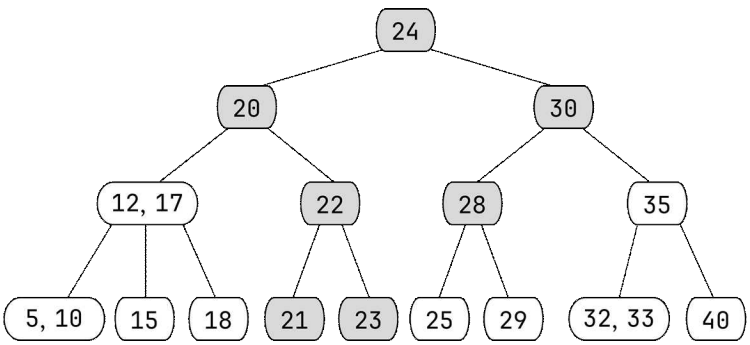
각각 키 20과 30을 가진 두 노드는 각각 두 개의 자식 노드를 가리키도록 합니다. 이제 올려진 키 24를 저장할 부모 노드는 없으므로, [유형 1]에 해당하여 새로운 루트 노드를 만들고 키 24를 저장합니다. 루트 노드가 두 자식 노드를 가리키면 삽입 작업이 종료됩니다(그림 13.22 (c) 참조).



(a) 각각 21과 23을 가진 분리된 노드와 부모 노드에 22를 삽입하려는 과정



(b) 각각 22와 28을 가진 분리된 노드와 부모 노드에 24를 삽입하려는 과정



(c) 각각 20과 30을 가진 분리된 노드와 새로운 루트 노드가 생성되는 과정

그림 13.22: 그림 13.21의 2-3 트리에 키 22를 삽입하는 과정

삽입 과정에서 맨 처음 삽입 위치를 찾는 데 $O(\log n)$ 시간이 걸립니다. 그다음, [유형 1]과 [유형 2]는 상수 시간에 작업이 완료되고, [유형 3]은 최악의 경우 루트까지 전파될 수 있습니다. 그렇지만 각 레벨에서의 작업은 상수 시간이므로, 전체 수행 시간은 $O(\log n)$입니다.

일반적으로 차수가 m인 B-트리의 경우, 노드에서 키의 삽입과 노드 분리 작업에 $O(m)$ 시간이 소요되므로, [유형 1]과 [유형 2]의 수행 시간은 $O(m)$입니다. 따라서 전체 삽입 시간은 $O(m\log_m n)$입니다. 디스크 I/O를 수행해야 하는 경우는 m을 상수로 취급해도 무방하므로 $O(\log_m n)$의 디스크 I/O가 발생한다고 말할 수 있습니다.

B-트리에서 키를 삭제하는 과정은 삽입 과정보다 복잡합니다. 삭제 후 노드가 최소 키 개수 조건을 위반할 경우, 재분배 또는 합병 과정을 수행해야 합니다. 하지만, 삽입 과정과 마찬가지로 시간복잡도는 $O(m\log_m n)$이고, 디스크 I/O 관점에서는 $O(\log_m n)$입니다. 삭제 과정에 대한 자세한 설명은 생략합니다.

데이터베이스 시스템에서는 B-트리의 변형인 **B+ 트리**가 사용됩니다. 인덱싱 구조로서 B-트리에서는 키와 데이터 포인터가 트리의 모든 노드에 저장되지만, B+ 트리에서는 모든 데이터 포인터가 오직 잎 노드에만 저장되고 내부 노드는 키만 가집니다. 또한, B+ 트리의 가장 큰 특징인 모든 잎 노드를 이중 연결리스트로 연결합니다. 따라서 임의의 키에 대한 탐색뿐만 아니라 연속된 구간에 속한 데이터의 순차적인 탐색도 효율적으로 수행할 수 있습니다. 이러한 특징이 B+ 트리가 데이터베이스 인덱싱에서 주요한 역할을 담당하는 주된 이유입니다.

13.3 해싱

해싱(hashing)은 탐색 트리와는 달리 자료 원소의 키를 서로 비교하지 않고, 단지 키의 값 자체만으로 원소가 저장될 위치를 계산하여 탐색 및 삽입, 삭제 연산 등을 수행하는 탐색 구조입니다. 자료 원소는 **해시 테이블(hash table)**이라고 부르는 배열 공간에 저장되는데, 해시 테이블에서 각 원소가 저장될 위치는 **해시 함수(hash function)**에 의해 계산됩니다. 해시 함수는 원소의 키로부터 **해시 주소(hash address)**라고 불리는 배열 첨자를 생성하고, 해시 테이블의 이 첨자 위치에서 해당 원소에 대한 탐색, 삽입, 삭제를 수행합니다. 그림 13.23에 해싱의 기본적인 동작 원리를 나타내는 그림이 나와 있습니다.

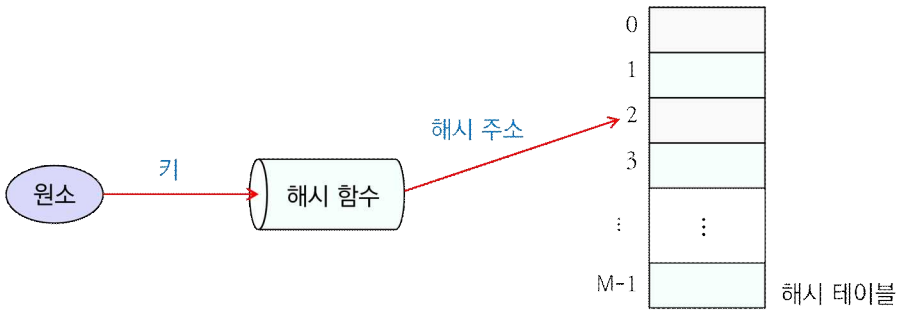


그림 13.23: 해싱의 기본적인 동작 원리

해시 함수가 모든 자료 원소에 대해 각기 다른 해시 주소를 생성하는 가장 이상적인 해싱에서는 탐색, 삽입, 삭제 연산의 수행 시간이 해시 함수를 계산하는 시간에 좌우됩니다. 따라서 해시 함수가 $O(1)$ 시간에 해시 주소를 생성하면, 세 연산 모두 $O(1)$ 시간에 수행됩니다. 그러나 작업 환경에 따라서는 아무리 잘 설계된 해시 함수라고 하더라도, 서로 다른 키로부터 같은 해시 주소가 생성되는 **충돌(collision)**을 피할 수 없는 상황이 나타납니다. 그러므로 충돌에 대한 해결 전략이 해싱의 전체적인 성능을 결정하게 됩니다.

해싱의 궁극적인 목표는 좋은 해시 함수의 설계와 효과적인 충돌 해결 전략을 사용하여 평균적으로 $O(1)$ 시간에 탐색, 삽입, 삭제 연산을 수행할 수 있도록 하는 것입니다. 이를 통해, 대규모 데이터 처리 시스템에서 데이터의 크기가 아무리 커져도 연산 속도는 거의 일정하게 유지해 줍니다. 높이 균형 탐색 트리는 탐색, 삽입, 삭제 연산이 모두 $O(\log n)$ 시간에 수행하므로 n이 커져도 효율적이긴 하지만, 잘 설계된 해싱은 모든 연산에서 압도적인 성능을 제공합니다. 이러한 점이 파이썬 설계자들이 높이 균형 탐색 트리 대신에 해싱을 기본 탐색 구조로 채택한 이유입니다. 균형 탐색 트리의 장점은 정렬에 관련된 연산, 즉 범위 탐색, 상한 및 하한 탐색 등의 연산을 효율적으로 수행할 수 있다는 점이지만 자주 사용되는 기능이 아니기 때문에, 일상적인 작업에서 평균 성능이 더 뛰어난 해싱 구조(`dict`, `set` 등)를 핵심 자료구조로 파이썬 내부에 포함하고 있습니다.

■ 해시 테이블

해시 테이블은 **버킷(bucket)**이라고 불리는 공간들의 배열이고, 버킷은 하나 이상의 **슬롯(slot)**으로 구성됩니다. 슬롯에는 하나의 자료 원소가 저장되는데, 슬롯이 여러 개 있는 경우는 보통 배열이나 연결리스트로 슬롯들을 연결합니다. 해시 테이블이 m개의 버킷과 h개의 슬롯으로 구성된다면, 이 해시 테이블은 모두 m×h개의 원소를 저장할 수 있습니다. 이 값을 **해시 테이블의 용량(capacity)**이라고 부릅니다. 해시 테이블에 실제 저장되는 원소들의 개수를 n이라고 하면, 해

시 테이블의 용량에 대한 n의 비율을 해시 테이블의 **적재율(load factor)** 또는 **적재 밀도(load density)**라고 부릅니다. 즉, m개의 버킷과 h개의 슬롯으로 구성된 해시 테이블의 적재율 α는 다음과 같습니다.

$$\alpha = \frac{n}{m \times h}$$

m개의 버킷을 가진 해시 테이블은 0부터 m-1까지의 정수로 구성되는 해시 주소를 가집니다. 각 해시 주소는 각 버킷에 대응합니다. 해시 함수는 원소의 키로부터 해시 주소를 생성하는 수학적 함수입니다. 그러므로 버킷은 해시 주소가 동일한 원소들을 저장하는 메모리 공간이라고 말할 수 있습니다. 동일한 해시 주소를 가진 원소들은 서로 **동의어(synonym)** 관계에 있다고 말하고, 이미 다른 원소가 저장된 버킷에 어떤 원소가 들어가려고 한다면 **충돌(collision)**이 발생하였다고 말합니다. 충돌을 발생시킨 원소는 해당 버킷의 비어 있는 슬롯에 저장하는데, 빈 슬롯이 남아 있지 않으면 **오버플로(overflow)**가 발생하였다고 말합니다. 한 개의 슬롯만 가진 버킷에는 충돌과 동시에 오버플로가 발생합니다.

해싱에서 충돌이 전혀 발생하지 않는다면 탐색 연산, 삽입 연산, 삭제 연산을 위한 수행 시간은 해시 함수를 계산하는 시간에 의해 결정됩니다. 그러나 충돌을 발생시키지 않는 해시 함수를 찾기란 매우 어려운 일입니다. 특수한 환경이 아니라면 어떤 해시 함수를 선택하더라도 충돌의 발생을 피하기 어렵습니다. 해싱의 여러 기법은 충돌의 발생 가능성을 인정하면서 얼마나 효율적으로 이를 해결하느냐에 초점을 맞춥니다. 해시 테이블을 충분하게 확보하여 적재율을 낮출수록 충돌 가능성은 줄어들게 되므로, 해싱은 메모리 공간을 다소 많이 사용하더라도 수행 시간을 최소화하는 전략을 사용합니다. 물론 해시 테이블의 용량을 늘리는 것만으로는 충돌 가능성과 효율성을 해결하지 못합니다. 해싱의 효율성은 충돌을 가능한 적게 일으키는 해시 함수의 선택과 충돌이 발생했을 때 이를 효과적으로 해결하는 오버플로 처리 전략에 의해 결정됩니다.

해싱의 사례를 들어보겠습니다. 알파벳 소문자로 구성된 문자열을 키로 하는 10개의 요소 집합을 대상으로 해싱을 수행한다고 가정합시다. 해시 함수 f(x)는 키 x의 첫 번째 문자, 즉 x[0]에 대해 0과 25 사이의 정수를 다음과 같이 계산하도록 정의합니다.

```
f(x) = ord(x[0]) - ord('a')
```

파이썬 함수인 ord(c)는 문자 c를 유니코드 값으로 변환합니다. 예를 들어, f('abs') = 0, f('chr') = 2, f('zip') = 25입니다. 해시 테이블 H는 26개의 버킷과 2개의 슬롯을 가지도록 구성합니다. 따라서 해시 테이블의 용량은 26×2 = 52이며, 적재율은 10/52 = 0.19입니다. 10개의 키가 다음의 순서로 삽입된다고 가정해 봅시다.

```
eval, abs, dir, bin, dict, chr, bool, dir, exec, format
```

4번째 키 bin을 삽입할 때까지, 해싱은 충돌 없이 성공적으로 수행됩니다. 5번째 키 dict가 삽입되면 충돌이 발생하지만, 비어 있는 슬롯에 요소를 저장할 수 있습니다. 그러나 8번째 키 dir이 삽입되면, 더 이상 남아 있는 슬롯이 없어 오버플로가 발생합니다. 해시 테이블의 적재율이 19%임에도 불구하고 충돌과 더불어 오버플로까지 발생하므로, 오버플로에 대한 대비가 없으면 해싱은 실패하게 됩니다. 그러므로, 해싱이 성공적으로 수행되기 위해서는 좋은 해시 함수의 설계와 함께 효과적인 오버플로 처리 전략이 필수적입니다.

■ 해시 함수

일반적으로 해시 함수는 자료 원소의 키를 이용하여 **해시 값(hash value)** 또는 **해시 코드(hash code)**라는 값을 생성한 다음, 이 값을 이용해서 해시 테이블의 크기에 맞게 조정하여 최종적으로 해시 주소를 반환합니다. 좋은 해시 함수는 다음과 같은 특징을 가져야 합니다.

① 동일한 키에 대해 항상 동일한 해시 주소를 반환한다.
② 해시 주소의 계산이 빠르다. 즉, $O(1)$ 시간에 해시 주소를 반환한다.
③ 해시 테이블의 버킷에 최대한 고르게 분포된다. 즉, 키의 특성이 해시 주소로 반영되지 않는다.
④ 충돌을 가능한 적게 발생시킨다.

①번 특징을 만족하지 않으면 앞서 저장된 원소를 다시 찾을 수 없고, ②번 특징을 만족하지 않으면 해싱 연산이 상수 시간에 수행될 수 없습니다. ③번 특징은 해싱의 성능을 결정하는 중요한 요소입니다. 키의 특성이 해시 주소로 반영되지 않으려면, 해시 주소를 계산할 때 키의 정보가 뒤죽박죽으로 뒤섞이도록 해야 합니다. 예를 들어, 키가 짝수이면 해시 주소가 짝수로, 키가 홀수이면 해시 주소가 홀수로 계산되는 해시 함수를 사용한다고 가정해 봅시다. 운이 나쁘게도 대부분 데이터가 짝수인 키로만 구성되어 있다면, 해시 테이블의 절반은 거의 사용되지 않을 것입니다. 그러면 그만큼 실제 사용되는 해시 테이블의 용량이 줄어들게 되므로, 충돌 발생 가능성이 커지게 됩니다. 충돌 발생을 최소화하기 위해서는 해싱을 사용하려는 환경에 가장 적합한 해시 함수를 설계하는 것이 효과적이지만, 맞춤식 해시 함수의 설계는 매우 어려운 문제입니다. 따라서 일반적으로 활용될 수 있는 다음과 같은 설계 방법이 주로 사용됩니다. 이들 방법은 단독으로 사용될 수도 있지만, 몇 개를 결합하여 해시 함수로 구성하면 더욱 효과적일 수 있습니다.

- **나눗셈 방법(division method)** : 가장 간단하고 널리 사용되는 방법입니다. 키를 해시 테이블의 버킷 개수 m으로 나눈 나머지를 해시 주소로 사용합니다. 이때 m의 선택이 매우

중요합니다. 만일 m이 짝수라면 홀수 키는 홀수 주소로, 짝수 키는 짝수 주소로 생성됩니다. m의 값으로 가장 좋은 것은 소수(prime)를 선택하는 것이지만, 소수는 드물게 존재하기 때문에, 원하는 범위에서 소수를 찾기가 어려울 때가 있습니다. 이 경우, m을 20보다 작은 소수를 약수로 가지지 않는 수로 정한다면 충분히 효과적이라는 실험 결과가 알려져 있습니다.

- **중간 제곱 방법(mid-square method)** : 이 방법은 키의 값을 제곱한 후에 결과의 중간 부분에 있는 적당한 자릿수를 추출하여 해시 주소로 사용합니다. 예를 들어, 1,000개의 버킷을 가진 해시 테이블을 사용한다고 하면, 해시 주소는 세 자리 숫자가 됩니다. 키가 1234인 경우, 이 수를 제곱한 수인 1522756으로부터 중간 부분의 수인 227을 추출하여 해시 주소로 사용합니다.
- **자릿수 폴딩 방법(digit folding method)** : 이 방법은 키를 동일한 자릿수로 끊어서 분할한 다음, 각 부분을 서로 더해 나온 결과를 이용해 해시 주소로 사용합니다. 예를 들어, 1,000개의 버킷을 가진 해시 테이블을 사용한다고 합시다. 키가 123456789인 경우, 세 자리씩 끊으면 123, 456, 789가 나오고, 이 값들을 모두 더한 수 1,035를 1,000으로 나눈 나머지인 35가 해시 주소가 됩니다.
- **곱셈 방법(multiplication method)** : 이 방법은 1보다 작은 실수를 키에 곱하여 나온 결과의 소수 부분을 사용하여 해시 주소로 계산합니다. 예를 들어, 1,000개의 버킷을 가진 해시 테이블을 사용한다고 할 때, 키가 12345678인 경우, 이 값에 실수 0.61803을 곱하면 나오는 7629999.37434에서 소수 부분인 0.37434를 1000으로 곱한 수의 정수 부분 374가 해시 주소가 됩니다.

■ 오버플로 처리

아무리 좋은 해시 함수라고 하더라도 충돌이 발생할 가능성은 존재합니다. 충돌 가능성은 오버플로가 발생할 수 있다는 사실을 의미하므로, 오버플로에 대한 적절한 대책을 마련하지 않으면 해싱의 운영 자체가 실패로 돌아갈 수 있습니다. 여기서는 오버플로를 효과적으로 해결하는 기본적인 기법인 개방 주소법과 해시 체인법을 소개합니다.

(1) 개방 주소법(open addressing)

이 방법은 충돌이 발생했을 때, 해시 테이블 내의 다른 비어 있는 슬롯을 찾아 데이터를 저장하는 방식입니다. 가장 간단한 개방 주소법은 **선형 개방 주소법(linear open addressing)** 또는 **선형 검색법(linear probing)**이라고 불리는 방법으로, 순차 탐색을 통해 충돌 문제를 해결합니다. 탐색 작업은 해시 테이블에서 해시 주소에 해당하는 버킷부터 시작하여, 탐색 키와 일치하는 원소를 찾습니다. 현재 조사하는 슬롯에 다른 원소가 저장되어 있으면 순차적으로 다음 슬롯을 조

사합니다. 이 과정에서 비어 있는 슬롯을 만나게 되면, 탐색 키를 가진 원소는 존재하지 않는다는 탐색의 실패를 알립니다. 삽입 작업은 해시 주소에 해당하는 버킷의 슬롯부터 시작하여 순차적으로 비어 있는 슬롯을 찾아 그 장소에 원소를 저장합니다. 삭제 작업은 탐색을 통해 삭제할 원소를 찾은 후 해당 원소를 삭제합니다. 삭제 후에 그 슬롯을 빈 상태 그대로 두면 다음에 수행되는 탐색 연산에서 문제가 발생합니다. 이는 삭제된 슬롯을 처음부터 비어 있는 것으로 판단하여 순차적 탐색의 진행을 끊어버리기 때문입니다. 그 결과로 실제로 존재하는 원소를 찾지 못하는 오류가 발생할 수 있습니다. 이 문제를 해결하는 방법으로 원소가 삭제된 슬롯에 "삭제되었다(DELETED)"는 특별한 값을 저장하는 것입니다. 탐색 중에 이 값을 만나면 다음 슬롯을 계속해서 탐색하고, 삽입 과정에서 만나면 새 원소를 해당 슬롯에 저장합니다. 이 방법은 탐색 경로가 계속 길어지는 단점이 있으므로, DELETED 값이 많아지거나 해시 테이블의 적재율이 높아지면, 해시 테이블을 **재해싱(rehashing)**해 주어야 성능 저하를 막을 수 있습니다.

해시 테이블에서 원소들이 일정 구역마다 집중적으로 저장된 **군집화(clustering)** 현상이 발생하면, 순차 탐색의 수행 시간이 늘어나 선형 검색법의 성능이 저하됩니다. 좋은 해시 함수를 사용하면 이 문제를 어느 정도 해소할 수 있지만, 군집화를 줄이기 위한 여러 가지 방안을 활용할 수 있습니다. 선형 검색법에 의해 발생하는 군집화를 **1차 군집화(primary clustering)**라고 합니다. 1차 군집화를 줄이기 위한 개방 주소법인 **이차 검색법(quadratic probing)**은 해시 주소로부터 시작하여 1, 2^2, 3^2, 4^2, … 만큼 떨어져 있는 위치들을 차례로 조사함으로써, 군집화의 영향력이 테이블 전체로 분산되도록 하는 방법입니다. 군집화를 줄이기 위한 또 다른 개방 주소법인 **이중 해싱(double hashing)**은 해시 함수를 하나 더 사용합니다. 그래서 충돌이 발생하면 해시 주소로부터 h(x), 2h(x), 3h(x), … 만큼 떨어져 있는 위치들을 차례로 검사하게 됩니다, 여기서 x는 주어진 키이고 h()는 추가로 사용하는 해시 함수입니다.

(2) 해시 체인법(hash chaining)

해시 체인법은 해시 테이블의 버킷을 연결리스트로 구성합니다. 따라서 동의어 관계에 있는 원소들은 같은 연결리스트에 저장됩니다. 탐색, 삽입, 삭제 연산은, 일단 해시 함수를 통해 해시 주소를 계산한 다음, 해당 버킷의 연결리스트에서 순차 탐색 방식으로 탐색, 삽입, 삭제를 수행합니다. 그림 13.24는 해시 체인법으로 구성된 해시 테이블의 예를 보여줍니다.

해시 체인법은 구현이 간단하고 좋은 해시 함수를 사용한다면 연결리스트의 길이가 그다지 길어지지 않기 때문에 효율적이면서 안정적으로 해싱을 운영할 수 있습니다.

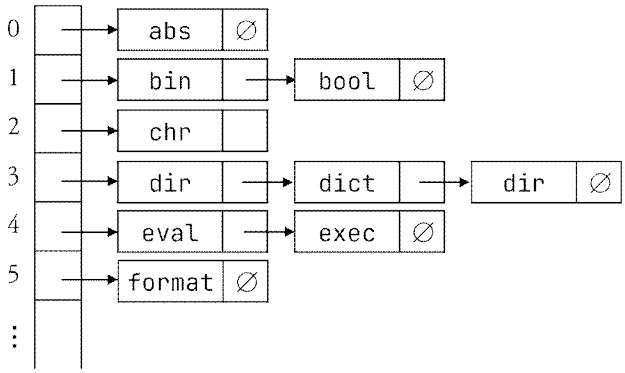


그림 13.24: 해시 체인법의 예 (해시 주소가 키의 첫 문자의 알파벳 순서인 경우)

13.4 사례 연구: 파이썬에서의 해싱

해싱은 파이썬의 핵심 자료구조인 dict(딕셔너리)와 set(집합)의 동작 원리입니다. 이 두 자료구조는 키나 요소를 해시 주소로 변환하여 데이터를 해시 테이블에 저장하고, 이를 통해 평균적으로 $O(1)$ 시간에 탐색, 삽입, 삭제를 수행합니다. 따라서 데이터의 양이 아무리 많아져도 이들 연산의 속도는 거의 일정합니다. 파이썬에서 해싱이 어떻게 동작하는지를 이해하는 것은 dict와 set을 올바르고 효율적으로 사용하는 첫걸음입니다.

■ hash() 함수와 해시 가능성

파이썬의 해싱은 내장 함수인 hash()에서 시작되는데, 이 함수는 파이썬 객체를 입력받아 **해시 값(hash value)**을 반환합니다. 이 해시 값은 객체의 식별자 역할을 하며, 해시 테이블의 크기에 맞춰 다음 식과 같이 나머지 연산(%)을 거쳐 실제 버킷 첨자로 변환됩니다.

```
bucket index = hash(key) % table_size
```

hash() 함수가 동일한 객체에 대해 항상 동일한 해시 값을 반환하기 위해서는, 인자로 주어지는 객체가 '**해시 가능(hashable)**'해야 합니다. 객체가 '해시 가능'하려면 다음 두 가지 조건을 충족해야 합니다.

1. 객체의 내용이 불변(immutable)해야 한다.
2. __hash__()와 __eq__() 메소드가 정의되어 있어야 한다.

첫 번째 조건은 객체가 생성된 후에는 객체의 상태가 바뀌지 않아야 한다는 것을 의미합니다. 객체의 내용이 바뀌면 해시 값도 바뀌어야 하는데, 해시 테이블은 객체의 첫 해시 값을 기반으로 위치를 결정하므로, 내용이 바뀌는 객체는 신뢰할 수 없는 해시 값을 가지게 됩니다. 이 조건 때문에, 파이썬의 자료형은 해시 가능한 객체인 **변경 불가능한 객체(immutable object)**와 해시 불가능한 객체인 **변경 가능한 객체(mutable object)**로 구분됩니다. 변경 불가능한 객체에는 `int`, `float`, `bool`, `str`, `tuple` 등이 있습니다. 이들은 생성된 후 내용이 변하지 않으므로 고유한 해시 값을 안정적으로 유지할 수 있습니다. 변경 불가능한 객체에 대해 `hash()` 함수는 다음과 같이 해시 값을 계산합니다.

- `int` : 정수 자체가 해시 값으로 사용된다.
- `float` : 부동소수점의 이진 표현을 기반으로 해시 값을 계산한다.
- `bool` : `True`는 1, `False`는 0을 해시 값으로 사용한다.
- `str` : 문자열의 모든 문자를 기반으로 해시 값을 계산한다.
- `tuple` : 튜플의 각 요소의 해시 값을 결합하여 하나의 해시 값을 계산한다.
  (단, 튜플의 모든 요소는 해시 가능해야 함)

변경 가능한 객체에는 `list`, `dict`, `set` 등이 있습니다. 이들의 내용은 언제든 바뀔 수 있으므로 해시 값이 고정될 수 없습니다. 따라서 이 객체들은 해시 불가능하며, `dict`의 키나 `set`의 요소로 사용될 경우 `TypeError`가 발생합니다.

두 번째 조건이 필요한 이유는 `hash()` 함수가 내부적으로 객체의 `__hash__()` 메소드를 호출하기 때문입니다. 즉, `__hash__()` 메소드는 객체의 해시 값을 계산하여 정수형으로 반환합니다. 또한, `__eq__()` 메소드는 해시 값의 충돌 시에 객체가 동일한지 여부를 판단하는 데 사용됩니다. 파이썬의 `dict`나 `set`은 충돌이 발생했을 때, 즉 `hash(a) == hash(b)`인 두 객체 a와 b가 같은 해시 주소를 가질 때, 단순히 해시 값만으로는 두 객체를 구분할 수 없습니다. 이때 `__eq__()` 메소드를 호출하여 두 객체의 내용이 실제로 동일한지 확인합니다. 만약 `a == b`가 참이면 동일한 객체로 간주하고, 거짓이면 충돌이 발생한 다른 객체로 간주하여 해시 테이블의 다음 빈 슬롯을 탐색하게 됩니다.

■ 딕셔너리(`dict`)의 동작 원리

파이썬의 `dict`는 개방 주소법을 기반으로 해싱의 원리를 그대로 구현한 자료구조입니다. 파이썬 3.6 이후 버전에서는 메모리 효율성을 위해 `list`와 유사하게 삽입 순서를 유지하는 새로운 최적화된 해시 테이블 구조가 도입되었습니다. 이 구조는 기존의 충돌 해결 방식보다 효율적인 메모리 사용과 빠른 접근 속도를 제공합니다.

dict는 키의 해시 값을 계산하여 해시 테이블의 배열 첨자를 직접 찾아가므로, 데이터의 양에 무관하게 평균적으로 매우 빠른 속도로 연산을 수행합니다. dict의 각 슬롯은 키에 대한 링크, 값에 대한 링크, 해시 값, 이 세 가지 정보를 저장합니다. 해시 값을 슬롯에 저장함으로써 키를 재해싱하는 비용을 감소시킵니다.

dict는 충돌이 발생하면 개방 주소법을 사용하여 다른 빈 슬롯을 찾아 데이터를 저장합니다. dict는 선형 검색법이나 이차 검색법보다 더 복잡하고 최적화된 검색 전략을 사용하여 군집화(clustering) 문제를 효과적으로 해결합니다. dict는 해시 테이블의 적재율(load factor)을 일정 수준(약 2/3)으로 유지합니다. 저장된 요소 수가 해시 테이블의 크기를 초과하면 자동으로 재해싱을 수행하여 테이블의 크기를 확장합니다. 이 과정은 모든 요소를 새로운 해시 테이블에 다시 삽입해야 하므로 일시적으로 선형 시간이 소요되지만, 이런 경우는 매우 드물게 발생하므로 평균적으로는 $O(1)$ 시간에 각 연산이 수행됩니다.

■ 집합(set)의 동작 원리

파이썬의 set은 dict와 마찬가지로 해시 테이블을 기반으로 구현된 자료구조로서, 중복을 허용하지 않는 유일한 요소들의 집합을 저장합니다. 즉 dict와 유사하지만, 값(value) 없이 오직 키(key)만 저장하는 구조입니다.

set의 각 요소는 dict의 키로 사용되고, dict의 값 부분은 사용되지 않거나 더미 값으로 채워집니다. 요소를 추가할 때마다, 파이썬은 해당 요소의 해시 값을 계산하여 해시 테이블의 존재 여부를 확인합니다. 만약 해시 값과 __eq__()를 통해 이미 동일한 요소가 존재한다고 판단되면, 추가를 무시하여 요소의 고유성을 보장합니다.

set은 해시 테이블의 장점을 활용하여 집합 연산을 매우 빠르게 수행합니다. 멤버십 검사, 합집합, 교집합, 차집합과 같은 연산도 내부적으로 해시 테이블을 활용하여 효율적으로 구현되어 있습니다. 이 연산들은 리스트를 사용해 같은 작업을 할 때보다 훨씬 빠릅니다.

■ collections 모듈의 defaultdict와 Counter

collections 모듈의 defaultdict와 Counter는 해싱을 기반으로 하는 대표적인 자료구조입니다.

defaultdict는 dict의 하위 클래스(subclass)로서 dict의 모든 해싱 동작을 상속받습니다. dict와 유일한 차이점은 존재하지 않은 키에 대한 처리 방식입니다. dict는 존재하지 않는 키에 대해 KeyError가 발생하지만, defaultdict는 객체를 생성할 때 **팩토리 함수(factory**

function)를 사용하여 기본값을 가진 새로운 요소를 자동으로 생성합니다. defaultdict의 팩토리 함수는 인자 없이 호출될 수 있는 callable 객체(함수, 클래스 등)여야 합니다. 이 함수는 존재하지 않는 키에 접근할 때 자동으로 호출되어 새로운 요소를 생성하고 기본값을 반환하는 역할을 합니다. 예를 들어, defaultdict(int)에서 int는 int()와 같이 호출되어 기본값 0을 생성하며, defaultdict(list)에서 list는 list()와 같이 호출되어 빈 리스트 []를 생성합니다. 프로그램 13.1에는 int와 list를 팩토리로 사용하여 defaultdict를 생성하는 프로그램 예가 나와 있습니다.

프로그램 13.1: defaultdict를 사용하는 프로그램의 예

```
1  from collections import defaultdict
2
3  # int를 기본값 팩토리로 사용
4  mydict = defaultdict(int)
5  mydict['a'] = 1
6  print(mydict['b'])   # 기본값 0
7  print(mydict)        # defaultdict(<class 'int'>, {'a': 1, 'b': 0})
8
9  # list를 기본값 팩토리로 사용
10 mydict2 = defaultdict(list)
11 mydict2[1] = ['a']
12 print(mydict2[2])    # 기본값 []
13 print(mydict2)       # defaultdict(<class 'list'>, {1: ['a'], 2: []})
```

Counter도 dict의 하위 클래스이며, dict의 모든 해싱 동작을 상속받습니다. Counter는 리스트나 튜플과 같은 객체에 포함된 요소들의 빈도수를 편리하게 계산하고 저장하는 데 사용됩니다. dict와의 차이점은 몇 가지 유용한 특화 기능과 편리한 초기화 방식을 제공한다는 점입니다. Counter는 요소와 그 빈도수를 키와 값으로 하는 dict를 생성합니다. 초기화 기능으로, 리스트, 튜플, 문자열 등의 객체를 인자로 받아 한 번에 각 요소의 빈도수를 셀 수 있습니다. Counter 객체끼리 덧셈, 뺄셈, 교집합, 합집합, 차집합 등과 같은 수학적 연산을 수행할 수 있으며, 다음과 같은 특화된 메소드를 제공합니다.

- elements() : 모든 요소를 그 개수만큼 반복하는 iterator를 반환한다.
- most_common(n) : 빈도수가 큰 순서대로 상위 n개의 요소와 그 빈도수를 리스트로 반환한다. n을 생략하면 모든 요소를 반환한다.
- update() : 순회 가능한 객체나 딕셔너리를 사용하여 빈도수를 갱신한다.

프로그램 13.2에는 Counter를 사용하여 리스트의 요소와 문자열의 문자에 대해 빈도수를 처리하는 프로그램 예가 나와 있습니다.

프로그램 13.2: Counter를 사용하는 프로그램의 예

```
from collections import Counter

# 리스트의 요소 개수 세기
L = [1, 4, 5, 5, 2, 1, 5, 2, 5, 2]
freq = Counter(L)
print(freq[3])                # 존재하지 않는 키 3에 대한 검색, 0 출력
freq.update([1, 1, 1])        # [1, 1, 1]을 추가하여 개수를 셈
print(freq.most_common(2))    # 상위 2개의 요소와 빈도수 출력
# 출력 : [(1, 5), (5, 4)]
print(freq.most_common())     # 모든 요소를 빈도수 순으로 나열
# 출력 : [(1, 5), (5, 4), (2, 3), (4, 1)]

# 문자열의 문자 개수 세기
A = Counter("hello")
B = Counter("world")
print(A)        # Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
print(B)        # Counter({'w': 1, 'o': 1, 'r': 1, 'l': 1, 'd': 1})

print(A + B)    # 덧셈 (요소별 개수를 더함)
# 출력 : Counter({'l': 3, 'o': 2, 'h': 1, 'e': 1, 'w': 1, 'r': 1, 'd': 1})
print(A - B)    # 뺄셈 (0보다 작거나 같아지면 포함하지 않음)
# 출력 : Counter({'h': 1, 'e': 1, 'l': 1})
```

연습문제

1. 다음의 각 탐색 구조에 대해서 주어진 범위 [min, max]에 속하는 키를 가진 모든 원소를 출력하는 알고리즘을 쓰시오.
   (1) 정렬된 배열
   (2) 정렬되지 않은 연결리스트
   (3) 이진 탐색 트리
   (4) 해싱

2. 그림 13.1의 AVL 트리에 키 60을 삽입할 때 발생하는 불균형 해결을 위해 수행해야 할 회전 변환의 유형이 무엇이며, 그 회전 변환의 결과를 설명하시오.

3. 어떤 레드-블랙 트리에서 루트부터 NIL 노드까지의 경로 중 가장 짧은 경로의 Black 노드 수가 5개이다. 이 트리에서 가장 긴 경로의 최대 길이는 얼마인지 설명하시오.

4. 다음의 각 탐색 트리에 대해 공백 트리에서 시작하여 다음과 같은 순서로 삽입할 때 만들어지는 트리를 그리시오.

   (12, 1, 4, 5, 7, 8, 10, 2, 9, 11, 3, 6)

   (1) 이진 탐색 트리
   (2) AVL 트리
   (3) 레드-블랙 트리
   (4) 2-3 트리

5. AVL 트리 T를 키 x를 기준으로 하여 두 개의 AVL 트리 T_1과 T_2로 분할하는 효율적인 알고리즘을 설계하시오. 이때, T_1의 모든 원소들은 x보다 작거나 같은 키를 가지고 T_2의 모든 원소들은 x보다 큰 키를 가진다.

6. 두 개의 AVL 트리 T_1과 T_2를 합병하여 하나의 AVL 트리로 만드는 알고리즘을 설계하고, 그 시간복잡도를 분석하시오.

7. 2-3-4 트리를 레드-블랙 트리로 변환하는 알고리즘을 설계하고, 그 시간복잡도를 분석하시오.

8. 레드-블랙 트리를 2-3-4 트리로 변환하는 알고리즘을 설계하고, 그 시간복잡도를 분석하시오.

9. 해싱의 오버플로를 선형 검색법으로 해결하고자 할 때, 삽입 연산을 수행하는 알고리즘을 쓰시오.

10. 사용자 데이터를 저장하고 사용자 이름으로 자주 검색해야 하는 프로그램을 개발하려고 한다. 이 프로그램에는 모든 사용자 이름을 알파벳 순서로 빠르게 표시하는 기능도 포함되어야 한다. 탐색 구조로서, 해시 테이블(dict)과 레드-블랙 트리 중 어느 쪽을 선택하는 것이 좋을지 수행 시간을 기준으로 설명하시오.

11. 크기가 11인 해시 테이블에서 선형 검색법을 사용하여 오버플로를 해결한다. 해시 함수는 "k를 11로 나눈 나머지"로 정의된다. 키 22, 35, 48, 33, 25, 11을 순서대로 삽입했을 때, 해시 테이블의 최종 상태를 나타내시오.

12. 크기가 11인 해시 테이블에서 이차 검색법을 사용하여 오버플로를 해결한다. 해시 함수는 "k를 11로 나눈 나머지"로 정의된다. 키 22, 35, 48, 33, 25, 11을 순서대로 삽입했을 때, 해시 테이블의 최종 상태를 나타내시오.

13. 크기가 11인 해시 테이블에서 이중 해싱(double hashing)을 사용한 개방 주소법을 사용한다고 하자. 주 해시 함수 $h_1(k)$과 보조 해시 함수 $h_2(x)$는 다음과 같다. 식에서 '%'는 나머지 연산을 의미한다.

    $h_1(k) = k \% 11$
    $h_2(x) = 7 - (k \% 7)$

    키 22, 35, 48, 33, 25, 11을 순서대로 삽입했을 때, 해시 테이블의 최종 상태를 나타내시오.

14. 일차원 리스트 A에서 A[i] ≤ A[i+1]이고 A[i] ≤ A[i-1]인 경우 A[i]를 지역 최솟값(local minimum)이라고 하자. 리스트 A에서 다음의 조건이 만족하면 항상 지역 최솟값이 존재한다.

    A[j] ≤ A[j-1]이고 A[k] ≤ A[k+1]이고 j < k인 두 첨자 j와 k가 존재한다.

    이 조건을 만족하는 리스트에서 지역 최솟값을 출력하는 선형 시간의 알고리즘을 고안하시오.

15. 해시 체인법을 이용하여 파이썬 dict와 유사한 딕셔너리 클래스 MyDict를 직접 구현하시오. __setitem__(삽입), __getitem__(탐색), __delitem__(삭제) 메소드를 포함해야 한다.

16. 길이가 각각 n과 m인 두 정수 리스트 A와 B가 주어질 때, 두 리스트의 교집합을 $O(n+m)$ 시간에 구하는 프로그램을 작성하시오.

17. 문자열 s에서 단 한 번만 나타나는 첫 번째 문자를 선형 시간에 찾아서 출력하는 파이썬 함수 first_non_repeating(s)를 작성하시오.

18. [실습 문제] 정렬되지 않은 정수 리스트가 주어질 때, 가장 긴 연속된 요소들의 순서의 길이를 찾는 프로그램을 작성하시오. 예를 들어, 리스트 [100, 4, 50, 1, 3, 2]가 주어지면, 가장 긴 연속된 요소들의 순서는 (1, 2, 3, 4)로서 길이가 4이다.

    입력으로, 첫 번째 줄에 리스트의 길이 n이 주어진다. n은 1 이상 10^6 이하이다. 두 번째 줄에는 n개의 정수가 공백으로 구분되어 들어온다. 각 정수는 -10^9과 10^9 사이이다. 출력으로, 가장 긴 연속된 요소 순서의 길이를 출력한다.

입력의 예	출력의 예
6 100 4 50 1 3 2	4

입력의 예	출력의 예
10 0 3 7 2 5 8 4 6 0 1	9

19. [실습 문제] 단어들의 리스트가 주어질 때, 이 리스트에서 **애너그램(anagram)** 관계인 단어들의 그룹을 찾는 프로그램을 작성하시오. 애너그램은 두 단어가 같은 문자를 같은 개수만큼 사용하여 구성된 경우를 말한다. 즉, 단어의 문자를 섞어서 만들 수 있는 단어들 관계이다. 예를 들어, "eat", "tea", "ate" 서로 애너그램 관계이다.

    입력으로, 첫 번째 줄에 단어의 개수 n이 주어진다. n은 1 이상 10^4 이하이다. 그다음 n개의 줄에는 각 줄에 하나의 단어가 들어온다. 각 단어는 영어 알파벳 소문자로만 구성되어 있으며, 길이는 1 이상 100 이하이다. 출력으로, 애너그램 관계에 있는 단어 그룹의 개수를 출력한다.

입력의 예	출력의 예
6 eat tea tan ate nat bat	3

    이 예에서는 3개의 애너그램 그룹 {bat}, {nat, tan}, {ate, eat, tea}이 존재한다.

입력의 예	출력의 예
4 a b c d	4

찾아보기

[ㄱ]

가변 인자(variable-length argument) 60
가상코드(pseudo-code) 13
가중치 규칙(weighting rule) 303
가중치 그래프(weighted graph) 350
가중치 행렬(weight matrix) 350
간선(edge) 250, 320
강 연결 요소(strongly connected component) 324
개방 주소법(open addressing) 405
객체 지향(object-oriented) 프로그래밍 64, 71
결정 트리(decision tree) 271
경로 압축 규칙(path collapsing rule) 303
경로(path) 250, 323
경로 압축 규칙(path collapsing rule) 303
경사 이진 트리(skewed binary tree) 252
계층적인 구조(hierarchical structure) 248
곱셈 방법(multiplication method) 405
공간복잡도(space complexity) 22
괄호 검사 135
군집화(clustering) 406
균형 인자(balance factor) 385
균형 탐색 트리(balanced search tree) 384
그래프(graph) 320
기대 시간(expected time) 22
기본 연산(primitive operation) 18
기수 정렬(radix sort) 233
깊은 복사(deep copy) 84
깊이 우선 신장 트리(depth first spanning tree) 338
깊이 우선 탐색(depth first search) 331
꼬리 노드(tail node) 189

[ㄴ]

나눗셈 방법(division method) 404
내부 노드(internal node) 250
내부 정렬(internal sort) 219
내부 클래스(nested class) 65
내장 자료형(built-in type) 42
내장 함수(built-in function) 61
너비 우선 신장 트리(breadth first spanning tree) 338
너비 우선 탐색(breadth first search) 331
널 링크(null link) 189
노드(node) 189, 249, 320

[ㄷ]

다익스트라 알고리즘(Dijkstra algorithm) 363
다중 그래프(multigraph) 322
다중 링크 표현법 267
단말 노드(terminal node) 250
단순 경로(simple path) 323
단일 연결리스트(singly linked list) 191
더미 노드(dummy node) 195, 201
데크(deque) 205
독스트링(docstring) 41
동적 데이터(dynamic data) 235
동적 배열(dynamic array) 76
동적계획법(dynamic programming) 369
동치 관계(equivalence relation) 305
동치 클래스(equivalence class) 305
등차급수의 합 15
디폴트 인자(default argument) 60

[ㄹ]

람다 함수(lambda function) 60, 91
레드-블랙 트리(red-black tree) 389
레벨 순위 순회(level order traversal) 261

레벨(level) 251
로무토 분할(Lomuto partition) 230
루트(root) 249
리스트 내포(list comprehension) 54, 83
링크(link) 189

[ㅁ]
마방진(magic square) 96
메모이제이션(memoization) 107
멱집합(power set) 126
모스 부호(Morse code) 96
무선조종 자동차 문제 163
무한 재귀(infinite recursion) 109
무향 그래프(undirected graph) 321
무향 에지(undirected edge) 321
미로(maze) 144, 173
밀집 그래프(dense graph) 329, 362

[ㅂ]
바이토닉 수열(bitonic sequence) 125
반동적 데이터(semi-dynamic data) 235
반복적 해법(iterative solution) 103
반전(inversion) 243
방향 경로(directed path) 323
방향 그래프(directed graph) 322
방향 비순환 그래프(directed acyclic graph) 341
방향 에지(directed edge) 321
방향 사이클(directed cycle) 323
배열 구조 188
버블 정렬(bubble sort) 87
버킷 정렬(bucket sort) 232
버킷(bucket) 402
벨만-포드 알고리즘(Bellman-Ford algorithm) 380
보루프카 알고리즘(Borůvka's algorithm) 454, 381
부그래프(subgraph) 323
부모 포인터 배열 270, 366
부모(parent) 250
부분합 구간의 개수 166
부트리(subtree) 250
분할정복법(divide and conquer) 220
불안정 정렬(unstable sort) 219
비단말 노드(nonterminal node) 250
비용 행렬(cost matrix) 350
B+ 트리 401
B-트리 395
bisect 모듈 236

[ㅅ]
사이클(cycle) 323
사진 트리(quad tree) 279
삽입 정렬(insertion sort) 88
상각 시간복잡도(amortized time complexity) 77
상향식 힙 조정(heapify up) 291
선입선출 160
선택 정렬(selection sort) 85
선행자(predecessor) 278
선형 개방 주소법(linear open addressing) 405
선형 검색법(linear probing) 405
선형리스트(linear list) 72
성능 분석 18
성능 측정 17
세그먼트 트리(segment tree) 308
센터(center) 379
셀프 루프(self loop) 322
셸프(shelf) 184
솔린 알고리즘(Sollin's algorithm) 381
수식 트리(expression tree) 272
순서리스트(ordered list) 72
순차 탐색(sequential search) 21, 118, 235
순회(traversal) 258
쉘 정렬(shell sort) 245
숲(forest) 359
스레드 이진 트리(threaded binary tree) 278

스크롤(scroll) 184
스택 오버플로(stack overflow) 135
슬라이딩 윈도우(sliding window) 167
슬롯(slot) 402
시간복잡도(time complexity) 19
시스템 스택(system stack) 113
신장 트리(spanning tree) 337
실타래 144
쓰레기 수집(garbage collection) 39, 191
Θ-표기법 25

[ㅇ]
아커만 함수(Ackermann function) 123, 304
안정 정렬(stable sort) 219
알고리즘적 사고(algorithmic thinking) 218
얕은 복사(shallow copy) 84
에라토스테네스의 체 16, 31
에지(edge) 250, 320
여왕 문제(n-queen problem) 127
역 인접 리스트(inverse adjacency list) 328
연결 구조 189
연결 그래프(connected graph) 324
연결 요소(connected component) 324, 336
연결리스트(linked list) 189
오일러 공식(Euler formula) 325
완전 그래프(complete graph) 325
완전 이진 트리(complete binary tree) 254
외부 정렬(external sort) 220
왼쪽 자식-오른쪽 형제 표현 268
요셉 문제(Josephus problem) 178, 199
욕심쟁이 원칙 363
욕심쟁이법(greedy method) 363
우선순위 큐(priority queue) 290
원형 단일 연결리스트(circular singly linked list) 198
원형 이중 연결리스트 201
위상 순서(topological sequence) 341
위상 정렬(topological sort) 341
유클리드 호제법 13
이분 그래프(bipartite graph) 346
이중 연결 그래프(biconnected graph) 345
이중 연결리스트(doubly linked list) 201
이중 해싱(double hashing) 406
이진 탐색 트리(binary search tree) 284
이진 탐색(binary search) 118, 236
이진 트리 표현법 268
이진 트리(binary tree) 251
이차 검색법(quadratic probing) 406
인접 리스트(adjacency list) 328
인접 행렬(adjacency matrix) 326
인접하다(adjacent) 322
인코딩 트리(encoding tree) 279
임계 경로(critical path) 377
잎 노드(leaf node) 250
AOA(Activity On Arrow) 네트워크 371
AOE 네트워크 371
AOV 네트워크 371
AVL 트리 384
f-문자열(f-string) 53
LL 회전 변환 385
LR 회전 변환 386
NP-hard 372
O-표기법 23
Ω-표기법 25
RL 회전 변환 386
RR 회전 변환 386
UFSet 클래스 304
union-find 301
1차 군집화(primary clustering) 406
2-3 트리 396
2-3-4 트리 396

[ㅈ]
자릿수 폴딩 방법(digit folding method) 405
자식 리스트-부모 포인터 표현법 269
자연 조인(natural join) 93
자연어(natural language) 13
자유 트리(free tree) 325

작업 네트워크(activity network) 371
재귀 관계(recurrence relation) 100
재귀 함수(recursive function) 104
재귀적 해법(recursive solution) 103
재해싱(rehashing) 406
적재 밀도(load density), 적재율(load factor) 403
전 이진 트리(full binary tree) 253
전위 수식 138
전위 순서(preorder sequence) 261
전위 순회(preorder traversal) 259
전입 차수(in-degree) 323
전출 차수(out-degree) 323
절단 속성(cut property) 354
절단 정점(cut vertex) 345
점근적 분석법 23
정적 데이터(static data) 235
정적 배열(static array) 75
정점(vertex) 320
제자리 정렬(in-place sort) 219
조상(ancestor) 250
중간 제곱 방법(mid-square method) 405
중대 작업(critical activity) 377
중앙(median) 380
중앙값의 중앙값(median of median) 227, 239
중위 선행자 278
중위 수식 138
중위 순서(inorder sequence) 261
중위 순회(inorder traversal) 259
중위 후속자(inorder successor) 278, 288
지름(diameter) 345, 379
지배적 연산(dominating operation) 20
지연 순서(lazy sequence) 93
지연 평가(lazy evaluation) 50

[ㅈ]
차수 표기법(order notation) 23
차수(degree) 250, 322
참조 계수(reference count) 194
참조 배열(reference array) 76
최단 경로 트리(shortest path tree) 339, 363
최단 경로(shortest path) 173
최대 재귀 깊이(maximum recursion depth) 109
최대 힙(max heap) 291
최대공약수 13
최상의 경우 시간복잡도 22
최소 신장 트리(minimum spanning tree) 353
최소 힙(min heap) 291
최소-최대 힙(min-max heap) 315
최악의 경우 시간복잡도 22
최장 방향 경로(longest directed path) 372
최적 알고리즘(optimal algorithm) 272
추상자료형(abstract data type) 70, 134
충돌(collision) 402, 403

[ㅋ]
카탈란 수(Catalan numbers) 266
칸토어 집합(Cantor set) 125
쾨니히스베르크(Königsberg) 다리 320
퀵 선택(quick select) 237
퀵 정렬(quick sort) 225
큐 오버플로(queue overflow) 163, 171
크루스칼 알고리즘(Kruskal's algorithm) 359
클래스 변수 65
클래스(class) 64
키워드 인자(keyword argument) 60

[ㅌ]
퇴각(backtrack) 331
트리 에지(tree edge) 355
트리 정점(tree vertex) 355
트리(tree) 325

[ㅍ]
편집기 문제 208
평균 시간복잡도 22

평면 그래프(planar graph) 325
폐쇄형(closed form) 100
포화 이진 트리(perfect binary tree) 253
프랙탈(fractal) 125
프림 알고리즘(Prim's algorithm) 354
플로이드-와샬 알고리즘(Floyd-Warshall algorithm) 363, 369
피벗(pivot) 225
피보나치 수열(Fibonacci sequence) 101

[ㅎ]

하노이 탑(Towers of Hanoi) 115
하이브리드 정렬(hybrid sort) 228
하향식 힙 조정(heapify down) 293
합병 정렬(merge sort) 221
해시 가능(hashable) 407
해시 값(hash value) 404, 407
해시 주소(hash address) 401
해시 체인법(hash chaining) 406
해시 코드(hash code) 404
해시 테이블(hash table) 401, 402
해시 함수(hash function) 401, 404
해싱(hashing) 401
헤더 노드(header node) 195
호너의 방법(Horner's method) 32
호어 분할(Hoare partition) 244
활성화 레코드(activation record) 113
회문(palindrome) 104, 155
후손(descendant) 250
후위 수식 138, 157
후위 순서(postorder sequence) 261
후위 순회(postorder traversal) 259
희소 그래프(sparse graph) 329, 362
힙 정렬(heap sort) 297
힙 초기화 294
힙(heap) 290
heapq 모듈 298

이 교재는 2025년도 정부(산업통상자원부)의 재원으로
한국산업기술진흥원의 지원을 받아 수행되었습니다.
(P0017124, 2025년 산업혁신인재성장지원사업)

비매품/무료
93000
9 791167 567444
ISBN 979-11-6756-744-4